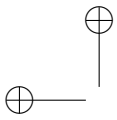
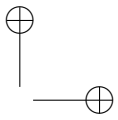
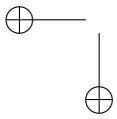
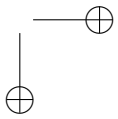
DESTRIPANDO INTERNET

19 de octubre de 2026

DESTRIPANDO INTERNET

David Villa
Ana Rubio

© 2026 David Villa, Ana Rubio

Primera edición: 2026
ISBN: XXX-XX-XXXXX-X

e-mail `destripando.internet@gmail.com`
web `https://github.com/destripando-internet`

© Los autores del documento. Se permite la copia, distribución y/o modificación de este documento bajo los términos de la licencia de documentación libre GNU, versión 1.1 o cualquier versión posterior publicada por la *Free Software Foundation*, sin secciones invariantes. Puede consultar esta licencia en http://www.gnu.org.

Índice general

Prefacio

1 Debian GNU/Linux **1**

1.1. UNIX 1
1.2. El fin del software libre 2
1.3. GNU 4
1.4. Linux 5
1.5. GNU/Linux 6
1.6. Debian GNU/Linux 6

2 Shell **9**

2.1. GNU Bash 10
2.2. Valor de retorno 11
2.3. Opciones 12
2.4. Procesos 13
2.4.1. Trabajos 14
2.4.2. Otros modos de identificar procesos 16
2.5. Entrada, salida y salida de error 16
2.6. Redirección 17
2.7. Usuarios y grupos 21
2.7.1. Propietarios y permisos 22
2.8. Paquetes 24
2.8.1. Repositorios de paquetes. 26
2.8.2. Cómo encontrar paquetes 28

2.9. Servicios 29
2.10. Parámetros del núcleo 30

3 Python **33**
3.1. ¡A programar! 33
3.2. Variables y tipos 34
3.3. Tipos de datos 35
3.3.1. Valor nulo 35
3.3.2. Booleanos 35
3.3.3. Numéricos 35
3.3.4. Secuencias 36
3.4. Módulos 38
3.5. Estructuras de control 38
3.6. Indentación estricta 38
3.7. Funciones 39
3.8. Python *is different* 39
3.9. Hacer un 'ejecutable' 39
3.10. Orientado a objetos 40
3.11. *Type checking* 41

4 Internet **43**
4.1. Tecnología Internet 45
4.2. Protocolos 47
4.3. Pila de protocolos 48
4.4. Modelo OSI 49
4.4.1. Direccionamiento físico vs. lógico 50
4.5. Modelo TCP/IP 50
4.6. Modelo híbrido 51
4.7. Qué NO es Internet 51
4.7.1. Internet no es la web 52

4.7.2. Internet no es la nube 52
4.7.3. Internet no es TCP/IP 53
4.7.4. Internet no es una empresa u organización 53

5 Protocolos esenciales **55**
5.1. Encapsulación 56
5.2. Protocolos de enlace 59
5.3. Ethernet 61
5.3.1. Interfaces de red 61
5.3.2. Trama Ethernet 62
5.3.3. Direcciones MAC 63
5.3.4. Conectividad Ethernet 65
5.4. PPP 66
5.5. IP 67
5.5.1. Interfaces de red 67
5.5.2. Paquete IP 68
5.5.3. Direcciones IP 70
5.5.4. Conectividad IP 71
5.6. ARP 71
5.7. ICMP 74
5.7.1. Conectividad IP 76
5.8. UDP 78
5.8.1. Datagrama UDP 79
5.8.2. Conectividad UDP 80
5.9. TCP 81
5.9.1. Segmento TCP 81
5.9.2. Capturando una conexión TCP 83
5.9.3. Conectividad TCP 85

6 Sockets **87**
6.1. Programación de redes 88

6.2. La clase `socket` . . . 88
6.3. Puertos . . . 89
6.4. Comunicación UDP . . . 92
6.5. Servidor TCP . . . 95
6.6. Cliente TCP . . . 96
6.7. Flujos de datos y E/S parcial . . . 98
6.7.1. Envío TCP . . . 98
6.7.2. Recepción TCP . . . 99
6.7.3. Bandera de fin de mensaje . . . 101
6.7.4. Tamaño del mensaje en la cabecera . . . 102
6.8. Sockets como archivos . . . 103
6.9. Gestión explícita de buffers . . . 105
6.10. Fin de la comunicación TCP . . . 107
6.11. Manejo de errores . . . 108
6.11.1. Context manager . . . 109
6.12. Netcat . . . 111
6.12.1. Sintaxis básica . . . 111

7 Direccionamiento IP **117**
7.1. Máscara de red . . . 118
7.2. Direccionamiento con clases . . . 119
7.2.1. *Subnetting* . . . 120
7.2.2. Ejemplo de *subnetting* . . . 121
7.3. Direccionamiento sin clases . . . 123
7.3.1. VLSM . . . 123
7.3.2. Bloques /30 . . . 124
7.3.3. Ejemplo de VLSM . . . 125
7.4. Direcciones especiales . . . 129

8 Interconexión **131**
8.1. Almacenamiento y reenvío . . . 132

8.2. Entrega directa vs. indirecta 135
8.3. Tabla de encaminamiento 135
8.3.1. Analizando la tabla de ejemplo 137
8.3.2. Tabla de nodo final 138
8.4. Agregación 140
8.5. Un par de ejemplos detallados 141
8.5.1. Escenario 1: entrega local 142
8.5.2. Escenario 2: dos saltos 144
8.6. Mensajes de control de interred (ICMP) 145
8.6.1. Destino inalcanzable *(Destination unreachable)* . . 147
8.6.2. Tiempo excedido *(Time exceeded)* 148
8.6.3. Problema en parámetro *(Parameter problem)* . . . 150
8.6.4. Supresión al origen *(Source quench)* 150
8.6.5. Redirección *(Redirect)* 150
8.6.6. Ping *(Echo)* 151
8.6.7. Marca de tiempo *(Timestamp)* 152
8.6.8. Información *(Information)* y máscara *(Address mask)* 154
8.6.9. Solicitud y anunciamiento de router 154
8.7. Fragmentación 154
8.7.1. No fragmentar 162
8.7.2. Descubrimiento de MTU 164

9 Encaminamiento dinámico **167**
9.1. Algoritmos y protocolos de encaminamiento 168
9.2. Sistemas autónomos 169
9.3. Tipos de encaminamiento 169
9.4. Topologías de referencia: Delta 170
9.5. Redundancia 171
9.6. La ruta más corta 172

9.7. Vector-distancia 174
9.7.1. Problemas y limitaciones de vector-distancia . . . 177
9.7.2. RIP 179
9.8. Estado de enlace 179
9.8.1. OSPF 180
9.9. Vector-ruta y BGP 181
9.10. Laboratorio de encaminamiento Delta-1 182
9.10.1. Encaminamiento estático 187
9.10.2. Zebra 188
9.10.3. Encaminamiento RIP 189
9.10.4. Encaminamiento OSPF 191
9.10.5. Reacción ante fallos 193

10 Configuración IP **195**
10.1. Configuración manual 195
10.2. Configuración automática con DHCP 197
10.3. Servidor DHCP 201
10.4. DHCP no siempre es la solución 203

11 Redes Privadas **205**
11.1. Líneas alquiladas 205
11.2. Redes privadas TCP/IP 207
11.3. Direccionamiento privado 207
11.4. Direccionamiento de enlace local 208
11.5. Conectividad en redes privadas 210
11.6. Traducción de Direcciones de Red (NAT) 211
11.6.1. Averiguar la dirección pública 215
11.6.2. Reenvío de puertos 215
11.6.3. Direcciones en la carga útil 217

11.7. Router NAT con Linux 218
11.8. CGNAT 220
11.9. Tipos de NAT 222
11.10. NAT transversal 224
11.10.1. STUN 224
11.10.2. Determinar el Mapeado 225
11.10.3. Determinar el Filtrado 225
11.10.4. Señalización 227
11.10.5. *hole punching* 228
11.10.6. TURN e ICE 229

12 Confiabilidad y control de flujo 231
12.1. Parada y espera 233
12.2. Repetición continua 235
12.3. Repetición selectiva 237
12.4. Confirmación acumulativa 239
12.5. ARQ y ancho de banda × retardo 240
12.6. Confirmación negativa. 243
12.7. Confiabilidad en comunicaciones dúplex. 243

13 Confiabilidad y control de flujo en TCP 245
13.1. Conexión 246
13.2. Tamaño máximo de segmento 248
13.3. Desconexión 250
13.3.1. Tiempo de silencio 252
13.4. Reset 252
13.5. Control de flujo en TCP 254
13.6. Ventanas de envío y recepción 256
13.7. RTO: el temporizador de retransmisión 260
13.8. El síndrome de la *ventana tonta* 264

13.9. Cierre de la ventana 265
13.10. Confirmación selectiva. 266
13.11. Aplicaciones interactivas 266
13.12. Control de errores 268
13.12.1. Segmento perdido/corrupto 268
13.12.2. Confirmación perdida/corrupta 268
13.12.3. Segmento duplicado 270
13.12.4. Segmento fuera de orden. 270
13.12.5. Confirmación duplicada 271
13.12.6. Demasiadas retransmisiones 272
13.13. *Keep alive*. 272

14 Control de congestión **275**
14.1. Carga, capacidad y congestión 275
14.2. Control de congestión 278
14.2.1. Técnicas preventivas 278
14.2.2. Técnicas reactivas 279
14.3. Control de flujo vs. control de congestión 280
14.4. Enfriamiento en origen 281
14.5. Control de congestión en TCP 282
14.5.1. Arranque lento (*slow start*) 282
14.5.2. Evitación de congestión *(congestion avoidance)* . . 284
14.5.3. Decrecimiento multiplicativo 285
14.5.4. Modelo simplificado 288
14.6. Gestión activa de colas 290
14.6.1. CoDel 291
14.6.2. ECN en TCP. 292
14.6.3. ECN con otros protocolos 293

15 Cliente-Servidor **295**
15.1. Upper 297

15.2. Servidor TCP 297
15.3. Cliente TCP 299
15.4. Servidor TCP multihilo 300
15.5. Forzando los límites 302
15.6. Servidor TCP multiproceso 303
15.7. Servidor UDP 307
15.8. Servidor UDP multiproceso 310
15.9. Servidor UDP multihilo 311
15.10. socketserver 313
15.11. Operaciones bloqueantes 315
15.12. Alternativas a la E/S bloqueante 315
15.13. Servidor TCP asíncrono con `select()` 317
15.14. Servidor TCP asíncrono con `asyncio` 319
15.15. Cliente TCP asíncrono con `asyncio` 322
15.16. Servidor UDP asíncrono con `asyncio` 324

16 Publicador-Suscriptor **327**

16.1. Chat UDP para dos 328
16.1.1. Paso 1: Mensaje unidireccional 328
16.1.2. Paso 2: Devuelve el saludo 330
16.1.3. Paso 3: Libertad de expresión 330
16.1.4. Paso 4: Habla cuando quieras 332
16.1.5. Paso 5: Todo en uno 334
16.2. Chat asíncrono con `select()` 335
16.3. Chatroom UDP 337
16.4. Chatroom TCP 339
16.5. Chatroom con MQTT 341

17 Calidad de servicio **345**

17.1. Latencia y *jitter* 347
17.1.1. Causas del retardo y del *jitter* 347

17.1.2. Medición del retardo 349
17.1.3. Cálculo del *jitter* 349
17.2. Cálculo de la tasa de transferencia 351
17.2.1. Tasa instantánea 353
17.2.2. Promedio acumulado (CA) 353
17.2.3. Media móvil simple (SMA) 354
17.2.4. Media móvil exponencial (EMA) 355
17.2.5. Consideraciones sobre el cálculo de tasa 356
17.3. Control de flujo TCP como limitador de tasa 357
17.3.1. Limitación de tasa en el receptor 358
17.3.2. Limitación de tasa en el emisor 363
17.4. Medición de tasa de transferencia y de pérdida 363
17.5. Garantizar prestaciones 365
17.6. Servicios diferenciados 365
17.6.1. Cubos de tokens 366
17.6.2. Clasificación y marcado 369
17.6.3. Policing 372
17.6.4. Encolado 372
17.6.5. Planificación 373
17.6.6. Shaping 374
17.7. QoS en Linux 375
17.7.1. qdisc 375
17.7.2. Configurando un sistema de colas 379
17.7.3. Marcado de paquetes con netfilter 390
17.8. QoS en origen 394

18 Serialización **397**

18.1. Representación, solo eso 398
18.2. Los enteros de Python 399
18.3. Caracteres 400

18.4. Tipos multibyte y ordenamiento 402
18.5. Cadenas de caracteres y secuencias de bytes 404
18.6. Empaquetado 406
18.7. Desempaquetado 408
18.8. Formatos de serialización binaria 408
18.9. Serialización textual 409

19 Captura y análisis **411**
19.1. `tshark` 412
19.1.1. Acceso privilegiado 413
19.1.2. Selección de la interfaz de captura 413
19.1.3. Limitando la captura 415
19.1.4. Filtros de captura 415
19.1.5. Formato de salida 415
19.1.6. Filtros de visualización 418
19.1.7. Estadísticas 419
19.2. wireshark 422
19.2.1. Interfaz gráfica 422
19.2.2. Captura y filtrado 424
19.3. Captura de flujos 427

20 Sockets raw **431**
20.1. Acceso privilegiado 432
20.2. Tipos de sockets raw 433
20.3. Sockets `AF_PACKET:SOCK_RAW` 433
20.3.1. Construir y enviar tramas 435
20.3.2. Implementando un `arping` 436
20.4. Sockets `AF_INET:SOCK_RAW` 439
20.4.1. Capturando mensajes 439
20.4.2. Enviando 440

21 DNS **443**

21.1. Dominios y zonas 446
21.2. La zona raíz 448
21.3. Resolución de nombres 449
21.4. Cachés e información obsoleta 452
21.5. Configuración de zona 454
21.6. Resolución inversa 455
21.7. Replicación 455
21.7.1. CDN 456
21.8. Transporte 457
21.9. Multicast DNS 458
21.10. Dynamic DNS 458
21.11. Sumideros DNS *(sinkhole)* 459

22 SSH **461**

22.1. Shell segura 461
22.2. Configuración 462
22.3. Acceso con clave pública 463
22.3.1. Agente SSH 464
22.4. Autenticación con certificado 464
22.4.1. Certificado de usuario 465
22.5. Copia de archivos con SCP 467

23 Invocación remota **469**

23.1. Protocolos de petición-respuesta 470
23.2. Medidas de tolerancia a fallos 472
23.3. Operaciones idempotentes 473
23.4. Semántica de invocación 473
23.5. Mecanismo de invocación remota 475
23.5.1. pico-rpc 477

23.5.2. ONC RPC. 480
23.5.3. gRPC. 483
23.5.4. JSON-RPC 489
23.6. RMI 491
23.6.1. pico-rmi 493
23.6.2. Java RMI 496
23.6.3. ZeroC Ice 499
23.7. Localización de servicios 505
23.7.1. Registro de servidores y objetos. 506
23.7.2. Resolución de nombres. 507
23.8. Invocación y despachado asíncrono 508
23.8.1. Invocación asíncrona 509
23.8.2. Despachado asíncrono 510
23.9. Cap'n Proto 512
23.9.1. ¿Sin serialización? 512
23.9.2. Referencias como *capabilities* 513
23.9.3. Promesas y *pipelining* 515
23.10. REST 516
23.10.1. Quizzy 517
23.10.2. Recursos y representaciones 518
23.10.3. Interfaz uniforme 520
23.10.4. Sin estado 521
23.10.5. Validación y documentación. 522
23.10.6. Errores y semántica de invocación 523
23.11. Versionado 524
23.12. Rendimiento 529

24 Epílogo **535**

A Comandos habituales **539**
A.1. Ficheros y directorios 539

A.2. Sistema 541
A.3. Procesos 542
A.4. Usuarios y permisos 542

B Otros comandos de red **543**
B.1. Otros *ping* 543
B.2. mtr 543
B.3. ss 544

C Docker **547**
C.1. Imágenes 547
C.2. Contenedores 548
C.3. `Dockerfile` 550
C.4. Docker Compose 550

D Unidades **553**
D.1. Memoria 553
D.2. Almacenamiento 553
D.3. Tasa o velocidad de transmisión 554

Referencias **563**

Listado de acrónimos

EIF	Endpoint-Independent Filtering
ADF	Address-Dependent Filtering
APDF	Address and Port-Dependent Filtering
EIM	Endpoint-Independent Mapping
ADM	Address-Dependent Mapping
APDM	Address and Port-Dependent Mapping
BDP	Bandwidth Delay Product
TOS	Type of Service
IOS	Internetwork Operating System
DRR	Deficit Round Robin
CSMA/CA	Carrier Sense Multiple Access with Collision Avoidance
htb	Hierarchical Token Bucket
EBS	Excess Burst Size
CBS	Committed Burst Size
PBS	Peak Burst Size
trTCM	Two Rate Three Color Marker
8P8C	Eight Position Eight Contact
A	Address
AAAA	Quad-A; DNS record for IPv6 addresses
ABR	Area Border Router
ACK	Acknowledgement
ADSL	Asymmetric Digital Subscriber Line
AFxy	Assured Forwarding
ALG	Application Layer Gateway
AMD	Asynchronous Method Dispatching
AMI	Asynchronous Method Invocation
ANSI	American National Standards Institute
API	Application Program Interface
AQM	Active Queue Management
ARP	Address Resolution Protocol

ARQ Automatic Repeat reQuest
AS Autonomous System
ASBR Autonomous System Border Router
ASCII American Standard Code for Information Interchange
ASIC Application-Specific Integrated Circuit
ASN Abstract Syntax Notation
ASPATH AS-PATH
ATM Asynchronous Transfer Mode
BGP Border Gateway Protocol
BOOTP Bootstrap Protocol
BSD Berkeley Software Distribution
CA Certificate Authority
CAD Computer-Aided Design
CAN Control Area Network
CBWFQ Class Based WFQ
CD Continuous Delivery
CDN Content Delivery Network
CE Congestion Experienced
CGNAT Carrier-Grade NAT
CGN Carrier-Grade NAT
CHAP Challenge Handshake Authentication Protocol
CI Continuous Integration
CIDR Classless Interdomain Routing
CIR Committed Information Rate
CLI Command Line Interface
CA Cumulative Average
CNAME Canonical Name
CoAP Constrained Application Protocol
CORBA Common Object Request Broker Architecture
CPU Central Processing Unit
CR Carriage Return
CRC Cyclic Redundancy Check
CSS Cascading Style Sheets
CSV Comma Separated Values
CWR Congestion Window Reduced
D-BUS Desktop BUS
DDNS Dynamic DNS
DF Don't Fragment

DGC	Distributed Garbage Collector
DHCP	Dynamic Host Configuration Protocol
DiffServ	Differentiated Services
DKIM	DomainKeys Identified Mail
DMARC	Domain-based Message Authentication, Reporting and Conformance
DNAT	Destination NAT
DNS-SD	DNS Service Discovery
DNS	Domain Name System
DNSSEC	DNS Security Extensions
DoH	DNS over HTTP
DoQ	DNS over QUIC
DoS	Denial of Service
DoT	DNS over TLS
DSCP	Differentiated Services CodePoint
E/S	Entrada/Salida
eBGP	External BGP
ECE	ECN Echo
ECMP	Equal-Cost Multi-Path
ECN	Explicit Congestion Notification
ECT	ECN Capable Transport
EGP	Exterior Gateway Protocol
EIR	Excess Information Rate
EMA	Exponential Moving Average
EOL	End Of Line
ESI	Escuela Superior de Informática
FCS	Frame Check Sequence
FIFO	First In First Out
FLSM	Fixed Length Subnet Mask
FPGA	Field-Programmable Gate Array
FQDN	Fully Qualified Domain Name
FRR	Free Range Routing
FSF	Free Software Foundation
FTP	File Transfer Protocol
FTTH	Fiber To The Home
GCC	GNU Compiler Collection
GID	Group IDentifier
GIL	Global Interpreter Lock

GNOME	GNU Network Object Model Environment
GNU	GNU is Not Unix
GPL	General Public License
GPS	Global Positioning System
gRPC	«g» Remote Procedure Calls
GSM	Global System for Mobile Communications
GUI	Graphical User Interface
HATEOAS	Hypermedia As The Engine Of Application State
HDLC	High-Level Data Link Control
HFC	Hybrid Fiber-Coaxial
HTTP	Hypertext Transfer Protocol
HTTP/2	HTTP versión 2
HTTP/3	HTTP versión 3
HTTPS	HTTP sobre SSL
Hurd	Hird of Unix-Replacing Daemons
I/O	Input/Output
IANA	Internet Assigned Numbers Authority
iBGP	Internal BGP
IBM	International Business Machines
ICANN	Internet Corporation for Assigned Names and Numbers
ICE	Interactive Connectivity Establishment
Ice	Internet Communications Engine
IceP	Ice Protocol
ICMP	Internet Control Message Protocol
IDL	Interface Definition Language
IEEE	Institute of Electrical and Electronics Engineers
IETF	Internet Engineering Task Force
IGMP	Internet Group Management Protocol
IGP	Interior Gateway Protocol
IHL	Internet Header Length
IMAP	Internet Message Access Protocol
IntServ	Integrated Services
IoT	Internet of Things
IP	Internet Protocol
IPC	Inter-Process Communication
IPTV	IP Television
IPv4	IP versión 4
IPv4LL	IPv4 Link-Local

IPv6 IP versión 6
IPX Internetwork Packet Exchange
IRDP ICMP Router Discovery Protocol
IS-IS Intermediate System to Intermediate System
ISN Initial Sequence Number
ISO International Organization for Standardization
ISP Internet Service Provider
JRMP Java Remote Method Protocol
JSON JavaScript Object Notation
JVM Java Virtual Machine
KiB Kibibyte (1024 bytes)
LAN Local Area Network
LCP Link Control Protocol
LED Light Emitting Diode
LF Line Feed
LLM Large Language Model
LLQ Low Latency Queuing
LPM Longest Prefix Match
LSA Link-State Advertisement
LSB Least Significant Bit
LSDB Link State Database
LSP Language Server Protocol
LTE Long Term Evolution
MAC Media Access Control
MCP Model Context Protocol
MD5 Message-Digest Algorithm 5
mDNS Multicast DNS
MF More Fragments
MiB Mebibyte (1024 KiB)
MIT Massachusetts Institute of Technology
MitM Man in the Middle
MQTT Message Queue Telemetry Transport
MSB Most Significant Bit
MSL Maximum Segment Lifetime
MSS Maximum Segment Size
MTU Maximum Transmission Unit
MX Mail eXchange
NACK Negative Acknowledgement

NAPT	Network Address Port Translation
NAS	Network Attached Storage
NAT	Network Address Translation
NCP	Network Control Protocol
NFS	Network File System
NIC	Network Interface Controller
NS	Name Server
NTP	Network Time Protocol
OAM	Operations, Administration and Maintenance
ONC	Open Network Computing
ONT	Optical Network Terminal
ORM	Object-Relational Mapping
OSI	Open Systems Interconnection
OSPF	Open Short Path First
OUI	Organizationally Unique Identifier
P2P	Peer to Peer
PAP	Password Authentication Protocol
PC	Personal Computer
AC	Class Selector
DF	Default Forwarding
EF	Expedited Forwarding
PHB	Per-Hop Behavior
PID	Process IDentifier
PIR	Peak Information Rate
PLPMTUD	Packetization Layer Path MTU Discovery
PMTU	Path MTU
PMTUD	Path MTU Discovery
POO	Programación Orientada a Objetos
PoP	Point of Presence
POP	Post Office Protocol
POSIX	Portable Operating System Interface; UNIX
PPP	Point to Point Protocol
PTR	Pointer
SP	Strict Priority
WFQ	Weighted Fair Queuing
WRR	Weighted Round Robin
QoS	Quality of Service
QUIC	Quick UDP Internet Connections

R	Request
RAE	Real Academia Española
RD	Recursion Desired
RDP	Remote Desktop Protocol
RED	Random Early Detection
REST	REpresentational State Transfer
RFC	Request For Comments
RIP	Routing Information Protocol
RIPE	Réseaux IP Européens
RIR	Regional Internet Registry
RJ45	Registered Jack 45
RMI	Remote Method Invocation
ROM	Read Only Memory
RPC	Remote Procedure Call
RR	Request-Reply
RRA	Request-Reply-Acknowledge
RSA	Rivest, Shamir y Adleman
RTO	Retransmission TimeOut
RTP	Real-time Transport Protocol
RTT	Round-Trip Time
RTTVAR	Round-Trip Time Variation
SA	Sistema Autónomo
SACK	Selective Acknowledgement
SCP	Secure Copy Protocol
SCTP	Stream Control Transmission Protocol
SDP	Session Description Protocol
SDSL	Symmetric Digital Subscriber Line
sfq	Stochastic Fairness Queueing
SI	Sistema Internacional
SIP	Session Initiation Protocol
SLA	Service Level Agreement
SLI	Service Level Indicator
Slice	Specification Language for Ice
SLO	Service Level Objective
SMA	Simple Moving Average
SMTP	Simple Mail Transport Protocol
SNAT	Source NAT
SND.UNA	Send Unacknowledged Number

SNMP	Simple Network Management Protocol
SO	Sistema Operativo
SOA	Service Oriented Architecture
SP	Strict Priority
SPF	Sender Policy Framework
SPT	Shortest Path Tree
SQL	Structured Query Language
SRT	Service Response Time
srTCM	Single Rate Three Color Marker
SRTT	Smoothed Round-Trip Time
SRV	Service
SSH	Secure SHell
SSL	Secure Socket Layer
STP	Spanning Tree Protocol
STUN	Session Traversal Utilities for NAT
SWS	Silly Window Syndrome
SYN	Synchronization
TCB	Transmission Control Block
TCP	Transmission Control Protocol
TCP/IP	Arquitectura de protocolos de Internet
TFTP	Trivial File Transfer Protocol
TIC	Tecnologías de la Información y las Comunicaciones
TLD	Top Level Domain
TLS	Transport Layer Security
TOML	Tom's Obvious, Minimal Language
TTL	Time To Live
TTY	TeleTYpewriter
TURN	Traversal Using Relays around NAT
TXT	Text
UCLM	Universidad de Castilla-La Mancha
UDP	User Datagram Protocol
UID	User IDentifier
UPnP	Universal Plug and Play
URI	Universal Resource Identifier
URL	Uniform Resource Locator
USB	Universal Serial Bus
UTF	Unicode Transformation Format
UTF8	UTF de 8 bits

UUID	Universally Unique Identifier
VLAN	Virtual LAN
VLSM	Variable Length Subnet Mask
VM	Virtual Machine
VoIP	Voice over IP
VPN	Virtual Private Network
WAN	Wide Area Network
WFQ	Weighted Fair Queuing
WiFi	Wireless Fidelity
WRED	Weighted RED
WRR	Weighted Round Robin
WWW	World Wide Web
XDR	eXternal Data Representation
XML-RPC	RPC sobre XML
XML	Extensible Markup Language
YAML	YAML Ain't Markup Language
ZWP	Zero Window Probe

Prefacio

Las redes de comunicaciones, y especialmente Internet, se convirtieron ya hace décadas en una parte esencial de nuestras vidas. Internet cambió radicalmente nuestra forma de comprar, viajar, aprender, relacionarnos y la gran mayoría de nuestras actividades cotidianas. Fue un cambio mucho más importante que la invención de la imprenta o la revolución industrial, supuso un nuevo concepto de comunicación, un medio de difusión de información y conocimiento a una escala sin precedentes. Y es precisamente la comunicación lo que nos convierte en seres sociales, en humanos. Las redes cambiaron cómo vivimos y lo que somos.

Todo ello es motivo suficiente para que cualquier persona —y en particular cualquier técnico— se interese por el funcionamiento de las redes de computadores. Entender cómo son y cómo funcionan las redes ayuda a tomar conciencia de sus posibilidades y oportunidades, y también de sus riesgos.

Destripando Internet

Según la RAE una de las acepciones de *destripar* es 'sacar lo interior de algo'; y eso vamos a hacer aquí, sacar y estudiar las tripas de Internet: la tecnología TCP/IP que la hace posible, los mecanismos y los protocolos clave en su funcionamiento. La mejor manera de conseguir esto es con un enfoque muy práctico, y por suerte, mucha de la tecnología de Internet es sorprendentemente accesible y está disponible para cualquiera. Esto es así en buena medida por la naturaleza abierta de las normas y protocolos que la rigen y por el omnipresente protagonismo del software libre en este ámbito. Basaremos este enfoque práctico en dos tecnologías muy concretas:

- El sistema operativo Debian GNU/Linux.
- El lenguaje de programación Python.

Debian GNU/Linux

GNU es un sistema operativo estilo POSIX concebido y desarrollado bajo el concepto de software libre. Esto tiene dos implicaciones muy importantes:

- GNU incorpora todas las ventajas prácticas y técnicas de la tradición de la familia de sistemas Unix. La tecnología base de Internet fue desarrollada sobre Unix, concretamente BSD. Internet y toda la tecnología que la hace posible nació en Unix. GNU es heredero de todo ese legado.
- GNU está disponible para cualquiera, en cualquier parte, sin ninguna limitación. Es la esencia del *software libre*: se puede estudiar, compartir, mejorar y, por supuesto, crear industria y riqueza con él. Debian GNU/Linux, la distribución que usaremos a lo largo del libro, hereda ese espíritu y cuenta con una enorme comunidad de usuarios y desarrolladores. En C1 descubrirás la historia y filosofía de GNU y del proyecto Debian.

El entorno de trabajo y herramientas que se utilizan en este libro están disponibles en el sistema operativo Debian GNU/Linux [1]. Por simplicidad y para un máximo aprovechamiento, te aconsejamos instalar esta distribución en tu computador, ya que este será el entorno que asumiremos cuando se hable de configuración o se mencionen comandos del sistema. Todos los programas que emplearemos están disponibles como paquetes oficiales y se pueden instalar con el gestor de paquetes `apt`.

Por supuesto, cualquier distribución basada en GNU puede ser perfectamente válida para la realización de los ejemplos y ejercicios prácticos propuestos, aunque quizá podrías encontrar ciertas diferencias. Simplemente ten en cuenta, que en ausencia de indicaciones concretas, debes asumir que hablamos de Debian o sus derivados, siendo Ubuntu el más popular.

También asumimos que dispones de un computador con al menos una interfaz de red Ethernet o WiFi con acceso a Internet convenientemente configurada y también que tienes una cuenta de usuario con privilegios de administrador en tu propio computador.

Python

Python es un lenguaje de programación moderno y muy popular. A pesar de ser un lenguaje muy completo y potente, su aprendizaje resulta sorprendentemente sencillo. Permite, incluso a un programador novato, ser

productivo en muy poco tiempo, en comparación con otros lenguajes como Java, C++, o C#. Python ha conseguido una gran notoriedad en los últimos años por su versatilidad en ciencia de datos (análisis, procesamiento y visualización) y por su amplio uso en muchas de las tecnologías relacionadas con la inteligencia artificial. Pero Python va mucho más allá.

Una de las características que lo hace especialmente adecuado para la «programación de redes» es que su librería estándar respeta la nomenclatura y conceptos del API de programación de POSIX, y eso resulta muy útil para encontrar documentación y bibliografía, además de facilitar la *traducción* a/desde Python de las llamadas al sistema POSIX, que están implementadas en C.

Python también es software libre, lo que significa que está disponible para cualquiera en cualquier plataforma. Está respaldado por una enorme comunidad de usuarios y desarrolladores, y suele ser muy fácil encontrar ayuda, incluso en temas muy específicos.

En todos los ejemplos, listados de código y programas que se proporcionan junto con el libro, utilizaremos siempre Python 3.

Contenido del libro

La mayoría de los libros de redes y comunicaciones suelen tener una estructura similar. Eso se debe a que las redes se estudian, e incluso implementan, mediante los denominados modelos de referencia, especialmente el «modelo OSI». Este modelo divide todo el conocimiento relacionado con las comunicaciones en redes en una serie de capas dispuestas como una pila. La primera (la inferior) es la capa física y se ocupa de los detalles de más bajo nivel llegando a los valores de voltaje o el tipo de cableado. La última es la capa de aplicación y se ocupa del más alto nivel, como por ejemplo, la estructura de un *e-mail*. Los autores suelen seguir este modelo ya sea empezando por la inferior hacia arriba o desde la superior hacia abajo, explicando las tecnologías y funcionamiento de cada componente capa a capa.

El problema con este enfoque es que esas capas no son ni de lejos tan independientes como se dice. Hay muchos mecanismos esenciales que requieren entender los detalles de lo que ocurre en otras capas. A nosotros no nos parece buena idea ignorarlos o postergar su explicación, que es lo que suele ocurrir. Para evitarlo, en este libro hemos preferido centrarnos en los conceptos y mecanismos de las redes aunque su explicación pueda involucrar varias capas. Pensamos que esto es más adecuado para novatos aunque quizá sea menos formal. En todo caso, no vamos a ignorar los modelos de referencia; son muy útiles para ubicar cada componente y entender cómo

se relaciona con el resto. Los mencionaremos con frecuencia, simplemente no los utilizaremos como hilo conductor.

Este es un pequeño resumen de la estructura general del libro:

Debian GNU/Linux (C1)
: presenta el sistema operativo que utilizaremos a lo largo del libro: su filosofía y características, así como la historia de sus precursores: el el sistema Unix, proyecto GNU y el núcleo Linux.

Shell (C2) y Python (C3)
: introducen dos tecnologías que vas a necesitar para seguir los frecuentes comandos de consola y pequeños programas que se utilizan en todo el libro. La shell y Python son el soporte pragmático del texto, de modo que si no tienes experiencia previa con ellos, es muy conveniente que los leas y practiques los ejemplos que incluyen. Si consideras que tienes suficiente familiaridad con ellos, puedes omitir estos capítulos.

Internet (C4)
: aborda algunos de los conceptos más básicos: red, interred o protocolo, y los modelos de referencia, incluido el citado modelo OSI.

Protocolos esenciales (C5)
: introduce los protocolos elementales de la familia TCP/IP. Aunque solo se cuenta lo mínimo de su funcionamiento, sí se presenta el formato de cada mensaje. Esto es así porque queremos que te pongas manos a la obra lo antes posible, que captures, veas, y entiendas tráfico real sin esperar a leer toda la teoría.

Sockets (C6)
: Los *sockets* son la herramienta de programación elemental para escribir software de redes. Aunque muchos autores los consideran un tema avanzado y muchos libros los omiten completamente, aquí incluimos esta introducción muy al principio. Entender lo que el programador puede hacer o no te va a ayudar a comprender todo lo demás.

Direccionamiento IP (C7)
: Las redes se utilizan para comunicar computadores —eso seguro que ya lo sabes— y para eso cada computador debe tener una dirección única. Este capítulo explica con detalle cómo se organizan las direcciones IP dentro de una red y cómo se puede saber a qué red está conectado cada computador a partir de su dirección.

Interconexión de redes (C8)
: La interconexión de redes es el concepto clave de Internet. Aquí verás cómo los routers son capaces de mover paquetes de una red a

otra, conceptos clave como la tabla de rutas, los métodos de entrega de paquetes o cómo se resuelve el problema de la fragmentación de paquetes.

Encaminamiento dinámico (C9)
Las redes cambian constantemente, y los routers necesitan adaptarse a esos cambios. Aquí verás los mecanismos que permiten a los routers trabajar juntos para encontrar la mejor ruta en cada momento y así llevar cada paquete hasta su destino.

Configuración IP (C10)
Aprenderás cuáles son los datos de configuración esenciales que requiere un nodo para poner conectarse de forma correcta a una red, cómo conseguir esos datos y cómo es posible automatizar el proceso.

Redes privadas (C11)
explica qué es una red privada y para qué sirve, sus distintos tipos, el direccionamiento privado y el mecanismo NAT: qué problema resuelve y cuáles provoca.

Confiabilidad y control de flujo (C12)
El protocolo IP no da ninguna garantía de que los datos vayan a llegar sin errores, ni siquiera de que vayan a llegar. Aquí verás cómo es posible añadir mecanismos que garanticen la entrega correcta de los mensajes, corrigiendo errores y retransmitiendo los mensajes perdidos, así como regular el flujo entre emisor y receptor.

Confiabilidad y control de flujo en TCP (C13)
muestra cómo TCP implementa en la práctica esos mecanismos: cómo se establece y termina una conexión, qué implicaciones tiene su control de flujo y cómo detecta y resuelve los errores de transmisión.

Control de congestión (C14)
explica qué es la congestión de la red, cómo aparece y cuáles son los mecanismos que permiten combatirla.

Cliente-servidor (C15)
El modelo cliente-servidor es el que siguen la mayoría de aplicaciones de red. En este capítulo verás en qué consiste y cómo se pueden diseñar e implementar servidores eficientes y escalables aprovechando el soporte que ofrece la librería estándar de Python. También verás cómo aplicar técnicas de concurrencia, paralelismo y entrada/salida asíncrona.

Publicador-subcriptor (C16)
: es otro de los modelos de interacción clásicos. Permite la comunicación entre múltiples participantes de forma más asíncrona y flexible que el modelo cliente-servidor, siempre claro, que el diseño de la aplicación lo permita. A pesar de no ser tan popular, es muy utilizado en aplicaciones de mensajería y propagación de notificaciones.

Calidad de servicio (C17)
: presenta los parámetros básicos que la caracterizan —tasa de transferencia, latencia, *jitter* y pérdida de paquetes—, cómo medirlos y cómo la priorización del reenvío y emisión de paquetes influye decisivamente para asegurar estos parámetros.

Serialización (C18)
: trata sobre cómo las aplicaciones de red pueden intercambiar datos arbitrariamente complejos en formato binario o texto de forma eficiente. Verás las bases de la codificación de datos aplicada a código listo para probar.

Captura y análisis (C19)
: profundiza en el potencial que ofrecen las herramientas de captura de tráfico `tshark` y `wireshark`. El análisis del tráfico de red es importante para muchas situaciones y es clave en campos tan pujantes como la ciberseguridad. Aquí verás cómo capturar, filtrar y analizar tráfico de forma práctica y sencilla.

Sockets raw (C20)
: explica qué son y para qué se utilizan este tipo de sockets y su papel en el tratamiento y generación de mensajes a bajo nivel. Verás también ejemplos prácticos para escribir programas capaces de capturar y generar mensajes de red a cualquier nivel de la pila de protocolos.

DNS (C21)
: presenta el servicio de nombres de las redes TCP/IP, que asocia nombres únicos a direcciones IP de forma similar a como una agenda asocia nombres a números de teléfono. Verás cómo se organiza y cómo funciona el proceso de resolución de nombres.

SSH (C22)
: describe el protocolo que permite establecer conexiones seguras con nodos remotos: ejecutar una shell remota, transferir archivos y crear túneles capaces de transportar otras conexiones.

Epílogo (C24)
: cierra el libro con algunas reflexiones finales sobre su origen, motivación y desarrollo.

Convenciones y formato

A lo largo del libro se emplea un marcado tipográfico coherente para que puedas reconocer de un vistazo cada tipo de elemento:

- El texto en `tipografía monoespaciada` identifica elementos software: comandos, aplicaciones, ficheros, funciones, identificadores o tipos de datos. Por ejemplo, la aplicación `tshark`, el fichero `/etc/passwd` o la función `open()`.
- La *cursiva* señala términos en inglés, como *checksum* o *overlay*. Cuando conviene aclarar el término original de un concepto ya traducido, este aparece entre paréntesis, por ejemplo servidor de nombres *(name server)*.
- Las comillas «latinas» acotan un término que se está definiendo, una etiqueta o el nombre propio de algún elemento, o bien un uso figurado o coloquial del lenguaje. Por ejemplo, el sistema operativo «mata» al proceso, o la columna «origen» de una tabla.
- La **negrita** se reserva para destacar, muy puntualmente, algún concepto especialmente relevante.
- En las capturas de consola (o simplimente «consolas»), el indicador de sistema *(prompt)* también aporta información: aparece '`$`' cuando el comando se ejecuta con una cuenta de usuario convencional y '`#`' cuando requiere privilegios de administrador (`root`).
- Los valores en hexadecimal se expresan siempre con el prefijo `0x`, como en `0x1234`.
- Cuando en el texto hagamos referencia a una llamada al sistema usaremos el superíndice ^{sc} *(system call)*, como por ejemplo en: `write()`^{sc}.
- La primera vez que se mencione un programa, comando o aplicación, aparecerá el nombre del paquete Debian que lo contiene, por ejemplo: `ip` (`iproute2`), a menos que el paquete se llame igual que el programa o se trate de un paquete *esencial*, es decir, que se instala por defecto con el sistema.

Anglicismos

En el texto se utilizan muchos términos en inglés incluso cuando existen equivalentes en español. Es así porque priorizamos la claridad y la precisión técnica por encima de la corrección lingüística. Nuestro principal objetivo

es que el lector entienda la redacción y comprenda los conceptos que se abordan, y eso no se consigue empleando términos que nadie utiliza en la ámbito profesional, por muy correctos que sean. Además, siempre que se introduzca un nuevo concepto, incluso si es de uso común en español, para evitar cualquier ambiguiedad se incluirá también el término en inglés entre paréntesis.

Hipertexto

En la versión `pdf` del libro, todas las referencias a capítulos, secciones (denotadas con §), figuras, tablas, listados de código y citas bibliográficas son enlaces que llevan al lugar dónde se definen. En la mayoría de los casos, si el visor `pdf` lo soporta, es suficiente con colocar el cursor del ratón encima para obtener una vista previa.

En cuanto a los acrónimos, como en cualquier documento técnico, aparecen en gran cantidad. Cuando se trate de un acrónimo poco habitual se definirá la primera vez que se utilice. Si es de uso más común tendrás que consultar el listado de acrónimos en la pagina XIX, aunque el visor `.pdf` también puede mostrarlos del mismo modo que las referencias.

Ejemplos y código fuente

Todos los ejemplos que aparecen en el libro están disponibles para descarga a través del repositorio `git`[1] en `https://github.com/destripando-internet/code`. Aunque es posible descargar este contenido individualmente o como un archivo comprimido, te aconsejamos utilizar el sistema de control de versiones `git`. Las rutas relativas a este repositorio que aparecen en el texto son 'clickables' y van precedidas del logotipo de github ( ).

Si encuentras alguna errata u omisión en los programas de ejemplo, por favor, utiliza la herramienta de gestión de incidencias (*issue tracker*) accesible desde `https://github.com/destripando-internet/code/issues` para notificarlo a los autores.

Sobre este documento

Este documento está tipografiado con LaTeX en un sistema Debian GNU/Linux. Las figuras y la mayoría de los diagramas están realizados con `inkscape`.

[1] `https://git-scm.com`

Los fuentes de este documento también se encuentran en un repositorio `git` en `https://github.com/destripando-internet/book`. Si quieres colaborar activamente en su desarrollo o mejora, ponte en contacto con los autores.

Al igual que con los ejemplos, también existe una herramienta pública de gestión de incidencias en la que puedes notificar problemas o errores de cualquier tipo que hayas detectado en el documento.

Exención de responsabilidad

Este libro contiene abundantes ejemplos de comandos y programas destinados exclusivamente a fines formativos. En muchas ocasiones estos comandos implican privilegios de administración que pueden alterar la configuración y estado del sistema, y por tanto pueden tener consecuencias imprevistas y/o no deseables incluyendo la pérdida o exposición de datos o la interrupción de servicios.

Los autores y editores de este libro no son responsables de ningún daño o pérdida, directa o indirecta, ni al lector y a terceros, como consecuencia de la utilización de los ejemplos o la información expuesta. Se recomienda al lector ejecutar siempre los ejemplos y pruebas en un entorno de laboratorio y nunca en sistemas en producción. El lector asume toda responsabilidad por el uso que realice de los contenidos del libro.

Capítulo 1

Debian GNU/Linux

Este capítulo —nada técnico— repasa el origen del sistema operativo Debian, su filosofía y características. También algo de la historia de sus precursores: el proyecto GNU, el núcleo Linux y el sistema UNIX. Nos parece interesante incluir un pequeño resumen de lo que es una parte crucial de la historia de la informática y que ayuda a entender cómo hemos llegado hasta aquí.

Este relato nos lleva directamente a los albores de la informática y de la incipiente industria del software. Empecemos...

1.1. UNIX

Corría el año 1969, cuando un pequeño equipo de investigadores de Bell Labs (AT&T), liderado por Ken Thompson y Dennis Ritchie, empezó a trabajar en un nuevo sistema operativo inspirado en el proyecto Multics, en el que Bell Labs había participado, pero abandonó. Como un juego de palabras, llamaron UNIX al nuevo sistema ya que pretendía ser más sencillo que Multics.

El primer prototipo, como era lo habitual entonces, se escribió en ensamblador. Para facilitar la experimentación, que era una parte fundamental de la labor de Bell Labs, decidieron reescribir UNIX en un nuevo lenguaje de programación que el mismo equipo estaba creando: el lenguaje C[2]. Desde entonces, y hasta la actualidad, pasado más de medio siglo, los SO se siguen escribiendo mayoritariamente en C.

La decisión de usar C le proporcionó a UNIX un nivel de portabilidad sin precedentes, y eso a pesar de que no era el objetivo principal del cambio. Eso facilitó su adaptación con rapidez a diferentes arquitecturas por parte de Universidades y otros centros de investigación, lo que resultó clave en su gran éxito y popularidad.

También fue determinante un conjunto de pautas bien definidas, conocidas después como *filosofía Unix*. Algunos de sus principios más importantes son:

- Cada programa hace solo una cosa y la hace bien.
- Los programas trabajan con flujos. La salida de un programa es la entrada de otro.
- El texto plano es un formato de intercambio universal.
- Las tareas repetitivas son fáciles de automatizar *(scripting)*.
- Minimiza la interacción con el usuario.
- Todo se trata como un fichero, con las mismas primitivas.
- Falla rápido e informa claramente del error.

Los distintos Unix[1] que surgieron dominaron la informática profesional desde entonces hasta comienzos del siglo XXI. De entre los muchos que surgieron como AIX, HP-UX, SunOS, etc., uno de los más influyentes fue BSD, creado por la Universidad de California en Berkeley en 1977. BSD añadió funcionalidades muy importantes: soporte para los protocolos TCP/IP o la abstracción de socket, y herramientas muy populares como el editor `vi` o la C Shell (`csh`). Además, su licencia mucho más permisiva que la del UNIX de AT&T, permitió que tanto su diseño arquitectural como su código fueran reutilizados incluso en productos comerciales privativos. De BSD surgieron variantes como FreeBSD, OpenBSD y NetBSD que siguen existiendo a día de hoy, y su influencia llega a otros SO de uso doméstico que surgieron después como Apple macOS o Microsoft Windows.

1.2. El fin del software libre

Ya antes de UNIX, empresas como IBM vendían y mantenían los grandes y caros computadores que se utilizaban en facultades e instituciones de investigación. Como parte de sus servicios, proporcionaban a los clientes el código fuente de los programas que se ejecutaban en sus computadores[2] como requisito para su funcionamiento. Los programadores e investigadores podían estudiar, modificar y mejorar esos programas. Los propios fabricantes fomentaban esa práctica, redistribuyendo esas modificaciones al resto de sus clientes, bajo la convicción de que el valor de su negocio estaba únicamente en el hardware.

[1] «UNIX» es la marca comercial propiedad de AT&T —hoy un estándar de Open Group— mientras que «Unix» es un término genérico para referirse a los sistemas de este tipo.

[2] Como hacía AT&T con UNIX.

En 1976, un estudiante de 20 años del Harvard College escribió una breve carta abierta titulada «Open Letter to Hobbyists»[3] [3] en la que sostenía que la copia no autorizada de software comercial —en concreto, de Altair BASIC— era un robo que privaba de ingresos a sus autores y desincentivaba la producción de software de calidad. La carta defendía por primera vez la idea de que el software es un producto con valor propio, separable del hardware, y que debe pagarse como tal.

La carta no tuvo mucha repercusión en un primer momento, pero el año siguiente (1977) el periodista Andrew Pollack publicó un artículo en «The New York Times» apoyando abiertamente estas ideas. Su difusión en un periódico tan influyente marcó un punto de inflexión que propició que rápidamente empresas como Apple, MITS, Tandy o Commodore comenzaran a cerrar su software. Posteriormente esos cambios llegaron también a la legislación de protección industrial e intelectual. El nombre del estudiante, autor de la carta, era William Henry Gates III, más conocido como Bill Gates.

> AN OPEN LETTER TO HOBBYISTS
> February 3, 1976
> By William Henry Gates III
>
> To me, the most critical thing in the hobby market right now is the lack of good software courses, books and software itself. Without good software and an owner who understands programming, a hobby computer is wasted. Will quality software be written for the hobby market?
>
> Almost a year ago, Paul Allen and myself, expecting the hobby market to expand, hired Monte Davidoff and developed Altair BASIC. Though the initial work took only two months, the three of us have spent most of the last year documenting, improving and adding features to BASIC. Now we have 4K, 8K, EXTENDED, ROM and DISK BASIC. The value of the computer time we have used exceeds $40,000.
>
> The feedback we have gotten from the hundreds of people who say they are using BASIC has all been positive. Two surprising things are apparent, however, 1) Most of these "users" never bought BASIC (less than 10 % of all Altair owners have bought BASIC), and 2) The amount of royalties we have received from sales to hobbyists makes the time spent on Altair BASIC worth less than $2 an hour.
>
> Why is this? As the majority of hobbyists must be aware, most of you steal your software. Hardware must be paid for, but software is something to share. Who cares if the people who worked on it get paid?
>
> Is this fair? One thing you don't do by stealing software is get back at MITS for some problem you may have had. MITS doesn't make money selling software. The royalty paid to us, the manual, the tape and the overhead make it a break-even operation. One thing you do do is prevent

[3] *hobbyist* se refiere al 'ecosistema hobbyist', una comunidad de aficionados cuyo máximo exponente fue el Homebrew Computer Club (California, 1975).

> good software from being written. Who can afford to do professional work for nothing? What hobbyist can put 3-man years into programming, finding all bugs, documenting his product and distribute for free? The fact is, no one besides us has invested a lot of money in hobby software. We have written 6800 BASIC, and are writing 8080 APL and 6800 APL, but there is very little incentive to make this software available to hobbyists. Most directly, the thing you do is theft.
>
> What about the guys who re-sell Altair BASIC, aren't they making money on hobby software? Yes, but those who have been reported to us may lose in the end. They are the ones who give hobbyists a bad name, and should be kicked out of any club meeting they show up at.
>
> I would appreciate letters from any one who wants to pay up, or has a suggestion or comment. Just write me at 1180 Alvarado SE, #114, Albuquerque, New Mexico, 87108. Nothing would please me more than being able to hire ten programmers and deluge the hobby market with good software.
>
> Bill Gates
> General Partner, Micro-Soft

1.3. GNU

Una de las instituciones que empezaron a notar el cambio de mentalidad de la industria fue el Laboratorio de Inteligencia Artificial del MIT. Uno de los investigadores que trabajaba allí, Richard Stallman, vio amenazada su forma de trabajar por los recientes cambios en la legislación y en el modo de operar de las empresas de software. En 1983, Stallman decidió iniciar el proyecto GNU [4] con el objetivo de crear un SO completo que permitiera a los usuarios recuperar la libertad perdida: la de usar, estudiar, modificar y distribuir el software sin restricciones.

El nombre GNU es un acrónimo recursivo. Significa GNU is Not Unix (GNU no es Unix) a la vez que juega con la pronunciación de la palabra *new* — prácticamente igual. El nombre plasma la intención de crear un SO similar a Unix, pero completamente libre, con el objetivo de que nadie tuviera la necesidad de usar ningún Unix propietario, terminando por reemplazarlo completamente.

Así nació la idea del software libre, aunque en realidad no era más que un intento por recuperar lo que algunos pensaban que era la forma original y natural de tratar el software. La decisión de crear un sistema similar a Unix no fue tanto por su superioridad técnica, sino porque era el SO más popular en ese momento.

Stallman y su equipo crearon la FSF para coordinar y promover el desarrollo del sistema GNU, y desarrollaron la licencia GPL, que constituye la base legal para proteger la libertad del software.

Con los años, el proyecto GNU fue creciendo y creando todo tipo de herramientas importantes como editores, compiladores, librerías, etc. A finales de los 80, el sistema tenía todos los componentes necesarios salvo uno: el núcleo. Trabajaban activamente en un núcleo llamado HURD, pero su diseño era muy ambicioso y complejo, y su desarrollo se retrasaba continuamente.

1.4. Linux

Linus Torvalds, otro estudiante, este de la Universidad de Helsinki, estaba aprendiendo acerca de Minix, un pequeño sistema operativo con fines educativos basado en la filosofía de Unix creado por Andrew Tanenbaum. Aunque Torvalds admiraba Minix como herramienta didáctica, encontraba limitaciones en su diseño y era crítico con ciertas decisiones que afectaban gravemente a su rendimiento. Frustrado por las limitaciones que Tanenbaum imponía a la modificación y evolución de Minix, en 1991 anunció a la comunidad de Minix [5] que estaba trabajando en Linux, su propio núcleo.

> From: torvalds@klaava.Helsinki.FI (Linus Benedict Torvalds)
> Newsgroups: comp.os.minix
> Subject: What would you like to see most in minix?
> Summary: small poll for my new operating system
> Date: 25 Aug 91 20:57:08 GMT
> Organization: University of Helsinki
>
> Hello everybody out there using minix -
>
> I'm doing a (free) operating system (just a hobby, won't be big and professional like gnu) for 386(486) AT clones. This has been brewing since april, and is starting to get ready. I'd like any feedback on things people like/dislike in minix, as my OS resembles it somewhat (same physical layout of the file-system (due to practical reasons) among other things).
>
> I've currently ported bash(1.08) and gcc(1.40), and things seem to work. This implies that I'll get something practical within a few months, and I'd like to know what features most people would want. Any suggestions are welcome, but I won't promise I'll implement them :-)
>
> Linus (torvalds@kruuna.helsinki.fi)
>
> PS. Yes - it's free of any minix code, and it has a multi-threaded fs. It is NOT protable (uses 386 task switching etc), and it probably never will support anything other than AT-harddisks, as that's all I have :-(.

Pronto Torvalds decidió liberar Linux bajo la licencia GPL, si bien sus motivaciones siempre fueron más técnicas y pragmáticas que filosóficas o éticas. La FSF y el proyecto GNU adoptaron el núcleo Linux como una solución temporal hasta tener lista una versión estable de HURD, que nunca llegó. FSF desarrolló la librería `glibc`, un gran hito poco valorado, que permite

que el sistema GNU pueda ejecutarse sobre el núcleo Linux. También se adaptaron muchas otras herramientas, entre las que destaca GCC, el compilador de C y C++ que se convirtió en el estándar de facto para Linux y otros sistemas, y que más tarde incorporó otros lenguajes. La mayoría de estas herramientas se siguen manteniendo y mejorando a día de hoy.

1.5. GNU/Linux

La combinación del sistema GNU y el núcleo Linux dio lugar a lo que conocemos como GNU/Linux, un SO completo y libre. Sin embargo, instalar un sistema así no era nada sencillo y solo personas con conocimientos técnicos avanzados podían conseguirlo. Implicaba compilar el núcleo y la mayoría de las herramientas.

Para simplificar esta tarea aparecieron las primeras «distribuciones» de GNU/Linux. Una distribución es una colección organizada de paquetes, precompilados y mantenidos por un equipo de desarrolladores con el objetivo de conseguir una coherencia entre ellos. Una distribución suele incluir el núcleo Linux —normalmente compilado con un objetivo específico—, el sistema GNU, un programa de instalación y una selección de programas y herramientas. Conforme todos esos componentes van evolucionando y se ofrecen versiones nuevas, la distribución define también nuevas versiones *(releases)*. De ese modo los usuarios pueden instalar y mantener un sistema funcional con ciertas garantías en cuanto a estabilidad.

A lo largo de la década de los 90, GNU/Linux fue ganando popularidad y para principios de los 2000 ya era una alternativa viable a nivel industrial. Rápidamente se convirtió en la plataforma dominante para servidores en Internet, al mismo tiempo que copaba el Top 500 de los supercomputadores a nivel mundial, un nicho tradicionalmente dominado por variantes de Unix. Para 2004, GNU/Linux había superado en cuota de mercado a todos los Unix comerciales, consiguiendo así su objetivo en poco más de 20 años.

1.6. Debian GNU/Linux

En 1993, otro estudiante de la Universidad de Texas, Ian Murdock, anunció el proyecto Debian [6]. Su objetivo era crear una nueva distribución de GNU/Linux que hiciera énfasis en la libertad.

Debian fue una de las primeras grandes distribuciones de GNU/Linux[4], y se estableció rápidamente como una de las más profesionales y respetadas por la comunidad hasta la actualidad. En su compromiso por la libertad,

[4]Solo por detrás de Slackware

Debian diferencia claramente entre software libre y no libre, de modo que los usuarios puedan tomar decisiones informadas sobre lo que instalan.

Uno de los aspectos distintivos de Debian es su Contrato Social [5], un documento que establece los principios éticos y los compromisos que el proyecto asume con sus usuarios. Los podemos resumir en sus epígrafes:

1. Debian permanecerá 100 % libre.
2. Contribuiremos con la comunidad de software libre.
3. No ocultaremos los problemas.
4. Nuestra prioridad son nuestros usuarios y el software libre.
5. Asumimos que algunos usuarios necesitan software no libre.

También define las Directrices de Software Libre de Debian (DFSG) que establecen los criterios que deben cumplir los programas que se incluyen en la distribución:

1. Libre redistribución.
2. Disponibilidad del código fuente.
3. Permiso para realizar trabajos derivados.
4. Integridad del código fuente del autor.
5. No discriminar personas o grupos.
6. No discriminar en función de la finalidad.
7. Distribución de la licencia.
8. La licencia no debe ser específica para Debian.
9. La licencia no debe contaminar otro software.

Aunque su esfuerzo por mantener y promover la libertad del software es ciertamente destacable, no es el único aspecto que hace de este un proyecto de referencia entre las distribuciones GNU/Linux. En el plano organizativo, Debian se basa principalmente en su comunidad de *desarrolladores oficiales*[6], en su mayoría, voluntarios. Convertirse en desarrollador oficial está abierto a cualquier persona, pero requiere superar un proceso de selección bastante riguroso.

Existe un conjunto de normas públicas que regulan todos los procesos importantes del proyecto, como pueden ser la adopción o eliminación de paquetes oficiales, cambios en la *policy*[7] o la elección del líder. Todas estas decisiones se toman mediante votaciones públicas con debates en los que participa la comunidad. La ausencia de una empresa tras el proyecto y la gran transparencia del proceso son aspectos muy valorados y muy poco habituales en la industria del software, incluso en la del software libre.

[5] `https://www.debian.org/social_contract.es.html`

[6] Abreviado como DD (Debian Developer)

[7] Se refiere a la *Debian Policy Manual*: `https://www.debian.org/doc/debian-policy/`

Debian hace mucho énfasis en la estabilidad y el soporte a largo plazo. Para ir incorporando nuevas versiones de los programas, mantiene un ciclo de desarrollo basado en tres ramas diferentes: *stable*, *testing* y *unstable*[8]. La rama *stable* corresponde con las versiones oficiales *(releases)* y está pensada para usar en servidores y entornos de producción. Las versiones estables se publican aproximadamente cada 2 años y tienen un soporte de unos 3 años[9]. La rama *testing* es el entorno de pruebas para preparar la siguiente versión estable y, aunque puede seguir incorporando nuevas versiones de los paquetes, debe estar justificado. La rama *unstable* es la versión de desarrollo, y en ella pueden aparecer versiones nuevas prácticamente a diario.

Aunque es perfectamente posible utilizar *unstable* en un equipo de escritorio, existe el riesgo no despreciable de quedar con paquetes o dependencias rotas en momentos puntuales. Esto permite a cada usuario decidir el equilibrio de estabilidad/novedad que quiera para su sistema de una manera muy sencilla, y además es posible tener un sistema mixto con paquetes de diferentes ramas.

Debian, además, es una de las distribuciones que mantiene soporte para más arquitecturas. En el momento de escribir esto, la última versión estable publicada (Debian 13: Trixie) cuenta con soporte para 9 arquitecturas. Esto da una idea clara de su compromiso para ofrecer al usuario un sistema operativo realmente universal.

A lo largo de sus muchos años de vida, Debian ha influido e influye en la industria del software. No solo por las muchas distribuciones derivadas[10], algunas de ellas muy populares como Ubuntu, Kali o Raspbian, que siguen bastante integradas en sus procesos de desarrollo, tomando los paquetes actuales de Debian como base. También son referencia sus prácticas de desarrollo y empaquetado (especialmente su gestor `apt`), sus procesos de mantenimiento y despliegue, y su sistema de versiones y ciclo de desarrollo.

[8] Y una mucho más inestable llamada *experimental*

[9] Puedes ver la lista oficial de versiones estables en `https://www.debian.org/releases/`

[10] Unas 120 según Distrowatch.com

Capítulo 2

Shell

Al terminar este capítulo, entenderás:

- Qué es una shell y por qué es tan importante.
- Cómo trabajar con procesos, trabajos y señales.
- Cómo redirigir la entrada y salida de los programas.
- Cómo gestionar usuarios, grupos, propietarios y permisos.
- Cómo instalar paquetes y configurar repositorios.
- Cómo gestionar servicios.
- Qué son los parámetros del núcleo y cómo configurarlos.

El intérprete de comandos (*shell*) es probablemente una de las herramientas más interesantes del sistema operativo GNU/Linux, y de cualquier POSIX en general. Una shell no es más que un programa interactivo que permite al usuario ejecutar otros programas manteniendo cierto control sobre ellos. Como con cualquier tecnología o herramienta que merezca la pena, es fácil encontrar grandes partidarios y detractores. Lo cierto es que para un novato la shell tiene una apariencia extraña, rudimentaria, obsoleta y de aspecto absolutamente espartano en un mundo en el que las pantallas multi-táctiles, la realidad virtual o el reconocimiento de voz son algo cotidiano.

Una interfaz de comandos —a menudo denominada CLI— es mucho menos intuitiva[1] que una GUI. Sin embargo, cuando la capacidad y el nivel de detalle (y por tanto la complejidad) de la herramienta aumentan, la interfaz gráfica resulta mucho más difícil de desarrollar, requiere tanto o más entrenamiento, y su uso suele ser menos productivo que una interfaz de comandos. Ejemplos muy evidentes de esto se pueden encontrar en programas de diseño 3D, herramientas CAD o de edición de imágenes o vídeo, con literalmente miles de opciones repartidas en innumerables menús. De hecho, no es extraño que algunos de estos programas con interfaces gráficas

[1] El manejo *intuitivo* se refiere a la posibilidad de realizar un uso productivo de una herramienta sin conocimiento o formación previa. El adjetivo se puede aplicar tanto a la herramienta como al usuario.

tan complejas ofrezcan también algún tipo de sistema de comandos para sus usuarios más avanzados, como es el caso de `AutoCAD`.

Otra buena razón por la que la interfaz de comandos puede resultar más productiva es porque te da la posibilidad de crear secuencias de comandos *(scripts)*, permitiendo automatizar tareas repetitivas. Por ejemplo, cuando se quiere explicar una secuencia de acciones, incluso de complejidad media, usar comandos es mucho más sencillo que dar instrucciones para acceder a menús, botones y listas desplegables, algo que a menudo requiere grabar en vídeo la secuencia de acciones —los llamados *screencast*. En el caso de la shell, además existe todo un lenguaje de programación —el propio *shell script*— que dispone de variables, condicionales, bucles, funciones, etc., así que se puede ir mucho más lejos que una mera secuencia de comandos.

En todo caso, sería un error subestimar o considerar anticuada una herramienta simplemente por el hecho de basarse en una interfaz de comandos, y desde luego lo es en el caso de la shell de GNU/Linux.

script

Un script en un archivo de texto que contiene secuencias de comandos. Estos comandos son interpretados por otro programa que lee y procesa el archivo. Hablamos de *shell scripts* cuando el programa que los interpreta es una shell.

A menudo, las aplicaciones bien diseñadas definen un conjunto de acciones en una librería, que pueden ser utilizadas indistintamente desde *front-ends*[2] gráficos, línea de comandos, o incluso desarrollando otras aplicaciones a medida.

2.1. GNU Bash

Entre la gran variedad de shells que los sistemas POSIX han conocido desde principios de los 70, `Bash` es probablemente una de las más veteranas, conocidas y potentes. Aunque `bash` tiene características muy interesantes (como el autocompletado contextual), primero debemos abordar cuestiones básicas que todo usuario avanzado debería conocer.

Además de la propia shell, hay una colección de programas (agrupados bajo el nombre de GNU `coreutils`) que realizan acciones relativamente sencillas, pero que están diseñados de modo que se puedan combinar para realizar operaciones de complejidad nada trivial de forma bastante simple, una vez que se entiende un concepto clave: la redirección de la entrada/salida.

[2] Se denomina *front-end* a la capa de software que proporciona una interfaz —del tipo que sea— a un programa o librería que ofrece su funcionalidad por medio de una serie de funciones o clases (API).

Estos y otros programas diseñados por terceros, pero que respetan la misma filosofía, junto con el propio lenguaje *shell script*, proporcionan un ecosistema con infinitas posibilidades para todo tipo de tareas, que pueden ir desde la administración de sistemas hasta las más sofisticadas técnicas de *pen-testing*, pasando por el desarrollo de aplicaciones de todo tipo, incluso maliciosas. Y como la shell es la forma más habitual y flexible para administrar un sistema GNU/Linux, aprovecharemos este capítulo para aprender lo básico sobre gestión de procesos, programas, usuarios, archivos y servicios.

2.2. Valor de retorno

Los sistemas POSIX asumen que cuando un programa acaba devuelve un entero. Este valor es recogido por su proceso padre (que puede ser una shell) y ofrece información muy útil acerca del resultado de la ejecución.

Un programa que realiza su función satisfactoriamente debería devolver siempre un valor 0. Puede devolver un valor distinto para indicar que hubo un problema concreto, que su página de manual debería explicar.

Éste es el motivo por el que el estándar del lenguaje C dice que toda función `main()` ha de devolver un entero y que por defecto su valor de retorno debería ser cero. Según eso, el 'hola mundo' correcto en C es el siguiente:

```c
#include <stdio.h>
int main(int argc, char* argv[]) {
    puts("hello world\n");
    return 0;
}
```

Por ejemplo, la siguiente consola muestra la ejecución de `ls /`, que lista el contenido del directorio raíz:

```
~$ ls /
bin   etc         lib         mnt   root  selinux  tmp  vmlinuz
boot  home        lost+found  opt   run   srv      usr
dev   initrd.img  media       proc  sbin  sys      var
```

En este caso, la shell `bash` crea un proceso en el que ejecuta el programa `ls` con el argumento `/`, que representa la raíz del sistema de archivos. Cuando el programa termina, la shell recoge el valor de retorno. Ese valor se almacena en una *variable de entorno* llamada `?`[3].

[3] Sí, el signo de interrogación

Puedes imprimir el valor de cualquier variable utilizando el programa `echo` y colocando el símbolo `$` delante del nombre de la variable. O sea que puedes ver el valor de retorno de un comando si inmediatamente después de terminar, ejecutas:

```
~$ echo $?
0
```

Si se produce un error, el programa retorna un código de error, independientemente de que el programa muestre un mensaje de error. Lo puedes comprobar en el siguiente ejemplo:

echo

Escribe la cadena de texto que se le indique como argumento, incluyendo las variables (identificadores precedidos del símbolo `$`).

```
~$ ls noexiste
ls: cannot access noexiste: No such file or directory
~$ echo $?
2
```

La documentación de cada programa indica el significado de cada uno de los posibles valores de retorno. Si consultas la página de manual de `ls` (con `man ls` (`man-db`)) verás algo como:

```
Exit status:
    0      if OK,
    1      if minor problems (e.g., cannot access subdirectory),
    2      if serious trouble (e.g., cannot access command-line argument).
```

Las *señales* también se utilizan para indicar que un programa terminó como consecuencia de una situación anómala, por ejemplo si el SO (u otro programa) terminó («mató») el proceso. Por ejemplo, si el usuario pulsa `Control-C` mientras un programa se encuentra en ejecución bajo el control de una shell, esta le enviará la señal SIGINT (número 2). El valor de retorno del programa será entonces 130, que se entiende como -2, si el bit MSB se interpreta como bit de signo.

2.3. Opciones

La mayoría de los programas CLI disponen de opciones que modifican su comportamiento aportando gran versatilidad. Las opciones normalmente presentan dos formas equivalentes: un guion y una letra (`-h`) o dos guiones y una palabra (`--help`).

El formato largo (con '`--`') es el adecuado cuando se quieren comandos autoexplicativos (como en este libro) o en un *script*. El formato corto es más conveniente cuando el usuario escribe comandos directamente en la consola, por el ahorro de tiempo, obviamente.

Para evitar sobrecargar las explicaciones sobre cada comando, en este capítulo vamos a obviar la utilidad precisa de cada opción. El lector puede consultar la página de manual del programa (`man comando`) o con la opción `--help` para conocer la función de cada opción.

2.4. Procesos

Un 'proceso' es una abstracción del SO para ejecutar un programa conforme a determinados parámetros de seguridad, prioridad y privilegios de acceso a recursos. Cualquier proceso puede crear procesos *hijos*, que heredan muchas de las propiedades de su *padre*, tales como privilegios y descriptores de archivos abiertos. Para listar procesos se utiliza `ps` (🧰`procps`), que es una abreviatura de *process status*. Por ejemplo, un usuario puede ver los procesos asociados al terminal al que está conectado con el comando `ps T`. Cada proceso tiene un identificador numérico único asignado por el SO, llamado PID. Es el número que aparece en la primera columna del siguiente ejemplo:

```
~$ ps T
  PID TTY          TIME CMD
 4970 pts/2    00:00:00 bash
 5107 pts/2    00:00:01 gedit
 5164 pts/2    00:00:00 bash
 6021 pts/2    00:00:01 firefox-bin
 6204 pts/2    00:00:00 ps
```

La segunda columna, TTY (TeleTYpewriter), es el terminal al que está asociado; la tercera, (TIME), el tiempo de procesador otorgado al proceso hasta el momento; y por último (CMD), el comando tal como se lanzó.

El programa `ps` dispone de una amplia variedad de opciones. Por ejemplo, la opción `f` muestra la «genealogía» de los procesos. Fíjate que las opciones de `ps` no van precedidas de guion, como es habitual. Esto es lo que devuelve `ps f`:

```
~$ ps f
  PID TTY      STAT   TIME COMMAND
 4970 pts/2    Ss     0:00 bash
 5107 pts/2    T      0:01  \_ gedit
 5164 pts/2    S      0:00  \_ bash
 5300 pts/2    Sl     0:01      \_ /usr/lib/iceweasel/firefox-bin
 6283 pts/2    R+     0:00      \_ ps f
 4592 pts/0    Ss     0:00 bash
 4755 pts/0    S+     0:28  \_ code shell.tex
```

También aparece una columna adicional (STAT) que indica el estado del proceso. El primer carácter: R (*running*), T (*stopped*) y S (*sleep*); el segundo: s (*session leader*), l (*multi-hilo*), + (*en primer plano*).

Imagina que un proceso no responde a su interfaz gráfica (suponiendo que la tiene) y el usuario no dispone de otra forma de controlarlo. La operación más habitual es terminarlo, «matarlo» (con el comando `kill` (🎒`procps`)). A pesar de su nombre, la utilidad del comando `kill` es enviar una señal a un proceso. La señal por defecto es SIGTERM, que efectivamente está pensada para terminar el proceso, pero se puede enviar cualquier otra señal que se especifique como argumento. Puedes ver la lista de todas las señales con `kill -l`. El siguiente comando envía la señal SIGKILL (9) al proceso con PID 5200:

```
~$ kill -9 5200
[1]+  Terminado (killed)      gedit
```

Algunas señales pueden ser ignoradas o bien capturadas por el programa para realizar acciones especificadas por el programador[4]. Este es el caso de SIGTERM, que se podría considerar una solicitud amistosa de terminación. Otras (como SIGKILL, que también termina el proceso) no pueden ser ignoradas, para así garantizar que el SO siempre tenga el control.

2.4.1. Trabajos

La shell crea un nuevo proceso hijo[5] para ejecutar cada comando que se le pide. El comportamiento normal de la shell es esperar a que ese proceso termine antes de permitir la introducción de un nuevo comando —fácil de comprobar con el programa `sleep`. Esto se llama ejecución en *primer plano* o *foreground*.

Sin embargo, en la ejecución de un programa puedes pedirle a la shell que no espere a su terminación, permitiendo incluso que el comando quede en ejecución indefinidamente. A eso se le llama ejecución en *segundo plano* o *background*. Para conseguir que la shell ejecute un comando en segundo plano basta con añadir el símbolo *ampersand* (`&`) al final de la línea de comando:

```
~$ gedit &
[1] 19777
~$
```

Como se aprecia en la consola anterior, la shell imprime una línea con dos números y luego queda disponible, a la espera de que se introduzca un nuevo comando. El primer número [entre corchetes] identifica el 'trabajo' en segundo plano (proceso hijo de la shell). El segundo número es el PID

[4] Ejecuta `man signal` para más detalles

[5] La propia shell también se ejecuta en un proceso, que puede haber sido creado a su vez por otra shell.

de dicho proceso, que lo identifica dentro del SO completo. Una shell puede ejecutar múltiples trabajos en segundo plano. La forma más sencilla de ver cuáles son es el comando `jobs`:

```
~$ firefox &
[2] 20847
~$ jobs
[1]-  Running                 gedit &
[2]+  Running                 firefox &
```

Si ejecutas el comando `fg` *(foreground)*, el último comando ejecutado en segundo plano (marcado con el símbolo `+`) pasará a primer plano, bloqueando la shell. Opcionalmente admite como argumento el identificador del trabajo que quieres llevar a primer plano.

```
~$ fg %1
gedit
```

Si la shell se encuentra bloqueada en espera de la finalización de un comando en primer plano, puedes pararlo (no cerrarlo) pulsando `Control-Z` (la shell lo representa con `^Z`). Partiendo de la situación anterior:

```
~$ fg %1
gedit
^Z
[1]+  Stopped                 gedit
~$ jobs
[1]+  Stopped                 gedit
[2]-  Running                 firefox &
~$
```

Un trabajo **parado** puede volver a estado de ejecución en primer plano si ejecutas el comando `fg`, o a segundo plano si ejecutas `bg`.

```
~$ jobs
[1]+  Stopped                 gedit
[2]-  Running                 firefox &
~$ bg
[1]+ gedit &
~$
```

Los indicadores de trabajo también se pueden usar con el comando `kill` en lugar de especificar el PID.

```
~$ kill -SIGKILL %2
[2]+  Killed                  firefox
```

2.4.2. Otros modos de identificar procesos

A veces, buscar en la lista de procesos (`ps`) no es la forma más cómoda de localizar un proceso. Puede ser más sencillo utilizar su nombre (con `pidof` o `pgrep` (🖿 `procps`)):

```
~$ pidof firefox
105894
```

O bien lo puedes identificar por un archivo o dispositivo que el proceso tenga abierto, o con un puerto al que esté vinculado (con `fuser` (🖿 `psmisc`)):

```
~$ fuser 17500/tcp
17500/tcp:           10589
```

También se puede enviar una señal (por defecto con la intención de matarlo) con estos valores (nombre y puerto):

```
~$ pkill firefox
~$ fuser --kill 17500/tcp
17500/tcp:           10589
```

Por supuesto, estos comandos tienen opciones que les permiten buscar procesos por otros criterios (como el propietario o el orden de creación), que además se pueden combinar.

2.5. Entrada, salida y salida de error

En los sistemas POSIX los programas interaccionan con el SO únicamente por medio de archivos—o de abstracciones que ofrecen la misma interfaz. Es decir, la lectura o escritura desde y hacia cualquier disco, pantalla, teclado, tarjeta de sonido o cualquier otro periférico, se realiza en términos de primitivas `read()`/`write()` de forma similar a un archivo.

Todo proceso —programa en ejecución— dispone, desde su inicio, de tres descriptores de archivo abiertos:

- entrada estándar (`stdin`) con el descriptor 0,
- salida estándar (`stdout`) con el descriptor 1, y
- salida de error estándar (`stderr`) con el descriptor 2.

Eso significa que cuando un programa trata de escribir algo (*p. ej.* con la función `puts()` en C o `System.out.println()` en Java) lo envía a su salida estándar. Del mismo modo, cuando el programa intenta leer o genera un mensaje de error, esas operaciones se realizan respectivamente sobre su entrada y salida de error estándar.

Normalmente, la entrada estándar está ligada al teclado, mientras que la salida y salida de error estándar están ligadas a la pantalla (teclado y pantalla se conocen comúnmente como 'consola' o 'terminal'[6]). Esta asociación entre la consola y los descriptores de archivo estándar la determina precisamente la shell y, mediante las órdenes adecuadas, el usuario puede alterar esa asociación para que la entrada y la salida de un programa queden ligadas a otra cosa, como un archivo en disco o incluso otro programa.

El resultado de la ejecución anterior del comando `ls /` 'el listado de archivos del directorio raíz', apareció en pantalla porque su salida estándar corresponde por defecto a la consola.

2.6. Redirección

Es posible alterar la salida estándar de cualquier programa para enviarla, por ejemplo, a un archivo. Simplemente debes escribir el símbolo `>` (mayorque) seguido del nombre del archivo:

```
~$ ls -l / > /tmp/root-files
~$
```

Repito: la 'redirección de salida' consigue que **cualquier** programa pueda enviar su salida en un archivo (o donde la shell determine) sin que el creador de ese programa tenga que haber hecho absolutamente nada para soportarlo.

`/tmp`

El directorio `/tmp` se utiliza por las aplicaciones para archivos *temporales* en los que almacenar logs o datos de sesión. El contenido de este directorio se elimina al apagar el computador.

Dos cosas han cambiado respecto a la ejecución anterior del comando `ls`: la primera es que se ha especificado la opción `-l` que hace que se muestren permisos, propietario y fecha de modificación de cada archivo o directorio. La segunda es que esta vez *nada* ha aparecido en la consola. La lista de los nombres de archivo ha sido almacenada en el archivo `/tmp/root-files`.

Puedes ver el contenido de ese archivo con el programa `cat`:

```
~$ cat /tmp/root-files
total 98
drwxr-xr-x   2 root root  4096 Aug 15 00:34 bin
drwxr-xr-x   4 root root  2048 Aug 17 22:22 boot
drwxr-xr-x  17 root root  3560 Aug 26 10:20 dev
drwxr-xr-x 193 root root 12288 Aug 25 18:29 etc
[...]
```

[6] Aunque 'consola' y 'terminal' no significan exactamente lo mismo.

La redirección simple (>) crea siempre un archivo nuevo. Si existe un archivo con el mismo nombre, su contenido se sobrescribe. Pero existe otro tipo de redirección de salida que añade el contenido *al final* del archivo especificado. Se indica con doble mayor-que (>>):

cat

Lee el contenido de los archivos que se le indiquen como argumentos y lo escribe en su salida estándar. Si no se le dan argumentos, leerá de su entrada estándar.

```
~$ date > /tmp/now
~$ date >> /tmp/now
~$ cat /tmp/now
Mon Jul 15 12:05:47 CEST 2024
Mon Jul 15 12:05:49 CEST 2024
```

Volviendo al archivo `root-files`, sus líneas se pueden filtrar de modo que aparezcan únicamente las que contengan la cadena «Jun» (los archivos modificados en junio). Para eso puedes utilizar el comando `grep` del siguiente modo:

date

Escribe en su salida estándar la fecha y hora actual con la zona horaria configurada.

```
~$ grep Jun /tmp/root-files
drwxr-xr-x   6 root root  4096 Jun  6 17:58 home
lrwxrwxrwx   1 root root    32 Jun 27 13:09 initrd.img
lrwxrwxrwx   1 root root    28 Jun 27 13:09 vmlinuz
```

`grep`, como muchos otros programas, usa su entrada estándar como fuente de datos si no se le dan argumentos. Por tanto, utilizando la 'redirección de entrada' (con el carácter <) se puede lograr lo mismo ejecutando:

grep

Escribe en su salida estándar aquellas líneas que coincidan con un criterio especificado. Lee de los archivos indicados como argumentos (o de su entrada estándar en su defecto).

```
~$ grep Jun < /tmp/root-files
drwxr-xr-x   6 root root  4096 Jun  6 17:58 home
lrwxrwxrwx   1 root root    32 Jun 27 13:09 initrd.img
lrwxrwxrwx   1 root root    28 Jun 27 13:09 vmlinuz
```

La diferencia es que, en este caso, el comando `grep` no tiene constancia de estar leyendo realmente de un archivo en disco. Resulta irrelevante.

Si se desea almacenar el resultado obtenido en otro archivo, basta con utilizar de nuevo la redirección de salida. Es decir, es posible combinar redirecciones de entrada y salida en el mismo comando:

```
~$ grep Jun < /tmp/root-files > /tmp/Jun-files
```

El contenido de ese archivo está ordenado por los nombres de los archivos (la última columna). Así se puede reordenar esa lista en función del tamaño (quinta columna), utilizando el programa `sort`:

```
~$ sort --numeric-sort --key=5 /tmp/Jun-files
lrwxrwxrwx   1 root root    28 Jun 27 13:09 vmlinuz
lrwxrwxrwx   1 root root    32 Jun 27 13:09 initrd.img
drwxr-xr-x   6 root root  4096 Jun  6 17:58 home
```

Para lograr este resultado se han creado dos archivos temporales (`root-files` y `Jun-files`), pero si lo único que te interesa es el resultado final, hay una forma más sencilla y rápida de conseguir lo mismo sin necesidad de crear esos archivos: las tuberías.

`sort`

Ordena las líneas de los archivos que se le indiquen como argumento (o de su entrada estándar) y las escribe en su salida estándar. Se pueden especificar diferentes criterios de ordenación mediante opciones.

La tubería *(pipe)* conecta la salida estándar de un proceso directamente con la entrada de otro, de modo que todo lo que el primer programa escriba podrá ser leído inmediatamente por el segundo. Para indicar a la shell que cree una tubería se utiliza el carácter `|` (AltGr-1 en el teclado español). El comando para obtener directamente los archivos del directorio raíz modificados en junio ordenados por tamaño sería:

```
~$ ls -l / | grep Jun | sort -n -k5
lrwxrwxrwx   1 root root    28 Jun 27 13:09 vmlinuz
lrwxrwxrwx   1 root root    32 Jun 27 13:09 initrd.img
drwxr-xr-x   6 root root  4096 Jun  6 17:58 home
```

Nótese que `grep` y `sort` no tienen nombres de archivo en sus argumentos y que, por tanto, están leyendo líneas desde sus respectivas entradas estándar.

Por supuesto, es posible combinar las tuberías y la redirección en un mismo comando. En este caso, el resultado del comando anterior se almacena en un archivo, del mismo modo que se hacía con los comandos simples:

```
~$ ls -l / | grep Jun | sort -n -k5 > /tmp/root-jun-files-sorted-by-size
```

También se puede hacer redirección de salida con la captura hacia una variable. Se utiliza la sintaxis `$(comando)`. Por ejemplo, si quieres almacenar el número de archivos del directorio raíz en una variable, puedes ejecutar:

```
~$ num_files=$(ls -1 / | wc -l)
~$ echo $num_files
24
```

Otra forma muy potente aunque no tan habitual es la *sustitución de procesos* cuya sintaxis es `>(comando)` o `<(comando)`. Permite utilizar la salida o la entrada de un comando como si fuera un archivo. En realidad es así, aunque ocurre fuera de tu vista. La shell crea un archivo temporal en `/dev/fd/` que contiene la salida del programa. Por ejemplo, podemos comparar la lista de procesos ahora y dentro de 2 segundos con:

`wc`

Cuenta el número de líneas, palabras y caracteres de los archivos indicados como argumentos (o de su entrada estándar) y lo escribe en su salida estándar.

```
~$ diff <(ps aux) <(sleep 2; ps aux)
```

Es similar a la tubería, pero es más versátil porque permite utilizar la salida o la entrada de varios comandos en el mismo comando.

`diff`

Compara el contenido de dos archivos línea a línea y escribe las diferencias en su salida estándar.

La salida de error también se puede redirigir. Por ejemplo, `cat`, al igual que la mayoría de programas, imprime un mensaje por su salida de error si le das un archivo que no existe. Puedes redirigir a un archivo con `2>\file`:

```
~$ cat no-existe
cat: no-existe: No existe el \file o el directorio
~$ cat no-existe 2> /tmp/cat-error
~$ cat /tmp/cat-error
cat: no-existe: No existe el \file o el directorio
```

Y por último, puedes redirigir un descriptor a otro, por ejemplo, la salida de error a la salida estándar. Mira este ejemplo:

```
1  ~$ echo hola > si-existe
2  ~$ cat si-existe no-existe
3  hola
4  cat: no-existe: No existe el \file o el directorio
5  ~$ cat si-existe no-existe > /dev/null
6  cat: no-existe: No existe el \file o el directorio
7  ~$ cat si-existe no-existe 2> /dev/null
8  hola
9  ~$ cat si-existe no-existe > salida 2>&1
10 ~$ cat salida
11 hola
12 cat: no-existe: No existe el \file o el directorio
```

El programa `echo` crea el archivo `si-existe` con el texto «hola». Después le pedimos a `cat` que lo imprima junto a otro archivo que no existe. Por eso aparece el «hola» en la **línea 3** (que envía a `stdout`) y el mensaje de error en la **línea 4** (que envía a `stderr`). Vemos las dos porque por defecto ambas salidas están conectadas a la consola. En la **línea 5** redirigimos `stdout` a `/dev/null` (que la descarta), y por eso solo vemos el mensaje de error. En la **línea 7** es `stderr` la que descartamos, así que vemos solo el «hola». Por

último en la **línea 9** redirigimos `stderr` a `stdout` y redirigimos esta última a un archivo `salida`, y vemos que al imprimirlo contiene las dos líneas. Aunque es contraintuitivo, la redirección `2>&1` debe ir al final del comando porque la shell evalúa las redirecciones de izquierda a derecha, y en ese punto `stdout` ya ha sido redirigida al archivo `salida`.

Es fácil apreciar el gran potencial de este sencillo mecanismo. Cualquier sistema POSIX (en especial GNU) dispone de una gran cantidad de pequeños programas especializados —como los que se han introducido aquí— que pueden combinarse mediante redirección para cubrir una gran variedad de necesidades puntuales de una forma rápida y eficiente[7].

2.7. Usuarios y grupos

Todo lo que ocurre en un sistema GNU/Linux depende de un usuario, que puede ser un humano o un servicio. El usuario se identifica con un número: el UID (User IDentifier). También hay grupos de usuarios, que también se identifican con un número: el GID (Group IDentifier). Cada usuario puede pertenecer a múltiples grupos y, para cada usuario, existe siempre un grupo que suele tener el mismo número.

Para simplificar el manejo de usuarios y grupos, se establece una correspondencia entre identificadores numéricos y nombres, que está almacenada en el archivo `/etc/passwd`. Cada fila de este archivo contiene el nombre del usuario, su UID, su GID, su nombre completo, su directorio personal (su *home*) y la shell que se ejecutará cuando el usuario inicie sesión. Estas son algunas líneas de un archivo `/etc/passwd`:

```
root:x:0:0:root:/root:/bin/bash
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
alice:x:1000:1000:Alice Doe:/home/alice:/bin/bash
```

El usuario `root` es el superusuario del sistema. Tiene acceso a todos los archivos y puede ejecutar cualquier comando. Siempre tiene el UID 0. El usuario `backup` es un usuario del sistema que no tiene acceso a la consola (su shell es `nologin`). El usuario `alice` es el primer usuario «normal», con su propio directorio personal (`/home/alice`) y `bash` como shell.

El archivo `/etc/group` contiene la correspondencia entre los nombres de grupo, su GID y los usuarios que pertenecen a cada uno. Esto es un fragmento del archivo `/etc/group`:

[7] La web `https://www.commandlinefu.com/` es una buena prueba de la gran versatilidad de la redirección y las capacidades de la shell.

```
root:x:0:
backup:x:34:
audio:x:29:alice,pulse
alice:x:1000:
```

Puedes ver el identificador de tu usuario y los grupos a los que pertenece con el programa `id`:

```
~$ id
uid=1000(alice) gid=1000(alice)
↪  grupos=1000(alice),29(audio),44(video),46(plugdev),111(bluetooth)
```

Los identificadores de usuario son números a partir de 1000, mientras que los servicios y grupos del sistema tienen valores menores. En el ejemplo, el grupo *audio* tiene el GID 29, y por cierto, identifica a los usuarios que pueden utilizar los dispositivos de sonido.

2.7.1. Propietarios y permisos

Todos los procesos son propiedad de algún usuario y ese usuario determina qué puede hacer el proceso, y a qué archivos y dispositivos puede acceder. Puedes ver el propietario de un proceso con la opción `u` del programa `ps`. En concreto el siguiente comando muestra todos los procesos del sistema y el propietario en la primera columna. Aquí solo incluimos algunas líneas para ilustrar el resultado. Si lo ejecutas en tu sistema, probablemente verás más de 200.

```
~$ ps aux
USER      PID %CPU %MEM  TTY     STAT START   TIME COMMAND
root        1  0.0  0.0  ?       Ss   16:55   0:01 /sbin/init splash
root        2  0.0  0.0  ?       S    16:55   0:00 [kthreadd]
alice    5021  0.0  0.0  ?       Ss   16:55   0:01 /lib/systemd/systemd --user
alice    8031  1.4  2.6  tty2  SLl+   16:56   3:44 /opt/google/chrome/chrome
alice   18950  0.0  0.4  ?      Ssl   17:31   0:07 /usr/libexec/gnome-terminal-server
alice   33354  0.4  1.3  ?     SLsl   18:33   0:38 /usr/share/code/code .
alice  196069  0.0  0.0  pts/5   Ss   21:05   0:00 bash
alice  202403  0.0  0.0  pts/5   R+   21:11   0:00 ps aux
```

Los archivos (y eso incluye los dispositivos representados como tales) también son propiedad de algún usuario y de un grupo. Los propietarios de archivos se pueden listar con el comando `ls -l`.

```
~$ ls -l /
total 98
lrwxrwxrwx   1 root root      7 sep 27  2023 bin -> usr/bin
drwxr-xr-x   3 root root   4096 may  2 18:43 boot
drwxr-xr-x  19 root root   4100 may  8 21:14 dev
drwxr-xr-x 207 root root  16384 may  8 21:14 etc
drwxr-xr-x   6 root root   4096 dic  1  2023 home
drwx------  22 root root   4096 may  4 15:00 root
-rw-r--r--   1 root root   1028 ene 19  2023 PKGBUILD
```

```
-rwx------   1 root root     27 feb 16  2024 vmlinuz
[...]
```

La tercera y cuarta columnas indican el usuario y el grupo propietario. En el ejemplo todos son del usuario `root` porque obviamente son archivos del sistema. La primera columna tiene información interesante codificada con una serie de letras y guiones:

La primera letra indica el tipo de archivo: `d` para directorios, `l` para enlaces simbólicos y `-` para archivos normales. Después aparecen 9 letras que tienes que interpretar como tres grupos. Expresan los permisos de lectura (`r`), escritura (`w`) y ejecución (`x`) del usuario, el grupo y del resto de usuarios (tres letras para cada uno). Si no tiene el permiso, aparece un guion (`-`).

Por ejemplo, la línea `drwxr-xr-x` de `boot` significa que es un directorio (`d`), que el propietario puede leer, escribir y ejecutar (`rwx`) el archivo y que tanto los miembros del grupo como el resto de usuarios solo pueden leer y ejecutar (`r-x`). Como es un directorio, 'leer' significa que se puede listar su contenido, 'escribir' que se puede crear o borrar archivos dentro de él y 'ejecutar' que puede acceder a su contenido.

Cuando se aplica a un archivo, 'ejecutar' significa que lo puede ejecutar como un programa. Por ejemplo, el archivo `vmlinuz` es un archivo normal (`-`) que solo puede ser leído, escrito (modificado o borrado) y ejecutado con el usuario `root`.

Cuando un usuario ejecuta un programa, el SO crea un proceso que hereda el propietario que lo ha ejecutado y el SO se encarga de que las operaciones que pueda realizar ese proceso respeten los permisos de los archivos a los que trate de acceder.

Por ejemplo, el archivo `/etc/shadow` almacena información sensible relacionada con las contraseñas de cada usuario. Por ese motivo, solo el usuario `root` y el grupo `shadow` tienen permiso de lectura.

```
~$ ls -l /etc/shadow
-rw-r----- 1 root shadow 2134 mar  3 12:24 /etc/shadow
```

Si intentas leerlo con un usuario normal, el SO te lo impedirá:

```
~$ cat /etc/shadow
cat: /etc/shadow: Permiso denegado
```



Aunque seas el administrador de tu propio PC, es una buena práctica de seguridad evitar utilizar el usuario `root` a menos que sea estrictamente necesario. Si ejecutas un programa malicioso como `root` le estarás dando acceso a todo el sistema, con lo que el daño que puede provocar es mucho mayor.

2.8. Paquetes

Muchas distribuciones de GNU/Linux utilizan *paquetes* para distribuir sus programas. Un paquete no es más que una colección de archivos estructurados en directorios para ser colocados en sus rutas adecuadas dentro del sistema de archivos cuando se instale.

En Debian GNU/Linux, Ubuntu u otras distribuciones derivadas, estos paquetes son archivos comprimidos con extensión `.deb`. Si descargas o dispones de uno de estos archivos, puedes ver su contenido con `dpkg`:

```
~$ dpkg -c gedit_48.1-9_amd64.deb
drwxr-xr-x root/root         0 2025-05-03 05:18 ./
drwxr-xr-x root/root         0 2025-05-03 05:18 ./usr/
drwxr-xr-x root/root         0 2025-05-03 05:18 ./usr/bin/
-rwxr-xr-x root/root     14568 2025-05-03 05:18 ./usr/bin/gedit
drwxr-xr-x root/root         0 2025-05-03 05:18 ./usr/lib/
[...]
```

Ahí aparece la ruta donde se va a instalar cada fichero contenido en el paquete. Son rutas relativas a la raíz del sistema de archivos. En este caso, el programa `gedit` se instalaría en `/usr/bin/gedit`.

Por supuesto, también puedes instalar el paquete:

```
~# dpkg -i gedit_48.1-9_amd64.deb
(Leyendo la base de datos ... 803908 ficheros o directorios instalados actualmente.)
Preparando para desempaquetar .../gedit_48.1-9_amd64.deb ...
Desempaquetando gedit (48.1-9) sobre (48.1-9) ...
Configurando gedit (48.1-9) ...
```

Instalar o eliminar paquetes —como otras muchas operaciones— requiere privilegios de superusuario. Sin embargo, por motivos de seguridad, en muchos sistemas modernos no es posible acceder directamente como `root`, el único usuario que cuenta con esos privilegios.

Para resolverlo, los usuarios autorizados tienen permiso para «encarnar» (temporalmente) a `root` por medio del comando `sudo`. En realidad, `sudo` te permite encarnar cualquier usuario para el que se tengas permiso, pero por defecto utiliza `root`. El fichero `/etc/sudoers` define qué permisos puede otorgar `sudo` y en qué condiciones. En adelante, para indicar que un comando

necesita privilegios de `root`, el indicador de sistema *(prompt)* muestra '`#`' en lugar de '`$`'. Cuando tengas que ejecutar uno de estos comandos, recuerda que necesitarás hacerlo con `sudo`.

Sin embargo, una característica muy importante de estos paquetes es que tienen especificadas sus *dependencias*, es decir, otros paquetes que deben estar instalados previamente. Puedes comprobar esas dependencias (y otra mucha información) sobre un paquete con `apt-cache`:

```
~$ apt-cache show gedit
Package: gedit
Version: 48.1-9
Installed-Size: 1319
Maintainer: Debian GNOME Maintainers <pkg-gnome-maintainers@lists.alioth.debian.org>
Architecture: amd64
Replaces: gedit-plugin-text-size (<< 48)
Depends: gedit-common (<< 49~), gedit-common (>= 48~), gsettings-desktop-schemas,
↪  iso-codes, gir1.2-gtk-3.0 (>= 3.22), gir1.2-gtksource-300, gir1.2-tepl-6 (>= 6.12),
↪  libc6 (>= 2.38), libcairo2 (>= 1.2.4), libgdk-pixbuf-2.0-0 (>= 2.22.0),
↪  libgedit-amtk-5-0
[...]
```

Para muchos de estos paquetes se indica la versión necesaria; en este ejemplo, se requiere una versión del paquete `libc6` que sea mayor o igual a la 2.38.

Obviamente, encontrar, descargar e instalar manualmente (con `dpkg`) todos esos paquetes (y sus respectivas dependencias) es una tarea terriblemente pesada. Afortunadamente, existe otro modo de hacerlo.

El programa `apt`[8] puede determinar, automática y recursivamente, las dependencias de un paquete, descargarlos e instalarlos en el orden correcto.

```
~# apt install gedit
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
Se instalarán los siguientes paquetes adicionales:
  gedit-common gir1.2-gtksource-4 libamtk-5-0 libamtk-5-common libtepl-5-0
Paquetes sugeridos:
  gedit-plugins
Se instalarán los siguientes paquetes NUEVOS:
  gedit gedit-common gir1.2-gtksource-4 libamtk-5-0 libamtk-5-common libtepl-5-0
0 actualizados, 6 nuevos se instalarán, 0 para eliminar y 172 no actualizados.
Se necesita descargar 59,0 kB/2.146 kB de archivos.
Se utilizarán 15,5 MB de espacio de disco adicional después de esta operación.
¿Desea continuar? [S/n]
```

[8]También existe `apt-get`, que es una versión de más bajo nivel más adecuada para scripts.

2.8.1. Repositorios de paquetes

La pregunta obvia es: ¿Cómo sabe `apt` de dónde descargar todos esos paquetes? Las distribuciones proporcionan servidores (normalmente web o FTP) donde comparten públicamente los archivos `.deb`. Por ejemplo, para Debian es `https://ftp.debian.org/debian/` y para Ubuntu `https://archive.ubuntu.com/ubuntu/`. De ambos existen cientos de «copias espejo» *(mirrors)*, normalmente una por país, como `https://ftp.es.debian.org/debian/`.

En estos repositorios, los paquetes se organizan en *releases* (o versiones de la distribución)[9]. En el caso de Debian, estas *releases* tienen un número de versión y un nombre asociado. Por ejemplo Debian 13 tiene el nombre *Trixie*.

En el archivo `/etc/apt/sources.list` se especifican los repositorios de paquetes y las *releases* que se quiere usar. Un ejemplo de ese archivo:

```
deb http://deb.debian.org/debian/ trixie main contrib non-free
```

Esta línea en concreto dice que es posible instalar paquetes oficiales *(main)*, contribuciones de terceros *(contrib)* y software con licencias no libres *(non-free)*, desde el repositorio de Debian para la versión *Trixie*.

Este archivo puede contener muchos repositorios, y también se pueden crear otros archivos con extensión `.list` dentro de `/etc/apt/sources.list.d` que tienen el mismo tipo de contenido.

Para saber qué paquetes (y versiones) están disponibles en los repositorios configurados, `apt` debe descargar una especie de índices que se encuentran allí mismo. Para eso ejecuta:

```
~# apt update
```

Es necesario hacer esto regularmente[10] porque el contenido de los repositorios cambia y se añaden nuevos paquetes o versiones.

En Debian hay siempre tres versiones activas:

stable
: Es la última versión publicada y su contenido no debería cambiar. Corresponde con *Trixie* en el momento de escribir este texto.

[9] En `https://www.debian.org/releases/` puedes encontrar las *releases* de Debian

[10] Las distribuciones actuales tienen aplicaciones que lo hacen automáticamente.

testing
: Contiene los paquetes que se están preparando para una futura versión estable y por tanto, cambia continuamente. En este momento se denomina *Forky* y ese será el nombre que tendrá la siguiente versión estable.

unstable
: Que tiene paquetes recién incorporados, experimentales o que tienen algunos problemas importantes para ser incluidos en una futura versión. Esta versión siempre se llama *sid*, que en realidad es el acrónimo de *Still In Development* (aún en desarrollo).

Por cuestiones de seguridad es importante que se puedan actualizar paquetes en los que se han descubierto vulnerabilidades o problemas graves, incluso aunque correspondan a una versión estable. Por eso, es conveniente tener los siguientes repositorios en el archivo `/etc/apt/sources.list`:

```
deb http://deb.debian.org/debian/ trixie-updates main
deb http://security.debian.org/debian-security trixie/updates main
```

Todo esto significa que los repositorios configurados determinan qué paquetes (y qué versiones) se pueden instalar. Si quieres tener un sistema completamente actualizado, es decir, con las últimas versiones de todos los paquetes disponibles en esos repositorios, debes ejecutar:

```
~# apt upgrade
```

Si incorporas repositorios de versiones nuevas y quieres actualizar tu sistema con ellos, tendrás que ejecutar:

```
~# apt dist-upgrade
```

Esto hará que la versión de tu sistema operativo corresponda con la versión del repositorio más actual. Eso lo puedes ver con:

```
~$ lsb_release -a
No LSB modules are available.
Distributor ID:^^IDebian
Description:^^IDebian GNU/Linux 13 (trixie)
Release:^^I13
Codename:^^Itrixie
```

También puedes saber qué versiones de un determinado paquete están disponibles en los repositorios configurados y en cuál de ellos están:

```
~$ apt-cache policy gedit
gedit:
  Instalados: 44.2-1
  Candidato:  44.2-1
```

```
  Tabla de versión:
     48.1-4 500
        500 http://deb.debian.org/debian sid/main amd64 Packages
     48.1-3 650
        650 http://deb.debian.org/debian testing/main amd64 Packages
 *** 44.2-1 700
        700 http://deb.debian.org/debian stable/main amd64 Packages
        100 /var/lib/dpkg/status
```

Esto quiere decir que no podrás instalar versiones de los paquetes que no estén en los repositorios configurados. La solución puede ser añadir un repositorio más reciente que sí lo contenga, pero teniendo cuidado porque las nuevas versiones de sus dependencias pueden afectar a otros paquetes.

2.8.2. Cómo encontrar paquetes

Es habitual necesitar un programa o una librería, pero no saber en qué paquete se encuentra. Estas son algunas formas de localizar paquetes, pensadas sobre todo para instalaciones de escritorio.

`command-not-found`

Este paquete modifica el comportamiento de la shell, de modo que al ejecutar un programa que no está instalado, en lugar de mostrar un simple mensaje «comando no encontrado», indica en qué paquete se encuentra ese programa que tratas de ejecutar. Por ejemplo:

```
~$ filezilla
No se ha encontrado la orden «filezilla», pero se puede instalar con:
sudo apt install filezilla
```

`apt-file`

El programa `apt-file` busca en los índices de paquetes de los repositorios de tu sistema. Puedes indicar una ruta o parte de ella y muestra todos los paquetes que coincidan. En este caso sirva para cualquier fichero, no únicamente ejecutables. Es necesario que mantengas ese índice actualizado con `apt-file update`.

```
~$ apt-file find bin/filezilla
filezilla: /usr/bin/filezilla
```

`dpkg -S`

Si ya tienes instalado el programa, pero necesitas saber a qué paquete corresponde puedes ejecutar lo siguiente:

```
~$ which gedit
/usr/bin/gedit
~$ dpkg -S /usr/bin/gedit
gedit: /usr/bin/gedit
```

2.9. Servicios

Los servicios del sistema[11] son procesos en segundo plano bajo el control del SO. Arrancan automáticamente al iniciar el sistema y se encargan de tareas específicas: firewall, sonido, bluetooth, etc.; servidores: web, ssh, etc. u otras tareas de gestión: registro de eventos del sistema, montaje de dispositivos de almacenamiento, etc.

Estos servicios son gestionados por un *gestor de servicios* que se encarga de arrancarlos, pararlos o reiniciarlos en función de su configuración o del estado del sistema. El sistema de gestión de servicios más frecuente en GNU/Linux en la actualidad es `systemd`. Aunque ofrece muchas funciones, estos son los comandos más básicos. Obviamente solo el administrador puede ejecutarlos.

`systemctl status <servicio>` (**`systemd`**)
: muestra el estado del servicio.

`systemctl stop/start/restart <servicio>`
: arranca el servicio.

`systemctl enable/disable <servicio>`
: habilita o deshabilita el servicio para que arranque automáticamente o no al iniciar el sistema.

`systemctl list-units`
: muestra el estado de todos los servicios disponibles.

Por ejemplo, puedes ver el estado del servicio `ssh` (**`openssh-server`**) (que permite conexiones remotas al sistema) con:

```
~# systemctl status ssh
● ssh.service - OpenBSD Secure Shell server
     Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: enabled)
     Active: active (running) since Fri 2025-08-01 15:21:30 CEST; 1 day 7h ago
 Invocation: a081694ecbb7403190db862ff993b604
       Docs: man:sshd(8)
             man:sshd_config(5)
    Process: 965 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
   Main PID: 1045 (sshd)
      Tasks: 1 (limit: 19048)
     Memory: 2.4M (peak: 3M)
```

[11] A menudo llamados *daemons*, un término acuñado por ingenieros del MIT en los primeros años de UNIX.

```
         CPU: 25ms
      CGroup: /system.slice/ssh.service
              └─1045 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

ago 01 15:21:29 maine systemd[1]: Starting ssh.service - OpenBSD Secure Shell server...
ago 01 15:21:30 maine sshd[1045]: Server listening on 0.0.0.0 port 22.
ago 01 15:21:30 maine sshd[1045]: Server listening on :: port 22.
ago 01 15:21:30 maine systemd[1]: Started ssh.service - OpenBSD Secure Shell server.
```

El servicio está habilitado *(enabled)* y activo desde hace 1 día y 7 horas.

En distribuciones GNU/Linux más antiguas se utiliza el sistema `SysVinit` que permitía hacer una gestión similar con el programa `service`. Aunque `SysVinit` está en desuso, `systemd` ofrece comandos compatibles en las distribuciones modernas:

`service <servicio> status`
: muestra el estado del servicio.

`service <servicio> stop/start/restart`
: para, arranca o reinicia el servicio.

`service ---status-all`
: muestra el estado de todos los servicios instalados.

2.10. Parámetros del núcleo

Existen muchos parámetros de configuración del núcleo del SO que se pueden consultar y modificar en tiempo de ejecución. Esa modificación se aplica inmediatamente sin necesidad de reiniciar el sistema.

Estos parámetros o variables del sistema están disponibles directamente para el usuario en el directorio `/proc` aunque lo que contiene no son archivos al uso. Se trata de un sistema de archivos virtual (llamado *procfs*) que expone una gran cantidad de información del núcleo y de los procesos en ejecución.

En el directorio `/proc/sys` en particular, es donde puedes encontrar los parámetros de configuración, organizados en subdirectorios en función de su categoría. Por ejemplo, el directorio `/proc/sys/fs` contiene los parámetros del sistema de archivos y `/proc/sys/net` contiene configuración de la red.

Para ver el valor de uno de estos parámetros, puedes usar simplemente el comando `cat`. Por ejemplo, el parámetro `/proc/sys/fs/file-max` indica el número máximo de archivos que pueden estar abiertos al mismo tiempo.

```
~$ cat /proc/sys/fs/file-max
9223372036854775807
```



Como todos los parámetros están en `/proc/sys`, cuando se haga referencia a alguno de ellos a lo largo del libro, se indicará su ruta relativa, es decir, para referirnos a `/proc/sys/fs/file-max` hablaremos del 'parámetro del núcleo `fs/file-max`'.

Para modificar el valor de un parámetro se requieren permisos de superusuario, y consiste en escribir el valor deseado en el archivo virtual, algo que puedes hacer con `echo`. Por ejemplo, el parámetro `vm/laptop\_mode` indica si el sistema está en «modo portátil». En este modo, el SO trata de ahorrar energía y optimizar el uso del disco duro. Para activarlo, basta con escribir un valor distinto de 0 en el archivo:

```
~# echo 1 > /proc/sys/vm/laptop_mode
~# cat /proc/sys/vm/laptop_mode
1
```

Existe un programa específico para manipular estos parámetros llamado `sysctl` (📦`procps`). La siguiente consola muestra su uso para leer y modificar el parámetro `vm/laptop\_mode` de forma equivalente a lo visto previamente:

```
~# sysctl vm.laptop_mode
vm.laptop_mode = 0
~# sysctl -w vm.laptop_mode=1
vm.laptop_mode = 1
```

Sin embargo, estos cambios no son permanentes. Si se reinicia el sistema, el valor del parámetro (y por tanto el comportamiento correspondiente) volverá al valor por defecto. Para que estos cambios sean permanentes, deben añadirse en el archivo `/etc/sysctl.conf`. Para el parámetro anterior, la línea que se debe añadir es:

```
vm.laptop_mode = 1
```

Y ¿qué más?

La shell representa como ningún otro programa la esencia de la *filosofía Unix*, que se puede resumir en una cita atribuida a Doug McIlroy, uno de los ingenieros de Bell Labs que diseñó el sistema UNIX:

Escribe programas que…
…hagan solo una cosa y la hagan bien.
…que trabajen juntos.
…que manejen flujos de texto, porque es una interfaz universal.

Aparte de los programas incluidos en este capítulo, existen muchos otros diseñados con estos mismos principios que pueden ser combinados con redirección para realizar tareas complejas. El anexo A incluye una lista de comandos habituales, aunque a lo largo del libro también se introducirán algunos más.

Capítulo 3

Python

Al terminar este capítulo, entenderás:

- Cuáles son los tipos de datos básicos de Python.
- Cómo se definen funciones, clases y módulos.
- Cuáles son las estructuras de control.
- Cómo funciona la comprobación opcional de tipos.

Python es un lenguaje interpretado, interactivo y orientado a objetos. Si ya conoces algún lenguaje como Java, la mayoría de lo que encontrarás en Python te sonará familiar. Tiene variables, clases, objetos, funciones y todo lo que se espera que tenga un lenguaje de programación «convencional». Cualquier programador puede empezar a trabajar con Python con poco esfuerzo. Según muchos expertos, Python es uno de los lenguajes más fáciles de aprender y que permiten al novato ser productivo en menos tiempo, y todo eso también explica que sea uno de los lenguajes más populares en la actualidad.

Python es perfecto como lenguaje «pegamento» y para prototipado. Dispone de librerías para desarrollar aplicaciones multihilo, distribuidas, bases de datos, con interfaz gráfica, juegos, gráficos 3D, cálculo científico, análisis de datos, inteligencia artificial y prácticamente todo lo que se te ocurra.

Este pequeño tutorial asume que dispones de Python versión 3.11 o posterior y tienes nociones básicas de programación con algún lenguaje imperativo u orientado a objetos.

3.1. ¡A programar!

Nada como programar para aprender un lenguaje, y en eso Python tiene una ventaja: el intérprete de Python se puede utilizar como una shell (y por eso se dice que es «interactivo»). Es algo tremendamente práctico para probar cosas, pero también para dar tus primeros pasos. Simplemente abre

una consola y ejecuta `python3`, o `python` dependiendo de la configuración de tu SO:

```
$ python3
Python 3.11.9 (main, Apr 10 2024, 13:16:36) [GCC 13.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

3.2. Variables y tipos

Las variables no se declaran explícitamente, se pueden usar desde el momento en que se inicializan y normalmente no hay que preocuparse por liberar la memoria que ocupan porque viene con un «recolector de basura».

```
$ python3
>>> a = "hola"
```

En el modo interactivo se puede ver el valor de cualquier objeto escribiendo simplemente su nombre:

```
>>> a
'hola'
```

Python es un lenguaje de tipado fuerte. Eso significa que no se puede operar alegremente con variables de tipos diferentes.

```
>>> a + 1
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: Can't convert 'int' object to str implicitly
```

Esa operación ha provocado que se dispare la excepción `TypeError` que avisa que no hay una forma implícita de convertir el `1` (un entero) para poder utilizar el operador `+` con `a` (una cadena). Las excepciones son el mecanismo más común de notificación de errores.

Sin embargo, puedes cambiar el tipo de la variable `a` sobre la marcha (esto se llama «tipado dinámico»).

```
>>> a = 3
```

En realidad no has cambiado el tipo. Has creado una nueva variable que reemplaza a la anterior.

3.3. Tipos de datos

3.3.1. Valor nulo

Una variable puede existir sin tener un valor, para ello se asigna el valor especial `None`:

```
>>> a = None
>>> a
>>> print(a)
None
```

Fíjate que al escribir simplemente `a` no aparece ningún resultado. Para verlo es necesario usar `print()` explícitamente.

3.3.2. Booleanos

Nada fuera de lo común:

```
>>> a = True
>>> a
True
>>> not a
False
>>> a and False
False
>>> 3 > 1
True
>>> b = 3 > 1
>>> b
True
```

3.3.3. Numéricos

Python dispone de tipos enteros de cualquier tamaño que soportan todas las operaciones aritméticas habituales, incluidas las de bits. También hay reales y ambos tipos pueden operar entre sí sin ningún problema. Incluso tiene soporte para números complejos de forma nativa.

```
>>> a = 2
>>> b = 3.0
>>> a + b
5.0
>>> b - 4j
(3-4j)
```

En algunos lenguajes existe el tipo carácter (`char`) que puede manejarse como dato numérico y permite operaciones aritméticas. En Python, sin embargo, se utilizan cadenas de caracteres de tamaño 1 y no permiten operaciones aritméticas.

3.3.4. Secuencias

La secuencia más simple y habitual es la cadena de caracteres (tipo `str`). Las cadenas admiten operaciones como la suma (concatenación) y la multiplicación:

```
>>> cad = 'hola '
>>> cad + 'mundo'
'hola mundo'
>>> cad * 3
'hola hola hola '
```

El tipo **`bytes`** permite manejar secuencias de bytes. Se utilizan habitualmente para serialización de datos. Por ejemplo, puedes codificar una cadena de caracteres a UTF8 como secuencia de bytes y viceversa:

```
>>> word = 'ñandú'
>>> coded = word.encode()
>>> coded
b'\xc3\xb1and\xc3\xba'
>>> coded.decode()
'ñandú'
```

Cuando se utiliza codificación UTF8 algunos caracteres ocupan más de un byte. Este es el caso de las letras 'ñ' y 'ú' del ejemplo. Veremos esta cuestión con más detalle en el capítulo 18.

Las **tuplas** son una agrupación de valores similar al concepto matemático homónimo. Se pueden empaquetar y desempaquetar varias variables, y pueden ser de tipos diferentes.

```
>>> x = cad, 'f', 3.0
>>> x
('hola ', 'f', 3.0)
>>> v1, v2, v3 = x
>>> v2
'f'
```

La tupla, al igual que la cadena, es inmutable, es decir, una vez construida no se puede modificar.

```
>>> cad = 'hola'
>>> cad[0] = 'm'
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'str' object does not support item assignment
```

Si la quieres cambiar, tienes que crear una nueva instancia a partir de la anterior:

```
>>> cad.upper()
'HOLA'
>>> cad
'hola'
```

Las **listas** son similares a los vectores o arrays de otros lenguajes, con la diferencia de que en Python se pueden mezclar tipos. El tipo `list` sí es mutable.

```
>>> x = [1, 'f', 3.0]
>>> x[0]
1
>>> x[0] = None
>>> x
[None, 'f', 3.0]
```

Las listas también se pueden «sumar» y «multiplicar»:

```
>>> x = [1, 'f', 3.0]
>>> x + ['adios']
[1, 'f', 3.0, 'adios']
>>> x * 2
[1, 'f', 3.0, 1, 'f', 3.0]
```

Cualquier tipo de secuencia (listas, tuplas o cadenas) se puede indexar del modo habitual, pero también desde el final con números negativos o mediante «rodajas» *(slicing)*:

```
>>> cad = 'holamundo'
>>> cad[-1]
'o'
>>> cad[1:6]
'olamu'
>>> cad[:3]
'hol'
>>> cad[3:]
'amundo'
>>> cad[-6:-2]
'amun'
```

Los **diccionarios** son tablas asociativas *(hash tables)*. Tanto las claves como los valores pueden ser de cualquier tipo[1], incluso de tipos diferentes en el mismo diccionario:

```
>>> notas = {'antonio':6, 'maria':9.3, 'juan': 'No presentado'}
>>> notas['antonio']
6
```

Python dispone de otros tipos de datos nativos como conjuntos (`set`), pilas, colas, listas multidimensionales, etc.

[1] En realidad tiene que ser un tipo *hashable*, es decir, que se pueda asimilar a un valor único

3.4. Módulos

Los módulos son análogos a las *librerías* o *paquetes* de otros lenguajes. Para usarlos se utiliza la sentencia `import`:

```
>>> import math
>>> math.cos(math.pi)
-1.0
```

3.5. Estructuras de control

Están disponibles las más comunes: `for`, `while`, `if`, `else`, `break`, `continue`, `return`, etc. Todas funcionan de la forma esperable excepto `for`, que no es un `while` disfrazado. Este `for` itera sobre una secuencia de modo que la variable de control toma el valor de cada uno de sus elementos en cada iteración; en realidad es más parecido al `for_each` de otros lenguajes:

```
>>> metales = ['oro', 'plata', 'bronce']
>>> for m in metales:
...     print(m)
...
oro
plata
bronce
```

3.6. Indentación estricta

En programación se aconseja «tabular» (indentar) el código para alinearlo con los bloques y así ayudar a su lectura. En Python este estilo es obligatorio porque el esquema de indentación es lo que define los bloques. Es una importante peculiaridad de este lenguaje y obliga a utilizar un editor de código que controle correctamente esta cuestión. Se aconseja tabulación 'blanda' de 4 espacios. Este fragmento ilustra el uso de la indentación para delimitar bloques:

```
total = 0
passed = 0
grades = {'antonio':6, 'maria':9}
for grade in grades.values():
    total += grade
    if grade >= 5:
        passed += 1

print('Average:', total / len(grades))
```

El cuerpo del bloque `for` queda definido por el código que está indentado un nivel debajo de él. Lo mismo ocurre con el `if`. La sentencia que define el bloque siempre lleva dos puntos al final. No se necesitan llaves ni ninguna otra cosa para delimitar los bloques.

3.7. Funciones

Las funciones utilizan una sintaxis muy sencilla e intuitiva. Nada mejor para explicar su sintaxis que un ejemplo:

```
def factorial(n):
    if n == 0:
       return 1

    return n * factorial(n-1)

print(factorial(10))
```

3.8. Python *is different*

Aunque Python se puede utilizar como un lenguaje convencional, siempre hay una manera más «pythónica» de hacer las cosas. Por ejemplo, el soporte de programación funcional que incluye Python permite hacer la función factorial de un modo menos convencional:

```
from functools import reduce
from operator import mul

factorial = lambda x: reduce(mul, range(1, x+1), 1)
print(factorial(10))
```

Si no conoces nada de programación funcional, sirva este ejemplo como mínima introducción. La palabra reservada `lambda` crea una función anónima, que por ejemplo se podría pasar directamente como argumento a otra función. En este caso se almacena en la variable `factorial`, de modo que la podremos usar del mismo modo que la función tradicional del ejemplo anterior. Esa función utiliza la `reduce()`, que es otra herramienta básica de programación funcional. La función `reduce()` aplica una función a los elementos de una secuencia. En este ejemplo, aplica el operador de multiplicación (`mul`) a los enteros del rango `1-x` usando un valor inicial de 1, es decir, multiplica $1 \cdot 1 \cdot 2 \cdot 3 \cdots x$.

3.9. Hacer un 'ejecutable'

Lo habitual en programación es escribir el programa en un archivo de texto y luego compilarlo. Pero como Python es un lenguaje interpretado, no es

necesario ningún tipo de procesamiento, al menos no explícito. Simplemente escribe el código en un archivo con extensión `.py`.

Para que la shell sepa qué programa debe ejecutar para interpretar el código de este archivo, es necesario añadir un comentario especial en la primera línea. Esta línea se conoce como «shebang» y tiene el siguiente aspecto:

```
#!/usr/bin/python3
```

Además debes darle permisos de ejecución al archivo:

```
$ chmod +x programa.py
```

Con eso ya puedes ejecutar el programa:

```
$ ./programa.py
```

Al encontrar el shebang, la shell ejecutará el programa como si hubieras escrito `/usr/bin/python3 programa.py`.

En el caso de Python, es más adecuado usar el programa `env` para localizar el intérprete adecuado de Python. El shebang queda así:

```
#!/usr/bin/env python3
```

3.10. Orientado a objetos

Casi todo en Python está orientado a objetos, incluyendo las variables de tipos básicos. ¡Incluso los literales!

```
>>> cad = 'holamundo'
>>> cad.upper()
'HOLAMUNDO'
>>> 'ADIOS'.lower()
'adios'
```

Escribir una clase, al menos algo básico, es muy sencillo:

```
class Book:
    def __init__(self, title, author):
        self.title = title
        self.author = author
        self.bookmarks = []

    def add_bookmark(self, page, label):
        self.bookmarks.append((page, label))

    def last_bookmark(self):
        return self.bookmarks[-1]

book = Book("1984", "George Orwell")
```

```
book.add_bookmark(101, "start of chapter 3")
print(book.last_bookmark())
```

Lo más importante es que el primer argumento de los métodos es siempre `self`, que representa la instancia de la clase sobre la que se ejecuta (`Book` en el caso del ejemplo). El método `__init__()` es el constructor[2] y se ejecuta al instanciar la clase: `book = Book("1984", "George Orwell")`.

3.11. *Type checking*

Que Python sea un lenguaje de tipado dinámico puede dar lugar a problemas cuando se utiliza un módulo o un API poco o mal documentado. Al no haber comprobación de tipos, el programador podría pasar variables de tipos incompatibles pero que solo se detectan en tiempo de ejecución. Por esa razón, desde la versión 3.5 ofrece la opción de hacer «anotaciones de tipo» *(type hints)* sobre variables y argumentos, y además existe un módulo donde se define la semántica y convenciones para especificar de forma opcional esas anotaciones. Un ejemplo:

```
>>> pi: float = 3.14
```

El comportamiento del lenguaje es el mismo, y las variables pueden «cambiar de tipo» independientemente de su definición sin producir errores:

```
>>> pi: float = 3.14
>>> pi = "Tres coma catorce"
```

¿Y para qué sirve entonces poner el tipo? Principalmente para facilitar la comprensión del código, y sí, también para poder llevar a cabo comprobaciones de tipo *(type checking)*, pero para eso se requieren herramientas adicionales como `MyPy`[3].

El archivo `factorial.py` contiene una versión de la función `factorial()`, pero que incluye anotaciones en sus argumentos y valor de retorno. También hemos añadido una llamada a la función que toma un argumento de la línea de comandos:

```
import sys

def factorial(n: int) -> int:
  if n == 0:
    return 1

  return n * factorial(n-1)
```

[2] En realidad es el *inicializador*, pero para un nivel básico nos sirve como idea.
[3] `http://mypy-lang.org/`

```
factorial(sys.argv[1])
```

Si ejecutas `MyPy` para que busque errores en esta implementación, verás lo siguiente:

```
$ mypy factorial.py
factorial.py:9: error: Argument 1 to "factorial" has incompatible type "str"; expected
↪  "int"
Found 1 error in 1 file (checked 1 source file)
```

El programa te avisa que el argumento que estás pasando a la función es una cadena, un tipo de dato que no se corresponde con el esperado (un entero). El error desaparece si transformas el parámetro de entrada del programa en un entero con `int(sys.argv[1])`.

Y ¿qué más?

Esto no ha sido más que una muy breve introducción. Como has podido comprobar, Python es un lenguaje muy accesible, con una sintaxis sencilla. Pero eso no significa que sea un lenguaje simple. Aparte de la extensa librería estándar, hay disponible una ingente cantidad de módulos de terceros para casi cualquier necesidad imaginable. Indudablemente es un lenguaje que vale la pena aprender a dominar, con una gran comunidad de usuarios y excelente documentación.

Capítulo 4

Internet

Al terminar este capítulo, entenderás:

- Qué es una red de computadores.
- Qué es una interred.
- Para qué sirve un protocolo y qué es una pila de protocolos.
- Qué son y para qué sirven los modelos de referencia OSI, TCP/IP e híbrido.

Como 'Internet' es una palabra tan habitual, es normal pasar por alto su significado incluso entre personas con formación técnica, a pesar de ser bastante evidente. La palabra 'internet' (en minúscula) surgió como una abreviatura de **inter-network** (interred), que es un concepto amplio para designar cualquier sistema formado por la interconexión de múltiples redes de computadores. 'Internet' (empezando por mayúscula y sin artículo) es un nombre propio que designa específicamente a la más grande y compleja de las interredes, la que todos conocemos y usamos a diario[1].

Esto implica que cualquier otra colección de redes interconectadas, incluso sin usar tecnología TCP/IP, también es un 'interred'. Como queremos que el lector tenga presente esta importante distinción, en el libro utilizaremos la palabra 'interred' siempre que la explicación se pueda aplicar a cualquier interred, y reservaremos la 'Internet' para la interred global.

También es interesante que sus creadores eligieran esa palabra. Significa que esa idea de interconectar redes es lo que les pareció su característica más destacable, al menos en ese momento.

[1] Ten en cuenta que por su obvia similitud semántica existe abundante bibliografía, sobre todo en inglés, en la que «Internet» se utiliza como sinónimo de «interred», por ejemplo la RFC 1819 [7]. Estos documentos asumen que el lector interpreta el significado en función del contexto.

Una **red de computadores** es un conjunto de nodos autónomos[2] conectados entre sí de modo que puedan compartir recursos (datos o dispositivos). Todos los computadores de una red utilizan una misma tecnología de comunicaciones (*p. ej.* WiFi). Como el término «red» es muy común y se utiliza para cosas diferentes, en muchas ocasiones decimos que esto es «un enlace», aunque técnicamente sería más correcto decir «una red». E incluso aislada del resto del mundo, puede tener utilidad, por ejemplo, en el control de una planta de fabricación industrial.

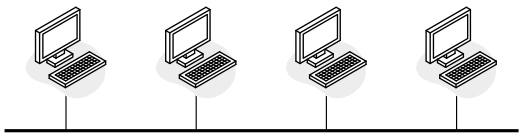


FIGURA 4.1: Representación típica de una red de computadores

Existen dos tipos de enlace: compartidos y punto a punto. En un enlace compartido (como en una LAN de la Figura 4.1) todos los computadores comparten un único medio físico (como el aire en el caso de WiFi), mientras que en un enlace punto a punto, el medio es utilizado únicamente por dos dispositivos (como un cable de cobre que los interconecta). Cada tecnología implica un tipo de enlace y responde a unas necesidades específicas. Veremos algunas de esas tecnologías más adelante.

La forma en la que están conectados físicamente los dispositivos es lo que llamamos **topología física**, pero es habitual que nos interese más la forma en la que se comunican que a qué está conectado cada cable; eso es la **topología lógica**. Por ejemplo, la topología lógica de una LAN Ethernet es de tipo bus (como la Figura 4.1), pero físicamente es un árbol multinivel gracias a los conmutadores (Figura 4.2).

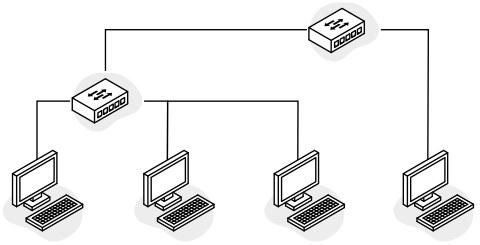


FIGURA 4.2: Topología física de una LAN Ethernet

[2]«autónomos» en el sentido de que pueden realizar tareas por sí mismos

En muchas ocasiones, la estructura de una red o interred es desconocida o irrelevante para el contexto específico que se está considerando. En estos casos se utiliza una nube como metáfora visual y lo único relevante es que hay ciertos dispositivos conectados a ella y gracias a eso pueden comunicarse.

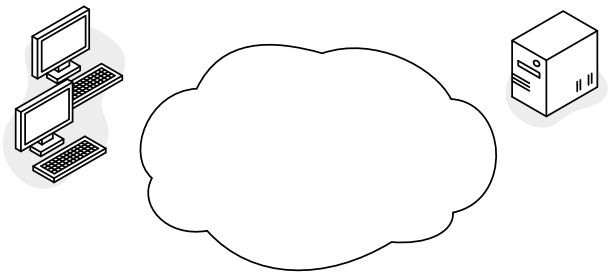


Figura 4.3: La nube como metáfora de una red o interred

4.1. Tecnología Internet

Como en tantos otros campos, no existe una única tecnología maravillosa que resuelve todos los problemas. Por ejemplo, no hay una tecnología universal para hacer redes locales o enlace punto a punto, hay decenas, y ninguna es perfecta (vaya, ¡menuda sorpresa!). La solución que propone la tecnología tras Internet es IP[3] y no consiste en sustituir todas esas tecnologías sino integrarlas, interconectando redes heterogéneas —y probablemente incompatibles entre sí.

La tecnología TCP/IP logra esa interconexión gracias a tres elementos:

- El protocolo IP, que significa literalmente *inter-net protocol*, es decir, **protocolo de interred** implica un formato de mensaje universal (el **paquete IP**), que utilizan tanto los dispositivos terminales como los de interconexión *(routers)*. IP no sustituye a los protocolo de enlace de cada red, sino que «se encapsula» en ellos[4].
- El **direccionamiento** IP. Es una forma de asignar direcciones (números únicos) a cada dispositivo. Cualquier computador con capacidad de enviar y recibir paquetes IP —en cualquier parte— debe disponer de una dirección IP adecuada[5].

[3] Por supuesto no es el único modo, pero es el que abordamos en este texto.
[4] Veremos más adelante qué es la encapsulación.
[5] Sí, todo se llama IP: protocolo IP, paquete IP y direcciones IP.

- Dispositivos capaces de reenviar paquetes entre redes y hacerlos llegar a sus correspondientes destinos. Estamos hablando los routers o pasarelas IP (*gateways*). Y por supuesto se necesitan los mecanismos y protocolos que coordinan esos *routers*, que recibe el nombre de **encaminamiento**, encaminamiento o *routing* en inglés.

La tecnología TCP/IP se basa en trocear la información en paquetes y en enviarlos hacia su destino como unidades independientes. Eso se conoce como **conmutación de paquetes** *(packet switching)*.

Es el modelo opuesto a la **conmutación de circuitos** *(circuit switching)*, propio de redes como la telefonía tradicional, donde antes de poder comenzar la comunicación es necesario establecer un camino (circuito) dedicado de extremo a extremo, ocupando ciertos recurso durante toda la duración de la conversación incluso aunque no haya nada que transmitir en un momento dado.

La conmutación de paquetes, por contra, permite que muchos flujos compartan los mismos enlaces sin que ninguno acapare recursos para sí. Esto es mucho más eficiente, pero a cambio no puede garantizar gran cosa: ni el orden de llegada, ni una tasa de transferencia, ni siquiera que los mensajes vayan a llegar.

¿Por qué se dividen los datos en bloques pequeños? ¿Por que no enviarlos todos de una vez? Hay varios motivos que se repiten en las distintas capas: un mensaje muy grande tardaría mucho en transmitirse, es frágil (un solo error obligaría a retransmitirlo completo), monopoliza el canal, y no cabe en las colas de los routers y menos aún en la memoria de las NIC. Además, hay aplicaciones que generan sus datos en vivo[6], a una tasa baja o en ráfagas espaciadas en el tiempo. Todo el funcionamiento y las ventajas de la conmutación de paquetes dependen de que la información se transmita en trozos manejables.

Internet es posible porque todos los dispositivos conectados se intercambian mensajes del protocolo IP, y eso junto con las direcciones IP y los routers, resuelve un problema clave: permite llevar paquetes con datos (sean lo que sean esos datos) desde un computador a cualquier otro, solo conociendo la dirección IP del destino, sin importar dónde esté o la tecnología que utilice para conectarse a **su** red.

Estos 3 aspectos son probablemente los más importantes en el funcionamiento de Internet y los estudiaremos en detalle en el resto del libro.

[6]Es muy común utilizar la expresión «en tiempo real» para este perfil de tráfico, pero no es correcto. Tiempo real tiene que ver con cumplimiento de plazos, no con inmediatez.

4.2. Protocolos

Pero ¿qué es un **protocolo**? Sin meternos ahora en detalles, podemos decir que un protocolo es una serie de normas y reglas que deben cumplir los participantes para comunicarse de forma eficaz y productiva. Por ejemplo, para hablar con un policía hay una serie de expresiones y palabras que nunca se te ocurriría usar a menos que quieras meterte en problemas, e igualmente, el tipo de respuestas que cabría esperar también cae dentro de unas pautas previsibles. Y el modo en que hablas con un policía es diferente al que utilizas con un médico, un niño, un jefe, un compañero, etc. De hecho, la palabra «protocolo» tiene acepciones que se aplican precisamente a las interacciones con personas en situaciones muy formales o solemnes, como las relaciones diplomáticas, el ámbito institucional o ciertas ceremonias oficiales. En esencia, la idea subyacente es la misma: establecer y cumplir pautas comunes para que la comunicación funcione.

Cuando dos dispositivos o procesos hablan entre sí a través de una red también deben cumplir una serie de normas y formatos; pero con mucha menos ambigüedad que entre personas. Los protocolos de red establecen una serie muy concreta de mensajes, con estructuras perfectamente especificadas que indican qué se puede enviar y responder, y cuándo está permitido enviar cada cosa. Por ejemplo, la comunicación entre un cliente y un servidor web utiliza exclusivamente el protocolo HTTP. Este protocolo solo admite una serie limitada de tipos de mensaje: GET, POST, HEAD, DELETE, etc. y cada uno de ellos tiene su estructura y formato. Si por cualquier motivo un navegador web envía un mensaje de tipo desconocido o que no respeta la estructura esperada, la comunicación acabará mal.

Los protocolos de bajo nivel (los de las capas inferiores) abordan problemas básicos generales como el transporte de mensajes, su división y reensamblado, la detección o corrección de errores, etc., cuestiones que son comunes a cualquier tipo de comunicación, sin importar el contenido de los mensajes. Pero hay otros problemas que resolver.

El sistema operativo debe ofrecer una forma que permita que dos procesos ejecutándose en nodos cualesquiera puedan «direccionarse» el uno al otro, es decir, que puedan designar unívocamente a otro proceso que se ejecuta en un nodo remoto accesible a través de la red. El direccionamiento de procesos dentro de un nodo se denomina **multiplexación**. Esto se logra mediante la combinación de dos datos: una dirección IP, que identifica al nodo, y un puerto, que identifica un proceso dentro de ese nodo.

Además, muchas aplicaciones necesitan garantías de que estos mensajes llegan correctamente a su destino, sin cambios (respetando su integridad), sin

partes ausentes (omisiones), recibidos en el mismo orden en el que se enviaron, asegurando la privacidad, sin saturar al receptor, evitando congestionar la red, etc.

La mayoría de estos problemas los resuelven los protocolos **de transporte**. En el caso de Internet, estos protocolos son:

TCP
: Proporciona un flujo *(stream)* ordenado y fiable de datos entre dos procesos.

UDP
: Ofrece un servicio de entrega de datagramas (mensajes independientes) entre dos procesos, pero sin ninguna garantía.

SSL/TLS
: Ofrece mecanismos para cifrado de mensajes y permite asegurar la legitimidad del proceso remoto.

Estos tres protocolos son genéricos porque se pueden aplicar sobre cualquier carga útil *(payload)*, es decir, son independientes del contenido de los mensajes que las aplicaciones necesiten enviar, aunque eso no implica que sean intercambiables.

El formato y significado específico de los datos que utiliza cada aplicación también está definido por protocolos —llamados **protocolos de aplicación**. Cuando hablamos de «aplicación» no nos estamos refiriendo necesariamente a un programa concreto. Por ejemplo, todo el software de cualquier fabricante que está relacionado con la web utiliza el mismo protocolo de aplicación HTTP. Al contrario de lo que sucede con los protocolos de interred y transporte, hay miles de protocolos de aplicación, pues cada aplicación o servicio tiene su propio objetivo y necesidades de comunicación específicas.

Hay muchos protocolos de aplicación privativos y secretos. Sin embargo, hay unos cuantos que son públicos y abiertos, bien documentados y de uso muy común. Algunos de estos son DNS, SMTP/IMAP/POP, FTP, SSH o el ya mencionado HTTP. Estos protocolos abiertos permiten precisamente que cualquier persona o empresa pueda desarrollar una nueva aplicación que interaccione con las que existen, lo que fomenta competencia e innovación, y favorece el crecimiento de Internet.

4.3. Pila de protocolos

Para manejar más cómodamente los distintos problemas relacionados con la comunicación, su estudio, diseño, especificación e implementación de sus

soluciones, los protocolos se organizan en capas o niveles formando una **pila** de protocolos *(protocol stack)*. Se organiza así porque los protocolos de cada capa se relacionan con los de la capa justo por debajo y por encima de la suya. Todo esto es principalmente una conveniencia de **abstracción**, aunque no por ello es menos útil para entender cómo funciona el conjunto. Como el criterio para definir esta pila y qué poner en cada capa no es único, existen varios modelos de referencia. Los más importantes son el modelo OSI, el TCP/IP y el híbrido, que combina los otros dos.

4.4. Modelo OSI

El modelo de referencia OSI —definido por la ISO— está pensado para aplicarse a cualquier tecnología de comunicaciones. El modelo describe las interfaces, protocolos y servicios que proporciona cada una de las siete capas que lo componen: aplicación, presentación, sesión, transporte, red, enlace y física. Cada capa se centra en resolver un conjunto de problemas específicos. Cada capa ofrece servicios a la capa inmediatamente superior y demanda servicios de la inmediatamente inferior. De ese modo se consigue aislar y desacoplar sus funciones simplificando cada elemento, y haciéndolo reemplazable.

Este es el objetivo de cada capa, brevemente, empezando desde abajo:

1. **Física** – Define las características eléctricas, mecánicas y temporales requeridas en una tecnología de comunicación de datos particular. Ejemplo: específica las dimensiones de los conectores, el voltaje que puede haber en cada pin y su significado, el tamaño y forma de las antenas, las frecuencias que utiliza un emisor, etc.

2. **Enlace** – Se ocupa del intercambio de mensajes entre nodos vecinos (directamente conectados) dentro de una misma red. Cuando la red es un medio de difusión (como una LAN), suele proporcionar un sistema de *direccionamiento físico* ya que, si puede haber varios vecinos escuchando, es necesario indicar de algún modo a quién está realmente dirigido el mensaje.

3. **Red** – Proporciona soporte para comunicaciones *extremo a extremo*, es decir, que implica también dispositivos intermediarios. Permite enviar mensajes individuales de tamaño variable y define un sistema de *direccionamiento lógico*. Una de sus funciones más importantes es la capacidad de interconectar redes de tecnología distinta formando interredes.

4. **Transporte** – Proporciona un canal de comunicación libre de errores entre procesos remotos. Incluye un mecanismo de multiplexación y un sistema de direccionamiento de procesos.
5. **Sesión** – Permite crear sesiones entre nodos remotos y se ocupa de la sincronización.
6. **Presentación** – Define la representación canónica de los datos (reglas de codificación) y su semántica correspondiente.
7. **Aplicación** – Incluye los detalles y protocolos específicos de cada aplicación.

4.4.1. Direccionamiento físico vs. lógico

Los dos tipos de direccionamiento a menudo resultan confusos para los principiantes, pero es importante entender su utilidad y, sobre todo, saber diferenciarlos. Lo primero que debes saber es que cada tecnología está ligada a un tipo de direccionamiento.

Los protocolos de red/interred (como el protocolo IP) utilizan direccionamiento lógico. La dirección lógica (la dirección IP en el caso del protocolo IP) sirve para determinar un destino global, es decir, identifica un nodo que habitualmente está conectado a una red diferente, eventualmente en el extremo opuesto del planeta.

Por contra, el direccionamiento físico es propio de los protocolos y tecnologías de enlace (como WiFi). La dirección física[7] identifica el nodo al que entregar un mensaje en un ámbito local, es decir, el destinatario del mensaje es un nodo vecino, un nodo al que se puede llegar sin pasar a través de ningún intermediario. Por eso, las direcciones físicas no tienen sentido ni utilidad si el destino está fuera del enlace o red local.

En próximos capítulos veremos qué aspecto tienen las direcciones físicas y lógicas, y cómo se utilizan en la práctica. De momento, lo importante es recordar que una dirección física identifica un vecino, mientras que una lógica puede identificar cualquier nodo conectado a la interred, en cualquier parte. Y también recuerda que cada tecnología de comunicación utiliza uno u otro tipo de direccionamiento, pero no ambos.

4.5. Modelo TCP/IP

El modelo de referencia TCP/IP se definió años después de las primeras implementaciones de esos protocolos y trata de formalizar y estandarizar

[7] También conocida como dirección hardware o MAC.

su uso para garantizar interoperabilidad entre los distintos fabricantes. Este modelo define únicamente cuatro capas, que vemos también de abajo hacia arriba:

1. **Host a red** – Asume que existen los mecanismos necesarios para conseguir que un paquete IP pueda ser enviado desde un computador a sus vecinos, es decir, otros computadores conectados al mismo enlace. En realidad, no aborda la problemática específica que ello implica, simplemente da por hecho que es un problema resuelto.

2. **Interred** – Proporciona los mecanismos de interconexión de redes y encaminamiento de paquetes. Define el protocolo IP, que proporciona un servicio de entrega de tipo «datagrama», es decir, que lleva cada paquete hasta el destino de forma independiente, sin considerar si tiene relación con el anterior o el siguiente.

3. **Transporte** – Proporciona mecanismos de comunicación entre procesos. Define, en esencia, dos protocolos de transporte: TCP, que proporciona un servicio confiable y orientado a conexión, y UDP, que únicamente proporciona multiplexación y detección muy básica de errores.

4. **Aplicación** – Incluye los protocolos para los servicios comunes, tales como: DNS, SMTP, HTTP, FTP, etc.

Una aclaración necesaria es que las siglas TCP/IP no se refieren exclusivamente al par de protocolos TCP e IP, sino a la pila completa y todas las implicaciones funcionales que conlleva.

4.6. Modelo híbrido

Por último, el modelo híbrido es una mezcla de los modelos OSI y TCP/IP eliminando «lo que sobra» al modelo OSI (las capas de presentación y sesión son aplicables en relativamente pocas ocasiones) y añadiendo «lo que falta» al modelo TCP/IP. La capa de enlace es decisiva para comprender el funcionamiento de la arquitectura de red. La Figura 4.4 muestra la correspondencia entre los tres modelos descritos.

4.7. Qué NO es Internet

El término «Internet» es de uso tan común que es inevitable que incluso usuarios con formación técnica acaben confundiendo otros conceptos más o menos relacionados.

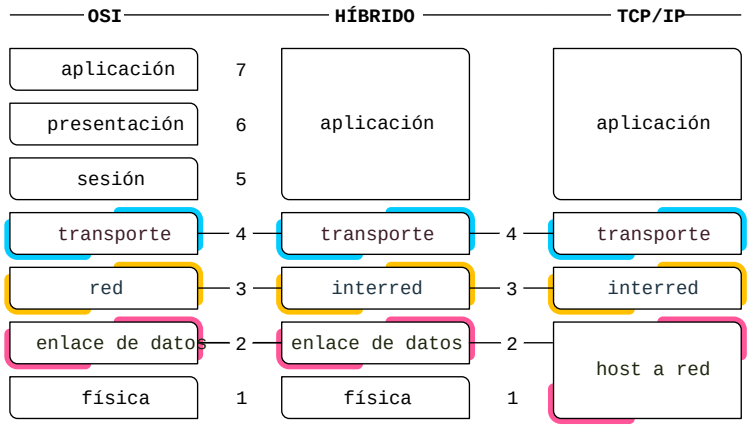


FIGURA 4.4: Correspondencia entre los modelos OSI, TCP/IP e híbrido

4.7.1. Internet no es la web

La web, o más precisamente la WWW (World Wide Web), es un servicio que, al menos inicialmente, proporcionaba un sistema de hipertexto que permite la navegación dentro de un documento (página) o entre varios. Obviamente, esta sigue siendo una de sus funciones básicas, pero hoy en día es mucho más. La web se basa en un protocolo de aplicación (HTTP) y necesita de toda la infraestructura de Internet descrita en este capítulo. Aunque ciertamente la web fue decisiva en el crecimiento y popularidad de Internet entre todo tipo de usuarios, se creó en 1990, mientras que Internet llevaba funcionando desde 1983.

4.7.2. Internet no es la nube

La «nube» es otro término común en el lenguaje cotidiano. No es raro escuchar expresiones como «tengo mis fotos en la nube» o «lo he subido a la nube» que son correctas si se refieren, por ejemplo, a almacenamiento remoto o a una aplicación que proporciona algún proveedor y que se controla por medio de la web.

Pero tampoco es raro que se utilice «la nube» prácticamente como sinónimo de «Internet», y eso sí es claramente incorrecto. Internet es la infraestructura de comunicaciones, y eso no incluye aplicaciones y servicios; de forma similar a como la red eléctrica no incluye los electrodomésticos, o la red de carreteras no incluye los vehículos. En todo caso hay excepciones y podemos considerar algunos servicios clave, como DNS o DHCP, parte de la infraestructura.

Cuestión aparte es la 'computación en la nube' *(cloud computing)*, que aunque también involucra computadores remotos, implica una forma muy particular de gestionarlos y utilizarlos, que veremos más adelante. De momento bastará con que sepas que no todo lo que está en la nube es *cloud computing*.

4.7.3. Internet no es TCP/IP

Es perfectamente posible crear una nueva red o interred basada en la pila de protocolos TCP/IP, con encaminamiento dinámico o características complejas al nivel de Internet, pero eso no la convierte en otra Internet. Solo hay una Internet y es la interred pública global. Es un nombre propio y por eso no puede haber más de una.

4.7.4. Internet no es una empresa u organización

Internet está compuesta por miles de redes interconectadas. Algunas de esas redes son propiedad o están bajo control de diferentes gobiernos, pero la mayoría de ellas son propiedad de operadores de telecomunicaciones, proveedores de acceso (ISP) o proveedores de contenidos y servicios, entre otros. Por eso es una infraestructura descentralizada y, aunque hay actores muy importantes en su gestión, no hay una sola organización que controle todo Internet.

Sin embargo, sí que hay algunos aspectos clave de coordinación que gestiona la ICANN o los RIR como son la asignación ordenada de direcciones, nombres o los números de los Sistemas Autónomos. Sin ellos, u organismos que los sustituyeran, la funcionalidad de Internet no podría seguir creciendo ni podría adaptarse a cambios en su estructura.

Y ¿qué más?

La tecnología TCP/IP es la base de Internet. A diferencia de muchas otras, no se diseñó para reemplazar completamente a las anteriores, sino para integrarse con ellas. El protocolo y el direccionamiento IP proporcionan un medio para que los mensajes que circulan por las redes LAN se puedan llevar (encapsulados) a cualquier otra red del mundo utilizando los routers IP. El protocolo IP es tan versátil y escalable que desde su creación se ha aplicado a tecnologías de enlace que ni siquiera existían cuando se diseñó. IP es una tecnología de integración de redes, que permite crear interredes heterogéneas, y eso es justamente lo que es Internet.

Aunque la interconexión es un problema importante, hay muchos otros. Los modelos de referencia OSI, TCP/IP e híbrido nos permiten desgranar esos problemas en capas y abordar sus soluciones de un modo ordenado y sistemático, y a eso dedicaremos el resto del libro.

Capítulo 5

Protocolos esenciales

Al terminar este capítulo, entenderás:

- Qué función cubre cada uno de los protocolos esenciales.
- Cómo es el formato de los mensajes de esos protocolos.
- Qué es y cómo funciona la encapsulación de mensajes.
- Qué es el direccionamiento lógico y físico.

Los protocolos esenciales son las auténticas tripas de Internet, y como tal, no suelen estar a la vista de los usuarios, pero sin ellos, Internet no sería posible, o sería muy diferente. Estos protocolos han condicionado la evolución de Internet para bien y para mal. Su diseño abierto y simple es en buena medida responsable de su gran éxito, rápida adopción, flexibilidad y de la siempre cuestionada «neutralidad de la red».

Sin embargo, ese mismo diseño también es responsable de sus limitaciones y problemas, como la ausencia de mecanismos de seguridad integrados o la escalabilidad limitada en ciertos aspectos. Hay que tener en cuenta que estos protocolos fueron diseñados para una red experimental, sin siquiera plantear el gran y rápido crecimiento que tendría. Aunque la tecnología de Internet ha evolucionado mucho desde sus comienzos, y se han añadido extensiones y protocolos nuevos para cubrir deficiencias y nuevas necesidades, lo cierto es que la parte esencial sigue funcionando sobre la misma base de protocolos que se diseñaron hace más de 40 años, lo que da una idea de la calidad de su diseño.

En este libro vamos a utilizar el modelo híbrido como guía (Figura 5.1). Esta figura incluye también los protocolos más importantes de TCP/IP, que vamos a estudiar a lo largo del libro. En la capa de enlace se incluye otros dos protocolos: PPP y Ethernet, que aunque no sean parte de TCP/IP son muy utilizados y parte esencial también del funcionamiento de Internet.

La implementación de estos protocolos es muy variopinta. Los protocolos de enlace se implementan típicamente en el hardware de la NIC, los de red

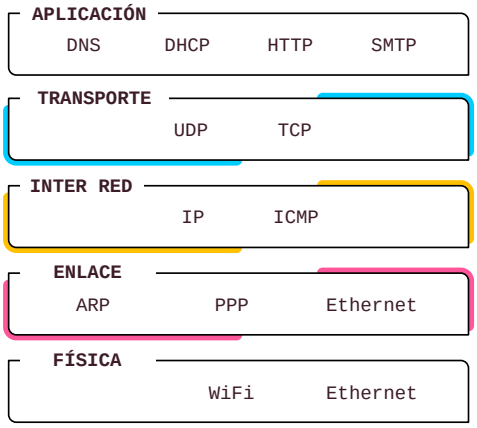


Figura 5.1: Modelo híbrido con los protocolos esenciales

y transporte en el subsistema de red, en el núcleo del sistema operativo, mientras que los de aplicación suelen estar disponibles en forma de librerías, o en el código de las propias aplicaciones si son muy específicos. Pero todo eso puede cambiar de un sistema a otro, por ejemplo los routers de gama media/alta implementan la mayoría de su funcionalidad (incluidos los protocolos de red) por medio de ASIC para conseguir mayor rendimiento y menor consumo.

También aprovecharemos esta primera toma de contacto con los protocolos esenciales para explicar cómo se realiza la comprobación de la conectividad en cada capa y protocolo, ya que además de ser técnicamente relevante, ayuda a entender mejor el rol que juega cada protocolo.

La **conectividad** es la capacidad de —normalmente— dos dispositivos o procesos para intercambiar información entre ellos. Cuando existe un problema de conectividad, no es posible realizar la comunicación. La comprobación de conectividad es una actividad habitual para aislar y localizar problemas en las comunicaciones; y también como forma efectiva de monitorizar el buen funcionamiento del equipamiento y aplicaciones de red.

5.1. Encapsulación

La encapsulación es un mecanismo fundamental de la pila de protocolos. Conceptualmente consiste en colocar el mensaje completo de la capa como «carga útil» del mensaje de la capa inmediatamente inferior. Un ejemplo real sería un datagrama UDP completo que se coloca como carga útil de un

paquete IP, que a su vez se coloca como carga útil de una trama Ethernet. La Figura 5.2 muestra este ejemplo.

Como analogía se puede pensar en un sobre utilizado por el servicio postal. En el exterior del sobre se indica la información del destinatario y en el interior se coloca la carga útil. Esa carga útil podría ser perfectamente otro sobre con indicaciones específicas de la entrega y así sucesivamente. En el último sobre tendríamos el mensaje que el usuario emisor realmente quiere enviar al usuario destino.

En la práctica, los campos de un protocolo, incluido el de carga útil, no son más que una secuencia de bytes, sin embargo, la información que contienen las cabeceras de cada protocolo permite determinar en qué punto de esa secuencia se encuentra cada campo y dónde comienza cada uno de los mensajes encapsulados. Veremos ejemplos concretos de esto más adelante.

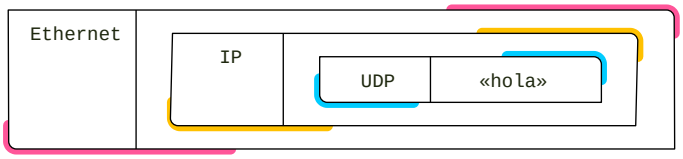


FIGURA 5.2: Ejemplo de encapsulación

Para llevar esto a la práctica utilizaremos `tshark`, una aplicación de análisis de protocolos *(sniffer)*, para capturar el tráfico de red. En un terminal ejecuta el siguiente comando:

```
~# tshark -c 1 -V -f "udp dst port 2000"
```

Para no entrar ahora en demasiados detalles, sirva decir que lo que estamos haciendo es capturar un único datagrama UDP dirigido al puerto 2000. El programa `sudo` es necesario porque la captura de tráfico requiere de permisos especiales[1]. Veremos con detalle cómo capturar y analizar tráfico en el capítulo 19.

Una vez tienes la captura en marcha, en otra consola puedes forzar la generación de un datagrama UDP con `netcat`.

```
~$ echo hola | ncat -u example.net 2000
```

[1] aunque `sudo` no es la forma más recomendable de hacerlo, nos sirve de momento. Consulta § 19.1.1 para una explicación detallada.

Esto crea un mensaje que reproduce exactamente la Figura 5.2, es decir, coloca el texto «hola» como carga útil de un datagrama UDP, que a su vez es la carga útil de un paquete IP, que a su vez es la carga útil de una trama Ethernet. El resultado de la captura de `tshark` será algo parecido al Listado 5.1. Tanto en este como en posteriores listados, hemos eliminado partes de la captura que no son relevantes para el concepto que abordamos en este momento. Aún así te recomendamos hacer la captura por ti mismo en tu propio computador para ver el resultado completo.

```
1  ~# tshark -c 1 -f "udp dst port 2000" -nVx
2  Frame 1: 47 bytes on wire (376 bits) on interface eth0, id 0
3      Interface id: 0 (eth0)
4      Encapsulation type: Ethernet (1)
5      Frame Length: 47 bytes (376 bits)
6      [Protocols in frame: eth:ethertype:ip:udp:data]
7
8  Ethernet II, Src: f8:5f:2a:c0:ff:ee, Dst: f8:99:47:ad:f0:0d
9      Type: IPv4 (0x0800)
10
11 Internet Protocol Version 4, Src: 192.168.0.37, Dst: 172.66.175.59
12     0100 .... = Version: 4
13     .... 0101 = Header Length: 20 bytes (5)
14     Total Length: 33
15     Time to Live: 64
16     Protocol: UDP (17)
17     Header Checksum: 0x4f76 [validation disabled]
18     Source Address: 192.168.0.37
19     Destination Address: 172.66.175.59
20
21 User Datagram Protocol, Src Port: 33262, Dst Port: 2000
22     Source Port: 33262
23     Destination Port: 2000
24     Length: 13
25     UDP payload (5 bytes)
26
27 0000  68 6f 6c 61 0a                                       hola.
28     Data: 686f6c610a
29
30 0000  f8 99 47 ad f0 0d f8 5f 2a c0 ff ee 08 00 45 00   ..G...._*.....E.
31 0010  00 21 cf 0a 40 00 40 11 4f 76 c0 a8 00 25 ac 42   .!..@.@.Ov...%.B
32 0020  af 3b 81 ee 07 d0 00 0d 7a f9 68 6f 6c 61 0a      .;......z.hola.
```

LISTADO 5.1: Captura de un datagrama UDP con `tshark`

Se pueden apreciar 6 secciones en la captura, que corresponden respectivamente a cada uno de los protocolos involucrados y por último al mensaje completo representado en hexadecimal. Se analizan someramente ahora y se van desgranando después en las siguientes secciones de este mismo capítulo.

- **Línea 2**: muestra un resumen del mensaje completo. Indica la cantidad de bytes y bits capturados, la interfaz de red, el tipo de trama y

la cadena de protocolos encapsulados: `eth:ethertype:ip:udp:data`, es decir, una trama Ethernet, que contiene un paquete IP, que contiene un datagrama UDP, que contiene datos.

- **Línea 8**: aparece información de la cabecera Ethernet, con sus direcciones MAC origen y destino, y tipo de carga.
- **Línea 11**: muestra información de la cabecera IP, con las direcciones IP origen y destino y otros campos relevantes.
- **Línea 21**: ofrece información de la cabecera UDP.
- **Línea 28**: representa la carga útil del datagrama UDP, que en este caso es el texto «hola» codificado en ASCII, es decir, la secuencia hexadecimal `686f6c610a`, aunque `tshark` también lo ofrece como texto a la derecha. El punto al final de «hola.» corresponde al byte `0a` que es un salto de línea `\n`.
- Por último, aparece el mensaje completo. En el margen izquierdo muestra el offset de cada fila con un valor de 4 dígitos. En la parte central representa el contenido organizado en filas de 16 bytes, y a la derecha el mismo contenido, pero codificado como texto ASCII, siempre que corresponda con un símbolo del código; en otro caso muestra puntos.

Esto que hemos hecho enviando un texto directamente como carga útil no es lo más habitual. Normalmente el datagrama UDP contiene un mensaje de algún protocolo de aplicación (*p. ej.* DNS), pero lo hemos hecho así para que el ejemplo sea ultra simple, y de todos modos, esto es algo factible, y no contradice ninguna norma o principio, hay protocolos sencillos y reales que hacen cosas similares.

Otro detalle que merece la pena destacar es que `example.net` aunque existe, no tiene ningún servidor UDP escuchando en el puerto 2000, pero por el modo en que funciona UDP, eso no impide que podamos crear el datagrama y enviarlo. Obviamente no va a llegar a ninguna parte y esto es buena prueba de las garantías que ofrece UDP: ninguna.

5.2. Protocolos de enlace

Los protocolos de la capa de enlace de datos se encargan de mover mensajes dentro del enlace local —por ejemplo, en la LAN a la que está conectado nuestro computador. Por tanto lleva mensajes desde un computador o equipo de comunicaciones hasta sus vecinos[2].

[2] dispositivos cercanos que pueden ser alcanzados sin la intervención de intermediarios



Para el modelo TCP/IP, la capa de enlace (incluida en lo que llama «host a red») simplemente se encarga —por medio del protocolo correspondiente— de llevar paquetes IP entre vecinos de la misma red, obviando todos los detalles que ello implica. El modelo OSI por otra parte, sí que entra a estudiar todos los problemas que implica la comunicación en un enlace local.

Dependiendo del medio físico se distingue entre dos tipos de medios:

Punto a punto que permite enviar mensajes únicamente entre dos dispositivos. Un ejemplo sencillo de esto puede ser un enlace de infrarrojos.

Difusión (o *broadcasting*) se caracteriza por el uso de un medio físico compartido por más de dos dispositivos. Esto permite que un mismo mensaje pueda llegar a todos o a un subconjunto determinado de vecinos. El aire en una comunicación WIFI es un ejemplo de medio físico compartido.

Los medios de difusión son típicos de las redes LAN aunque se da en otros ámbitos, como por ejemplo, las comunicaciones satelitales. En función del tipo de medio físico, se pueden clasificar también los protocolos y tecnologías de enlace. Así, Ethernet es un protocolo de enlace de difusión, mientras que PPP es un protocolo para enlaces punto a punto.

En los medios de difusión se requiere una dirección física para determinar a qué nodo se debe enviar el mensaje, mientras que en los enlaces punto a punto no es necesario, porque los mensajes solo tienen un sitio a dónde ir: el otro extremo. Consulta § 4.4.1 para más información.

Los mensajes que circulan por un enlace se llaman «tramas» *(frames)* y son construidos por las NIC a partir de los mensajes que les proporciona el SO. Normalmente, las NIC también se encargan de convertir estas tramas en señales eléctricas, ópticas o de radiofrecuencia que se transmiten por el medio físico correspondiente.

Cada tecnología de enlace tiene sus limitaciones y características específicas como puede ser la velocidad de transmisión (también llamado «ancho de banda»), la distancia máxima que puede haber entre los dispositivos conectados o la cantidad máxima de dispositivos que pueden estar conectados.

5.3. Ethernet

Con mucha diferencia sobre el resto, las tecnologías LAN más utilizadas en la actualidad son Ethernet IEEE 802 y WiFi que, salvando las distancias, podríamos ver como una especie de «Ethernet inalámbrica».

Ethernet cubre los detalles tecnológicos de la transmisión de señales (la capa física), pero también la funcionalidad, codificación y formateo de los mensajes (la capa de enlace) que es en lo que nos vamos a centrar en este momento.

5.3.1. Interfaces de red

Para conectar nuestro computador a una LAN Ethernet o cualquier otra tecnología de enlace como WiFi, Bluetooth, GSM, LTE o incluso infrarrojos, necesitamos hardware específico, es decir, una interfaz de red (NIC) que soporte Ethernet, WiFi, o la tecnología correspondiente, y también un controlador *(driver)* de dispositivo que se encargue de la gestión y transferencia de datos con el SO.

Hoy en día en el ámbito del PC o *smartphone*, muchos de estos dispositivos suelen estar integrados en la placa base (especialmente WiFi y Bluetooth), pero también están disponibles como periféricos externos, normalmente mediante conexión USB.



FIGURA 5.3: Interfaz de red Ethernet a USB

La forma más sencilla de ver con qué interfaces de red cuenta el nodo es el comando `ip addr` (`iproute2`)[3] como en el ejemplo del Listado 5.2. Se omite la información no relevante en este momento. Puedes probar a ejecutarlo en tu propio computador para ver la salida completa en tu caso.

```
1 ~$ ip addr
2 1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN
3     link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
4     inet 127.0.0.1/8 scope host lo
5 2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP
6     link/ether f8:5f:2a:c0:ff:ee brd ff:ff:ff:ff:ff:ff
7     inet 192.168.0.37/24 brd 192.168.0.255 scope global dynamic noprefixroute eth0
```

LISTADO 5.2: Salida del comando `ip addr`

La salida tiene dos secciones diferenciadas, una especie de «fichas», para cada una de las dos interfaces de red que ha encontrado, en este caso: `lo` y `eth0`. La interfaz `eth0` corresponde a una NIC de tipo Ethernet (**línea 6**) con la dirección física: `f8:5f:2a:c0:ff:ee`.

Si hubiera una segunda NIC Ethernet se llamaría `eth1`, y del mismo modo las NIC WiFi tienen nombres como `wlan0`, aunque dependiendo del SO, su versión o su configuración pueden llamarse `eno1` o `enp0s32d5` para Ethernet, `wlp1s0` para WiFi, `s0` o `ppp1` para interfaces serie, etc.

5.3.2. Trama Ethernet

El formato de la trama Ethernet (Figura 5.4) es bastante simple.

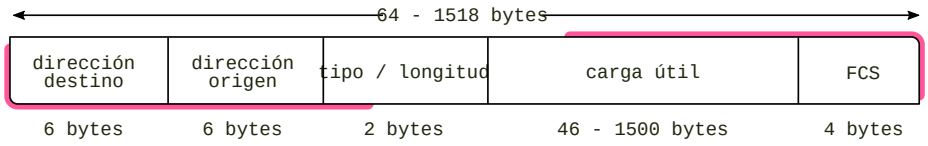


FIGURA 5.4: Formato de la trama Ethernet

Un pequeño resumen de los campos que forman la cabecera:

- Las direcciones MAC de las NIC origen y destino.
- `Tipo/longitud` puede indicar el tamaño en bytes del campo `payload` o bien un código que indica de qué tipo (qué protocolo) es la carga útil.

[3] Se puede abreviar como `ip a`.

- La carga útil *(payload)* que es de tamaño variable (mínimo 46 bytes y máximo 1500). En este campo se coloca un mensaje de la capa superior, en Internet un paquete IP, aunque puede transportar otras cosas, como ARP.
- FCS, una especie de suma de comprobación que permite al receptor verificar que la trama no ha sufrido cambios.

Esta es la parte de la captura 5.1 que corresponde a la cabecera Ethernet:

```
Ethernet II, Src: f8:5f:2a:c0:ff:ee, Dst: f8:99:47:ad:f0:0d
  Type: IPv4 (0x0800)
```

Aparecen claramente las direcciones origen y destino y el tipo, que en este caso es `0x0800` e indica que la carga útil es un paquete IP. Este campo se conoce comúnmente como `EtherType`[4]. El tamaño de la cabecera Ethernet es por tanto de 14 bytes (6 para dirección origen, 6 para dirección destino y 2 para tipo).

Obviando todos los detalles del funcionamiento de Ethernet, que no son pocos, el servicio que ofrece es simple: lleva la trama hasta la NIC con la dirección destino que aparece en la cabecera, siempre que sea un vecino de la misma LAN. Las tramas Ethernet no pueden salir nunca de la LAN en la que se crearon.

Podemos ver la trama Ethernet como un contenedor en el que colocar una secuencia de bytes arbitraria (el campo `payload`) y Ethernet se encargará de llevar esa trama hasta el computador destino. La NIC emisora calcula el FCS y lo añade al final. Al llegar a su destino, la NIC receptora realiza el mismo cálculo; si corresponde con el valor FCS, la trama es correcta y será entregada a la capa superior, probablemente IP. Si el resultado de ese cálculo no coincide, la trama será descartada...y eso es todo. Ethernet no informa del error a nadie, no pide una retransmisión, no hay temporizadores ni confirmaciones. Si una trama se corrompe durante su viaje, el receptor la descartará sin más. Obviamente habrá muchas aplicaciones que no puedan tolerar ese comportamiento, pero resolver ese problema no es responsabilidad de Ethernet.

5.3.3. Direcciones MAC

En los protocolos de enlace de medio compartido (como Ethernet) se identifica cada dispositivo con una dirección física o hardware. En el caso de

[4] Puedes encontrar la lista oficial de tipos Ethernet en `https://www.iana.org/assignments/ieee-802-numbers/ieee-802-numbers.xhtml`

Ethernet, también se le llama «dirección MAC» o simplemente «dirección Ethernet».

Se trata de un número grabado por el fabricante en la memoria ROM de la NIC. La dirección es única, no existen dos tarjetas Ethernet con la misma dirección en todo el mundo, o al menos no deberían. Es lo que se llama un «identificador único global» o UUID. A pesar de eso, se utiliza únicamente para identificar esta tarjeta solo en el ámbito de la red LAN a la que está físicamente conectada. Todo esto es igualmente aplicable a una NIC WiFi y otros protocolos y tecnologías similares.

En concreto, la dirección MAC Ethernet es un número de 6 bytes que por convenio se representa como una lista de dígitos en hexadecimal separados por el carácter ':', por ejemplo: `f8:5f:2a:c0:ff:ee`. Puedes ver la dirección MAC de una interfaz de tu computador con el programa `ip`.

```
~$ ip link show eth0
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP
    link/ether f8:5f:2a:c0:ff:ee brd ff:ff:ff:ff:ff:ff
```

La dirección MAC está formada por dos partes:

OUI (3 bytes) Identifica al fabricante de la tarjeta. Este prefijo lo otorga la IANA. Puedes consultar la lista de todos los OUI asignados actualmente en su web[5].

NIC (3 bytes) Es un número de serie que el fabricante garantiza que es único en su producción.

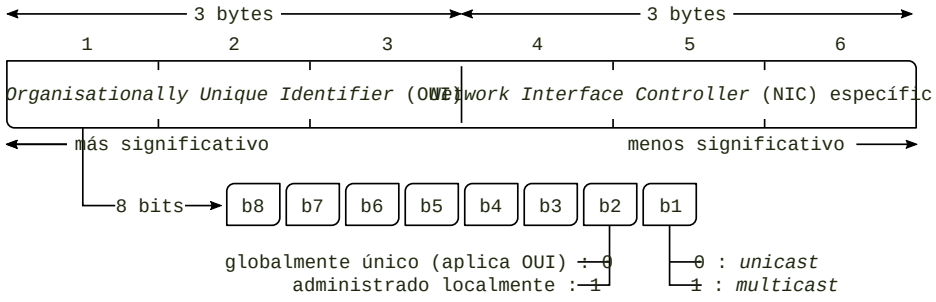


FIGURA 5.5: Formato de la dirección MAC Ethernet

Aparte de la entrega para un único destinatario *(unicast)*, Ethernet proporciona otros dos métodos de entrega:

[5] `http://www.iana.org/assignments/ethernet-numbers`

broadcast Permite enviar un mensaje a todos los vecinos conectados a la misma LAN. Para ello se utiliza como destino una dirección especial que tiene todos sus bits a uno; en hexadecimal `FF:FF:FF:FF:FF:FF`.

multicast Permite enviar un mensaje a un subconjunto de los vecinos. Para ello se utilizan direcciones que tienen prefijos reservados.

5.3.4. Conectividad Ethernet

La comprobación de conectividad en el caso de Ethernet consiste en verificar que la NIC está en disposición de comunicarse con un vecino de la LAN por sí mismo, sin la participación de ningún otro protocolo.

Si tu computador dispone de una NIC Ethernet normalmente estará conectada a un conmutador *(switch)*, ya sea directamente o a través de una roseta en la pared. Físicamente el conector 8P8C[6] hembra suele presentar dos LEDs (ver Figura 5.6), que ofrecen información interesante sobre el estado del enlace.

El LED de la izquierda aparece encendido si se ha podido establecer el enlace de datos con el vecino (el conmutador). Es habitual también que su color indique el modo (velocidad) a la que ha realizado la negociación: verde para 10 Mbps, naranja para 100 Mbps y amarillo para 1 Gbps.

El LED de la derecha (de color ámbar) parpadea cada vez que se transmite o recibe una trama.

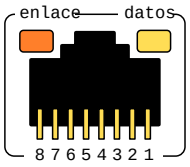


FIGURA 5.6: Conector 8P8C (RJ45) hembra de un equipo Ethernet

Aparte de la comprobación visual anterior, es posible preguntar al controlador del dispositivo con un programa específico. Uno de esos programas es `ethtool`. Lo siguiente es la salida de dicho programa para una interfaz Ethernet de un PC conectado a un conmutador.

```
1 ~# ethtool eth0
2 Settings for eth0:
3 Supported ports: [ TP ]
4 Supported link modes:   10baseT/Half 10baseT/Full
```

[6]El conector 8P8C se conoce más común, aunque erróneamente, como RJ45.

```
5                          100baseT/Half 100baseT/Full
6                          1000baseT/Half 1000baseT/Full
7   Supports auto-negotiation: Yes
8   Advertised link modes:  10baseT/Half 10baseT/Full
9                          100baseT/Half 100baseT/Full
10                         1000baseT/Half 1000baseT/Full
11  Advertised pause frame use: No
12  Advertised auto-negotiation: Yes
13  Speed: 100Mb/s
14  Duplex: Full
15  Port: Twisted Pair
16  PHYAD: 1
17  Transceiver: internal
18  Auto-negotiation: on
19  MDI-X: Unknown
20  Supports Wake-on: g
21  Wake-on: g
22  Current message level: 0x000000ff (255)
23  Link detected: yes
```

Se trata de un controlador Gigabit (**línea 6**) conectado a un conmutador (**línea 14**)[7] de 100 Mbps (**línea 14**) y el enlace de datos está correctamente establecido (**línea 23**).

5.4. PPP

PPP (Point to Point Protocol) [8] es un protocolo de enlace, pero a diferencia de Ethernet, se utiliza en enlaces punto a punto, como por ejemplo, una conexión directa entre dos routers, o entre un router doméstico y el ISP (la llamada «ultima milla»). También se utiliza de forma habitual sobre enlaces virtuales, túneles y VPNs.

Es un protocolo muy versátil, lo que explica sus usos tan variados:

- Tiene la capacidad de encapsular distintos protocolos de red.
- Puede configurar los detalles de esos protocolos de red (*p. ej.* la dirección IP del suscriptor) mediante los llamados NCP.
- Permite establecer, configurar, probar y cerrar el enlace, con el protocolo LCP.
- Ofrece mecanismos para autenticar al suscriptor mediante distintos protocolos de autenticación como PAP o CHAP.

El formato básico de la trama PPP es relativamente sencillo (Figura 5.7), aunque puede variar en función del enlace, opciones o protocolos que encapsule.

Por compatibilidad con HDLC los campos `Dirección` y `Control` tienen un valor fijo de `0xFF` y `0x03` respectivamente. El campo `Proto` es similar al

[7] Solo se puede establecer un enlace *full duplex* en Ethernet conmutada, es decir, el PC está conectado a un conmutador (*switch*).

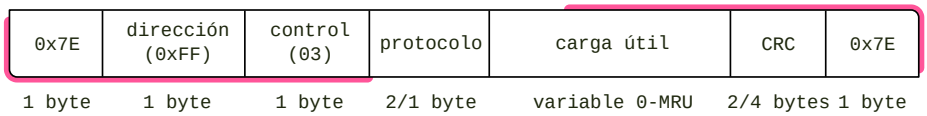


FIGURA 5.7: Formato de la trama PPP

campo `Type` de Ethernet e indica el tipo de carga útil: IP, IPX, AppleTalk, etc. El campo `Carga útil` es el mensaje encapsulado y CRC es un código de verificación. El flag `0x7E` indica el inicio y fin de cada mensaje.

5.5. IP

IP[8] es el protocolo de interred. Su tarea es llevar paquetes IP desde un punto del planeta a cualquier otro mediante comunicación 'extremo a extremo' *(end-to-end)*. Sin embargo, no ofrece garantías de entrega: IP es un protocolo *best-effort*. Eso significa que hará lo posible por llevar el paquete a su destino, pero bajo determinadas condiciones, lo «mejor posible» puede ser absolutamente nada.

IP es el acrónimo de 'Internet Protocol', pero cuidado, no significa 'el protocolo de la Internet', sino 'protocolo de interred', es decir, el que hace posible la interconexión de redes y la creación de interredes.

5.5.1. Interfaces de red

Las interfaces de red tienen una dirección lógica (una dirección IP), aparte de la dirección hardware. La dirección IP, en el caso de un dispositivo doméstico (PC, smartphone, etc.), se obtiene habitualmente de forma automática solicitándola a un servidor DHCP. Desde el punto de vista de su ámbito, se distingue entre 2 tipos de direcciones:

- **Privadas**: Se asignan a dispositivos inaccesibles desde Internet. Se usan en redes privadas y solo sirven para comunicarse con computadores dentro de la misma red o interred privada[9].
- **Públicas**: Se asignan a dispositivos accesibles desde cualquier parte. Se asignan a computadores directamente conectados a Internet, como servidores web, correo, etc.

[8] Cuando veas escrito «IP» estamos hablando de IPv4 (IP versión 4), que sigue siendo la más utilizada hoy en día. Para referirnos a 'IP versión 6' siempre lo escribiremos como IPv6.

[9] Existe un mecanismo llamado NAT que permite superar parcialmente esa limitación.

Como veremos más adelante, se han definido unos rangos de direcciones específicos para cada tipo de dirección.

En el caso de una red doméstica, el proveedor de servicios (el ISP) puede asignar una única dirección IP pública al *router* doméstico, y el *router* a su vez, asigna direcciones privadas a los dispositivos de la red doméstica.

En el caso del Listado 5.2, la dirección IP de la interfaz `eth0` es `192.168.0.37`, que es privada.

5.5.1.1. Interfaz *loopback*

Cualquier SO que implemente IP ofrece una interfaz de red llamada *loopback*[10]. Es una interfaz de red bastante especial. Se dice que es una interfaz «virtual» porque no está ligada a ninguna NIC. A pesar de ello, se puede utilizar para cualquier servicio basado en TCP/IP teniendo en cuenta que su ámbito de trabajo está limitado al propio nodo, tal como indica la **línea 4**: `scope host` del Listado 5.2.

Como no hay una NIC asociada, la interfaz *loopback* no tiene dirección física y tiene siempre la dirección IP especial `127.0.0.1`, independiente del tipo de computador o sistema operativo. El nombre simbólico para esa dirección IP es «localhost».

La **línea 2** muestra otros datos interesantes:

- Es de tipo bucle: `LOOPBACK`.
- Tiene una MTU de 16 436 bytes.

5.5.2. Paquete IP

El formato del paquete IP [9] se muestra en la Figura 5.8.

Significado y utilidad de cada campo:

`versión`
: Versión del protocolo. Siempre contiene el valor `4` porque estamos describiendo el protocolo IP versión 4, que es la que se utiliza desde el nacimiento de Internet y sigue siendo la de mayor uso en la actualidad, aunque convive con la versión 6.

IHL
: (Internet Header Length) es el tamaño de la cabecera expresado en palabras de 4 bytes. Eso significa que el valor mínimo posible es `5`, es decir, una cabecera estándar de 20 bytes, y el máximo es 15 que sería una cabecera de 60 bytes.

DSCP
: contiene un código qué indica el nivel de servicio deseado para el paquete (capítulo 17).

[10] que podríamos traducir como «bucle local»

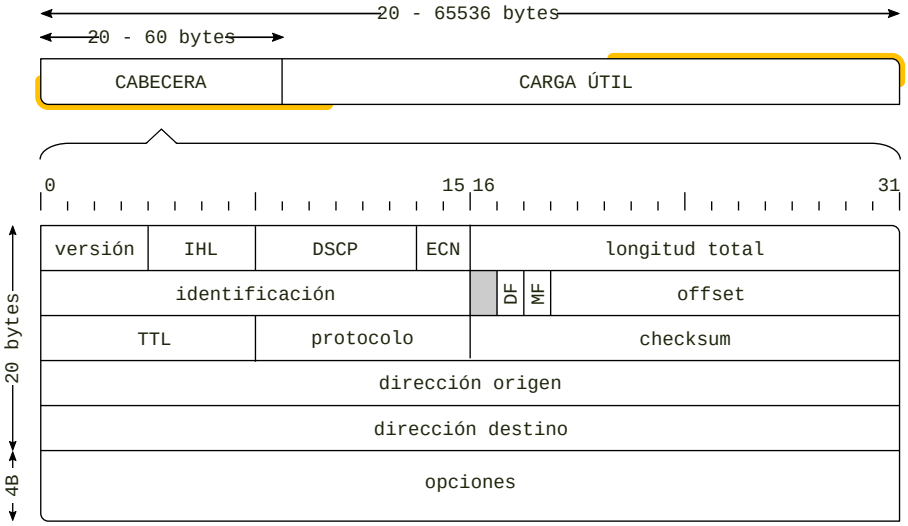


FIGURA 5.8: Formato del paquete IP

ECN
: se utiliza para el control de congestión (capítulo 14).

`longitud total`
: La longitud total del paquete incluyendo cabecera y carga útil. Como es un entero de 16 bits implica que el tamaño máximo de un paquete IP es 65 535 bytes.

`identificación`
: Un número que identifica a todos los fragmentos que proceden de un mismo paquete IP original.

DF (Don't Fragment), le indica a los routers que no deben fragmentar este paquete.

MF (More Fragments), indica que éste no es el último fragmento.

`offset`
: Indica qué posición ocupa este fragmento en el paquete original.

TTL Indica cuántos routers puede atravesar este paquete (saltos) antes de ser descartado.

`protocolo`
: Indica el protocolo al que corresponde la carga útil del paquete.

`checksum`
: Una suma de comprobación para verificar que la cabecera no ha sufrido cambios inesperados.

`dirección origen`
: La dirección IP del nodo que crea el paquete.

dirección destino
: La dirección IP del nodo destinatario del paquete.

Esta es la parte de la captura 5.1 que corresponde con la cabecera IP (esta vez incluyendo más detalles):

```
Internet Protocol Version 4, Src: 192.168.0.37, Dst: 172.66.175.59
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
    Identification: 0xcf0a (53002)
    010. .... = Flags: 0x2, Don't fragment
    ...0 0000 0000 0000 = Fragment Offset: 0
    Total Length: 33
    Time to Live: 64
    Protocol: UDP (17)
    Header Checksum: 0x4f76 [validation disabled]
    Source Address: 192.168.0.37
    Destination Address: 172.66.175.59
```

Muestra la versión (4), la longitud de la cabecera (5=20 bytes), el identificador (`0xcf0a`), el flag DF (no fragmentar), el offset del fragmento (0), la longitud total del paquete (33 bytes), el TTL (64), el protocolo de la carga útil (17=UDP), la suma de comprobación de la cabecera (`0x4f76`), la dirección origen (`192.168.0.37`) y, por último, la dirección destino (`172.66.175.59`).

5.5.3. Direcciones IP

A diferencia de la dirección MAC, la dirección IP no viene grabada en el dispositivo sino que es asignada cada vez que el computador se conecta a una red[11]. La dirección IP es un número de 32 bits y tiene una estructura jerárquica que la relaciona con esa red y se utiliza para identificar al computador en toda la interred, y no solo en ese enlace.

La dirección IP se representa como 4 cifras en base decimal separadas por puntos. Puedes averiguar la dirección IP de una interfaz de tu computador también con el programa `ip`.

```
~$ ip addr show eth0
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP
    link/ether f8:5f:2a:c0:ff:ee brd ff:ff:ff:ff:ff:ff
    inet 192.168.0.37/24 brd 192.168.0.255 scope global dynamic noprefixroute eth0
```

Como en Ethernet, aparte del direccionamiento *unicast*, hay otros modos:

broadcast Es posible direccionar, al menos en teoría, todos los hosts de una red.

[11] Por esto se las denomina «direcciones lógicas»

multicast Se puede direccionar un grupo arbitrario de hosts utilizando direcciones de grupo (las que empiezan por 240). Este mecanismo no es trivial ya que implica la suscripción activa de los hosts a grupos y requiere soporte en los *routers* que deben propagar la información de membresía. Para ello se utiliza un protocolo específico llamado IGMP [10] y protocolos de encaminamiento específicos.

En el capítulo 7 veremos todos los detalles del direccionamiento IP.

5.5.4. Conectividad IP

Una vez has verificado que el computador puede establecer un enlace de datos con sus vecinos puedes comprobar si es posible enviar (y recibir) tráfico IP hacia o desde otro PC, independientemente si es un vecino o está al otro lado del planeta. Las herramientas habituales para comprobar la conectividad IP se basan en el protocolo ICMP, por lo que se explican más adelante, en § 5.7.1, una vez se haya presentado ese protocolo.

5.6. ARP

ARP (Address Resolution Protocol) es un protocolo auxiliar de IP necesario cuando un nodo conectado a un enlace de difusión (asumamos Ethernet) necesita comunicarse con sus vecinos. En este caso, el nodo emisor debe construir una trama, en la que debe colocar la dirección MAC del vecino destino. Sin embargo, el nodo emisor no conoce ese dato, pero lo que sí conoce es la dirección IP del destino.

ARP resuelve este problema. El nodo emisor envía una petición ARP a todos los vecinos (es un mensaje *broadcast*) preguntando si hay alguno que tenga asignada la IP destino. Si lo hay, el vecino que la tenga, construirá una respuesta ARP con su dirección MAC y la enviará específicamente al nodo solicitante (este mensaje es *unicast*). El vecino conoce la dirección MAC del solicitante porque aparece en la petición ARP.

El formato del mensaje ARP se muestra en la Figura 5.9. Es un mensaje de tamaño variable porque se puede utilizar con distintos protocolos de enlace y/o red, que pueden tener direcciones (físicas y lógicas) de tamaños distintos.

los campos de un mensaje ARP se listan a continuación. Para cada campo, se indica entre paréntesis el valor que tendría para el caso en que el protocolo de enlace sea Ethernet o WiFi y el de red sea IP, que es el caso más habitual.

***hardware type* (HTYPE)**
Es un código que indica el tipo de dirección física (Ethernet=1).

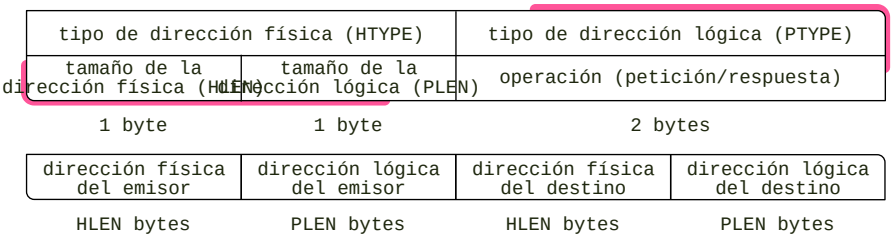


FIGURA 5.9: Formato del mensaje ARP

***protocol type* (PTYPE)**
: Tipo de dirección lógica (IP =0x0800).

***hardware length* (HLEN)**
: Número de bytes de la dirección física (en bytes) (Ethernet=6).

***protocol length* (PLEN)**
: Número de bytes de la dirección lógica (en bytes) (IP =4).

operation
: Operación a realizar: 1=petición, 2=respuesta.

sender hardware address
: Dirección física del emisor (6 bytes para Ethernet).

sender protocol address
: Dirección lógica del emisor (4 bytes para IP).

target hardware address
: Dirección física del destinatario.

target protocol address
: Dirección lógica del destinatario.

Podemos realizar una captura de una petición y una respuesta ARP con el siguiente comando. Probablemente solo hay que esperar unos segundos para que aparezcan algunas tramas. Como en las capturas anteriores, se muestran solo las partes relevantes en este momento:

```
~# tshark -f arp -nVx
Frame 1: 60 bytes on wire (480 bits) on interface eth0, id 0
    [Protocols in frame: eth:ethertype:arp]
Ethernet II, Src: f8:99:47:ad:f0:0d, Dst: ff:ff:ff:ff:ff:ff
    Type: ARP (0x0806)
    Padding: 000000000000000000000000000000000000
Address Resolution Protocol (request)
    Hardware type: Ethernet (1)
    Protocol type: IPv4 (0x0800)
    Hardware size: 6
    Protocol size: 4
    Opcode: request (1)
    Sender MAC address: f8:99:47:ad:f0:0d
    Sender IP address: 192.168.0.1
    Target MAC address: 00:00:00:00:00:00
    Target IP address: 192.168.0.37
```

```
0000  ff ff ff ff ff ff f8 99 47 ad f0 0d 08 06 00 01   ...vd.....7....
0010  08 00 06 04 00 01 f8 99 47 ad f0 0d c0 a8 00 01   ..........7....
0020  00 00 00 00 00 00 c0 a8 00 25 00 00 00 00 00 00   .............
0030  00 00 00 00 00 00 00 00 00 00 00 00               ............

Frame 2: 42 bytes on wire (336 bits) on interface eth0, id 0
    [Protocols in frame: eth:ethertype:arp]
Ethernet II, Src: f8:5f:2a:c0:ff:ee, Dst: f8:99:47:ad:f0:0d
    Type: ARP (0x0806)
Address Resolution Protocol (reply)
    Hardware type: Ethernet (1)
    Protocol type: IPv4 (0x0800)
    Hardware size: 6
    Protocol size: 4
    Opcode: reply (2)
    Sender MAC address: f8:5f:2a:c0:ff:ee
    Sender IP address: 192.168.0.37
    Target MAC address: f8:99:47:ad:f0:0d
    Target IP address: 192.168.0.1

0000  f8 99 47 ad f0 0d f8 5f 2a c0 ff ee 08 06 00 01   .....7...vd.....
0010  08 00 06 04 00 02 f8 5f 2a c0 ff ee c0 a8 00 25   .........vd.....
0020  f8 99 47 ad f0 0d c0 a8 00 01                     .....7....
```

Lo que vemos aquí es una petición ARP (frame 1) enviada por el router local (`192.168.0.1`) y la respuesta ARP correspondiente (frame 2), que la envía el mismo computador que está haciendo la captura (`192.168.0.37`). Claramente el formato de ambos mensajes es el mismo, solo cambia el valor del campo `Opcode` que indica precisamente si es una petición o una respuesta. Observa que la petición va sobre una trama broadcast (dirección destino `ff:ff:ff:ff:ff:ff`), mientras que la respuesta va en una trama unicast (dirigida a la dirección MAC del emisor de la petición). Quizá lo más relevante es que el campo `Target MAC address` de la petición tiene un valor cero, porque ese es precisamente el dato que desconoce y pretende averiguar. Observa también que mientras la petición incluye un relleno (*padding*) para lograr que la trama Ethernet llegue al mínimo de 64 bytes, en la respuesta no aparece. Esto es porque esta respuesta ha sido capturada antes de salir del computador. Cuando la trama se envíe, la NIC añadirá el relleno.

ARP es lo que se llama un protocolo de descubrimiento de vecinos *(neighbor discovery)* o de resolución de direcciones, ya que en la práctica permite averiguar la dirección MAC de un nodo a partir de su dirección IP. De hecho, los programas de captura de tráfico suelen resumir un mensaje de petición ARP como «Who has `192.168.0.37`? Tell `192.168.0.1`» y la respuesta como «`192.168.0.37` is `f8:5f:2a:c0:ff:ee`».

Cabe señalar que los nodos no preguntan por las MAC de los vecinos cada vez que tienen que enviar una trama. El nodo dispone de una caché donde

se guardan las últimas respuestas, y así puede reutilizarlas evitando tráfico innecesario.

Puedes ver la caché ARP de tu computador con el programa `arp` (`net-tools`).

```
~# arp -n
Address           HWtype  HWaddress           Flags Mask   Iface
192.168.0.200     ether   44:09:17:e0:b8:5c   C            eth0
192.168.0.1       ether   f8:99:47:ad:f0:0d   C            eth0
192.168.0.198     ether   7c:83:cb:a4:34:b7   C            eth0
192.168.0.184     ether   00:21:66:58:5c:c8   C            eth0
```

Ahora que conoces ARP puedes utilizarlo como forma de comprobar la conectividad Ethernet. Puedes generar una consulta ARP con el programa `arping`. Si el vecino responde, significa que está conectado y operativo. Eso es lo que hace el siguiente comando:

```
~# arping -c2 192.168.0.1
ARPING 192.168.0.1
60 bytes from f8:99:47:ad:f0:0d (192.168.0.1): index=0 time=2.279 msec
60 bytes from f8:99:47:ad:f0:0d (192.168.0.1): index=1 time=10.207 msec

--- 192.168.0.1 statistics ---
2 packets transmitted, 2 packets received,   0% unanswered (0 extra)
rtt min/avg/max/std-dev = 2.279/6.243/10.207/3.964 ms
```

A diferencia del ping ICMP, `arping`, ni IP ni ICMP entran en juego, y por eso puedes analizar problemas específicos de Ethernet.

5.7. ICMP

ICMP [11] también es un protocolo auxiliar de IP, que sirve para:

- Enviar notificaciones sobre problemas o errores en la entrega de paquetes.
- Realizar diagnósticos e informar sobre el estado de la red.

Los mensajes ICMP se encapsulan sobre datagramas IP, pero a pesar de ello lo consideramos un protocolo de la capa de interred[12].

El formato del mensaje ICMP es sencillo (Figura 5.10). Sin embargo, dependiendo del tipo de mensaje, puede incluir campos con significados específicos y distinta carga útil.

Los mensajes ICMP de error se envían siempre al nodo que creó el paquete que provocó el problema. Por ejemplo, si un *router* no sabe cómo llevar

[12]Para muchos autores (y también para nosotros) la capa en la que se ubica un protocolo tiene que ver más con su función que con el protocolo sobre el que se encapsula.

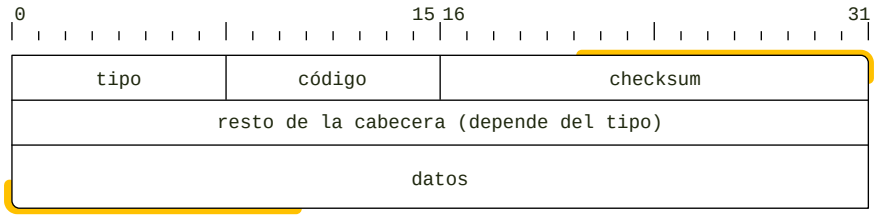


FIGURA 5.10: Formato del mensaje ICMP

un paquete a su destino final, creará un mensaje ICMP de tipo 3 (destino inalcanzable) y lo enviará al nodo que lo envió, es decir, colocará el mensaje ICMP en un paquete IP cuya dirección destino es la dirección origen del paquete problemático, y colocará su propia dirección IP (la del *router*) como dirección origen. Eso significa que el nodo sabe quién le está informando del problema. En este caso además, el mensaje ICMP incluye una copia de la cabecera del paquete IP problemático más 8 bytes de su carga útil. Así el SO puede determinar cuál de sus procesos fue el responsable del paquete problemático y, a su vez, ese proceso podría informar al usuario mediante una excepción u otro mecanismo de reporte de error disponible.

Para ilustrar el formato de la cabecera ICMP, esta es la captura de una petición *Echo Request* —más conocido como *ping*— y su correspondiente respuesta. Este tipo de mensaje se puede enviar fácilmente con el programa del mismo nombre (`ping` (`iputils-ping`)), una herramienta sencilla, pero muy útil y común para comprobar la conectividad IP.

Por un lado, vamos a poner a `tshark` a capturar mensajes ICMP:

```
~# tshark -f icmp -nV
```

Y por otro, vamos a enviar un único mensaje *ping* al nodo `example.net`:

```
$ ping -c 1 example.net
PING example.net (172.66.175.59) 56(84) bytes of data.
64 bytes from 172.66.175.59: icmp_seq=1 ttl=52 time=159 ms

--- example.net ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 159.448/159.448/159.448/0.000 ms
```

La captura de `tshark` ofrece algo parecido a esto:

```
Frame 1: 98 bytes on wire (784 bits), on interface wlp1s0, id 0
  [Protocols in frame: eth:ethertype:ip:icmp:data]
Ethernet II, Src: 08:00:46:de:ca:f0, Dst: f0:f5:a5:de:fa:ce
  Type: IPv4 (0x0800)
Internet Protocol Version 4, Src: 192.168.0.37, Dst: 172.66.175.59
```

```
    Total Length: 84
    Protocol: ICMP (1)
Internet Control Message Protocol
    Type: 8 (Echo (ping) request)
    Code: 0
    Checksum: 0x8d86 [correct]
    Identifier (BE): 65286 (0xff06)
    Identifier (LE): 1791 (0x06ff)
    Sequence Number (BE): 1 (0x0001)
    Sequence Number (LE): 256 (0x0100)
    Timestamp from icmp data: May  6, 2024 21:54:10.393995000 CEST
    Data (40 bytes)
      101112131415161718191a1b1c1d1e1f202122232425262728292a2b2c2d2e2f3031323334353637

Frame 2: 98 bytes on wire (784 bits), on interface wlp1s0, id 0
    [Protocols in frame: eth:ethertype:ip:icmp:data]
Ethernet II, Src: f0:f5:a5:de:fa:ce, Dst: 08:00:46:de:ca:f0
    Type: IPv4 (0x0800)
Internet Protocol Version 4, Src: 172.66.175.59, Dst: 192.168.0.37
    Total Length: 84
    Protocol: ICMP (1)
Internet Control Message Protocol
    Type: 0 (Echo (ping) reply)
    Code: 0
    Checksum: 0x9586 [correct]
    Identifier (BE): 65286 (0xff06)
    Identifier (LE): 1791 (0x06ff)
    Sequence Number (BE): 1 (0x0001)
    Sequence Number (LE): 256 (0x0100)
    Timestamp from icmp data: May  6, 2024 21:54:10.393995000 CEST
      101112131415161718191a1b1c1d1e1f202122232425262728292a2b2c2d2e2f3031323334353637
```

Puedes comprobar fácilmente que los valores de los campos de la cabecera ICMP coinciden con la descripción de cada campo. Puedes ver los tipos de mensaje ICMP en detalle en la sección 8.6.

5.7.1. Conectividad IP (ahora sí)

El programa `ping` permite verificar que el nodo consultado está activo, accesible, y es capaz de recibir y enviar paquetes IP hacia y desde el nodo desde el que hemos lanzado el comando, es decir, verifica conectividad simétrica. En el caso del ejemplo anterior, está verificando que existe conectividad entre el nodo local y nodo `example.net`, pero eso no significa necesariamente que tenga la misma conectividad con otros nodos de la misma red.

Cabe destacar que `ping` no sirve para demostrar lo contrario, es decir, un nodo perfectamente funcional y con plena conectividad IP podría no responder a *ping* por distintos motivos. Es posible configurar un cortafuegos para ignorar mensajes ICMP o específicamente mensajes *Echo Request*. Tampoco se puede usar `ping` para demostrar conectividad asimétrica, es decir, que el tráfico IP es posible en un sentido, pero no en el contrario.

En cualquier caso, se puede utilizar ICMP para comprobar la conectividad IP de otras formas y con otras herramientas, entre ellas `ping`, que se detalla a continuación junto con algunas de las funciones que puede ofrecer.

5.7.1.1. ping

Aparte de la verificación de conectividad IP, que obviamente es la funcionalidad principal, una consecuencia interesante del funcionamiento del programa `ping` es que puede calcular el RTT (Round-Trip Time) o «tiempo de vuelo», es decir, el tiempo transcurrido desde que se envió la petición hasta que se recibe la respuesta.

Prueba a ejecutar `ping` hacia tu *router* local:

```
1  $ ping 192.168.0.1
2  PING 192.168.0.1 (192.168.0.1) 56(84) bytes of data.
3  64 bytes from 192.168.0.1: icmp_req=1 ttl=255 time=2.02 ms
4  64 bytes from 192.168.0.1: icmp_req=2 ttl=255 time=0.730 ms
5  64 bytes from 192.168.0.1: icmp_req=3 ttl=255 time=1.66 ms
6  ^C
7  --- 192.168.0.1 ping statistics ---
8  3 packets transmitted, 3 received, 0% packet loss, time 2000ms
9  rtt min/avg/max/mdev = 0.730/1.471/2.023/0.546 ms
```

Ejecutado de esta forma (sin argumentos), el programa enviará mensajes *ping* de forma indefinida (uno por segundo) hasta que pulses `Control-C` (lo que ha pasado en la **línea 6**).

Las líneas 3 a 5 aparecen al recibirse cada respuesta. Para cada una se muestra el número de secuencia (icmp_req), el TTL (Time To Live) del paquete IP y el RTT (en milisegundos). Como el nodo destino (el *router* local en este caso) es un vecino, ese tiempo ronda los 2 ms.

Cuando el programa termina, muestra unas estadísticas sobre lo ocurrido (**línea 8**). Aparece el número de peticiones enviadas, respuestas recibidas, porcentaje de peticiones perdidas (sin respuesta) y el tiempo total que ha requerido. En la última línea aparecen los RTT mínimo, medio, máximo y la desviación estándar, que son datos interesantes para tener una idea de la latencia entre estos dos nodos y también de cómo varía con el tiempo. Por cierto, esta variación se conoce como *jitter* y es importante para cierto tipo de aplicaciones como la telefonía IP, el streaming multimedia o los videojuegos en línea.

El programa `ping` es una de las formas más fáciles y habituales para establecer un proceso de monitorización automática para una serie de nodos que formen parte de la infraestructura de red de una organización, y de hecho prácticamente todos los sistemas de monitorización lo proporcionan de un modo u otro.

Como todos los comandos, `ping` también tiene opciones. Estas son algunas de las más interesantes para modificar su comportamiento:

`-c` Envía la cantidad de peticiones indicada y termina.

`-i` Permite fijar el intervalo entre envíos. Se especifica en segundos y admite decimales.

`-A` Modo adaptativo, calcula el intervalo en función del RTT medido.

`-f` Envía la siguiente petición tan pronto como vuelve la respuesta anterior.

5.7.1.2. ping «extendido»

Algunas implementaciones de `ping` proporcionan opciones para características avanzadas que, en realidad, se corresponden con campos y parámetros de los mensajes: TTL, tamaño de la carga del mensaje ICMP, etc. Algunas de las más interesantes:

`-p` Permite indicar el contenido de los bytes de relleno de la carga del paquete ICMP.

`-R` Graba y muestra la ruta seguida por la petición y su respuesta.

`-s` Indica el tamaño (en bytes) de la carga útil del mensaje de petición. El tamaño por defecto es 56 bytes.

`-t` Fija el valor del TTL de la cabecera IP en los mensajes de petición.

5.8. UDP

UDP es un protocolo de transporte. Eso significa que se encarga de mover mensajes entre procesos, que lógicamente pueden estar ejecutándose en nodos diferentes. Los mensajes UDP se llaman «datagramas» y se encapsulan sobre paquetes IP. Para determinar a qué nodo se debe enviar el datagrama se utiliza la dirección IP destino; para determinar el proceso dentro de ese nodo se utiliza el puerto destino. A esto se le llama «multiplexación» y es la que permite que distintos procesos, incluso de la misma pareja de nodos, puedan mantener flujos de tráfico simultáneos e independientes.

El puerto es un simple entero de 16 bits sin estructura. Su función es permitir la comunicación entre procesos: el proceso servidor solicita al SO vincularse a un número específico para permanecer en estado de *escucha* pasiva, es decir, ese proceso queda inactivo en espera de una petición proveniente de un cliente. Cada puerto solo puede estar vinculado a un proceso, aunque un proceso puede vincularse a múltiples puertos.

Para organizar los distintos servicios los puertos están organizados en tres rangos:

Bien conocidos Desde el 1 hasta el 1023 están reservados para servicios comunes y muy habituales. Se requieren permisos específicos para arrancar un servidor vinculado a un puerto de este rango.

Registrados Desde el 1024 hasta el 49 151 están reservados para servicios registrados.

Dinámicos y efímeros Desde el 49 152 hasta el 65 535 están reservados para servicios dinámicos y puertos efímeros, que son los que el SO asigna de forma automática cuando el programador no especifica uno concreto.

UDP es un protocolo muy simple. De hecho, prácticamente la única funcionalidad que ofrece sobre IP es la citada multiplexación. No tiene corrección de errores, ni control de flujo, congestión, o duplicados... Simplemente, se coloca una secuencia de bytes como carga útil del datagrama y se envía al destino como una unidad individual e independiente, y sin ninguna garantía de que vaya a llegar. Tampoco proporciona conexión o desconexión, y por eso se dice que es un «protocolo sin conexión» *(connectionless)*.

A pesar de que a primera vista parezca un protocolo de poca utilidad o interés, el hecho es que UDP es un protocolo ligero y eficiente, con mínima sobrecarga. Eso lo hace adecuado para situaciones en las que no es crítico que todos los mensajes lleguen a su destino, o cuando los recursos son limitados o la sobrecarga de un protocolo más complejo es inaceptable; para transmisiones en tiempo real, transmisión de audio o vídeo en tiempo real, actualización de estado en videojuegos, monitorización o telemetría.

5.8.1. Datagrama UDP

La Figura 5.11 muestra el formato del datagrama UDP. Por supuesto la cabecera UDP también es muy simple:

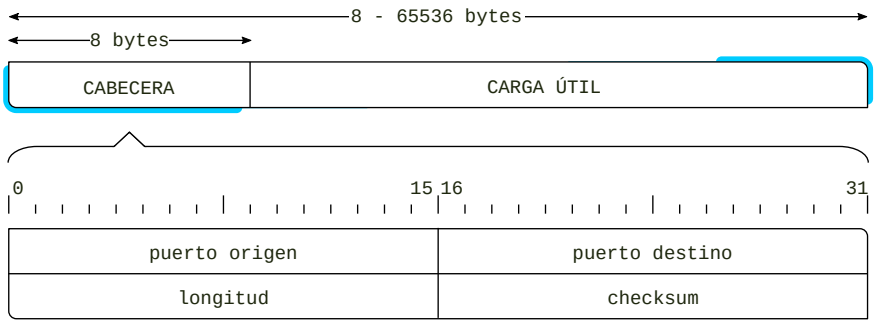


FIGURA 5.11: Formato del datagrama UDP

Puerto origen
: Puerto asociado al proceso que envía el datagrama. Como no tiene por qué haber respuesta, este campo es opcional (en cuyo caso contendrá un 0).

Puerto destino
: Puerto asociado al proceso al que va dirigido el datagrama.

Longitud
: Longitud total del datagrama UDP incluyendo cabecera y carga útil.

Checksum
: Suma de comprobación, que sirve para verificar si el datagrama ha sufrido corrupción en los datos durante la transmisión. Esto ofrece una detección básica de errores, aunque también es opcional.

Cabecera UDP correspondiente a la captura 5.1:

```
User Datagram Protocol, Src Port: 33262, Dst Port: 2000
  Source Port: 33262
  Destination Port: 2000
  Length: 13
  Checksum: 0x995d [unverified]
  UDP payload (5 bytes)

0000  68 6f 6c 61 0a                                    hola.
  Data: 686f6c610a
```

Se distinguen claramente los cinco campos: el datagrama UDP indica que el puerto destino es 2000, el origen el 33 262, tiene una longitud de 13 bytes (incluyendo la carga útil), y la suma de comprobación es `0x995d`. La carga útil es la palabra «hola» seguida de un salto de línea, que codificado en ASCII y hexadecimal son los 5 bytes `686f6c610a`, y más los 8 bytes de la cabecera, hacen el total de 13 bytes.

5.8.2. Conectividad UDP

La conectividad UDP implica la posibilidad de que un cliente sea capaz de enviar un datagrama UDP y el proceso servidor al que va dirigido lo pueda recibir adecuadamente.

El mensaje que acabamos de analizar corresponde con los comandos de la sección 5.1, pero no sirve para verificar conectividad UDP, ya que solo es un mensaje enviado por un cliente y capturado desde el mismo computador, ni siquiera hay servidor.

Para comprobar la conectividad vamos a usar `ncat` creando un servidor UDP en un nodo y también con `ncat` un cliente UDP en el otro nodo. El servidor se crea con:

```
alice@node1$ ncat -u -l 2000
```

Y en otra consola ejecutamos el cliente con:

```
bob@node2$ ncat -u node1 2000
```

Ahora escribe algo de texto en la consola del cliente, pulsa ENTER y debería aparecer en la del servidor. Si quieres comprobar la conectividad en el sentido opuesto escribe algo de texto en la del servidor, pulsa ENTER y debería aparecer en la del cliente.

Es importante destacar que, como en el caso de IP, es perfectamente posible tener conectividad en uno de los sentidos, pero no en el otro. Un problema de encaminamiento o un cortafuegos en uno de los dos nodos o en cualquiera de los routers del camino podría descartar los datagramas UDP solo en uno de los sentidos.

5.9. TCP

TCP es «el otro» protocolo de transporte de TCP/IP. Podríamos decir que TCP es lo contrario a UDP en casi todo. Para empezar, TCP está orientado a conexión, lo que implica que antes de enviar o recibir datos, cliente y servidor obligatoriamente tienen que intercambiar información de sincronización —durante el proceso de conexión— y, de forma similar, también se aplica un procedimiento de desconexión que asegura que ambos extremos saben que el otro da por terminada la comunicación.

La conexión asegura que solo cuando ambas partes tienen la información necesaria sobre el otro puede comenzar la transmisión de datos. A diferencia de UDP, TCP garantiza la integridad de los datos, es decir, evita que se producen cambios ni haya partes ausentes o duplicadas. Para lograr esto, TCP utiliza un mecanismo de confirmación, temporizadores y retransmisión de los segmentos. «Segmento» es el nombre que se da a los mensajes que se intercambian en TCP. Además, incluye mecanismos de control de flujo para evitar saturar al receptor, y de congestión para evitar colapsar la red.

La comunicación TCP se comporta como un flujo continuo y bidireccional de bytes. Ambos procesos pueden enviar datos adicionales en cualquier momento y de forma indefinida. Sin embargo, los procesos no controlan cuándo se envían realmente los datos a través de la red. Es el SO el que se encarga de agrupar esos bytes en segmentos y enviarlos en el momento más adecuado para maximizar la eficiencia, al mismo tiempo que se aplican los mecanismos de control de flujo y congestión.

5.9.1. Segmento TCP

TCP es un protocolo bastante complejo, de modo que comprender el funcionamiento de estos mecanismos, incluso de una forma somera, requiere

abordarlos cada uno por separado, y eso lo haremos en sucesivos capítulos. Por supuesto esta complejidad se ve reflejada en el formato de su cabecera, que se muestra en la Figura 5.12.

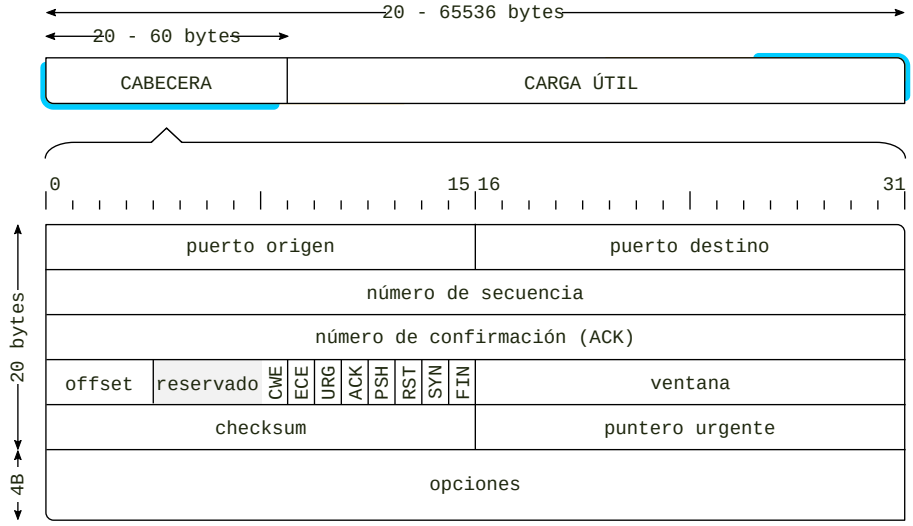


FIGURA 5.12: Formato del segmento TCP

Aunque veremos la función que desempeña cada uno de estos campos a lo largo del libro, se describen brevemente a modo de referencia. En este contexto, cuando hablemos de «emisor» nos estaremos refiriendo al nodo que ha enviado específicamente **este** segmento, y cuando hablemos de «destinatario» para referirnos al nodo al que va dirigido.

`puerto origen`
: Puerto asociado al proceso emisor.

`puerto destino`
: Puerto asociado al proceso destinatario.

`número de secuencia`
: Número de secuencia del primer byte de la carga útil del segmento.

`número de confirmación`
: Número de secuencia del siguiente byte que el emisor espera recibir. Dicho de otro modo, es un acuse de recibo de todos los datos hasta el inmediatamente anterior al indicado.

`offset`
: Tamaño de la cabecera expresado en palabras de 32 bits. Indica el lugar donde comienza la carga útil, por lo que también se llama *data offset*.

`flags`
: Los flags son un conjunto de bits que ofrecen información del segmento o sobre su emisor. Los más importantes son:

Flag	Descripción
CWR	(Congestion Window Reduced) El emisor ha recibido un segmento con el bit ECE activado y ha reducido su tasa de emisión.
ECE	(ECN Echo) El emisor sufre congestión.
URG	El campo `puntero urgente` es significativo.
ACK	El campo `acuse de recibo` es significativo.
PSH	El destino debe pasar los datos al proceso tan pronto como pueda.
RST	El emisor rechaza (o cierra) la conexión.
SYN	El emisor quiere establecer una conexión.
FIN	El emisor ha terminado de enviar datos.

Cuadro 5.1: Descripción de los flags TCP

`ventana`
: Tamaño de la ventana del receptor. El receptor indica al emisor de cuánto espacio libre (en bytes) dispone para aceptar datos nuevos.

`checksum`
: Suma de comprobación.

`puntero urgente`
: Indica la posición del último byte urgente.

`opciones`
: Cabeceras adicionales para propósitos específicos, que tienen su propio formato.

5.9.2. Capturando una conexión `TCP`

Como en las secciones anteriores, podemos capturar mensajes TCP con `tshark`. Vamos a replicar con TCP una situación equivalente a la de UDP. En una consola ejecuta:

```
~# tshark -i lo -f "tcp port 2000"
```

En otra consola ejecuta un servidor TCP:

```
~$ ncat -l -p 2000
```

Y por último, en una tercera consola, ejecuta un cliente TCP, escribe algo y pulsa `Control-D` (que es el carácter de fin de archivo) que cerrará la conexión.

```
~$ ncat 127.0.0.1 2000
hola
C-d
```

La captura de `tshark` ofrece lo siguiente:

```
1  127.0.0.1 → 127.0.0.1  TCP 74 47178 → 2000 [SYN] Seq=0 Win=65495 Len=0 MSS=65495
2  127.0.0.1 → 127.0.0.1  TCP 74 2000 → 47178 [SYN, ACK] Seq=0 Ack=1 Win=65483 Len=0
↪  [...]
3  127.0.0.1 → 127.0.0.1  TCP 66 47178 → 2000 [ACK] Seq=1 Ack=1 Win=65536 Len=0
4  127.0.0.1 → 127.0.0.1  TCP 71 47178 → 2000 [PSH, ACK] Seq=1 Ack=1 Win=65536 Len=5
5  127.0.0.1 → 127.0.0.1  TCP 66 2000 → 47178 [ACK] Seq=1 Ack=6 Win=65536 Len=0
6  127.0.0.1 → 127.0.0.1  TCP 66 47178 → 2000 [FIN, ACK] Seq=6 Ack=1 Win=65536 Len=0
7  127.0.0.1 → 127.0.0.1  TCP 66 2000 → 47178 [FIN, ACK] Seq=1 Ack=7 Win=65536 Len=0
8  127.0.0.1 → 127.0.0.1  TCP 66 47178 → 2000 [ACK] Seq=7 Ack=2 Win=65536 Len=0
```

Esta vez hemos hecho una captura más sencilla. Al no añadir el argumento `-v` a `tshark`, solo muestra un resumen de algunos campos importantes de las cabeceras.

Cada línea incluye varios datos:

- Las direcciones IP de origen y destino. En este caso es `127.0.0.1` para ambos ya que los estás ejecutando en dos consolas de tu computador y además el cliente ha conectado mediante la interfaz *loopback*.
- Los puertos origen (47178, asignado automáticamente para el cliente) y 2000 (que es donde has puesto el servidor).
- El protocolo del mensaje más específico. En este ejemplo, el payload del segmento TCP no encapsula un mensaje de un formato que `tshark` pueda reconocer, de modo que indica TCP.
- El tamaño del segmento completo. 74 bytes para el primer segmento.
- Los flags activos. Por ejemplo, en el primer segmento está activo el flag SYN.
- El número de secuencia (Seq) y de confirmación (Ack).
- El tamaño de la ventana de recepción (Win).
- La longitud de la carga útil (Len). Solo el mensaje 4 porta datos (la cadena «hola\n») que corresponde a esos 5 bytes que indica.

Lo primero que llama la atención es que escribir un solo mensaje (con el texto «hola») ha provocado 8 segmentos. Esto es porque TCP realiza un proceso de conexión previo y un proceso de desconexión al final.

- Los mensajes 1 a 3 corresponden al proceso de conexión.
- Los mensajes 4 y 5 corresponden al envío de la carga útil y su correspondiente segmento de acuse de recibo.
- Los mensajes 6 y 7 corresponden a la desconexión.

Para poder entender plenamente toda esta información y también otros campos de las cabeceras que ni se están mostrando aquí, tendrás que esperar a próximos capítulos donde veremos el funcionamiento detallado de TCP. Pero, de momento, esto ya te da una idea de la considerable diferencia de complejidad y funcionalidad que ofrece respecto a UDP, incluso para un ejemplo tan simple como ese.

5.9.3. Conectividad TCP

El ejemplo que has realizado con `ncat` en la sección anterior para la captura es también una comprobación de conectividad TCP perfectamente válida, siempre que, lógicamente, ejecutes cliente y servidor en nodos diferentes.

Fíjate que comprobar la conectividad TCP implica obligatoriamente comprobar la simetría, ya que es imposible enviar datos si primero no se establece la conexión, y la conexión requiere segmentos en ambos sentidos. Tener conectividad en un sentido no implica necesariamente que exista en el otro. Un cortafuegos podría filtrar específicamente el segmento de conexión (SYN) de A hacia B impidiendo la conexión, pero no a la inversa. Por eso, si necesitas verificar la posibilidad de establecer la conexión en ambos sentidos, tendrás que repetir la prueba cambiando los roles como cliente y servidor.

Y ¿qué más?

Esto ha sido un vistazo rápido al formato y utilidad de los protocolos principales. Es probable que como lector tengas ahora más preguntas y dudas que antes de empezar el capítulo. No te preocupes, es normal. La intención era abrir camino y mostrar la parte más tangible de los protocolos: sus mensajes. Por eso hemos hecho énfasis en las capturas de tráfico real y también por eso es muy importante que las realices por tu cuenta.

En próximos capítulos veremos con más detalle el funcionamiento de cada protocolo y cómo cada campo de estas cabeceras tiene consecuencias clave en el comportamiento de la red. Nos quedan algunos protocolos importantes como DNS, DHCP, HTTP ... pero esos todavía tendrán que esperar hasta que tengas una bases sólidas sobre direccionamiento, encaminamiento y programación de redes.

Capítulo 6

Sockets

Al terminar este capítulo, entenderás:

- Qué son los sockets UDP y TCP, y cuáles son sus diferencias.
- Cómo se vincula un socket a un puerto.
- Cómo enviar y recibir datos con sockets UDP y TCP.
- Qué es la E/S parcial y cómo afecta a la programación de redes.
- Cómo delimitar los mensajes en una comunicación TCP.
- Qué es `netcat` y cómo usarlo para probar servidores UDP y TCP.

Un socket es un punto de conexión a la red de comunicaciones, de forma análoga a como un enchufe —que por cierto es la traducción de *socket*— es un punto de conexión a la red eléctrica. Ahí acaba el parecido porque, obviamente, los sockets de los que hablamos aquí no son dispositivos físicos, sino una mera abstracción de programación.

Para ser más precisos, técnicamente 'socket' es un API, 'un conjunto de funciones', para hacer posible el intercambio de datos entre dos procesos. Los sockets se parecen un poco a las tuberías *(pipes)* de POSIX; el proceso emisor escribe en un extremo y el proceso receptor lee del otro extremo.



A menudo se les llama «BSD sockets» porque el sistema operativo BSD 4.2 fue el primero en incluirlos allá por 1983. A partir de ahí se convirtió en el estándar de facto y fue adoptado por otros sistemas operativos como GNU/Linux, Windows, macOS, etc. Y así sigue siendo en la actualidad.

Ambos, tuberías y sockets, son mecanismos de IPC (Inter-Process Communication) pero, a diferencia de las tuberías, que solo permiten comunicar procesos que se ejecutan en un mismo nodo, los sockets comunican procesos que se ejecutan en nodos diferentes (incluso en lugares opuestos del planeta), y también son bidireccionales, entre otras importantes diferencias.

Aunque los sockets son una abstracción relativamente simple, son la base de todo lo demás. No es descabellado decir que prácticamente todo el software que utiliza la red para comunicarse, con cualquier SO, escrito en cualquier lenguaje y sobre cualquier plataforma, lo hace a través de sockets o de librerías que se basan en sockets.

6.1. Programación de redes

La *programación de redes* como traducción directa del inglés *network programming* engloba las técnicas, herramientas, librerías, patrones, etc. que permiten crear aplicaciones que se comunican a través de la red, o más específicamente, a la parte del desarrollo de una aplicación que se encarga de todo eso. La programación de redes engloba dos aspectos clave: los sockets y los modelos de interacción. El más utilizado de estos modelos es cliente-servidor, pero existen muchos otros como publicador-suscriptor o comunicación de pares *(peer-to-peer)*.

En este capítulo nos centraremos en los sockets y las ideas básicas del modelo cliente-servidor. Vamos a tratar *solo lo esencial* acerca de los sockets porque el objetivo es que el lector adquiera las nociones que le permitan aplicar una perspectiva pragmática a los conceptos que veremos en el resto del libro.

Dejaremos para más adelante otros asuntos más avanzados como la serialización de datos, el rendimiento, la seguridad, etc. Aún así «lo esencial» no es poca cosa. La programación de redes, y los sockets en particular, son un campo amplio, lleno de sutilezas y rincones oscuros.

6.2. La clase `socket`

La librería estándar de Python ofrece soporte para sockets en el módulo `socket`. En concreto este módulo proporciona una clase que también se llama `socket`. Incluye los métodos necesarios para manejar conexiones, enviar y recibir datos, etc. El constructor de la clase es:

```
socket(family, type, proto)
```

El parámetro `family` define un conjunto particular de sockets para una tecnología o pila de protocolos de red. Hay familias para Bluetooth (`AF_BLUETOOTH`), ATM (`AF_ATMPVC`), CAN (`AF_CAN`), comunicaciones infrarrojas (`AF_IRDA`) y muchas otras. En este capítulo nos limitaremos a la familia `AF_INET` que corresponde a la pila TCP/IP con IPv4.

El parámetro `type` determina el modelo de comunicación. Esencialmente, hay dos tipos:

- `SOCK_STREAM`: Para comunicaciones de flujo *(stream)*. La proporcionan protocolos orientados a conexión y habitualmente confiables.
- `SOCK_DGRAM`: Para comunicaciones de datagramas. La proporcionan protocolos no orientados a conexión y normalmente no confiables.

El parámetro `proto` indica qué protocolo quieres usar. Como en la familia `AF_INET` el único protocolo de tipo `SOCK_DGRAM` es UDP y el único protocolo de tipo `SOCK_STREAM` es TCP, es decir, como solo hay un protocolo de cada, este parámetro se puede y se suele omitir.

Resumiendo, puedes crear sockets UDP con:

```
socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
```

Y sockets TCP con:

```
sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

Pero como `AF_INET` es la familia por defecto, y `SOCK_STREAM` es el tipo por defecto, se puede simplificar la creación de un socket TCP a:

```
sock = socket.socket()
```

6.3. Puertos

Ambos, UDP y TCP, son protocolos de transporte, es decir, sirven para mover datos entre procesos, que es el objetivo principal de los sockets. Para lograrlo, cada socket necesita «direccionar» al otro extremo —el proceso remoto. Esto se logra con dos datos: la dirección IP del nodo remoto y el **puerto** vinculado al proceso remoto. Los puertos son enteros de 16 bits y aparecen en las cabeceras de ambos protocolos. Lógicamente, el puerto origen identifica al proceso emisor.

La tupla (IP, puerto) que identifica a un proceso globalmente en Internet se denomina «endpoint» o también «socket»[1]. Un **flujo** *(flow)* corresponde con el conjunto de paquetes que se intercambian dos procesos, definido por una pareja de endpoints: (IP origen, puerto origen) e (IP destino, puerto destino) y un protocolo de transporte. Ambas direcciones IP aparecen en la cabecera IP, y ambos puertos aparecen en la cabecera de transporte (TCP o UDP).

[1] En este libro utilizaremos «endpoint» para evitar la ambigüedad de «socket».

Los routers encaminan los paquetes hacia el nodo que tiene asignada la dirección destino, que aparece en la cabecera IP. Cuando el paquete llega al nodo, el SO extrae el mensaje de transporte y entrega la carga útil al proceso al que corresponda el puerto destino que aparece en esa cabecera de transporte. Este mecanismo, por el cual el SO determina a qué proceso corresponden los datos en función de los puertos, se llama *multiplexación*.

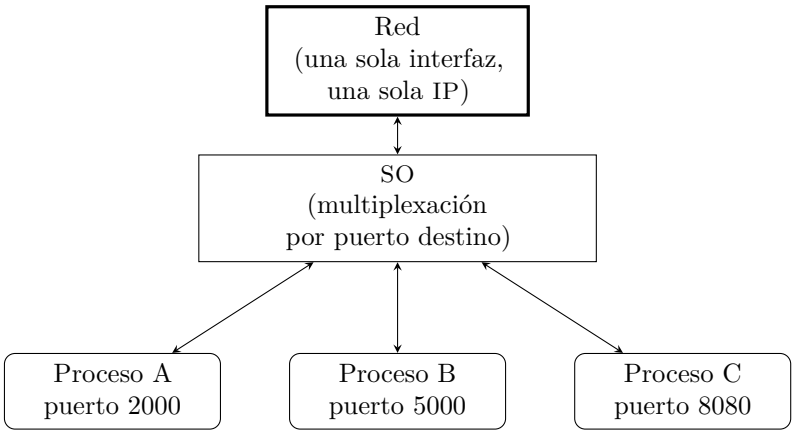


FIGURA 6.1: Varios procesos comparten la misma IP y se distinguen por el puerto

Para vincular un socket a un puerto se utiliza el método `bind()`. Esta operación es imprescindible para crear un servidor porque el servidor tiene el rol pasivo de la comunicación. El servidor espera a que un cliente le contacte y por eso su endpoint debe ser conocido de antemano por el cliente. El servidor, por su parte, puede averiguar el endpoint del cliente en el momento que este le contacte. El Listado 6.1 muestra cómo utilizar `bind()` para un socket UDP, aunque es igual para un socket TCP.

```
import socket

sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
sock.bind(('192.168.0.37', 2000))
```

LISTADO 6.1: Vinculando un socket a un puerto UDP

El argumento de `bind()` es una tupla (ip, puerto)[2], no solo el puerto. Esto debe ser así porque el nodo dispone de varias interfaces de red, cada una con su dirección IP, y puedes querer que el socket quede vinculado solo a alguna de esas interfaces. Cuando quieras que el servidor sea accesible

[2]Fíjate que hay otra pareja de paréntesis dentro de los de la llamada a `bind()`.

a través de cualquier interfaz del nodo —algo bastante frecuente— puedes usar la dirección especial `INADDR_ANY` (`0.0.0.0`), aunque en Python se puede usar también una cadena vacía o `None`, como en `('', 2000)`.

```
sock.bind(('', 2000))
```

El puerto 2000 no tiene nada de especial, es solo un ejemplo. Técnicamente hablando puedes usar cualquier puerto libre entre 1 y 65 535, aunque hay ciertos convenios. Los puertos inferiores a 1024 están reservados para servicios importantes y requieren privilegios especiales. Por ejemplo, el puerto 80 es el puerto asignado para el protocolo HTTP y el 443 para HTTPS. Entre el 1024 y el 49 151 son puertos registrados, asociados también a servicios o protocolos específicos[3], aunque cualquier usuario puede ejecutar un servidor que los utilice. Por último, los puertos por encima de 49 151 son «puertos efímeros» y son asignados automáticamente por el SO cuando no se especifica, que es lo habitual en los clientes.

Tener un número limitado de puertos implica también una cantidad limitada de procesos (en un mismo nodo) con capacidad de comunicarse con la red. Para ser exactos, un nodo puede tener un máximo de 65 535 procesos utilizando sockets TCP y otros 65 535 utilizando sockets UDP. No es un problema, es más que suficiente para cualquier uso razonable.

Desde el momento que se invoca `bind()`, se dice que el puerto está «abierto». Por contra, si no hay ningún proceso vinculado a determinado puerto, decimos que está «cerrado». Es posible comprobar el estado de los puertos con el comando `ss` (`iproute2`)[4] como se ve en el Listado 6.2. Solo un puerto abierto es susceptible de recibir y aceptar mensajes. Es el primer paso para crear un servidor UDP. El servidor TCP es algo más complejo.

```
$ ss -lpun
State    Recv-Q Send-Q     Local Address:Port     Peer Address:Port
UNCONN   0      0               0.0.0.0:2000            0.0.0.0:*
```

LISTADO 6.2: Comprobando un socket UDP abierto con `ss`

El programa `ss` utiliza los servicios del SO para averiguar esta información, de modo que puedes considerarla 100 % fidedigna. Desde fuera del nodo, también es posible averiguar el estado de los puertos, aunque en ese caso los programas especializados —llamados escáners de puertos *(port scanners)*—

[3] La lista oficial completa de todos los puertos y sus asignaciones en `https://www.iana.org/assignments/service-names-port-numbers/service-names-port-numbers.txt`

[4] `ss` sustituye a `netstat` (`net-tools`), aunque este último sigue estando disponible aún en muchas distribuciones de GNU/Linux.

recurren a técnicas de sondeo que no son tan fiables. Uno de los escáneres de puertos más conocidos es `nmap` y lo veremos más adelante en el libro.

Para liberar un puerto, se debe invocar el método `socket.close()`, aunque también ocurre cuando el proceso termina. Cada puerto solo puede estar vinculado a un socket, y por tanto a un proceso. Si se intenta ejecutar `bind()` con el mismo puerto en otro socket se producirá un error:

```
OSError: [Errno 98] Address already in use
```

6.4. Comunicación UDP

UDP es un protocolo muy simple (§ 5.8). Su cabecera solo tiene cuatro campos: los puertos origen y destino, la longitud del mensaje y un checksum (que además es opcional). La única funcionalidad real que ofrece UDP respecto a IP es precisamente esa *multiplexación*.

El SO asigna un puerto al socket cliente cuando contacta con el servidor. En el caso de UDP, eso ocurre cuando envía el primer mensaje. El Listado 6.3 muestra un ejemplo práctico.

```
1 $ python
2 >>> import socket
3 >>> sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
4 >>> sock.getsockname()
5 ('0.0.0.0', 0)
6 >>> sock.sendto(b'hello', ('192.168.0.37', 2000))
7 >>> sock.getsockname()
8 ('0.0.0.0', 48845)
```

LISTADO 6.3: Cliente UDP mínimo

El método `getsockname()` devuelve el endpoint *local* actual, es decir, la IP y puerto al que está asociado el socket en ese momento. El puerto 0 que aparece en la **línea 5** significa que el socket aún no tiene puerto asociado. En la **línea 6** se utiliza el método `sendto()`, que es la manera de enviar datos con un socket UDP. El primer argumento son los datos a enviar y el segundo, el endpoint destino al que deben enviarse. Al hacer esto, el SO le asigna un puerto al socket. Puedes ver cuál ejecutando de nuevo `getsockname()` como hace el listado.

Al ser UDP un protocolo sin garantías, es posible ejecutar sin quejas este código sin que de hecho exista un servidor escuchando en el endpoint destino indicado. Por supuesto, el mensaje se perderá, pero esto es lo que ofrece UDP.

Y otra cuestión importante. Como UDP es un protocolo sin conexión, no hay un destino prefijado para los datos. Por eso, es necesario indicar el endpoint en cada llamada al método `sendto()`. Eso implica que un cliente UDP puede indicar destinos diferentes para cada mensaje que envía, y también que el servidor puede recibir mensajes de clientes distintos en cada recepción.

Un servidor UDP mínimo (Listado 6.4) tiene una estructura sencilla. Después de crear y vincular el socket a un puerto, se utiliza un bucle que recibe un mensaje, lo procesa y devuelve una respuesta al emisor.

```
import socket

sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
sock.bind(('', 2000))

while 1:
    data, client = sock.recvfrom(1024)
    print(f"Se ha recibido el mensaje '{data}' desde '{client}'")
    sock.sendto(f"Enviaste {len(data)} bytes".encode(), client)
```

Listado 6.4: Un servidor UDP mínimo

El método `recvfrom()` es la contraparte del método `sendto()`. Su valor de retorno es una tupla con los datos recibidos y el endpoint del emisor. A continuación el código del listado imprime lo recibido en pantalla y devuelve al cliente un mensaje que indica la longitud de los datos recibidos. Tanto `sendto()` como `recvfrom()` manejan secuencias de bytes (no texto), por eso los mensajes llevan el prefijo `b`, que es el modo que tiene Python para indicar que son `bytes`[5].

El argumento de `recvfrom()` indica la cantidad máxima de memoria que el programa está dispuesto a utilizar para almacenar el mensaje recibido. Si el emisor envía un mensaje mayor, el receptor solo recibirá los primeros 1024 bytes (para el caso del ejemplo) y el resto se perderá. Esto es propio de UDP al ser un protocolo orientado a datagramas. Los mensajes son y se tratan de forma independiente, y sin garantías. El programador debe decidir cuál es el tamaño de mensaje adecuado para el propósito concreto de la aplicación que está desarrollando y asegurarse de que no se truncan.

El cliente UDP mínimo (Listado 6.5) es aún más sencillo. Solo necesita enviar un mensaje y esperar la respuesta.

```
import socket

sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
sock.sendto(b'hola', ('127.0.0.1', 2000))
reply, client = sock.recvfrom(1024)
```

[5] «`bytes`» es el tipo Python para secuencias de bytes.

```
print(f"El servidor ha respondido: {reply}")
sock.close()
```

LISTADO 6.5: Cliente UDP mínimo

Este cliente envía un mensaje con el texto `'hola'` al servidor y espera a recibir una respuesta, que imprime en pantalla. Por supuesto, podría enviar y recibir varios mensajes, y podría usar ese mismo socket para comunicarse con otros servidores diferentes durante la misma ejecución.

La implementación de los sockets por parte del SO se está encargando de la mayor parte del trabajo que implica la construcción y reconocimiento de las cabeceras y el envío de esos datos a través de la red. Recuerda que los datagramas UDP se encapsulan en paquetes IP, que a su vez se encapsulan en alguna tecnología de enlace. Esos protocolos tienen sus propias cabeceras que contienen muchos detalles como las direcciones, el tamaño de los mensajes, checksums, etc. Gracias a los sockets, el programador no tiene que preocuparse de nada de todo eso. Lo único que tiene que especificar en el cliente es el endpoint del servidor y los datos a enviar. Del resto se encarga el SO.

El diagrama de la Figura 6.2 muestra la estructura y esquema de iteración típico de un cliente y un servidor UDP básicos.

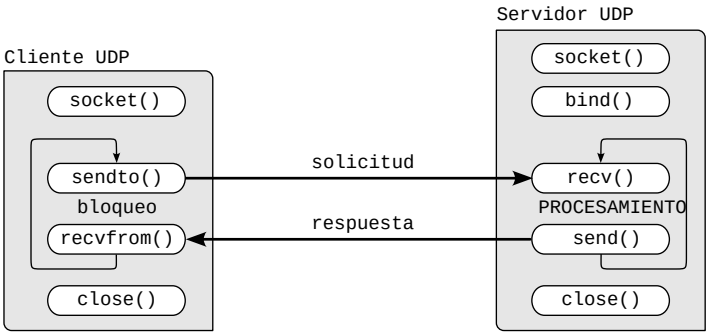


FIGURA 6.2: Diagrama de interacción de un cliente y un servidor UDP

La única diferencia significativa respecto a los ejemplos mínimos anteriores es que representa bucles para el par de invocaciones `sendto()`/`recvfrom()` y su contraparte en el servidor, que encaja con el comportamiento muy habitual en aplicaciones reales.

También es interesante mencionar que en el servidor, después de la recepción de un mensaje lo habitual es realizar algún tipo de procesamiento o acceso/modificación de un recurso antes de devolver un resultado o un códi-

go que indique el resultado de la operación. Mientras eso ocurre, lo habitual es que es cliente quede bloqueado en espera de la respuesta.

6.5. Servidor TCP

TCP es un protocolo mucho más complejo que UDP. El intercambio de datos no es posible si no se realiza primero un proceso de conexión exitoso. El protocolo garantiza que todos los datos llegan al otro extremo, sin errores, en el orden correcto, sin partes ausentes ni duplicadas.

Aunque se utiliza la misma clase `socket` y se usan métodos similares, o incluso los mismos que con UDP, el modo en que funcionan y la forma en que logran su objetivo es significativamente diferente. Por eso es importante conocer esas diferencias para evitar confusiones y errores.

El servidor TCP mínimo (Listado 6.6) suele tener dos partes bien diferenciadas: la primera parte se llama *acceptor* y se encarga de atender nuevas conexiones —el bucle `while` con la llamada `accept()`— y la segunda parte (el manejador) se encarga de gestionar la conversación con un cliente concreto con la función `handle()`. Esta estructura se suele respetar incluso en servidores mucho más complejos.

```
1  import socket
2
3  def handle(conn):
4      data = conn.recv(1024)
5      print(f"Se ha recibido el mensaje '{data}'")
6      conn.send(f"Enviaste {len(data)} bytes".encode())
7      conn.close()
8
9  sock = socket.socket()
10 sock.bind(('', 2000))
11 sock.listen(5)
12
13 while 1:
14     conn, client = sock.accept()
15     handle(conn)
```

Listado 6.6: Servidor TCP mínimo

Además de la llamada a `bind()`, que funciona igual que con UDP, con TCP es necesario llamar también a `listen()`. Este método le pide al SO que ponga el socket a la escucha. Su argumento fija el tamaño del *backlog*, que es una cola en la que se insertan las conexiones en espera de ser aceptadas. Una cola de tamaño 5, como la del ejemplo, significa que podrá haber un máximo de 5 clientes a la espera. Si un sexto cliente intenta conectar, será rechazado.

Al socket `sock`, sobre el que se invoca `bind()` y `listen()`, se le llama *listening socket* o *master socket*, y solo sirve para eso: aceptar nuevas conexiones. No se puede utilizar para enviar o recibir datos.

Para aceptar conexiones (**línea 14**) se invoca el método `accept()`. El proceso queda bloqueado en ese punto hasta que un cliente inicie una conexión. Cuando eso ocurre, el método retorna una tupla con dos elementos: un nuevo socket para intercambiar mensajes con ese cliente (`conn`) y el endpoint de ese cliente (`client`).

Una vez establecida la conexión, el bucle llama a `handle()` pasando como argumento el socket `conn`. Esta función se encarga de manejar la conexión con ese cliente hasta su desconexión.

En este servidor tan simple, la función `handle()` recibe datos del cliente —con el método `recv()`—, los imprime en pantalla e invoca `send()` para enviar una respuesta indicando la longitud[6]. Solo hay un mensaje de petición y uno de respuesta, pero como en el esquema de UDP, habitualmente la comunicación suele implicar varios mensajes. Para acabar cierra la conexión.

Una vez termina la función `handle()`, el bucle `while` llama de nuevo a `accept()` y espera al siguiente cliente. Se trata de un bucle infinito, que es lo habitual en un servidor. En una versión en producción, el servidor tendría algún mecanismo para terminar de forma ordenada, habitualmente mediante un manejador de señal.

Es un programa sencillo, y todo parece correcto, salvo que... **NO LO ES**. Este código está pasando por alto la forma en que funciona realmente la comunicación TCP. Volveremos sobre el asunto enseguida, después de echar un vistazo al cliente.

6.6. Cliente TCP

La estructura del cliente TCP para enviar un único mensaje al servidor y recibir una respuesta es muy similar al cliente UDP. Se muestra en el Listado 6.7.

```
import socket

sock = socket.socket()
sock.connect(('127.0.0.1', 2000))

sock.send(b'hola')
reply = sock.recv(1024)
```

[6] como en el ejemplo de UDP

```
print(f"El servidor ha respondido: '{reply}'")
sock.close()
```

Listado 6.7: Cliente TCP mínimo

La diferencia más evidente es que el cliente TCP invoca `connect()` para establecer la conexión, que es la contraparte de `accept()` en el servidor. También es una llamada bloqueante, el proceso queda bloqueado hasta que el servidor acepta la petición y la conexión queda establecida. A partir de ese momento, puede comenzar el intercambio de mensajes, y, a diferencia del servidor, todo se hace con el mismo socket.

Este esquema de interacción entre cliente y servidor TCP está bien representado en la figura 6.3. Como en el caso de UDP, representa una situación más general en la que cliente y el servidor intercambian varios mensajes. Por lo demás, corresponde con la idea básica de los programas cliente y servidor anteriores.

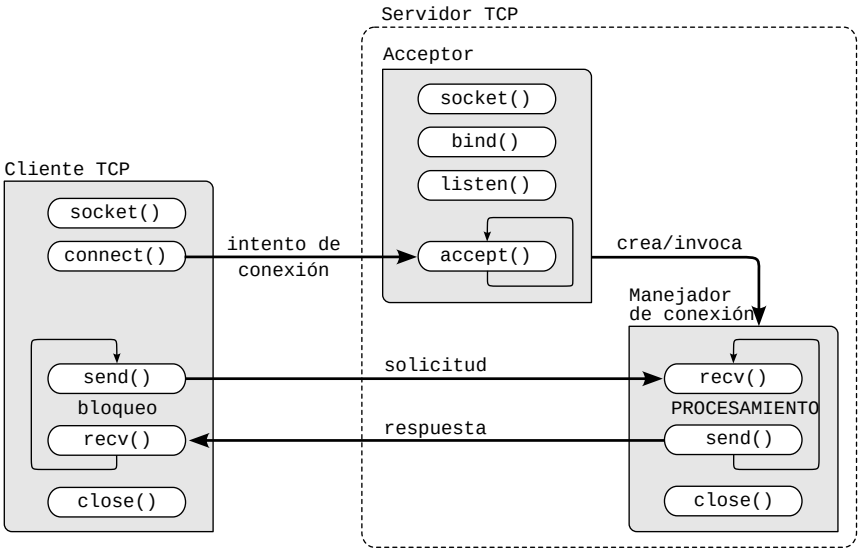


Figura 6.3: Diagrama de interacción de un cliente y un servidor TCP

Las dos cajas grises en el lado del servidor representan los dos elementos: el acceptor y el manejador de la conexión. En el Listado 6.6 el acceptor invoca directamente la función `handle`, pero como veremos más adelante en el capítulo 15, es muy habitual que esa función se ejecute en un nuevo hilo o proceso, como forma de conseguir un mayor rendimiento.

6.7. Flujos de datos y E/S parcial

En el servidor y en el cliente TCP hemos pasado por alto —a propósito— un detalle muy importante. Las primitivas `send()` y `recv()` no funcionan como en principio cabría esperar. Desde luego, distan bastante de cómo lo hacen `sendto()` y `recvfrom()` de UDP.

Por extraño que pueda sonar, invocar `send(datos)` de TCP no garantiza ni mucho menos que se vaya a enviar un segmento cuya carga útil sean exactamente esos `datos`. Aunque en condiciones de baja/media carga es probable que sí ocurra, habrá situaciones en las que varias llamadas a `send()` impliquen el envío de un único segmento (o ninguno), o en las que un único `send()` conlleve el envío de varios segmentos; a fin de cuentas, no hay correspondencia alguna entre los límites de los datos que se pasan a `send()` y la carga útil de los segmentos que se envían a la red. Aunque la probabilidad de que se den estas situaciones sea relativamente baja, el programa debe estar preparado para funcionar correctamente cuando ocurran o fallará, y eso pasa por entender exactamente cómo funcionan estas primitivas. Y sí, eso también implica que el código de los Listados 6.6 y 6.7 está **MAL**.

6.7.1. Envío TCP

La comunicación TCP trata los datos como un flujo *(stream)*. Los datos que el emisor pasa como argumento a `send()` no se envían inmediatamente a la red como ocurre en UDP. En su lugar, van a parar a un buffer de envío *(sending buffer)* bajo el control del SO, y cuando él lo considere oportuno, construirá un segmento e incluirá en él la parte de ese buffer que estime adecuada. Hay varias variables y condicionantes no triviales que influyen en estas decisiones y que veremos en próximos capítulos.

Además de la incertidumbre que conlleva para el programador el mecanismo de construcción y envío de segmentos, ocurre que el método `send()` funciona de un modo peculiar (o no tanto). En realidad, es muy similar a la llamada al sistema[7] `write()`^{sc} y sufre del mismo efecto, llamado «E/S parcial» *(partial I/O)*. de POSIX Implica que, en el momento de la invocación, el SO no asegura que pueda aceptar el bloque de datos completo que se le pasa, sino solo una parte. Puede que en ese momento el driver de dispositivo no esté listo o no tenga suficiente espacio en su buffer interno, entre otros motivos. Por esa razón, el valor de retorno de `send()`, la función `os.write()` de Python o la llamada al sistema `write()`^{sc} devuelven un entero que indica cuántos bytes pudieron escribir realmente. En realidad,

[7] Puedes consultar la documentación de `write()` con `man 2 write`. El '2' es porque se encuentra en la Sección 2: «Llamadas al sistema» del manual de POSIX.

`os.write()` es un envoltorio Python para `write()`[sc] así que es lógico que tenga un comportamiento análogo.

La consecuencia directa de la E/S parcial es que potencialmente son necesarias varias llamadas para asegurar que se ha enviado todo el bloque de datos. Por suerte, la solución a este problema es sencilla, al menos para envío/escritura. En un bucle se puede comprobar si quedan datos por enviar, y de ser así, invocar de nuevo `send()` con los datos que «no entraron». El bucle termina cuando el SO acepta por fin todos los datos que se pretendían enviar. Este patrón de envío es tan común que Python lo ofrece como otro método de la clase `socket` llamado `sendall()`. El Listado 6.8 podría ser una implementación simplificada a efectos explicativos.

```
def sendall(sock, data):
    sent = 0
    while sent < len(data):
        sent += sock.send(data[sent:])
```

Listado 6.8: implementación simplificada de `sendall()`

Una solución similar la aplican muchos lenguajes de programación cuando se escribe en archivos convencionales almacenados en disco[8]. Así, el método *file*.`write()`[9] de las clases para manipulación de archivos de muchos lenguajes internamente efectúan varias invocaciones a `os.write()` hasta conseguir escribir todos los datos.

6.7.2. Recepción TCP

La recepción TCP funciona de forma análoga. Cuando se invoca `recv()`, los límites de los datos que se obtienen no tienen por qué coincidir con los segmentos recibidos y menos aún con lo que el emisor colocó en cada llamada a `send()`. También pueden ser necesarias varias llamadas a `recv()` para obtener todos los datos esperados, del mismo modo que pueden ser necesarias varias invocaciones a `os.read()` para leer la cantidad deseada de bytes desde un archivo. Por eso, el método `recv()`, la función `os.read()` y `read()`[sc] devuelven la cantidad de bytes que realmente se han recibido/leído. Como antes, no pierdas de vista que los métodos tipo *file*.`read()` de las clases de manejo de archivos internamente pueden realizar varias llamadas a `os.read()` para conseguir leer la cantidad de datos que se le solicita.

[8] Utilizamos la expresión 'en disco' como una forma común para referirnos al almacenamiento secundario, a pesar de que la tecnología actual no involucra discos.

[9] Usamos *file* para referirnos en genérico a clases para manipulación de archivos. Python llama a esto *file object*, aunque no existe ningún tipo `file`.

Lamentablemente, no hay una solución genérica para implementar un hipotético método `recvall()`, como contraparte de `sendall()`, para resolver esta situación.

El argumento de `recv()` no es de mucha ayuda en esto. Poner un número muy grande no garantiza en absoluto que vaya a obtener el mensaje completo, porque simplemente los datos esperados pueden no haber llegado aún. Como en el caso de UDP, ese argumento simplemente le dice al SO cuanto espacio como máximo estás dispuesto a utilizar para almacenar esos datos. A diferencia de UDP, si los datos recibidos no caben en ese espacio, no se pierden, sino que se quedan en el buffer de recepción *(receiving buffer)* a la espera de que invoques `recv()` de nuevo.

Lo importante aquí es que es responsabilidad del programador delimitar los mensajes y decidir hasta cuando debe seguir llamando a `recv()` y eso no siempre es sencillo. Depende completamente del objetivo y finalidad de la aplicación, o siendo más precisos, del protocolo de aplicación. Si la aplicación envía mensajes de una longitud fija, es sencillo, solo hay que recibir hasta conseguir esa cantidad de bytes. Pero si los mensajes son de longitud variable o impredecible, la cosa se complica.

En nuestro pequeño servicio contador de caracteres (Listado 6.6), el objetivo del servidor es informar de la longitud del mensaje que recibe, y obviamente, es desconocido para él. Eso significa que el servidor no puede saber cuántas veces debe llamar a `recv()` para obtener el mensaje completo. Al cliente le pasa algo parecido cuando recibe: no puede saber qué tamaño tendrá el mensaje de respuesta.

El problema de fondo es que el programador diseña su aplicación para intercambiar mensajes que tienen sentido para el objetivo del programa. Algo como:

```
cliente: "hola"
servidor: "Enviaste 4 bytes"
```

Pero TCP no entiende nada de eso. Para TCP todo es una única secuencia de bytes que abarca toda la conexión y que tiene que llevar de un lado al otro del modo más eficiente posible. Para TCP no hay mensajes individuales, solo un flujo continuo de datos en cada sentido.

Para reconciliar ambos enfoques aparentemente enfrentados, el programador debe incluir algo en el mensaje que le permita, en el otro extremo, saber cuándo ha recibido un mensaje completo (desde el punto de vista de la aplicación). Y es crítico, porque si no recibe suficiente, el mensaje estará incompleto; y si intenta recibir demasiado, el proceso quedará bloqueado porque no hay más datos en ese momento.

framing

El proceso que determina el comienzo y fin de cada mensaje dentro de un flujo continuo de datos en función de la lógica propia de cada aplicación específica se denomina *framing* en inglés, y se puede traducir como *delimitación de mensajes*.

6.7.3. Bandera de fin de mensaje

Para el propósito del contador de caracteres, una posible solución sería incluir un *token* especial al final del mensaje a modo de bandera. Por ejemplo, tanto cliente como servidor podrían enviar un carácter de nueva línea (`\n`) al final de cada mensaje y el receptor simplemente tiene que seguir recibiendo hasta que aparezca. El Listado 6.9 muestra una versión corregida del cliente, que aplica esta solución, y utiliza `sendall()` para el envío.

```python
import socket

def read_message(conn):
    data = bytes()
    while (i := data.find(b'\n')) == -1:
        chunk = conn.recv(1024)
        if not chunk:
            return data
        data += chunk
    line = data[:i]
    return line

sock = socket.socket()
sock.connect(('127.0.0.1', 2000))

sock.sendall(b'hola\n')
reply = read_message(sock)

print(f"El servidor ha respondido: {reply.decode()}")
sock.close()
```

Listado 6.9: Cliente TCP recibiendo un mensaje con bandera de fin /socket/tcp-client-flag.py

Del otro lado, el Listado 6.10 muestra una versión corregida del servidor. Este tiene en cuenta la E/S parcial, incluye la bandera de fin de mensaje y el manejador de señal para la finalización adecuada. Con esto, si pulsas `Ctrl+C` en la consola en la que se está ejecutando el servidor, la shell enviará al proceso la señal SIGINT y el servidor ejecutará la función `int_handler()` que se encarga de cerrar el socket y terminar ordenadamente el proceso.

Aunque añadir este tipo de manejador es ciertamente una práctica muy conveniente en código de producción, por legibilidad no los incluiremos en los ejemplos del libro.

```python
import socket
import signal

def int_handler(sig, frame):
    print("\nSIGINT recibido. Cerrando socket y saliendo...")
    sock.close()
    exit(0)

def read_message(conn):
    data = bytes()
    while (i := data.find(b'\n')) == -1:
        chunk = conn.recv(1024)
        if not chunk:
            return data
        data += chunk
    line = data[:i]
    return line

def handle(conn):
    line = read_message(conn)
    print(f"Se ha recibido el mensaje: {line}")
    conn.sendall(f"Enviaste {len(line)} bytes\n".encode())
    conn.close()

sock = socket.socket()
sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
sock.bind(('', 2000))
sock.listen(5)
signal.signal(signal.SIGINT, int_handler)

while True:
    try:
        conn, client = sock.accept()
        handle(conn)
    except OSError:
        break
```

LISTADO 6.10: Servidor TCP recibiendo un mensaje con bandera de fin
/socket/tcp-server-flag.py

La función `read_message()` que busca la bandera de fin de mensaje es frágil porque podría leer más de lo debido y provocar pérdida de datos, pero de momento sirve para ilustrar este ejemplo trivial. Más adelante en este mismo capítulo veremos una solución más robusta usando el método *file*`.readline()`.

6.7.4. Tamaño del mensaje en la cabecera

Otra solución más flexible y robusta es incluir la longitud del mensaje en su cabecera. De ese modo no se necesita elegir un carácter especial

como bandera. Esta solución es de las más habituales en los protocolos de aplicación (*p. ej.* HTTP).

Hasta este momento ni siquiera había una cabecera en nuestro ejemplo del contador de caracteres, solo se enviaba la carga útil y, con la solución anterior, la bandera de fin. Como este es un protocolo[10] muy simple, puede tener una cabecera también muy simple. El mensaje tendrá el formato `<longitud>:<datos>` siendo `<longitud>` un número en decimal que indica la cantidad de bytes que mide el resto del mensaje, *p. ej.* `17:you all everybody`. La cabecera está formada solo por `<longitud>:` y el resto es cuerpo[11].

El Listado 6.11 muestra un fragmento de código en el que se realiza la recepción conforme a ese formato de mensaje. Fíjate que en este caso no hay problema en que el carácter ':' aparezca en el cuerpo del mensaje, solo el primero funciona como separador entre cabecera y cuerpo.

```
size = bytes()
while b':' not in reply:
    size += sock.recv(1)

size = int(size[:-1])

body = bytes()
while len(body) < size:
        body += sock.recv(size - len(body))
```

Listado 6.11: Recepción TCP con una mensaje con formato `<longitud>:<datos>`

Tanto esta solución con longitud en la cabecera como la bandera de fin en realidad son soluciones sencillas. Pero date cuenta que el verdadero problema es que aunque estas puedan ser soluciones razonables para este caso, pueden no ser adecuadas en otras situaciones. ¿Qué ocurre si el carácter que hemos elegido como bandera forma parte del mensaje? ¿Y si no es aceptable añadir nada al mensaje? ¿Y si la cabecera debe ser de tamaño fijo o es crítico mantener el tamaño del mensaje al mínimo? Este tipo de problemas son comunes en la programación de redes y es lo que justifica la necesidad de protocolos de aplicación y la razón por la que hay tantos y tan variados.

6.8. Sockets como archivos

La mayoría de los métodos de la clase `socket` que hemos estado usando son «envoltorios» para las llamadas al sistema `socket()`[sc], `connect()`[sc], `send()`[sc]

[10] Sí, técnicamente estamos diseñando un protocolo de aplicación.

[11] Ya dije que sería una cabecera simple.

cuyos prototipos aparecen en `<sys/socket.h>` de la librería estándar del lenguaje C.

No es casualidad que hayamos estado haciendo analogías entre sockets y archivos. En los sistemas POSIX se aplica un principio que reza «todo es un archivo» ya sea un dispositivo de caracteres o de bloques (un disco), tubería, terminal, zona de memoria, entrada y salida estándar, etc. Por supuesto, los sockets deben considerar muchas situaciones que no ocurren con archivos convencionales y por eso `recv`/`send` disponen de un argumento opcional `flags` que permite modificar su comportamiento. Pero, en la mayoría de situaciones las primitivas `read`/`write` son equivalentes a `recv`/`send` y, de hecho, cuando se programa en C, lo son.

En Python no existen los métodos `socket.read()`/`socket.write()`, pero si hay dos opciones que permiten tratar a los sockets como archivos en caso de que convenga.

El método `socket.fileno()` devuelve el descriptor de archivo (un entero) asociado al socket. Con este descriptor puedes usar las primitivas `os.read()` y `os.write()` para recibir y enviar datos. Esto no cambia la forma de trabajar con el socket porque estas funciones también aplican la E/S parcial. Sin embargo, disponer del descriptor del archivo te da la posibilidad de pasar el descriptor a un subproceso o de utilizar código de librería que no está pensado para sockets, pero acepta descriptores de archivo.

```
>>> sock.fileno()
12
>>> os.write(sock.fileno(), b'hello')
```

La otra opción es `socket.makefile()`. Este método crea un objeto de estilo archivo (un *file object*) para manipular el socket:

```
>>> sock.makefile('r')
<_io.TextIOWrapper name=5 mode='r' encoding='UTF-8'>
```

El método `sock.makefile()` tiene parámetros similares a los de la función `open()`.

- `mode`: `'r'`, `'w'`, `'rb'`, `'wb'`, etc.
- `buffering`: tamaño del buffer, o `0` sin buffer.
- `encoding`: Para hacer (de)codificación automática (si se ha indicado modo texto).
- `newline`: Para indicar qué carácter se usa como salto de línea (consulta `open()` en la documentación de Python).

Y como archivo de alto nivel, sus métodos tienen el comportamiento habitual, es decir, ocupándose de la E/S parcial haciendo varias llamadas a `os.read()`/`os.write()` si es necesario.

- *file*.`read(size)` lee `size` bytes.
- *file*.`readline()` lee hasta que reciba el carácter `newline` indicado en `makefile()`.
- *file*.`readlines()` lee todas las líneas hasta el final de la conexión.
- *file*.`write()` envía `data` completo, que es equivalente a `socket.sendall()`.
- *file*.`flush()` fuerza la escritura (envío) de los datos pendientes.

Pero recuerda que para que todo funcione, el protocolo de la aplicación que estás creando debe asegurar que es factible que lo que esperas recibir (sea por cantidad o formato) puede llegar eventualmente sin producir un bloqueo.

Al tener disponible el método `readline()`, puedes implementar una versión más robusta de la recepción del mensaje con bandera de fin (§ 6.7.3).

```
import socket

sock = socket.socket()
sock.connect(('127.0.0.1', 2000))

sock.sendall(b'hola\n')
reply = sock.makefile().readline()

print(f"El servidor ha respondido: {reply}")
sock.close()
```

LISTADO 6.12: Cliente TCP recibiendo un mensaje con bandera de fin usando *file*.`readline()`

Aparte de ahorrarnos el escribir la función `read_message()`, se encarga de convertir a texto (`reply` es una cadena ya decodificada) y si quedan datos después del salto de línea lo guarda para la siguiente invocación, algo que no hacía nuestra función.

6.9. Gestión explícita de buffers

Como corresponde a un lenguaje dinámico como Python, la memoria necesaria para los buffers de las llamadas a `send()`, `recv()` y otras se reserva y libera automáticamente. Un ejemplo: un emisor que envía a un receptor un archivo en bloques de 1024 bytes sobre un flujo TCP. Date cuenta que en este ejemplo no importa quién es el servidor y quién el cliente, la transferencia se puede hacer en ambos sentidos exactamente de la misma forma. En los siguientes fragmentos no se incluye la creación de los sockets, conexión o desconexión, solo la transferencia.

Emisor

```
with open('outgoing.data', 'rb') as f:
    while 1:
        chunk = f.read(1024)
        if not chunk:
            break
        sock.sendall(chunk)
```

```
with open('incoming.data', 'wb') as f:
    while 1:
        chunk = sock.recv(1024)
        if not chunk:
            break
        f.write(chunk)
```

FIGURA 6.4: Transferencia de un archivo creando buffers nuevos en cada iteración

En cada iteración de estos bucles, `read()` y `recv()` crean un buffer interno de 1024 bytes para realizar la correspondiente llamada al sistema. Después crean el buffer `chunk` con los datos que han conseguido leer/recibir. Repito: ¡en cada iteración! Esto puede suponer una ineficiencia nada despreciable, especialmente en plataformas con recursos limitados.

Tanto para *file*.`read()` como para `recv()`, el 1024 indica el tamaño del buffer que crean[12] y determina la cantidad máxima de bytes que pueden retornar. Sin embargo, no trabajan exactamente igual (§ 6.7). La llamada `f.read(1024)` bloquea hasta conseguir los 1024 bytes (o se acabe el fichero/flujo); la llamada `sock.recv(1024)` bloquea hasta que lleguen algunos datos, retorna incluso si dispone solo de un byte. Si en lugar de usar *file*.`read()`/*file*.`write()` usases `os.read()`/`os.write()`, los dos bloques de código tendrían una semántica análoga.

Sin embargo, es posible evitar tanta creación y destrucción de buffers. Python ofrece una alternativa que permite crear un único buffer y reutilizarlo en cada iteración del bucle mientras dure la transferencia. Quedaría algo así:

Emisor

```
buffer = bytearray(1024)
view = memoryview(buffer)

with open('outgoing.data', 'rb') as f:
    while 1:
        nbytes = f.readinto(buffer)
        if nbytes == 0:
            break
        sock.sendall(view[:nbytes])
```

Receptor

```
buffer = bytearray(1024)
view = memoryview(buffer)

with open('incoming.data', 'wb') as f:
    while True:
        nbytes = sock.recv_into(buffer)
        if nbytes == 0:
            break
        f.write(view[:nbytes])
```

FIGURA 6.5: Transferencia de un archivo reutilizando un único buffer

[12] La destrucción de todos esos buffers es tarea del recolector de basura.

El tipo `bytearray` es similar a `bytes`, pero mutable. Por eso se puede reutilizar en cada iteración para almacenar los datos procedentes del archivo en el envío o desde el socket en la recepción. Los métodos `readinto()` y `recv_into()`[13] son variantes diseñadas para este propósito.

El tipo `memoryview` permite acceder a una parte del buffer sin copiar nada, por lo que es mucho más eficiente que la primera alternativa en la que *file*.`read()` y `recv()` creaban buffers. La variable `nbytes` indica en ambos casos cuántos bytes se han podido obtener realmente. Como el propósito del código es enviar y recibir un archivo, realmente no supone ningún problema si `socket.recv_into()` no recupera la misma cantidad de datos en todas las iteraciones.

6.10. Fin de la comunicación TCP

En los listados anteriores, tanto el cliente como el servidor invocan `close()` para cerrar el socket, y con ello la conexión si es TCP. El receptor TCP necesita conocer esta situación. Cuando invoca `recv()`, si el otro extremo ya ha cerrado la conexión, el método devolverá una secuencia vacía. De ese modo el receptor puede terminar también de forma ordenada. Y si estaba en un bucle recibiendo mensajes sucesivos, puede interrumpirlo como corresponda.

El siguiente listado ilustra este comportamiento. Es un receptor que va concatenando los datos que va recibiendo en el buffer `reply` y sale del bucle al recibir una secuencia vacía (`if not data`).

```
reply = bytes()
while True:
    data = sock.recv(1024)
    if not data:
        break
    reply += data
```

El socket TCP dispone de una forma más fina que el método `close()` para controlar el cierre de la conexión: el método `shutdown()`. Este acepta 3 valores:

- `socket.SHUT_RD`: El proceso ya no va a invocar `recv()`. Eso le dice al SO que a partir de ese momento puede descartar cualquier mensaje entrante sin llegar a almacenarlo en el buffer. Podrá seguir enviando mensajes.

[13] También existe un `recvfrom_into()` para sockets de datagramas.

- `socket.SHUT_WR`: El proceso ya no va a invocar `send()`. Eso implica el envío de un mensaje con el flag FIN al otro extremo. Podrá seguir recibiendo mensajes.
- `socket.SHUT_RDWR`: Es la combinación de las dos anteriores.

Cuando el emisor invoca `shutdown(SHUT_WR)`, el receptor experimenta el mismo comportamiento descrito anteriormente: el método `recv()` devuelve una cadena vacía. Es decir, el receptor no puede distinguir si el emisor cerró con `SHUT_WR` o `close()`. En todo caso el proceso debería invocar `close()` para liberar el socket aunque haya invocado `shutdown()` previamente.

6.11. Manejo de errores

En los ejemplos anteriores —y en la mayoría de los que encontrarás en el libro— no se han incluido manejadores de errores y excepciones. La razón para esto es proporcionar listados más compactos y fáciles de explicar y entender. Sin embargo, en un código destinado a producción es fundamental tratar los errores de forma adecuada, porque de lo contrario el programa puede comportarse de forma errática o simplemente terminar abruptamente. Lógicamente, este tratamiento de errores es especialmente importante para los servidores, donde no puedes permitir que la operación incorrecta de un cliente provoque la caída del servidor.

La tabla 6.1 muestra los errores más habituales cuando se opera con sockets y conexiones.

Excepción	**Situación**
`socket.gaierror`	Error en la resolución de nombres.
`TimeoutError`	`connect()` llevó demasiado tiempo (incluso para sockets bloqueantes).
`BrokenPipeError`	Se invocó `send()` cuando el otro extremo ya había desconectado.
`ConnectionAbortedError`	La conexión se abortó durante la ejecución de `connect()` o `accept()` y no se llegó a completar.
`ConnectionResetError`	Se invocó `send()`/`recv()` cuando el otro extremo ya había cerrado de forma abrupta (RST en lugar de FIN).
`ConnectionRefusedError`	Se invocó `connect()` a un puerto cerrado.

CUADRO 6.1: Errores comunes en programación de sockets

Estos errores aparecen como excepciones que heredan de la clase `OSError` (Figura 6.13). Conocer la jerarquía es útil porque permite al programador decidir lo genérico o específico que quiere hacer el manejador.

```
Exception
 └── OSError (== socket.error)
      ├── socket.gaierror
      ├── TimeoutError
      └── ConnectionError
           ├── ConnectionAbortedError
           ├── ConnectionRefusedError
           ├── ConnectionResetError
           └── BrokenPipeError
```

Listado 6.13: Jerarquía de excepciones de `socket`

Por ejemplo, si capturas `ConnectionError`, eso incluye todas las causas de error de la conexión.

```
while 1:
    try:
        conn, client = sock.accept()
    except ConnectionError as e:
        print(f"Error de conexión: {e}")
        continue
```

Estas excepciones son «nombres propios» para números de error (`errno`) específicos de la excepción `OSError`, pero hay muchos otros valores de `errno` que no tienen un nombre específico. En esos casos, el programador puede capturar la excepción y comprobar el valor para determinar el problema concreto.

```
try:
    [...]
except OSError as e:
    if e.errno == errno.ECONNREFUSED:
        print("Conexión rechazada")
    elif e.errno == errno.EHOSTUNREACH:
        print("Host inalcanzable")
    elif e.errno == errno.ENETUNREACH:
        print("Red inalcanzable")
    else:
        print(f"Error desconocido: {e}")
```

Algunos de ellos, están disponibles como constantes en el módulo `errno`:

6.11.1. Context manager

Los sockets de Python se pueden usar como *context managers*, es decir, con la clausula `with`. Esto ayuda a gestionar automáticamente los recursos asociados, ahorrando al programador la invocación de `close()` y capturando excepciones comunes. El Listado 6.14 muestra un servidor para el servicio Echo que aprovecha esa característica. Tal como indica su nombre, el servidor Echo devuelve al cliente los mensajes que este le envía.

nombre	errno	descripción
EDESTADDRREQ	89	Se requiere dirección de destino.
EPROTONOSUPPORT	93	Protocolo no soportado.
EAFNOSUPPORT	97	Familia de direcciones no soportada.
EADDRNOTAVAIL	99	Dirección no disponible.
ENETDOWN	100	La red está caída.
ENETRESET	102	La conexión se ha perdido por un fallo de red.
EISCONN	106	El socket ya está conectado.
ENOTCONN	107	El socket no está conectado.
EHOSTUNREACH	113	No se puede determinar una ruta hacia el host.
ENETUNREACH	101	La red no es alcanzable.
ETIMEDOUT	110	El tiempo de conexión se ha agotado.
ECONNREFUSED	111	El destino rechaza el intento de conexión.
ECONNRESET	104	El destino ha cerrado la conexión abruptamente.

CUADRO 6.2: Códigos `errno` relacionados con errores de red

```python
import socket

def handle(conn):
    while 1:
        data = conn.recv(1024)
        if not data:
            break

        conn.sendall(data)

with socket.socket() as sock:
    sock.bind(('', 7))
    sock.listen(5)

    while 1:
        conn, client = sock.accept()
        with conn:
            handle(conn)
```

LISTADO 6.14: Servidor de Echo TCP con *context manager*

Una vez el servidor acepta una conexión, recibe mensajes del cliente y los devuelve con `sendall()`. Si `recv()` retorna una cadena vacía, significa que el cliente ha desconectado. Entonces el bucle `while` de `handle()` termina con `break` y el servidor espera la siguiente conexión. Es interesante destacar que para el caso concreto de Echo utilizar `recv()` sin más no es un problema. El servidor no necesita identificar límites en los datos que recibe. Simplemente devuelve lo que recibe. Da igual si eso ocurre en bloques pequeños o grandes, iguales o diferentes, y al cliente tampoco le va a afectar, porque todo es un único flujo.

6.12. Netcat

El programa `netcat` es, a pesar de su sencillez, una de las herramientas más potentes y versátiles que existen en el campo de la programación, análisis y manipulación de redes y servicios TCP/IP. Es un recurso imprescindible para profesionales de la programación, administración o gestión de redes, expertos en seguridad y hackers.

Existen muchas implementaciones de `netcat` desarrolladas por distintos fabricantes, con sus propias características, pero todas ellas comparten una misma funcionalidad básica y simple que se resume en:

- `netcat` crea un socket TCP o UDP, que puede ser cliente o servidor en función de los argumentos que se le indiquen.
- Una vez establecida la comunicación, envía por el socket todo lo que lea de su entrada estándar y a la vez, envía a su salida estándar todo lo que reciba por ese mismo socket.

Esto lo convierte en un cliente o servidor absolutamente genérico, que combinado con la potencia de la *redirección* que ofrece la `shell` (§ 2.6) permite proporcionar conectividad de red a cualquier programa que consuma y genere datos de su entrada y salida estándar, respectivamente. Por eso se podría decir que `netcat` es un «socket» para usar desde línea de comandos.

Para que todos los usos de `netcat` que se hacen a lo largo del libro sean homogéneos, vamos a usar siempre la implementación de `insecure.org`, que se llama `ncat` y está disponible en la mayoría de distribuciones GNU/Linux en el paquete del mismo nombre.

6.12.1. Sintaxis básica

Aunque `ncat` tiene muchas opciones y funcionalidad, en la gran mayoría de las situaciones se utilizan dos comandos básicos:

- Crear un servidor TCP: `ncat -l <puerto>`
- Crear un cliente TCP: `ncat <nodo> <puerto>`

El parámetro `-l` (*listen*) indica que se trata de un servidor. Para crear un servidor o un cliente UDP simplemente añade el argumento `-u` en ambos casos.

El siguiente es el uso más básico: funciona como una especie de chat tremendamente básico y permite a cualquiera de los dos extremos enviar y recibir en cualquier momento. Lógicamente, debes ejecutar primero el servidor y luego el cliente. Después puedes escribir en la entrada de cualquiera

de los dos y el texto aparecerá en el otro extremo. En este caso, ambos se estarán ejecutando en tu PC, pero por supuesto puede hacerse en nodos diferentes.

Cliente

```
$ ncat 127.0.0.1 2000
hola[ENTER]
```

Servidor

```
$ ncat -l 2000
hola
```

FIGURA 6.6: Uso básico de `ncat`: servidor y cliente TCP

Veremos unos cuantos ejemplos para ilustrar las muchas posibilidades de `ncat`, pero ten en cuenta que para casi todos existen programas específicos que pueden lograr el mismo objetivo con más posibilidades y, sobre todo, más seguridad.



Prueba esos ejemplos únicamente si sabes lo que estás haciendo. Algunos pueden suponer un problema para la integridad de tu sistema y tus datos.

Transferencia de archivos

Aplicando redirección puedes enviar un archivo (tanto texto como binario) del cliente al servidor y viceversa. Suponiendo que tienes el archivo `song.mp3` en el nodo cliente, puedes ejecutar lo siguiente:

Cliente

```
$ ncat sidney 2000 < song.mp3
```

Servidor

```
$ ncat -l 2000 > copia-song.mp3
```

FIGURA 6.7: Transfiriendo un archivo con `ncat`

Servidor Echo

`ncat` puede ejecutar cualquier programa conectando directamente el socket que crea con la entrada/salida de ese programa. Si el programa que ejecutas es `cat` conseguirás un comportamiento equivalente al servicio Echo, es decir, devolver al cliente todo lo que reciba de él:

Cliente

```
$ ncat sidney 2000
hola[ENTER]
hola
```

Servidor

```
$ ncat -l 2000 -e /bin/cat
```

FIGURA 6.8: Servidor Echo con `ncat`

Servidor *daytime*

También muy básico. Cuando el cliente conecta y sin enviar nada, el servidor que emula el servicio *daytime* le envía la fecha y hora actual y cierra la conexión. Para lograrlo, basta con ejecutar `date` en lugar de `cat`:

Cliente

```
$ ncat sidney 2000
jue 24 abr 2025 23:50:01 CEST
```

Servidor

```
$ ncat -l 2000 -e /bin/date
```

FIGURA 6.9: Servidor daytime con `ncat`

Lógicamente, puedes hacer lo mismo con cualquier programa que produzca una salida.

Shell remota

De forma similar a los ejemplos anteriores, si el programa que ejecutas con `ncat` es una shell, esta ejecutará los comandos que se le envíen y devolverá el resultado de la ejecución de esos comandos (Figura 6.10). Y para tener una *shell inversa* simplemente ejecuta la shell en el cliente (Figura 6.11).

Cliente

```
$ ncat sidney 2000
hostname[ENTER]
sidney
```

Servidor

```
$ ncat -l 2000 -e /bin/bash
```

FIGURA 6.10: Shell remota con `ncat`

Cliente

```
$ ncat sidney 2000 -e /bin/bash
```

Servidor

```
$ ncat -l 2000
hostname[ENTER]
sidney
```

FIGURA 6.11: Shell remota inversa con `ncat`

Desde luego esta no es la forma más conveniente de ejecutar una shell remota en un entorno de producción, para eso existe `ssh`. Sin embargo es habitual encontrar cosas parecidas en software malicioso. Se suele utilizar como *backdoor* y normalmente explota alguna vulnerabilidad que permite al atacante introducir comandos en la máquina vulnerable.

Streaming multimedia

Del mismo modo que se envía el contenido de un archivo para guardarlo en disco, se puede consumir directamente por el receptor. Si dispones de un programa que reproduce audio o vídeo desde su entrada estándar (como `mplayer`) puedes enviar un archivo compatible que se irá transfiriendo a medida que se consume.

Cliente

```
$ ncat sidney 2000 | mplayer -
```

Servidor

```
$ ncat -l 2000 < song.mp3
```

FIGURA 6.12: Streaming multimedia con `ncat`

Web

Puedes servir una página web con `ncat`, si bien un cliente estándar espera que envíes también la cabecera HTTP. Puedes usar este archivo `index.html` poco ortodoxo porque incluye la cabecera HTTP de respuesta:

```
HTTP/1.1 200 OK
Content-Type: text/html
Content-Length: 115

<!DOCTYPE html>
<html>
  <head><title>Ejemplo</title></head>
  <body>
  <h1>Hola desde ncat</h1>
  </body>
</html>
```

Lo puedes servir con el comando:

```
$ ncat -l 2000 < index.html
```

Y cargarlo con `firefox` o cualquier otro navegador web.

```
$ firefox http://127.0.0.1:2000
```

Fíjate que la consulta HTTP GET aparece en la salida del servidor.

```
GET / HTTP/1.1
Host: 127.0.0.1:2000
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: es-ES,es;q=0.8,en-US;q=0.5,en;q=0.3
Accept-Encoding: gzip, deflate, br, zstd
Connection: keep-alive
Upgrade-Insecure-Requests: 1
```

Proxy

Puedes utilizar `ncat` para redirigir una conexión entrante a otro servidor, es decir, otro puerto y/u otro nodo:

```
$ ncat -l 2000 -e "ncat example.net 3000"
```

El tráfico recibido en el puerto 2000 de este nodo se redirige al nodo `example.net`:3000. ¡Permite incluso que el flujo entrante sea UDP, pero la redirección sea a un servidor TCP o viceversa!

Medir el ancho de banda

En este caso, puedes usar un cliente que envía datos a la mayor velocidad posible —leyendo del dispositivo `/dev/zero`— y el servidor que los recibe los redirige hacia el programa `pv` que calcula la velocidad de llegada. Con esto puedes medir la velocidad máxima (el ancho de banda) entre ambos nodos.

Cliente

```
$ ncat sidney 2000 < /dev/zero
```

Servidor

```
$ ncat -l 2000 | pv > /dev/null
2,84GiB 0:00:04 [1,35GiB/s] [<=>     ]
```

FIGURA 6.13: Midiendo el ancho de banda con `ncat`

Imprimir un archivo PostScript

Este ejemplo funciona con impresoras que soportan el estándar AppSocket/JetDirect, que son la mayoría de las que se conectan por Ethernet o WIFI y, obviamente, debe estar habilitado en su configuración. El archivo debe estar en formato PostScript, pero lo puedes convertir fácilmente desde PDF con `pdf2ps` (`ghostscript`).

```
$ pdf2ps documento.pdf
$ cat documento.ps | ncat ip.de.la.impresora 9100
```

Y ¿qué más?

Este capítulo ha mostrado cómo los sockets proporcionan la base de la programación de redes en prácticamente cualquier sistema operativo y lenguaje de programación: cómo crear servidores y clientes UDP, un protocolo que proporciona comunicaciones sin conexión, rápidas, ligeras pero nada fiables; y servidores y clientes TCP, que proporcionan un flujo de datos orientado a conexión, con entrega garantizada y corrección de errores, in-

cidiendo también en la importancia de la multiplexación y los puertos, o las implicaciones de la entrada/salida parcial y la gestión eficiente de los buffers.

Los sockets son en realidad solo el punto de partida. Hay muchos temas pendientes, que cubriremos en los siguientes capítulos, y que también son fundamentales para entender y construir aplicaciones de red: los modelos cliente-servidor y publicación-suscripción, la serialización de datos, el diseño de protocolos de aplicación, la gestión eficiente de múltiples conexiones concurrentes, etc. Además, conocer los sockets te va a ayudar a entender los detalles del funcionamiento del control de flujo, de errores, el control de congestión, etc., que veremos en los siguientes capítulos.

También la seguridad es un tema fundamental, y muchos de los ataques se aprovechan de errores relacionados con la programación de sockets o de su uso incorrecto por parte del programador.

Capítulo 7

Direccionamiento IP

Al terminar este capítulo, entenderás:

- Qué es una dirección IP y cuál es su formato.
- Para qué sirve la máscara de red.
- Qué es el direccionamiento con clases y sus limitaciones.
- Qué es el direccionamiento sin clases y la técnica VLSM.
- Cómo calcular el direccionamiento para una topología dada.

Una dirección IP[1] es un identificador único asignado a cada nodo de la interred. Se trata de un número entero de 32 bits, por lo que teóricamente el espacio de direcciones es de 2^{32}, 4 «gibidirecciones»[2] o 4.294.967.296 direcciones. Aunque parezcan muchas —y ciertamente lo son— por varias razones que veremos, no todas se pueden asignar a un nodo.

A diferencia de las direcciones físicas, las direcciones IP tienen una estructura jerárquica. Como norma general, se consideran dos partes[3]: la primera es el prefijo de red *(network ID)* e identifica la red a la que pertenece el dispositivo. La segunda es el identificador de nodo *(host ID)* y sirve para eso, indica qué nodo es dentro de esa red. La Figura 7.1 muestra el formato de la dirección IP.

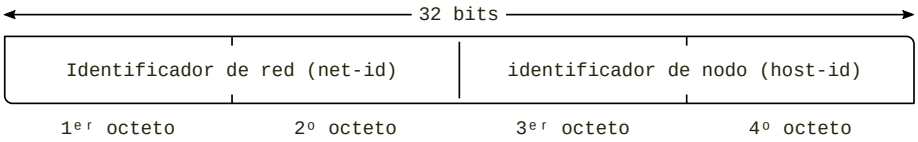


Figura 7.1: Formato de la dirección IP

[1] Este capítulo aborda únicamente IPv4.
[2] Nos tomamos la licencia de usar el prefijo «gibi» del SI para 2^{30}.
[3] Más adelante veremos que puede tener más.

El prefijo de red es imprescindible para que los routers puedan hacer su trabajo. Esencialmente la función de un router es decidir por cual de sus interfaces debe enviar cada paquete que recibe. Para eso, dispone de una tabla (la de encaminamiento) en la que busca cuál de sus filas encaja con la dirección IP destino que aparece en el paquete. El prefijo de red permite que la tabla pueda incluir las direcciones de las redes conocidas, en lugar de tener que incluir todos los posibles nodos destino. De ese modo las tablas pueden tener unas decenas de filas en lugar de cientos o miles.

La dirección de un nodo no se puede elegir arbitrariamente, tiene que respetar obligatoriamente el prefijo de la red (o subred) a la que está conectado. Lo único que puede variar de un nodo a otro es el identificador de nodo, y tampoco en eso hay total libertad. Quedan excluidos los valores que corresponden a la dirección de red y a la de broadcast, y obviamente también los que ya estén asignados a otros nodos.

7.1. Máscara de red

Para determinar qué parte de la dirección corresponde al prefijo de red y cuál al identificador de nodo se necesita la **máscara de red**. Es simplemente otro número de 32 bits que, en binario, se ve como una secuencia de unos seguida de una secuencia de ceros, aunque normalmente se representa en notación decimal, como una dirección IP. Así, una máscara formada por 16 unos seguidos de 16 ceros, en decimal sería `255.255.0.0`.

Para obtener el prefijo de red se realiza una operación `AND` a nivel de bits entre la dirección y la máscara. En el ejemplo, dada la dirección `150.20.45.76`, y la máscara `255.255.0.0`, el resultado que se obtiene al hacer el `AND` es `150.20.0.0`. El prefijo de red es `150.20` (la parte que ha «superado» la máscara), y el identificador de dispositivo es `45.76` (la parte eliminada por la máscara) (ver Cuadro 7.1). La dirección 150.20.0.0 (la que tiene todo ceros en la parte de identificador de nodo) representa a la red completa y por eso se conoce como **dirección de red**.

También es importante la **dirección broadcast**. Representa a todos los dispositivos de la red, y se obtiene también a partir del prefijo de red, pero con todos los bits restantes a 1, en lugar de 0. En el ejemplo, la dirección de broadcast de la red `150.20.0.0` para la máscara aplicada es `150.20.255.255`.

Como en total son 32 bits, cuando necesitamos una red grande con muchos nodos, el prefijo ea corto, mientras que si es una red pequeña, el prefijo es largo. Esto también implica que si creamos demasiadas redes grandes, quedará poco espacio para las pequeñas, y viceversa.

	decimal	1er octeto	2° octeto	3er octeto	4° octeto
Dirección	150.20.45.76	10010110	00010100	00101101	01001100
Máscara	255.255.0.0	11111111	11111111	00000000	00000000
Dir. de red	150.20.0.0	10010110	00010100	00000000	00000000
Dir. broadcast	150.20.255.255	10010110	00010100	11111111	11111111

CUADRO 7.1: Ejemplo de uso de la máscara de red

En todo caso, una organización cualquiera no puede decidir por su cuenta qué prefijo usar para su red. Para hacer un reparto organizado y evitar conflictos, existe la ICANN y sus delegados regionales (las RIR) que se encargan de asignar —previo pago, por supuesto— estos prefijos de red a las organizaciones que los soliciten.

Pero, ¿cómo se sabe cuál es la máscara de red que hay que aplicar? Hay dos respuestas a esta pregunta: la que se definió originalmente (que llamamos «direccionamiento con clases») y la que se utiliza en la actualidad (que llamamos «direccionamiento sin clases»).

7.2. Direccionamiento con clases

Para hacer un reparto racional entre redes grandes y pequeñas, la IETF originalmente definió un sistema que llamamos direccionamiento con clases o *classful addressing* [9] Divide el espacio de direcciones en cinco clases nombradas desde la A a la E, aunque únicamente las clases A, B y C se utilizan para direccionar nodos (unicast).

clase	prefijo (bits)	máscara	# redes	direcciones por red
A	0	`255.0.0.0`	128	16 777 216
B	10	`255.255.0.0`	16 384	65 536
C	110	`255.255.255.0`	2 097 152	256

CUADRO 7.2: Direccionamiento con clases

Con este sistema, dada una dirección IP se puede determinar directamente a qué clase pertenece, pero también mucha otra información útil. Por ejemplo, con la dirección `161.67.21.13`:

- Es una dirección de clase B porque los dos primeros bits de 161 son `10`.
- Por ser de clase B, le corresponde una máscara `255.255.0.0`.
- Su dirección de red es `161.67.0.0`.
- Su dirección broadcast es `161.67.255.255`.
- Su rango de direcciones asignables va de `161.67.0.1` a `161.67.255.254`, es decir, $2^{16} - 2$ direcciones.

Hablamos de «direcciones asignables» precisamente porque, ni la dirección de red ni la de broadcast, se pueden asignar a una interfaz de red, y por eso se descuentan del total.

7.2.1. *Subnetting*

El direccionamiento con clases tiene varios problemas. Hay relativamente pocas redes de clase A y B, porque son demasiado grandes (sobre todo las clase A), y las de clase C son demasiado pequeñas, más a día de hoy. Esto provocaba un gran desperdicio de direcciones porque muchas organizaciones que solicitaron bloques clase A y B hace años solo utilizan una mínima parte, pero nadie más las puede aprovechar.

Para tratar de resolver esto, se desarrolló el concepto de *subnetting* o más formalmente FLSM. Consiste en dividir una red cualquiera en subredes más pequeñas. Estas nuevas subredes son útiles para la misma organización que ya poseía la red original. Desde un punto de vista administrativo, permite a la organización delegar la gestión de cada subred al departamento o unidad a la que se le asigna. Eso facilita la gestión de los servicios, visibilidad hacia el exterior y privilegios de acceso específicos de cada subred, adaptándose a las necesidades, restricciones y políticas de cada departamento.

Para crear subredes, se toman bits de la parte del identificador de host y se utilizan para direccionar las subredes. A este nuevo campo se le llama «prefijo de subred» o *subnet ID* (consulta la Figura 7.2).

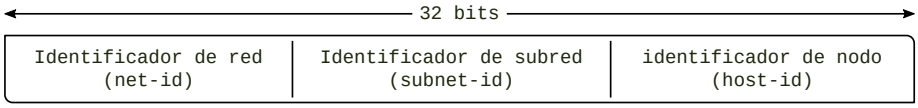


FIGURA 7.2: Formato de la dirección IP con *subnetting*

Si se toman n bits para el prefijo de subred, se obtendrán 2^n subredes. Siempre que se aplica *subnetting* se obtiene una potencia de dos de subredes. Eso significa que si, por ejemplo, se necesitan 3 subredes, habrá que crear al menos 4 y una de ellas quedará sin uso. Obviamente, tomar bits del identificador de host para el de subred, implica que las subredes resultantes serán más pequeñas, es decir, habrá menos direcciones para asignar a nodos.

Lo interesante del *subnetting* es que se aplica solo desde el punto de vista de la organización. Desde fuera, para el resto de Internet, el conjunto se sigue viendo como la red original completa. Nadie fuera de esa red necesita

saber que se ha aplicado subnetting y mucho menos cómo se ha hecho, en cuantas subredes se ha dividido, etc.

Inicialmente, el primer y el último bloque obtenidos al dividir la red causaban problemas de encaminamiento, ya que esas direcciones podían confundirse con la dirección de red y de broadcast de la red original. Por ello se prohibía su uso: son los conocidos *subnet-zero* y *subnet-all-ones*. Es una limitación muy grave, ya que en el caso de usar pocos bits para el prefijo de subred, se perdían muchas direcciones. En el ejemplo anterior implicaría perder la mitad del espacio de direcciones original. Más tarde la RFC 1878 [12] especificó la forma de evitar esta limitación. A día de hoy la mayoría de equipos de red y sistemas operativos la soportan sin problema y no existe ese desperdicio.

7.2.2. Ejemplo de *subnetting*

Supongamos que una organización dispone del bloque `70.50.0.0/16` y necesita dar servicio a la topología de la Figura 7.3: tres redes con distintas necesidades (A con 5000 nodos, B con 2000 y C con 1000) más tres enlaces serie punto a punto entre routers.

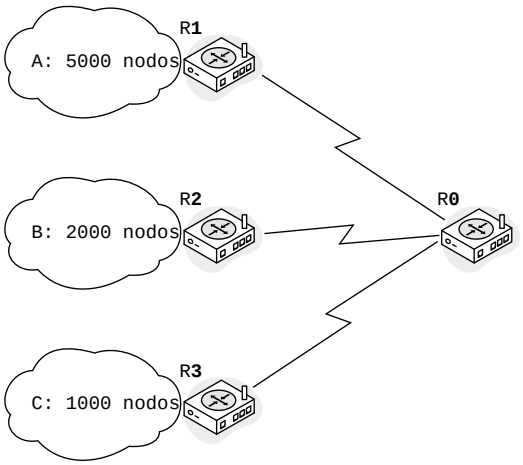


FIGURA 7.3: Topología de ejemplo para *subnetting* y VLSM

En *subnetting* todas las subredes deben tener el mismo tamaño. Por eso el bloque tiene que ser lo bastante grande como para cubrir la mayor: la red A con 5000 nodos (5003 direcciones contando red, broadcast e interfaz de `R1`). Eso requiere 13 bits de *host-ID* (máscara `/19`). Además, direccionar las 6 redes (A, B, C y los 3 enlaces serie) requiere al menos $2^3 = 8$ subredes iguales, que es lo que se obtiene tomando 3 bits adicionales de un bloque

`/16`: 16 + 3 = 19. La tabla 7.3 muestra las 8 subredes resultantes y su asignación. Para todas ellas, la máscara de subred será `255.255.224.0`.

#	Subred	Tercer octeto	Dir. de subred	Dir. broadcast
0	Red A	**000** 00000	70.50.0.0/19	70.50.31.255
1	Red B	**001** 00000	70.50.32.0/19	70.50.63.255
2	Red C	**010** 00000	70.50.64.0/19	70.50.95.255
3	R1-R0	**011** 00000	70.50.96.0/19	70.50.127.255
4	R2-R0	**100** 00000	70.50.128.0/19	70.50.159.255
5	R3-R0	**101** 00000	70.50.160.0/19	70.50.191.255
6	Libre	**110** 00000	70.50.192.0/19	70.50.223.255
7	Libre	**111** 00000	70.50.224.0/19	70.50.255.255

CUADRO 7.3: Ejemplo de *subnetting*

Fíjate en el enorme desperdicio que provoca esta técnica: cada enlace serie (R1-R0, R2-R0 y R3-R0) solo necesita un bloque `/30` (4 direcciones), pero al exigir *subnetting* que todas las subredes tengan igual tamaño, cada uno consume un bloque `/19` con 8192 direcciones. Eso no ocurre solo con los enlaces serie. Por ejemplo la red C solo necesita 1003 direcciones, de modo que se desaprovechan 8192-1003=7189 direcciones. Este desperdicio lo resuelve VLSM, como veremos más adelante.

Existen muchas herramientas para realizar el cálculo de subredes con *subnetting*. Una de las más sencillas es (`sipcalc`). En el Listado 7.1 reproduce la misma división.

```
~$ sipcalc -bi 70.50.0.0/16 -s /19
-[ipv4 : 70.50.0.0/16] - 0

[CIDR]
Host address            - 70.50.0.0
Host address (decimal)  - 1177681920
Host address (hex)      - 46320000
Network address         - 70.50.0.0
Network mask            - 255.255.0.0
Network mask (bits)     - 16
Network mask (hex)      - FFFF0000
Broadcast address       - 70.50.255.255
Cisco wildcard          - 0.0.255.255
Addresses in network    - 65536
Network range           - 70.50.0.0 - 70.50.255.255
Usable range            - 70.50.0.1 - 70.50.255.254

[CIDR bitmaps]
Host address            - 01000110.00110010.00000000.00000000
Network address         - 01000110.00110010.00000000.00000000
Network mask            - 11111111.11111111.00000000.00000000
Broadcast address       - 01000110.00110010.11111111.11111111
Cisco wildcard          - 00000000.00000000.11111111.11111111
```

```
Network range           - 01000110.00110010.00000000.00000000 -
                          01000110.00110010.11111111.11111111
Usable range            - 01000110.00110010.00000000.00000001 -
                          01000110.00110010.11111111.11111110

[Split network]
Network                 - 70.50.0.0       - 70.50.31.255
Network                 - 70.50.32.0      - 70.50.63.255
Network                 - 70.50.64.0      - 70.50.95.255
Network                 - 70.50.96.0      - 70.50.127.255
Network                 - 70.50.128.0     - 70.50.159.255
Network                 - 70.50.160.0     - 70.50.191.255
Network                 - 70.50.192.0     - 70.50.223.255
Network                 - 70.50.224.0     - 70.50.255.255
```

LISTADO 7.1: Ejemplo de uso de `sipcalc`

7.3. Direccionamiento sin clases

Aunque la técnica de *subnetting* facilitó un uso más eficiente, no era suficiente. Años más tarde se modificó el sistema de direccionamiento para proporcionar una mayor flexibilidad, sobre todo para el espacio que aún quedaba por asignar. Se llamó a esto «direccionamiento sin clases» o *classless addressing*, aunque la designación técnica es «encaminamiento interdominio sin clases» o CIDR (Classless Interdomain Routing), recogido en la RFC 1519 [13].

En el direccionamiento sin clases la máscara de red (y por tanto el prefijo) no está restringida a 8, 16 o 24 bits correspondientes a las clases A, B y C, sino que puede tener cualquier tamaño, por supuesto siempre dentro de los 32 bits de la dirección IP.

Con la llegada del direccionamiento sin clases, los bits iniciales de la dirección IP pierden el significado que tenían para las clases A, B y C. Eso quiere decir que ya no es posible determinar la máscara a partir de la dirección. Y sin la máscara no se puede determinar la dirección de red o broadcast. Por ese motivo, con *classless* es necesario conocer siempre la máscara junto con la dirección.

Para facilitar esta tarea, se introdujo una sintaxis más compacta: la llamada «notación CIDR». Consiste simplemente en añadir el carácter `/` seguido del número de bits que forman el prefijo de red, y que ya hemos usado en la sección anterior.

7.3.1. VLSM

VLSM es la generalización del concepto de *subnetting*. Permite dividir una red en subredes de diferentes tamaños, lo que permite que las máscaras

varíen de tamaño entre subredes —de ahí su nombre. Esto permite un aprovechamiento mucho mayor del espacio de direccionamiento.

En un caso real, el objetivo de un reparto de direcciones es parte esencial del diseño de la infraestructura de comunicaciones de una organización. No se trata simplemente de dividir sin más el espacio de direcciones disponible en cierta cantidad de bloques, como en el ejemplo de *subnetting*. Lo normal es que una organización que disponga de un determinado bloque deba proporcionar direcciones a distintos departamentos, o usos específicos, como servidores, impresoras, telefonía IP, etc.

Por eso, la organización necesita determinar primero cuáles son las necesidades de direccionamiento de la empresa y, a partir de ahí, diseñar su esquema de direccionamiento. En ese sentido, es importante tener en cuenta las expectativas de crecimiento de cada uno de los departamentos contando con el margen de direcciones libres requerido dentro de cada bloque, dado que una vez en explotación, cambiar el esquema de direccionamiento puede ser complejo.

Aunque todo esto es igualmente importante con el direccionamiento con clases y *subnetting*, VLSM resuelve mucho mejor este problema ya que ofrece mayor flexibilidad y reduce notablemente el desperdicio de direcciones.

7.3.2. Bloques /30

VLSM resulta especialmente importante cuando es necesario asignar direcciones a redes pequeñas. La red más pequeña posible es una bastante habitual: enlaces punto a punto, como los que se utilizan para interconectar dos routers. En ese caso, solo se necesita una dirección para cada extremo del enlace. Se utiliza la subred de tamaño mínimo, que es la que tiene máscara de 30 bits y *host ID* de 2 bits. por tanto 4 direcciones: la de red, la de broadcast y dos direcciones asignables. Por ejemplo, el bloque `170.10.20.160/30` aplicado a un enlace punto a punto (ver Figura 7.4) tiene las siguientes direcciones:

Dirección de red:	170.10.20.160/30
Dirección asignable 1:	170.10.20.161/30
Dirección asignable 2:	170.10.20.162/30
Dirección de broadcast:	170.10.20.163/30

Idealmente esta necesidad la deberían cubrir los bloques `/31`, pero siguiendo las reglas establecidas, las únicas dos direcciones que ofrece son la de red y la de broadcast, de modo que no quedaría ninguna para asignar a un nodo. Sin embargo, la RFC 3021 [14] establece que es posible utilizar un bloque

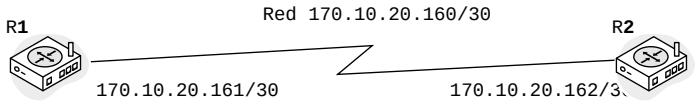


FIGURA 7.4: Direccionamiento IP de un enlace punto a punto (serie)

`/31` específicamente para este propósito, bajo la premisa —que puede no cumplirse en todos los casos— de que un enlace punto a punto no necesita una dirección de broadcast. Siguiendo el ejemplo anterior, el bloque `170.10.20.160/31` tendría las siguientes direcciones:

Dirección asignable 1:	170.10.20.160/31
Dirección asignable 2:	170.10.20.161/31

En todo caso, ten presente que aunque esta norma no es nueva, puede haber equipos de red y SO que no lo soportan.

Y por último, la máscara `/32` no define una red o subred: identifican exclusivamente un nodo individual, por ejemplo, `10.0.0.1/32`.

7.3.3. Ejemplo de VLSM

Retomando la misma topología de la Figura 7.3, vamos a definir ahora el esquema de direccionamiento con VLSM, partiendo esta vez del mismo bloque `70.50.0.0/16` que usamos en el ejemplo de *subnetting*. A diferencia de *subnetting*, en VLSM cada subred consume solo el tamaño que realmente necesita, así que antes de repartir direcciones tendremos que calcular cuánto espacio requiere realmente la topología.

Asumiendo que estas cantidades de nodos expresan las necesidades de cada red considerando el posible crecimiento futuro, el proceso de división debería minimizar la cantidad de direcciones que sobran (no se asignan a nodos) en cada bloque. Al mismo tiempo, es muy conveniente que los bloques sin asignar sean lo mayor posible, para conseguir un mejor aprovechamiento futuro.

Lo primero que haremos es determinar las necesidades de la topología. Por ejemplo, la red A requiere 5000 nodos. A eso tenemos que sumar 1 dirección más para la interfaz interna del router R1 además de las direcciones de red y broadcast. En total 5003 direcciones. Ahora debemos calcular su logaritmo en base 2 para determinar cuántos bits de *host-ID* se necesitan:

$$\log_2(5003) = 12{,}28$$

Obviamente no podemos tener bits decimales, de modo que necesitaremos 13 bits para conseguir las 5003 direcciones. La máscara de esa subred será por tanto $32 - 13 = /19$. Ten en cuanta que con 13 bits tenemos realmente $2^{13} = 8192$ direcciones, así que «sobran» $8192 - 5003 = 3189$ direcciones.

Si aplicas estos mismos cálculos al resto de las subredes, obtendrás los resultados que aparecen en el Cuadro 7.4. Para cada red, se muestra el nombre, la cantidad de nodos que necesita (Necesidad), el tamaño en bits del *host-ID*, la máscara de la subred resultante (Máscara), la cantidad total de direcciones (Total) y la cantidad de direcciones libres/desperdiciadas.

Subred	Necesidad	Máscara	host-ID	Total	Libres
A	5003	/19	13	8192	3189
B	2003	/21	11	2048	45
C	1003	/22	10	1024	21
R1-R0	2	/30	2	4	0
R2-R0	2	/30	2	4	0
R3-R0	2	/30	2	4	0
Total	8015	/18	14	11 276	3255

CUADRO 7.4: Ejemplo de VLSM: Necesidades de las subredes

La fila «Total» indica la cantidad equivalente a único bloque contiguo. Aplicando el logaritmo, obtenemos 14 bits de *host-ID* y, por tanto, máscara `/18`.

$$\log_2(11276) = \lceil 13,46 \rceil = 14$$

Es decir, se pueden cubrir todas las necesidades con un 1/4 del bloque de partida. Por tanto, antes de repartir las subredes, dividimos el bloque `70.50.0.0/16` en 4 bloques `/18` y repartimos solo el primero (Tabla 7.5).

Bloque	3er byte	Dirección de red
Topología	**00** 00 0000	70.50.0.0/18
Libre	**01** 00 0000	70.50.64.0/18
Libre	**10** 00 0000	70.50.128.0/18
Libre	**11** 00 0000	70.50.192.0/18

CUADRO 7.5: Ejemplo de VLSM: división inicial

Por conveniencia se suele hacer el reparto ordenando las subredes de mayor a menor, con un algoritmo de tipo *greedy first-fit* (voraz de primer encaje). Como el bloque de partida es /18 y la red A necesita un bloque /19 debemos utilizar 1 bit. La tabla 7.6 muestra esta división. En la segunda columna

se muestra el tercer byte de la dirección que es donde está la frontera entre *net-ID* y *host-id*. El bit marcado en negrita es el que se ha utilizado para dividir el bloque de partida.

Subred	3er byte	Dirección de red
Red A	00**0**0 0000	70.50.0.0/19
Libre	00**1**0 0000	70.50.32.0/19

CUADRO 7.6: Ejemplo de VLSM: Red A

El siguiente paso es asignar espacio a la red B. Como es un bloque `/21` y partimos de un bloque `70.50.32.0/19`, significa que necesitamos solo una cuarta parte, de modo que utilizaremos 2 bits adicionales del *host-id*. La tabla 7.7 muestra la división.

Subred	3er byte	Dirección de red
Red B	001**0 0**000	70.50.32.0/21
Libre	001**0 1**000	70.50.40.0/21
Libre	001**1 0**000	70.50.48.0/21
Libre	001**1 1**000	70.50.56.0/21

CUADRO 7.7: Ejemplo de VLSM: Red B

La red C necesita un bloque `/22`, así que dividimos el primer bloque libre (`70.50.40.0/21`) con 1 bit adicional. La tabla 7.8 muestra la división.

Subred	3er byte	Dirección de red
Red C	0010 1**0**00	70.50.40.0/22
Libre	0010 1**1**00	70.50.44.0/22

CUADRO 7.8: Ejemplo de VLSM: Red C

Por último, utilizaremos el bloque libre `70.50.44.0/22` para asignar subredes a los 3 enlaces serie. En este caso se utilizan 8 bits adicionales para obtener máscaras `/30`. La tabla 7.9 muestra la división. Al realizar la división con 8 bits tendríamos 256 de estos bloques `/30`, pero solo necesitamos 3 (Cuadro 7.9).

El Cuadro 7.10 muestra para cada bloque: dirección de red, máscara en decimal, primera y última dirección asignable y dirección de broadcast.

Por último, en el Cuadro 7.11 aparecen todos los bloques que han quedado libres. La primera columna es solo un identificador para referirnos a ellos. Los bloques 1 a 7 corresponden con la fragmentación provocada al asignar los enlaces serie. Hemos agregado en lo posible el espacio disponible para

Subred	3er byte	4o byte	Dirección de red
R1-R0	0010 11**00**	**0000 00**00	70.50.44.0/30
R2-R0	0010 11**00**	**0000 01**00	70.50.44.4/30
R3-R0	0010 11**00**	**0000 10**00	70.50.44.8/30
Libre	0010 11**00**	**0000 11**00	70.50.44.12/30
...	...	...	...
Libre	0010 11**11**	**1111 11**00	70.50.w55.12/30

CUADRO 7.9: Ejemplo de VLSM: Enlaces serie

Subred	Dir. red	Máscara	Primera	Última	Broadcast
A	70.50.0.0/19	255.255.224.0	70.50.0.1	70.50.31.254	70.50.31.255
B	70.50.32.0/21	255.255.248.0	70.50.32.1	70.50.39.254	70.50.39.255
C	70.50.40.0/22	255.255.252.0	70.50.40.1	70.50.43.254	70.50.43.255
R1-R0	70.50.44.0/30	255.255.255.252	70.50.44.1	70.50.44.2	70.50.44.3
R2-R0	70.50.44.4/30	255.255.255.252	70.50.44.5	70.50.44.6	70.50.44.7
R3-R0	70.50.44.8/30	255.255.255.252	70.50.44.9	70.50.44.10	70.50.44.11

CUADRO 7.10: Ejemplo de VLSM: Resultado

evitar esos 253 bloques `/30` que no necesitamos. El bloque 8 también es una agregación de los bloques `70.50.48.0/21` y `70.50.56.0/21` que quedaron libres. Como son contiguos y difieren solo en el primer bit del *net-id* es posible expresarlos como un solo bloque `/20`.

Los bloques 9 y 10 corresponden a las 3/4 partes del bloque `70.50.0.0/16` que quedaron sin usar en la división inicial (Cuadro 7.5). Los bloques `70.50.128.0/18` y `70.50.192.0/18` son contiguos y difieren solo en el primer bit del *net-id*; se pueden agregar en un único bloque `/17`. El bloque `70.50.64.0/18` no cumple esta condición y no se puede agregar.

Id	3er byte	4o byte	Dirección del bloque
1	0010 1100	0000 1100	70.50.44.12/30
2	0010 1100	0001 0000	70.50.44.16/28
3	0010 1100	0010 0000	70.50.44.32/27
4	0010 1100	0100 0000	70.50.44.64/26
5	0010 1100	1000 0000	70.50.44.128/25
6	0010 1101	0000 0000	70.50.45.0/24
7	0010 1110	0000 0000	70.50.46.0/23
8	0011 0000	0000 0000	70.50.48.0/20
9	0100 0000	0000 0000	70.50.64.0/18
10	1000 0000	0000 0000	70.50.128.0/17

CUADRO 7.11: Ejemplo de VLSM: Bloques libres

Frente a *subnetting*, donde las seis subredes —incluidos los enlaces serie— consumían 8192 direcciones cada una por tener que compartir máscara, con

VLSM cada bloque se ajusta mucho más a la necesidad real. El resultado es que toda la topología cabe en un único bloque `/18`, la cuarta parte del espacio original, quedando además el resto disponible para crecimiento futuro. La contrapartida es un reparto más laborioso, al calcularse subred a subred en lugar de aplicar una máscara única.

7.4. Direcciones especiales

Además de las direcciones de red y broadcast, hay otras direcciones especiales que tienen usos particulares o predefinidos. Estas son las más importantes:

Dirección nula La dirección `0.0.0.0` —conocida como `INADDR_ANY`— se utiliza en varias situaciones diferentes cuando no se conoce, no se dispone o no tiene sentido utilizar una dirección IP concreta. Por ejemplo, en las peticiones ARP, en una tabla de encaminamiento para indicar entrega directa o router por defecto y otros casos que veremos a lo largo del libro.

Direcciones *loopback* Cualquier dirección del rango `127/8` solo tiene sentido en el propio nodo y no se puede asignar a ninguna interfaz de red que pueda ser accesible desde fuera. La dirección más común para este uso es `127.0.0.1`.

Direcciones privadas Son direcciones que se pueden utilizar únicamente en redes privadas, y por ello no necesitan autorización ni asignación por parte de ninguna autoridad, pero son ignoradas por los routers fuera de la red privada. Las veremos en detalle en el capítulo 11.

Direcciones de enlace local Son direcciones que se utilizan para comunicar dispositivos en la misma red local, sin necesidad de un router. Hablaremos de ellas también en el capítulo 11.

Direcciones multicast Son direcciones que identifican a grupos. No están asignadas a ningún interfaz de red. Sirven para enviar un mensaje a todos los dispositivos que pertenezcan a ese grupo. Se utilizan en aplicaciones como la IPTV, radio por IP, distribución de contenidos, etc. Son las direcciones de clase D y empiezan por 1110. El uso de esta clase no fue descartado con la llegada del direccionamiento sin clases, y se sigue utilizando en la actualidad.

Y ¿qué más?

En realidad no hay mucho más que saber sobre el direccionamiento IP. Este capítulo te da una buena base para diseñar por ti mismo un esquema de direccionamiento de cualquier organización. El diseño puede ser arbitrariamente complejo y tendrás que considerar muchos factores, pero el proceso se basa en estos mismos conceptos. Incluso con IPv6 la base es prácticamente la misma en lo que concierne a la subdivisión de bloques, ya que nunca existió direccionamiento con clase en IPv6. En el capítulo 10 veremos cómo se asignan estas direcciones a las interfaces de red, ya sea manual o automáticamente con DHCP, pero esa tarea en realidad es independiente del esquema de direccionamiento.

Capítulo 8

Interconexión

Al terminar este capítulo, entenderás:

- Cómo se mueven los paquetes IP a través de una interred.
- Qué son los routers y cuál es exactamente su trabajo.
- Qué es la tabla de encaminamiento y cómo funciona.
- Cómo los routers colaboran entre sí para llevar los paquetes hasta su destino.
- Cómo los paquetes IP atraviesan redes con tecnologías de enlace diferentes y porqué se produce la fragmentación de paquetes.

Una de las tecnologías clave para que Internet alcance prácticamente cualquier punto del planeta es la interconexión de redes, particularmente, cuando se trata de tecnologías heterogéneas. Aunque es cierto que existieron y existen otras tecnologías que ofrecen una funcionalidad similar, ha sido TCP/IP la que ha logrado convertirse con diferencia en la más popular.

Por supuesto, llevar paquetes a través de múltiples redes es uno de los objetivos prioritarios de TCP/IP y en particular del protocolo IP. Citando la RFC 791 [9]:

> "The purpose of internet protocol is to move datagrams through an interconnected set of networks".

En español:

> "El propósito del protocolo de interred es mover datagramas a través de un conjunto interconectado de redes".

De nuevo insistimos aquí, por su importancia, que «Protocolo de Internet» debe entenderse como «protocolo de interconexión de redes», porque es IP el que hace posible las interredes.

Para interconectar redes es necesario resolver varios problemas. El principal es el encaminamiento, el mecanismo que determina la ruta que siguen los paquetes hasta su destino. Los routers juegan un papel clave. Son de hecho los dispositivos que crean las interredes. Disponen de varias interfaces, conectadas a redes distintas, y por eso pueden mover paquetes entre ellas. Pero también es necesario resolver los problemas que aparecen cuando se utilizan tecnologías de enlace diferentes al interconectar redes heterogéneas. Y además debemos entender el modo en que los routers informan de errores o situaciones especiales a otros routers y a los nodos finales.

8.1. Almacenamiento y reenvío

Definamos primero lo que es un router: Es un dispositivo conectado a más de una red, que además, tiene la capacidad de aceptar paquetes *dirigidos a terceros* y reenviarlos (del inglés *forwarding*) para colaborar en la tarea de llevarlos hasta su destino.

La diferencia clave entre un PC que actúa como un simple «nodo final» y otro que actúa como router, es que el segundo reenvía los paquetes que no son para él, es decir, aquellos cuya dirección destino no coincide con la dirección asignada a la interfaz por la que llegan. El nodo final en cambio, los descarta.

En un sistema GNU/Linux, ese *reenvío* se activa con el parámetro del núcleo `net.ipv4.ip_forward`:

```
~# echo 1 > /proc/sys/net/ipv4/ip_forward
```

Recuerda que para hacer este cambio persistente, debes editar el archivo `/etc/sysctl.conf` (ver § 2.10).

La Figura 8.1 muestra una topología de ejemplo que se utilizará para ilustrar varios conceptos y mecanismos a lo largo del capítulo. Esta interred está formada por 5 nodos (`PC1` a `PC5`) conectados a 3 redes Ethernet (`N1` a `N3`). Los routers `R1` y `R2` tienen 2 interfaces Ethernet y una interfaz serie con encapsulación PPP. Para todas las interfaces se indica la dirección IP y el último octeto de la dirección MAC[1].

Los routers tienen interfaces en varias redes que pueden ser de distinta tecnología y protocolo de enlace (Ethernet y PPP en este caso).

[1] Es solo una simplificación para poder usar aquí datos más sencillos.

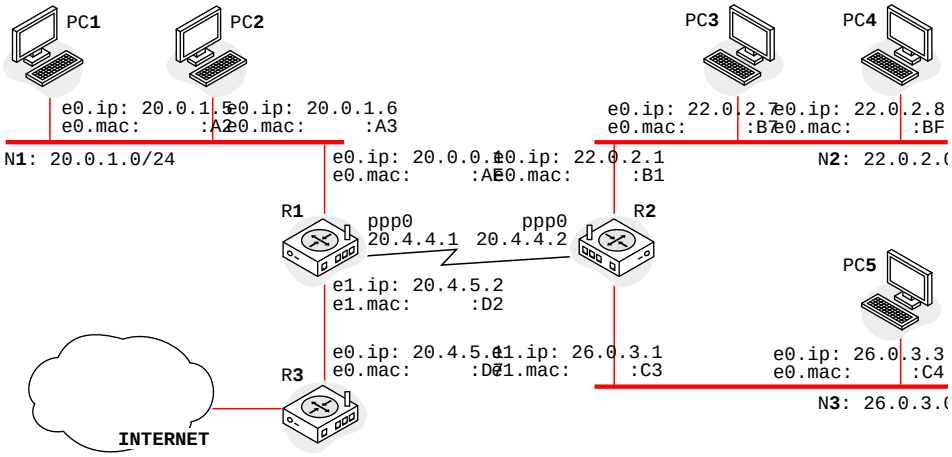


FIGURA 8.1: Topología de ejemplo

IP es una tecnología de conmutación de paquetes, que también se suele llamar «red de datagramas». Un datagrama es un bloque de datos independiente, que lleva incorporada —en su cabecera— toda la información necesaria para llegar a su destino. El router desconoce —o simplemente pasa por alto— si una secuencia de paquetes que está recibiendo pertenecen a la misma conexión o flujo, proceden del mismo origen o van al mismo destino. Aunque tuvieran relación, son tratados como si no la tuvieran. Por eso, paquetes que sí están relacionados podrían seguir rutas diferentes, y eso tiene consecuencias importantes.

Un router IP recibe y procesa los paquetes que van llegando a cualquiera de sus interfaces, sin importar la tecnología de enlace que emplean. Como la unidad de procesamiento del router puede estar ocupada cuando llega un nuevo paquete, es necesario almacenar el paquete temporalmente en una cola (hay una cola de entrada por interfaz). De forma similar, cuando el router quiere reenviar un paquete, la línea de salida puede estar ocupada, por lo que también hace falta una cola de salida para almacenar los paquetes pendientes de enviar.

Por esta forma de trabajar se dice que el router es un dispositivo de *almacenamiento y reenvío* (*store and forward*). Si incluimos algo más de detalle, el proceso que realiza el router cada vez que llega un paquete es el siguiente:

1. Una vez recibido por la interfaz, el paquete se extrae de su envoltorio de enlace (descarta la cabecera de la trama). Es una tarea que realiza la propia NIC como en cualquier otro dispositivo.

2. El paquete se almacena en memoria, en concreto, en la cola de entrada de esa interfaz; y espera allí hasta que el router disponga de tiempo/recursos para procesarlo.
3. Cuando por fin el router procesa el paquete, comprueba su *checksum*. Si la comprobación falla, descarta el paquete.
4. Comprueba el campo TTL de la cabecera. Si es 0, se descarta el paquete e informa al origen (§ 8.6.2).
   En otro caso (mayor que 0), lo decrementa en 1, calcula el nuevo *checksum* y actualiza ambos campos en la cabecera. El campo TTL evita que un paquete pueda quedar viajando por la red indefinidamente en caso de producirse un bucle de encaminamiento.



El checksum del que hablamos es una función de suma de comprobación que permite detectar cualquier modificación que haya sufrido el paquete durante la transmisión, incluso aunque afecte a un único bit. Se conoce como *Internet checksum* y lo utilizan varios protocolos de la familia TCP/IP. Aparte de la cabecera IPv4 que estamos tratando aquí, lo usan TCP, UDP, ICMP, e IGMP, entre otros. En el repositorio de código del libro puedes encontrar una implementación en Python de este algoritmo (`/inet-checksum/checksum.py`) y un notebook con una explicación detallada de su funcionamiento (`/inet-checksum/checksum.ipynb`).

5. Determina la interfaz de salida en función exclusivamente de la dirección destino del paquete y del contenido de la tabla de encaminamiento. Si la tabla no contiene información sobre cómo proceder, el paquete se descarta y, de nuevo, informa del problema al origen con un mensaje ICMP *destination unreachable* (destino inalcanzable).
6. El paquete se coloca en la cola de la interfaz de salida elegida.
7. Cuando el medio físico asociado a la interfaz queda libre, la NIC encapsula el paquete IP con el formato de trama según la tecnología de ese enlace que lo transforma en una señal capaz de alcanzar el siguiente dispositivo.
   Cuando esta interfaz de salida está conectada a un enlace de difusión (*p. ej.* una LAN Ethernet), el router debe determinar primero la dirección física destino de la trama.

Todo este proceso es lo que llamamos «un salto», una expresión que enfatiza la idea de que el paquete ha pasado (saltado) de una red a otra. Los sucesivos routers a lo largo de la ruta efectúan cada uno de estos saltos hasta el destino. Es la base del funcionamiento de cualquier interred IP.

8.2. Entrega directa vs. indirecta

El trabajo del router es entregar el paquete «al siguiente» en la ruta hacia su destino. En este camino, se efectúan dos tipos de entrega:

- La entrega directa (o local) la realiza cuando el destino del paquete es un vecino del origen. Por ejemplo, si ambos (origen y destino) están conectados a una red Ethernet, el origen encapsula el paquete en una trama que lleva en su cabecera la dirección física del destino. Tanto la dirección MAC destino (en la cabecera de la trama) como la IP destino (en la cabecera del paquete) identifican al nodo destino.
- La entrega indirecta ocurre, como puedes suponer, cuando el destino no es vecino y, de hecho, probablemente el origen no tiene ninguna información en absoluto de dónde se encuentra ese destino. En ese caso, el origen debe entregar el paquete a un router asumiendo que él sabe cómo llevarlo al destino. Aquí también se encapsula el paquete en una trama, pero la dirección física destino es la del siguiente router, no la del destino, es decir, la trama va dirigida al router, pero el paquete encapsulado va dirigido al destino.

Esta explicación sirve tanto para routers como para nodos finales. Ambos, cuando necesitan enviar un paquete a un vecino, realizan una entrega directa, y cuando el destino está fuera de la red, realizan una entrega indirecta.

En la sección 8.5 veremos un ejemplo práctico con datos y mensajes concretos que ilustra ambos tipos de entrega en una interred sencilla, pero no trivial.

8.3. Tabla de encaminamiento

La tabla de encaminamiento contiene la información que necesita el router para hacer su trabajo principal: decidir dónde enviar cada paquete que llega. Cada fila de esta tabla especifica un destino (normalmente una red) y qué hacer para llegar a él. Sin embargo, la tabla no tiene instrucciones precisas de cómo llevar el paquete hasta allí, solo habla del siguiente salto. Veamos un ejemplo en la Tabla 8.1.

Vale la pena hacer aquí una aclaración para evitar malentendidos. La tabla de encaminamiento *(routing table)* se denomina muy a menudo en español «tabla de rutas». Pero si entendemos que una «ruta» es un «itinerario o camino para un viaje»[2], la traducción «tabla de rutas» es incorrecta, o al menos, confusa, porque no es eso lo que contiene la tabla. Por eso

[2] Así lo define la RAE.

#	dst	mask	next hop	iface
1	30.0.0.0	255.255.255.0	0.0.0.0	e0
2	40.0.0.0	255.255.255.0	0.0.0.0	e1
3	130.10.20.0	255.255.255.0	30.0.0.2	e0
4	210.20.30.0	255.255.128.0	40.0.0.3	e1
5	0.0.0.0	0.0.0.0	50.1.1.2	s0

CUADRO 8.1: Ejemplo de tabla de encaminamiento

optaremos aquí por la traducción «tabla de encaminamiento». A pesar de eso, en muchas ocasiones (incluso en inglés) para hacer referencia a las filas de la tabla, se habla de «rutas».

Toda tabla de encaminamiento contiene al menos la siguiente información:

Destino *(dst)*
: Una dirección IP que identifica una red. Aunque menos frecuente, también puede ser una dirección de nodo. Un caso especial es la dirección IP nula (`0.0.0.0`) que indica que esa fila es un *router por defecto*.

Máscara *(mask)*
: La máscara asociada al destino (la primera columna) y determina qué parte de aquella corresponde al prefijo de red (ver sección 7.1). En ocasiones la columna «destino» se combina con la máscara en formato CIDR (*p. ej.* `120.10.12.0/24`).

Siguiente salto *(next-hop)*
: La dirección IP de un router vecino, o bien la dirección IP nula (`0.0.0.0`) que aquí significa entrega directa.

Interfaz *(iface)*
: El nombre de la interfaz por la que debe salir el paquete.

Para cada nuevo paquete que llega, el router busca la correspondencia entre la dirección destino del paquete y la combinación destino-máscara de cada fila. Como varias filas podrían cumpliir esta correspondencia, se aplica el siguiente criterio de prioridad:

1. Entrega directa.
2. Entrega indirecta.
3. Router por defecto.

Dentro de cada categoría se aplica LPM (Longest Prefix Match), es decir, la máscara más larga (más específica) tiene más prioridad. Si aún así hay empate, se aplica la primera fila que aparezca en la tabla.

Si no es posible encontrar ninguna coincidencia, el paquete es descartado (y el origen informado con un mensaje ICMP). Sin embargo, la tabla puede

incluir un *router por defecto (default router)*. Suele aparecer en la última fila (como en la Tabla 8.1), pero eso es solo una convención y en realidad su posición es irrelevante.

Pero exactamente, ¿cuál es esa «correspondencia» que se evalúa para cada fila? Consiste simplemente en una operación AND a nivel de bits entre la dirección destino del paquete y la máscara de esa fila. La fila es adecuada si el resultado de esa operación coincide con el valor de la columna «destino».

Cuando el router determina qué fila utilizar, envía el paquete a la dirección que aparece en la columna «siguiente salto» a través de la interfaz indicada en la columna «interfaz». Fíjate que la columna «interfaz» siempre identifica interfaces del propio router.

Siguiendo el ejemplo, supongamos que al router de la Tabla 8.1 llega un paquete con destino `40.0.0.9`. La única fila coincidente sería la 2, lo cual se determina al aplicar la operación indicada:

$$(40.0.0.9 \text{ AND } 255.255.255.0) == 40.0.0.0$$

Por tanto, el paquete se reenviará al destino, que es un vecino (es una entrega directa) a través de la interfaz `e1`. El procesamiento del paquete termina y el router procede con el siguiente paquete.

El siguiente fragmento de pseudocódigo ilustra el modo en que el router procesa la tabla de encaminamiento por cada paquete que llega a través de cualquiera de sus interfaces:

```
destino = datagrama.ip_destino
for fila in tabla_de_encaminamiento:
    if (destino AND fila.máscara) == fila.destino:
        reenviar(datagrama, fila.siguiente_salto)
        return

if router_por_defecto:
    reenviar(datagrama, router_por_defecto)
else:
    enviar_ICMP(destino_inalcanzable, datagrama.origen)
```

8.3.1. Analizando la tabla de ejemplo

La Tabla 8.1 corresponde con un hipotético router `R0` que podría ser perfectamente realista. Esta es, en detalle, la información que ofrece. La primera columna se ha añadido únicamente para numerar las filas y referirnos a ellas.

- La tabla corresponde a un router que dispone de 3 interfaces: `e0`, `e1` y `s0`: dos Ethernet y una serie.
- Las filas 1 y 2 inidcan entrega directa. Se reconocen porque el valor para el campo *next-hop* es la dirección IP nula: `0.0.0.0`. Esto significa que si el destino está en la red `30.0.0.0/24` (accesible por medio de su interfaz `e0`) o en la red `40.0.0.0/24` (a través de su interfaz `e1`), el router y el destino son vecinos y el propio router se encarga de entregar el paquete.
- Las filas 3 y 4 especifican entregas indirectas. La 3 indica que todo paquete dirigido a la red `130.10.20.0/24` debe reenviarse al router `30.0.0.2`. La fila 4 indica que todo paquete dirigido a la red `210.20.30.0/17` debe reenviarse al router `40.0.0.3`.
- Por último, la fila 5 establece un router por defecto. Cualquier paquete que no haya satisfecho ninguna de las otras filas, será enviado al router `50.1.1.2`.

Es importante señalar que la tabla de encaminamiento no dice nada sobre dónde están las redes `130.10.20.0/24` y `210.20.30.0/17` ni a cuántos saltos se encuentran. La tabla solo dice que para llegar a las redes *distantes*, se debe delegar la entrega al router vecino indicado. Pero hay parte de la topología que podemos deducir (ver Figura 8.2), la que involucra a las redes *locales*, es decir, de las que `R0` es vecino.

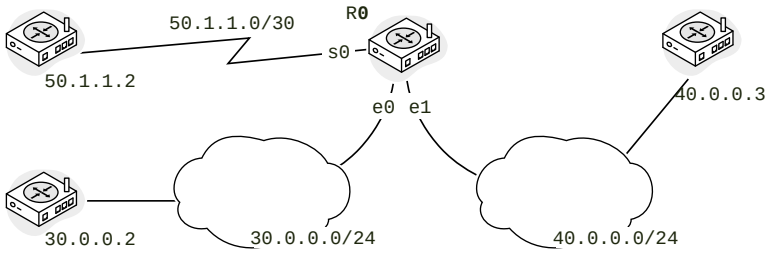


FIGURA 8.2: Topología (parcial) que se deduce de la Tabla 8.1

8.3.2. Tabla de nodo final

Cualquier nodo o dispositivo conectado a Internet (incluso un *smart phone* o una consola de videojuegos) tiene una tabla de encaminamiento, aunque eso no lo convierte en router. Para comportarse como tal debe ser capaz de hacer reenvío (§ 8.1) y eso no está activado por defecto.

La tabla de encaminamiento indica al nodo final cómo enviar tráfico a sus vecinos y cómo enviar tráfico fuera del enlace (una LAN normalmente). Una

tabla de encaminamiento típica de un nodo en una red doméstica con enlace WiFi[3] es muy parecida a la Tabla 8.2.

dst	mask	next hop	iface
192.168.0.0	255.255.255.0	0.0.0.0	wlan0
0.0.0.0	0.0.0.0	192.168.0.1	wlan0

Cuadro 8.2: Tabla de encaminamiento de un nodo final en una red doméstica

¿Qué significa?

- La primera fila dice dos cosas: quiénes son los vecinos (los conectados a la red `192.168.0.0/24`), y que para llegar a ellos el nodo debe hacer una entrega directa (*next-hop* = `0.0.0.0`).
- La segunda fila indica el router por defecto, que identifica un router de la red domestica (`192.168.0.1`); y a través de éste, el sistema enviará tráfico hacia Internet. Los sistemas operativos de escritorio suelen denominar a este router «pasarela de enlace».



Al único router que aparece en la tabla de encaminamiento de un nodo final, se le suele llamar *puerta de enlace* (*gateway*) porque se entiende que es el punto de acceso a redes externas.

En un sistema GNU/Linux, puedes ver la tabla de encaminamiento con el comando `ip route`. Como probablemente tu computador es un PC conectado a una red doméstica, verás algo muy similar a lo siguiente:

```
~$ ip route
default via 192.168.0.1 dev wlan0 proto dhcp metric 600
192.168.0.0/24 dev wlan0 proto kernel scope link metric 600
```

Aunque el formato es bastante espartano, obviando los datos adicionales, es fácil interpretar la siguiente información:

dest/mask	next hop	iface
default	192.168.0.1	wlan0
192.168.0.0/24	scope link	wlan0

Esto se corresponde directamente con la Tabla 8.2. Las diferencias son escasas:

- Las columnas «destino» y «máscara» aparecen como un solo dato en notación CIDR: *destino/máscara*.

[3]Suele estar detrás de un router NAT (§ 11.6).

- Como destino *nulo*, en lugar de `0.0.0.0`, muestra *default* (por *default router*).
- Para el siguiente salto, en lugar de `0.0.0.0` muestra *scope link*, que significa que esos dispositivos (`192.168.0.0/24`) son accesibles en el «ámbito del enlace», es decir, que son vecinos.

La designación *proto dhcp* indica que esa entrada se ha creado a partir de información recibida por medio del servicio DHCP, mientras que *proto kernel* indica que ha sido creada por el SO, probablemente cuando se activó la interfaz.

El valor *metric* indica la prioridad. Si hay varias filas aplicables para un mismo paquete, se elige la que tenga menor valor de prioridad. Si la prioridad también coincide, en el caso de GNU/Linux, el SO aplica un algoritmo de balanceo de carga llamado ECMP que puede tener en cuenta cosas como la carga de la interfaz o la caché (prioriza la última usada). Un router dedicado, normalmente implementado como un ASIC o FPGA puede utilizar los criterios que considere el fabricante o su configuración específica.

8.4. Agregación

En ocasiones una tabla de encaminamiento puede contener filas que especifican redes que comparten un prefijo y son contiguas. En estos casos, es posible realizar *agregación (summarization)* de esos destinos. Por ejemplo, supongamos la topología de la Figura 8.3.

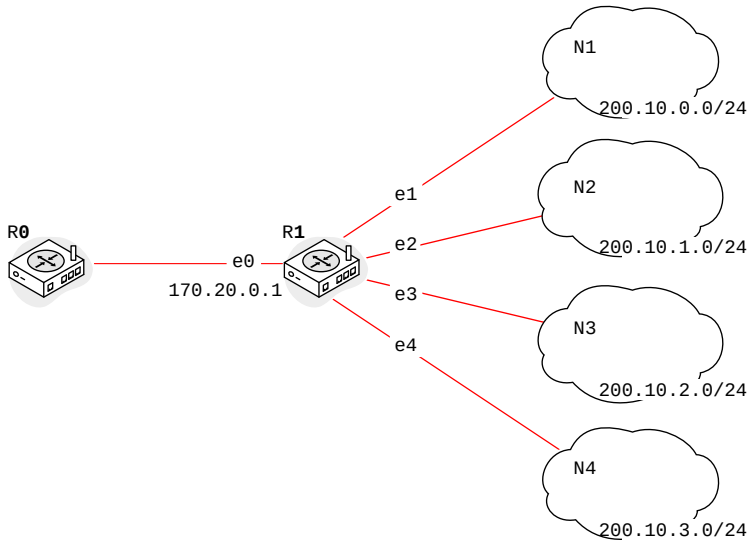


FIGURA 8.3: Topología con redes agregables

Siendo la tabla de encaminamiento de `R1`:

dst/mask	next hop	iface
200.10.0.0/24	0.0.0.0	e0
200.10.1.0/24	0.0.0.0	e1
200.10.2.0/24	0.0.0.0	e2
200.10.3.0/24	0.0.0.0	e3
170.20.0.0/24	0.0.0.0	e4

La tabla de `R0` en principio debería incluir también una fila para llegar a cada una de esas redes:

dst/mask	next hop	iface
200.10.0.0/24	170.20.0.1	e0
200.10.1.0/24	170.20.0.1	e0
200.10.2.0/24	170.20.0.1	e0
200.10.3.0/24	170.20.0.1	e0
170.20.0.0/24	0.0.0.0	e0

Pero una mejor opción sería «agregar» esas 4 filas en una sola, es decir, puede indicar que para llevar el paquete a una hipotética red `200.10.0.0/22` se debe enviar a `R1`. La tabla de `R0` quedaría así:

dst/mask	next hop	iface
200.10.0.0/22	170.20.0.1	e0
170.20.0.0/24	0.0.0.0	e0

Esto reduce el tamaño de la tabla de `R0` (y de todos a los que llegue este destino) sin perder información. Conlleva un aumento de rendimiento y una mejora de la escalabilidad de la red. Lo podemos ver como una especie de *supernetting* con la diferencia de que esta red agregada no existe en ningún sentido, salvo a afectos de encaminamiento.

8.5. Un par de ejemplos detallados

Sobre la topología de ejemplo (Figura 8.1), estos dos escenarios concretos ilustran cómo funciona el encaminamiento. Analizamos con todo detalle el proceso que sigue un paquete IP desde su creación en el origen hasta alcanzar su destino, incluyendo los cambios en su encapsulación a lo largo de la ruta.

dst/mask	next hop	iface
20.0.1.0/24	0.0.0.0	e0
22.0.2.0/24	20.4.4.2	ppp0
26.0.3.0/24	20.4.4.2	ppp0
20.4.4.2/32	0.0.0.0	ppp0
20.4.5.0/30	0.0.0.0	e1
0.0.0.0	20.4.5.1	e1

dst/mask	next hop	iface
22.0.2.0/24	0.0.0.0	e0
26.0.3.0/24	0.0.0.0	e1
0.0.0.0/0	20.4.4.1	ppp0
20.4.4.1/32	0.0.0.0	ppp0

CUADRO 8.3: Tablas de encaminamiento de `R1` y `R2`

Obviamente hace falta conocer las tablas de encaminamiento de routers y nodos. Las de `R1` y `R2` aparecen en la Figura 8.3.

Las demás tablas de encaminamiento son tablas básicas típicas que suelen tener los nodos finales como la vista en § 8.3.2. Por ejemplo, la tabla de `PC1` sería algo como:

dst/mask	next hop	iface
20.0.1.0/24	0.0.0.0	e0
0.0.0.0	20.0.1.1	e0

8.5.1. Escenario 1: entrega local

Supongamos que `PC1` envía un paquete IP a `PC2`, por ejemplo, ejecutando `ping -c1 20.0.1.6`. El proceso implica los siguientes pasos:

1. `PC1`[4] construye el mensaje ICMP y lo encapsula en un paquete IP.

IP	ver+IHL 0x45 | tos 0 | len 0x0054 id 0x1C46 | flags+offset 0x0000 ttl 64 | proto 1 (ICMP) | cksum src 20.0.1.5 | dst 20.0.1.6
ICMP	type 8 (Echo Request) | code 0 | cksum id 0x1234 | seq 1

2. `PC1` consulta su tabla de encaminamiento. La primera fila es aplicable (`20.0.1.6` AND `255.255.255.0` == `20.0.1.0`), de modo que realiza entrega directa.
3. Como la interfaz de salida es Ethernet, `PC1` debe encapsular el paquete en una trama Ethernet, pero desconoce la dirección MAC de `PC2`.
4. Para averiguarla, envía una petición ARP a la dirección broadcast, por lo que llegará a todos los nodos de la red. La petición ARP pregunta «¿Quién tiene la dirección IP `20.0.1.6`?». La cabecera ARP incluye esencialmente lo siguiente:

[4]Nombramos los nodos, pero lógicamente estas tareas las realizan sus SO.

- IP origen: `20.0.1.5`, IP destino: `20.0.1.6`.
- MAC origen: `:A2`, MAC destino: `00:00:00:00:00:00`.

El mensaje completo sería:

Ethernet	dst FF:FF:FF:FF:FF:FF | src :A2 | type 0x0806 (ARP)
ARP	hw 0x0002 (eth) | proto 0x0800 (IP) hw-len 6 | proto-len 4 | op 0x0001 sender :A2 | 20.0.1.5 target 00:00:00:00:00:00 | 20.0.1.6

5. `PC2` recibe la petición ARP y responde con su dirección MAC (`:A3`) en una respuesta ARP.

Ethernet	dst :A2 | src :A3 | type 0x0806 (ARP)
ARP	hw 0x0002 | proto 0x0800 hw-len 6 | proto-len 4 | op 0x0002 sender :A3 | 20.0.1.6 target :A2 | 20.0.1.5

6. `PC1` puede ahora construir y enviar el paquete IP encapsulado en otra trama Ethernet.

Ethernet	dst :A3 | src :A2 | type 0x0800 (IP)
IP	ver+IHL 0x45| tos 0 | len 0x0054 id 0x1C46 | flags+offset 0x0000 ttl 64 | proto 1 (ICMP) | cksum src 20.0.1.5 | dst 20.0.1.6
ICMP	type 8 (Echo Request) | code 0 | cksum id 0x1234 | seq 1

Como ya dice el título de la sección, se trata de una entrega **directa** dado que `PC1` y `PC2` son vecinos, pero en todo caso a la vista del mensaje es fácil de comprobar. En una entrega directa ambas direcciones destino, tanto física (`:A3`) como lógica (`20.0.1.6`) corresponden al nodo destino (`PC2`).

7. La trama llega a `PC2`, que la desencapsula y procesa el paquete IP.
8. Para responder al mensaje `ping`, `PC2` crea una respuesta *Echo Reply* y la envía a `PC1` aplicando el mismo procedimiento que se acaba de describir, solo que a la inversa.

IP	ver+IHL 0x45| tos 0 | len 0x0054 id 0x1C46 | flags+offset 0x0000 ttl 64 | proto 1 (ICMP) | cksum src 20.0.1.6 | dst 20.0.1.5
ICMP	type 0 (Echo Reply) | code 0 | cksum id 0x1234 | seq 1

8.5.2. Escenario 2: dos saltos

Probemos algo un poco más interesante. Supongamos que `PC1` ejecuta ahora `ping -c1 26.0.3.3`. En este caso, el paquete debe atravesar 2 routers y 3 redes hasta `PC5`.

1. `PC1` construye el paquete IP con las direcciones correspondiente:

IP	ver+IHL 0x45| tos 0 | len 0x0054 id 0x2D57 | flags+offset 0x0000 ttl 64 | proto 1 (ICMP) | cksum src 20.0.1.5 | dst 26.0.3.3
ICMP	type 8 (Echo Request) | code 0 | cksum id 0x5678 | seq 1

2. `PC1` consulta su tabla de encaminamiento. Ninguna fila convencional es aplicable a la dirección destino, pero hay un router por defecto (`20.0.1.1`), así que realiza una entrega indirecta hacia él.
3. Como antes, necesita averiguar la dirección MAC de la interfaz *e0* del router, para lo cual envía una petición ARP con las siguientes direcciones:
   - IP origen: `20.0.1.5`, IP destino: `20.0.1.1` (`R1`).
   - MAC origen: `:A2`, MAC destino: `00:00:00:00:00:00`.
4. `R1` responde con la dirección MAC (`:AE`) en una respuesta ARP.
5. `PC1` puede ahora construir y enviar el paquete IP encapsulado en otra trama Ethernet.

Ethernet	dst :AE | src :A2 | type 0x0800 (IP)
IP	ver+IHL 0x45 | tos 0 | len 0x0054 id 0x2D57 | flags+offset 0x0000 ttl 64 | proto 1 (ICMP) | cksum src 20.0.1.5 | dst 26.0.3.3
ICMP	type 8 (Echo Request) | code 0 | cksum id 0x5678 | seq 1

   En este caso se puede comprobar que la dirección física destino (`:AE`) corresponde a `R1` mientras que la lógica (`26.0.3.3`) corresponde a `PC5`, es decir, se trata de una entrega **indirecta**.
6. La trama llega a `R1`, que la desencapsula y procesa el paquete IP.
7. Como el paquete no es para él (`26.0.3.3` $\neq$ `20.0.1.1`) procede a reenviarlo. Consulta su tabla de encaminamiento. La única fila aplicable es la tercera (`26.0.3.3` AND `255.255.255.0` == `26.0.3.0`) de modo que realiza una nueva entrega indirecta a `R2`.
8. El enlace `R1`–`R2` es serie y utiliza una encapsulación PPP. En este caso no se utiliza ARP porque no hay otro destino posible que el otro extremo del enlace. Tampoco hay direcciones MAC en este tipo de interfaz. La trama PPP será algo como:

PPP	addr 0xFF | control 0x03 | proto 0x0021 (IP)
IP	ver+IHL 0x45| tos 0 | len 0x0054 id 0x2D57 | flags+offset 0x0000 ttl 64 | proto 1 (ICMP) | cksum src 20.0.1.5 | dst 26.0.3.3
ICMP	type 8 | code 0 | cksum id 0x5678 | seq 1

9. `R2` recibe la trama PPP, la desencapsula y procesa el paquete IP. Tampoco es para él (`26.0.3.3` ≠ `20.4.4.2`), así que lo reenvía. La segunda fila de su tabla de encaminamiento es aplicable, e indica una entrega directa por la interfaz `e1`.
10. `R2` tiene que averiguar la dirección MAC del destino. Envía una petición ARP preguntando por `26.0.3.3` y `PC5` responde con su dirección: `:C4`.
11. `R2` puede ahora construir y enviar el paquete IP en una trama Ethernet con las siguientes direcciones:
    - MAC origen: `:C3`, MAC destino: `:C4`.
12. `PC5` recibe la trama, desencapsula el paquete IP y a su vez el mensaje ICMP. Como es un *Echo request*, procede a construir una respuesta *Echo reply* dirigida a `PC1`, que recorrerá el camino en sentido contrario.

8.6. Mensajes de control de interred (ICMP)

ICMP ya se presentó en § 5.7, y se ha utilizado a menudo con el comando `ping`, que emplea uno de sus tipos de mensajes; en este mismo capítulo han aparecido otros usos. Esta sección detalla los distintos mensajes y su finalidad [11]. Recordemos primero el formato general del mensaje:

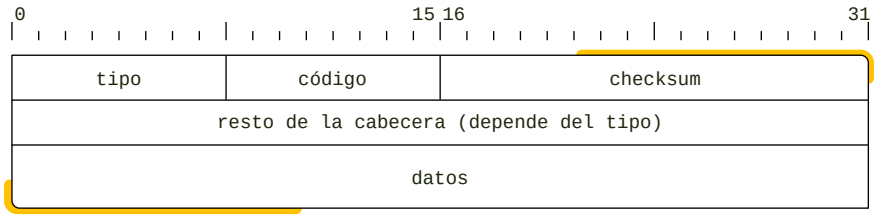


FIGURA 8.4: Formato del mensaje ICMP

Hay dos grupos de mensajes ICMP: los de notificación de errores y los de consulta/diagnóstico. El campo `tipo` indica el tipo de mensaje mientras que el campo `código` indica un subtipo. A continuación se describen los más comunes.

Los mensajes de notificación de errores son enviados cuando un router detecta un problema en el encaminamiento o entrega de un paquete[5]. Estos mensajes son enviados siempre al nodo origen del paquete que ha causado el problema. Es importante entender que el objetivo de estos mensajes es meramente informativo. IP es un protocolo esencialmente no confiable e ICMP no pretende cambiar eso. Del mismo modo que no hay garantías de que un paquete IP pueda ser entregado, tampoco las hay para un mensaje ICMP, de hecho los mensajes ICMP se encapsulan sobre paquetes IP. Algunos de los mensajes de notificación son los siguientes (se muestra entre corchetes el valor del campo `tipo`):

- Destino inalcanzable [3]
- Tiempo excedido [11]
- Problema en parámetro [12]
- Supresión al origen [4]
- Redirección [5]

Los mensajes ICMP de error incluyen como carga útil al menos la cabecera IP del paquete problemático y los primeros 8 bytes de la carga útil de aquel, que corresponde con los puertos origen y destino de un mensaje TCP o UDP que este estuviera transportando en el momento de producirse el error.

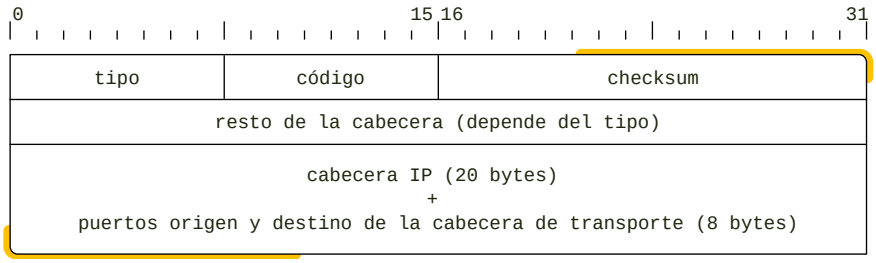


FIGURA 8.5: Carga útil de los mensajes ICMP de error

Esta información permite al SO del nodo origen determinar qué proceso es el responsable de ese paquete y así puede informar al programador mediante un error o excepción, para que a su vez el programa pueda identificar y tratar el problema.

Ni los routers ni los nodos generan nunca mensajes ICMP cuando el paquete problemático porta un mensaje ICMP de notificación de error. Hacerlo podría generar a su vez otro problema de entrega, provocando un bucle. Como ICMP Echo-request no es un mensaje de error, por regla general sí se envían mensajes ICMP de error si hay problemas al entregarlo.

[5]Ya han aparecido algunos casos en este mismo capítulo.

Los mensajes de consulta/diagnóstico, como su nombre indica, tratan de averiguar cierta información o comprobar el estado. Por eso estos mensajes vienen por pares petición-respuesta. Son los siguientes:

- Ping (petición [8]/respuesta [0])
- Marca de tiempo (petición [13]/respuesta [14])
- Información (petición [15]/respuesta [16])
- Máscara de subred (petición [17]/respuesta [18])
- Router (solicitud [10]/anunciamiento [9])

Los siguientes apartados los describen uno a uno.

8.6.1. Destino inalcanzable *(Destination unreachable)*

Es una notificación muy frecuente. Un router envía un mensaje *Destino inalcanzable* (`tipo`=3) cuando no puede entregar un paquete a su destino. Hay varios motivos por lo que eso puede ocurrir, y corresponden con los distintos valores para el campo `código`:

0. Red inalcanzable *(network unreachable)*. Lo utiliza el router si no encuentra una fila en su tabla de encaminamiento que pueda utilizar para reenviar el paquete.
1. Nodo inalcanzable *(host unreachable)*. El router tiene acceso a la red destino, pero no puede efectuar la entrega directa.
2. Protocolo inalcanzable *(protocol unreachable)*. El destino no soporta el protocolo encapsulado en el paquete IP.
3. Puerto inalcanzable *(port unreachable)*. El puerto destino en el nodo destino está cerrado.
4. Fragmentación necesaria *(fragmentation needed and DF set)*. El paquete necesita ser fragmentado, pero el origen ha solicitado que no se fragmente (ver § 8.7).
5. Fallo en la ruta en origen *(source route failed)*. No se ha podido aplicar la opción *source route* de IP[6].

Es sencillo reproducir algunos de estos errores porque tu propio sistema los genera en caso de problemas cuando intenta enviar tráfico. Puedes provocar un mensaje *nodo inalcanzable* enviando `ping` a una dirección IP que sabes que no está asignada a ningún nodo de la red. En el siguiente ejemplo asumimos que la IP de tu nodo es `192.168.0.37/24` y la del nodo que no existe es `192.168.0.66/24`. Cambia estas direcciones por las adecuadas en tu caso y pruébalo por ti mismo.

[6] *Source route* es una función de diagnóstico de IP con un uso muy marginal.

```
~$ ping -c 1 192.168.0.66
PING 192.168.0.66 (192.168.0.66) 56(84) bytes of data.
From 192.168.0.37 icmp_seq=1 Destination Host Unreachable
```

Si capturas el tráfico verás el mensaje ICMP en detalle. Como siempre eliminamos los campos irrelevantes para el propósito que nos ocupa:

```
1  ~# tshark -i any -f icmp -V
2  Internet Protocol Version 4, Src: 192.168.0.37, Dst: 192.168.0.37
3      Total Length: 112
4      Protocol: ICMP (1)
5      Source Address: 192.168.0.37
6      Destination Address: 192.168.0.37
7  Internet Control Message Protocol
8      Type: 3 (Destination unreachable)
9      Code: 1 (Host unreachable)
10     Internet Protocol Version 4, Src: 192.168.0.37, Dst: 192.168.0.66
11         Total Length: 84
12         Protocol: ICMP (1)
13         Source Address: 192.168.0.37
14         Destination Address: 192.168.0.66
15     Internet Control Message Protocol
16         Type: 8 (Echo (ping) request)
17         Code: 0
```

Fíjate que el mensaje ICMP se lo envía el sistema a sí mismo (`192.168.0.37`) porque ha sido el propio sistema el que ha detectado el problema. También se puede apreciar el mensaje ICMP Echo-request (con su cabecera IP) encapsulado en el mensaje de error (**líneas 10-17**).

El programador puede acceder a esta información desde el código. Así pues, un intento de conexión a un nodo inexistente eleva una excepción `socket.error` como consecuencia directa de que el SO reciba y procese el mensaje ICMP (Listado 8.1). Al ejecutar este programa aparece un mensaje similar a «[Errno 113] No route to host».

```
import socket

try:
    sock = socket.socket()
    sock.connect(('192.168.0.66', 1234))
except socket.error as e:
    print(e)
```

LISTADO 8.1: Captura del error «nodo inalcanzable» con Python

8.6.2. Tiempo excedido *(Time exceeded)*

Recuerda que los routers decrementan el campo TTL cuando procesan un paquete IP. Pues bien, si el valor de TTL llega a cero, el router descarta el paquete y envía un mensaje ICMP de este tipo con `código`=0, que significa

formalmente *tiempo de vida excedido en tránsito (time to live exceeded in transit)*.

Los nombres tanto del error como del campo de la cabecera IP *time to live* (tiempo de vida) al que está asociado, son una reminiscencia de la Internet primigenia cuando los routers tardaban aproximadamente un segundo en procesar cada paquete. El campo TTL expresaba (en segundos) la vida que aún le quedaba al paquete. Hoy en día hay routers que pueden procesar millones de paquetes por segundo y el significado práctico del campo TTL ahora es el número de saltos que el paquete tiene permitido dar aún. En esta línea y para ser coherente con el significado actual, en el diseño de IPv6 el campo equivalente pasó a llamarse `hop-count` (cuenta de saltos).

Si el campo `código`=1, el mensaje lo envía el nodo destino para indicar que ha agotado el tiempo límite en espera de recibir todos los fragmentos de un paquete. En este caso significa *tiempo excedido en el reensamblado (time to live exceeded in reassembly)*. El valor recomendado para ese tiempo límite es de 60 segundos [9], aunque por ejemplo en GNU/Linux es configurable mediante el parámetro del núcleo `net/ipv4/ipfrag_time`.

traceroute

El programa `traceroute` aprovecha este mecanismo para descubrir la ruta que sigue un paquete IP hacia su destino. `traceroute` comienza enviando tres mensajes ICMP *Echo*, o un segmento UDP o TCP encapsulado en un paquete IP con TTL=1. Eso provoca que el router local descarte el paquete, informe del error y con ello revele su dirección IP (porque aparece como dirección origen del mensaje de error). A continuación, `traceroute` repite el envío con TTL=2, obteniendo la dirección del segundo router en la ruta, y así sucesivamente hasta alcanzar el nodo destino.

Resultado de una ejecución de `traceroute` dirigida a `rediris.es` desde la ESI de Ciudad Real:

```
1  ~$ traceroute rediris.es
2  traceroute to rediris.es (130.206.13.20), 30 hops max, 60 byte packets
3   1  161.67.101.1 (161.67.101.1)  0.694 ms  1.015 ms  1.256 ms
4   2  172.16.160.5 (172.16.160.5)  1.153 ms  1.465 ms  1.710 ms
5   3  ro-vlan170.ctic.cr.red.uclm.es (172.16.160.22)  0.514 ms  0.517 ms  0.571 ms
6   4  GE1-2-0.EB-CiudadReal0.red.rediris.es (130.206.200.1)  0.803 ms  0.939 ms  1.073 ms
7   5  CLM.SO1-1-1.EB-IRIS4.red.rediris.es (130.206.250.133)  4.529 ms  4.568 ms  4.656 ms
8   6  XE3-0-0-264.EB-IRIS6.red.rediris.es (130.206.206.133)  4.806 ms  4.318 ms  4.332 ms
9   7  www.rediris.es (130.206.13.20)  4.456 ms  4.465 ms  4.518 ms
```

El primer resultado es para un TTL=1 (**línea 3**) y corresponde al router local de esa LAN (`161.67.101.1`). Al lado aparecen los tiempos RTT para cada uno de los tres paquetes enviados. Para cada router aparece primero

su nombre (si tiene) y después su dirección IP. Como se puede apreciar la ruta completa requiere 6 saltos, es decir, el paquete atraviesa 6 routers. La última entrada es el destino solicitado.

8.6.3. Problema en parámetro *(Parameter problem)*

Este mensaje de error lo envía un router o un nodo destino cuando detecta un valor incorrecto en la cabecera IP que le impide procesarla. El mensaje incluye un campo `puntero` que indica el campo de la cabecera en el que ha encontrado el problema.

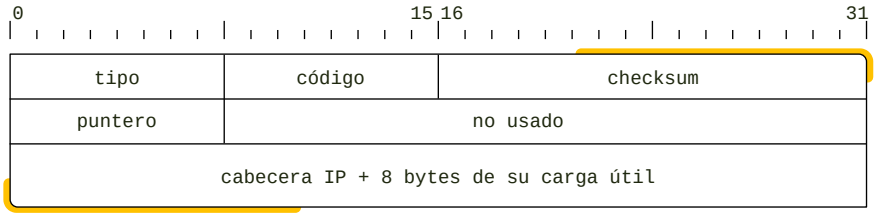


FIGURA 8.6: Mensaje ICMP de problema en parámetro

8.6.4. Supresión al origen *(Source quench)*

Este mensaje notifica un problema de exceso de tráfico, que puede estar relacionado con un problema de congestión si lo envía un router o como mecanismo de control de flujo si lo envía el destino. En todo caso, este mecanismo está obsoleto, no se implementa en sistemas modernos y no se recomienda su uso [15]. Trataremos este tema con más detalle en § 14.4.

8.6.5. Redirección *(Redirect)*

Este mensaje lo envía un router para indicar a un nodo origen que hay una ruta mejor para llegar a su destino. El mensaje incluye la dirección IP del router alternativo. Este mensaje no es técnicamente un error, es más bien informativo. En todo caso, el uso de este mensaje también está obsoleto porque podría ser explotado por un atacante malicioso en una técnica MITM.

Existe un parámetro del núcleo `net/ipv4/conf/all/accept_redirects` que permite configurar el comportamiento de los mensajes de redirección (por defecto se ignoran).

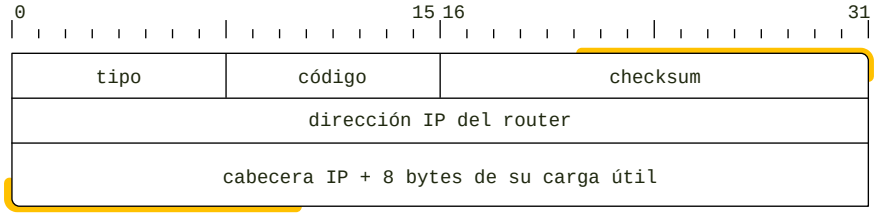


Figura 8.7: Mensaje ICMP de redirección

8.6.6. Ping *(Echo)*

`ping` ya se ha usado en varias ocasiones, incluso en algunas capturas (§ 5.7) de los mensajes ICMP que genera. El programa `ping` envía un mensaje ECHO request (`tipo`=8, `código`=0) a un nodo remoto. Éste al recibirlo envía de vuelta un mensaje ECHO reply (`tipo`=0, `código`=0).

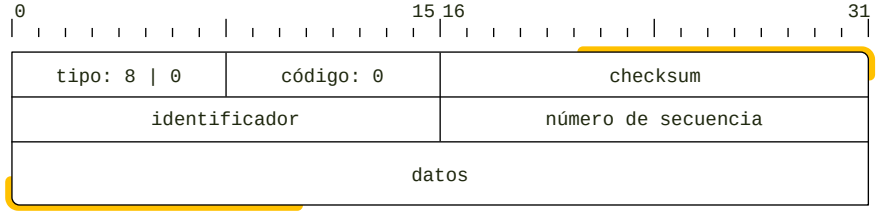


Figura 8.8: Mensaje ICMP Echo

El mensaje tiene un campo `identificador` y un campo `número de secuencia`. En una determinada ejecución del programa `ping` todos los mensajes de petición utilizan el mismo identificador y cada mensaje lleva un número de secuencia creciente empezando en 1. Captura del comando `ping example.net`:

```
~# tshark -f icmp
Capturing on 'eno1'
1 192.168.0.37 → 104.18.5.106 ICMP 98 Echo (ping) request  id=0x0002, seq=1/256,  ttl=64
2 104.18.5.106 → 192.168.0.37 ICMP 98 Echo (ping) reply    id=0x0002, seq=1/256,  ttl=57
3 192.168.0.37 → 104.18.5.106 ICMP 98 Echo (ping) request  id=0x0002, seq=2/512,  ttl=64
4 104.18.5.106 → 192.168.0.37 ICMP 98 Echo (ping) reply    id=0x0002, seq=2/512,  ttl=57
5 192.168.0.37 → 104.18.5.106 ICMP 98 Echo (ping) request  id=0x0002, seq=3/768,  ttl=64
6 104.18.5.106 → 192.168.0.37 ICMP 98 Echo (ping) reply    id=0x0002, seq=3/768,  ttl=57
```

Ese campo `id` hace posible dos o más ejecuciones de `ping` con el mismo origen y destino. Recuerda que ICMP se encapsula directamente sobre IP,

aquí no hay puertos que permitan multiplexar[7]. El segundo número tras '/' en el campo `seq` es el mismo valor, pero expresado en *little endian*.

La carga útil del mensaje de petición tiene por defecto una longitud de 56 bytes. Los primeros 16 bytes suelen ser una marca de tiempo y después 40 bytes de datos aleatorios o consecutivos, como confirma esta captura:

```
~# tshark -i lo -f icmp -V
Internet Control Message Protocol
  Type: 8 (Echo (ping) request)
  Code: 0
  Timestamp from icmp data: Apr 21, 2025 18:38:03.966021000 CEST
  Data (40 bytes)

0000  10 11 12 13 14 15 16 17 18 19 1a 1b 1c 1d 1e 1f   ................
0010  20 21 22 23 24 25 26 27 28 29 2a 2b 2c 2d 2e 2f    !"#$%&'()*+,-./
0020  30 31 32 33 34 35 36 37                          01234567

Data: 101112131415161718191a1b1c1d1e1f202122232425262728292a2b2c2d2e2f3031323334353637
```

Realmente no hay un campo `timestamp` en el mensaje ICMP, pero es tan habitual que la carga útil sea un *timestamp*, que `tshark` así lo interpreta. Los siguientes bytes empiezan con el valor `0x10` y van creciendo a partir de ahí hasta `0x37`.

El mensaje Echo reply que envía el receptor debe enviar de vuelta la misma carga útil que le envió el emisor —por eso se llama *eco*. El emisor puede entonces tomar la hora de llegada y restar la que él mismo envió en el mensaje de petición. Con ello obtiene el lapso de tiempo que ha transcurrido entre la petición y la respuesta. Ese es el tiempo de ida y vuelta o RTT (Round-Trip Time), que muestra la salida del programa con la etiqueta *time* el imprimir cada respuesta.

```
~$ ping -c2 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=117 time=4.25 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=117 time=4.27 ms

--- 8.8.8.8 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1001ms
rtt min/avg/max/mdev = 4.481/4.526/4.571/0.045 ms
```

8.6.7. Marca de tiempo *(Timestamp)*

El emisor envía su marca de tiempo en el mensaje de petición (`tipo`=13). El receptor envía un mensaje de respuesta (`tipo`=14) con tres marcas de tiempo:

- `origen`: La marca de tiempo que contenía el mensaje de petición.

[7] distinguir entre procesos

- `recepción`: El instante en que llegó el mensaje de petición.
- `transmisión`: El instante en que envió el mensaje de respuesta.

Como el mensaje Echo, incluye un campo `identificador` y `número de secuencia` para que el emisor pueda realizar la correspondencia entre petición y respuesta.

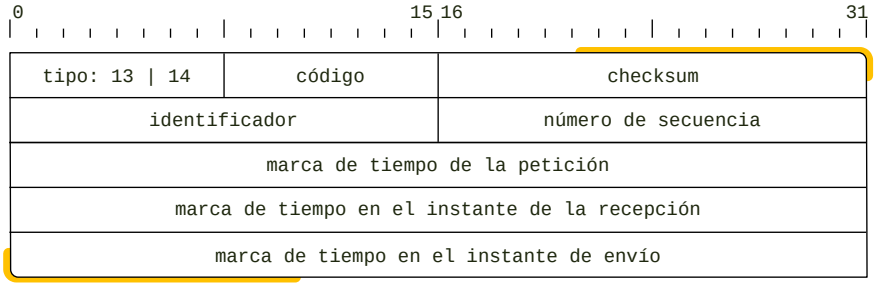


Figura 8.9: Mensaje ICMP Timestamp

Puedes enviar un mensaje de marca de tiempo con el programa `hping3`, incluido en el paquete del mismo nombre. Un ejemplo:

```
~# hping3 -c 4 --icmp --icmp-ts example.net
HPING example.net (eno1 23.215.0.141): icmp mode set, 28 headers + 0 data bytes
len=46 ip=23.215.0.141 ttl=50 id=42928 icmp_seq=0 rtt=91.9 ms
ICMP timestamp: Originate=63921874 Receive=63921915 Transmit=63921915
ICMP timestamp RTT tsrtt=92

len=46 ip=23.215.0.141 ttl=50 id=43150 icmp_seq=1 rtt=91.8 ms
ICMP timestamp: Originate=63922874 Receive=63922915 Transmit=63922915
ICMP timestamp RTT tsrtt=92

len=46 ip=23.215.0.141 ttl=50 id=43869 icmp_seq=2 rtt=91.7 ms
ICMP timestamp: Originate=63923875 Receive=63923915 Transmit=63923915
ICMP timestamp RTT tsrtt=91

len=46 ip=23.215.0.141 ttl=50 id=44269 icmp_seq=3 rtt=91.6 ms
ICMP timestamp: Originate=63924875 Receive=63924915 Transmit=63924915
ICMP timestamp RTT tsrtt=91

--- example.net hping statistic ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max = 91.6/91.7/91.9 ms
```

La salida contiene los mensajes de respuesta, con las tres marcas: `Originate`, `Receive` y `Transmit`. Las marcas `Receive` y `Transmit` son idénticas porque el tiempo que transcurre entre la recepción de la petición y el envío de la respuesta es despreciable.

El mensaje ICMP Timestamp permite calcular un RTT preciso sólo si los relojes de ambos nodos están sincronizados o el desfase (sesgo) es conocido.

Como esto es poco habitual, un uso que puede ser interesante es precisamente para realizar una sincronización o medir el sesgo. El emisor puede combinar el mensaje Echo para estimar y corregir el sesgo de su reloj, y de ese modo sincronizar su reloj con el del destino.

8.6.8. Información *(Information)* y máscara *(Address mask)*

Los mensajes de petición/respuesta de información (tipos 15 y 16), y petición/respuesta de máscara (tipos 17 y 18) constituían un mecanismo rudimentario para que un nodo pudiera averiguar la dirección IP o máscara asignada a ese nodo. Están obsoletos [16] y no se utilizan en la actualidad. El protocolo DHCP suple con creces esta necesidad.

8.6.9. Solicitud y anunciamiento de router

Estos mensajes forman parte del protocolo IRDP (ICMP Router Discovery Protocol) [17], y como sus nombres indican, sirven respectivamente para que un nodo pueda solicitar routers locales para poder salir de la LAN y para que un router pueda anunciare, ya sea como respuesta al mensaje de solicitud o de forma proactiva para informar de su presencia.

Como podrás suponer, es una funcionalidad en desuso ya que el protocolo DHCP cubre esta necesidad con creces. La sección § 10.2 explica con detalle la configuración automática de los parámetros de red.

8.7. Fragmentación

La función principal del router es interconectar redes, y en muchas situaciones esas redes pueden ser heterogéneas, es decir, estar basadas en tecnologías de enlace diferentes, como el caso de la Figura 8.1.

Uno de los problemas que surge al interconectar redes heterogéneas es la disparidad en los tamaños de trama. En concreto el problema aparece cuando se extrae la carga útil que llega en la trama entrante y se necesita encapsular en una trama de un tamaño menor para ser enviada por una interfaz diferente. Al tamaño máximo (en bytes) que puede transportar una trama (la carga útil) de una determinada tecnología se le llama MTU (Maximum Transmission Unit).

La MTU es una característica básica inherente a cualquier interfaz de red y tecnología de enlace, es decir, todas la interfaces de una determinada tecnología tienen siempre la misma MTU en cualquier lugar o instalación; es una característica dada por diseño y fabricación. En GNU/Linux lo puedes consultar con el comando `ip link show`. El siguiente ejemplo muestra que la interfaz `eno1` (una NIC Ethernet) tiene una MTU de 1500 bytes.

```
~$ ip link show eno1
2: eno1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP mode DEFAULT
  group default qlen 1000
  link/ether f8:5f:2a:c0:ff:ee brd ff:ff:ff:ff:ff:ff
```

Aunque es muy poco habitual y a pesar de lo dicho, el administrador puede cambiar la MTU de una interfaz. Salvo que haya una buena razón no es nada recomendable y desde luego nunca para configurar valores mayores que la MTU real del enlace. Provocaría todo tipo de problemas y errores difíciles de detectar.

```
~# ip link set dev eth0 mtu 1400
```

La discrepancia entre MTUs puede ocurrir, por ejemplo, cuando un paquete IP que viaja encapsulado en una trama Ethernet o WiFi (MTU=1500 bytes) llega a un router que lo debe enviar por un enlace serie (MTU=256 bytes). La solución a este problema es que el router **fragmente** (trocee) la carga útil del paquete IP original y cree nuevos paquetes IP (fragmentos) que quepan en las tramas del enlace de salida. A este proceso es a lo que se llama «fragmentación».

El problema no termina ahí. En su viaje hacía el destino, los fragmentos podrían tener que atravesar otras redes con MTU aún menores, con lo que el router correspondiente debería dividirlos a su vez en fragmentos más pequeños. Aunque la probabilidad es baja, podría ocurrir además que los fragmentos, que son a todos los efectos paquetes IP individuales, sigan eventualmente rutas distintas. Eso significa que, para un mismo flujo, podrían llegar al destino paquetes completos y fragmentos de distintos tamaños dependiendo de la ruta que ha seguido cada uno.

Por estos dos motivos: fragmentaciones sucesivas y rutas distintas, el «reensamblado», es decir, la unión de los fragmentos para obtener de nuevo el paquete original, solo lo puede realizar el destinatario. Es el único lugar donde es seguro que van a pasar todos los fragmentos.

La fragmentación es una tarea costosa que consume tiempo y recursos tanto en los routers como en el destino. El destino debe almacenar en memoria los fragmentos que va recibiendo hasta que llegue el último de ellos, y esto puede estar ocurriendo para varios datagramas en varios flujos al mismo tiempo. En muchas ocasiones es preferible evitar la fragmentación. Eso es posible si el nodo origen crea paquetes lo suficientemente pequeños como para quepan en la menor MTU de la ruta probable, que llamamos PMTU (Path MTU).

La cabecera del paquete IP tiene 4 campos relacionados con la fragmentación. Todos ellos están en la segunda palabra de 32 bits, es decir, la segunda fila tal como se suele representar.

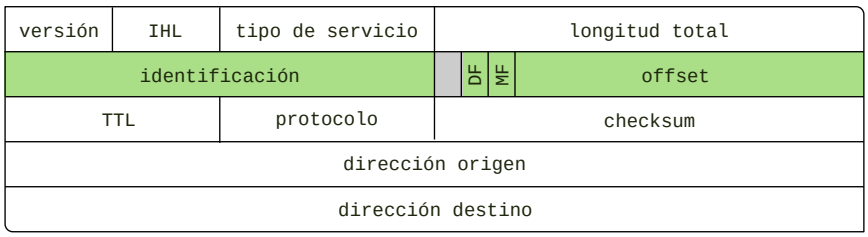


FIGURA 8.10: Campos relacionados con la fragmentación en la cabecera IP

Este es su significado, con más detalle:

- `identificación` *(identification)*: es un número único que identifica el paquete original. Todos los fragmentos creados a partir de un mismo paquete comparten el mismo número de identificación. De este modo, el destinatario puede determinar qué fragmentos corresponden con qué paquete original.
- Flag DF. Si está activo, indica que el paquete no debe ser fragmentado: DF (Don't Fragment).
- Flag MF. Si está activo significa que ese paquete no es el último fragmento, hay más: MF (More Fragments).
- `offset`: Es un número que indica la posición que ocupa la carga útil de ese fragmento respecto al paquete original. El primer fragmento siempre tiene un `offset`=0. Fíjate que la longitud de este campo es de solo 13 bits, mientras que el tamaño del paquete IP se expresa con un entero de 16 bits. Eso es porque este campo está expresado en múltiplos de 8 bytes, lo que lógicamente implica que los fragmentos no pueden tener cualquier tamaño, tiene que ser múltiplo de 8 bytes.

Con esto, se pueden distinguir cuatro situaciones al inspeccionar una cabecera IP:

1. Si `offset`=0 y el flag MF no está activo, se trata de un paquete completo no fragmentado.
2. Si `offset`=0 y el flag MF está activo, el paquete ha sido fragmentado y este es el primer fragmento.
3. Si `offset` es distinto de 0 y el flag MF está activo, el paquete ha sido fragmentado y este es un fragmento intermedio.
4. Si `offset` es distinto de 0 y el flag MF no está activo, el paquete ha sido fragmentado y este es el último fragmento.

Fíjate que cuando un paquete está fragmentado no es posible saber cuántos fragmentos lo componen. El destino los irá recibiendo y almacenando temporalmente, pero no sabrá cuantos son hasta que rellene todos los huecos.

El siguiente es un ejemplo de fragmentación con la topología de la Figura 8.11.

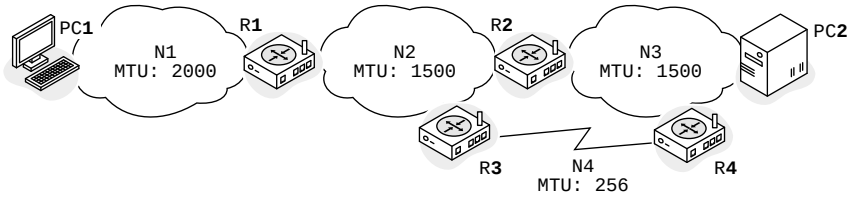


FIGURA 8.11: Topología para el ejemplo de fragmentación

Supongamos que `PC1` envía un paquete IP a `PC5` con una carga útil de 1980 bytes, lo que implica un tamaño total de 2000 bytes considerando que incluye una cabecera IP estándar sin opciones (20 bytes). Por regla general (como este caso) todo nodo evita crear paquetes mayores que la MTU de la red a la que está conectado, pues eso le obligaría a fragmentar ya desde el origen.

El valor para los campos relevantes sería el siguiente cuando el paquete sale a la red `N1`:

- Tamaño total: 2000 bytes.
- `identificación`: `0x1234` (ejemplo).
- Flag DF: 0 (fragmentación permitida).
- Flag MF: 0 (es el último fragmento).
- `offset`: 0 (es el primer fragmento).
- Tamaño de la carga útil: 1980 bytes.
- Rango de datos: 0-1979.

Es el primer y último fragmento, es decir, no está fragmentado (el primer caso de la lista anterior). Al llegar a `R1`, éste debe fragmentarlo para poder enviarlo a la red `N2` que requiere una encapsulación con MTU=1500 bytes, lo que implica que la carga útil máxima es de 1480 bytes (1500 - 20). Crea por tanto dos fragmentos:

- *Fragmento 0*
  - Tamaño total: 1500 bytes.
  - `identificación`: `0x1234`.
  - Flag MF: 1.
  - `offset`: 0.
  - Tamaño de la carga útil: 1480 bytes.

    - Rango de datos: 0–1479.
- *Fragmento 1*:
    - Tamaño total: 520 bytes.
    - `identificación`: `0x1234`.
    - Flag MF: 0.
    - `offset`: 185 (1480 / 8).
    - Tamaño de la carga útil: 500 bytes.
    - Rango de datos: 1480–1979.

Ahora supongamos que `R1` encamina el fragmento 0 por `R2`, que está conectado a otra red con MTU=1500 (`N3`), es decir, no requiere más fragmentación y llega tal cual al destino. Sin embargo, `R1` encamina el fragmento 1 por `R3` y este a su vez hacía `R4` hasta el destino. Eso obliga a `R3` a fragmentar de nuevo pues el tamaño de este segundo fragmento (520 bytes) supera el MTU del enlace serie `N4` (256 bytes). Restando la cabecera, la carga útil del paquete podría tener un máximo de 236 bytes (256 - 20). Pero 236 no es múltiplo de 8, así que el primer fragmento deberá tener 232 bytes de carga útil.

Los campos relevantes de estos nuevos fragmentos serían:

- *Fragmento 1.1*:
    - Tamaño total: 252 bytes.
    - `identificación`: `0x1234`.
    - Flag MF: 1.
    - `offset`: 185 (1480 / 8).
    - Tamaño de la carga útil: 232 bytes.
    - Rango de datos: 1480–1711.
- *Fragmento 1.2*:
    - Tamaño total: 252 bytes.
    - `identificación`: `0x1234`.
    - Flag MF: 1.
    - `offset`: 214 (1712 / 8).
    - Tamaño de la carga útil: 232 bytes.
    - Rango de datos: 1712–1943.
- *Fragmento 1.3*:
    - Tamaño total: 56 bytes.
    - `identificación`: `0x1234`.
    - Flag MF: 0.
    - `offset`: 243 (1944 / 8).
    - Tamaño de la carga útil: 36 bytes.
    - Rango de datos: 1944–1979.

La Figura 8.12 representa las cabeceras de los fragmentos que se han creado. En la primera fila aparece el tamaño total del paquete (cabecera + carga útil), en la segunda fila los campos relacionados con la fragmentación: `iden-`

`tificador`, flag DF y `offset`, y en la parte inferior la carga útil, en la que se representa un rango que numera los bytes empezando en 0.

Al entrar el paquete original en la red `N2` se crean los fragmentos 0 y 1, y posteriormente cuando el fragmento 1 entra en la red `N4` se crean a partir de él los fragmentos 1.1, 1.2 y 1.3. Por tanto al destino llegan 4 fragmentos: 0, 1.1, 1.2 y 1.3. Puedes comprobar que los rangos de datos representados en la carga útil son contiguos y coinciden con el paquete original.

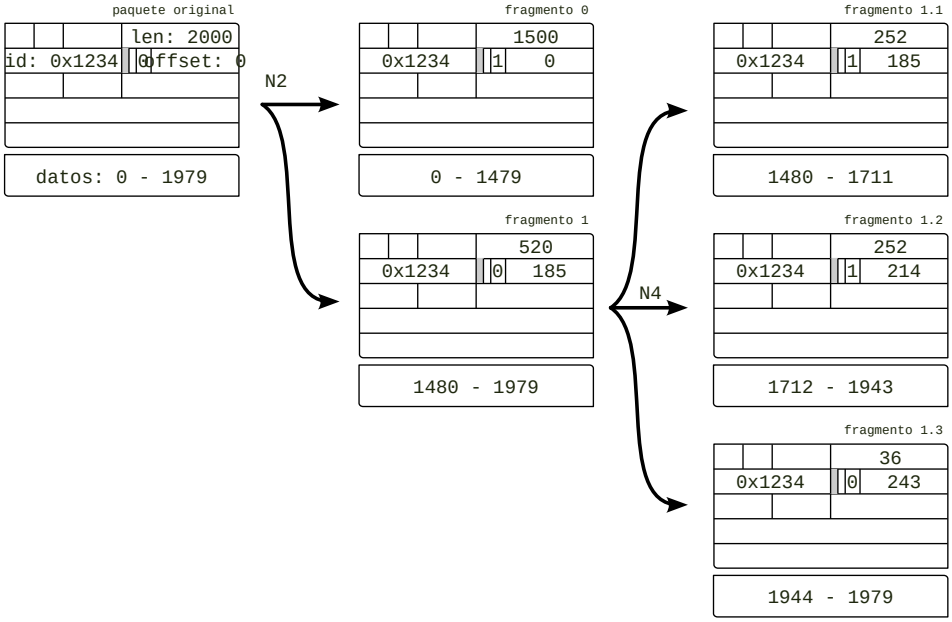


FIGURA 8.12: Ejemplo de fragmentación

La fragmentación también agrava la sobrecarga de cabeceras porque crea paquetes IP adicionales. Hemos pasado de un solo paquete IP a cuatro, es decir, 3 nuevas cabeceras IP, además de las cabeceras de las correspondientes tramas adicionales. Si por ejemplo esas tramas fuesen Ethernet, esta sobrecarga se puede cuantificar en $3 \times (14 + 20) = 102$ bytes.

Esto se puede llevar a la práctica con un pequeño laboratorio llamado `lab-frag`, basado en `docker` (`docker.io`), que recrea la topología trivial de la Figura 8.13:

Esta topología está especificada en el archivo `compose.yml` para `docker-compose`.

```
x-common: &common-settings
  privileged: true
  restart: "no"
```

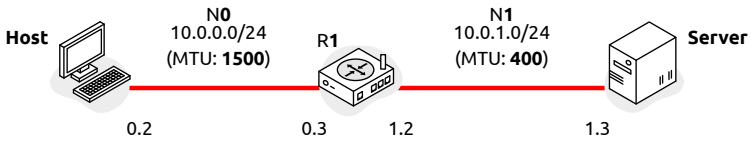


FIGURA 8.13: Topología del laboratorio de la fragmentación /lab-frag

```
  ulimits:
    nofile:
      soft: 100000
      hard: 200000

services:
  router:
    build: ./router
    image: lab-frag-router
    container_name: router
    hostname: router
    <<: *common-settings
    networks:
      N0: { ipv4_address: 10.0.0.3 }
      N1: { ipv4_address: 10.0.1.2 }

  server:
    build: ./server
    image: lab-frag-server
    container_name: server
    <<: *common-settings
    networks:
      N1: { ipv4_address: 10.0.1.3 }

networks:
  N0:
    ipam: { config: [{subnet: 10.0.0.0/24}] }
  N1:
    driver_opts: { com.docker.network.driver.mtu: 400 }
    ipam: { config: [{subnet: 10.0.1.0/24}] }
```

LISTADO 8.2: Descripción del laboratorio de fragmentación /lab-frag/compose.yml

Con este tenemos un router y un nodo `Servidor` en la red `N1` que tiene una MTU de 400 bytes. El nodo `host` es tu computador real. Lo único que debes tener en cuenta es que previamente a la ejecución del ejemplo tu dirección de red no esté dentro de `10.0.0.0/8` ni tengas nada en tu tabla de encaminamiento que lleve a esa red. De otro modo tendrías un conflicto y no funcionaría.

En el directorio `lab-frag` ejecuta `make` para arrancar el laboratorio. Una vez en marcha puedes ejecutar `ping` y `traceroute` para comprobar que puedes llegar al nodo `Server`.

```
~$ ping -c1 10.0.1.3
PING 10.0.1.3 (10.0.1.3) 56(84) bytes of data.
64 bytes from 10.0.1.3: icmp_seq=1 ttl=63 time=0.049 ms

--- 10.0.1.3 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.049/0.049/0.049/0.000 ms
~$ traceroute
traceroute to 10.0.1.3 (10.0.1.3), 30 hops max, 60 byte packets
 1  10.0.0.3 (10.0.0.3)  0.494 ms  0.409 ms  0.372 ms
 2  10.0.1.3 (10.0.1.3)  0.349 ms  0.316 ms  0.275 ms
```

Este primer *ping* es un mensaje muy pequeño (64 bytes) y no se ha visto afectado por la fragmentación. Ahora puedes enviar un mensaje mucho mayor (700 bytes de carga útil) con `ping -c1 -s700 10.0.1.3` para forzar a `R1` a fragmentar.

Si ejecutas `tshark` en `Server` y vuelves a ejecutar `ping` podrás ver los fragmentos.

```
~$ docker exec server tshark -f icmp
Capturing on 'eth0'
1  10.0.0.1 ? 10.0.1.3  IPv4 410 Fragmented IP protocol (proto=ICMP 1, off=0, ID=073c)
2  10.0.0.1 ? 10.0.1.3  ICMP 366 Echo (ping) request  id=0x0008, seq=1/256, ttl=63
3  10.0.1.3 ? 10.0.0.1  IPv4 410 Fragmented IP protocol (proto=ICMP 1, off=0, ID=d41e)
4  10.0.1.3 ? 10.0.0.1  ICMP 366 Echo (ping) reply    id=0x0008, seq=1/256, ttl=64
```

Los mensajes 1 y 2 corresponden a la petición ICMP Echo. En el primer mensaje `tshark` ofrece información del fragmento (`offset` y `identificación`) porque en ese momento obviamente no tiene toda la información. Al llegar el segundo fragmento ya puede reensamblar y determina que es un mensaje Echo. Los mensajes 3 y 4 son la respuesta que envía `Server`, que necesariamente tiene que ser del mismo tamaño, y por esa razón el propio nodo se ve obligado a fragmentar *en origen*.

Puedes hacer una captura más detallada para ver el valor de los campos de la cabecera IP. Mostramos aquí solo la información relevante de las cabeceras IP de los 2 primeros mensajes (el Echo request).

```
~$ docker exec server tshark -f icmp -V
Frame 1:
Internet Protocol Version 4, Src: 10.0.0.1, Dst: 10.0.1.3
    Total Length: 396
    Identification: 0x85b8 (34232)
    001. .... = Flags: 0x1, More fragments
    ...0 0000 0000 0000 = Fragment Offset: 0
    Source Address: 10.0.0.1
    Destination Address: 10.0.1.3

Frame 2:
Internet Protocol Version 4, Src: 10.0.0.1, Dst: 10.0.1.3
    Total Length: 352
```

```
    Identification: 0x85b8 (34232)
    000. .... = Flags: 0x0
    ...0 0000 0010 1111 = Fragment Offset: 376
    Source Address: 10.0.0.1
    Destination Address: 10.0.1.3
    [2 IPv4 Fragments (708 bytes): 1(376), 2(332)]
        [Frame: 1, payload: 0-375 (376 bytes)]
        [Frame: 2, payload: 376-707 (332 bytes)]
        [Fragment count: 2]
        [Reassembled IPv4 length: 708]
        [Reassembled IPv4 data: [...]]
```

Es interesante señalar que el tamaño del primer fragmento es 376 y no 400, ya que 380 (400-20) no es múltiplo de 8. Fíjate en estos campos:

- Mensaje 1
  - `longitud total`: 396 bytes (376 + 20).
  - `identificación`: 0x85b8.
  - Flag MF: 1.
  - `offset`: 0.
- Mensaje 2
  - `longitud total`: 352 bytes (332 + 20).
  - `identificación`: 0x85b8.
  - Flag MF: 0.
  - `offset`: 101111=47 (376/8).

Además en el último fragmento `tshark` muestra información detallada del reensamblado. Indica el tamaño total del paquete original (700) y el de los fragmentos y el rango de los datos de ambos fragmentos.

```
[2 IPv4 Fragments (708 bytes): 1(376), 2(332)]
   [Frame: 1, payload: 0-375 (376 bytes)]
   [Frame: 2, payload: 376-707 (332 bytes)]
```

Puedes probar a enviar *ping* de otros tamaños o cambiar la MTU en el archivo `compose.yml`, y comprobar los resultados capturados en cada caso.

8.7.1. No fragmentar

Existen algunas situaciones en las que el origen necesita evitar la fragmentación, para lo cual marca el flag DF. Algunas de estas situaciones son:

- El destino es un dispositivo empotrado con capacidades muy limitadas y sin capacidad para reensamblar fragmentos.
- La ruta atraviesa una VPN o túnel. La fragmentación rompería la encapsulación.
- El flujo utiliza algún protocolo de tiempo real como RTP o VoIP en el que la fragmentación introduciría latencia y jitter muy perjudicial.

Cuando un router recibe un paquete marcado con el bit DF, descarta el paquete y envía un mensaje ICMP de tipo *Destination unreachable* y código *Fragmentation Needed* al nodo origen. El mensaje de error indica el tamaño máximo de la carga útil que puede aceptar la interfaz de salida de ese router.

Es sencillo provocar esta situación con el comando `ping` con los argumentos `-M do`, que fija el flag DF, y `-s`, que indica el tamaño de la carga útil. Para el caso de una interfaz Ethernet o WiFi fijamos una carga útil de 1472 bytes, que sumando la cabecera ICMP (8 bytes) y la cabecera IP (20 bytes) da un total de 1500 bytes, que es la MTU de esa tecnología. El comando por tanto sería:

```
~$ ping -c 1 -M do -s 1472 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 1472(1500) bytes of data.
From 172.20.21.254 icmp_seq=1 Frag needed and DF set (mtu = 1370)
```

Si pones en marcha una captura antes de ejecutar el comando anterior recibirás algo similar a lo siguiente. Se omiten los campos poco relevantes:

```
~# tshark -f "icmp" -V
Internet Protocol Version 4, Src: 172.20.21.254, Dst: 192.168.0.37
    Total Length: 576
    Time to Live: 62
    Protocol: ICMP (1)
    Source Address: 172.20.21.254
    Destination Address: 192.168.0.37
Internet Control Message Protocol
    Type: 3 (Destination unreachable)
    Code: 4 (Fragmentation needed)
    MTU of next hop: 1370
    Internet Protocol Version 4, Src: 192.168.0.37, Dst: 8.8.8.8
        Total Length: 1500
        Identification: 0x0000 (0)
        010. .... = Flags: 0x2, Don't fragment
        ...0 0000 0000 0000 = Fragment Offset: 0
        Time to Live: 62
        Protocol: ICMP (1)
        Source Address: 192.168.0.37
        Destination Address: 8.8.8.8
    Internet Control Message Protocol
        Type: 8 (Echo (ping) request)
        Code: 0
        [...]
```

Fíjate que el mensaje ICMP que informa del error incluye la cabecera IP y parte de la carga útil (el mensaje ping) que provocó el problema. También puedes comprobar que el mensaje de error lo está enviando `172.20.21.254`, que es el router que no ha podido reenviar el paquete. En concreto indica que el enlace saliente al que trataba de entregar el paquete tiene una MTU de 1370 bytes.

8.7.2. Descubrimiento de MTU

El mensaje ICMP «*fragmentation needed*» (§ 8.6.1) se puede utilizar para descubrir el mayor valor de MTU que permite la ruta hasta un determinado destino. El origen puede empezar enviando un paquete *ping* con un tamaño igual a la MTU de su interfaz de salida y el flag DF activa. Si recibe un mensaje ICMP de este tipo, envía un nuevo mensaje del tamaño indicado en el error. Así sucesivamente hasta recibir la respuesta a *ping* por parte del destino. Este mecanismo se denomina PMTUD (Path MTU Discovery). Indirectamente, también determina el valor de MSS que se utilizará en las conexiones TCP.

Es posible consultar o modificar si el SO aplica PMTUD con el parámetro del núcleo `ip_no_pmtu_disc`. Esa abreviatura se puede leer como «no IP PMTU discovery», es decir, un valor de 0 significa que el descubrimiento está habilitado, que es la situación por defecto.

```
~$ cat /proc/sys/net/ipv4/ip_no_pmtu_disc
0
```

Como algunos routers o incluso el destino podrían no procesar mensajes ICMP, existe una estrategia diferente basada en TCP llamada PLPMTUD (Packetization Layer Path MTU Discovery) o «*TCP MTU probing*». También es posible elegir qué estrategia PMTUD se ha de utilizar con la variable `tcp_mtu_probing`.

```
~$ cat /proc/sys/net/ipv4/tcp_mtu_probing
0
```

Esta variable puede tomar 3 valores:

- 0: Utiliza siempre la estrategia basada en ICMP (PMTUD clásico).
- 1: Inicialmente utiliza la estrategia basada en ICMP, pero si no recibe respuestas, cambia a PLPMTUD.
- 2: Utiliza siempre PLPMTUD.

Por último, el socket también permite configurar la fragmentación y el comportamiento de PMTUD para un flujo específico. Se realiza por medio de la opción `IP_MTU_DISCOVER`. Puede tomar 6 valores con los siguientes significados:

- `IP_PMTUDISC_DONT (0)`: No realizar descubrimiento de PMTU y permitir la fragmentación.
- `IP_PMTUDISC_WANT (1)`: Intenta PMTUD clásico. Si no funciona, permite la fragmentación.
- `IP_PMTUDISC_PROBE (2)`: Aplicar PMTUD clásico, nunca fragmentar.
- `IP_PMTUDISC_DO (3)`: Aplicar PLPMTUD.

- `IP_PMTUDISC_INTERFACE (4)`: Utilizar la MTU de la interfaz.
- `IP_PMTUDISC_OMIT (5)`: Como la anterior, pero permite fragmentación si el tamaño excede la MTU de la interfaz.

Por ejemplo, puedes utilizar el siguiente código:

```
import socket
IP_MTU_DISCOVER = 10
IP_PMTUDISC_DO = 2

sock = socket.socket()
sock.setsockopt(socket.IPPROTO_IP, IP_MTU_DISCOVER, IP_PMTUDISC_DO)
```

La fragmentación es costosa y conviene evitarla cuando sea posible. Por eso la mayoría de los SO activan PMTUD por defecto y ajustan el MSS de las conexiones TCP al valor descubierto. El propio origen genera paquetes que nunca llegan a fragmentarse. Esta estrategia también evita un problema adicional: si se pierde/corrompe uno solo de los fragmentos de un paquete, el destino no puede completar el reensamblado y termina descartando todos los fragmentos recibidos, lo que en el caso de TCP obliga a retransmitir el segmento completo. IPv6 lleva esta idea al extremo y elimina por completo la fragmentación en los routers intermedios, dejando esa responsabilidad exclusivamente al nodo origen.

Y ¿qué más?

Este capítulo ha abordado una de las funciones clave de cualquier interred IP: llevar paquetes desde un origen a un destino remoto atravesando múltiples redes con tecnologías heterogéneas y probablemente incompatibles. Aquí se ve el papel *homogeneizador* del protocolo IP y la importancia crucial de los routers como elementos de interconexión. También se ha tratado el protocolo ICMP y el problema que supone la disparidad de tamaños de trama propios de cada tecnología de enlace. En el capítulo 9 complementaremos toda esta funcionalidad estudiando como una interred puede descubrir automáticamente las rutas óptimas a cada posible destino gracias a los algoritmos y protocolos de encaminamiento dinámico.

Capítulo 9

Encaminamiento dinámico

Al terminar este capítulo, entenderás:

- La necesidad y el funcionamiento del encaminamiento dinámico.
- Qué son los sistemas autónomos y su importancia en el funcionamiento de Internet.
- Cómo funcionan los algoritmos de encaminamiento vector-distancia, estado de enlace y vector ruta.
- Las nociones básicas de los protocolos RIP y OSPF.

Cuando la interred es muy pequeña (un par de routers), las tablas de encaminamiento las puede definir y mantener manualmente el administrador. Tanto si las define «de cabeza» como si utiliza algún programa de vez en cuando. Llamamos a eso «encaminamiento estático».

El problema es que la red tiene la mala costumbre de cambiar de vez en cuando. Algunos de esos cambios pueden ser intencionados, como instalar un nuevo router, pero la mayoría son más imprevisibles, como un equipo de comunicaciones que se avería o un enlace que falla. Si el administrador decide el contenido de las tablas, él es el responsable de modificarlas para que encajen con los cambios.

En cuanto la interred crece un poco, y esos cambios empiezan a producirse de forma habitual, mantener las tablas manualmente se convierte en una tarea compleja y propensa a errores. Se necesita una forma de detectar cambios en la topología y en las condiciones de la red, y algoritmos que puedan adaptar las tablas de encaminamiento inmediata y automáticamente de acuerdo a esos cambios. En eso consiste el encaminamiento dinámico y su principal característica es que es **adaptativo**, es decir, detecta los cambios en la red y modifica las tablas de encaminamiento para adecuarse a la nueva situación.

Por contra, con encaminamiento estático, las tablas no cambian automáticamente, no hay nada que detecte los cambios, calcule nuevas rutas ni modifique las tablas.

La tabla de encaminamiento no es diferente en un caso u otro. De hecho no es raro que una misma tabla contenga filas especificadas de forma manual con otras obtenidas por un protocolo de encaminamiento dinámico, o incluso varios simultáneamente.

9.1. Algoritmos y protocolos de encaminamiento

Un algoritmo de encaminamiento es utilizado por un grupo de routers que colaboran para deducir toda o parte de la topología de la subred que los conecta. Con la información que es capaz de recabar por sí mismo y la que los otros le pueden proporcionar, cada router calcula el mejor modo para llevar cada paquete a su destino. Con esa información decide el nuevo contenido de su tabla de encaminamiento.

Cuando se aplica encaminamiento dinámico, los routers colaboran compartiendo información sobre ellos mismos y sobre lo que han aprendido. Obviamente esta información viaja dentro de mensajes, por lo que, asociados a los algoritmos, existen también protocolos de encaminamiento, con sus correspondientes formatos de mensaje y patrones de intercambio.

Estos protocolos definen cómo los routers intercambian información sobre la topología y cómo la utilizan para calcular las rutas óptimas. Pero del uso del protocolo y del hecho de que se comuniquen mediante mensajes deriva una limitación importante: existe cierto grado de obsolescencia, y por tanto de inconsistencia, en la información disponible.

Este efecto se denota diciendo que «no existe un reloj global», es decir, que es físicamente imposible sincronizar las acciones de componentes que se comunican mediante mensajes. Es el caso de los algoritmos de encaminamiento dinámico, pero también el de cualquier otra aplicación distribuida. No hay forma de que los routers envíen (y mucho menos reciban) la información topológica exactamente en el mismo instante. Esto implica que los routers trabajan siempre con datos eventualmente obsoletos. A determinado nivel de precisión (quizá centésimas o milésimas de segundo) tampoco pueden saber cuan obsoletos son los datos recibidos, ni las diferencias entre mensajes recibidos de orígenes diferentes. Esto debe considerarse en el diseño de los algoritmos y protocolos para que, a pesar de ello, las decisiones sean lo suficientemente robustas.

9.2. Sistemas autónomos

Antes de continuar, debemos abordar el concepto de *sistema autónomo* (AS). Un AS es un conjunto de routers y enlaces que pertenecen o están bajo el control de una misma entidad administrativa.

Es habitual que grandes empresas, gobiernos, universidades, proveedores de acceso a Internet (ISP) tengan su propio AS. Las empresas más grandes pueden tener varios, como es el caso de Google o Amazon.

Cada AS se identifica con un número único de 16 bits (ASN) que, como es habitual, es asignado por la IANA y gestionado por los RIR. Con el crecimiento de Internet, en 2007 se crearon los ASN de 32 bits. Actualmente ambos tipos de identificadores coexisten. Se han asignado más de 100 000 ASN públicos entre ambos tipos. Además existe un rango de ASN privados que no se anuncian a Internet, y que se utilizan por las organizaciones para sus redes internas.

Los AS se interconectan entre sí por medio de un tipo especial de router que se denomina *pasarela de borde* (*border gateway*) y que es capaz de encaminar paquetes a través de varios AS. La definición más precisa de Internet sería decir que es una colección de AS interconectados.

La existencia de los AS da lugar a dos tipos diferentes de encaminamiento dinámico:

- **Encaminamiento interno** designa los algoritmos y protocolos de pasarela interior o IGP (Interior Gateway Protocol). Se refiere al encaminamiento de paquetes dentro de un mismo AS. Como cada AS es una unidad independiente, cada administrador puede decidir el protocolo de encaminamiento IGP que más le convenga, así como los detalles de su configuración concreta.
- **Encaminamiento externo** designa los algoritmos y protocolos de pasarela exterior o EGP (Exterior Gateway Protocol). Se refiere al encaminamiento de paquetes entre AS independientes. A diferencia del interno, en este caso, todos los sistemas autónomos deben utilizar el mismo protocolo de encaminamiento.

9.3. Tipos de encaminamiento

Aparte de los tipos estático/dinámico e interno/externo, podemos considerar otros dos tipos de encaminamiento que no disponen de información topológica ni calculan rutas.

- Inundación. Consiste esencialmente en realizar copias de los paquetes recibidos y enviarlas por algunas o todas las interfaces del router (excluyendo la de recepción). Lógicamente, este método es extremadamente ineficiente, pero puede ser útil cuando no se dispone de ninguna información topológica.
- Aleatorio. Se plantea como una optimización de la inundación. Consiste en elegir una o varias interfaces de salida al azar por las que enviar copias de los paquetes recibidos. Hay variantes que puede utilizar información local al router (como la carga de los enlaces) para ponderar la probabilidad de elegir una u otra interfaz.

Según todo lo anterior podemos clasificar los algoritmos de encaminamiento según varios criterios:

- Según el ámbito respecto del AS: interno o externo.
- Según su capacidad para adaptarse a los cambios: estático, dinámico.
- Según la disponibilidad de información topológica: inundación, aleatorio.
- Dentro de los algoritmos dinámicos internos: vector distancia y estado de enlace.
- Dentro de los algoritmos dinámicos externos: vector ruta.

La siguiente tabla resume esta clasificación y los protocolos más representativos:

Ámbito	**Tipo**	**Algoritmos**	**Protocolos**
Interior	Estático	—	—
	Dinámico	Vector distancia Estado de enlace	RIP, EIGRP OSPF, IS-IS
	Inundación	—	—
	Aleatorio	—	—
Exterior	Dinámico	Vector-ruta	BGP

CUADRO 9.1: Clasificación de protocolos de encaminamiento

Las siguientes secciones tratan estos algoritmos.

9.4. Topologías de referencia: Delta

Para estudiar los algoritmos de encaminamiento interno, utilizaremos unas topologías sencillas y concretas que llamaremos «delta», porque visualmente forman triángulos. Así Delta-1 forma 1 triángulo, Delta-2 forma 2, etc.

Considerando las misma topologías será más fácil entender las diferencias entre algoritmos y comparar más fácilmente sus ventajas y desventajas. La Figura 9.1 representa esas topologías.

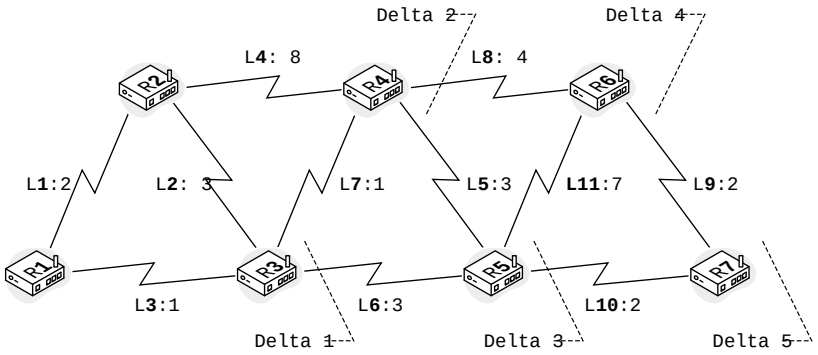


FIGURA 9.1: Topologías «Delta»

Aunque estas topologías incluyen detalles como la tecnología y tipo de enlaces (serie, en este caso), eso no es importante para el algoritmo de encaminamiento. Lo relevante es el grafo equivalente. En este, los nodos representan a los routers y las aristas a los enlaces. Esto se puede aplicar a cualquier red. Las redes a las que los routers dan acceso tampoco son relevantes para el cálculo de rutas dado que lo importante es poder llegar hasta el router que proporciona acceso a cada red.

Por la misma razón, para el algoritmo tampoco son importantes las direcciones de los dispositivos. Verás que cuando se estudian los algoritmos desde un enfoque teórico la columna «destino» de las tablas de encaminamiento no contiene direcciones de red como habitualmente, sino simplemente los nombres de los routers.

Por ejemplo, el grafo equivalente a la topología Delta-5 sería el de la Figura 9.2. Como en el original, los enlaces del grafo indican su coste. En este ejemplo, se utiliza una métrica de retardo, pero podría ser cualquier otra y a efectos del algoritmo no es relevante. El objetivo siempre es minimizar el coste total de la ruta.

9.5. Redundancia

Estas topologías de referencia cuentan con varios caminos para llegar a cualquier destino desde cualquier origen. Las redes e interredes suelen diseñarse de este modo porque explota una gran ventaja que proporciona la conmu-

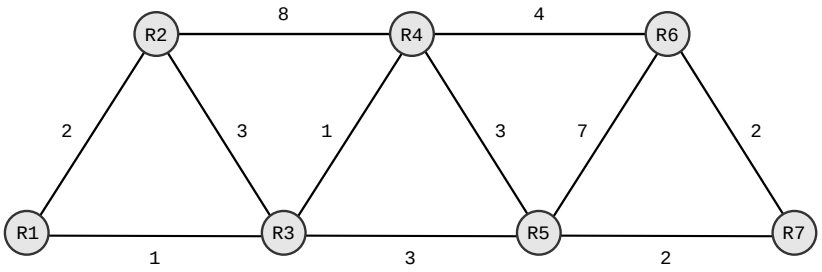


FIGURA 9.2: Grafo equivalente de la topología Delta-5

tación de paquetes: la **redundancia**. Disponer de caminos alternativos es la base para proporcionar tolerancia a fallos y balanceo de carga.

La *tolerancia a fallos* es la capacidad de la red[1] para seguir funcionando —llevando paquetes a sus destinos— aún en presencia de fallos. En muchas situaciones, las prestaciones se verán afectas, porque quizá el camino óptimo no esté disponible a causa del fallo[2], pero esencialmente la red seguirá haciendo su trabajo. Por el modo en que funciona la conmutación de paquetes es muy probable que el servicio no llegue a interrumpirse en ningún momento y los usuarios no noten nada en absoluto. Del mismo modo, si el problema es temporal, la red volverá a funcionar con normalidad cuando se resuelva. En estas situaciones se dice que el servicio proporciona «transparencia de fallos».

El *balanceo de carga* consiste en distribuir el tráfico (la carga de la red) entre los recursos disponibles (routers y enlaces en este caso) evitando la saturación de unos mientras otros están infrautilizados. El balanceo de carga puede ser complejo y puede requerir algoritmos capaces de manejar múltiples variables, criterios y estimaciones.

Los algoritmos de encaminamiento dinámico sacan partido de la redundancia y proporcionan tolerancia a fallos y, los más sofisticados, también balanceo de carga.

9.6. La ruta más corta

Todos los algoritmos de encaminamiento tratan de encontrar la mejor ruta (la óptima) desde un nodo origen a un nodo destino. Pero la «mejor ruta» o incluso aunque hablemos de la «ruta más corta» no es necesariamente la ruta que recorre la menor distancia. La «mejor ruta» es la que tiene menor

[1] O cualquier otro sistema

[2] En ingeniería se habla de «modo degradado» o «degradación gradual» *(graceful degradation)*

coste. El coste de la ruta normalmente se calcula como suma del coste de todos los enlaces que la forman. El coste del enlace puede ser cualquier propiedad medible que el administrador considere relevante. Esta propiedad determina la 'métrica' del algoritmo. La métrica más sencilla es el número de saltos, pero hay muchas otras: retardo, ancho de banda, fiabilidad, coste económico, etc., y también combinación de ellas.

Algunas de estas métricas no cambian con el tiempo. Una ruta de 5 saltos seguirá teniendo 5 saltos independientemente de que la red esté libre o saturada, es decir, no depende de la carga. Otras, como el retardo o el ancho de banda sí dependen; incluso el propio algoritmo de encaminamiento podría influir en esas propiedades. Por ejemplo, si existe un enlace con bajo retardo y el algoritmo la prioriza, es probable que la carga adicional eleve su valor haciendo que el propio algoritmo la considere menos adecuada en el futuro inmediato. Este es un efecto muy perjudicial porque afecta a la convergencia del algoritmo. Lo deseable es que el algoritmo mantenga las misma rutas a menos que ocurran cambios en la topología.

El cálculo de la ruta óptima se basa en una idea sencilla: el «principio de optimalidad». Este principio dice que si existe una ruta óptima entre el nodo `A` y el `C`, que pasa por `B`, entonces la ruta entre `B` y `C` también es necesariamente óptima. En esta idea se fundamentan los algoritmos de Dijkstra, Bellman-Ford y otros, que son los más utilizados en encaminamiento dinámico.

El algoritmo Dijkstra [18] calcula el camino más corto (de menor coste) entre dos nodos de un grafo no dirigido y ponderado, es decir, un grafo cuyas aristas son transitables en ambas direcciones y tienen costes asociados a una métrica arbitraria. La aplicación general del algoritmo al grafo completo calcula la ruta más corta a todos los nodos. Como esas rutas tienen partes comunes, el resultado es el árbol de rutas más cortas (SPT) para el origen considerado. Por ejemplo, para la topología Delta-3, el árbol de rutas más cortas desde `R1` sería el de Figura 9.3.

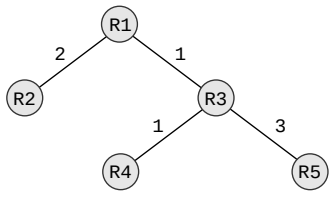


Figura 9.3: Árbol de rutas más cortas (SPT) para `R1` en la topología Delta-3

De manera resumida, el algoritmo de Dijkstra hace lo siguiente:

1. Inicializa el coste a todos los nodos a infinito, excepto el nodo origen, que se fija en 0.
2. Marca todos los nodos como no visitados.
3. Selecciona el nodo no visitado alcanzable al menor coste.
4. Calcula el coste acumulado de todos sus nodos vecinos y lo marca como visitado. Si el coste calculado fuera menor que el coste previo, lo actualiza.
5. Repite los pasos 3 y 4 hasta que todos los nodos hayan sido visitados o no queden nodos accesibles.

En el repositorio de ejemplos puedes encontrar una implementación del algoritmo Dijkstra (/`dijkstra/dijkstra.py`) y un notebook que lo aplica a un ejemplo (/`dijkstra/dijkstra.ipynb`).

9.7. Vector-distancia

En el algoritmo vector-distancia *(vector-distance routing)* inicialmente cada router solo tiene información sobre sus vecinos (routers directamente conectados). Esa información, que consiste esencialmente en la dirección/identificador del vecino y la distancia/coste hasta él, se coloca en una tabla llamada *vector-distancia*. La obtención de esta información es lo que llamamos *inicialización*. Una vez que el algoritmo empieza a trabajar cada router envía su vector solo a los vecinos, que la utilizan para actualizar sus propios vectores-distancia y, a partir de estos, sus tablas de encaminamiento. A esa transferencia de vectores es a lo que llamamos una *iteración* del algoritmo, que ocurre periódicamente o cuando se detecta un cambio. La inicialización no se considera una iteración puesto que no hay propagación de vectores-distancia, y en algunos protocolos, como RIP, incluso se define manualmente, con lo que el algoritmo ni tan siquiera se está ejecutando.

Este es un ejemplo trivial con 4 routers empleando una métrica de saltos:



FIGURA 9.4: Topología «lineal» de 4 routers

El Cuadro 9.2 muestra cómo se propagan los vectores-distancia. La primera fila es la mencionada *inicialización*. Los elementos de estos vectores tienen el formato `destino:coste:vecino`, que corresponden respectivamente al router destino, el total hasta él y el vecino que se usará como siguiente

salto —siendo '-' la forma de indicar entrega directa. Por ejemplo, para `R1` aparece `R1:0:-` que significa que el coste para llegar a sí mismo es 0 y que puede acceder con entrega directa. También aparece `R2:1:-`, que indica que puede llegar a `R2` con un coste de 1 y entrega directa. Fíjate que los vectores incluyen a los propios routers (los que tienen coste 0) dado que esta información debe enviarse necesariamente.

inicialización				
R1	R1:0:-	R2:1:-		
R2	R1:1:-	R2:0:-	R3:1:-	
R3		R2:1:-	R3:0:-	R4:1:-
R4			R3:1:-	R4:0:-
primera iteración				
R1	R1:0:-	R2:1:-	R3:2:R2	
R2	R1:1:-	R2:0:-	R3:1:-	R4:2:R3
R3	R1:2:R2	R2:1:-	R3:0:-	R4:1:-
R4		R2:2:R3	R3:1:-	R4:0:-
segunda iteración				
R1	R1:0:-	R2:1:-	R3:2:R2	R4:3:R2
R2	R1:1:-	R2:0:-	R3:1:-	R4:2:R3
R3	R1:2:R2	R2:1:-	R3:0:-	R4:1:-
R4	R1:3:R3	R2:2:R3	R3:1:-	R4:0:-

CUADRO 9.2: Propagación de vectores-distancia para la topología de la Figura 9.4

La segunda y tercera filas muestran los vectores que incluye la nueva información incorporada (resaltada con fondo oscuro) en cada iteración del algoritmo. Así pues, en la primera iteración `R1` envía el vector `(R1:0:-)`, `(R2:1:-)`, recibe el vector de `R2` que incluye información sobre `R3` y actualiza su vector-distancia añadiendo `R3:2:R2`, es decir, puede llegar a `R3` con un coste de 2 a través de `R2`.

En la tercera fila (segunda iteración) `R1`, después de recibir información acerca de `R4`, añade `R4:3:R2`, es decir, puede llegar a `R4` con un coste de 3 a través de `R2`. En esta segunda iteración, los routers de los extremos ya tienen la información necesaria para llegar a todos los demás routers con al menor coste posible. Cuando ocurre esto se dice que el algoritmo ha *convergido*. Para esta topología, la convergencia se produce en 2 iteraciones.

El siguiente es un ejemplo algo más complejo, sobre la topología Delta-5, empleando una métrica de coste de los enlaces. La tabla 9.3 muestra el vector-distancia inicial de cada router (uno por fila) tras la inicialización.

Dado que en cada iteración la información solo avanza un salto, si emplea una métrica de saltos, el algoritmo converge en un número de iteraciones

R1	R1:0:-	R2:2:-	R3:1:-				
R2	R1:2:-	R2:0:-	R3:3:-	R4:8:-			
R3	R1:1:-	R2:3:-	R3:0:-	R4:1:-	R5:3:-		
R4		R2:8:-	R3:1:-	R4:0:-	R5:3:-	R6:4:-	
R5			R3:3:-	R4:3:-	R5:0:-	R6:7:-	R7:2:-
R6				R4:4:-	R5:7:-	R6:0:-	R7:2:-
R7					R5:2:-	R6:2:-	R7:0:-

CUADRO 9.3: Vector-distancia inicial de cada router

igual al diámetro del grafo menos uno. Pero si la métrica es otra, podría requerir más iteraciones, porque rutas con más saltos podrían tener costes menores. El diámetro del grafo indica la cantidad de aristas (saltos) de la más costosa de entre las rutas menos costosas. Se define formalmente como:

$$D(G) = \max_{u,v \in V} d(u, v) \tag{9.1}$$

donde:

> V es el conjunto de todos los vértices del grafo G y $d(u, v)$ representa el coste del camino óptimo entre los nodos u y v.

La tabla 9.4 muestra los vectores de todos los routers después de la primera iteración. Como antes, todos los elementos nuevos (o mejorados) aparecen resaltados con fondo oscuro. Fíjate por ejemplo en el vector de `R2`. La información sobre `R4` ha cambiado de `R4:8:-` a `R4:4:R3`. Eso se debe a que al recibirel vector inicial de `R3`, `R2` ha descubierto que puede llegar a `R4` con un coste de 4 a través de `R3`, en lugar de con un coste de 8 con entrega directa. Puedes comprobar que eso ha ocurrido en varios casos más.

R1	R1:0:-	R2:2:-	R3:1:-	R4:2:R3	R5:4:R3		
R2	R1:2:-	R2:0:-	R3:3:-	R4:4:R3	R5:6:R3	R6:12:R4	
R3	R1:1:-	R2:3:-	R3:0:-	R4:1:-	R5:3:-	R6:5:R4	R7:5:R5
R4	R1:2:R3	R2:4:R3	R3:1:-	R4:0:-	R5:3:-	R6:4:-	R7:5:R5
R5	R1:4:R3	R2:6:R3	R3:3:-	R4:3:-	R5:0:-	R6:4:R7	R7:2:-
R6		R2:12:R4	R3:5:R4	R4:4:-	R5:4:R7	R6:0:-	R7:2:-
R7			R3:5:R5	R4:5:R5	R5:2:-	R6:2:-	R7:0:-

CUADRO 9.4: Vector-distancia de cada router después de la primera iteración

Por último, la tabla 9.5 muestra los vectores después de la segunda iteración.

Esta topología converge en dos iteraciones porque después de eso, todos los routers conocen las rutas más cortas a todos los demás. Sin embargo, el

R1	R1:0:-	R2:2:-	R3:1:-	R4:2:R3	R5:4:R3	R6:6:R3	R7:6:R3
R2	R1:2:-	R2:0:-	R3:3:-	R4:4:R3	R5:6:R3	R6:8:R3	R7:8:R3
R3	R1:1:-	R2:3:-	R3:0:-	R4:1:-	R5:3:-	R6:5:R4	R7:5:R5
R4	R1:2:R3	R2:4:R3	R3:1:-	R4:0:-	R5:3:-	R6:4:-	R7:5:R5
R5	R1:4:R3	R2:6:R3	R3:3:-	R4:3:-	R5:0:-	R6:4:R7	R7:2:-
R6	R1:6:R4	R2:8:R4	R3:5:R4	R4:4:-	R5:4:R7	R6:0:-	R7:2:-
R7	R1:6:R5	R2:8:R5	R3:5:R5	R4:5:R5	R5:2:-	R6:2:-	R7:0:-

CUADRO 9.5: Vector-distancia de cada router después de la segunda iteración

algoritmo no acaba al alcanzar la convergencia. Sigue trabajando (ejecutando nuevas iteraciones) indefinidamente porque la topología puede cambiar en cualquier momento. El algoritmo debe adaptar las tablas de encaminamiento de los routers tan pronto como se produzcan los cambios, y pueden ocurrir en cualquier momento.

9.7.1. Problemas y limitaciones de vector-distancia

El principal problema del algoritmo de vector-distancia ya lo hemos visto: es lento. Tarda mucho en converger aunque todo vaya bien. Pero cuando las cosas van mal, es mucho más lento. Para verlo, partamos de una topología muy simple, formada solo por tres routers `R1`, `R2` y `R3` y sus vectores-distancia una vez que algoritmo ha convergido.

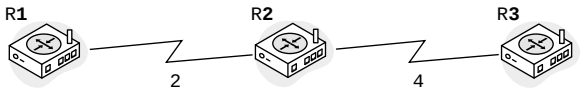


FIGURA 9.5: Topología «lineal» de 3 routers

R1	R1:0:-	R2:2:-	R3:6:R2
R2	R1:2:-	R2:0:-	R3:4:-
R3	R1:6:R2	R2:4:-	R3:0:-

CUADRO 9.6: Vectores-distancia de la topología lineal de 3 routers

Supongamos ahora que el `R3` desaparece o el enlace que lo conecta con `R2` se rompe. A partir de ese momento, `R1` y `R2` intentan utilizar la información disponible para determinar la nueva ruta y coste para llegar a `R3`.

Eso provoca las siguientes acciones:

- `R2` ya no recibe información de `R3`, solo de `R1`, que le indica que puede llegar a `R3` con un coste de 6 (`R3:6:R2`). `R2` actualiza su vector con `R3:8:R1`, es decir, puede llegar a `R3` con un coste de 8 (6+2) a través de `R1`. Obviamente esto no es cierto, pero `R2` no tiene forma de saberlo.
- `R1` recibe el nuevo vector de `R3`, de modo que actualiza su vector a `R3:10:R2`.
- `R2` actualiza su vector a `R3:12:R1`.
- ...

Este proceso continúa indefinidamente, aumentando el coste en cada iteración, y por eso se conoce como *cuenta a infinito* (*count to infinity*). Es un problema grave porque `R1` y `R2` se estarán reenviado entre sí los paquetes dirigidos a `R3` hasta que el valor de TTL llegue a 0 y acaben por descartarse. Esto se conoce como un *bucle de encaminamiento*, aunque también puede tener otras causas.

Existen algunas formas de paliar este problema:

- **«Definir» o acotar el infinito**. Consiste en fijar un valor máximo de coste. Un valor mayor implica considerar inaccesible el router afectado. El inconveniente es que limita también el tamaño de la red. Ninguna ruta real podrá tener un coste mayor al valor fijado.
- **Horizonte dividido (*split horizon*)**. Consiste en no enviar a un vecino la información obtenida de ese vecino. Para el ejemplo implicaría que `R1` no envíe a `R2` el vector `R3:6:R2`, ya que `R1` ha aprendido sobre `R3` por medio de `R2`.
- **Ruta inversa envenenada *(poison reverse)***. Se utiliza en combinación con horizonte dividido. Consiste en informar al router del que ha aprendido un destino, que ese destino es inalcanzable para él. En el ejemplo, `R1` enviaría a `R2` el vector `R3:-1:`, en el entendido de que -1 indica inalcanzable.

Aunque *horizonte dividido* y *ruta inversa envenenada* mejoran el desempeño del algoritmo, no resuelven completamente el problema de la cuenta a infinito. Esto es porque estas técnicas solo afectan a vecinos, pero no evitan que la información errónea llegue a través de otros routers cuando la topología no es tan simple como la del ejemplo.

Además de la cuenta a infinito, el algoritmo de vector-distancia tiene otros problemas:

- La ya citada convergencia lenta. La información de cada router se propaga solo de uno en uno.

- Los routers envían información de encaminamiento periódicamente aunque no haya cambios en la topología, lo que consume ancho de banda y recursos.
- Puede ser inestable en redes grandes o topologías complejas, provocando cambios continuos en las tablas de encaminamiento.

9.7.2. RIP

Routing Information Protocol (RIP) es un protocolo de encaminamiento que emplea el algoritmo vector-distancia. Fue el protocolo de encaminamiento más utilizado en Internet hasta mediados de los 90. A partir de entonces empezó a ser reemplazado por OSPF, BGP e IS-IS. La primera versión de RIP [19] empleaba el algoritmo de Bellman-Ford para el cálculo de la ruta más corta.

Utiliza una métrica de saltos y para mitigar el problema de la cuenta a infinito define un valor máximo de 15 saltos y aplica horizonte dividido. Es un protocolo *classful*, es decir, no envía información sobre la máscara de subred y por tanto solo puede trabajar con las redes de clase A, B y C originales.

Envía actualizaciones de encaminamiento (vectores-distancia) cada 30 segundos a la dirección broadcast `255.255.255.255`.

El mensaje RIP indica el comando (1 para petición, 2 para respuesta) y la versión del protocolo. Después incluye un máximo de 25 vectores-distancia. Cada uno de ellos consta de:

- Familia de direcciones.
- Dirección IP de la red destino.
- Número de saltos hasta la red destino.

En 1993 se publicó RIP versión 2 [20] que sigue utilizando el algoritmo Bellman-Ford para el cálculo de la ruta óptima. Incluye varias mejoras que solventan los problemas del RIP original. Incluye soporte para VLSM y CIDR, utiliza multicast para enviar las actualizaciones (grupo `224.0.0.9`) e incluye un mecanismo de autenticación.

Aunque a día de hoy muchos equipos aún ofrecen soporte, su uso es residual y se limita a redes pequeñas y simples, o entornos de laboratorio.

9.8. Estado de enlace

El «otro» algoritmo importante para encaminamiento interno es estado de enlace *(link state routing)*. La diferencia principal con vector-distancia es

que en este los routers consiguen información topológica detallada de toda la subred, y no solo una medida de coste a cada router.

Por su cuenta cada router:

- Descubre sus vecinos y sus direcciones.
- Mide el coste a cada vecino.
- Construye un mensaje con la información obtenida.
- Envía ese mensaje a todos los routers de la subred (o AS).
- Con la información propia y la recibida crea el grafo de la subred.
- Calcula la ruta más corta a todos los routers.

Cuando un enlace o router cambia de estado, es decir, se activa, desactiva o cambia su coste, los routers que lo detecten envían inmediatamente una actualización a todos los demás.

Este algoritmo requiere más cantidad de memoria al tener que almacenar la topología, requiere más computación al tener que calcular la ruta a cada destino y más ancho de banda ya que genera más tráfico. A cambio, converge más rápido, es más estable y se puede utilizar en subredes más grandes y complejas que vector-distancia.

9.8.1. OSPF

Open Short Path First (OSPF) es el protocolo de encaminamiento de estado de enlace más utilizado en Internet en la actualidad. La primera versión funcional (la 2) fue publicada en 1991 [21] y se diseñó para sustituir a RIP, resolviendo la mayoría de sus problemas, aunque a costa de mayor complejidad y consumo de recursos. Es un protocolo *classless* desde su diseño y por tanto soporta CIDR y VLSM. Utiliza Dijkstra para el cálculo de la ruta más corta y ancho de banda como métrica.

Segmenta el SA en áreas para mejorar la escalabilidad y reducir el tráfico del propio protocolo. La distribución de áreas es jerárquica y hay dos niveles: áreas de tránsito y áreas de borde. El área *backbone* (la 0) es la principal área de tránsito y debe estar en cualquier despliegue de OSPF. Las áreas de tránsito conectan con las de borde routers ABR (Area Border Router), es decir, el tráfico de las otras áreas debe pasar por el área *backbone*. Las áreas de borde solo manejan tráfico propio. Utilizan los ABR para comunicarse con el resto del AS y los ASBR (Autonomous System Border Router) para comunicarse con otros AS.

Cada router mantiene una LSDB (Link State Database) con la información de la topología. Se actualiza periódicamente o cuando se producen cambios a partir del intercambio de mensajes LSA. Hay 7 tipos distintos de LSA en

función del tipo de router que lo envía, la información que contiene y su alcance. Los mensajes LSA se envían al grupo multicast `224.0.0.5`.

9.9. Vector-ruta y BGP

A diferencia de vector-distancia y estado de enlace, vector-ruta es un algoritmo de encaminamiento externo, es decir, se utiliza para encaminamiento entre AS. Los EGP son los encargados de este tipo de encaminamiento. BGP (Border Gateway Protocol) es el EGP más utilizado. Obviamente hay muchas diferencias entre los EGP, que utilizan vector-ruta, respecto a los IGP, pero hay dos que resultan muy evidentes:

- Las rutas se determinan entre AS, no entre routers.
- Las tablas indican la ruta completa hasta el AS destino, no solo el siguiente salto. Estas sí son literalmente «tablas de rutas».

Cada AS se identifica con un ASN (§ 9.2). Una ruta a través de distintos AS se especifica con una secuencia de ASN (se llama ASPATH). Los routers de frontera *(border router)*, como su nombre indica, son los que conectan un AS a otros. Son los encargados de anunciar las ASPATH a los demás AS. Cuando un router BGP recibe una ruta de este tipo puede comprobar fácilmente si esta pasa por su AS. Si ese es el caso, la descarta, y de ese modo evita bucles de encaminamiento. En caso contrario, añade su ASN al final y anuncia esa nueva ruta a los demás AS. La métrica más sencilla para medir la bondad de una ruta es su longitud. Una ruta será mejor si implica menos AS. Las preferencias del administrador son determinantes en la elección, mucho más que en los IGP, incluso por encima de las métricas puramente técnicas.

Fíjate que los routers de frontera tienen que ejecutar tanto un IGP como un EGP. Un IGP (por ejemplo OSPF para encaminar los paquetes dentro del AS) y un EGP (como BGP) para encaminarlos entre AS. Además BGP está compuesto por dos protocolos: eBGP es el que comunica rutas BGP entre distintos AS, mientras que iBGP lo hace entre los routers de un mismo AS.

En la Figura 9.6 aparecen tres AS interconectados. Los marcados en amarillo son routers frontera que tiene que ejecutar tanto un IGP como un EGP.

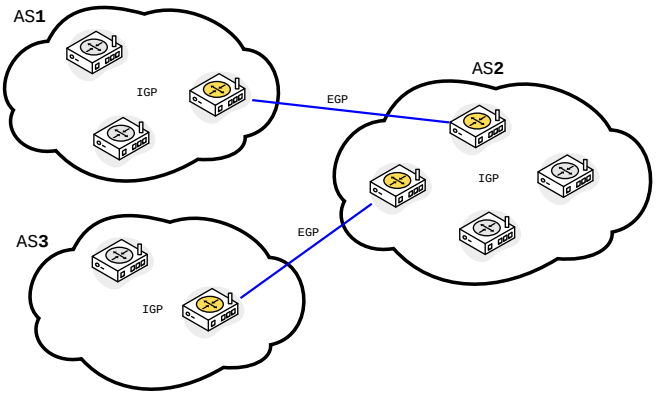


FIGURA 9.6: Sistemas Autónomos interconectados

9.10. Laboratorio de encaminamiento Delta-1

Para ver los protocolos de encaminamiento desde un punto de vista mucho más práctico, vamos a montar un pequeño laboratorio emulando la topología Delta-1 (Figura 9.7).

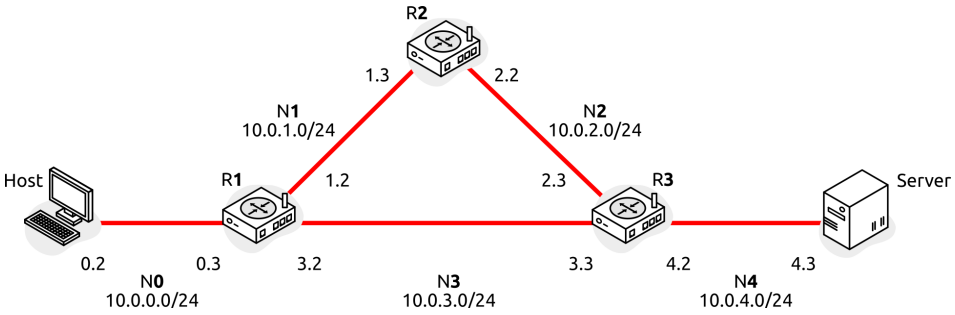


FIGURA 9.7: Topología Delta-1 con direccionamiento

A pesar de ser una topología ciertamente muy simple, es redundante. Los protocolos de encaminamiento deben encontrar la mejor de las rutas disponibles. Sin embargo, los protocolos modernos no descartan las rutas no elegidas. Es mejor idea mantenerlas como rutas alternativas para poder tenerlas disponibles inmediatamente en caso de fallo o de querer balancear el tráfico.

El nodo `Host` de la figura es tu PC real. Los routers y el nodo `Server` son contenedores `docker`. Eso significa que te puedes comunicar con ellos ejecutando comandos reales en tu terminal.

Todos los archivos necesarios para ejecutar este laboratorio (llamado `lab-routing-delta`) están disponibles en el repositorio de ejemplos del libro (ver página XXXVI).

La topología completa está especificada en el archivo `compose.yml` (Listado 9.1).

```yaml
x-common: &common-settings
  privileged: true
  restart: "no"
  command: /entrypoint.sh
  ulimits:
    nofile:
      soft: 100000
      hard: 200000

services:
  r1:
    image: lab-delta-router
    container_name: r1
    hostname: r1
    <<: *common-settings
    volumes:
      - ./router/entrypoint.sh:/entrypoint.sh
      - ./daemons:/etc/frr/daemons
      - ./frr-r1.conf:/etc/frr/frr.conf
    networks:
      N0: { ipv4_address: 10.0.0.3 }
      N1: { ipv4_address: 10.0.1.2 }
      N3: { ipv4_address: 10.0.3.2 }

  r2:
    image: lab-delta-router
    container_name: r2
    hostname: r2
    <<: *common-settings
    volumes:
      - ./router/entrypoint.sh:/entrypoint.sh
      - ./daemons:/etc/frr/daemons
      - ./frr-r2.conf:/etc/frr/frr.conf
    networks:
      N1: { ipv4_address: 10.0.1.3 }
      N2: { ipv4_address: 10.0.2.2 }

  r3:
    image: lab-delta-router
    container_name: r3
    hostname: r3
    <<: *common-settings
    volumes:
      - ./router/entrypoint.sh:/entrypoint.sh
      - ./daemons:/etc/frr/daemons
      - ./frr-r3.conf:/etc/frr/frr.conf
    networks:
      N2: { ipv4_address: 10.0.2.3 }
      N3: { ipv4_address: 10.0.3.3 }
      N4: { ipv4_address: 10.0.4.2 }
```

```
  server:
    image: lab-delta-server
    container_name: server
    privileged: true
    networks:
      N4: { ipv4_address: 10.0.4.3 }

networks:
  N0:
    driver: bridge
    driver_opts: { com.docker.network.bridge.name: N0 }
    ipam: { config: [{subnet: 10.0.0.0/24}] }
  N1:
    driver: bridge
    driver_opts: { com.docker.network.bridge.name: N1 }
    ipam: { config: [{subnet: 10.0.1.0/24}] }
  N2:
    driver: bridge
    driver_opts: { com.docker.network.bridge.name: N2 }
    ipam: { config: [{subnet: 10.0.2.0/24}] }
  N3:
    driver: bridge
    driver_opts: { com.docker.network.bridge.name: N3 }
    ipam: { config: [{subnet: 10.0.3.0/24}] }
  N4:
    driver: bridge
    driver_opts: { com.docker.network.bridge.name: N4 }
    ipam: { config: [{subnet: 10.0.4.0/24}] }
```

LISTADO 9.1: Descripción del laboratorio de encaminamiento Delta
`lab-routing-delta/compose.yml`

Este archivo se procesa con el comando `docker-compose`[3]. Además se necesitan otros archivos de configuración y scripts.

Se proporciona un archivo `Makefile` en el mismo directorio que ejecuta algunas tareas básicas necesarias —que se explican a continuación. Es importante utilizarlo en lugar de lanzar directamente los comandos de `docker`. Es necesario tener instalado el paquete Debian `make`. Para arrancar la **configuración básica** del laboratorio ejecuta:

```
$ make up
[...]
[+] Running 4/4
   Container r1      Started
   Container r2      Started
   Container r3      Started
   Container server  Started
```

Una vez todo en marcha, puedes ver los contenedores con:

```
$ docker compose ps
NAME    IMAGE             COMMAND                  SERVICE  CREATED         STATUS
r1      lab-delta-router  "/entrypoint.sh"         r1       26 seconds ago  Up
```

[3]En versiones más recientes puede estar disponible como `docker compose` (sin guion).

```
r2      lab-delta-router  "/entrypoint.sh"       r2      26 seconds ago  Up
r3      lab-delta-router  "/entrypoint.sh"       r3      26 seconds ago  Up
server  lab-delta-server  "sh -c 'ip route del…" server  26 seconds ago  Up
```

Y las redes con:

```
$ docker network ls
NETWORK ID     NAME                   DRIVER    SCOPE
266d2cb29af4   bridge                 bridge    local
059b37dd8ae6   host                   host      local
030f578f0ff0   none                   null      local
cbcf75bf8048   lab-routing-delta_N0    bridge    local
ab4139a63510   lab-routing-delta_N1    bridge    local
2520be21d05b   lab-routing-delta_N2    bridge    local
6fe6bdc69c3f   lab-routing-delta_N3    bridge    local
270e2a372326   lab-routing-delta_N4    bridge    local
```

Estas redes se implementan como *bridges*. Un bridge es una especie de conmutador *(switch)* virtual que proporciona Linux y que funciona como una red Ethernet. Docker crea un *bridge* por cada red (de N0 a N4). También crea interfaces Ethernet virtuales para conectar los contenedores a las redes. Todo esto lo puedes ver si ejecutas el comando `ip a` en tu PC. Aquí se muestra un fragmento de lo que verás. Los *bridges* normalmente tienen nombres tipo `br-<número>`, pero en este caso les hemos dado nombres `N0-N4` en el fichero `compose.yml`. Los `veth<número>@<iface>` son las interfaces virtuales.

```
1  $ ip a
2  320: N0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 state UP
3      link/ether 26:28:ce:09:e6:7f brd ff:ff:ff:ff:ff:ff
4      inet 10.0.0.1/24 brd 10.0.0.255 scope global N0
5         valid_lft forever preferred_lft forever
6  325: vethc0ba461@if2: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 master N0 state UP
7      link/ether ba:19:52:86:af:65 brd ff:ff:ff:ff:ff:ff link-netnsid 6
8      inet6 fe80::b819:52ff:fe86:af65/64 scope link proto kernel_ll
9         valid_lft forever preferred_lft forever
```

Vale la pena mencionar lo que hace la regla `up` del `Makefile` para evitar confusiones. Es esta:

```
up: build
    >frr-r1.conf; >frr-r2.conf; >frr-r3.conf
    >frr-r{1..3}.conf
    docker compose up --remove-orphans -d
    sudo ./rm-bridge-addrs.sh
    sudo ./setup-hosts.sh
```

Esta es su función:

- `>frr-r1.conf; ...` trunca a cero los tres ficheros de configuración de los routers.
- `docker compose up` arranca los contenedores y los mantiene en *background*. También crea las redes que acabamos de comentar.

- `rm-bridge-addrs.sh` elimina las direcciones IP de esos bridge, excepto el que conecta el host con la red `N0`. Esto es necesario porque de otro modo tu PC tendría acceso directo a todas la redes invalidando el ejercicio. Lo que nosotros pretendemos es que sean los routers los que lleven el tráfico a esas redes.
- `setup-hosts.sh` configura en tu PC la ruta estática que envía a `r1` el tráfico dirigido a `10.0.0.0/16` (todas las redes de la figura). También configura `r3` como router por defecto para `Server`.

Ahora puedes ejecutar comandos en cualquiera de los contenedores. Por ejemplo, puedes ver la tabla de encaminamiento de `r1` con el siguiente comando:

```
$ docker exec r1 ip route
10.0.0.0/24 dev eth0 proto kernel scope link src 10.0.0.3
10.0.1.0/24 dev eth1 proto kernel scope link src 10.0.1.2
10.0.3.0/24 dev eth2 proto kernel scope link src 10.0.3.2
```

Las filas de esta tabla las ha creado automáticamente el SO al activar las interfaces del router. Estamos seguros de eso porque la propia tabla lo indica con *proto kernel*. Estas filas indican que para llegar a las redes `N0`, `N1` y `N3` se debe hacer entrega directa a través de las interfaces `eth0`, `eth1` y `eth2` respectivamente. Docker asigna los nombres a estas interfaces automáticamente, y podrían cambiar en cada ejecución, así que quizá no coincidan con los que ves en tu PC. Puedes probar el mismo comando con `r2` y `r3` para ver sus tablas.

A diferencia de los routers, el nodo `Server` solo tiene una interfaz y está conectada a la red `N4`. Mira el resultado en este caso:

```
$ docker exec server ip route
default via 10.0.4.2 dev eth0
10.0.4.0/24 dev eth0 proto kernel scope link src 10.0.4.3
```

Esta tabla de encaminamiento solo tiene una fila para entrega directa a la red `N3`. La segunda fila la ha configurado el script `setup-hosts.sh`, del que hablamos antes, indicando que `r3` (`10.0.4.2`) es su router por defecto. Solo de ese modo podrá enviar tráfico a las otras redes.

Con esta configuración básica, el nodo `Server` tiene conectividad con `r3`:

```
$ docker exec server ping -c1 10.0.4.2
PING 10.0.4.2 (10.0.4.2) 56(84) bytes of data.
64 bytes from 10.0.4.2: icmp_seq=1 ttl=64 time=0.188 ms
```

Pero no con las otras redes, porque los routers no las conocen:

```
$ docker exec server ping -c1 10.0.1.2
PING 10.0.1.2 (10.0.1.2) 56(84) bytes of data.
From 10.0.3.2 icmp_seq=1 Destination Net Unreachable
```

En las siguientes secciones vamos a configurar el encaminamiento de esta interred de tres formas: encaminamiento estático, RIP y OSPF.

9.10.1. Encaminamiento estático

El encaminamiento estático es muy sencillo en este caso. Simplemente hay que decirle a los routers cómo llegar a las redes distantes, es decir, a las que no están directamente conectados. Por ejemplo, a `r1` debes indicarle cómo llegar a las redes `N2` y `N3`. Los comandos necesarios se incluyen en el script `setup-static.sh`.

```
1 docker exec r1 ip route add default via 10.0.3.3
2 docker exec r2 ip route add 10.0.0.0/24 via 10.0.1.2
3 docker exec r2 ip route add default via 10.0.2.3
4 docker exec r3 ip route add default via 10.0.3.2
```

LISTADO 9.2: Configuración para encaminamiento estático
/lab-routing-delta/setup-static.sh

Fíjate que `r3` (**línea 1**) se configura como router por defecto para `r1`, porque él sabrá cómo llegar a las otras 2 redes. Hacemos algo equivalente con `r3` para `r2` (**línea 3**) y con `r1` para `r3` (**línea 4**). Sin embargo, para que `r2` pueda llegar a la red `N0` debes indicar explícitamente que lo envíe a través de `r1` (**línea 2**), porque no puede tener dos routers por defecto.

Para reiniciar la topología, aplicar los scripts y configurar el encaminamiento estático, ejecuta:

```
$ make static
```

Ahora puedes comprobar que tienes conectividad con `Server`. Desde una consola en tu PC puedes probarle ping:

```
$ ping -c1 10.0.4.3
PING 10.0.4.3 (10.0.4.3) 56(84) bytes of data.
64 bytes from 10.0.4.3: icmp_seq=1 ttl=62 time=0.121 ms
```

O `traceroute`:

```
$ traceroute 10.0.4.3
traceroute to 10.0.4.3 (10.0.4.3), 30 hops max, 60 byte packets
 1  10.0.0.3 (10.0.0.3)  0.411 ms  0.363 ms  0.347 ms
 2  10.0.3.3 (10.0.3.3)  0.332 ms  0.304 ms  0.284 ms
 3  10.0.4.3 (10.0.4.3)  0.264 ms  0.231 ms  0.206 ms
```

En esta salida de `traceroute` se puede ver cómo contestan `r1`, `r3` y el nodo `Server` para cada uno de los 2 saltos necesarios.

También puedes arrancar un servidor en `Server` y acceder a él desde tu PC para comprobar así la conectividad a nivel de transporte. Con el siguiente comando puedes arrancar un servidor `ncat` que simplemente envía la hora al cliente que conecte:

```
$ docker exec server ncat -lp 2000 -c date
```

Y en otra consola, ejecuta el cliente (también con `ncat`):

```
$ ncat 10.0.4.3 2000
Mon Mar  2 20:08:19 CET 2026
```

Con estas tres pruebas no queda ninguna duda de que el encaminamiento está haciendo su trabajo.

9.10.2. Zebra

Actualmente[4] muchos de los SO basados en GNU/Linux utilizan un servicio llamado FRR (Free Range Routing) para implementar encaminamiento dinámico. FRR ofrece un servicio para cada uno de los protocolos de encaminamiento soportados. Todos ellos se comunican con otro servicio llamado `Zebra`, que es quien coordina y decide qué información se incluye finalmente en la tabla de encaminamiento, permitiendo incluso que varios protocolos trabajen a la vez.

Los contenedores de los routers ejecutan `Zebra`, aunque no lo hemos utilizado en la sección anterior. Lo puedes ver en el archivo `router/Dockerfile`:

```
CMD ["sh", "-c", \
    "ip route del default && \
    sysctl -w net.ipv4.ip_forward=1 && \
    /usr/lib/frr/frrinit.sh start && \
    sleep infinity"]
```

Cuando el contenedor arranca, elimina la ruta por defecto que configura docker para evitar que interfiera con nuestro esquema de encaminamiento, activa el reenvío y arranca `zebra` y queda en ejecución mientras el contenedor esté activo (con `sleep`). Es posible utilizar `vtysh`[5] para contactar con `Zebra` y obtener información o configurar parámetros de encaminamiento. Con el siguiente comando puedes ver la tabla de encaminamiento de `r1` tal como quedó configurada en la sección anterior:

```
$ docker exec -it r1 vtysh
r1# show ip route
Codes: K - kernel route, C - connected, L - local, S - static,
```

[4] Escribo esto en 2026.

[5] `vtysh` está incluido en el paquete `frr`.

```
       R - RIP, O - OSPF, I - IS-IS, B - BGP, E - EIGRP, N - NHRP,
       T - Table, v - VNC, V - VNC-Direct, A - Babel, F - PBR,
       f - OpenFabric, t - Table-Direct,
       > - selected route, * - FIB route, q - queued, r - rejected, b - backup
       t - trapped, o - offload failure

IPv4 unicast VRF default:
K>* 0.0.0.0/0 [0/0] via 10.0.3.3, eth2, weight 1, 00:04:06
C>* 10.0.0.0/24 is directly connected, eth0, weight 1, 00:04:06
L>* 10.0.0.3/32 is directly connected, eth0, weight 1, 00:04:06
C>* 10.0.1.0/24 is directly connected, eth1, weight 1, 00:04:06
L>* 10.0.1.2/32 is directly connected, eth1, weight 1, 00:04:06
C>* 10.0.3.0/24 is directly connected, eth2, weight 1, 00:04:06
L>* 10.0.3.2/32 is directly connected, eth2, weight 1, 00:04:06
```

Puedes ejecutar un comando de `vtysh` directamente con la opción `-c` como en `docker exec r1 vtysh -c "show ip route"`. En la salida verás las 3 redes directamente conectadas (C), las interfaces locales (L) y la ruta por defecto (K) que hemos configurado. Al final de cada fila aparece el tiempo transcurrido desde que se estableció.

9.10.3. Encaminamiento RIP

Es momento de configurar RIP en nuestra topología. Partiendo del despliegue original, es necesario proporcional un archivo de configuración para cada uno de los routers. Por ejemplo, para el router `r1` el archivo es:

```
router rip
    network 10.0.0.0/24
    network 10.0.1.0/24
    network 10.0.3.0/24
```

LISTADO 9.3: Configuración RIP para el laboratorio de encaminamiento Delta
/lab-routing-delta/ripd-r1.conf

Esta configuración le indica al router las redes que debe anunciar, es decir, qué vectores-distancia incluir en la inicialización. Los 3 routers anuncian las 3 redes a las que están conectados. Para utilizar el protocolo RIP, el router debe ejecutar el servicio `ripd`. El script `setup-rip.sh` se encarga de copiar los ficheros de configuración como `frr-<router>.conf` para que `Zebra` los aplique.

Para hacer efectiva esa configuración, aplicar los scripts y configurar el encaminamiento RIP, simplemente ejecuta:

```
$ make rip
```

Puedes consultar la configuración de cualquiera de los routers con `vtysh`:

```
$ docker exec r3 vtysh -c "show running-config"
Building configuration...
```

```
Current configuration:
!
frr version 10.6.1
frr defaults traditional
hostname r3
service integrated-vtysh-config
!
router rip
 network 10.0.2.0/24
 network 10.0.3.0/24
 network 10.0.4.0/24
exit
!
end
```

Ahora espera hasta que RIP converja. Para eso puedes simplemente hacer `ping` a `Server` desde tu PC y esperar. Puede tardar alrededor de un minuto. Hasta que las rutas se establezcan, los mensajes no podrán llegar hasta `Server` y verás mensajes de error que te lo indican. Cuando RIP haya establecido las rutas, `ping` empezará a funcionar.

```
$ ping 10.0.4.3
PING 10.0.4.3 (10.0.4.3) 56(84) bytes of data.
From 10.0.0.3 icmp_seq=1 Destination Net Unreachable
From 10.0.0.3 icmp_seq=2 Destination Net Unreachable
From 10.0.0.3 icmp_seq=27 Destination Net Unreachable
64 bytes from 10.0.4.3: icmp_seq=47 ttl=62 time=0.092 ms
64 bytes from 10.0.4.3: icmp_seq=48 ttl=62 time=0.072 ms
64 bytes from 10.0.4.3: icmp_seq=49 ttl=62 time=0.075 ms
```

En ese momento puedes comprobar la tabla de encaminamiento de cualquiera de los routers con `ip route`:

```
$ docker exec r3 ip route
10.0.0.0/24 nhid 9 via 10.0.3.2 dev eth1 proto rip metric 20
10.0.1.0/24 nhid 10 via 10.0.2.2 dev eth0 proto rip metric 20
10.0.2.0/24 dev eth0 proto kernel scope link src 10.0.2.3
10.0.3.0/24 dev eth1 proto kernel scope link src 10.0.3.3
10.0.4.0/24 dev eth2 proto kernel scope link src 10.0.4.2
```

Fíjate en el *proto rip* en las dos primeras filas. Indica que esas rutas las ha aprendido por medio de RIP. También puedes comprobar la tabla de encaminamiento de `r1` con `vtysh`:

```
$ docker exec r3 vtysh -c "show ip route"
R>* 10.0.0.0/24 [120/2] via 10.0.3.2, eth1, weight 1, 00:02:29
R>* 10.0.1.0/24 [120/2] via 10.0.2.2, eth0, weight 1, 00:02:29
C>* 10.0.2.0/24 is directly connected, eth0, weight 1, 00:02:30
L>* 10.0.2.3/32 is directly connected, eth0, weight 1, 00:02:30
C>* 10.0.3.0/24 is directly connected, eth1, weight 1, 00:02:30
L>* 10.0.3.3/32 is directly connected, eth1, weight 1, 00:02:30
C>* 10.0.4.0/24 is directly connected, eth2, weight 1, 00:02:30
L>* 10.0.4.2/32 is directly connected, eth2, weight 1, 00:02:30
```

También puedes comprobar el estado del protocolo RIP con `vtysh`:

```
$ docker exec r3 vtysh -c "show ip rip"
Codes: K - kernel route, C - connected, L - local, S - static,
       R - RIP, O - OSPF, I - IS-IS, B - BGP, E - EIGRP, N - NHRP,
       T - Table, v - VNC, V - VNC-Direct, A - Babel, F - PBR,
       f - OpenFabric, t - Table-Direct
Sub-codes:
      (n) - normal, (s) - static, (d) - default, (r) - redistribute,
      (i) - interface

     Network            Next Hop         Metric From            Tag Time
R(n) 10.0.0.0/24        10.0.3.2              2 10.0.3.2          0 02:39
R(n) 10.0.1.0/24        10.0.2.2              2 10.0.2.2          0 02:48
C(i) 10.0.2.0/24        0.0.0.0               1 self              0
C(i) 10.0.3.0/24        0.0.0.0               1 self              0
C(i) 10.0.4.0/24        0.0.0.0               1 self              0
```

Prueba también `vtysh -c "show ip rip status"` para obtener información adicional.

Fíjate que aunque hay dos formas de llegar desde `r3` a la red `N1` o desde `r1` a la red `N2`, RIP solo anuncia una de ellas.

Igual que para encaminamiento estático, puedes comprobar la conectividad con `Server` (o cualquiera de las routers) desde tu PC mediante `ping`, `traceroute` o `ncat`.

9.10.4. Encaminamiento OSPF

Por último, vamos a configurar OSPF en la topología Delta, partiendo del despliegue básico. Es necesario proporcionar un archivo de configuración para el servicio `ospfd`, aunque en este caso podemos usar el mismo archivo para los 3 routers. El archivo es `ospfd.conf` y se monta también en el archivo `compose.yml`. El archivo contiene simplemente:

```
router ospf
    ospf router-id 10.0.0.3
    redistribute connected
    network 0.0.0.0/0 area 0
```

LISTADO 9.4: Configuración OSPF en el laboratorio de encaminamiento Delta
/lab-routing-delta/ospfd-r1.conf

Puedes reiniciar el laboratorio y aplicar OSPF con:

```
$ make ospf
```

Como antes, puedes consultar la configuración de cualquiera de los routers con `vtysh`:

```
$ docker exec r3 vtysh -c "show running-config"
Building configuration...

Current configuration:
!
frr version 10.6.1
frr defaults traditional
hostname r3
service integrated-vtysh-config
!
router ospf
 ospf router-id 10.0.2.3
 redistribute connected
 network 0.0.0.0/0 area 0
exit
!
end
```

Y también, cuando el protocolo haya convergido, puedes comprobar las tablas de encaminamiento con `ip route`. En este caso pueden pasar hasta 2 minutos hasta que las tablas se estabilicen.

```
$ docker exec r3 ip route
10.0.0.0/24 nhid 11 via 10.0.3.2 dev eth1 proto ospf metric 20
10.0.1.0/24 nhid 15 proto ospf metric 20
        nexthop via 10.0.3.2 dev eth1 weight 1
        nexthop via 10.0.2.2 dev eth0 weight 1
10.0.2.0/24 dev eth0 proto kernel scope link src 10.0.2.3
10.0.3.0/24 dev eth1 proto kernel scope link src 10.0.3.3
10.0.4.0/24 dev eth2 proto kernel scope link src 10.0.4.2
```

OSPF, a diferencia de RIP, proporciona dos formas de llegar a la red `N1`: a través de `r1` y de `r2`. También puedes verlo con `vtysh`:

```
$ docker exec r3 vtysh -c "show ip route"
O>* 10.0.0.0/24 [110/20] via 10.0.3.2, eth1, weight 1, 00:02:03
O>* 10.0.1.0/24 [110/20] via 10.0.2.2, eth0, weight 1, 00:02:03
  *                      via 10.0.3.2, eth1, weight 1, 00:02:03
O   10.0.2.0/24 [110/10] is directly connected, eth0, weight 1, 00:02:48
C>* 10.0.2.0/24 is directly connected, eth0, weight 1, 00:02:48
L>* 10.0.2.3/32 is directly connected, eth0, weight 1, 00:02:48
O   10.0.3.0/24 [110/10] is directly connected, eth1, weight 1, 00:02:48
C>* 10.0.3.0/24 is directly connected, eth1, weight 1, 00:02:48
L>* 10.0.3.3/32 is directly connected, eth1, weight 1, 00:02:48
O   10.0.4.0/24 [110/10] is directly connected, eth2, weight 1, 00:02:48
C>* 10.0.4.0/24 is directly connected, eth2, weight 1, 00:02:48
L>* 10.0.4.2/32 is directly connected, eth2, weight 1, 00:02:48
```

Fíjate que OSPF también anuncia las redes directamente conectadas, pero `Zebra` prioriza las que configuró el SO (las que llevan el prefijo `>`).

9.10.5. Reacción ante fallos

Esta sección muestra cómo reaccionan los protocolos de encaminamiento ante fallos. Está claro que la ruta óptima para llegar de `Host` a `Server` es `Host->r1->r3->Server`. Ya lo has comprobado con `traceroute`.

```
$ traceroute 10.0.4.3
traceroute to 10.0.4.3 (10.0.4.3), 30 hops max, 60 byte packets
 1  10.0.0.3 (10.0.0.3)  0.118 ms  0.038 ms  0.031 ms
 2  10.0.3.3 (10.0.3.3)  0.097 ms  0.083 ms  0.053 ms
 3  10.0.4.3 (10.0.4.3)  0.086 ms  0.068 ms  0.065 ms
```

Ahora vamos a desactivar la interfaz de `r1` en la red `N3` (la que tiene asignada la `10.0.3.2`) para ver cómo OSPF reacciona a este «fallo». Esta es la interfaz que tiene esa dirección IP:

```
$ docker exec r1 ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
2: eth2@if485: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 state UP group default
    link/ether a6:28:d3:14:09:34 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 10.0.3.2/24 brd 10.0.3.255 scope global eth2
       valid_lft forever preferred_lft forever
3: eth0@if487: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 state UP group default
    link/ether ea:6e:00:6e:88:77 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 10.0.0.3/24 brd 10.0.0.255 scope global eth0
       valid_lft forever preferred_lft forever
4: eth1@if490: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 state UP group default
    link/ether f2:63:cd:db:36:4f brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 10.0.1.2/24 brd 10.0.1.255 scope global eth1
       valid_lft forever preferred_lft forever
```

La interfaz que buscamos es `eth2`. Para desactivarla, ejecuta:

```
$ docker exec r1 ip link set eth2 down
```

Comprueba que está desactivada (`DOWN`) con:

```
$ docker exec r1 ip link show eth2
2: eth2@if485: <BROADCAST,MULTICAST> mtu 1500 state DOWN mode DEFAULT group default
```

Después de converger, comprueba qué ruta siguen los paquetes hasta `Server`:

```
$ traceroute 10.0.4.3
traceroute to 10.0.4.3 (10.0.4.3), 30 hops max, 60 byte packets
 1  10.0.0.3 (10.0.0.3)  0.116 ms  0.040 ms  0.031 ms
 2  10.0.1.3 (10.0.1.3)  0.071 ms  0.050 ms  0.047 ms
 3  10.0.2.3 (10.0.2.3)  0.083 ms  0.065 ms  0.062 ms
 4  10.0.4.3 (10.0.4.3)  0.094 ms  0.079 ms  0.078 ms
```

Ahora la ruta que sigue es `Host->r1->r2->r3->Server`. OSPF ha reaccionado al fallo en `N4` y ha elegido la ruta alternativa. Puedes hacer este mismo ejercicio con RIP y verás que también se adapta al fallo.

Y ¿qué más?

El encaminamiento dinámico permite a las interredes descubrir como es su topología, calcular las rutas óptimas a cada destino y adaptarse a los cambios que se produzcan, sean estos debidos a fallos puntuales o a la aparición de nuevos equipos, enlaces o rutas.

Este capítulo ha repasado el funcionamiento de los algoritmos más elementales: vector-distancia, estado de enlace y vector-ruta, y sus problemas y limitaciones. También ha mostrado en funcionamiento implementaciones reales de los protocolos RIP y OSPF, y del sistema FRR de Linux, gracias a despliegues docker sencillos, pero funcionales.

Sin encaminamiento dinámico sería imposible construir interredes de gran tamaño, incluso de mediano tamaño, y obviamente Internet no podría existir.

En todo caso, el encaminamiento dinámico genérico (el descrito en este capítulo) tiene sus limitaciones: se centra en encontrar rutas óptimas y adaptarse a los cambios, pero no comtempla las necesidades particulares de usuarios o servicios. Existen muchas aplicaciones: videoconferencia, streaming de video, VoIP, tráfico en tiempo real; que son muy sensibles a la latencia, el *jitter* o cambios imprevistos en las condiciones del tráfico. Para paliar estas limitaciones existe toda una disciplina: la Calidad de Servicio. Lo abordaremos en el capítulo 17.

Capítulo 10

Configuración IP

Al terminar este capítulo, entenderás:

- Cuál es la información esencial para que un nodo pueda comunicarse en una red IP.
- Cómo se configura un nodo de forma manual.
- Cómo se configura un nodo de forma automática con DHCP.

Cuando un nodo arranca y el SO accede a las interfaces de red conectadas, puede determinar sus direcciones MAC[1], porque esas direcciones físicas están grabadas en una memoria ROM en el propio periférico.

Sin embargo, eso no ocurre con la dirección IP. La dirección IP es parte de la configuración del nodo y es totalmente dependiente de la red a la que cada NIC esté conectada. Sin una dirección IP válida, el nodo no podrá comunicarse con otros nodos ni en la misma red ni fuera de ella.

10.1. Configuración manual

Si no hay disponible un servicio que ayude a realizar esta configuración automáticamente (que veremos más tarde), el usuario del nodo debe realizar la configuración de forma manual. En ese caso, el administrador de la red tiene que proporcionar al usuario la dirección IP para ese nodo y la máscara de subred. Probar asignando una dirección a ciegas es poco probable que funcione, y puede provocar problemas de conectividad a otros usuarios si se elige una dirección en uso.

Supón que el administrador de la red te ha asignado la dirección `20.3.4.13/24` para la interfaz Ethernet de tu computador. Lo primero que necesitas es saber cómo se llama la interfaz, algo que puedes averiguar con el comando `ip addr` (Figura 10.1).

[1] Este es el caso de Ethernet o WiFi, pero no todas las NIC tienen dirección MAC, solo las que utilizan una tecnología de medio compartido.

```
~# ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eno1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default
    qlen 1000
    link/ether f8:5f:2a:c0:ff:ee brd ff:ff:ff:ff:ff:ff
```

FIGURA 10.1: Salida del comando `ip addr` para mostrar las interfaces de red

La interfaz loopback es `lo` y la interfaz Ethernet, en este caso, es `eno1`. Ahora puedes asignarle la nueva dirección y ver el resultado:

```
~# ip addr add 20.3.4.13/24 dev eno1
~# ip addr show eno1
2: eno1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default
    qlen 1000
    link/ether f8:5f:2a:c0:ff:ee brd ff:ff:ff:ff:ff:ff
    inet 20.3.4.13/24 scope global eno1
       valid_lft forever preferred_lft forever
```

Al hacer esto, el SO crea automáticamente una entrada en la tabla de encaminamiento que indica cómo llegar a los vecinos:

```
$ ip route
20.3.4.0/24 dev eno1 proto kernel scope link src 20.3.4.13
```

En esta situación el nodo tiene conectividad con cualquier vecino de la red local, aunque no puede llegar más allá. Para salir de la red local, se necesita la IP del router local[2]. Este dato también lo debe proporcionar el administrador de la red. Supongamos para este ejemplo que esa dirección es `20.3.4.1`. Puedes crear la entrada en la tabla de encaminamiento con el siguiente comando:

```
~# ip route add default via 20.3.4.1
~# ip route
default via 20.3.4.1 dev eno1
20.3.4.0/24 dev eno1 proto kernel scope link src 20.3.4.13
```

Esto corresponde con la tabla de encaminamiento básica de § 8.3.2. La tabla dice cómo llegar a los vecinos y cómo llegar a cualquier otro nodo fuera de la red. Por eso se define como router por defecto.

[2]La llamada *puerta de enlace*

Si no haces nada más, esta información se perderá cuando el nodo se reinicie. Para que sea persistente, tienes que escribirla en el archivo de configuración `/etc/network/interfaces`, que quedaría así:

```
auto eno1
iface eno1 inet static
    address 20.3.4.13
    netmask 255.255.255.0
    gateway 20.3.4.1
```

Este es el método clásico de Debian, pero existen otros métodos como `systemd-networkd`, `NetworkManager` (🎁`network-manager`) o `netplan` (🎁`netplan.io`), que aunque más versátiles, para un uso tan básico como este, resultan equivalentes.

Otra cuestión importante es el nombre del nodo. Se utiliza para identificar el nodo en la red y puede ser necesario para configurar ciertos servicios. El nombre del nodo se puede consultar y cambiar con el comando `hostname`:

```
$ hostname
maine
$ hostname atlantis
$ hostname
atlantis
```

Pero también es temporal. Si quieres que se mantenga después de reiniciar debes escribir ese nombre en el archivo `/etc/hostname`.

Este tipo de configuración se suele denominar «manual» o «estática» porque el usuario la tiene que definir explícitamente y no cambia a menos que lo haga el usuario. Cualquier SO (incluidos los móviles) tiene algún medio para configurar esta información de forma manual, ya sea con una shell o con una interfaz gráfica.

Es habitual que los entornos de escritorio proporcionen algún asistente para realizar la configuración de las interfaces de red manualmente. Como ejemplo, la Figura 10.2 muestra la ventana de configuración que proporciona `GNOME`.

10.2. Configuración automática con DHCP

Pedir a un usuario sin conocimientos técnicos básicos que realice una configuración manual no es lo más conveniente. Además cada nuevo usuario para cada nuevo nodo u otro equipo como una impresora, debe solicitar la configuración al administrador de la red. El administrador a su vez debe llevar registro de las direcciones asignadas por cada subred, y cerciorarse de que se siguen usando, pues lo más probable es que un usuario que se va olvide notificar que ya no necesita esas direcciones.

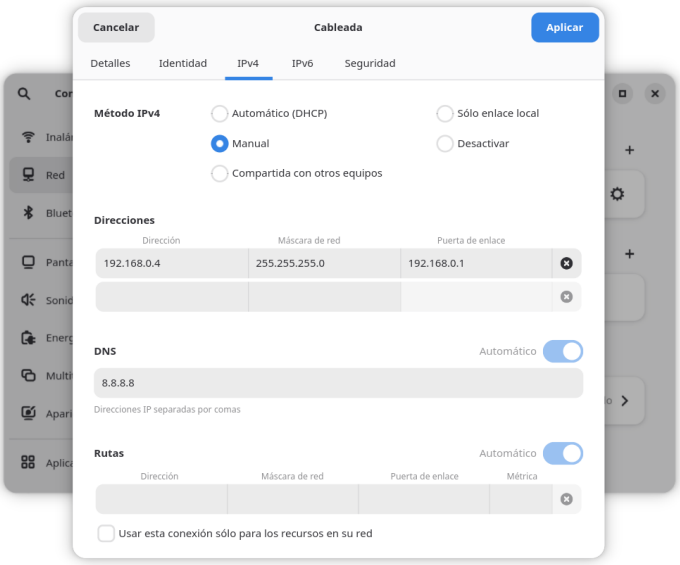


FIGURA 10.2: Configuración de red del entorno GNOME

Para resolver todos estos inconvenientes existe el protocolo DHCP (Dynamic Host Configuration Protocol) [22]. DHCP es un servicio que proporciona la configuración completa para un nodo, cuando éste lo solicita. DHCP se suele considerar un protocolo de aplicación que se encapsula sobre UDP y utiliza los puertos reservados 67 y 68 para servidor y cliente respectivamente.

Pero, ¿cómo puede un nodo sin dirección solicitar algo a un servidor? ¿Cómo sabe el nodo la dirección del servidor? Es posible porque el cliente DHCP envía mensajes de difusión (dirigidos a `255.255.255.255`), algo que es posible incluso aunque la interfaz no tenga una dirección asignada.

En realidad, la conversación entre cliente y servidor DHCP es algo más compleja. Este es el escenario típico, con más detalle:

- El cliente, en el nodo que necesita configuración, envía un mensaje Discover a la dirección de difusión `255.255.255.255`. El propósito de este mensaje es «descubrir» servidores DHCP en la red.
- El servidor (o servidores) DHCP recibe esos mensajes y en función de su configuración responde con un mensaje Offer que contiene la dirección IP y otros parámetros de configuración. Este mensaje también va dirigido a la dirección de difusión.

- El cliente recibe estas «ofertas» y envía un mensaje Request para confirmar la que ha elegido.
- El servidor envía un mensaje Ack como confirmación. Este mensaje contiene la información de configuración completa, incluyendo la dirección IP, la máscara de subred, la puerta de enlace y también los servidores DNS. Desde este momento, el servidor considera que la dirección IP está asignada a ese cliente y no la ofrecerá a otros.
- Al recibir el mensaje Ack, el cliente aplica la configuración a la interfaz de red, y a partir de ese momento puede comunicarse con normalidad.
- Cuando el nodo se apaga o ya no necesita la dirección, envía un mensaje Release para liberarla. Así el servidor puede disponer de ella y asignarla a otro nodo.

Puedes hacer una captura de una conversación DHCP real. En este ejemplo se utiliza `eno1`, que es una interfaz Ethernet, pero lo puedes probar igualmente con una interfaz WiFi. En un terminal arranca una captura con `tshark -i eno1 -Y bootp`. El argumento `-Y bootp` es un filtro que le dice a `tshark` que te interesan solo los mensajes BOOTP, que incluye los mensajes DHCP, porque el segundo es una extensión del primero.

En otro terminal, ejecuta el cliente tal como aparece a continuación. Lo más probable es que ya tengas el programa `dhclient`, pero si no es así, instala el paquete (`isc-dhcp-client`).

El siguiente comando libera la dirección que tienes asignada (envía el mensaje Release) para que así puedas capturar el proceso completo. Opcionalmente después de ese comando, puedes ver que ahora la interfaz no tiene dirección si ejecutas `ip a`.

```
~# dhclient -v -r eno1
Killed old client process
Internet Systems Consortium DHCP Client 4.4.3-P1

Listening on LPF/eno1/f8:5f:2a:c0:ff:ee
Sending on   LPF/eno1/f8:5f:2a:c0:ff:ee
DHCPRELEASE of 172.24.27.53 on eno1 to 172.20.32.167 port 67
```

Este segundo comando es el que realiza la solicitud. Su salida resume los mensajes anteriores:

```
~# dhclient -v -i eno1
Internet Systems Consortium DHCP Client 4.4.3-P1

Listening on LPF/eno1/f8:5f:2a:c0:ff:ee
Sending on   LPF/eno1/f8:5f:2a:c0:ff:ee
DHCPDISCOVER on eno1 to 255.255.255.255 port 67 interval 5
```

```
DHCPDISCOVER on eno1 to 255.255.255.255 port 67 interval 10
DHCPOFFER of 172.24.27.53 from 172.20.32.167
DHCPREQUEST for 172.24.27.53 on eno1 to 255.255.255.255 port 67
DHCPACK of 172.24.27.53 from 172.20.32.167
bound to 172.24.27.53 -- renewal in 1571 seconds.
```

En la captura que tenías en marcha en el otro terminal ha debido aparecer algo similar a esto:

```
~# tshark -i eno1 -Y bootp
1        0.0.0.0 → 255.255.255.255 DHCP 342 DHCP Discover - Transaction ID 0x9b6d7a5a
2  172.20.32.167 → 172.24.27.53    DHCP 342 DHCP Offer    - Transaction ID 0x9b6d7a5a
3        0.0.0.0 → 255.255.255.255 DHCP 342 DHCP Request  - Transaction ID 0x9b6d7a5a
4  172.20.32.167 → 172.24.27.53    DHCP 342 DHCP ACK      - Transaction ID 0x9b6d7a5a
```

Para ver más detalles, repite la captura añadiendo el argumento `-V` al comando `tshark`. Se muestran aquí los campos más interesantes de los mensajes Discover y Offer.

```
~# tshark -i eno1 -Y bootp -v
Dynamic Host Configuration Protocol (Discover)
    Message type: Boot Request (1)
    Hardware type: Ethernet (0x01)
    Hardware address length: 6
    Transaction ID: 0x014ff92a
    Client IP address: 0.0.0.0
    Your (client) IP address: 0.0.0.0
    Next server IP address: 0.0.0.0
    Relay agent IP address: 0.0.0.0
    Client MAC address: f8:5f:2a:c0:ff:ee
    Option: (50) Requested IP Address (0.0.0.0)
    Option: (12) Host Name
        Length: 5
        Host Name: maine
    Option: (55) Parameter Request List
        Length: 13
        Parameter Request List Item: (1) Subnet Mask
        Parameter Request List Item: (28) Broadcast Address
        Parameter Request List Item: (2) Time Offset
        Parameter Request List Item: (3) Router
        Parameter Request List Item: (15) Domain Name
        Parameter Request List Item: (6) Domain Name Server
        Parameter Request List Item: (119) Domain Search
        Parameter Request List Item: (12) Host Name
        Parameter Request List Item: (26) Interface MTU
        Parameter Request List Item: (121) Classless Static Route
        Parameter Request List Item: (42) Network Time Protocol Servers
    Option: (61) Client identifier
        Length: 19
        IAID: 96a502be
        DUID Type: link-layer address plus time (1)
        Hardware type: Ethernet (1)
        Time: 799853663
        Link layer address: f8:5f:2a:c0:ff:ee

Dynamic Host Configuration Protocol (Offer)
```

```
Message type: Boot Reply (2)
Hardware type: Ethernet (0x01)
Hardware address length: 6
Transaction ID: 0x014ff92a
Client IP address: 0.0.0.0
Your (client) IP address: 172.24.27.53
Next server IP address: 172.20.32.167
Client MAC address: f8:5f:2a:c0:ff:ee
Option: (1) Subnet Mask (255.255.255.0)
Option: (58) Renewal Time Value: 30 minutes (1800)
Option: (59) Rebinding Time Value: 52 minutes, 30 seconds (3150)
Option: (51) IP Address Lease Time: 1 hour (3600)
Option: (54) DHCP Server Identifier (172.20.32.167)
Option: (3) Router: 172.24.27.1
Option: (15) Domain Name: uclm.es
Option: (6) Domain Name Server:
    Domain Name Server: 172.20.32.3
    Domain Name Server: 172.20.32.4
```

En el mensaje Discover el cliente solicita diversa información. Lo puedes ver en la sección *Parameter Request List*. El servidor responde a estas solicitudes como opciones al final del mensaje Offer y Ack. No tiene por qué responder a todo, pero a parte por supuesto de la dirección del nodo (campo «Your (client) IP address»), normalmente siempre aparecerán los ya nombrados:

- Subnet Mask
- Router
- Domain Name Server

El servidor asigna una dirección al cliente por un período limitado: el «tiempo de concesión» *(lease time)*, que es configurable en el servidor. En la captura esto aparece en el campo `IP Address Lease Time`. Durante ese plazo, el servidor asegura que la dirección no se va a asignar a otro cliente. Si el cliente quiere seguir utilizando la misma dirección puede enviar un mensaje Renew para renovar la concesión antes de que termine el plazo de renovación (campo `Renewal Time Value` en la captura). En la captura puedes comprobar que el tiempo de concesión es de 1 hora y el de renovación es de 30 min. Estos valores son bastante bajos porque hemos tomado la captura en un entorno empresarial. En el entorno doméstico el tiempo de concesión suele ser de 24 horas o incluso más. Comprueba qué valores aparecen en tu caso analizando tu captura.

10.3. Servidor DHCP

El programa `dnsmasq` es un servidor DHCP sencillo, ligero y fácil de configurar, ideal para redes domésticas, despliegues pequeños o para realizar

pruebas de laboratorio. También funciona como servidor DNS aunque para este ejemplo lo vamos a desactivar.

El servidor se configura mediante un archivo de texto que suele estar en `/etc/dnsmasq.conf`. El siguiente listado es un ejemplo perfectamente funcional de ese fichero que sirve para asignar direcciones en la red `192.168.2.0/24` incluyendo la información básica necesaria.

```
port=0
listen-address=192.168.2.1
dhcp-range=192.168.2.2,192.168.2.100,12h
dhcp-option=3,192.168.2.1
dhcp-option=6,8.8.8.8,1.1.1.1
```

Significado de cada opción:

`port`
: es el puerto en el que debe escuchar el servidor DNS. Al asignar un valor de 0, el servidor DNS queda inoperante.

`listen-address`
: indica la dirección IP de la interfaz de red en la que el servidor DHCP atenderá peticiones.

`dhcp-range`
: es el rango de direcciones que puede asignar a los clientes. Desde `192.168.2.2` hasta `192.168.2.100`, con una concesión de 12 horas. El nodo que ejecuta este servidor DHCP probablemente actúa de router de la LAN (aunque no es obligatorio) y tendría la dirección `192.168.2.1`.

`dhcp-option`
: indica distintas opciones con información adicional que se puede ofrecer a los clientes. Los números de opción son los de la especificación de DHCP y corresponden con la captura de la sección anterior: 3 para indicar la dirección del router y 6 para las direcciones IP de dos servidores DNS, que en este caso son servidores públicos de Google y Cloudflare.

Esto funciona aplicando este mismo fichero de configuración con el ejemplo /`dnsmasq`, utilizando un contenedor `docker`, como muestra la siguiente consola:

```
dnsmasq$ docker compose up
[+] Running 1/1
 - Container dnsmasq  Create
Attaching to dnsmasq
dnsmasq  | dnsmasq[1]: started, version 2.90 DNS disabled
dnsmasq  | dnsmasq-dhcp[1]: DHCP, IP range 192.168.2.2 -- 192.168.2.100, lease time 12h
```

En este ejemplo `docker` crea una red llamada `dnsmasq_net` en la que el host (tu PC) y el contenedor son vecinos. Eso significa que puedes solicitar una dirección a este DHCP. Debes listar las interfaces para averiguar cuál tiene una IP en el bloque `192.168.2.0/24`. Sí, la interfaz ya tiene una IP asignada, pero no importa, aún así puedes solicitar otra al servidor DHCP y comprobar su funcionamiento. La interfaz en cuestión tendrá un nombre

similar a `br-f215a045cd10`. Por tanto, en otro terminal, puedes solicitar una dirección con:

```
$ ip a
[...]
54: br-f215a045cd10: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP
    link/ether 5a:47:ab:01:41:42 brd ff:ff:ff:ff:ff:ff
    inet 192.168.2.254/24 brd 192.168.2.255 scope global br-f215a045cd10
       valid_lft forever preferred_lft forever

~# dhclient -v -i br-f215a045cd10
```

Al ejecutar esto deberías ver un registro detallado de las solicitudes y respuestas en la salida de `docker compose` con información similar a la captura `tshark` de la sección anterior. Después puedes comprobar que la interfaz tiene ahora una dirección del rango (`192.168.2.85`).

```
$ ip addr show br-f215a045cd10
54: br-f215a045cd10: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP
    link/ether 5a:47:ab:01:41:42 brd ff:ff:ff:ff:ff:ff
    inet 192.168.2.254/24 brd 192.168.2.255 scope global br-f215a045cd10
       valid_lft forever preferred_lft forever
    inet 192.168.2.85/24 brd 192.168.2.255 scope global secondary dynamic br-f215a045cd10
       valid_lft 43196sec preferred_lft 43196sec
```

10.4. DHCP no siempre es la solución

DHCP resulta muy útil y cómodo, pero no es adecuado en todos los casos. Hay situaciones en las que la configuración manual sigue siendo necesaria, o preferible. La mayoría de los dispositivos que conforman la infraestructura (como los routers) o que ofrecen servicios a la red (servidores, impresoras, etc.) deben tener direcciones fijas y conocidas. Aunque es posible configurar un servidor DHCP para que asigne siempre las mismas direcciones a dispositivos específicos (identificados por su MAC), es más seguro que tengan su dirección correcta incluso cuando el servidor DHCP no funcione.

El servicio DHCP está fuertemente orientado a los usuarios finales, por lo que suelen atender peticiones solo en las LANs. En ciertas subredes en las que solo hay equipos controlados, como cámaras de seguridad o telefonía IP, o en los enlaces entre routers, no suele haber servicios DHCP. Es de hecho una buena medida de seguridad que así sea. Un servidor DHCP ilegítimo podría ofrecer configuraciones fraudulentas.

El propio servidor DHCP es otro caso evidente: no puede depender de un servicio que él mismo proporciona, así que al menos la interfaz por la que atiende peticiones necesita una dirección fija asignada de antemano.

Y ¿qué más?

La dirección IP del nodo, la del router y las de los servidores DNS constituyen la configuración mínima esencial para que un nodo pueda conectarse a la red, sin embargo hay mucho más. Conforme una red y un despliegue TIC se vuelve más y más complejo es necesario administrar muchos otros parámetros como inventarios de equipos, usuarios, programas instalados, versiones y un largo etcétera.

Esto da lugar a toda una disciplina que es la «Gestión y administración de redes y sistemas» en sus muchas variantes. En esta disciplina se utilizan protocolos y herramientas específicas. Y muy relacionado con todo esto quedan las tareas de monitorización y seguridad, que suponen otra gran área de conocimiento y estudio. Este capítulo solo ha sido el primer paso hacia ese mundo.

Capítulo 11

Redes Privadas

Al terminar este capítulo, entenderás:

- Qué es una red privada y por qué es útil.
- Los diferentes tipos de redes privadas: LAN, interredes privadas, redes híbridas, intranets y extranets.
- Qué es el direccionamiento privado y cuáles son bloques de direcciones reservadas.
- Qué el NAT, qué problema resuelve y cuáles provoca.

Una red privada, es solo eso, una red[1] de uso privado, en contraposición a una red pública, que es utilizada por el tráfico de mucha gente. Su objetivo principal suele ser compartir recursos dentro de una organización, por lo que todos los elementos físicos que conforman la red (computadores, dispositivos de interconexión, cableado, etc.) están bajo el control exclusivo de esa organización, y por tanto, también es responsable de su diseño, gestión y explotación.

El concepto «red privada» no es equivalente a «red aislada». La red privada más común hoy día es la red WiFi doméstica que encontramos en la mayoría de los hogares y que, obviamente, está conectada a Internet a través del llamado «router doméstico» y la red del ISP.

11.1. Líneas alquiladas

También una interred puede ser privada, es decir, puede ser una colección de LANs interconectadas mediante routers, ya sea en una o varias localizaciones, todo eso bajo el control y uso exclusivo de una misma organización. Durante los años 80 a 90, cuando Internet no existía o el acceso era demasiado caro, era habitual que muchas empresas tuvieran una red privada que conectaba las LAN de sus oficinas —por ejemplo, los concesionarios de una

[1] Habitualmente una LAN

marca de automóviles. Para conectar estas oficinas entre sí, la organización podía instalar cableado propio o bien alquilar líneas a una compañía de telecomunicaciones[2]. La primera opción solo era económicamente viable para distancias muy cortas (muy pocos cientos de metros). Este planteamiento es el que muestra la Figura 11.1.

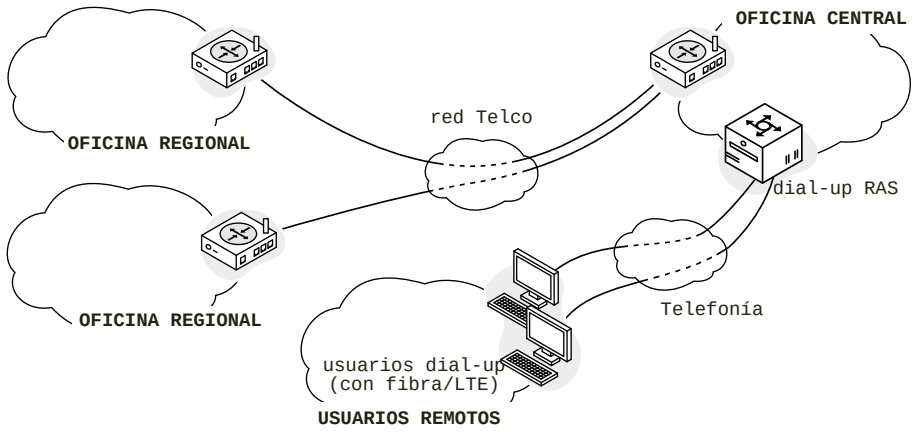


FIGURA 11.1: Red privada que utiliza líneas alquiladas

Partiendo de una topología de este tipo, si alguna de las oficinas tiene acceso a Internet, puede ofrecer conectividad a algunas o todas las demás oficinas. Este esquema (que se muestra en la Figura 11.2) es lo que se denomina *red híbrida*.

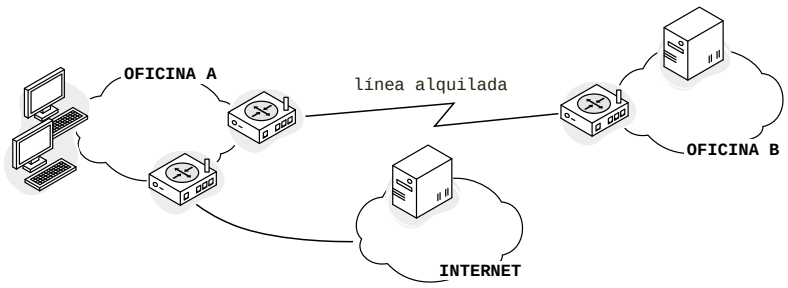


FIGURA 11.2: Red privada híbrida

[2]Abreviado habitualmente «telco».

11.2. Redes privadas TCP/IP

Precisamente por ser privada, la organización tiene plena libertad para elegir cualquier pila de protocolos disponible, o incluso implementar una tecnología propia.

En la práctica, parece buena idea utilizar la pila TCP/IP por variedad y disponibilidad de software, hardware de red, soporte y sobre todo, como forma de simplificar la conexión de la red privada con otras redes. La única razón para no utilizar TCP/IP en la red privada es que no cumpla con algún requisito técnico específico.



Durante los primeros 90, Novell NetWare IPX tuvo gran popularidad como protocolo para redes privadas, debido a varios factores: tarjetas NIC asequibles, soporte por defecto en Microsoft Windows y en los primeros videojuegos multijugador en red (local) como *p. ej.* Quake, etc. Pocos años después, con la llegada de Internet, IPX fue desbancado rápidamente por IP.

Una *intranet* es justamente una red privada que utiliza tecnología TCP/IP. Sin embargo, el nombre *intranet* se utiliza hoy día, erróneamente, para identificar una o varias aplicaciones o servicios (normalmente web) destinados específicamente al personal de una organización para lo que requiere autenticación específica.

Como caso particular de *intranet*, la *extranet* permite a ciertos usuarios o servicios acceder a los recursos de la red privada desde el exterior. Este acceso está controlado mediante algún sistema de autenticación y autorización.

11.3. Direccionamiento privado

Si la red privada está aislada, la organización tiene libertad para decidir cómo plantear el direccionamiento de sus dispositivos. El diseñador de la red privada tiene tres alternativas:

- Solicitar un bloque de direcciones públicas globales. Implica realizar una petición, y su correspondiente pago, a las autoridades de Internet: IANA o las entidades regionales (RIR) en las que haya delegado la tarea de asignación de direcciones. Si la red privada está aislada, o al menos sus computadores no van a proporcionar servicios hacia Internet, puede ser un gasto injustificado.

- Utilizar un bloque público arbitrario sin conocimiento de las autoridades. Si efectivamente la red privada va a estar esencialmente aislada, no supone ningún problema técnico, pero puede plantear graves problemas logísticos y administrativos si en el futuro esa red acaba siendo conectada a Internet.
- Utilizar uno de los bloques reservados específicamente para redes privadas.

Esta tercera es la opciónrecomendada por las autoridades en la RFC 1918 [23] y consiste en la elección arbitraria de uno de los bloques definidos en el Cuadro 11.1. A estas direcciones se las denomina simplemente «direcciones privadas».

inicio		fin	prefijo CIDR
10.0.0.0	-	10.255.255.255	10/8
172.16.0.0	-	172.31.255.255	172.16/12
192.168.0.0	-	192.168.255.255	192.168/16

CUADRO 11.1: Bloques IP reservados para direccionamiento privado

La direcciones privadas deben ser consideradas *no rutables* en Internet, es decir, los routers del ISP y de la WAN en general descartarán cualquier paquete IP que tenga como destino una dirección privada. Precisamente por esto, cualquier organización puede elegir uno de estos bloques privados sin necesidad de autorización. Aunque existan millones de redes alrededor del mundo que estén utilizando exactamente el mismo bloque no hay problema, ya que su tráfico nunca podrá ser confundido con el de otra red. Es decir, las direcciones privadas deben ser localmente únicas, pero al contrario de las públicas, no son globalmente únicas.

Aunque el núcleo de Internet (esencialmente BGP) y los ISP descarten este tráfico, la organización puede encaminarlos dentro de su interred corporativa. Esto le otorga gran flexibilidad a la organización, pudiendo crear varias subredes interconectadas para diferentes propósitos o comunidades de usuarios.

11.4. Direccionamiento de enlace local

Las direcciones privadas permiten construir una interred corporativa completa, con múltiples subredes interconectadas. Existe un ámbito todavía más restringido: el de las «direcciones de enlace local» *(link-local addresses)*, que solo sirven para comunicar directamente a dispositivos conectados a un mismo enlace. A diferencia de las direcciones privadas, un router nunca

debe encaminar un paquete con origen o destino en este rango, ni siquiera hacia otra subred de la propia organización: quedan necesariamente confinadas al enlace en el que se generan.

En IPv4 el bloque reservado para este fin es `169.254.0.0/16` [24]. Cuando un nodo no logra contactar con ningún servidor DHCP, se puede autoasignar aleatoriamente una dirección de este bloque, tras comprobar con ARP que ningún otro vecino la está utilizando. Es un mecanismo de último recurso que permite que los equipos de una misma LAN se puedan comunicar aunque falle la infraestructura de configuración automática.

En Debian/GNU-Linux esta autoasignación la hace el servicio `avahi-autoipd`. Basta con instalar el paquete para activarlo. Si una petición DHCP falla, expira o agota el tiempo de espera, `avahi-autoipd` arranca automáticamente y asigna la dirección; si más tarde `dhclient` consigue una concesión real, el mismo mecanismo lo detiene.

Se puede probar con una interfaz virtual ejecutando los siguiente:

```
# modprobe dummy
# ip link add dummy0 type dummy
# ip link set dummy0 arp on
# ip link set dummy0 up
```

Esta interfaz `dummy0` no está conectada con nada, no tiene acceso a un servidor DHCP y por tanto, no tiene dirección IP.

```
~$ ip addr show dev dummy0
4: dummy0: <BROADCAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UNKNOWN group default
↪  qlen 1000
    link/ether 5e:38:c9:81:30:e1 brd ff:ff:ff:ff:ff:ff
```

Al arrancar `avahi-autoipd` se asigna una dirección de enlace local:

```
~# avahi-autoipd --debug dummy0
[...]
Successfully claimed IP address 169.254.12.100
[...]
```

Una vez conseguido, la interfaz tiene una dirección IP de enlace local:

```
~$ ip addr show dev dummy0
4: dummy0: <BROADCAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UNKNOWN group default
↪  qlen 1000
    link/ether 5e:38:c9:81:30:e1 brd ff:ff:ff:ff:ff:ff
    inet 169.254.12.100/16 brd 169.254.255.255 scope link dummy0:avahi
       valid_lft forever preferred_lft forever
```

La etiqueta *scope link* indica precisamente que la dirección solo tiene alcance en el enlace local.

Todo este mecanismo de autoconfiguración tiene se denomina IPv4LL, según la propia RFC 3927 [24].

11.5. Conectividad en redes privadas

Las redes domésticas actuales, y la mayoría de las que se utilizan en empresas y organizaciones pequeñas y medianas, técnicamente son *redes privadas híbridas con tecnología TCP/IP*. En una red doméstica, el ISP proporciona un dispositivo (ver Figura 11.3), que llamamos informalmente como *router doméstico*. Este router es el que proporciona la conexión de tus dispositivos a Internet. En realidad ese dispositivo (esa caja) no es solo que un router. Incorpora normalmente varios dispositivos y funciones:

- Un router IP con 2 interfaces: LAN y WAN.
- Un conmutador Ethernet.
- Un punto de acceso WiFi.
- Un módem que puede utilizar distintas tecnologías: ADSL, SDSL o actualmente fibra óptica con FTTH, HFC y otras. En las conexiones de fibra, el módem puede ser un dispositivo aparte llamado ONT o estar integrado también en la misma caja.
- Un pequeño computador habitualmente con alguna variante de Linux empotrado.
- Un servidor DHCP.

En la Figura 11.3 se pueden ver las conexiones del router doméstico. El primer conector de la derecha (WAN) conecta el equipo con el ONT. Los conectores amarillos numerados de 1 a 4 son los puertos del conmutador Ethernet incorporado. Y la antena, es parte del punto de acceso WiFi.

Cuando conectas tu computador, móvil, televisor, etc. —sea con Ethernet o WiFi— el servidor DHCP incorporado le asigna una dirección privada. El bloque de direcciones configurado suele ser `192.168.0.0/24`, aunque puede ser cualquiera del Cuadro 11.1, y normalmente el propio usuario tiene la posibilidad de cambiarlo accediendo a la web de administración del router.

Si la Internet pública descarta los paquetes IP dirigidos a direcciones privadas (§ 11.3), ¿cómo pueden entonces estos nodos comunicarse con servidores públicos? La solución consiste en *traducir* las direcciones privadas a direcciones públicas al salir de la red privada. Esta traducción la realiza un proceso llamado NAT que se ejecuta en el router doméstico, y por eso se le llama «router NAT».

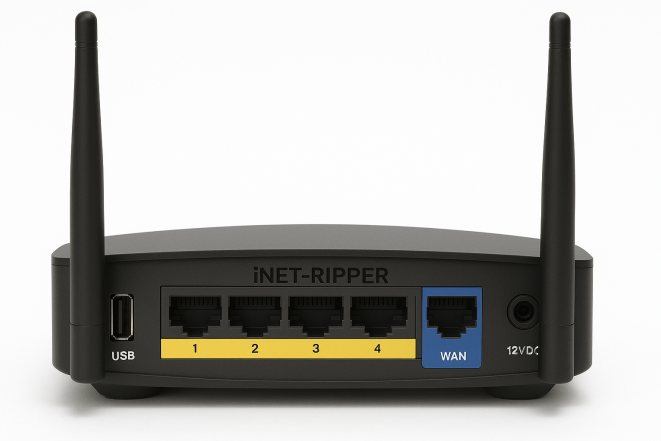


Figura 11.3: Conexiones de un router doméstico

11.6. Traducción de Direcciones de Red (NAT)

NAT (Network Address Translation) [25] es un programa (software) que traduce (reescribe) las direcciones de los paquetes IP que reenvía. La topología es similar a la Figura 11.4. El router NAT tiene una interfaz WAN (que conecta con la red del ISP) con una dirección IP pública (*p. ej.* `180.20.30.13`) asignada por un servidor DHCP del ISP e interfaces LAN con direcciones privadas (`192.168.0.0/24`) asignadas por el servidor DHCP del router doméstico.

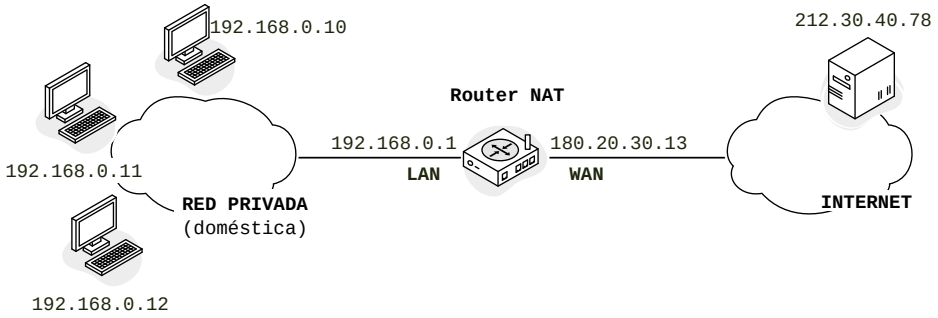


Figura 11.4: Ejemplo de configuración NAT en una red doméstica

El uso más habitual de NAT es el que permite a los nodos de la red privada establecer conexiones con servidores de la red pública. Es el llamado SNAT que se muestra en la Figura 11.5. Si por ejemplo, el nodo A quiere conectar con un servidor público, cuando sus paquetes IP salen de la red, el router

substituye la dirección **origen**[3] (privada) por la dirección pública (WAN) del router. Al volver la respuesta procedente del servidor remoto, el router substituye la dirección destino (que es la pública del router) de nuevo por la dirección privada del nodo (ver Figura 11.5). Ten en cuenta que este mecanismo es transparente para el servidor y para cualquier otro elemento en Internet, es decir, ni participan ni son conocedores de que esta traducción está ocurriendo.

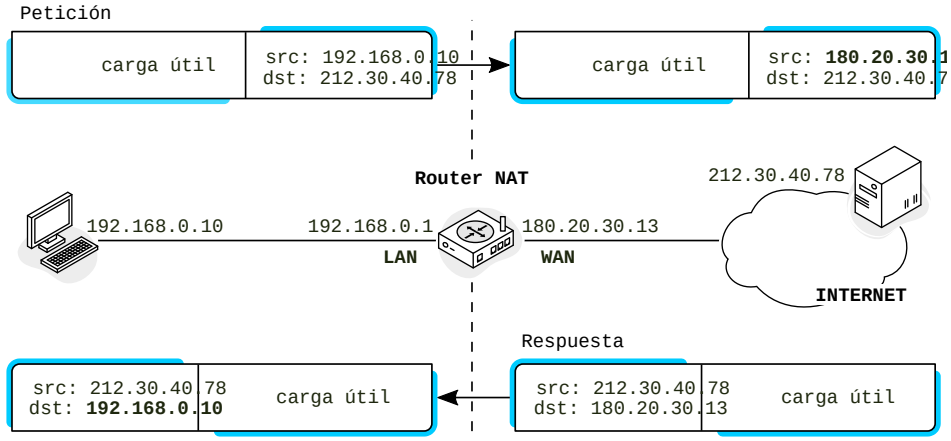


FIGURA 11.5: Ejemplo de SNAT

Para que la traducción de la respuesta funcione, el router debe saber cuál de los computadores de la red privada hizo la petición. Para ello, el software NAT apunta en la tabla NAT[4] esas correspondencias en el momento de hacer la traducción de salida. Lógicamente, estas entradas en la tabla desaparecen tras un tiempo de inactividad o bien cuando se detecta el cierre de la conexión (si es que el flujo es TCP). El Cuadro 11.2 muestra la tabla tras dos conexiones desde los nodos `A` y `B` a dos servidores remotos diferentes.

Dirección local	Dirección remota
192.168.0.10	212.30.40.78
192.168.0.12	200.25.34.56

CUADRO 11.2: Tabla NAT para el envío de la Figura 11.5

Sin embargo, la información de esta tabla es insuficiente. La traducción de direcciones en la respuesta es ambigua si dos o más nodos de la red privada

[3]Por eso se llama *Source* NAT
[4]Formalmente: *Address Translation Table*

contactan con el mismo servidor remoto a la vez. Es ambigua en el sentido de que el router no tiene forma de saber qué nodo privado fue el que envió la petición que corresponde a esta respuesta.

Para reducir esta ambigüedad existe una técnica mejorada denominada NAPT (Network Address Port Translation) [26]. Consiste en incorporar a la tabla NAT los puertos origen y destino TCP o UDP. La probabilidad de que dos nodos de la red privada elijan el mismo puerto origen en conexiones simultáneas a un mismo servidor remoto es realmente muy baja, y ese sería el único caso en el que se presentaría la ambigüedad. La Figura 11.6 y la Tabla 11.3 ilustran esta técnica. Prácticamente cualquier traducción de direcciones de este tipo es alguna variante de NAPT, pero comúnmente se llama simplemente NAT a todas ellas, y así lo haremos en adelante.

Endpoint local	Endpoint remoto	Proto
192.168.0.10:4045	212.30.40.78:80	TCP
192.168.0.12:32400	200.25.34.56:22	TCP

Cuadro 11.3: Tabla NAPT para el envío de la Figura 11.6

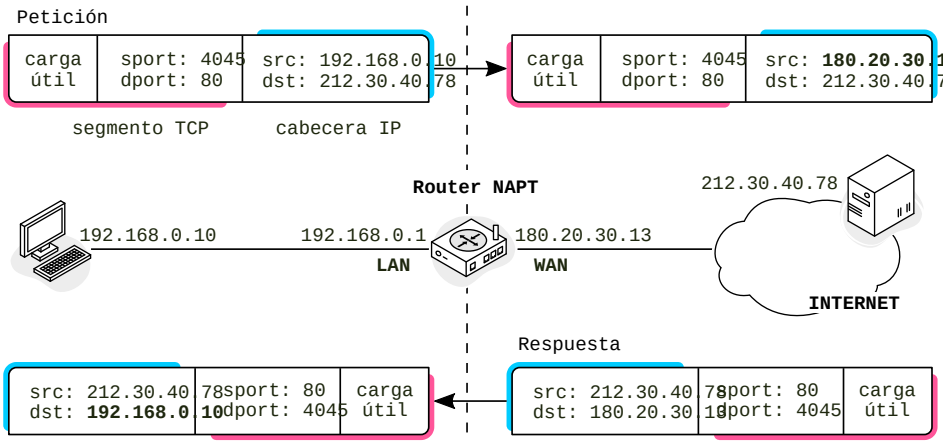


Figura 11.6: Ejemplo de NAPT

En todo caso, muchas implementaciones de NAT optan por evitar completamente la ambigüedad traduciendo el puerto del cliente local por un puerto ficticio que el router puede elegir garantizando así que será único en la tabla. Este puerto se conoce como «puerto mapeado» o «sintético». La IP externa del router (normalmente pública) junto con un puerto mapeado forman un *endpoint mapeado*, que es la forma de identificar un flujo desde el exterior de la red privada.

Esta situación se muestra en la Figura 11.7 y el Cuadro 11.4.

Endpoint local	Endpoint mapeado	Endpoint remoto	Proto
192.168.0.10:4045	180.20.30.13:7312	212.30.40.78:80	TCP
192.168.0.12:2400	180.20.30.13:45012	200.25.34.56:22	TCP

CUADRO 11.4: Tabla NAPT con puertos mapeados para el envío de la Figura 11.6

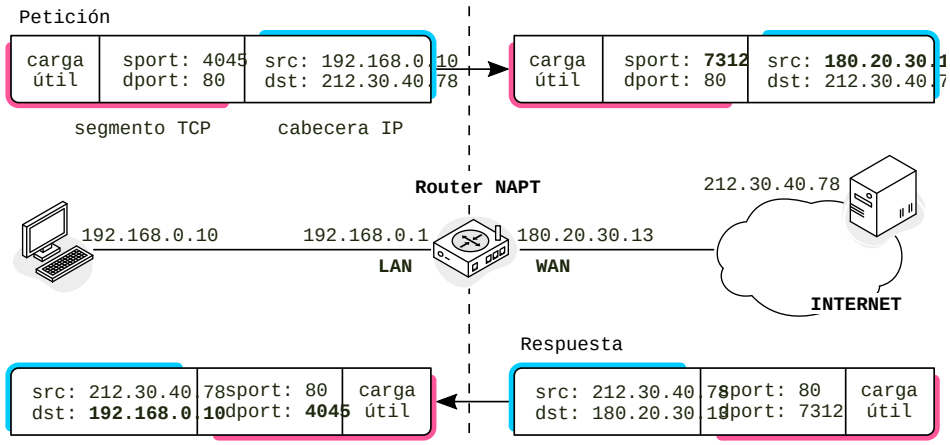


FIGURA 11.7: Ejemplo de NAPT con puertos mapeados

Las entradas de la tabla NAT no son permanentes; si lo fueran, la tabla crecería sin límite y el router acabaría por agotar los puertos mapeados disponibles. Por eso cada entrada tiene asociado un tiempo de inactividad *(idle timeout)*: es un contador que se reinicia con cada paquete del flujo y que, al expirar, provoca la eliminación de la entrada.

El tiempo de inactividad depende del protocolo. Con TCP, el router puede detectar cuando se cierra la conexión analizando las cabeceras, y eliminar la entrada de la tabla en ese momento. Por eso, puede otorgar tiempos de inactividad amplios: al menos dos horas según la RFC 5382 [27], pero en casos reales puedes ser días. UDP, en cambio, no tiene conexión ni cierre, así que el router no puede saber si el flujo ha terminado. La estrategia en ese caso es fijar tiempos cortos, entre 30 segundos y pocos minutos según la RFC 4787 [28].

Esto tiene una consecuencia negativa para estos flujos UDP. Si la frecuencia de los mensajes es menor a la que implica ese tiempo, la entrada se borra y el flujo se interrumpe. Para evitarlo, la aplicación debe envíar periódicamente

mensajes de mantenimiento *(keep alive)* para así reiniciar el contador y que la entrada siga activa.

11.6.1. Averiguar la dirección pública

Una consecuencia directa de NAT es que un nodo de la red privada no conoce con qué dirección IP se le ve desde Internet. El comando `ip addr` (Figura 10.1) solo muestra su dirección privada; la pública pertenece a la interfaz WAN del router, y la forma de averiguarla sin la intervención de terceros es consultar el propio router, quizá a través de su web de administración, algo que es distinto para cada fabricante.

Cuando eso no es posible o no resulta adecuado, la alternativa es consultar un servidor externo. Cualquier servidor de Internet con el que se comunique el nodo del usuario recibe necesariamente los paquetes con la dirección traducida, de modo que basta un pequeño servicio que responda con la dirección origen de la petición que le llega. Hay dos medios habituales para lograr esto:

- Servicios web del tipo «cuál es mi IP». Muchos devuelven la dirección en texto plano cuando la petición no procede de un navegador, lo que permite consultarlos desde el terminal con un cliente HTTP como `curl`:

  ```
  ~$ curl checkip.amazonaws.com
  203.0.113.2
  ```

  Estos servicios no están estandarizados y pueden variar mucho en su funcionamiento y disponibilidad. Para ilustrar lo sencillo que es implementar uno de estos servicios mira el ejemplo /`whats-my-ip/whats-my-ip.py`.

- Servidores STUN, un protocolo diseñado específicamente para este fin, que además de la dirección devuelve el puerto mapeado. Se describe en § 11.10.

11.6.2. Reenvío de puertos

Otro inconveniente de NAT, que NAPT tampoco resuelve, es que todo el mecanismo se basa en que el router aprende del tráfico saliente de la red privada, y usa esa información para realizar la traducción inversa. Eso implica que no es posible poner servidores en los nodos privados que puedan ser alcanzados desde clientes fuera de la red privada. Para resolver esto, el router proporciona un medio para incluir filas permanentes en la tabla NAT.

Estas entradas en la tabla se conocen como «reenvío de puertos» *(port forwarding)*. Ten en cuenta que en este caso, el router traduce la dirección IP destino, en lugar de la origen, y por eso se denomina *Destination NAT* (DNAT). Como esta configuración está pensada para dar acceso a un servidor interno, lo habitual es que el usuario que crea una entrada DNAT en la tabla indique tanto el puerto destino del router como el puerto destino del nodo privado, que es donde realmente hay un servidor vinculado.

El caso más evidente es el de un servidor convencional —un servidor web o SSH en casa, un NAS, una cámara IP—, pero hay muchas aplicaciones cotidianas que también necesitan recibir flujos entrantes, aunque el usuario no las perciba como «servidores»:

- Las aplicaciones P2P *(peer-to-peer)*, como BitTorrent, en las que cada nodo es cliente y servidor al mismo tiempo. Un nodo que no admite conexiones entrantes solo puede intercambiar datos con los que sí las admiten, lo que reduce mucho su rendimiento; y si todos estuvieran en esa situación, la red no funcionaría en absoluto.
- Las videollamadas, en las que lo deseable es que el audio y el vídeo viajen directamente entre los participantes, sin pasar por un servidor intermedio que añada retardo. Ese flujo directo es, por definición, entrante para al menos uno de los dos extremos.
- Los videojuegos en línea en los que uno de los jugadores hace de anfitrión *(host)* de la partida, o en los que las consolas intercambian directamente el estado del juego. Algunos fabricantes de consolas clasifican el «tipo de NAT» del usuario en función de si es posible o no recibir esos flujos.

En todos estos casos el reenvío de puertos es la solución más directa, pero requiere que el usuario pueda y sepa configurar su router. Cuando no es así, o cuando hay un CGNAT (§ 11.8), las aplicaciones tienen que recurrir a las técnicas de § 11.10.



Muchos routers domésticos ofrecen UPnP, que permite a las aplicaciones que se ejecutan en nodos de la red privada pedir al router la creación de reenvíos de puertos automáticamente, sin intervención del usuario. Y justo por eso se considera un riesgo de seguridad.

11.6.3. Direcciones en la carga útil

El router NAT reescribe únicamente las cabeceras de red y transporte, pero algunos protocolos de aplicación incluyen direcciones IP y puertos del nodo en la carga útil como medio para que el receptor pueda contactar después a través de flujos entrantes nuevos. Cuando eso ocurre tras un NAT, el nodo privado envía su endpoint local, que resulta inútil para la aplicación remota. Estos son dos ejemplos clásicos:

- FTP [29] utiliza una conexión de control y otra distinta para cada transferencia de datos. En el modo «activo», el cliente indica al servidor, con el comando `PORT`, la dirección y el puerto en los que espera la conexión de datos, y es el servidor quien la inicia. Si el cliente está en una red privada, el servidor recibe una dirección como `192.168.0.10`, inalcanzable desde Internet, y la transferencia falla.
- SIP [30], el protocolo de señalización de la telefonía IP, negocia los flujos de audio y vídeo mediante descripciones de sesión (SDP [31]) que viajan en el cuerpo de sus mensajes. Cada extremo anuncia en ellas el endpoint en el que quiere recibir los paquetes RTP del otro. Con una dirección privada, la llamada se establece, pero el audio nunca llega.

Hay dos formas de abordar este problema. La primera, en el propio router, con una ALG (Application Layer Gateway) [26]: un módulo que reconoce el protocolo de aplicación, localiza las direcciones dentro de la carga útil, las reescribe con el endpoint mapeado y crea por adelantado la entrada en la tabla NAT para el flujo que va a llegar; en la práctica, un reenvío de puertos automático e implícito. Es transparente para la aplicación, pero exige un módulo específico para cada protocolo, con todas sus particularidades, y resulta inútil si el payload viaja cifrado. Por eso los routers suelen implementar solo los de unos pocos protocolos muy extendidos, como FTP o SIP.

Los ALG de SIP de muchos routers domésticos causan más problemas de los que resuelven —modifican mensajes que no deberían o dejan a medias la reescritura—, hasta el punto de que los proveedores de telefonía IP suelen recomendar desactivarlos.

La segunda forma, más robusta, es que la propia aplicación evite anunciar direcciones privadas: bien porque no las necesita —el modo «pasivo» de FTP, en el que también la conexión de datos la inicia el cliente—, bien porque averigua previamente su endpoint mapeado por los medios de § 11.6.1

y es ese el que escribe en el payload. Es la estrategia que recomienda la RFC 3235 [32] y la que siguen las aplicaciones de videollamada actuales (§ 11.10).

11.7. Router NAT con Linux

Cualquier nodo con Linux[5] y dos interfaces de red también puede ser un router NAPT. Sirva como escenario la misma topología de la Figura 11.4: la interfaz `eth0` (WAN) tiene la dirección pública `180.20.30.13` asignada por el ISP, y la interfaz `eth1` (LAN) tiene la dirección privada estática `192.168.0.1`, que es la puerta de enlace de los nodos de la red privada `192.168.0.0/24`.

Lo primero es activar el reenvío de paquetes (§ 8.1) para que el nodo se comporte como un router:

```
~# echo 1 > /proc/sys/net/ipv4/ip_forward
```

La traducción de direcciones es tarea de *netfilter*, el sistema de manipulación y filtrado de paquetes del núcleo Linux. El sistema recomendado actualmente para configurarlo es `nftables`[6], por medio del programa `nft` (`nftables`). La configuración se organiza en «tablas» *(tables)*, cada una asociada a una familia de protocolos (`ip` para IPv4, `ip6` para IPv6 o `inet` para ambas). Una tabla es una colección de «cadenas» *(chains)* y otros artefactos, como conjuntos o mapas. Una cadena es un conjunto de reglas que se aplican a los paquetes que la atraviesan. Para el SNAT hace falta una tabla y una cadena con estos parámetros:

```
~# nft add table ip nat
~# nft add chain ip nat postrouting '{ type nat hook postrouting priority srcnat; }'
```

`type nat`
: Es una cadena de traducción de direcciones. Solo la atraviesa el primer paquete de cada flujo; los siguientes se traducen automáticamente según el contenido de la tabla NAT.

`hook postrouting`
: Se aplica a los paquetes que salen del nodo, una vez tomada la decisión de encaminamiento.

`priority srcnat`
: Orden de evaluación estándar para las cadenas SNAT.

La cadena solo necesita una regla:

```
~# nft add rule ip nat postrouting oif eth0 masquerade
```

[5] Eso incluye también a un móvil Android.
[6] Evolución del veterano `iptables`.

Se aplica a los paquetes que salen por la interfaz WAN (`oif eth0`). La acción `masquerade` es una variante de SNAT que utiliza como dirección origen la que tenga la interfaz de salida en cada momento, lo que resulta muy conveniente si la dirección pública es dinámica: aunque el ISP asigne otra, la regla sigue funcionando. Con una dirección fija sería `snat to 180.20.30.13`.

Con esto el router está completo: los nodos de la red privada ya pueden establecer conexiones con servidores de Internet. La tabla NAPT la mantiene el subsistema de seguimiento de conexiones del núcleo *(connection tracking)* y se puede consultar con el programa `conntrack`:

```
~# conntrack -L
tcp  6 431998 ESTABLISHED src=192.168.0.10 dst=212.30.40.78 sport=4045 dport=80
  src=212.30.40.78 dst=180.20.30.13 sport=80 dport=7312 [ASSURED] use=1
```

Cada entrada contiene dos tuplas. La primera describe los paquetes salientes tal como los envía el nodo privado; la segunda, los paquetes de respuesta que el router espera recibir: dirigidos a su dirección pública y al puerto mapeado 7312. Es decir, esta entrada equivale a la primera fila del Cuadro 11.4. El número 431 998 es el tiempo de inactividad de la entrada (§ 11.6). En Linux es de 5 días para las conexiones TCP establecidas. Para UDP el plazo es de 30 segundos para tráfico en un sentido, y 120 para tráfico en ambos sentidos.

Los ALG de § 11.6.3 en Linux se denominan *helpers*: módulos del núcleo como `nf_conntrack_ftp` o `nf_conntrack_sip` que inspeccionan la carga útil de los flujos que tienen asignados y crean las entradas adicionales que necesitan. Con `nft`, se puede usar la sentencia `ct helper` para asignar un *helper* a un flujo.

El reenvío de puertos requiere una segunda cadena, asociada al *hook prerouting*, que procesa los paquetes recibidos **antes** de la decisión de encaminamiento. Tiene que ser así porque la dirección destino debe reescribirse antes de decidir hacia dónde debe ir el paquete.

```
~# nft add chain ip nat prerouting '{ type nat hook prerouting priority dstnat; }'
~# nft add rule ip nat prerouting iif eth0 tcp dport 8080 dnat to 192.168.0.10:80
```

Esta regla DNAT crea la entrada permanente en la tabla: las conexiones que lleguen por la interfaz WAN al puerto 8080 se entregan al puerto 80 del nodo `192.168.0.10`, donde hay un servidor web. Como corresponde a una entrada de reenvío de puertos, la regla indica tanto el puerto destino en el router como el puerto real del servidor interno.

Configuración completa de la tabla:

```
~# nft list table ip nat
table ip nat {
  chain postrouting {
    type nat hook postrouting priority srcnat; policy accept;
    oif "eth0" masquerade
  }
  chain prerouting {
    type nat hook prerouting priority dstnat; policy accept;
    iif "eth0" tcp dport 8080 dnat to 192.168.0.10:80
  }
}
```

Como el resto de la configuración de red, estas reglas también se pierden al reiniciar el equipo. En Debian, el servicio `nftables` carga en el arranque las reglas guardadas en el archivo `/etc/nftables.conf`. Con el siguiente comando se puede guardar la configuración actual para cargar en el siguiente reinicio del nodo o el servicio —lo que hace el segundo comando.

```
~# nft list ruleset > /etc/nftables.conf
~# systemctl enable nftables.service
```

El parámetro `net.ipv4.ip_forward` se hace persistente en el archivo de configuración `/etc/sysctl.conf` (§ 2.10).

11.8. CGNAT (Carrier-Grade NAT)

El direccionamiento privado y NAT sirvieron para aliviar la escasez de direcciones IPv4 durante años. Con estos mecanismos cada hogar solo necesita una dirección pública, aunque tenga muchos dispositivos conectados. Pero con el crecimiento de Internet incluso eso es insuficiente. IANA repartió sus últimos bloques libres en 2011, y los registros regionales fueron agotando después sus reservas, el último fue RIPE, en 2019.

La solución que aplican los ISP consiste en aplicar doble NAT: un router CGN o CGNAT (Carrier-Grade NAT) [33] es un router NAPT de gran capacidad situado en la red del ISP, que permite compartir cada dirección pública entre cientos de clientes. Con CGNAT, la interfaz WAN del router doméstico del abonado no recibe una dirección pública, sino una del bloque `100.64.0.0/10` [34], reservado específicamente para este propósito. El tráfico hacia Internet sufre entonces *dos* traducciones consecutivas —en el router doméstico y en el router CGNAT—, esquema que se conoce como NAT444 por las tres direcciones IPv4 que tiene cada paquete durante su tránsito.

En redes de telefonía móvil la práctica está aún más extendida: el número de abonados por estación base es tan alto que las conexiones de datos por

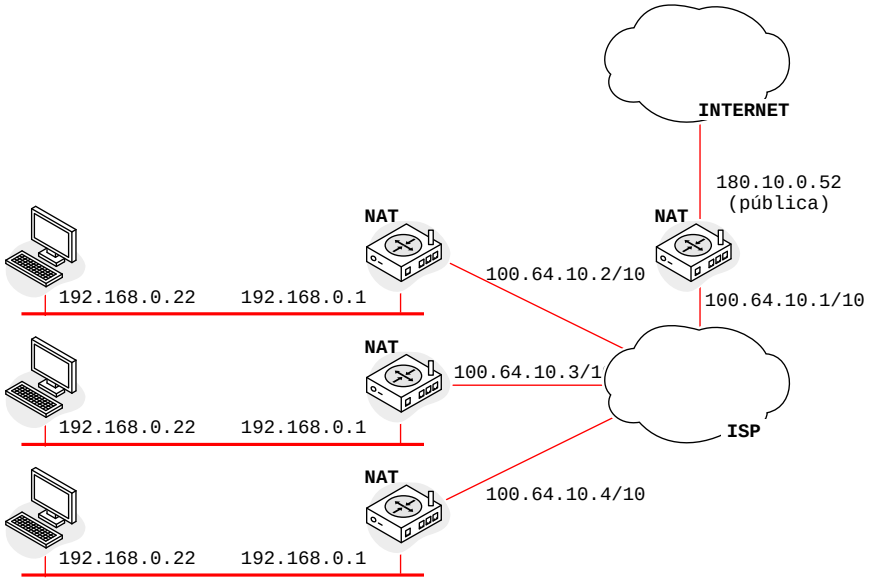


Figura 11.8: Conguración típica de CGNAT

LTE o 5G suelen estar tras CGNAT incluso cuando la línea fija del mismo operador no lo está.

Para el usuario final, CGNAT tiene todos los inconvenientes de NAT, pero por partida doble y, además, fuera de su control:

- El reenvío de puertos (§ 11.6.2) no sirve en este caso. Montar un servidor en la red privada accesible desde Internet ya no es posible.
- La dirección pública se comparte con cientos de otros abonados del ISP. Si un proveedor de servicio bloquea esa dirección por el abuso de uno de ellos, el bloqueo afecta a todos.
- La dirección pública ya no identifica a un cliente, por lo que el ISP está obligado a registrar también los puertos mapeados para poder llevar un registro de conexiones[7].

Con CGNAT, la web de administración del router doméstico ya no sirve para conocer la dirección pública, porque su interfaz WAN no la tiene. Los servicios externos vistos en § 11.6.1 sí la devuelven, ya que ven la dirección origen de los paquetes, es decir, la del router CGNAT tras las dos traducciones. Eso proporciona también una forma sencilla de comprobar si la conexión está detrás de un CGNAT: basta comparar ambas direcciones.

[7]Este registro es obligatorio por ley en muchos países.

Si no coinciden, hay una traducción adicional por el camino; y si además la WAN pertenece a `100.64.0.0/10`, es un CGNAT con seguridad. También suele delatarlo el primer salto después del router doméstico:

```
~$ traceroute -n 8.8.8.8
 1  192.168.0.1  1.104 ms  1.087 ms  1.075 ms
 2  100.64.0.1  9.371 ms  9.355 ms  9.342 ms
 3  180.20.30.1  10.240 ms  10.227 ms  10.213 ms
...
```

Muchos ISP ofrecen la posibilidad de obtener una dirección pública exclusiva[8], aunque puede conllevar un coste.

11.9. Tipos de NAT

El mecanismo NAT explicado en § 11.6 se denomina «simétrico» y se caracteriza por utilizar un puerto mapeado diferente para cada flujo saliente —identificado por endpoint remoto. No es la única forma de implementar NAT.

Distintas implementaciones pueden aplicar restricciones diferentes que pueden afectar a su comportamiento y esto da lugar a diferentes tipos. La taxonomía *clásica* [35] utiliza el concepto de «cono» como metáfora de la correspondencia entre el endpoint mapeado (el vértice del cono) y los endpoints remotos que tienen permitido utilizarlo (la base del cono).

Los tipos son los siguientes (ordenados desde el más permisivo al más restrictivo). Se utilizan los datos de la Figura 11.7 para los ejemplos:

Full cone
: El endpoint mapeado acepta mensajes de cualquier endpoint externo: mensajes desde cualquier IP y puerto.

Endpoint local	endpoint mapeado	Endpoint remoto	
`192.168.0.10:4045` →	`180.20.30.13:7312` →	`212.30.40.78:80`	petición
`192.168.0.10:4045` ←	`180.20.30.13:7312` ←	`*:*`	respuesta aceptada

Restricted cone
: El endpoint mapeado solo acepta mensajes de endpoints externo que provengan de una IP contactada.

Endpoint local	endpoint mapeado	Endpoint remoto	
`192.168.0.10:4045` →	`180.20.30.13:7312` →	`212.30.40.78:80`	petición
`192.168.0.10:4045` ←	`180.20.30.13:7312` ←	`212.30.40.78:*`	respuesta aceptada
`192.168.0.10:4045` ←	`180.20.30.13:7312` ←	`1.2.3.4:1200`	respuesta **rechazada**

Port restricted cone
: Solo acepta mensajes que provengan del endpoint contactado exacto.

[8] Coloquialmente se dice «sacarte del CGNAT»

Endpoint local	endpoint mapeado	Endpoint remoto	
192.168.0.10:4045 →	180.20.30.13:7312 →	212.30.40.78:80	petición
192.168.0.10:4045 ←	180.20.30.13:7312 ←	212.30.40.78:80	respuesta aceptada
192.168.0.10:4045 ←	180.20.30.13:7312 ←	212.30.40.78:1200	respuesta **rechazada**
192.168.0.10:4045 ←	180.20.30.13:7312 ←	1.2.3.4:80	respuesta **rechazada**

Simétrico
Aplica la misma restricción que *port restricted*, pero crea un endpoint mapeado diferente (un puerto diferente) para cada flujo saliente.

Algunas aclaraciones necesarias:

- Todos los tipos requieren que los mensajes entrantes utilicen el mismo protocolo (UDP o TCP) que la petición que creó la entrada en la tabla NAT.
- Para un mismo endpoint local, los tres primeros tipos (cono) utilizan el mismo puerto mapeado aunque el endpoint remoto cambie; hay una correspondencia 1:N, ese es el vértice del cono. En el NAT simétrico la correspondencia es N:N; no hay cono.

Es posible compartir el puerto mapeado para varios flujos porque la tabla NAT puede determinar cuál es el endpoint local a partir del endpoint remoto. Un ejemplo para *full cone*:

Endpoint local	endpoint mapeado	Endpoint remoto	
192.168.0.10:4045 →	180.20.30.13:7312 →	212.30.40.78:80	petición
192.168.0.10:4045 →	180.20.30.13:7312 →	212.30.40.**79**:**81**	petición
192.168.0.10:4045 ←	180.20.30.13:7312 ←	212.30.40.78:80	respuesta aceptada
192.168.0.10:4045 ←	180.20.30.13:7312 ←	212.30.40.79:81	respuesta aceptada
192.168.0.10:4045 ←	180.20.30.13:7312 ←	*:*	respuesta aceptada

Estos son los cuatro tipos mas habituales, pero la RFC 4787 [28] descompone el comportamiento de NAT en 9 tipos organizados en dos ejes ortogonales: cómo se asigna el puerto mapeado y restricción de acceso, es decir, qué endpoints remotos tienen permitido utilizarlo:

Asignación de puerto mapeado:

EIM (Endpoint-Independent Mapping) El endpoint mapeado es el mismo para todos los flujos salientes de un mismo endpoint local.

ADM (Address-Dependent Mapping) El endpoint mapeado es específico para cada IP remota.

APDM (Address and Port-Dependent Mapping) El endpoint mapeado es específico para cada endpoint remoto.

Restricción de acceso:

EIF (Endpoint-Independent Filtering) El endpoint mapeado acepta tráfico de cualquier endpoint remoto.

ADF (Address-Dependent Filtering) Solo acepta tráfico de IPs remotas contactadas previamente.

APDF (Address and Port-Dependent Filtering) Solo acepta tráfico de endpoints remotos contactados previamente.

Lógicamente, los tipos clásicos se corresponden con combinaciones de estos dos ejes: *full cone* es EIM/EIF, *restricted cone* es EIM/ADF, *port restricted cone* es EIM/APDF y *simétrico* es APDM/APDF.

11.10. NAT transversal

En muchas situaciones no es posible configurar el reenvío de puertos, sea porque el usuario no tiene acceso, permiso o conocimientos para ajustar la configuración de su router doméstico o porque se encuentra detrás de un CGNAT. En estos casos se aplican las denominadas técnicas de *NAT traversal* [36]. Todas se apoyan en la misma idea: la entrada en la tabla NAT que crea el tráfico saliente indica qué hacer con el tráfico entrante, que en teoría son respuestas, pero que podemos aprovechar para permitir flujos entrantes.

La idea básica de NAT *transversal* es abrir un flujo saliente, determinar cuál es el endpoint mapeado que el router registra para ese flujo, y comunicárselo al cliente remoto para que pueda enviar sus mensajes a ese endpoint.

11.10.1. STUN

La primera tarea es averiguar el endpoint mapeado asociado a un nuevo flujo. Para eso existen los servidores y el protocolo STUN [37]. Es el segundo de los mecanismos de §11.6.1, y el único que devuelve también el puerto. Se trata de un protocolo muy sencillo encapsulado sobre UDP: el cliente local envía una solicitud `Binding Request` a un servidor STUN público, y este le responde con un mensaje `Binding Response` que contiene justamente el endpoint deseado. Suponiendo que el endpoint fuera `180.20.30.13:7312`, la petición y la respuesta de la sesión STUN serían algo como:

```
Message Type: Binding Response (0x0101)
Magic Cookie: 0x37A3FACD
Transaction ID: 0x5EE3564D15A18B6AB42E0489

Message Type: Binding Response (0x0101)
Transaction ID: 0x5EE3564D15A18B6AB42E0489
```

```
Attribute: XOR-MAPPED-ADDRESS (0x0020)
  Family: IPv4 (0x01)
  Port: 7312 XOR 0x37A3
  Addr: 180.20.30.13 XOR 0x37A3FACD
```

Tanto la dirección IP como el puerto sufren una codificación trivial con `xor` para evitar que un ALG (§ 11.6.3) los reconozca y los reemplace al encontrarlos en el payload. La respuesta puede incluir también el mismo endpoint sin codificar, en el atributo `MAPPED-ADDRESS`, por compatibilidad con clientes antiguos; si ambos coinciden es que ningún ALG lo ha modificado, y eso es lo que indica la línea `No ALG: Mapped == XOR-Mapped` de `turnutils_natdiscovery`, un poco más adelante.

Con el programa `turnutils_stunclient` (`coturn`) se puede efectuar una consulta a un servidor STUN público y ver la respuesta:

```
~$ turnutils_stunclient stunserver2025.stunprotocol.org
RFC 5780 response 1
0: : IPv4. Response origin: : 3.22.142.132:3478
0: : IPv4. Response origin: : 3.22.142.132:3478
0: : IPv4. Other addr: : 3.135.212.85:3479
0: : IPv4. UDP reflexive addr: 203.0.113.2:58961
```

El protocolo STUN también se puede utilizar para determinar (al menos parcialmente) el tipo de NAT que aplica el router local. Para lograrlo el cliente realiza distintas peticiones para ver cómo se comporta en los dos ejes de la taxonomía de la RFC 4787 [28]: asignación de puerto mapeado y restricción de acceso. La RFC 5780 [38] describe dos procedimientos para determinar ambos ejes, que resumimos aquí. Supongamos que el servidor STUN es `stun.example.net:3478`.

11.10.2. Determinar el Mapeado

En cada petición, el cliente recibe el endpoint mapeado en el atributo `XOR-MAPPED-ADDRESS` de la respuesta (la línea `UDP reflexive addr` del comando anterior). La primera respuesta incluye además el atributo `OTHER-ADDRESS`, un endpoint alternativo del mismo servidor STUN (otra IP y otro puerto, línea `Other addr`), que se usa en los tests siguientes.

11.10.3. Determinar el Filtrado

El procedimiento de la Figura 11.10 reutiliza el atributo `CHANGE-REQUEST` para pedirle al servidor STUN que responda desde su endpoint alternativo (IP y puerto, o solo puerto), en lugar de desde el primario, y así comprobar si esa respuesta consigue atravesar el NAT.

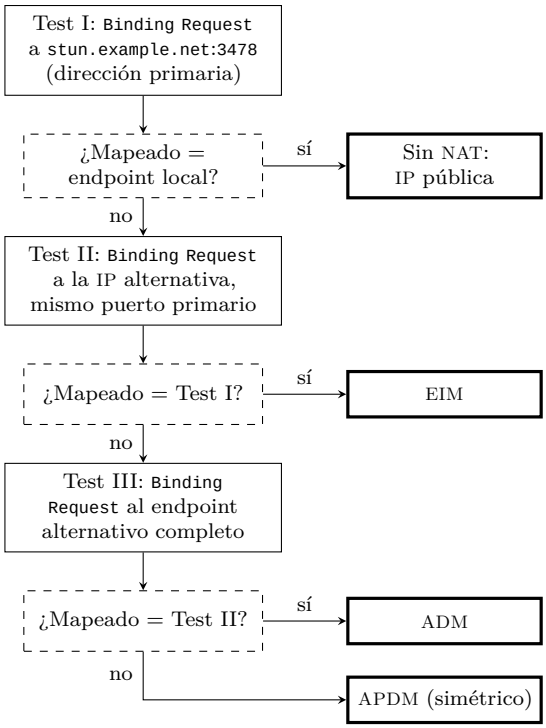


FIGURA 11.9: Determinación del tipo de mapeado NAT con STUN

Es importante precisar que el cliente debe utilizar el mismo socket (mismo puerto local) para todas las peticiones, de manera que el router mantienga la misma entrada en la tabla NAT.

El programa `turnutils_natdiscovery` aplica este procedimiento:

```
$ turnutils_natdiscovery -m -f stunserver2025.stunprotocol.org
-= Mapping Behavior Discovery =-
=======================================
No ALG: Mapped == XOR-Mapped
0: : IPv4. Response origin: : 3.22.142.132:3478
0: : IPv4. Response origin: : 3.22.142.132:3478
0: : IPv4. Other addr: : 3.135.212.85:3479
0: : IPv4. UDP reflexive addr: 203.0.113.2:37140
0: : IPv4. Local addr: : 0.0.0.0:37140
=======================================
No ALG: Mapped == XOR-Mapped
0: : IPv4. Response origin: : 3.135.212.85:3478
0: : IPv4. Other addr: : 3.22.142.132:3479
0: : IPv4. UDP reflexive addr: 203.0.113.2:37140
0: : IPv4. Local addr: : 0.0.0.0:37140
=======================================
NAT with Endpoint Independent Mapping!
=======================================
```

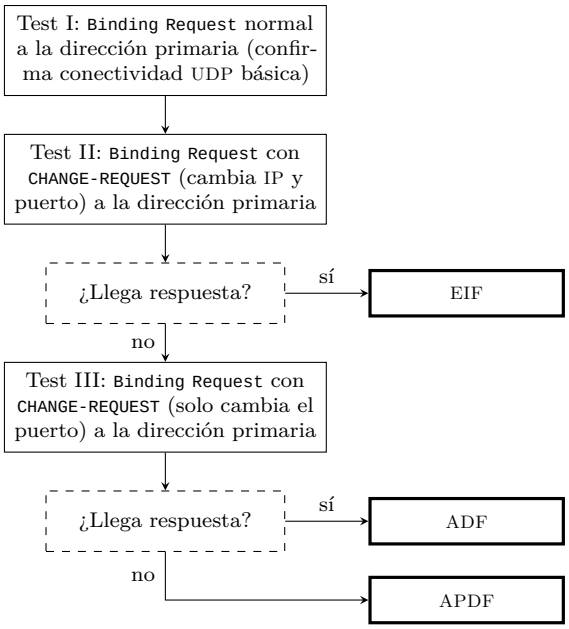


FIGURA 11.10: Determinación del tipo de filtrado NAT con STUN

```
-= Filtering Behavior Discovery =-
=====================================
No ALG: Mapped == XOR-Mapped
0: : IPv4. Response origin: : 3.22.142.132:3478
0: : IPv4. Other addr: : 3.135.212.85:3479
0: : IPv4. UDP reflexive addr: 203.0.113.2:60547
0: : IPv4. Local addr: : 0.0.0.0:60547
STUN receive timeout..
STUN receive timeout..
=====================================
NAT with Address and Port Dependent Filtering!
=====================================
```

11.10.4. Señalización

Con el procedimiento anterior, ambos extremos deben determinar su endpoint mapeado, si es que ambos están detrás de un NAT. La segunda tarea es comunicar sus endpoints (público o mapeado) a su contraparte. Para intercambiar estos endpoints se utiliza el servidor de señalización o *rendezvous* [36].

El servidor de señalización es un nodo público que ambos extremos conocen y al que pueden conectarse (usualmente con TCP). Piensa que estos extremos podrían ser clientes de un servicio de videollamada, o de un videojuego en línea. La comunicación con el servidor de señalización ocurriría en la fa-

se de espera a otros participantes o jugadores (el «lobby»). El servidor de señalización no interviene en la comunicación entre los extremos, solo sirve para intercambiar sus endpoints mapeados.

No hay un protocolo o especificación normalizada para la señalización porque depende mucho de cada aplicación específica. Debe manejar datos particulares del dominio de la aplicación (creación y registro en salas, partidas, reuniones, etc.) y seguramente también tendrá un sistema de registro y autenticación de usuarios.

El siguiente es un ejemplo simplificado, en el que `A` es un nodo detrás de NAT, `B` es un nodo con IP pública y `S` es un servidor de señalización y el objetivo es que `B` pueda establecer una conexión activa con un servidor (up programa) en `A`.

```
A -> S: Quiero unirme a la partida AX23.
A -> S: Mi endpoint mapeado es 180.20.30.13:4500 (descubierto con STUN).
B -> S: Mi endpoint público es 200.10.20.12:3200 (descubierto con STUN).
S -> B: El endpoint de A es 180.20.30.13:4500.
S -> A: El endpoint de B es 200.10.20.12:3200.
```

En realidad, siendo `B` un nodo público, no sería necesario que `A` obtenga su endpoint a través del servidor. En una situación en la que los dos extremos están detrás de sendos NAT sería imprescindible el intercambio mutuo de endpoints.

En este punto, si el NAT de A es de tipo *full-cone*/EIM, `B` puede abrir una conexión TCP directamente con `A` a través su endpoint mapeado. El router le entragará esos paquetes, porque la tabla NAT así lo indica: acepta cualquier origen remoto.

Sin embargo, si el NAT de A es de tipo *restricted-cone*/ADF o *port-restricted-cone*/APDF, los mensajes de `B` serán descartados. Es necesario aplicar una técnica adicional: *hole punching*.

11.10.5. *hole punching*

La técnica de *hole punching*[9] consiste en que una vez conocido el endpoint mapeado de la contraparte, el nodo privado envía un mensaje en el sentido contrario, para que el router registre el origen remoto como permitido. Dicho de otro modo, la «perforación» la realiza el nodo que actuará de servidor en su propio NAT. En el ejemplo anterior, si `A` crea una conexión al endpoint de `B`, el NAT de `A` aceptará mensajes entrantes de `B` siempre que el NAT sea tipo ADF o APDF.

[9] en español: «perforación de agujeros», traducción que por suerte nadie usa.

Esto también funciona si ambos extremos (A y B) están tras un NAT. El siguiente es un listado de mensajes que ilustra la secuencia de eventos para ese caso:

```
A -> S: Quiero unirme a la partida AX23.
B -> S: Quiero unirme a la partida AX23.
A -> S: Mi endpoint mapeado es 180.20.30.13:4500 (descubierto con STUN).
B -> S: Mi endpoint mapeado es 190.33.44.10:6500 (descubierto con STUN).
S -> B: El endpoint de A es 180.20.30.13:4500.
S -> A: El endpoint de B es 190.33.44.10:6500.
A -> B: Perforación. El \NAT de A registra el endpoint de B como permitido, El \NAT de B
↪  lo descarta.
B -> A: Perforación. El \NAT de B registra el endpoint de A como permitido, El \NAT de A
↪  lo descarta.
A -> B: Mensaje aceptado.
B -> A: Mensaje aceptado.
```

El «agujero» no es más que una entrada en la tabla NAT de cada extremo y, como tal, tiene asociado un tiempo de inactividad (§ 11.6). Si el flujo puede pasar periodos en silencio —una partida en pausa, una llamada sin audio—, ambos extremos deben enviar los mensajes *keep alive* con la frecuencia adecuada para reiniciar el contador antes de que expire (cada 15–20 segundos). El contenido del mensaje puede ser ignorado por el receptor. También se puede enviar una petición STUN, que además permite comprobar que el endpoint mapeado no ha cambiado.

Sin embargo, ni siquiera con esto se puede atravesar un NAT simétrico (o ADM/ADF) porque el endpoint mapeado descubierto con STUN no sirve para la conexión inversa. Cuando A trate de crear un flujo hacia B, el NAT de A asignará un puerto mapeado distinto y el NAT de B filtrará esos paquetes.

La solución es utilizar un servidor intermedio para la tranferencia de datos. Se denominan servidores *relay* o TURN.

11.10.6. TURN e ICE

El servidor TURN [39] es un nodo público que actúa de proxy. Esencialmente ambos extremos establecen una conexión con el servidor y le dan los datos que quieren enviar a su contraparte.

TURN es el último recurso, porque añade latencia y consume ancho de banda en el servidor; efectos muy poco deseables en aplicaciones de videollamadas o videjuegos, que son las principales candidatas a utilizar *NAT traversal*.

Es sencillo entender que todas estas técnicas de *NAT traversal* implican mayor coste y menor rendimiento cuanto más restrictivo es el NAT. Para simplificar y sistematizar la elección de la técnica más adecuada se utiliza

el protocolo ICE [40]. ICE permite a los extremos descubrir sus endpoints mapeados, intercambiarlos y evaluar cuál es la mejor técnica para efectuar la comunicación.

ICE no evalúa la situación. En su lugar, genera una lista de endpoints candidatos y los evalúa por orden de prioridad. La lista incluye (en este orden):

host
: endpoints locales (porque los extremos podrían estar en la misma red)

srflx
: endpoints mapeados descubiertos con STUN.

relay
: endpoints de servidores TURN.

La lista se intercambia a través del servidor de señalización y cada extremo va probando los candidatos hasta que encuentra uno que funciona.

Tanto WebRTC como SIP (dos protocolos muy utilizados en aplicaciones de videollamadas) emplean ICE para establecer la comunicación con NAT transversal.

Y ¿qué más?

Este capítulo ha introducido el concepto de red privada —algo tan cotidiano como desconocido—, sus variantes —líneas alquiladas, redes híbridas, intranets y extranets— y el papel que juega NAT como su variante CGNAT para paliar el agotamiento de direcciones IPv4. También tiene un alto precio: NAT rompe la conectividad extremo a extremo, perjudica a los protocolos que transportan direcciones en su carga útil e impide la simetría —los nodos de la red privada puede iniciar flujos hacia nodos de Internet, pero no al revés—, de modo que cualquier aplicación que necesite recibir flujos entrantes —videollamadas, videojuegos en red, P2P— debe recurrir a técnicas de *NAT traversal*, que también se han abordado.

Sin embargo, estos problemas son propios de IPv4 y su alarmante excasez de direcciones, que no permite asignar una IP pública a cada dispositivo, como sería deseable —al menos desde la lógica de la arquitectura de Internet.

Con IPv6 el, espacio de direcciones crece de forma exponencial: literalmente miles de millones de direcciones públicas por hogar. Veremos esa y otras ventajas de IPv6 más adelante.

Capítulo 12

Confiabilidad y control de flujo

Al terminar este capítulo, entenderás:

- Qué es la confiabilidad y cómo se logra.
- Qué es el control de flujo y por qué es necesario para lograr confiabilidad.
- Qué son y cómo funcionan los protocolos ARQ.
- En qué consiste y cómo funciona la ventana deslizante.
- Qué es la confirmación acumulativa y negativa.

La red no garantiza por sí misma que los datos lleguen correctamente a su destino. Tampoco impide que un emisor envíe datos más rápido de lo que el receptor es capaz de procesarlos. Para resolver cada uno de estos problemas existen dos mecanismos: la confiabilidad y el control de flujo, respectivamente. Ambos se pueden implementar en protocolos de capas diferentes y con enfoques dispares, aunque lo habitual es encontrarlos en la capa de transporte (como TCP) o en la de enlace (como HDLC).

Se dice que una comunicación es fiable o **confiable** si los datos llegan a su destino exactamente tal cual partieron del origen, sin ningún cambio ni omisión, pero también sin partes duplicadas o desordenadas. Hay muchas razones por las que podrían darse todos esos problemas: interferencias, atenuación, retrasos, disparidad de rutas, limitación en el almacenamiento intermedio, etc.

Por otro lado, el **control de flujo** es el mecanismo que permite a los participantes limitar (incluso interrumpir temporalmente) el flujo de datos entre ellos. Aunque el concepto se puede aplicar a varias situaciones, aquí emplearemos la expresión «control de flujo» para un caso específico: el ajuste que el receptor solicita al emisor para evitar que lo sature, es decir, que sobrepase su capacidad de procesar los datos entrantes (error de *overrun*).

La confiabilidad y el control de flujo se suelen estudiar a la vez porque habitualmente se diseñan e implementan de forma conjunta. Además, se

complementan; sin control de flujo, probablemente se producirían pérdidas que el mecanismo de confiabilidad tendría que resolver mediante retransmisiones, desperdiciando capacidad del canal.

Todo mensaje que viaja entre dos dispositivos lo hace a través de algún medio físico, sea el aire, un cable de cobre o una fibra óptica, y por eso está restringido por las limitaciones físicas. Incluso en el vacío, la señal sufre cambios causados por interferencias o ruido electromagnético, y la intensidad de la señal se degrada por la distancia o la resistencia del medio. Por eso, cualquier mensaje que se envíe a través de una red podría llegar modificado o incompleto, o incluso no llegar. Llamamos a todo esto «errores de transmisión» y para solucionarlos existen distintas técnicas, que se clasifican en dos categorías: detección y corrección de errores.

La detección de errores más simple consiste en añadir información redundante al mensaje original, normalmente solo unos pocos bytes. Puede ser algo tan sencillo como un bit de paridad, pero típicamente se utiliza una comprobación de redundancia cíclica (CRC). El emisor aplica ese algoritmo a los datos de partida e incluye el resultado en el propio mensaje. Al llegar, el receptor realiza el mismo cálculo y compara el resultado con el que ha recibido. Si estos valores coinciden, el mensaje es correcto; si difieren, el mensaje ha sufrido algún cambio.

Este tipo de algoritmo solo detecta la alteración, pero no determina en qué parte del mensaje y menos aún cuál fue la alteración. Este es el caso del FCS de Ethernet, o del checksum utilizado en las cabeceras de IP, UDP, TCP, etc., que es un valor de tan solo 16 bits. Se adjunta una explicación detallada de este algoritmo en `/inet-checksum/checksum.ipynb`.

Si la probabilidad de error es baja, lo más sencillo y eficiente es detectar los errores y retransmitir los mensajes afectados. Si por el contrario la probabilidad es alta, o no es posible o conveniente realizar la retransmisión, entonces se utilizan algoritmos de corrección de errores. Estos algoritmos añaden una cantidad significativa de información redundante a los mensajes, lo que también conlleva recursos de cómputo adicionales. El más conocido de estos algoritmos de corrección es Reed-Solomon, que se utiliza en comunicaciones vía satélite.

Para la retransmisión de mensajes erróneos se utilizan variantes de los protocolos ARQ (Automatic Repeat reQuest) en los que el receptor envía mensajes de **confirmación** (también llamados «reconocimientos» o ACK). Estos protocolos permiten al emisor obtener información actualizada sobre el estado del receptor. Cada vez que el emisor recibe una confirmación tiene constancia de que el receptor está listo para continuar, y envía un nuevo

mensaje, es decir, al mismo tiempo proporciona confirmación y controla el flujo.

Aunque se denominan «protocolos», los ARQ no definen formato o sintaxis concreta para los mensajes. Únicamente definen una serie de reglas y pautas que deben cumplir los mensajes de datos y confirmaciones. Puedes entenderlos como una especie de patrones para crear protocolos confiables concretos.

Las siguientes secciones explican cómo funciona cada uno de los ARQ clásicos.

12.1. Parada y espera

En el protocolo *parada y espera (stop and wait)* el emisor envía un mensaje y espera su confirmación. Si no la recibe en un tiempo determinado, reenvía el mensaje. Si recibe la confirmación, envía el siguiente mensaje.

En detalle:

- El emisor envía un mensaje de datos que incluye un checksum.
- El emisor inicia un temporizador de retransmisión.
- El receptor recibe el mensaje y calcula el checksum.
  - Si coincide, envía una confirmación.
  - Si no coincide, descarta el mensaje.
- Si el emisor recibe la confirmación, cancela el temporizador y envía el siguiente mensaje.
- Si el temporizador expira antes de recibir la confirmación, reenvía el mensaje.

El temporizador debe ajustarse en relación al tamaño de los mensajes y al RTT, dado que un valor demasiado corto provocaría retransmisiones prematuras e innecesarias[1] y, como resultado, el receptor recibe un duplicado del mensaje. Para que el receptor pueda detectar esta situación utiliza un número de secuencia de un único bit: alterna entre 0 y 1. Si el receptor recibe dos mensajes consecutivos con el mismo número de secuencia, sabrá que es un duplicado y lo descartará, pero debe confirmarlo igualmente. El duplicado también aparece cuando se pierde la confirmación y, si el receptor no la repite, el emisor seguirá retransmitiendo ese mensaje indefinidamente.

Hay un detalle que puede resultar confuso y merece la pena aclarar. Es frecuente que muchos protocolos tomen el convenio de enviar en la confirmación el número de secuencia del **siguiente** mensaje esperado en lugar del que corresponde al mensaje que acaba de llegar al receptor. Es decir, para

[1] llamada a menudo «retransmisión espuria»

confirmar el mensaje de datos con número de secuencia 0, el receptor envía una confirmación con el número de secuencia 1. En este texto tomaremos también este convenio. La confirmación necesita ese número de secuencia; si el temporizador expira antes de tiempo, al emisor puede llegarle tarde una segunda confirmación de un mensaje que ya estaba confirmado. Como indica cuál es el siguiente mensaje que espera el receptor, el emisor sabe que no se refiere al mensaje que acaba de enviar y la descarta. Sin ese número, la tomaría por la confirmación de ese mensaje aunque se hubiera perdido.

La Figura 12.1 muestra los escenarios que contempla *parada y espera*:

- El emisor envía el mensaje 0 e inicia un temporizador de retransmisión (reloj verde). El mensaje llega satisfactoriamente al receptor, que lo acepta y envía una confirmación (ack 1), que también llega correctamente. Al recibir esa confirmación, el emisor cancela el temporizador de retransmisión (reloj tachado).
- El emisor envía el mensaje 1, pero se pierde. Como consecuencia su temporizador expira (reloj rojo) y retransmite el mensaje 1 (RTX). El receptor lo confirma (ack 0) y el temporizador es cancelado.
- El segundo mensaje 0 (está reciclando el número de secuencia) llega al receptor, que lo acepta y envía una confirmación (ack 1), pero se pierde. El temporizador expira y el emisor retransmite el mensaje 0. Esta situación genera un duplicado porque el receptor ya tenía esos datos. Descarta el duplicado, pero lo confirma igualmente.

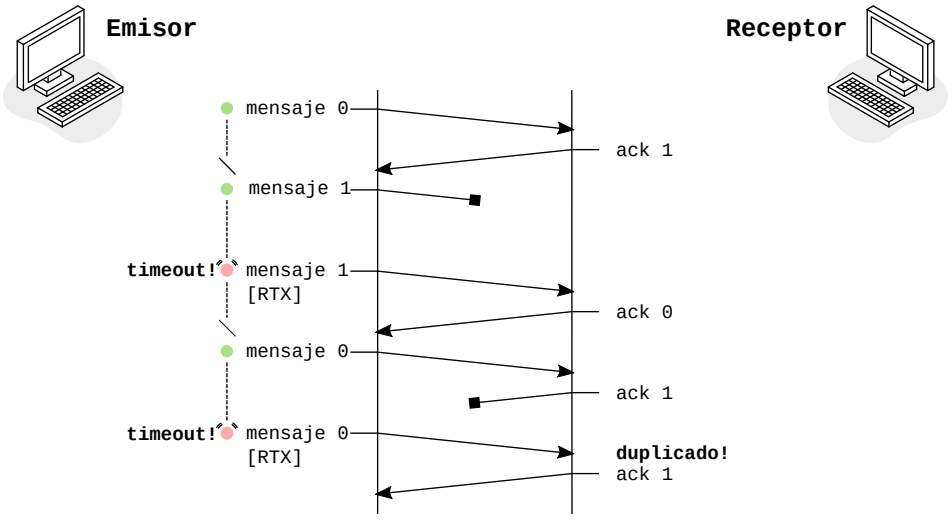


FIGURA 12.1: Diagrama de secuencia del protocolo *parada y espera*

La implementación de *parada y espera* es simple, pero ineficiente. Como el emisor debe esperar la confirmación antes de enviar el siguiente mensaje, el canal de comunicación se infrautiliza en gran medida, afectando gravemente a la velocidad de transmisión neta.

12.2. Repetición continua

Repetición continua (go back n) intenta maximizar la ocupación del canal. Para ello, el emisor envía varios mensajes consecutivos, sin esperar una confirmación antes de enviar el siguiente. Esa técnica se llama *canalización (pipelining)* y consigue una mejora sustancial del ancho de banda efectivo.

Para que funcione, el emisor tiene que guardar temporalmente el contenido de los mensajes no confirmados por si fuera necesario retransmitirlos. El espacio donde se almacenan estos mensajes suele ser una cola circular que se conoce como *buffer de envío*.

Cuando el receptor recibe un mensaje, comprueba el *checksum*, envía una confirmación al emisor y entrega el mensaje a la capa superior. Si algún mensaje se pierde, el receptor recibe mensajes fuera de secuencia. Cuando eso sucede, el receptor **descarta los mensajes** aunque sean correctos y, para cada uno de ellos, envía obligatoriamente una confirmación para el último mensaje recibido correctamente (y que respetaba la secuencia sin huecos), es decir, el emisor puede recibir múltiples confirmaciones para un mismo mensaje de datos. Se las llama «confirmaciones duplicadas».

Por tanto, las confirmaciones de repetición continua son acumulativas (§ 12.4). La confirmación con número n se aplica a todos los mensajes anteriores a n, no solo al $n - 1$. No es opcional; como el receptor solo confirma el último mensaje recibido en orden, si fuera una confirmación individual bastaría con perder una para que el receptor respondiera a cada retransmisión de ese mensaje con el número del siguiente esperado. El mensaje nunca quedaría confirmado y el emisor lo retransmitiría indefinidamente.

Cada mensaje tiene su propio temporizador de retransmisión. Si expira el de un mensaje, el emisor lo retransmite junto con todos los posteriores que ya había enviado, porque el receptor los habrá descartado al llegar fuera de secuencia. Como cada transmisión vuelve a iniciar el temporizador de su mensaje, los temporizadores de esos mensajes posteriores se reinician en lugar de expirar.

Los mensajes incluyen también un número de secuencia que el receptor utiliza para confirmarlos. El espacio de números de secuencia es una potencia de 2 porque se representa en un *campo de bits* en la cabecera del mensaje.

Con k bits se obtiene un espacio de números de secuencia de 2^k mensajes, si bien no es posible enviar 2^k mensajes sin confirmación, porque provoca una situación ambigua que hay que evitar. Imagina una implementación hipotética con un número de secuencia de 2 bits (2^2 mensajes) con el siguiente escenario:

- El emisor envía los 4 mensajes posibles, con números de secuencia 0 a 3 incluidos.
- Todos los mensajes llegan correctamente a su destino.
- Todas las confirmaciones se pierden, pero el receptor lo desconoce.
- El temporizador del mensaje 0 expira.
- El emisor retransmite el mensaje 0.

En esa situación el receptor tomaría esa retransmisión por un mensaje nuevo que está reutilizando el número de secuencia 0, provocando con ello el fallo del protocolo. Para evitar esta situación, se define una *ventana de emisión* con un tamaño menor:

$$W_s \leq 2^k - 1 \tag{12.1}$$

donde:

W_s: tamaño de la ventana de emisión

La ventana de emisión determina qué mensajes se pueden enviar en cada momento, sin esperar confirmaciones y sin provocar ambigüedad. Cuando un mensaje es confirmado, su número de secuencia queda libre para ser reutilizado con un nuevo mensaje, y el buffer se reutiliza una y otra vez durante la transmisión. Esa reutilización cíclica de los números de secuencia es la *ventana deslizante (sliding window)*.

El ejemplo de la Figura 12.2 utiliza números de secuencia de $k = 2$ bits y una ventana de emisión $W_s = 3$. Empieza enviando los 3 mensajes permitidos [0–2], perdiéndose el 1. Al recibir el mensaje 0 el receptor envía ack 1; eso desliza la ventana y permite al emisor enviar el mensaje 3. Los mensajes 2 y 3 llegan al receptor fuera de secuencia (falta el 1). Por eso el receptor los descarta y envía confirmaciones ack 1.

Como el mensaje 1 no ha sido confirmado, su temporizador expira poco después y el emisor retransmite el 1 y los posteriores que ya había enviado, el 2 y el 3, cuyos temporizadores se reinician. El receptor responde con las confirmaciones 2, 3 y 0 respectivamente. La figura en el lado del emisor representa el buffer de envío. Los cuadritos verdes representan los mensajes enviados no confirmados y el marco naranja corresponde con la ventana de emisión, es decir, los mensajes que puede enviar en cada momento.

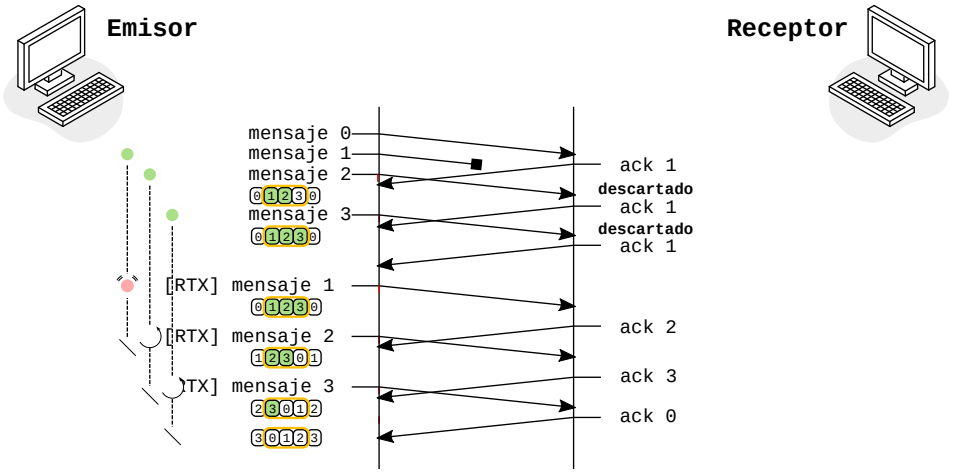


Figura 12.2: Ejemplo de *repetición continua*

Nótese en todo caso que el receptor no establece temporizadores ni retransmisiones cuando envía confirmaciones, y por supuesto el emisor no confirma las confirmaciones que recibe.

El nombre *go back n* hace referencia al hecho de que, en caso de perder un mensaje, quizá haya que «volver n mensajes atrás» y reenviarlos.

12.3. Repetición selectiva

La *repetición selectiva (selective repeat)* también aplica la técnica de canalización y tiene un temporizador de retransmisión por mensaje, pero, a diferencia de *repetición continua*, los mensajes que llegan fuera de secuencia se guardan en lugar de descartarse y cada confirmación se refiere a un único mensaje. Cuando expira un temporizador, se retransmite solo ese mensaje.

Su objetivo es evitar que el emisor tenga que retransmitir mensajes que llegaron correctamente, y de ahí el nombre del protocolo. Para lograrlo, el receptor necesita también un *buffer de recepción* que es donde guarda los mensajes fuera de secuencia. Cuando recibe el mensaje que «rellena el hueco», entrega al nivel superior todos los mensajes contiguos que haya en el buffer.

Si lo que se pierde es una confirmación, el emisor retransmite el mensaje correspondiente al expirar su temporizador, y el receptor debe volver a confirmarlo aunque ya lo tuviera.

El límite para el tamaño de la ventana es menor que en repetición continua. El receptor puede almacenar mensajes fuera de secuencia siempre que no se puedan confundir retransmisiones con mensajes de la siguiente ventana. Por eso, para un número de secuencia de k bits, el tamaño de ventana debe cumplir:

$$W_s = W_r \leq 2^{k-1} \tag{12.2}$$

donde:

> W_s es el tamaño de la ventana del emisor y W_r es el tamaño de la ventana del receptor

Un ejemplo en la Figura 12.3: con números de secuencia de $k = 3$ bits ($W_s = 2^{k-1} = 4$), un emisor envía la ventana completa (mensajes [0–3]) y se pierde el 2. El receptor confirma los mensajes 0, 1 con los ACK 1 y 2, y guarda el mensaje 3 en su buffer de recepción. Las confirmaciones recibidas por el emisor deslizan su ventana, que ahora puede enviar los mensajes 4 y 5, pero no más (tiene 4 mensajes en vuelo: [2–5]).

A diferencia de repetición continua, los mensajes fuera de secuencia no provocan confirmaciones duplicadas: el receptor guarda los mensajes 3, 4 y 5 y confirma cada uno de ellos (ACK 4, 5 y 6), de modo que el emisor cancela sus temporizadores.

Cuando expira el temporizador del mensaje 2, el emisor retransmite solo ese mensaje. Al recibirlo, el receptor entrega al nivel superior los mensajes [2–5] y lo confirma con el ACK 3.

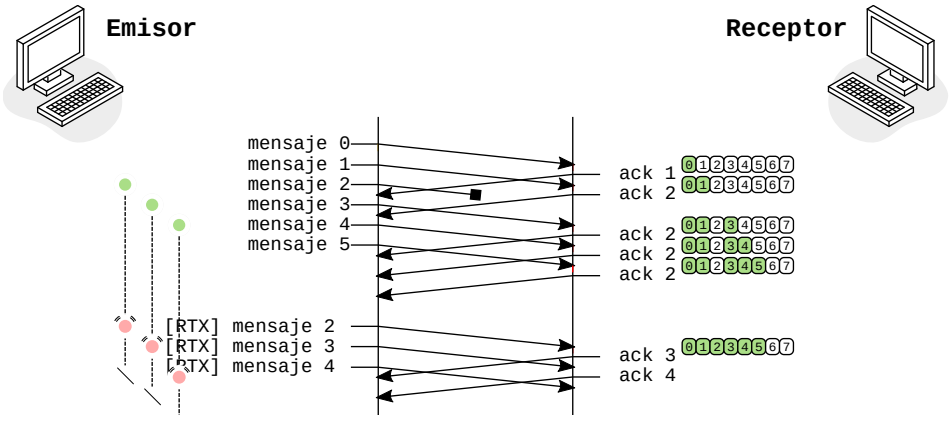


FIGURA 12.3: Ejemplo de *repetición selectiva*

12.4. Confirmación acumulativa

La *confirmación acumulativa* es un tipo diferente de confirmación. En lugar de aplicarse a un único mensaje, se aplica a todo lo anterior.

En repetición selectiva cada confirmación se refiere a un único mensaje, de modo que la pérdida de una confirmación obliga a retransmitir un mensaje que ya había llegado. Con *confirmación acumulativa*, en cambio, cada confirmación se aplica también a todos los mensajes anteriores. En repetición continua esto ocurre de forma implícita, porque el receptor solo acepta mensajes en orden (§ 12.2). En repetición selectiva, sin embargo, debe hacerse de forma explícita: como el receptor guarda mensajes fuera de secuencia, una confirmación acumulativa solo puede cubrir la parte de la secuencia que no tiene huecos.

Por ejemplo, un ACK acumulativo con número de secuencia 4 significa que todos los mensajes hasta el 3 incluido han llegado correctamente. TCP utiliza este tipo de confirmación.

La confirmación acumulativa tiene dos ventajas. Por un lado, la pérdida de una confirmación no tiene consecuencias si llega otra posterior, que la sustituye. Por otro lado, no es necesario enviar una confirmación para cada mensaje de datos. Muchos protocolos retrasan intencionadamente la confirmación (se llama «ACK retardado» o *delayed ACK*) para poder enviar una única confirmación para varios mensajes de datos.

A cambio, cuando hay un hueco en la secuencia, el receptor solo puede confirmar los mensajes anteriores al hueco. Cada mensaje que llega fuera de secuencia provoca de nuevo esa misma confirmación, por lo que el emisor recibe «confirmaciones duplicadas» y no puede saber cuáles de los mensajes posteriores han llegado.

La Figura 12.4 muestra un ejemplo con repetición selectiva y confirmación acumulativa. Con números de secuencia de $k = 3$ bits, un emisor ha enviado los mensajes [0–3] y se ha perdido el 2. El receptor envía confirmaciones con secuencia 1 y 2, que reconocen respectivamente los mensajes 0 y 1 (por ser acumulativa, bastaría con enviar la 2). Cuando llega el mensaje 3, lo almacena en su buffer de recepción y envía de nuevo una confirmación con secuencia 2. Más tarde, cuando expira el temporizador del mensaje 2, el emisor lo retransmite. El receptor, al recibirlo, dispone de la secuencia completa [0–3] y envía un ACK con secuencia 4 que confirma todos los mensajes recibidos. Mientras tanto es posible que expiren más temporizadores (el del mensaje 3 en la figura) y que haya retransmisiones innecesarias, porque el emisor no puede saber que el mensaje 3 ya llegó. Al llegar el ACK 4 se confirman todos los mensajes pendientes y se cancelan sus temporizadores

(la figura solo muestra los de los mensajes [2–3], que son los relevantes para el ejemplo).

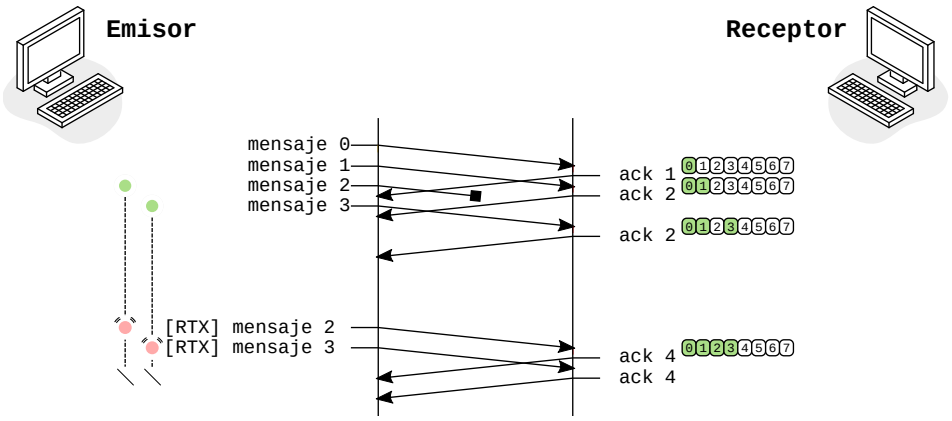


FIGURA 12.4: Ejemplo de *repetición selectiva* con confirmación acumulativa

12.5. ARQ y ancho de banda × retardo

En realidad, los tres protocolos ARQ cumplen una misma condición general:

$$W_s + W_r \leq 2^k \tag{12.3}$$

La Tabla 12.1 muestra cómo encaja esa condición en cada protocolo:

Protocolo	Restricción	Aplicación de (12.3)	W_s máx	W_r máx
Parada y espera	$W_s = W_r = 1$	$1 + 1 \leq 2^k$	1	1
Repetición continua	$W_r = 1$	$W_s + 1 \leq 2^k$	$2^k - 1$	1
Repetición selectiva	$W_s = W_r = W$	$2W \leq 2^k$	2^{k-1}	2^{k-1}

CUADRO 12.1: Aplicación de la condición $W_s + W_r \leq 2^k$ a los protocolos ARQ

Los límites de la Tabla 12.1 fijan el tamaño **máximo** de ventana que admite cada protocolo para un número de secuencia de k bits sin ambigüedad. Sin embargo, para que la canalización aproveche el canal, la ventana debe ser lo suficientemente grande como para que el emisor pueda seguir enviando mensajes continuamente, sin bloquearse a la espera de confirmaciones.

Para estimar ese tamaño intervienen dos valores:

- El *retardo de transmisión* (t_{trans}): lo que tarda el emisor en poner el mensaje completo en el canal. Depende del tamaño del mensaje (L, en bits) y de la velocidad de transmisión (R, en bits/s): $t_{\text{trans}} = L/R$.
- El *retardo de propagación* (t_{prop}): lo que tarda cada bit en recorrer el canal de extremo a extremo. Depende de la longitud del enlace (d, en metros) y de la velocidad de propagación de la señal en el medio (v, en m/s): $t_{\text{prop}} = d/v$.

Si el emisor empieza a transmitir el mensaje 0 en el instante 0, termina de enviarlo en t_{trans} y el último bit llega al receptor en $t_{\text{trans}} + t_{\text{prop}}$. Despreciando el retardo de procesamiento y el de transmisión de la confirmación (mucho más corto que el mensaje de datos), el ACK llega al emisor en $t_{\text{trans}} + 2t_{\text{prop}}$. En ese intervalo el emisor puede transmitir como mucho los W_s mensajes de su ventana, que ocupan el canal durante $W_s \cdot t_{\text{trans}}$. Para no quedar bloqueado debe cumplirse $W_s \cdot t_{\text{trans}} \geq t_{\text{trans}} + 2t_{\text{prop}}$, es decir:

$$W_s \geq 1 + 2a \tag{12.4}$$

donde:

W_s es la ventana de emisión y $a = t_{\text{prop}}/t_{\text{trans}}$

El término $2a$ indica el número de mensajes que se pueden transmitir mientras el primero llega al receptor y vuelve su ACK o, dicho de otro modo, los mensajes que se pueden enviar durante el RTT: $2 \cdot t_{\text{prop}}$. Es una simplificación; el RTT que se mide por ejemplo con `ping` incluye también los retardos de transmisión, cola y procesamiento, mientras que aquí consideramos solo propagación.

Multiplicando $2a$ por el tamaño del mensaje se obtiene el BDP (Bandwidth Delay Product), la cantidad de datos en vuelo necesaria para mantener ocupado el canal:

$$\begin{aligned} \text{BDP} &= 2a \cdot L = 2 \cdot \frac{t_{\text{prop}}}{t_{\text{trans}}} \cdot L = 2 \cdot \frac{t_{\text{prop}}}{L/R} \cdot L \\ &= R \cdot 2t_{\text{prop}} = R \cdot \text{RTT} \end{aligned} \tag{12.5}$$

El tamaño del mensaje se cancela, de modo que el BDP solo depende de la velocidad de transmisión y del retardo del enlace, y de ahí su nombre: producto ancho de banda[2] por retardo.

[2]En el sentido de velocidad de transmisión

Si la ventana es menor que $1+2a$ mensajes, el emisor transmite su ventana completa y debe esperar la primera confirmación. En ausencia de errores, el aprovechamiento del canal (U), o fracción de tiempo que el emisor transmite, es:

$$U = \begin{cases} 1 & \text{si } W_s \geq 1+2a \\ \dfrac{W_s}{1+2a} & \text{si } W_s < 1+2a \end{cases} \tag{12.6}$$

Parada y espera es $W_s = 1$, con $U = 1/(1+2a)$, que se corresponde con su bajo rendimiento en canales con mucho retardo o con una velocidad de transmisión alta.

Por tanto, la ventana tiene dos límites: $W_s \geq 1+2a$ es el mínimo para que el protocolo sea eficiente y $W_s \leq W_{s,\text{máx}}$ es el máximo para que sea correcto y que depende del protocolo según la Tabla 12.1 ($2^k - 1$ para repetición continua y 2^{k-1} para repetición selectiva):

$$1 + 2a \leq W_s \leq W_{s,\text{máx}} \tag{12.7}$$

Un ejemplo: en un enlace de fibra óptica[3] de 1000 km a 10 Mbps con mensajes de 1000 bytes:

- $t_{\text{trans}} = L/R = 8000$ bits/10 Mbps $= 0{,}8$ ms
- $t_{\text{prop}} = d/v \approx 10^6$ m$/(2 \cdot 10^8$ m/s$) = 5$ ms
- $a = 5/0{,}8 = 6{,}25$
- $1 + 2a = 13{,}5$. En la ventana deben caber al menos 14 mensajes.

La Tabla 12.2 recoge el valor mínimo de k que necesita cada protocolo en este enlace y en un enlace satelital ($t_{\text{prop}} \approx 250$ ms) para conseguir un aprovechamiento óptimo.

Es posible utilizar ventanas mayores de $1 + 2a$, pero no mejora el aprovechamiento del canal y además aumenta el coste: buffers de envío mayores (y de recepción, en repetición selectiva), un campo de número de secuencia más largo en la cabecera y, en repetición continua, más mensajes que retransmitir tras cada pérdida.

[3] La luz se propaga en la fibra a unos $2 \cdot 10^8$ m/s.

	Fibra (1000 km)	Satélite
Retardo de propagación (t_{prop})	5 ms	250 ms
$a = t_{\text{prop}}/t_{\text{trans}}$ (mensajes)	6,25	312,5
$W_{s,\text{mín}} = 1 + 2a$	13,5	626
k en repetición continua	4	10
$W_{s,\text{máx}} = 2^k - 1$	15	1023
k en repetición selectiva	5	11
$W_{s,\text{máx}} = 2^{k-1}$	16	1024

CUADRO 12.2: Elección de k para un enlace de fibra y otro por satélite

12.6. Confirmación negativa

Es posible mejorar la repetición selectiva, aunque, como siempre, a cambio de complejidad adicional. El receptor puede utilizar un nuevo tipo de confirmación —la confirmación negativa o NACK— para notificar explícitamente cuál es el mensaje corrupto o perdido. De ese modo, el emisor puede retransmitir inmediatamente ese mensaje en lugar de esperar a que expire su temporizador, aprovechando mejor el canal.

Retomando el ejemplo de la Figura 12.4 —números de secuencia de 3 bits, mensajes [0–3] enviados con pérdida del mensaje 2—, con NACK el receptor no necesita esperar a que el hueco se rellene para reaccionar: en cuanto recibe el mensaje 3 fuera de secuencia detecta el hueco en 2 y envía un NACK con ese número, junto con la confirmación que le corresponda. El emisor retransmite el mensaje 2 de inmediato en lugar de esperar a que expire su temporizador, que se cancela tan pronto como llega el NACK. El resto de la recuperación es igual que antes: al completar la secuencia [0–3], el receptor envía un ACK con secuencia 4 confirmando todos los mensajes pendientes.

12.7. Confiabilidad en comunicaciones dúplex

Los protocolos de confiabilidad tratan la comunicación solo en un sentido de la comunicación. Hemos hablado todo el tiempo de «emisor» y «receptor». En realidad se explica de este modo por simplicidad y para ayudar a entender su funcionamiento. Nada impide que en un canal dúplex ambos extremos actúen como emisores y receptores a la vez. En ese caso, esos protocolos trabajan de forma independiente para cada uno de los sentidos

de la comunicación empleando ventanas y números de secuencia independientes.

En realidad, utilizar protocolos de confiabilidad en ambos sentidos permite optimizar algunas situaciones. Por ejemplo, resulta ineficiente enviar varios mensajes pequeños consecutivos. Por eso, cuando uno de los extremos necesita enviar una confirmación, que solo ocupa una simple cabecera, puede esperar «un poco» y aprovechar un mensaje de datos saliente para colocar la confirmación en su cabecera. A este tipo de optimizaciones se las llama *piggybacking* y es tan habitual que los formatos de las cabeceras se diseñan específicamente para hacerlo posible.

Y ¿qué más?

Los protocolos ARQ son la base de la mayoría de los protocolos que proporcionan confiabilidad en distintas capas. Algunos de estos protocolos son IEEE 802.11 (WiFi), HDLC, Bluetooth o LTE, que son protocolos de la capa de enlace[4], TCP o SCTP que son protocolos de transporte, o TFTP que es un protocolo de aplicación. Sin duda alguna, el más importante de todos estos protocolos es TCP. Por eso trataremos en detalle su funcionamiento y características en el capítulo 13.

[4]En realidad, algunos de estos protocolos abarcan varias capas, pero los mecanismos de confiabilidad los ofrecen en la capa de enlace.

Capítulo 13

Confiabilidad y control de flujo en TCP

Al terminar este capítulo, entenderás:

- Cómo se establece y termina una conexión TCP.
- Qué implicaciones tiene el control de flujo de TCP.
- Cómo detecta y resuelve TCP los errores de transmisión.

El protocolo de transporte TCP ya se ha presentado brevemente; este capítulo trata el soporte que ofrece para confiabilidad y control de flujo [41]. TCP utiliza un mecanismo de retransmisión de mensajes y confirmaciones de tipo repetición selectiva con confirmación acumulativa (§ 12.4) y ventana deslizante. Es *full duplex* de modo que aplica lo dicho en § 12.7. También puede utilizar un sistema de confirmación selectiva para mejorar la eficiencia, aunque es algo opcional que deben negociar explícitamente los extremos durante el establecimiento de la conexión.

Para cada conexión establecida, el SO crea un TCB (Transmission Control Block). Se trata de una estructura de datos que almacena toda la información relacionada con la conexión: dirección del otro extremo, números de secuencia, temporizadores, puertos, estado, etc. y también el buffer donde coloca los datos que el proceso emisor envía —`socket.send()`—, llamado *buffer de envío (sending buffer)*. En el otro extremo, el receptor tiene un *buffer de recepción (receiving buffer)* donde almacena los datos hasta que el proceso receptor los solicite, con `socket.recv()`.

A partir de los datos almacenados en el buffer de envío, el procedimiento de **segmentación**, determina cuántos bytes se incluirán en el siguiente mensaje TCP (llamado «segmento») que se envíe. El tamaño del segmento depende de varias variables que veremos en las siguientes secciones y también cuando hablemos de congestión (§ 14.5).

Una diferencia interesante respecto a los protocolos de estilo ARQ es que TCP no identifica los segmentos. En lugar de ello, numera cada uno de los bytes del flujo completo, para lo que usa un entero de 32 bits. En particular, el número de secuencia que corresponde al primer byte de la carga útil de cada segmento aparece en el campo `sequence number` de su cabecera y es por eso que se le llama «número de secuencia del segmento». Si por ejemplo se envía un segmento con número de secuencia 1000 y su carga útil tiene 500 bytes, el número de secuencia del siguiente byte (que será el primero del siguiente segmento) será 1500. Del mismo modo, la confirmación que envía el receptor corresponde al número de secuencia del siguiente byte que espera recibir, y que aparecerá en el campo `acknowledge number` de la cabecera.

13.1. Conexión

El procedimiento de conexión de TCP permite a los dos extremos intercambiar varios parámetros de configuración. El más importante es el *número de secuencia inicial* de cada extremo (o ISN). Cada uno de los extremos genera su ISN mediante una función pseudo-aleatoria, que aparecerá únicamente en el primer segmento que envíen. A partir de ese valor, se numera cada byte del flujo de datos. La conexión responde a un patrón muy concreto de tres mensajes, llamado *three-way handshake*, que se suele traducir literalmente como *triple apretón de manos*.

```
1. --> seq:1200, SYN
2  <-- seq:4800, SYN/ACK, ack:1201
3. --> seq:1201, ACK, ack:4801
```

LISTADO 13.1: TCP: Secuencia de una conexión

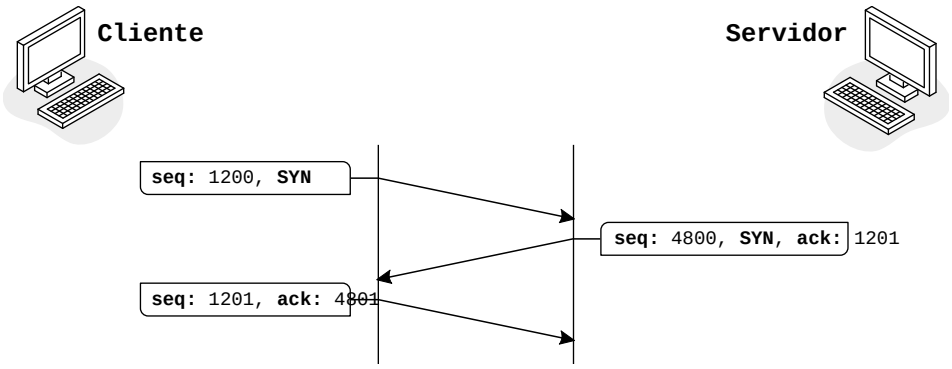


FIGURA 13.1: TCP: Patrón *triple handshake* durante la conexión

La conexión la inicia siempre el cliente, por lo que el primer segmento es suyo. Este segmento incluye el ISN del cliente (es el 1200 del ejemplo de la Figura 13.1 y del esquema equivalente del Listado 13.1) y lleva activo el flag SYN[1] únicamente. El servidor responde con un segmento que incluye su ISN (4800 en la figura) y lleva activos los flags SYN y ACK, y por tanto el contenido del campo `acknowledge number` es relevante y sirve como confirmación del primer mensaje del cliente (1201). En realidad el flag ACK ya siempre estará activo hasta el fin de la conexión. El segundo segmento del cliente lleva el ACK 4801, reconociendo así el segmento del servidor. El número de secuencia de este tercer segmento es 1201 a pesar de que el cliente no ha enviado aún ningún byte de datos. Esto ocurre tanto en la conexión como en la desconexión y es una excepción al modo en que se incrementan los números de secuencia. A partir de ese instante, la conexión queda establecida y ambos extremos conocen los números de secuencia que va a utilizar el otro, es decir, los números de secuencia están sincronizados *(SYNchronized)*.

La captura de la conexión real aparece en la Figura 13.2. El comando ejecuta `tshark` en segundo plano, espera 1 segundo y luego conecta al puerto 80 del servidor `example.net` con `ncat`. Si lo pruebas en tu computador, recuerda parar `tshark` al terminar, por ejemplo con `kill%1`[2].

```
~# tshark -f "tcp port 80" & sleep 1; ncat example.net 80
1 192.168.0.37 → 93.184.215.14 TCP 74 50192 → 80 [SYN] Seq=0 Win=32120 Len=0
                                                 MSS=1460 SACK_PERM WS=128
2 93.184.215.14 → 192.168.0.37 TCP 74 80 → 50192 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0
                                                 MSS=1452 SACK_PERM WS=512
3 192.168.0.37 → 93.184.215.14 TCP 66 50192 → 80 [ACK] Seq=1 Ack=1 Win=32128 Len=0
```

Listado 13.2: TCP: Captura de una conexión

La captura indica tanto para el cliente como para el servidor que los ISN son 0 (Seq=0), pero cuidado, porque esto no es real. Para poder hacer un seguimiento más sencillo de los números de secuencia, `tshark`[3] no muestra el número de secuencia que aparece en el segmento, si no un número relativo al ISN; por eso siempre es 0 en el primer segmento. Hay una opción para forzar a `tshark` a mostrar los números de secuencia reales. El mismo ejemplo

[1] por *synchronize*

[2] El comando `kill` envía una señal (por defecto SIGTERM) al proceso indicado, en este caso con la intención de terminarlo. En el capítulo 2 vimos el uso de este y otro muchos comandos y recuerda también que el anexo A es un pequeño catálogo de comandos habituales. Y por supuesto, siempre puedes usar el comando `man` para obtener información de cualquier comando.

[3] Y la mayoría de los *sniffers*

con números de secuencia reales aparece en la captura 13.3. Se han omitido algunos campos que no son relevantes para este ejemplo.

```
~# tshark -f "tcp port 80" -o tcp.relative_sequence_numbers:FALSE &
$ sleep 1; ncat example.net 80
1 192.168.0.37 → 93.184.215.14 TCP 74 48200 → 80 [SYN] Seq=7967281 Win=32120 Len=0
2 93.184.215.14 → 192.168.0.37 TCP 74 80 → 48200 [SYN, ACK] Seq=6696166 Ack=7967282 Len=0
3 192.168.0.37 → 93.184.215.14 TCP 66 48200 → 80 [ACK] Seq=7967282 Ack=6696167 Len=0
```

LISTADO 13.3: TCP: Captura de una conexión (con números de secuencia reales)

13.2. Tamaño máximo de segmento

La especificación de TCP dice que todo computador debe ser capaz de recibir segmentos de como mínimo 536 bytes en IPv4 y 1220 en IPv6 [41]. Este requisito está relacionado con la capacidad de almacenamiento en el *buffer de recepción*. Sin embargo, si el emisor tiene la certeza de que el receptor puede manejar segmentos mayores, es preferible que lo haga ya que eso aumenta el rendimiento del canal.

En general es más eficiente enviar pocos mensajes grandes que muchos pequeños. Enviando menos mensajes se reducen las interacciones entre capas a través de las interfaces, se reduce el peso relativo de las cabeceras en el total de datos transmitidos —la *sobrecarga de cabeceras*—, se minimiza el impacto de las retransmisiones y el trabajo que conlleva para routers y conmutadores.

Sin embargo, hay un límite superior relacionado con el tamaño máximo de datagrama IP, que a su vez está limitado por la MTU de la red. Si el tamaño del segmento elegido provoca que se supere el MTU, tendremos un paquete que no cabe en una única trama y se tendrá que fragmentar. La fragmentación debería evitarse cuando sea posible porque aumenta el impacto de los errores de transmisión. Si alguno de los fragmentos se pierde o corrompe, habrá que retransmitir el segmento completo, que de nuevo será fragmentado.

MSS determina el tamaño máximo que tendrá la carga útil de los segmentos durante la conexión. Si por ejemplo, cliente y servidor son vecinos de un mismo enlace Ethernet (que tiene una MTU de 1500 bytes) podrían utilizar un MSS de $1500 - 20 - 20 = 1460$ bytes. Esos 40 bytes que descontamos corresponden a las cabeceras estándar[4] de TCP e IP.

[4] estándar = sin opciones

Para utilizar un MSS distinto al valor por defecto, un receptor puede incluir la opción TCP MSS (tipo 2) en el primer segmento (el que lleva el flag SYN). Esta opción anuncia cuál es el tamaño máximo de segmento que puede manejar ese extremo. Hay que destacar que esto no es una negociación, no están acordando un valor común para el MSS; cada extremo decide por su cuenta y puede ser un valor diferente en cada uno de los los dos sentidos de la comunicación. La opción MSS proporciona un campo de 2 bytes para especificar el tamaño de segmentos, es decir, el valor máximo que se puede anunciar es 64 KiB, aunque el valor 65 535 es especial: significa «infinito» y está pensado para utilizar con los *jumbogramas* de IPv6.

En la captura 13.2, los dos mensajes iniciales llevan la opción MSS. El cliente indica 1460 bytes mientras que el servidor indica 1452 bytes.

Como los extremos de una conexión TCP pueden no conocer el MTU de la ruta, podrían elegir valores que obligan a realizar fragmentación. Un router o firewall intermedio podría evitar este problema aplicando una técnica conocida como *MSS clamping*, que consiste en *sobrescribir* el valor de MSS en las cabeceras de la conexión fijando un valor más bajo, adecuado para la ruta que seguirán los paquetes de esa conexión.

El programador también tiene la posibilidad de consultar y modificar el MSS de una conexión establecida. El Listado 13.4 muestra un cliente Python que conecta con un servidor escuchando en el mismo nodo (localhost). Consulta el valor de MSS antes de establecer la conexión y obtiene 536 bytes, que es el valor mínimo. Después, lo vuelve a consultar una vez establecida la conexión y obtiene 32 741 bytes. Este valor tan alto se debe precisamente a que ambos cliente y servidor se ejecutan el propio nodo y por eso el SO sabe de antemano que no habrá fragmentación.

```
$ python
>>> import socket
>>> sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
>>> sock.getsockopt(socket.IPPROTO_TCP, socket.TCP_MAXSEG)
536
>>> sock.connect(('127.0.0.1', 2000))
>>> sock.getsockopt(socket.IPPROTO_TCP, socket.TCP_MAXSEG)
32741
```

Listado 13.4: TCP: Consulta del MSS de una conexión

Si se fija el valor de MSS antes de establecer la conexión, el socket lo aplicará en la opción MSS de la cabecera al establecerse la conexión. Sin embargo, cambiarlo después normalmente no tiene ningún efecto.

13.3. Desconexión

La desconexión se parece a la conexión en que ambos extremos necesitan asegurarse que el otro ha terminado su tarea y está de acuerdo en dar por terminada la conexión (al menos en la modalidad más *civilizada*). La desconexión, sin embargo, puede implicar 3 o 4 mensajes.

A diferencia de la conexión, que solo la puede iniciar el cliente, la desconexión la puede comenzar cualquiera de los dos extremos en cualquier momento. La desconexión se basa en el envío del flag FIN[5] que indica que su emisor ha terminado de enviar sus datos, pero a pesar de eso todavía está dispuesto a aceptar nuevos datos. El receptor confirma la recepción de ese mensaje aumentando el número de secuencia en su siguiente confirmación de manera análoga a lo que ocurre con el flag SYN durante la conexión. Si el receptor también ha terminado de enviar datos, ese mismo segmento llevará también el flag FIN. El Listado 13.5 y la Figura 13.2 representan esta desconexión de 3 mensajes. El Listado 13.6 muestra una captura de una desconexión real.

```
1. --> seq:4000, FIN
2. <-- seq:5000, FIN, ack:4001
3. --> seq:4001, ack:5001
```

LISTADO 13.5: TCP: Secuencia de una desconexión en 3 segmentos

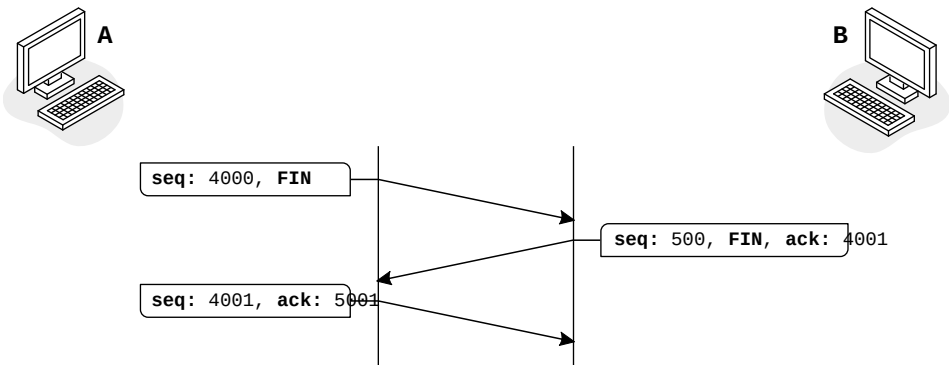


FIGURA 13.2: TCP: Patrón de mensajes de la desconexión en 3 segmentos

```
$ tshark -f "tcp port 80"
[...]
4 192.168.0.37 → 93.184.215.14 TCP 66 50192 → 80 [FIN, ACK] Seq=1 Ack=1 Win=32128 Len=0
```

[5]por *finish*

```
5 93.184.215.14 → 192.168.0.37 TCP 66 80 → 50192 [FIN, ACK] Seq=1 Ack=2 Win=65536 Len=0
6 192.168.0.37 → 93.184.215.14 TCP 66 50192 → 80 [ACK] Seq=2 Ack=2 Win=32128 Len=0
```

LISTADO 13.6: TCP: Captura de una desconexión

También puede ocurrir que el receptor del mensaje FIN tengo datos por enviar. En ese caso enviará una confirmación, y continuará enviando sus datos pendientes. Cuando termine emitirá por su cuenta un segmento con el flag FIN. En esta situación la desconexión requiere 4 segmentos $(2 + 2)$, en lugar de 3. El Listado 13.7 y la Figura 13.3 representa una situación en que el nodo que no inicia la desconexión todavía tenía 100 bytes por enviar. Fíjate que el mensaje que lleva datos y su correspondiente confirmación no forman parte de la desconexión; por eso no se contabilizan en el Listado 13.7 y aparecen en gris en la Figura 13.3.

```
1. --> seq:4000, FIN
2. <-- seq:5000, ack:4001
   <-- seq:5000, len:100
   --> seq:4001, ack:5100
3. <-- seq:5100, FIN
4. --> seq:4001, ack:5101
```

LISTADO 13.7: TCP: Secuencia de una desconexión en 4 segmentos

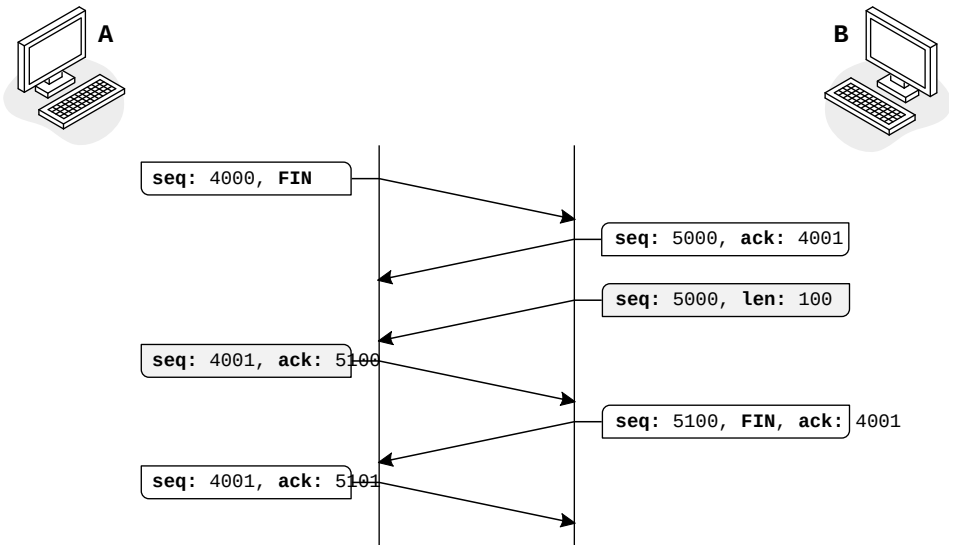


FIGURA 13.3: TCP: Patrón de mensajes de la desconexión en 4 segmentos

Una vez terminada la conexión, los dos extremos pueden liberar sus respectivos TCB, que incluyen los buffers de envío y recepción, y otros recursos asociados.

13.3.1. Tiempo de silencio

Cuando una conexión termina y vuelve a establecerse inmediatamente después, existe una pequeña posibilidad de que los números de secuencia empleados en la nueva conexión se repitan o solapen con los de la conexión anterior. Si esto ocurriera, el receptor podría confundir los segmentos de la nueva conexión con los de la anterior, con resultados impredecibles.

Para evitar este problema, cuando se termina un servidor, el SO impone un *tiempo de silencio (quiet time)*. Durante ese tiempo el socket se mantiene en un estado de *limbo* (llamado `TIME_WAIT`) que impide vincular un nuevo socket (`bind`) al mismo puerto. Si se intenta levantar de nuevo un servidor en el mismo puerto se obtendrá el mismo error que si realmente estuviera ocupado: *Dirección en uso (Address already in use)*.

El tiempo de silencio es igual a MSL (Maximum Segment Lifetime), que como su nombre indica, es el tiempo máximo de vida de un segmento, es decir, el tiempo máximo que un segmento puede estar moviéndose por la red antes de llegar a su destino o ser descartado. Se asume que este tiempo es aproximadamente 2 minutos, aunque depende del implementador.

Aunque es una restricción de seguridad importante para un servidor en producción, el programador puede desactivar esta salvaguarda para un socket concreto. Esto es útil por ejemplo para un servidor en su fase de desarrollo o pruebas, y para el que sabe que una confusión entre segmentos no tendrá consecuencias. El Listado 13.8 muestra cómo desactivar el tiempo de silencio.

```
sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
```

Listado 13.8: TCP: Desactivación del tiempo de silencio

Puedes profundizar más en esta cuestión consultando la sección «The TCP Quiet Time Concept» en la RFC 793 [42].

13.4. Reset

La utilidad principal del flag RST es rechazar una conexión entrante. Ante un intento de conexión (segmento con el flag SYN), el SO del nodo receptor responderá con un segmento con el flag RST si la conexión no puede ser establecida. Esto puede ocurrir por varios motivos:

- El puerto destino está cerrado, es decir, no está vinculado a ningún proceso servidor (Listado 13.9).
- Hay un servidor, pero no tiene posibilidad de aceptar nuevas conexiones en ese momento, quizá por una limitación de recursos.
- El segmento de inicio de conexión no es válido.

```
$ tshark -f "tcp port 2000"
1 192.168.1.235 → 192.168.1.1 TCP 74 48318 → 2000 [SYN] Seq=0 Win=32120 Len=0 MSS=1460
2 192.168.1.1 → 192.168.1.235 TCP 60 2000 → 48318 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
```

LISTADO 13.9: TCP: Rechazo de un intento de conexión a un puerto cerrado

El flag RST también se utiliza para cerrar una conexión activa de forma inmediata sin esperar a que el otro extremo esté de acuerdo, de hecho ni siquiera espera una confirmación. Esto se puede hacer en situaciones anómalas, cuando el otro extremo está infringiendo las reglas del protocolo, como por ejemplo usando números de secuencia incorrectos, flags ausentes o inesperados, etc. Todas estas situaciones las maneja automáticamente el SO y no es necesario que el programador intervenga.

Pero el programador también puede usar RST para finalizar la conexión de manera abrupta aunque esté funcionando correctamente. Obviamente puede conllevar la pérdida de datos si el otro extremo no ha terminado de enviar, pero es un riesgo que el emisor asume. El Listado 13.10 muestra cómo desactivar la «permanencia» *(linger)* de la conexión hasta recibir todos los datos. De este modo, el método `close()` cierra la conexión con un segmento RST en lugar del método convencional (§ 13.3).

```
import socket, struct

sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
sock.setsockopt(socket.SOL_SOCKET, socket.SO_LINGER, struct.pack('ii', 1, 0))
sock.connect((host, port))
[... envío de datos ...]
sock.close()  # envía RST, no habrá handshake de desconexión
```

LISTADO 13.10: TCP: Cerrar una conexión de forma abrupta con RST

El SO del otro extremo (el que recibe el RST) enviará al proceso un error de tipo `errno.ECONNRESET` para indicarle esta situación.

13.5. Control de flujo en TCP

El control de flujo en TCP se basa en el mecanismo de *ventana deslizante*. El receptor se preocupa de informar en cada mensaje del espacio disponible (en bytes) en su buffer de recepción —a esto se le llama «actualizar la ventana». Esa cantidad de espacio libre es lo que llamamos *ventana de recepción* y es el valor que aparece en el campo `window` de la cabecera TCP. Determina también los números de secuencia de los bytes que puede recibir. Por ejemplo, si el campo `window` tiene un valor 1000 y el campo `acknowledge number` tiene un valor de 3000, significa que el receptor puede aceptar los bytes desde el 3000 hasta el 3999, y no otros. Al recibir este mensaje, el emisor sabe que como mucho podrá enviar segmentos por un total de esos 1000 bytes a partir del último byte confirmado. De este modo, el receptor TCP influye continuamente en la tasa de salida del emisor. A este mecanismo concreto es a lo que llamamos *control de flujo de receptor*, aunque en este capítulo lo llamaremos simplemente *control de flujo*.

La Figura 13.4 representa el flujo de datos completo desde que salen del proceso emisor hasta que son consumidos por el proceso receptor. Recuerda que estamos representando solo un sentido de la comunicación de una única conexión. Cada conexión establecida conlleva una estructura como esta en cada sentido de la comunicación.

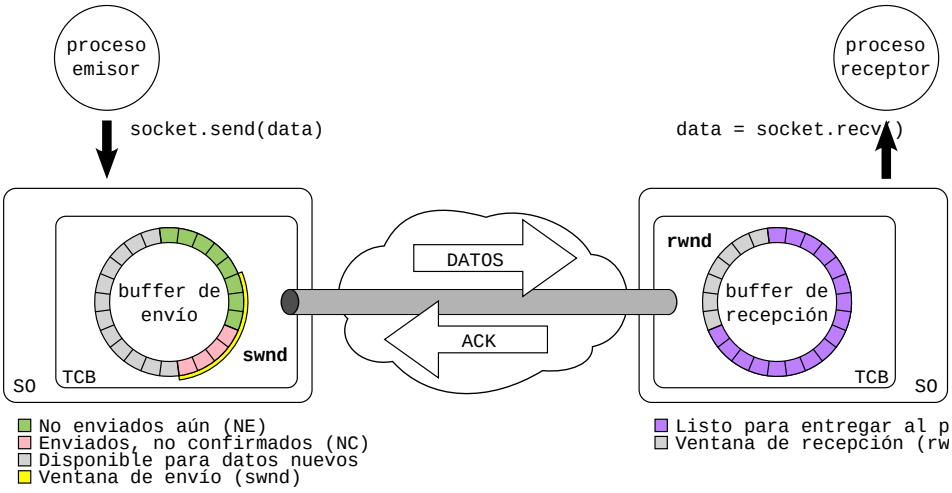


FIGURA 13.4: TCP: Flujo de datos
(se representa un solo sentido de la comunicación)

El emisor mantiene el buffer de envío *(sending buffer)* implementado como una cola circular. Los datos entran en este buffer vía el método `soc-`

`ket.send()` y salen cuando el SO decide construir un nuevo segmento y enviarlo a la red, pero no hay una relación directa entre la invocación de `send()` y el envío de segmentos, y de hecho varios `send()` podrían corresponder a uno, varios o ningún segmento en ese instante.

El buffer de envío está formado por:

- Espacio disponible para almacenar datos nuevos procedentes del proceso emisor.
- Bytes enviados que aún no han sido confirmados. Nos referiremos a esta parte del buffer como NC (no confirmados).
- Bytes que aún no han sido enviados. Nos referiremos a esta parte del buffer como NE (no enviados).

El buffer de recepción solo tiene datos en dos estados: recibidos confirmados y espacio libre. Cuando el computador recibe mensajes a través de una interfaz de red, el SO procesa el segmento, identifica el proceso al que corresponde el puerto destino que aparece en la cabecera de transporte y finalmente inserta su carga útil en este buffer. Los datos salen del buffer cuando el proceso receptor los solicita mediante el método `socket.recv()`.

Eso significa que si un proceso receptor, que mantiene una conexión abierta, deja de recoger datos —con `recv()`— mientras el emisor sigue enviando, ambos buffers (envío y recepción) se llenarán y como consecuencia el emisor quedará bloqueado en una invocación a `send()`.

Es importante aclarar que el SO envía el mensaje de confirmación cuando los datos son almacenados en el buffer de recepción, es decir, antes de que hayan sido procesados por el proceso receptor. Si el proceso receptor no llega a pedirlos y termina, el buffer de recepción se destruye y los datos se pierden. Dicho de otro modo, el mecanismo garantiza que los datos llegan al SO destino, no al proceso receptor.

El tamaño de estos buffers depende de cada SO y de su configuración específica. Puedes consultar estos valores para un socket concreto por medio de los identificadores `socket.SO_SNDBUF` y `socket.SO_RCVBUF`. El Listado 13.11 muestra una *shell* Python con el código que permite conseguirlo. Los valores mostrados: 16 384 y 131 072 bytes, son los valores reales (por defecto) que se han obtenido en un sistema Debian con Linux 6.12.9.

El programador puede modificar el tamaño de estos buffers, aunque si lo pruebas (como en el listado) verás algo curioso. El SO reserva el doble de lo que pides. Esto se debe a que debe contar con el espacio necesario para almacenar cabeceras y metadatos, que estima que puede llegar a ser el

doble de la carga útil. Estos tamaños tienen un valor mínimo y máximo que depende del SO. En Linux el mínimo ronda los 2-4 kB y un máximo 400 kB, y estas primitivas truncarán valores fuera de ese rango.

También es posible consultar el tamaño de los buffers, y muchos otros datos, para un socket activo con el comando `ss` como veremos un poco más adelante.

```
$ python
>>> import socket
>>> sock = socket.socket()
>>> sock.getsockopt(socket.SOL_SOCKET, socket.SO_SNDBUF)
16384
>>> sock.getsockopt(socket.SOL_SOCKET, socket.SO_RCVBUF)
131072
>>> sock.setsockopt(socket.SOL_SOCKET, socket.SO_SNDBUF, 4096)
>>> sock.setsockopt(socket.SOL_SOCKET, socket.SO_RCVBUF, 4096)
>>> sock.getsockopt(socket.SOL_SOCKET, socket.SO_SNDBUF)
8192
>>> sock.getsockopt(socket.SOL_SOCKET, socket.SO_RCVBUF)
8192
```

Listado 13.11: TCP: Consultar y modificar el tamaño de los buffers

13.6. Ventanas de envío y recepción

La *ventana de recepción* (abreviado como *rwnd*) es el espacio libre (no ocupado) en el buffer de recepción. De la otra parte, la *ventana de envío* (abreviado como *swnd*) define la **parte concreta** del buffer de envío (los bytes específicos) que el emisor tiene permitido enviar en ese momento. El tamaño de la ventana de envío está limitado en todo momento por la ventana de recepción.

$$\text{swnd} \leq \text{rwnd} \tag{13.1}$$

Como la velocidad a la que el proceso receptor consume datos del buffer de recepción puede variar drásticamente a lo largo del tiempo, la ventana de recepción puede cambiar del mismo modo, y a su vez estos cambios, notificados al emisor, condicionan la ventana de envío. Dicho de otro modo, la tasa a la que el proceso receptor consume datos condiciona indirectamente la tasa a la que el proceso emisor puede enviar. La ventana de envío por tanto puede crecer, decrecer o incluso cerrarse en cualquier momento.

El campo `window` de la cabecera TCP tiene una longitud de 16 bits, por lo que el valor máximo que un receptor puede anunciar en su ventana de recepción es $2^{16} - 1 = 65535$ bytes. Como la ventana de envío está siempre acotada por la ventana de recepción anunciada (Fórmula 13.1), este límite también condiciona el tamaño máximo de la ventana de envío. En redes con un BDP alto (§ 12.5), ese límite resulta insuficiente: se necesita una ventana mayor para conseguir un aprovechamiento óptimo del canal, es decir, no quedar bloqueado en espera de confirmaciones.

Para superar esta limitación, TCP dispone de la opción *escala de ventana (Window Scale)* [43] que, al igual que MSS (§ 13.2), solo puede aparecer en el segmento SYN. Cada extremo la anuncia de forma independiente, no negocian un valor común. La opción indica un factor de desplazamiento *(shift count)* (`s`) entre 0 y 14, que ambos extremos aplican al valor bruto del campo `window` durante el resto de la conexión: el tamaño real de la ventana se calcula como window $\times\, 2^S$ = window $\ll S$, por eso se le llama desplazamiento. El factor máximo (14) permite ventanas de 2^{30} bytes ≈ 1 GiB. Si alguno de los dos extremos no incluye la opción en su segmento SYN, esa conexión se limita a los 65 535 bytes originales en ambos sentidos.

Control de flujo en una conexión real

Para ver el control de flujo en acción vamos a usar el ejemplo /`flow-control`. Consta de un servidor que únicamente recibe, pero que nos da la opción de limitar la tasa a la que consume. El siguiente comando arranca el servidor en el puerto 2000 y limita la tasa a 200 kB/s.

```
code/flow-control$ ./server.py --limit 200 2000
```

Y un cliente que se conecta al servidor indicado y sólo envía. Lo hace tan rápido como pueda. Para ejecutarlo, abre otra terminal y hazlo así:

```
code/flow-control$ ./client.py 127.0.0.1 2000
```

El cliente informa del tamaño del buffer de envío, la cantidad de datos que ha enviado (pero que pueden estar aún en el buffer) y la tasa media de envío. El servidor informa del tamaño del buffer de recepción, la cantidad de datos recibidos y la tasa media de recepción. Esto se aprecia en la Figura 13.5. Más adelante en el capítulo 17 veremos cómo miden y limitan la tasa, de momento solo los vamos a usar para generar tráfico.

Si los ejecutas ambos en tu computador, tanto el emisor como «la red» pueden enviar más rápido de esa tasa de 200 kB/s. Por eso, tanto el buffer de recepción del servidor como el de envío del cliente eventualmente se llenan, lo que produce un bloqueo intermitente del emisor durante unos

Cliente

```
~$ ./client.py 127.0.0.1 2000
Sending buffer size: 2,626 kB
 (-) sent:10,630 kB, rate:268.2 kB/s
```

Servidor

```
~$ ./server.py 2000 200
Receiving buffer size: 131 kB
 received:9,067 kB, rate:200.1 kB/s
```

Figura 13.5: Flujo continuo con tasa limitada por el receptor (servidor) — /flow-control

segundos. Cada bloqueo (cuando se detiene la cuenta de bytes enviados) corresponde al momento en que el receptor cierra la ventana. Esto es una prueba clara de cómo el receptor condiciona absolutamente la capacidad del emisor para enviar datos, a punto de detenerlo por completo.

Una vez tienes en marcha los dos programas en sendas consolas, puedes usar el comando `ss` que consulta el TCB del socket para obtener muchos datos interesantes. El comando concreto que se muestra filtra la conexión establecida que incluye el socket cliente y el socket conectado en el lado del cliente, es decir, los flujos con destino y origen en el puerto 2000. Si no se aplica un filtro, listaría todos los sockets activos.

```
$ ss -tin 'sport = :2000 or dport = :2000'
State         Recv-Q   Send-Q     Local Address:Port        Peer Address:Port
ESTAB         0        2260096    127.0.0.1:33188           127.0.0.1:2000
    wscale:7,7 rto:220 backoff:1 rtt:17.291/0.018 mss:54976 bytes_sent:58781120
    bytes_retrans:164928 bytes_acked:58616193 segs_out:1608 segs_in:1602
    data_segs_out:1073 send 254356602bps lastsnd:512 lastrcv:293028 lastack:272
    pacing_rate 508687456bps delivery_rate 50765624bps delivered:1074 busy:293024ms
    rwnd_limited:293016ms(100.0%) retrans:0/3 dsack_dups:3 rcv_space:65495
    rcv_ssthresh:65495 notsent:2260096 minrtt:0.01 rcv_wnd:65536
ESTAB         7552      0         127.0.0.1:2000            127.0.0.1:33188
    wscale:7,7 rto:200 rtt:0.017/0.008 ato:40 mss:32768 bytes_received:58616192
    segs_out:1601 segs_in:1608 data_segs_in:1073 send 154202352941bps lastsnd:293028
    lastrcv:512 lastack:512 pacing_rate 308404705880bps delivered:1 app_limited rcv_rtt:1
    rcv_space:65483 rcv_ssthresh:109848 minrtt:0.017 snd_wnd:65536
```

Listado 13.12: TCP: Información de la conexión con `ss`

En la primera línea aparece la cabecera con unos campos que se aplican a todos los sockets listados. En esas filas se observa lo siguiente:

```
State         Recv-Q   Send-Q     Local Address:Port        Peer Address:Port
ESTAB         0        2260096    127.0.0.1:33188           127.0.0.1:2000
ESTAB         7552     0          127.0.0.1:2000            127.0.0.1:33188
```

Estas columnas indican:

- State: es el estado de la conexión. En ambos es `ESTAB`, es decir, la conexión está establecida.

- Recv-Q: es la ocupación (en bytes) de la cola de recepción. La primera fila corresponde con el cliente, que no recibe nada, y la segunda con el servidor.
- Send-Q es la ocupación de cola de envío.
- Local Address:Port es la dirección IP y el puerto local del socket.
- Peer Address:Port es la dirección IP y el puerto del socket remoto.

La lista de variables que hay debajo de esas filas corresponde a cada socket. Incluimos solo las que resultan relevantes en este momento. La Tabla 13.1 muestra los datos del cliente mientras que la Tabla 13.2 muestra los del servidor.

Variable	**Valor**	**Descripción**
mss	54 976	Tamaño máximo de segmento (MSS)
bytes_sent	58 781 120	Bytes enviados
bytes_retrans	164 928	Bytes retransmitidos
bytes_acked	58 616 193	Bytes confirmados (ACK)
segs_out	1608	Segmentos totales
segs_in	1602	Segmentos totales
data_segs_out	1073	Segmentos de datos
send	254,36	Tasa de envío (Mbps)
delivery_rate	50,77	Tasa de entrega (Mbps)
delivered	1074	Segmentos entregados
rwnd_limited	293 016	La tasa está limitada por rwnd (ms) – 100 %
rcv_space	65 495	Espacio en el búfer de recepción
rcv_wnd	65 536	Tamaño de la ventana de recepción
wscale	7,7	Escala de ventana del otro extremo y propia ($2^7 = 128$)

CUADRO 13.1: Métricas del estado del socket cliente relacionadas con buffers y ventanas — /flow-control

Variable	**Valor**	**Descripción**
mss	32 768	Tamaño máximo de segmento (MSS)
bytes_received	58 616 192	Total de bytes recibidos
segs_out	1601	Segmentos totales enviados
segs_in	1608	Segmentos totales recibidos
data_segs_in	1073	Segmentos de datos recibidos
send	154.20	Tasa de envío (Gbps)
delivered	1	Segmentos entregados
app_limited	-	Indica limitación por la aplicación
snd_wnd	65 536	Tamaño de la ventana de envío
wscale	7,7	Shift count del otro extremo y propio ($2^7 = 128$)

CUADRO 13.2: Métricas de estado del socket servidor relacionadas con buffers y ventanas — /flow-control

En próximas secciones y capítulos veremos otros datos interesantes que aparecen en la información que ofrece `ss`.

13.7. RTO: el temporizador de retransmisión

En un protocolo confiable de la capa de enlace, la duración del temporizador de retransmisión es fácil de calcular. Como el envío de datos se realiza directamente a un vecino, resulta sencillo determinar el tiempo necesario para que un mensaje de datos llegue al destino y, de vuelta, la confirmación correspondiente. Considerando el tamaño del mensaje y el ancho de banda del enlace se puede calcular este tiempo con precisión. Se debe añadir obviamente el tiempo de procesamiento de los mensajes en los extremos, su almacenamiento, etc., pero en general todo esto es bastante predecible, determinista y constante.

Con TCP la situación es muy distinta [44]. El envío de un segmento a un destino arbitrario depende de muchas variables ya que el segmento debe atravesar probablemente muchas redes y dispositivos intermedios:

- La distancia física entre emisor y receptor.
- La topología de la red y la ruta que van a tomar los datagramas.
- La carga de la red.
- El ancho de banda de los enlaces.
- La posible congestión en la red.
- La carga de trabajo de los routers intermedios, y de los nodos finales.

Y la situación es más compleja aún. Todas estas variables toman valores diferentes en función del destino y ruta, y pueden cambiar rápida y significativamente a lo largo de una misma conexión por cambios en las condiciones en distintos lugares de la interred.

Para resolverlo, TCP mide constantemente el RTT, es decir, el tiempo que transcurre desde que se envía un segmento hasta que llega su correspondiente confirmación. A partir de esta medida, calcula el valor del temporizador de retransmisión (RTO) que va a aplicar. Así pues, es posible que se el período de retransmisión sea distinto incluso para segmentos consecutivos. Para la obtención del valor final de RTO, TCP calcula una versión 'suavizada' (SRTT) del RTT con la Fórmula 13.2. El suavizado proporciona estabilidad al cálculo del RTO, evitando que variaciones puntuales en la medida provoquen cambios bruscos.

$$\mathrm{SRTT} = (1 - \alpha) \cdot \mathrm{SRTT} + \alpha \cdot \mathrm{RTT} \tag{13.2}$$

El coeficiente α es el *factor de suavizado*. Un valor entre 0 y 1 que determina si se le da más importancia al valor RTT que se acaba de medir (α alto) o si por el contrario es más importante el valor SRTT previo (α bajo). El valor de α suele ser 1/8, es decir, conservador. Para el cálculo inicial $SRTT = RTT$.

TCP también calcula RTTVAR, que representa la desviación media del RTT respecto a SRTT. Permite estimar la variabilidad de RTT según la Fórmula 13.3 (algoritmo de Jacobson [45]).

$$\text{RTTVAR} = (1 - \beta) \cdot \text{RTTVAR} + \beta \cdot |\text{SRTT} - \text{RTT}| \tag{13.3}$$

La fórmula es similar a la de SRTT, β también es un factor de suavizado cuyo valor habitual es 1/4. Para el cálculo inicial se toma $RTTVAR = RTT/2$. Esta variable se actualiza antes que SRTT al llegar un ACK adecuado.

A partir de SRTT y RTTVAR se calcula el RTO según la Fórmula 13.4.

$$\text{RTO} = \text{SRTT} + max(G, K \cdot \text{RTTVAR}) \tag{13.4}$$

donde:

G es la precisión del reloj (*p. ej.* 1ms).
K es un factor de ponderación, normalmente 4.

Lo que dice esta formula es que el RTO es el SRTT más un margen de seguridad que será mayor cuanto más variable sea la latencia de la red (mayor sea RTTVAR). Adicionalmente se aplica un valor mínimo de 1 segundo y un valor máximo de 60 segundos.

Una retransmisión es un mensaje idéntico al original: lleva el mismo número de secuencia y la misma carga útil. Por eso el receptor no puede distinguir un segmento original de una retransmisión posterior del mismo segmento, a menos claro, que acabe recibiendo dos o más copias.

TCP utiliza el algoritmo de Karn [46], que estipula que no se deben realizar medidas de RTT con segmentos retransmitidos, ya que la ambigüedad entre posibles ACK correspondientes a segmentos duplicados podría falsear el cálculo. En todo caso, no se recalcula el RTO para cada segmento, se realiza solo una vez por cada ronda. La única excepción a esta norma es la opción TCP Timestamp.

Cada emisor TCP mantiene un único RTO, que está asociado al primer segmento enviado no confirmado (SND.UNA). Eso implica que si todos los datos están confirmados, el RTO se desactiva. Cuando se envía un nuevo segmento, se reactiva el temporizador con el último valor RTO calculado. Cuando llega un ACK, se reinicia el temporizador con el valor del RTO actualizado.

Cuando el RTO expira, el emisor retransmite el segmento no confirmado más antiguo y duplica el valor del RTO. Si vuelve a expirar se duplicará de nuevo hasta un máximo según la Fórmula 13.5. Esto se denomina *backoff exponencial* y pretende adaptar las retransmisiones a las condiciones de la red.

SND.UNA

Es el acrónimo de Send Unacknowledged Number, y es el número de secuencia relacionado con el segmento más antiguo enviado y no confirmado. Más concretamente, indica el limite inferior de la ventana de envío.

$$\text{RTO} = min(2 \cdot \text{RTO}, \text{RTO}_{max}) \tag{13.5}$$

donde:

RTO_{max} es el valor máximo que puede tomar el RTO, normalmente 60s.

Cuando el emisor consiga realizar una nueva medida de RTT, es decir, cuando pueda enviar un nuevo segmento (no una retransmisión) y éste sea confirmado, el valor de RTO volverá a sus cotas habituales.

Ejemplo de cálculo de RTO

Un pequeño ejemplo ilustra cómo evoluciona el valor del RTO. Supongamos un valor calculado de $SRTT = 100ms$, $RTTVAR = 25ms$ que resulta en $\text{RTO} = \text{SRTT} + 4 \cdot \text{RTTVAR} = 200ms$. Cuando correspond, se recalculan SRTT, RTTVAR y RTO según las Fórmulas 13.2, 13.3 y 13.4.

- Se envían 3 segmentos S1 en $t = 0ms$, S2 en $t = 1ms$ y S3 en $t = 2ms$, asumiendo que la ventana así lo permite. El RTO queda asociado a S1.
- En $t = 90$ llega el ACK de S1. El RTT medido es 90ms. Se recalcula RTO para S2 en 183,75ms, expira en el instante $t = 273,75$.
- En $t = 120$ lleva un ACK duplicado para S1. No tiene ningún efecto.
- En $t = 273,75$ expira el temporizador de S2. Se aplica *backoff* y RTO se duplica hasta 367,5ms. Expirará en el instante $t = 641,25$.
- En $t = 330$ llega un ACK para S3. No se recalcula el RTO porque según Karn, el RTT medido debe descartarse. Como los tres segmentos enviados están confirmados, el RTO se desactiva.
- En $t = 400$ se envía S4. El temporizador se reinicia con el valor del RTO actualizado (367,5ms). Expirará en $t = 767,5$.

- En $t = 500$ llega el ACK de S4. Se recalcula el valor RTO con una nueva muestra válida, resultando en 163,91ms, y se desactiva porque no hay confirmaciones pendientes.

La Tabla 13.3 resume el ejemplo y la evolución de las variables implicadas. Todos los valores están expresados en ms.

t	evento	RTT	RTTVAR	SRTT	RTO	expira en
0,00	envío S1	–	25,00	100,00	200,00	200,00
1,00	envío S2	–	25,00	100,00	–	–
2,00	envío S3	–	25,00	100,00	–	–
90,00	ACK S1	90	21,25	98,75	183,75	273,75
120,00	ACK (dup) S1	–	21,25	98,75	183,75	273,75
273,75	timeout S2	–	21,25	98,75	367,50	641,25
330,00	ACK (acum) S3	–	21,25	98,75	367,50	–
400,00	envío S4	–	21,25	98,75	367,50	767,50
500,00	ACK S4	100	16,25	98,91	163,91	–

CUADRO 13.3: TCP: Evolución de las variables en el ejemplo de RTO

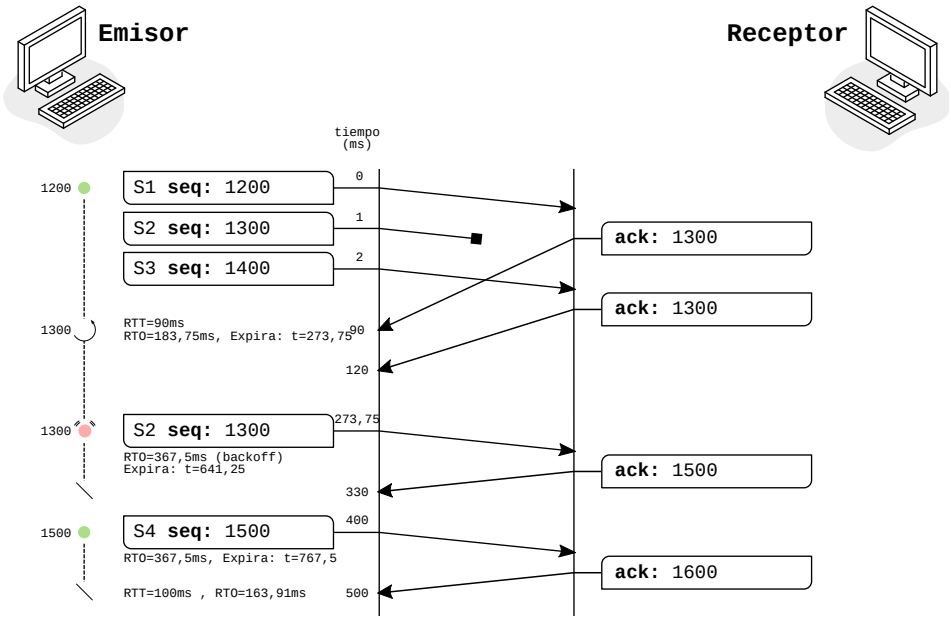


FIGURA 13.6: TCP: Ejemplo de cálculo del RTO

RTO en una conexión real

Con `ss` se puede consultar información importante sobre el cálculo de RTO. La Tabla 13.4 recoge las variables del cliente relacionadas, retomando el ejemplo de la sección 13.6.

Variable	Valor	Descripción
rto	220	Valor actual del RTO (ms)
backoff	1	Multiplicador de tiempo de espera (por timeout)
rtt	17.291 / 0.018	RTT medido (promedio / desviación) ms
minrtt	0.01	RTT mínimo observado

CUADRO 13.4: Métricas del estado del socket cliente relacionadas con el RTO— /flow-control

13.8. El síndrome de la *ventana tonta*

El *síndrome de la ventana tonta* (o SWS) es un efecto que conlleva una importante ineficiencia en la transferencia de datos propiciada por el envío de segmentos cada vez más pequeños. Ambos, receptor y emisor, son responsables de este efecto: el receptor porque anuncia tamaños de ventana pequeños tan pronto como dispone de algo de espacio en el buffer de recepción, y el emisor porque envía segmentos aunque tenga pocos datos disponibles. Por eso, resolver el problema requiere la colaboración de ambos extremos.

El receptor puede ayudar a paliar el problema esperando a disponer de una cantidad de espacio libre significativa antes de anunciarlo (normalmente entre MSS y la mitad del buffer de recepción) y reservando espacio libre adicional que no se anuncia. Además el receptor puede hacer uso del ACK retardado (con un máximo de medio segundo) y así favorecer la liberación de espacio adicional.

Por su parte, el emisor puede esperar a disponer de una cantidad significativa de datos antes de enviar un nuevo segmento. El algoritmo de Nagle [47] se ocupa de esto. Lo que dice Nagle básicamente es:

«Si el buffer de envío ya contiene datos sin confirmar, los nuevos datos que lleguen desde el proceso emisor se añaden al buffer hasta que se confirmen los datos pendientes o bien se acumulen MSS bytes nuevos».

13.9. Cierre de la ventana

El cierre de la ventana de recepción ocurre cuando el receptor anuncia un valor de 0 bytes en el campo `window` de la cabecera TCP, y provoca una situación peculiar. Esta acción suspende el tráfico (en ese sentido de los datos) a pesar de que el emisor tenga aún datos que enviar. Termina cuando el receptor, en algún momento, envía una confirmación indicando un valor distinto de 0. Pero hay un problema: ¿qué ocurre si se pierde esa confirmación? El emisor quedará esperando indefinidamente el mensaje de apertura, que no llegará; y el receptor queda esperando que el emisor envié datos nuevos, que tampoco van a llegar, es decir, tenemos un bloqueo.

Para evitarlo, TCP dispone de un mecanismo conocido como *prueba de ventana cero*. Consiste en el uso del llamado *temporizador de persistencia*, que al expirar, le dice al emisor que envíe una *prueba de ventana cero* (Zero Window Probe, ZWP), que puede ser un segmento sin datos o una retransmisión sobre datos ya confirmados. Eso fuerza al receptor a enviar una confirmación que permite al emisor saber si la ventana sigue cerrada. Emisor y receptor deberían permitir que la ventana quede cerrada indefinidamente. La figura 13.7 muestra un ejemplo de este mecanismo.

El temporizador de persistencia comienza siendo igual a RTO, pero como el RTO, se duplica cada vez que expira, hasta un valor máximo de 60 segundos.

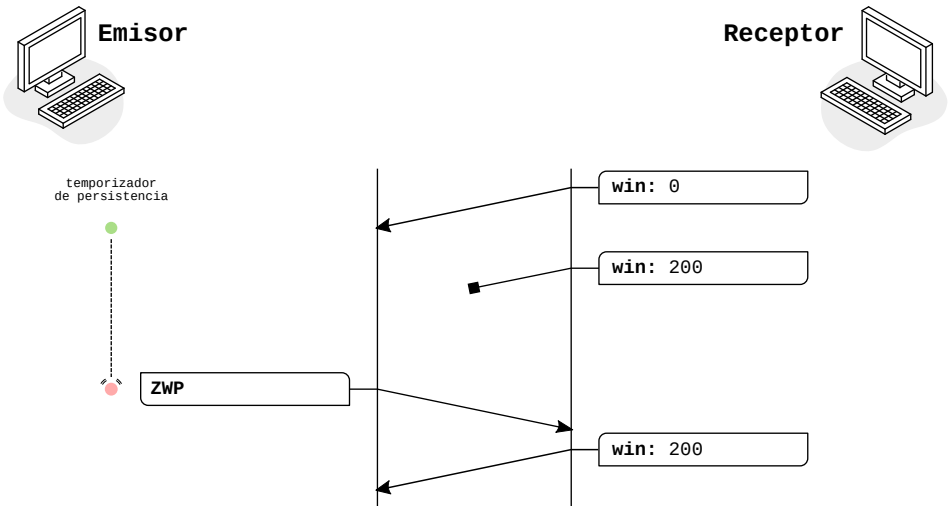


Figura 13.7: TCP: «Prueba de ventana cero» tras expirar el temporizador de persistencia

13.10. Confirmación selectiva

Con la confirmación convencional, cuando se pierde un segmento, pero los siguientes llegan correctamente, el emisor recibe repetidamente confirmaciones para el último segmento que llegó correcto y en orden (confirmaciones duplicadas), y conforme vaya expirando el temporizador para los segmentos afectados, los irá retransmitiendo, a pesar de que hubieran llegado correctamente (el emisor no puede saberlo), y seguirá haciéndolo hasta recibir una confirmación sobre datos nuevos. Eso por supuesto conlleva retransmisiones innecesarias y un desperdicio significativo de tiempo y recursos.

Para mejorar esta situación, el receptor puede enviar *confirmaciones selectivas*[6] o SACK. En ese caso, el receptor puede informar de los segmentos que han llegado correctamente incluso fuera de orden. Entonces el emisor puede enviar inmediatamente los segmentos que faltan sin tener que esperar a que expire el temporizador.

Para proporcionar esta funcionalidad, TCP dispone de dos opciones. La primera se llama SACK-permitted (tipo 4) y se incluye en el segmento SYN para indicar que quién la emite es capaz de procesar confirmaciones selectivas. La segunda opción es la confirmación SACK en sí misma (tipo 5), que lógicamente solo se debería enviar a un emisor que haya indicado que puede manejarlas.

La opción SACK contiene una lista de bloques que han llegado correctamente. Cada bloque (que puede incluir varios segmentos) se especifica con los números de secuencia del primer byte del bloque y del siguiente al último. Como cada número de secuencia requiere 4 bytes, la opción SACK puede contener un máximo de 4 bloques. Cuando el emisor retransmite los segmentos que faltan, el receptor envía una confirmación convencional actualizada.

La Figura 13.8 muestra un ejemplo de confirmación SACK en la que el receptor informa que ha recibido correctamente los datos fuera de orden (con números de secuencia 1601 a 2000). El emisor procede a retransmitir inmediatamente el segmento que falta (con número de secuencia 1401) sin esperar a que expire el temporizador.

13.11. Aplicaciones interactivas

TCP transmite un flujo continuo de bytes entre emisor y receptor. El emisor (gracias al algoritmo de Nagle entre otros) trata de realizar una gestión

[6] No confundir con *repetición selectiva*

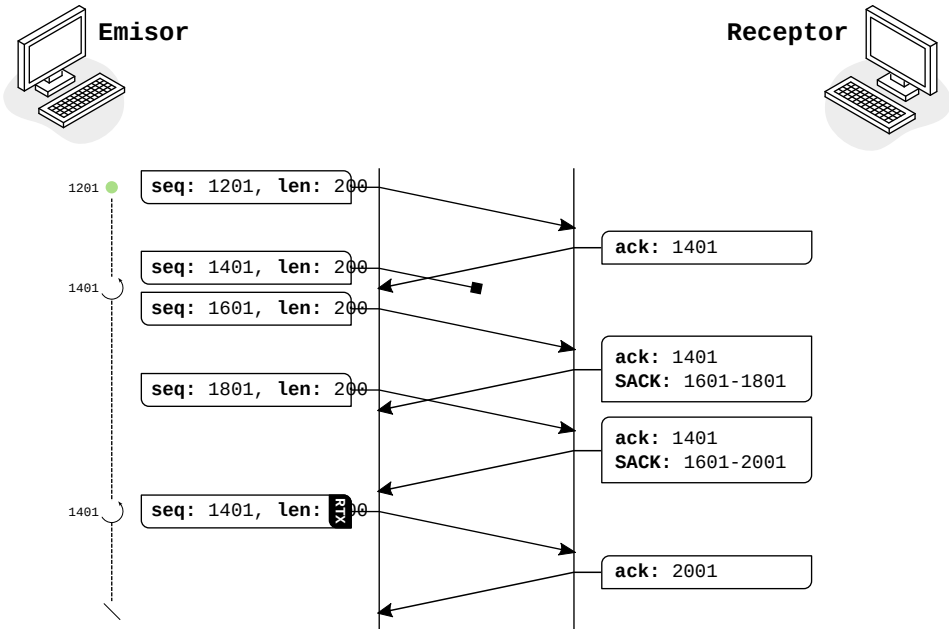


FIGURA 13.8: TCP: Confirmación selectiva

óptima que maximice el uso del canal respetando las limitaciones del receptor y evitando efectos negativos como el envío de segmentos demasiado pequeños. El SO decide cuando enviar cada segmento y qué cantidad de bytes colocar en cada uno.

Sin embargo, esta búsqueda de la eficiencia es muy perjudicial para las aplicaciones interactivas tales como shell remota, escritorio remoto o videojuegos. Si por ejemplo movemos el ratón o pulsamos una tecla y esperamos ver el resultado de forma inmediata, el segmento que lleva esos datos al servidor debe salir inmediatamente, no puede esperar a que se acumulen MSS bytes. Este tipo de aplicaciones serían inviables.

Para lograrlo, hay varias cosas que podemos hacer. Una de ellas es desactivar el algoritmo de Nagle para la conexión particular que lo requiera. Así se desactiva en Python (Listado 13.13):

```
sock.setsockopt(socket.IPPROTO_TCP, socket.TCP_NODELAY, 1)
```

LISTADO 13.13: TCP: Desactivación del algoritmo de Nagle

Además de desactivar Nagle, TCP proporciona en su cabecera 2 flags que modifican el comportamiento habitual y que pueden ayudar a llevar los datos hasta el receptor lo antes posible.

Cuando el emisor incluye el flag PSH *(push)* en el segmento, el receptor lo interpreta como una señal de que debe entregar los datos a la capa superior inmediatamente, sin esperar a nuevos mensajes, pero no tiene ningún efecto en el emisor, es decir, no le obliga a enviar el segmento antes de los habitual. El flag PSH puede ser añadido automáticamente por el SO en ciertas circunstancias, por ejemplo, si los datos llevan determinado tiempo en el buffer de envío.

El flag URG *(urgent)* indica que la primera parte de la carga útil del segmento deben tratarse de forma prioritaria. El flag habilita el campo `puntero urgente` que corresponde con el offset (sumado al número de secuencia) que indica dónde acaban los datos urgentes (es el puntero al último byte urgente). En todo caso, es un mecanismo que históricamente ha tenido interpretaciones diferentes en las distintas implementaciones, ha tenido poco uso y no se recomienda.

13.12. Control de errores

Esta sección explica cómo TCP resuelve los errores que pueden surgir durante una conexión. En adelante, «segmento» se refiere a un segmento de datos y «confirmación» a un segmento ACK efectivo, es decir, que confirma la llegada de datos nuevos.

13.12.1. Segmento perdido/corrupto

A efectos prácticos no hay diferencias entre un segmento que llega a su destino con un checksum incorrecto (corrupto) y uno que nunca llega. El receptor no puede procesar la información que contiene un segmento corrupto y por tanto lo descarta, de modo que las consecuencias son las mismas. En ambos casos, el receptor no envía confirmación, el RTO expirará y el emisor lo retransmitirá. La Figura 13.9 ilustra esta situación.

13.12.2. Confirmación perdida/corrupta

Una confirmación corrupta es descartada igual que cualquier otro segmento corrupto. La consecuencia, como en el caso anterior, es que el RTO expirará y habrá una retransmisión. Sin embargo, como las confirmaciones son acumulativas, si llegara una confirmación sobre un número de secuencia posterior, el segmento quedaría confirmado a pesar de todo y la pérdida/corrupción del segmento de confirmación no tendría ninguna consecuencia (Figura 13.10).

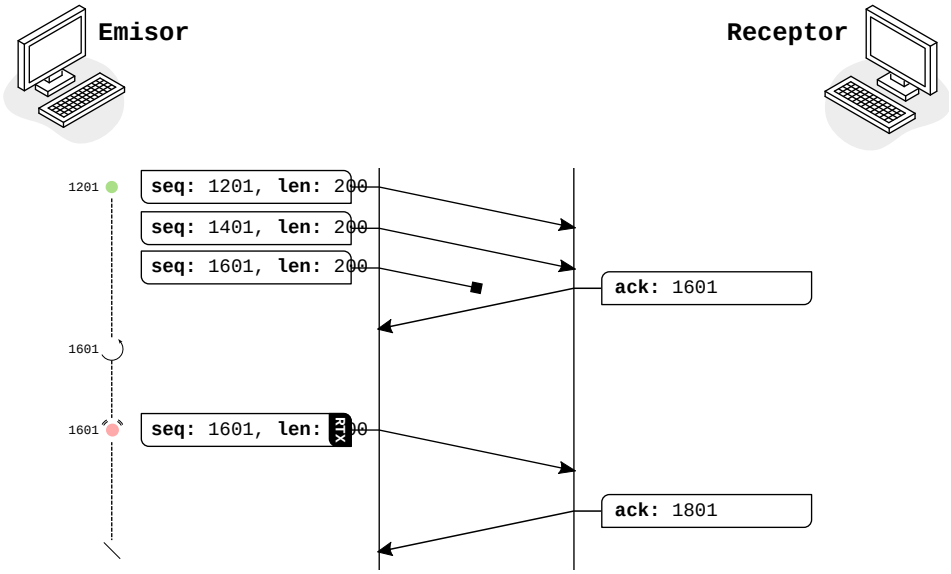


FIGURA 13.9: TCP: Segmento perdido

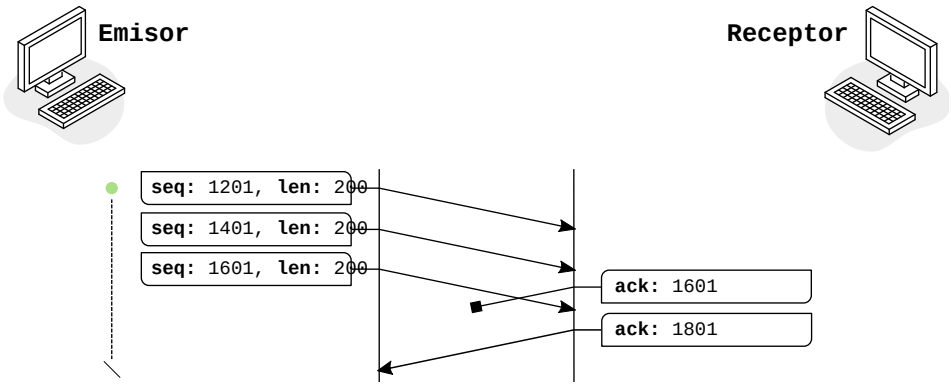


FIGURA 13.10: TCP: Confirmación perdida

13.12.3. Segmento duplicado

Cuando una confirmación se pierde y no existe otra que la sustituya de forma acumulativa, el emisor retransmite el segmento correspondiente y el receptor recibirá un duplicado. En esta situación el receptor ignora el duplicado, pero está obligado a enviar una confirmación inmediatamente (ver Figura 13.11).

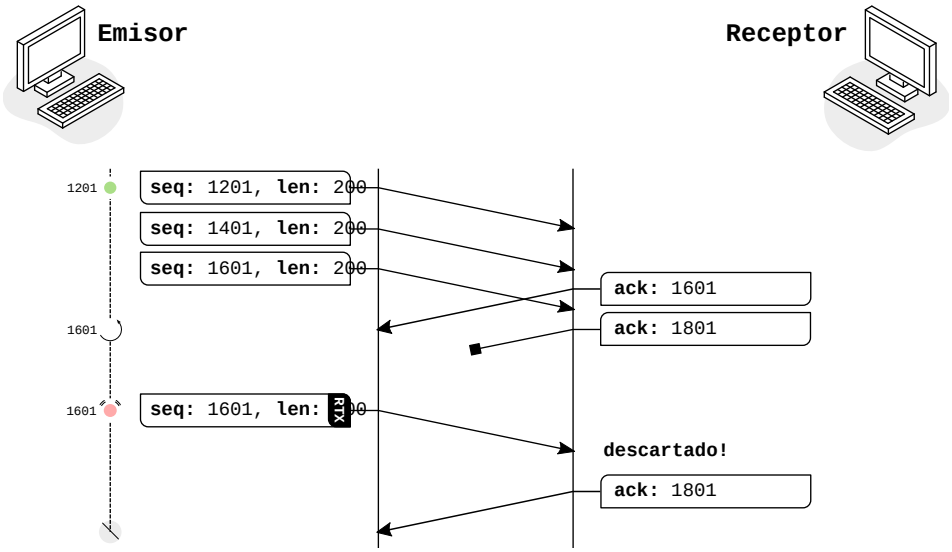


FIGURA 13.11: TCP: Segmento duplicado

13.12.4. Segmento fuera de orden

En una red de conmutación de paquetes como Internet es posible que segmentos de un mismo flujo o conexión lleguen al receptor en un orden diferente al que fueron enviados (Figura 13.12).

Esto puede ocurrir por varias razones, por ejemplo, porque el segmento que llega tarde se perdió y fue retransmitido, o porque el segmento que llegó antes se retrasó por alguna razón (congestión, colas en los routers, etc.). En cualquier caso, el receptor no puede procesar un segmento fuera de orden hasta que lleguen los segmentos anteriores.

Como hemos visto anteriormente, el receptor almacenará los segmentos fuera de secuencia en el buffer de recepción y enviará confirmaciones para el último segmento que llegó respetando la secuencia. Cuando finalmente llegue el segmento que falta, el receptor enviará una confirmación por todo el conjunto.

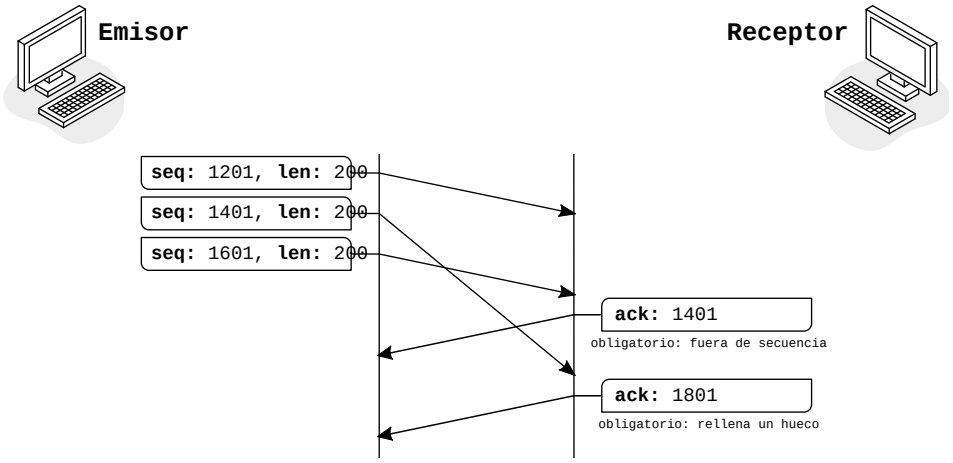


FIGURA 13.12: TCP: segmento fuera de orden

13.12.5. Confirmación duplicada

Tanto si un segmento de datos se pierde como si llega fuera de orden, el receptor enviará varias veces la misma confirmación para los siguientes segmentos que lleguen correctamente a continuación. Al emisor no le afectan las confirmaciones duplicadas, pero como veremos más adelante, le dan información valiosa sobre el estado de la red. La Figura 13.13 muestra un escenario con confirmaciones duplicadas.

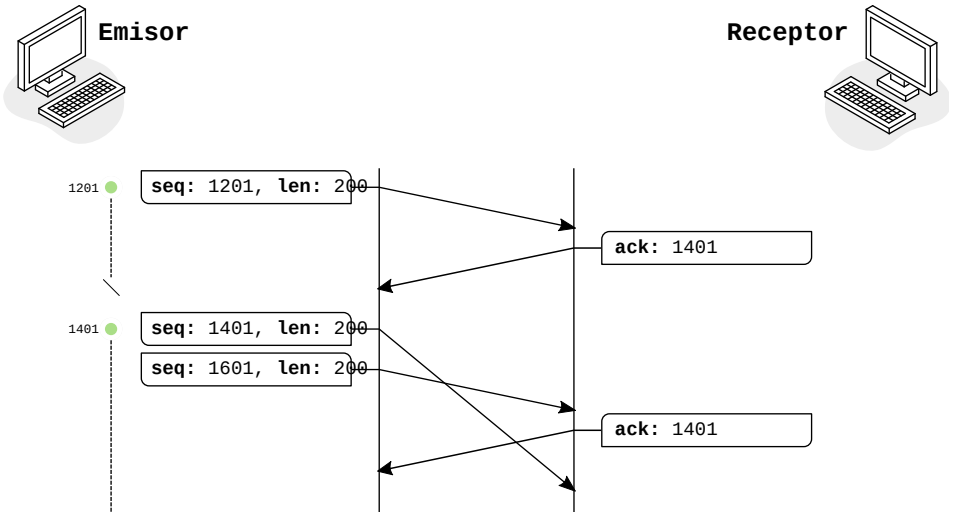


FIGURA 13.13: TCP: Confirmación duplicada

13.12.6. Demasiadas retransmisiones

Si durante el transcurso de una conexión, se suceden las retransmisiones sin lograr confirmar nuevos datos es una señal clara de que algo va realmente mal. Se definen dos umbrales que implican medidas distintas (ver [48]). Si el número de retransmisiones sobrepasa R1, que tiene normalmente un valor de 3, se entiende como un indicio de que podría haber un problema en el rutado de paquetes y se informa al proceso. Si sobrepasa R2, que suele tener un valor de 15, se cierra la conexión. En el caso de Python, eleva una excepción `OSError` con el valor `errno.ETIMEDOUT`.

En Linux puedes consultar estos valores como parámetros del núcleo (§ 2.10): R1 se corresponde con `tcp_retries1` y R2 con `tcp_retries2`. El Listado 13.14 muestra cómo hacerlo, con los valores por defecto.

```
$ sysctl net.ipv4.tcp_retries1
net.ipv4.tcp_retries1 = 3
$ sysctl net.ipv4.tcp_retries2
net.ipv4.tcp_retries2 = 15
```

LISTADO 13.14: TCP: Configuración de los umbrales de retransmisión

Estos valores raramente se modifican. Solo ocurre en situaciones muy concretas, con enlaces de muy alta latencia o mucha inestabilidad, como enlaces vía satélite.

13.13. *Keep alive*

La conexión TCP puede quedar abierta durante mucho tiempo sin que ninguno de los dos extremos envíe segmentos (datos ni confirmaciones). Es estas condiciones se dice que la conexión está «inactiva» *(idle)*. Para asegurarse de que el otro extremo sigue ahí, TCP proporciona un mecanismo llamado *keep alive* (mantener vivo) que consiste en enviar un segmento «sonda» para provocar una respuesta (una confirmación) del otro extremo y así verificar que sigue «vivo».

El segmento sonda se envía cuando expira el temporizador *keep-alive* (2 horas). El temporizador se reinicia cada vez que se recibe un segmento. Si el otro extremo no responde a la sonda, el emisor repetirá la operación cada 75 segundos hasta un máximo de 10 intentos. Si a pesar de eso no recibe respuesta, el emisor cierra la conexión. Todos estos valores son configurables por el programador para cada conexión.

- `SO_KEEPALIVE`: Activa o desactiva el mecanismo *keep alive*.
- `TCP_KEEPIDLE`: Tiempo de inactividad antes de la primera sonda.
- `TCP_KEEPINTVL`: Intervalo entre sondas.

- `TCP_KEEPCNT`: Número máximo de intentos antes de cerrar la conexión.

El Listado 13.15 muestra cómo fijar estos valores en un programa Python.

```
ç#ç Activar keep-alive
sock.setsockopt(socket.SOL_SOCKET, socket.SO_KEEPALIVE, 1)

# Configurar el temporizador keep-alive (en segundos)
sock.setsockopt(socket.IPPROTO_TCP, socket.TCP_KEEPIDLE, 3600)
```

Listado 13.15: TCP: Configuración del mecanismo *keep alive*

Y ¿qué más?

Este capítulo ha repasado el control de flujo que, junto al control de errores, son los mecanismos fundamentales que proporcionan confiabilidad a TCP y por tanto a la mayoría de las comunicaciones en Internet en la actualidad.

Sin embargo, queda al menos otra gran cuestión que abordar: la congestión. Se trata de un problema más complejo que el control de errores, porque no afecta solo a una conexión concreta, sino a una parte o a toda la interred. En el próximo capítulo veremos cómo TCP aporta soluciones para paliar este problema.

Capítulo 14

Control de congestión

Al terminar este capítulo, entenderás:

- Qué es la congestión y cuáles son sus causas.
- Por qué la congestión es un problema grave.
- Qué técnicas se pueden aplicar para prevenir o mitigar la congestión.
- Qué técnicas de control de congestión utiliza TCP y cómo funcionan.

Intuitivamente una congestión es una situación en la que la demanda de recursos supera la capacidad del sistema para proporcionarlos. En el caso de las redes de conmutación de paquetes, la congestión se produce cuando la carga, es decir, la cantidad de paquetes que se introduce en la red, supera su capacidad para manejarlos adecuadamente y llevarlos a su destino.

Este capítulo trata las causas de la congestión, sus consecuencias y las técnicas que se pueden aplicar para prevenirla o bien controlarla una vez que se ha producido.

14.1. Carga, capacidad y congestión

Podemos cuantificar la **carga** de la subred como la cantidad de paquetes que se encuentran en tránsito en un determinado momento. La subred está formada por líneas de comunicaciones y dispositivos de intercambio, de modo que los bytes que forman los paquetes pueden estar físicamente viajando a través de un enlace o bien en las colas de entrada y salida de los routers. Desde este punto de vista, podemos entender la subred como una especie de almacenamiento temporal, una 'memoria' que contiene paquetes durante el tiempo que transcurre desde que el emisor los envía hasta que son entregados a su destino. La cantidad máxima de paquetes que cabe en esta 'memoria' la podemos entender, por analogía, como la **capacidad** de la subred.

Por simplicidad, hemos hablado de paquetes, pero obviamente esos paquetes son de tamaño muy variado, y lógicamente para una misma cantidad de paquetes, la carga y la capacidad son mayores si hablamos de paquetes de mayor tamaño. Así que, para obtener un cálculo más preciso de carga y capacidad, habría que medirlas en bytes.

Una vez están claros estos conceptos, podemos entender la **congestión** como toda situación en la que la carga supera la capacidad de la subred (ver Fórmula 14.1), es decir, se produce una *sobrecarga* de la subred.

$$\text{congestión} = \text{carga} > \text{capacidad} \tag{14.1}$$

Es necesario analizar con más detalle las colas de entrada y salida de los routers. Los paquetes llegan al router a través de las interfaces de red y se almacenan en las colas de entrada, asociadas a cada interfaz. El componente del router que realiza el reenvío (ver § 8.1) va tomando y procesando paquetes desde la cabeza de cada cola. Eso significa que cuantos más paquetes entran al router, mayor es la ocupación de las colas de entrada y más tiempo pasan los paquetes en ellas, lo que aumenta la latencia. Si cuando llega un nuevo paquete, la cola ya está llena, el router lo descarta *(tail drop)*[1]. Esta situación se representa en la Figura 14.1. La cola de entrada de la interfaz superior está llena y no puede aceptar nuevos paquetes.

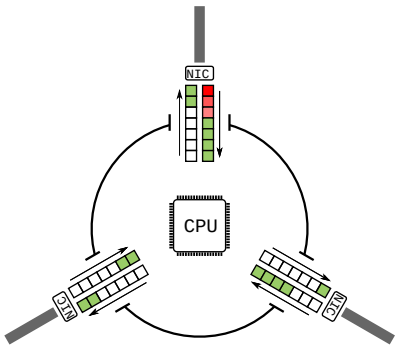


FIGURA 14.1: Esquema de las colas de entrada y salida de un router

Una vez procesado, el paquete se coloca en la cola de salida asociada a la interfaz de red por la que será enviado. La interfaz de salida necesita cierto tiempo para enviar un paquete, por lo que si la velocidad con la que el router reenvía paquetes por esa interfaz (los encola para su envío) es mayor que la velocidad con la que se transmiten al medio, la cola de salida

[1] A menos que el router aplique una política de descarte selectivo

también se puede llegar a llenar y también habrá paquetes descartados por ese motivo.

Como puede verse, cuando la carga se acerca o supera la capacidad de la subred, los routers empezarán a descartar paquetes, que es precisamente la consecuencia más grave de la congestión. Pero hay otros efectos negativos, como el incremento de la latencia, que pueden ocurrir mucho antes. La Figura 14.2 muestra cómo la latencia aumenta con la carga y el impacto que tiene sobre el rendimiento al acercarse la carga a la capacidad de la subred. El rendimiento puede medirse como la cantidad de paquetes que la subred es capaz de llevar a su destino por unidad de tiempo.

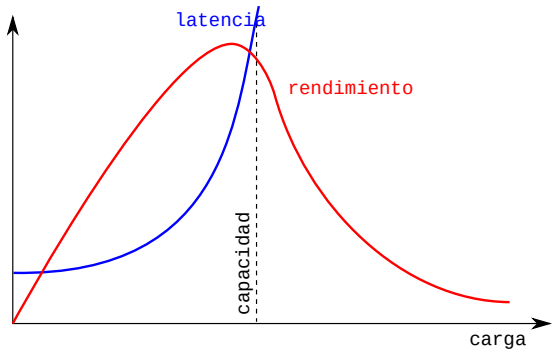


Figura 14.2: Carga, capacidad, latencia y rendimiento

Quizá pienses que la causa de todos estos problemas es que las colas del router son demasiado pequeñas. Sin embargo, aumentar el tamaño no soluciona el problema. Si la cola es mayor, los paquetes pasarán más tiempo en ella, aumentando la latencia del flujo y, en el caso extremo, provocando la expiración de los temporizadores de retransmisión, cuando se trate de tráfico confiable. Incluso obviando ese efecto indeseado, mayores colas solo retrasarían el momento en que la congestión se hace patente. Este efecto se conoce como *bufferbloat* y es muy perjudicial para transmisiones sensibles a la latencia como videollamadas y juegos en línea.

El tamaño ideal de las colas debería ser cercano al BDP de la ruta, es decir, a la cantidad máxima teórica de datos que pueden estar en tránsito en ella. Para un único enlace, el BDP depende de su velocidad de transmisión R (§ 12.5). Una ruta, en cambio, atraviesa varios enlaces con velocidades distintas y su ritmo lo marca el más lento, el cuello de botella. La velocidad de ese enlace es la *capacidad* de la ruta (C):

$$C = \min_i R_i \qquad \text{BDP} = C \times \text{RTT}$$

donde:

R_i es la velocidad de transmisión de cada uno de los enlaces de la ruta.

Un ejemplo: en una ruta cuyo enlace más lento es de 50 Mbps y con un RTT de 20 ms, BDP = 50 Mbps × 20 ms = 1 Mb = 125 kB. Si la cola del router es menor, se desborda y provoca descarte de paquetes. Si la cola es mayor, los paquetes pasan más tiempo del necesario en la cola, aumentando la latencia. Lamentablemente, como este valor depende de la ruta, no es posible determinar el tamaño óptimo de las colas, pero sirve como referencia.

En todo caso, la congestión raramente ocurre a la vez en toda la subred. Suele localizarse en algunos routers o enlaces involucrados en determinadas rutas con más uso en ese momento. Mientras tanto quizá en otras partes todo va bien, aunque de no tomarse medidas, la congestión puede propagarse rápidamente.

14.2. Control de congestión

Por supuesto, la congestión es un problema muy grave que puede inutilizar parte de una interred. La solución es lo que llamamos **control de congestión**, que consiste en aplicar técnicas que permitan evitar, o al menos minimizar, los efectos de la congestión.

Estas son algunas de las técnicas habituales, agrupadas en dos categorías: preventivas y reactivas. En ambos casos, se basan en la idea de reducir la tasa de emisión de mensajes, o en menor medida, el tamaño de estos. Reduciendo la cantidad de paquetes que se introducen en la red, lógicamente se reduce la carga y, con ello, la posibilidad de alcanzar la capacidad. Sin embargo, recuerda que el rendimiento óptimo de la subred está cerca de la capacidad, de modo que el objetivo es buscar el equilibrio, llegar al punto de rendimiento máximo sin sobrepasarlo.

14.2.1. Técnicas preventivas

Como su nombre indica, se aplican para evitar que la congestión llegue a producirse. Tanto los emisores como los receptores pueden colaborar. También se llaman *técnicas de bucle abierto* porque no se realizan medidas ni se obtiene información explícita de lo que realmente ocurre en la red.

Política de retransmisión

Cuando un paquete se pierde, el emisor puede aumentar la duración

del temporizador de retransmisión, para que en una situación de congestión, los paquetes se retransmitan más tarde de lo habitual, reduciendo así la tasa de emisión.

Política de confirmación
Si el receptor estima una situación de congestión puede retrasar el envío de mensajes de confirmación, lo que indirectamente reduce la tasa de emisión. Obviamente este retardo adicional debe estar dentro de los límites que eviten retransmisiones innecesarias, que provocarían un aumento de carga, agravando la congestión. También se puede aprovechar la *confirmación acumulativa* que además reduce el número de mensajes en tránsito.

Política de ventana
El tamaño de la ventana de envío determina la cantidad de datos que pueden estar en tránsito sin confirmar, de modo que una ventana limitada acota la carga que ese emisor introduce en la red. Además, un protocolo de confiabilidad de tipo *repetición selectiva* o *confirmación negativa* reduce las retransmisiones innecesarias, que también contribuyen a la carga.

Política de descarte
Los routers pueden decidir qué paquetes resulta más conveniente descartar en caso de congestión. Por ejemplo, descartar paquetes que corresponden a protocolos confiables provocará retransmisiones, y eso no ayuda a resolver el problema. Pueden ser técnicas muy efectivas, pero requieren conocimiento detallado de la topología de la red y de los protocolos de aplicación.

Política de admisión
Determina si la red dispone de recursos suficientes para aceptar un nuevo flujo de datos, rechazándolo si no fuera así. Esta técnica es tremendamente efectiva, pero solo es posible en tecnologías de conmutación de circuitos o circuitos virtuales.

14.2.2. Técnicas reactivas

Se aplican una vez que la congestión ha empezado a producirse y algunos de sus síntomas son perceptibles. Se denominan también *técnicas de bucle cerrado* porque se basan en la retroalimentación de información sobre el estado de la red que procede, bien de los receptores, o de otros dispositivos intermedios.

Backpressure
: Se trata de una técnica típica de las redes de circuitos virtuales. Cuando un dispositivo de interconexión (conmutador según su terminología[2]) detecta congestión, puede enviar un mensaje al conmutador inmediatamente anterior para que reduzca su tasa. Esto implicará el descarte de mensajes, de modo que este conmutador enviará a su vez un mensaje de notificación de congestión al conmutador previo y así hasta llegar al dispositivo emisor. Este último reducirá su tasa de emisión y la congestión se irá reduciendo progresivamente.

Paquete de estrangulamiento *(choke)*
: Es similar a *backpressure*. Se envía un mensaje específico de notificación de congestión directamente al emisor, mientras que los dispositivos intermedios no son notificados.

Señalización implícita
: En este caso el emisor interpreta ciertos indicios en el comportamiento del flujo o conexión como señales de que está ocurriendo un episodio de congestión. Uno de estos indicios podría ser la ausencia o retraso en los mensajes de confirmación.

Señalización explícita
: Es similar al paquete *choke*, pero no se utiliza un mensaje de notificación de congestión específico, sino que se utilizan campos de los propios mensajes de datos. En algunas tecnologías, como en Frame Relay, esta información podía enviarse tanto al emisor *(backward signaling)* como al receptor *(forward signaling)*.

14.3. Control de flujo vs. control de congestión

Ambos, control de flujo y control de congestión se basan en ajustar la tasa a la que nuevos datos entran en la red. Sin embargo, su objetivo es muy diferente. El control de flujo (de receptor) intenta evitar que un emisor específico sature a un receptor específico, es decir, es un problema que atañe únicamente a un único flujo de datos (normalmente una conexión). Por otro lado, el control de congestión busca evitar que una subred o parte de ella colapse por sobrecarga de tráfico, es decir, la congestión afecta a muchos flujos de datos simultáneamente, y para resolverla probablemente múltiples emisores tendrán que colaborar reduciendo sus tasas de emisión.

[2] No confundir con conmutadores Ethernet

Como son dos problemas independientes pueden ocurrir de forma independiente. Pueden darse situaciones en las que un emisor tenga que aplicar control de flujo porque está saturando a su receptor, y sin embargo, la red se encuentre en condiciones de baja carga. También puede ocurrir lo contrario: que un emisor tenga que aplicar control de congestión porque la red está saturada, pero su receptor no tenga ningún problema con la tasa de llegada de los datos. Y obviamente también pueden coincidir ambos problemas al mismo tiempo.

14.4. Enfriamiento en origen

El protocolo IPv4 ofrecía un mecanismo para control de congestión muy rudimentario mediante el mensaje ICMP *source quench*, que se podría traducir como «enfriamiento en origen».

Cuando un router IP tiene un alto nivel de ocupación en una de sus colas (señal inequívoca de congestión) y se ve obligado a descartar un paquete, puede enviar un paquete *source quench* al emisor para que reduzca su tasa de emisión. El mensaje incluye como carga útil la cabecera del paquete descartado más los primeros 8 bytes de sus datos (ver § 8.6), de modo que el emisor puede identificar a qué flujo corresponde. Conforme a la clasificación anterior, el uso de este tipo de notificación se puede entender como una técnica reactiva similar al *choke packet*.

Un receptor también puede enviar mensajes *source quench* si no puede manejar la tasa de recepción de datos, de modo que en ese caso funciona como un sistema de control de flujo, no de congestión.

Desde hace años el uso de paquetes *source quench* se considera obsoleto por varias razones [15, 49], y de hecho, no se incluyó ningún mecanismo equivalente en IPv6. Las razones más importantes son las siguientes:

- Su eficacia es limitada si el emisor recibe el mensaje tarde, o simplemente el router no lo envía.
- El mecanismo no incluye una medida de la gravedad de la congestión. El emisor debe decidir cuánto y durante cuánto tiempo reducir la tasa después de recibir las notificaciones.
- Un agente malicioso podría enviar notificaciones falsas para reducir la tasa de una víctima consiguiendo así una denegación de servicio (DoS).
- Los mecanismos de control de congestión de TCP son mucho más eficaces.

14.5. Control de congestión en TCP

TCP cuenta con mecanismos de control de congestión muy sofisticados. Se trata de mecanismos de «caja negra», es decir, los elementos intermedios de la red, como los routers, no participan y no tienen que hacer nada específico para que el control de congestión de TCP funcione correctamente.

El concepto clave es la *ventana de congestión (congestion window)*, abreviado como *cwnd*. La ventana de congestión limita en todo momento la ventana de envío, es decir, controla la tasa de emisión de cada conexión, por lo que claramente es una técnica de «política de ventana» según la clasificación de § 14.2. Por supuesto, el valor de la ventana de congestión no es fijo, sino que se recalcula continuamente para adaptarse a las condiciones de la red; en concreto cwnd trata de estimar continuamente el valor BDP. Con esto podemos refinar el modo en que se calcula el valor de la ventana de envío (ver Fórmula 14.2), en contraposición a la versión simplificada de la Fórmula 13.1.

$$\text{swnd} \leq \text{mín}(\text{rwnd}, \text{cwnd}) \tag{14.2}$$

Esta fórmula tan sencilla dice algo interesante: ambos mecanismos, control de flujo y control de congestión, son igualmente importantes y tienen la misma capacidad de limitar la tasa de emisión.

La forma en la que se determina el valor de la ventana de congestión y el comportamiento de las retransmisiones depende de 5 algoritmos:

- Arranque lento *(Slow Start)*
- Evitación de congestión (*Congestion Avoidance*)
- Decrecimiento multiplicativo *(Multiplicative Decrease)*
- Retransmisión rápida *(Fast Retransmit)*
- Recuperación rápida *(Fast Recovery)*

Las siguientes secciones tratan cada uno de ellos.

14.5.1. Arranque lento (*slow start*)

El *arranque lento* pretende alcanzar una tasa de emisión significativa partiendo de la mínima, pero con un margen de seguridad que evite provocar congestión. Se aplica en el inicio de la conexión o justo después de un RTO. Aunque el arranque lento es solo una de las fases, es habitual encontrar bibliografía que se refiere a todo el control de congestión de TCP como «arranque lento», probablemente porque en las primeras versiones de TCP era de hecho el único algoritmo que había.

El valor de la ventana de congestión se actualiza en base a rondas. Una «ronda» es el tiempo que transcurre desde que se envían los segmentos asociados a una ventana hasta que comienzan a llegar sus respectivos ACK, lo que normalmente equivale a la medición de un RTT.

Inmediatamente después de establecerse la conexión, se fija el tamaño de la ventana de congestión *(initial window)* en MSS[3]. Así, el emisor puede empezar enviando un segmento de tamaño MSS. Al llegar su correspondiente confirmación, cwnd crece hasta 2 MSS. En la segunda ronda puede enviar 2 segmentos de tamaño MSS. Sus respectivas confirmaciones aumentan cwnd hasta 4 MSS, y así sucesivamente. La evolución de cwnd aparece en la Figura 14.3.

Este crecimiento exponencial (2^r, siendo r la ronda) continúa hasta que alcanza *ssthresh* (abreviatura de *slow start threshold*) o bien se detecta congestión. Inicialmente *ssthresh* se fija en un valor alto, habitualmente el tamaño máximo de ventana ($2^{16} - 1$), pero cuando se detecte congestión, se reducirá sensiblemente.

Es necesario aclarar que este crecimiento solo ocurre si rwnd no es un limitante, es decir, siempre que se cumpla $rwnd > cwnd$. En otro caso, la tasa de emisión estará condicionada por rwnd que a su vez, limitará el crecimiento de cwnd. Si el emisor envía un segmento de, por ejemplo, MSS/2 debido a un valor de rwnd bajo, la confirmación correspondiente aumentará cwnd únicamente en esa cantidad.

En resumen, durante el arranque lento, cwnd aumenta en la cantidad de nuevos bytes (N) enviados por cada mensaje de confirmación, siempre que esa cantidad no supere MSS (ver Fórmula 14.3). Si efectivamente, los segmentos son de tamaño MSS, cwnd crece a un ritmo MSS por cada confirmación efectiva, o lo que es lo mismo, cwnd se duplica al final de cada ronda.

$$\text{cwnd} \mathrel{+}= \text{mín}(\text{N}, \text{MSS}) \tag{14.3}$$

Cuando $cwnd > ssthresh$, la fase de arranque lento termina y comienza una fase de *evitación de congestión*.

[3]Esto es una simplificación —de las muchas que aplicaremos en este capítulo— pero sirve para el propósito del libro. Como referencia, las versiones actuales de GNU/Linux suelen fijar un mínimo de cwnd=10 MSS.

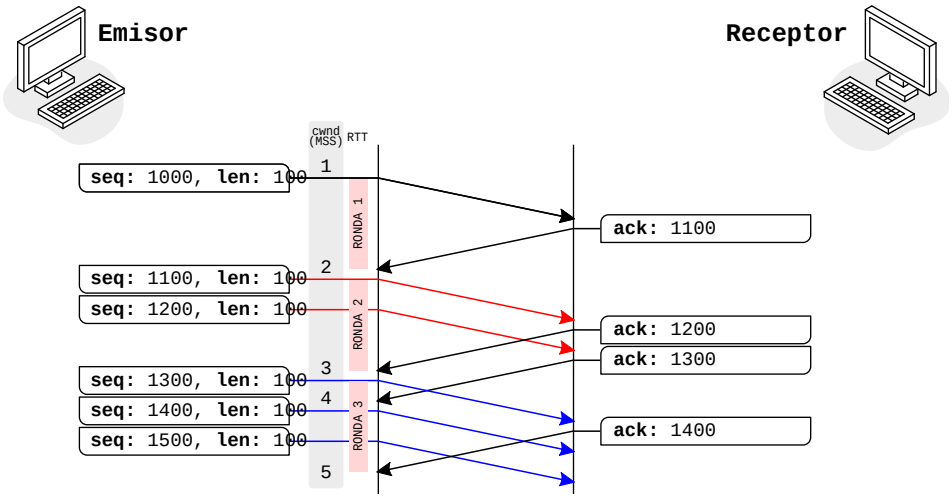


FIGURA 14.3: Evolución de la ventana de congestión durante una fase de arranque lento

14.5.2. Evitación de congestión *(congestion avoidance)*

En esta fase cwnd crece de forma lineal según la Fórmula 14.4 con cada confirmación neta recibida, que corresponde aproximadamente a un MSS/k siendo k el número de segmentos enviados en esa ronda. Dicho de otro modo, al final de cada ronda, cwnd habrá crecido aproximadamente MSS, siempre que como se indicó antes, rwnd no sea un limitante. La Figura 14.4 muestra un ejemplo de esta fase.

$$\text{cwnd} \mathrel{+}= \text{MSS}^2/\text{cwnd} \tag{14.4}$$

En este caso no se aplica ningún umbral, la ventana continúa creciendo indefinidamente hasta que se detecte algún problema. Sin embargo, el valor de cwnd respecto a *ssthresh* se utiliza para determinar en qué fase se encuentra la conexión:

- Si $cwnd < ssthresh$, la conexión está en una fase de arranque lento.
- Si $cwnd > ssthresh$, la conexión está en una fase de evitación de congestión.
- Si $cwnd = ssthresh$, el emisor puede aplicar cualquiera de los dos. El estándar [50] no establece ningún criterio y deja la elección a la implementación de TCP de cada SO.

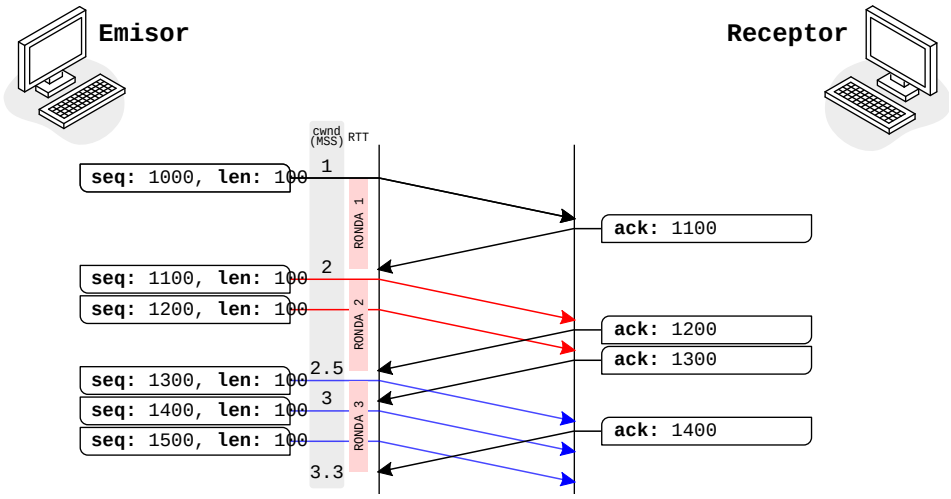


Figura 14.4: Evolución de la ventana de congestión durante una fase de evitación de congestión

14.5.3. Decrecimiento multiplicativo

Esta fase, como indica su nombre, es una reducción drástica de la ventana de congestión que se aplica cuando se detecta congestión. Aunque estaba diseñada para limitar la tasa durante cierto tiempo, en la actualidad solo se aplica durante una ronda, por lo que en realidad actúa más como transición que como fase estable.

14.5.3.1. Indicios de congestión

Aunque «detección de congestión» es una expresión habitual, en realidad TCP asume que la conexión está sufriendo los efectos de un episodio de congestión debido a dos *indicios* muy concretos:

- Ha expirado el RTO. TCP asume que la explicación más probable de la expiración del RTO y la correspondiente pérdida de un segmento es que un router ha descartado el paquete debido a la alta ocupación de sus colas, señal inequívoca de congestión.
- El emisor ha recibido 3 confirmaciones duplicadas (*3dupack*[4]), es decir, han llegado un total de 4 mensajes de confirmación idénticos desde un receptor al que se le están enviando datos nuevos. Las confirmaciones

[4]En muchos documentos técnicos es habitual encontrar «dupack» para indicar las 3 confirmaciones duplicadas. Aquí utilizaremos el más explícito «3dupack» para evitar cualquier confusión.

duplicadas se producen porque el receptor está recibiendo segmentos correctos, pero fuera de secuencia (ver § 13.12.5). La Figura 14.5 muestra un ejemplo. Por supuesto, un emisor podría recibir más de 3 en una ronda, dependiendo de cuántos segmentos la formen.

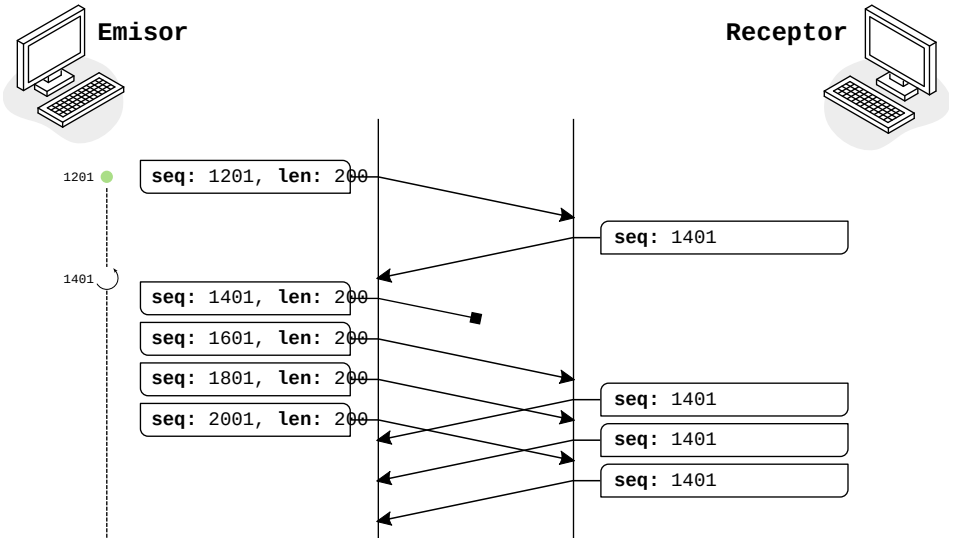


FIGURA 14.5: 3 ACK duplicados, como consecuencia de la pérdida (o retraso) de un segmento

La detección de la congestión en base a estos indicios se puede interpretar como una técnica de 'señalización implícita' según la clasificación de § 14.2. Estos indicios tienen consecuencias diferentes. La expiración del RTO es un evento más grave y es más probable que sea síntoma de congestión. La aparición de *3dupack* en cambio indica que los mensajes siguientes, correspondientes a la ronda, siguen llegando al destino. Por tanto, la congestión, de haberla, no parece tan grave.

Cuando ocurre una expiración del RTO, la fase en curso termina inmediatamente. Se inicia entonces una nueva fase de arranque lento, y se reajustan los valores de cwnd (Fórmula 14.5) y *ssthresh* (Fórmula 14.6) del siguiente modo.

$$\text{cwnd} = \text{MSS} \tag{14.5}$$

$$\text{ssthresh} = \text{máx}(2\text{MSS}, \frac{\text{Flightsize}}{2}) \tag{14.6}$$

donde:

Flightsize es la cantidad de bytes que el emisor ha enviado y que todavía no han sido confirmados: cantidad de datos en vuelo.

El nuevo valor de *ssthresh* provoca que, en la siguiente fase de arranque lento, el crecimiento de cwnd sea más conservador que en el inicio de la conexión. Por eso, la tasa de emisión se adapta con más suavidad a las condiciones de la red.

Cuando se produce *3dupack*, la fase en curso termina. En ese momento se aplica el algoritmo de 'retransmisión rápida' *(fast retransmit)* que inmediatamente realiza una retransmisión del segmento al que corresponde la confirmación duplicada. Se llama así porque no espera la expiración del RTO.

Fíjate que la aparición de los ACK duplicados no implica necesariamente que el mensaje se haya perdido. Podría acabar llegando y el receptor enviaría una confirmación que, de llegar a tiempo, evitaría la retransmisión.

Después de la retransmisión rápida se aplica el algoritmo de 'recuperación rápida' *(fast recovery)*. No siempre fue así: la retransmisión rápida se introdujo en la versión Tahoe de TCP [45], que tras retransmitir iniciaba una nueva fase de arranque lento. Pronto se comprobó que esa reducción de tasa era demasiado agresiva y provocaba una caída innecesaria de rendimiento, por eso la versión Reno incorporó la recuperación rápida. Este algoritmo primero fija *ssthresh* según la Fórmula 14.6 y asigna ese mismo valor a cwnd. Después lleva a cabo un período de «inflación» que incrementa cwnd en 3 MSS para compensar las confirmaciones duplicadas. A partir de ahí, incrementa cwnd en MSS por cada confirmación duplicada adicional porque se entiende que los segmentos siguen fluyendo. Durante este período, el emisor puede enviar segmentos con datos nuevos. Cuando por fin se reciba la confirmación para el segmento retransmitido, se produce la «deflación», es decir, cwnd vuelve al valor *ssthresh* calculado previamente. La conexión continúa en una fase de evitación de congestión.

La Figura 14.6 muestra el diagrama de transición entre fases según lo explicado.

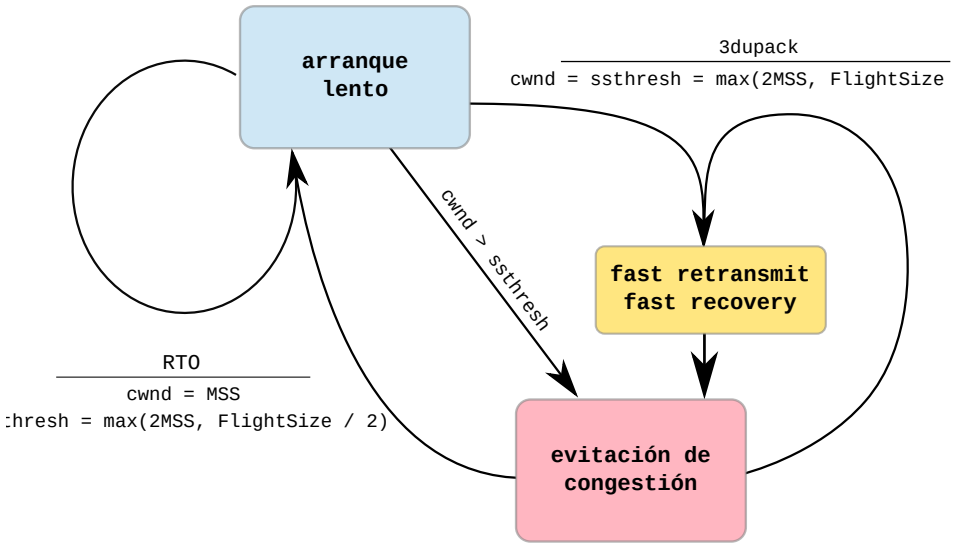


FIGURA 14.6: Transición entre los algoritmos de control de congestión de TCP

14.5.4. Modelo simplificado

Por facilitar el estudio y comprensión de estos mecanismos podemos aplicar una serie de simplificaciones que no afectan significativamente a su finalidad, pero que nos permiten realizar cálculos más sencillos.

En la fase de evitación de congestión, en lugar de aplicar la Fórmula 14.4 para determinar el crecimiento de cwnd, utilizaremos la Fórmula 14.7, que divide el crecimiento de MSS de la ronda en los k segmentos que se envían en ella, con lo que al final de la ronda cwnd también habrá crecido MSS.

$$\text{cwnd} += \text{MSS}/k \tag{14.7}$$

Cuando expira el RTO, calcularemos el nuevo valor de *ssthresh* según la Fórmula 14.6 que utiliza swnd como aproximación a *Flightsize*. También obviamos el mínimo de 2 MSS. El cálculo simplificado es el de la Fórmula 14.8.

$$\text{ssthresh} = \text{swnd}/2 \tag{14.8}$$

Esta misma simplificación la aplicamos a la reducción de cwnd cuando se detecta congestión por *3dupack*. También obviamos los algoritmos de retransmisión rápida y recuperación rápida.

$$\text{cwnd} = \text{swnd}/2 \tag{14.9}$$

Algunos autores aplican la simplificación cwnd/2, pero eso implica que el receptor nunca limita la ventana de envío, es decir, que siempre se cumple $rwnd > cwnd$. Ambas pueden ser simplificaciones razonables, pero consideramos que swnd/2 contempla ambos casos: tasa limitada por el receptor o por la congestión de la red; por lo que lo consideramos más cercano a la intención de la fórmula original.

La Figura 14.7 muestra el diagrama de estados entre las fases arranque lento y evitación de congestión, aplicando las simplificaciones indicadas.

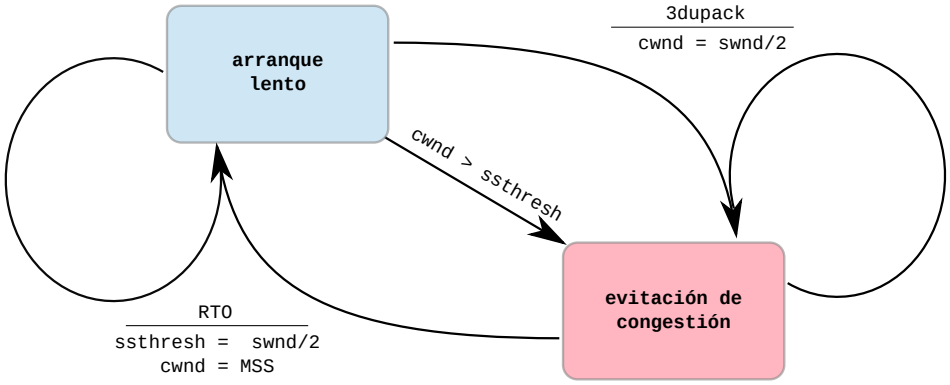


FIGURA 14.7: Transición entre los algoritmos de control de congestión de TCP (modelo simplificado)

En la Figura 14.8 aparece un ejemplo más completo para entender cómo evoluciona la ventana de congestión conforme ocurren diferentes eventos a lo largo de la conexión. El valor de cwnd aparece en el eje vertical expresado en MSS mientras que el eje horizontal muestra las rondas, o períodos RTT equivalentes. Por simplicidad este ejemplo supone que $rwnd > cwnd$ durante toda la conexión, de forma que $swnd = cwnd$.

Así progresa la ventana de congestión respondiendo a los indicios que ocurren durante la conexión:

Ronda 0 – Al inicio de la conexión hay siempre una fase de arranque lento (AL) que fija $cwnd = MSS$ y lo duplica en cada ronda.

Ronda 4 – Se detecta una situación 3dupack. TCP responde según el diagrama de la Figura 14.7 fijando $cwnd = swnd/2 = 16/2 = 8MSS$ con una transición de disminución multiplicativa (DM) y cambia a evita-

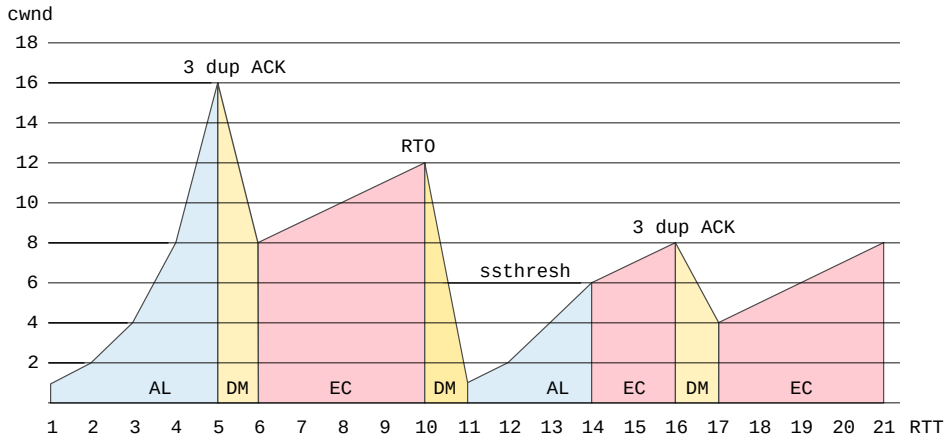


FIGURA 14.8: Ejemplo de evolución de la ventana de congestión en TCP

ción de congestión (EC). La fase EC incrementa el valor de cwnd en MSS en cada ronda.

Ronda 9 – Aparece otro problema: la expiración del RTO. Esta vez la respuesta es más agresiva, cwnd se reduce a MSS y comienza una nueva fase de arranque lento, pero también se ajusta $ssthresh = swnd/2 = 12/2 = 6MSS$.

Ronda 13 – El valor de cwnd alcanza *ssthresh*, por lo que entra en evitación de congestión.

Ronda 15 – De nuevo otra situación 3dupack, y de nuevo se aplica $cwnd = swnd/2 = 8/2 = 4MSS$. A partir de aquí cwnd sigue creciendo de forma lineal hasta la ronda 20.

14.6. Gestión activa de colas

Que los routers descarten paquetes cuando sus colas de salida desbordan tiene algunas consecuencias negativas adicionales a la propia pérdida de los paquetes. Es muy probable que paquetes que corresponden a conexiones diferentes lleguen al router mientras la cola está llena, y todos ellos sufrirán la misma suerte. Esto puede provocar que esas conexiones *sincronicen* sus retransmisiones, un efecto que no es deseable en absoluto.

La Gestión Activa de Colas (en inglés AQM) es un conjunto de técnicas para detectar la congestión antes de que las colas de salida del router se llenen por completo y, en ese caso, informar de la situación a los nodos finales. En Internet, este tipo de notificaciones se realizan mediante Notificación Explícita de Congestión (en inglés ECN) que, como su nombre indica, es una técnica de bucle cerrado. Los routers AQM no crean mensajes específi-

cos sino que etiquetan con la notificación Congestion Experienced (CE) la cabecera de los paquetes IP en tránsito. Estos paquetes «marcados» siguen su camino del modo habitual hacia su destino.

Con AQM, los routers tienen dos opciones al detectar una congestión inminente. Pueden *notificar* la congestión de forma implícita: descartando un paquete aunque no haya desbordamiento de la cola de salida, o de forma explícita: haciendo llegar información específica sobre la congestión al emisor responsable de ese tráfico, es decir, aplicando el marcado ECN.

Con AQM la congestión no se notifica cuando la cola desborda, sino que se utiliza un algoritmo como RED. Este algoritmo calcula la ocupación media de la cola en cierto período de tiempo. Si ese valor sobrepasa determinado umbral, considera que existe un riesgo significativo de congestión y optará por descartar un paquete de la cola o bien aplicar ECN. Es interesante destacar que incluso en el caso de descarte de paquetes, los efectos negativos no son tan graves como cuando ocurre porque la cola desborda, ya que se reduce mucho la probabilidad no deseada de retransmisiones sincronizadas entre varias conexiones.

AQM utiliza ECN únicamente si la carga útil del paquete es un ECT (ECN Capable Transport), es decir, un protocolo de transporte que soporta ECN. En caso contrario, descarta el paquete. ECN utiliza un campo del mismo nombre que hay en la cabecera del paquete IP[5]. Este campo de 2 bits sirve para que el router notifique CE, pero también para indicar a los routers que el paquete encapsula un ECT. Por concretar el significado de estos bits:

00: El protocolo de transporte no soporta ECN.
01: Es un ECT de tipo 1 (ECT 1), el modelo avanzado.
10: Es un ECT de tipo 0 (ECT 0), el modelo estándar, que usa TCP.
11: Un router ha detectado congestión (CE).

14.6.1. CoDel

RED necesita que el administrador fije de antemano los umbrales de ocupación de la cola que determinan cuando se empiezan a marcar o descartar. Esos umbrales dependen del tamaño de la cola, la velocidad del enlace u otros condicionantes, así que no hay valores fijos universales.

CoDel *(Controlled Delay)* resuelve el mismo problema que RED —evitar que la ocupación de la cola crezca sin control— pero mide algo distinto: en lugar de la ocupación de la cola, usa el tiempo que cada paquete lleva esperando en ella. Eso le permite autoajustarse a la velocidad del enlace

[5]Tanto en IPv4 como en la de IPv6

y a su ocupación, sin necesitar la configuración manual que requiere RED. Además, al fijarse en el tiempo mínimo de espera dentro de cada intervalo (y no en la media), CoDel distingue una cola que crece de forma pasajera para absorber una ráfaga —inevitable e inofensiva, porque se resuelve pronto— de una cola con alta ocupación persistente, que es la que realmente añade retardo al flujo.

CoDel se configura con dos parámetros, `target` e `interval`, los mismos con los que Linux configura `fq_codel` por defecto en cada interfaz de red. En `target` se establece el tiempo máximo que un paquete debería permanecer en la cola en condiciones normales (5 ms de forma predeterminada); `interval` es el plazo durante el que se tolera que ese tiempo de espera supere `target` antes de intervenir (100 ms por defecto). Si el tiempo mínimo de espera se mantiene por encima de `target` durante todo ese `interval`, CoDel empieza a descartar paquetes, o a marcarlos con ECN si está disponible, cada vez con más frecuencia mientras se mantenga la situación.

De este modo, CoDel ataja directamente el *bufferbloat* (§ 14.1): la alta ocupación de la cola no es un problema en sí mismo, siempre que los paquetes no pasen demasiado tiempo esperando en ella.

14.6.2. ECN en TCP

Un emisor TCP con soporte ECN reacciona a una notificación CE reduciendo su ventana de congestión a la mitad ($cwnd = cwnd/2$) [51], de forma similar a lo que ocurre con la detección de *3dupack*.

Sin embargo, para que el emisor TCP sea informado se necesita la colaboración del receptor. Para ello se utilizan 2 flags en la cabecera TCP:

ECE (ECN Echo) Lo envía el receptor (probablemente en un segmento de confirmación) para indicar al emisor la llegada de un paquete marcado con CE.

CWR (Congestion Window Reduced) Lo envía el emisor (probablemente en el siguiente segmento de datos) para indicar al receptor que ha recibido la notificación de congestión y ya está reduciendo su tasa de emisión. El receptor puede entonces dejar de marcar sus ACK con el bit ECE.

Para contar con la colaboración del receptor, es necesario que ambos extremos verifiquen que el otro también soporta ECN. Para ello, un cliente que soporta ECN debería activar los flags CWR y ECE en el segmento inicial junto al flag SYN. Si el servidor también soporta ECN, debe responder activando el flag ECE junto con los flags SYN y ACK, propios del proce-

so de conexión habitual. El uso de estos flags durante el establecimiento de conexión se llama «negociación ECN» y durante su ejecución no tienen el significado de notificación previamente descrito. Solo en el caso en que se haya dado esta negociación ECN con éxito, los participantes marcarán los paquetes IP con ECN, y reconocerán y activarán los flags ECE y CWR durante la conexión.

Esta es una secuencia de mensajes que aparecerían en una conexión que utiliza ECN, representada también en la Figura 14.9.

1. El cliente realiza la conexión enviando un segmento marcando los flags SYN, ECE y CWR.
2. El servidor responde con un segmento marcando los flags SYN, ACK y ECE. A partir de este momento, cliente y servidor marcarán los paquetes IP que envíen con el valor ECT(0)[6].
3. En algún momento, un router detecta congestión y elige un paquete que corresponde a esta conexión. Cambia el valor de su campo ECN a 11, es decir, lo marca como CE.
4. El paquete marcado con CE llega al receptor TCP.
5. En las siguientes confirmaciones, el receptor activa el flag ECE en la cabecera TCP para notificar al emisor.
6. El emisor TCP recibe un segmento con ECE, reduce su tasa de emisión y, en el siguiente segmento de datos, activa el flag CWR.
7. El receptor recibe el segmento marcado con CWR y deja de enviar segmentos marcados con ECE para que el emisor deje de aplicar la reducción de tasa.

14.6.3. ECN con otros protocolos

ECN no es para uso específico de TCP, si bien requiere que los protocolos de transporte proporcionen la funcionalidad necesaria para realizar la necesaria gestión de tasa de emisión por sus propios medios. Los protocolos de aplicación también pueden beneficiarse de ECN si están diseñados para ello. Por ejemplo, protocolos modernos como QUIC, que de hecho se encapsula en UDP, lo incorporan [52].

[6] En realidad, solo marcan los paquetes IP que contengan segmentos TCP con datos.

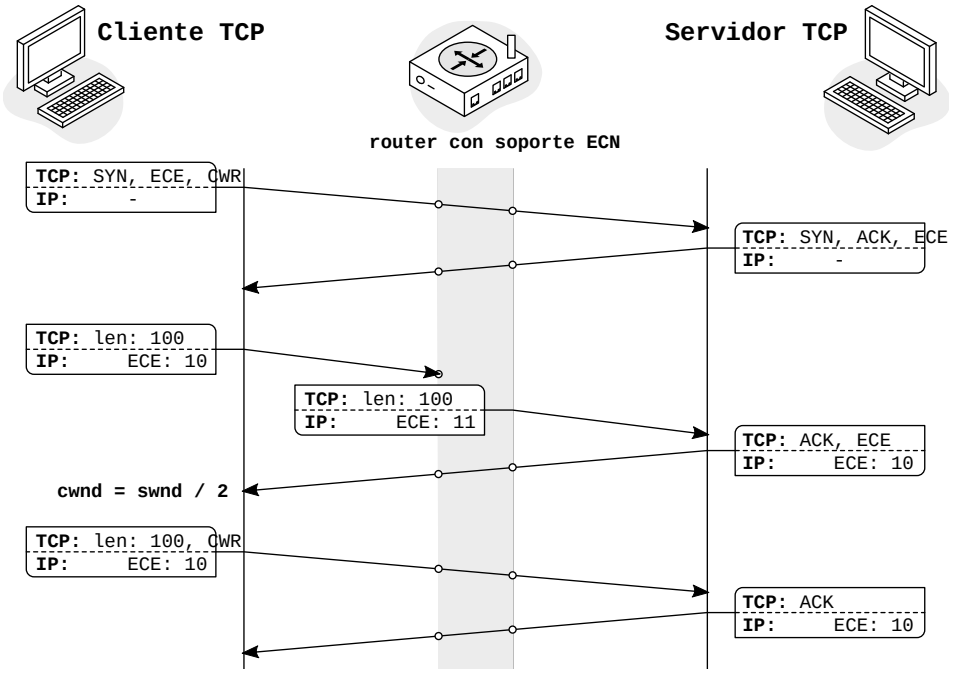


FIGURA 14.9: Conexión TCP con soporte ECN bajo un episodio de congestión

Y ¿qué más?

TCP es un protocolo complejo, lleno de detalles y algoritmos que han sido refinados y optimizados a lo largo de sus muchos años de vida. La experiencia empírica ha permitido a varias generaciones de ingenieros ir adaptando su funcionamiento para trabajar con aplicaciones y servicios que ni se concebían cuando se diseñó a finales de los 70. A pesar de ello, su funcionamiento básico sigue siendo esencialmente el mismo, y su desempeño es francamente robusto para la mayoría de los casos, algo que resulta especialmente meritorio si se piensa por un momento la tremenda evolución que ha experimentado hardware y software desde entonces.

Eso no significa que sea perfecto, ni mucho menos. Algunos de sus problemas más graves son: ausencia total de mecanismos de seguridad, bajo desempeño en comunicaciones móviles, rendimiento pobre en enlaces de alta latencia, el problema del «bloqueo de línea de cabecera» *(head-of-line blocking)*, etc.

Estos problemas y sus soluciones son temas avanzados que superan el alcance actual de este libro, pero pueden dar al lector una idea de cómo el estudio y desarrollo de nuevos protocolos de transporte sigue siendo un tema de investigación abierto aún hoy día.

Capítulo 15

Cliente-Servidor

Al terminar este capítulo, entenderás:

- Qué características tiene el modelo cliente-servidor.
- Cómo se crean servidores concurrentes multiproceso o multihilo
- Qué es el *preforking* y el *pool* de procesos e hilos.
- Qué consecuencias tienen las operaciones bloqueantes.
- Cómo utilizar los mecanismos de programación asíncrona para conseguir concurrencia eficiente.

El capítulo 6 ya presentó las ideas básicas del modelo cliente-servidor como forma de acercarnos a los sockets; este capítulo profundiza en el modelo y en sus posibilidades y limitaciones.

El modelo (o paradigma) cliente-servidor (Figura 15.1) es, sin ninguna duda, el enfoque más simple y habitual para crear aplicaciones distribuidas. La mayoría de los **protocolos de aplicación** clásicos de TCP/IP: HTTP, DNS, IMAP, SMTP, etc. están basados en este modelo, y a día de hoy, también la mayoría de aplicaciones que se desarrollan lo siguen usando; tal es el caso de la omnipresente arquitectura REST.

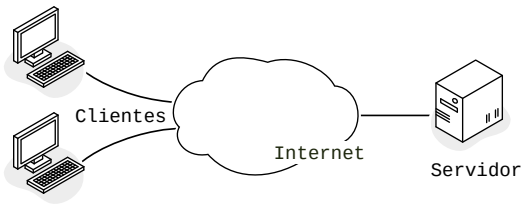


FIGURA 15.1: Modelo Cliente-Servidor

Una *aplicación distribuida* es una que está compuesta por varios componentes que se ejecutan en distintos nodos y colaboran entre sí mediante el intercambio de mensajes para realizar alguna tarea. En esa definición cabe

la gran mayoría de aplicaciones que utilicen la red para comunicarse, que a su vez son la mayoría de las que usamos hoy en día. Sí, casi todo el software que usamos a diario son aplicaciones distribuidas.

En este modelo cliente-servidor aparecen exactamente esos dos **roles** bien diferenciados que le dan el nombre:

- El **servidor** es la parte pasiva. Permanece inactiva a la espera de una petición para realizar una tarea o proporcionar un recurso a través de la red.
- El **cliente** es la parte activa. Envía una petición a un servidor para que éste realice la tarea especificada, o bien le proporcione acceso a un recurso remoto.

Que sean 'roles' es importante porque un mismo programa puede tomar uno, otro o ambos en distintos momentos o circunstancias, es decir, no definen tipos de aplicaciones. Es cierto que coloquialmente hablamos de 'servidores' y 'clientes' refiriéndonos a programas concretos: *p. ej.* el servidor web y el cliente de correo. Pero eso se debe simplemente a que uno de los roles destaca muy claramente sobre el otro, y ciertamente también es habitual encontrar programas que solo actúan como cliente.

También es común hablar de nodos como 'clientes' y, sobre todo, como 'servidores' para denotar que su actividad principal es la de alojar ese tipo de programas. Aunque la arquitectura hardware de un nodo de cómputo puede ser exactamente la misma independientemente de si ejecuta programas servidores o clientes, es fácil identificar algunas características diferenciadoras: el que llamamos 'servidor' suele disponer de una mejor conexión a la red, mayor ancho de banda, está montado en un *rack*, tiene fuente de alimentación redundante y no dispone de pantalla, teclado, ratón u otros periféricos habituales, ya que ninguna persona lo va a utilizar como PC de escritorio. Es hardware diseñado para funcionar ininterrumpidamente porque los programas servidores suelen estar pensados para ejecutarse de forma continua e indefinida.

Sin embargo, en un entorno de desarrollo las aplicaciones servidoras se ejecutan en los PC de los desarrolladores o bien en máquinas virtuales o contenedores. Incluso en un entorno doméstico es relativamente común que los usuarios ejecuten en sus máquinas servidores de diversa índole, aunque por supuesto es más habitual que ejecuten clientes.

El modelo cliente-servidor implica un patrón de comunicación característico conocido como **petición-respuesta**. En éste, el cliente realiza una petición —que suele incluir una operación y un identificador de recurso— y

el servidor devuelve una respuesta con un resultado o un valor de retorno, indicando si la operación se pudo realizar satisfactoriamente o se produjo un error. El formato de estos mensajes de petición y respuesta está especificado en un protocolo de aplicación. Por ejemplo, el protocolo HTTP encaja bastante bien en esta idea. Las operaciones son `GET`, `POST`, `PUT` y `DELETE`, etc., y los recursos se identifican con una URL. Por supuesto, existe una gran cantidad de protocolos, pero la mayoría siguen esta pauta. Los recursos que ofrece un servidor pueden ir desde una impresora hasta elementos de menor granularidad, como los registros de una base de datos o los archivos de un directorio. Sin embargo, en este capítulo no vamos a hablar de protocolos de aplicación.

En su lugar, vamos a hablar de productividad y rendimiento. En la mayor parte de las situaciones, el modelo cliente-servidor implica un servidor atendiendo a múltiples clientes simultáneamente. Para lograr eso de una forma razonable, se requiere algún mecanismo de ejecución concurrente, ya sean procesos, hilos, o E/S asíncrona. Todo eso es lo que vamos a tratar en este capítulo.

15.1. Upper

Para concretar las técnicas que vamos a estudiar, utilizaremos un servicio muy básico: un conversor a mayúsculas, es decir, el servidor devuelve una versión en mayúsculas de la cadena de texto que el cliente le envía. Es equivalente al típico servicio *echo*, pero en lugar de simplemente devolver la misma cadena que se le envía, ese pequeño cambio de pasar a mayúsculas que introduce *upper* demuestra que el servidor está haciendo trabajo, por simple que sea. El funcionamiento del cliente es trivial: obtiene cadenas desde la consola, las envía al servidor, recibe las respuestas y las imprime en pantalla.

Para explorar las distintas opciones vamos a empezar por TCP, ya que al estar orientado a conexión, da más juego a la hora de evaluar su rendimiento. Después se explica qué puede hacer UDP y qué diferencias prácticas existen.

15.2. Servidor `TCP`

En esta primera versión (Listado 15.1) tenemos un bucle al final del listado que acepta conexiones y las delega a una función `handle()` que se encarga de la comunicación con el cliente hasta su desconexión. Este esquema se llama patrón *acceptor* y es el mismo del Listado 6.6, que se repite en todos los ejemplos del capítulo.

```python
import sys
import time
import socket

def upper(msg):
    time.sleep(1)  # simulates a complex job
    return msg.upper()

def handle(sock, client):
    print(f"Client connected: {client}")
    while 1:
        data = sock.recv(32)
        if not data:
            break

        sock.sendall(upper(data))

    sock.close()
    print(f"Client disconnected: {client}")

def main(port):
    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    sock.bind(('', port))
    sock.listen(5)

    while 1:
        conn, client = sock.accept()
        handle(conn, client)

if len(sys.argv) != 2:
    print("Usage: {0} <port>".format(sys.argv[0]))
    sys.exit(1)

try:
    main(int(sys.argv[1]))
except KeyboardInterrupt:
    print("shut down")
```

LISTADO 15.1: Servidor TCP
/upper/tcp_server.py

En un servidor monoproceso y monohilo (como este) existen dos puntos de bloqueo: esperar un nuevo cliente —en `accept()`—, o esperar un nuevo mensaje del cliente conectado —en `recv()`—, y el proceso solo puede estar en uno de ellos. Por supuesto, eso también implica que solo puede haber un cliente conectado; hasta que no acabe el bucle de la función `handle()`, no podrá ejecutar la siguiente iteración del bucle de la función `main()`.

Se le llama servidor *iterativo*, porque *itera* atendiendo cliente tras cliente, propiciando una *serie* de conexiones. Dicho de otro modo: un cliente no será atendido hasta que el servidor termine con el anterior. Aquí 'iterativo' es por definición lo contrario a 'concurrente'.

Un detalle relevante que hay que comentar es la forma en la que se hace la conversión a mayúsculas mediante la función `upper()`. Aparte de llamar al método `bytes.upper()`, que es la que realmente hace el trabajo, invoca `time.sleep(1)` para provocar una pausa artificial de 1 segundo. El motivo es simular que pasar a mayúsculas es una tarea compleja, para así poder apreciar más fácilmente el efecto de las técnicas de concurrencia que veremos después y la mejora que suponen.

Otro detalle que te puede llamar la atención es la llamada a `setsockopt()`. Como su nombre indica sirva para modificar el valor de una *opción* del socket. Estas opciones sirven para modificar ciertos comportamientos del socket. En concreto esta (`SO_REUSEADDR`) permite arrancar un servidor en un puerto que había sido utilizado recientemente. Esto es solo una conveniencia para los ejemplos y no es recomendable en producción. Veremos este asunto con detalle más adelante en § 13.3.1.

15.3. Cliente TCP

El cliente, después de conectar con el servidor, ejecuta un bucle que lee una línea de texto desde consola —llamada `sys.stdin.readline()`—, la envía al servidor y espera la respuesta.

El tratamiento de la E/S parcial en este caso es simple. Los mensajes que se mueven entre cliente y servidor no necesitan límites definidos, no importa si se trocean o dónde, al final todo el texto llegará al servidor y toda la secuencia de respuestas llegará de vuelta al cliente. En todo caso, para que el funcionamiento resulte más intuitivo en cualquier circunstancia, el cliente se asegura de que recibe una respuesta del mismo tamaño que el último mensaje enviado; en el bucle `while len(msg) < sent` al final de la función `main()`.

```
import sys
from socket import socket

def main(host, port):
    with socket() as sock:
        sock.connect((host, port))

        while 1:
            data = sys.stdin.readline().strip().encode()
            if not data:
                break

            sock.sendall(data)

            msg = bytes()
            while len(msg) < len(data):
                msg += sock.recv(32)
```

```
            print("Reply is '{0}'".format(msg.decode()))

if len(sys.argv) != 3:
    print("Usage: {0} <host> <port>".format(sys.argv[0]))
    sys.exit(1)

try:
    main(sys.argv[1], int(sys.argv[2]))
except KeyboardInterrupt:
    print("shut down")
```

LISTADO 15.2: Cliente TCP
/upper/tcp_client.py

Para probar el programa simplemente ejecuta servidor y cliente en consolas distintas tal como aparece a continuación. Para terminar la conexión simplemente cierra la entrada estándar del cliente, lo que puedes conseguir pulsando `Ctrl+D`.

Cliente

```
$ ./tcp_client.py
hola
Reply is 'HOLA'
```

Servidor

```
$ ./tcp_server.py 2000
Client connected: ('127.0.0.1', 43172)
Client disconnected: ('127.0.0.1', 43172)
```

FIGURA 15.2: Servidor y cliente upper TCP

Un pequeño detalle al que debes prestar atención es la codificación de los mensajes. La función `readline()` devuelve una cadena de caracteres (`str`) y del mismo modo la función `print()` acepta una cadena. Sin embargo, los métodos `send()` y `recv()` del socket solo aceptan y devuelven secuencias de tipo `bytes`. Por eso es necesario convertir entre estos tipos con los métodos `encode()` y `decode()` respectivamente. Curiosamente el servidor no ha tenido que hacer estas conversiones porque también existe un método `bytes.upper()`, que es el que está usando la función `upper()` del Listado 15.1. Veremos más sobre esta cuestión en el capítulo 18.

Mientras tienes un cliente conectado, puedes ejecutar un nuevo cliente en otra consola, para así comprobar que efectivamente el servidor no acepta esta segunda conexión hasta que la primera termine.

Por cierto, este cliente es tan simple que de hecho lo puedes sustituir directamente con `ncat`, y el funcionamiento es idéntico.

15.4. Servidor TCP multihilo

Un enfoque muy habitual para permitir que el servidor atienda múltiples clientes es crear un hilo (una instancia de la clase `threading.Thread`) en

el que ejecutar la función `handle()` para cada nueva conexión. El código completo de este servidor multihilo *(threading server)* es muy similar al servidor iterativo (Listado 15.3).

```
import sys
import time
import socket
from threading import Thread

def upper(msg):
    time.sleep(1)
    return msg.upper()

def handle(sock, client, n):
    print(f"Client {n:>3} connected: {client}")
    while True:
        data = sock.recv(32)
        if not data:
            break
        sock.sendall(upper(data))

    sock.close()
    print(f"Client {n:>3} disconnected: {client}")

def main(port):
    sock = socket.socket()
    sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    sock.bind(('', port))
    sock.listen(30)

    n = 0
    while True:
        conn, client = sock.accept()
        t = Thread(
            target=handle, args=(conn, client, n := n+1), daemon=True)
        t.start()

if len(sys.argv) != 2:
    print("Usage: {0} <port>".format(sys.argv[0]))
    sys.exit(1)

try:
    main(int(sys.argv[1]))
except KeyboardInterrupt:
    print("shut down")
```

LISTADO 15.3: Servidor TCP multihilo
/upper/tcp_thread.py

El programa no limita el número de hilos, aunque por supuesto el SO sí que lo hará. Si quieres detectar un exceso de hilos debes capturar la excepción `RuntimeError`. Este límite tiene que ver principalmente con la cantidad de memoria que el SO asigna a cada hilo. Los hilos son muy ligeros porque todos ellos comparten el mismo espacio de direcciones (la misma memoria) que el proceso original y solo necesitan memoria adicional para la 'traza

de pila' *(call stack)*. La traza de pila almacena las llamadas a las funciones con sus argumentos, es decir, supone poca memoria, con lo que es posible crear varios miles de hilos sin problema.

Crear y destruir hilos es rápido, de modo que si la tarea que realiza el servidor es sencilla y corta, el servidor multihilo es bastante eficiente.



Python tiene una limitación técnica importante debida al GIL, que impide que varios hilos puedan ejecutar código Python al mismo tiempo, es decir, impide el paralelismo. Esto es un grave problema para programas intensivos en CPU, pero no lo es en absoluto para programas intensivos en E/S, que es precisamente lo que son los servidores.

15.5. Forzando los límites

Para dejar patente la mejora de rendimiento que supone el servidor multihilo, se proporciona un programa llamado `/upper/tcp_stress_client.py` que crea la cantidad de clientes que se le solicita y trata de conectarlos al servidor.

En paralelo, cada uno de esos clientes envía 8 palabras. En concreto cada uno envía *twenty tiny tigers take two taxis to town* en 8 mensajes distintos y al terminar, desconecta. Con el siguiente comando puedes probar el cliente de *stress* con el servidor iterativo. Verás que aparecen las 8 respuestas a los mensajes del 'cliente 0' (están identificados con un índice entre corchetes) antes de empezar a ver los mensajes del 'cliente 1'.

Cliente

```
$ ./tcp_stress_client.py 127.0.0.1 2000 10
- [  0] Reply: TWENTY
- [  0] Reply: TINY
...
- [  0] Reply: TO
- [  0] Reply: TOWN
- [  1] Reply: TWENTY
- [  1] Reply: TINY
- [  1] Reply: TIGERS
...
```

Servidor

```
$ ./tcp_server.py 2000
Client connected: ('127.0.0.1', 47900)
Client disconnected: ('127.0.0.1', 47900)
Client connected: ('127.0.0.1', 42700)
Client disconnected: ('127.0.0.1', 42700)
Client connected: ('127.0.0.1', 42714)
Client disconnected: ('127.0.0.1', 42714)
Client connected: ('127.0.0.1', 42724)
Client disconnected: ('127.0.0.1', 42724)
Client connected: ('127.0.0.1', 42738)
```

FIGURA 15.3: Servidor y cliente upper TCP

Para que los clientes no esperen indefinidamente, el programa fija un *timeout* para la conexión, que de expirar provoca que el cliente desista. Lo verás si lo ejecutas con demasiados clientes.

Si los 10 clientes consiguen conectarse, la ejecución completa necesitará unos 80 segundos (10 clientes × 8 mensajes × 1 segundo por mensaje).

Para ver la diferencia con el nuevo servidor, que es la intención, ejecuta ahora el mismo comando con el servidor multihilo de esta sección. En este caso la ejecución se completará en algo más de 8 segundos. Mientras el servidor sea capaz de crear tantos hilos como clientes se conecten a la vez, el tiempo total será aproximadamente esos mismos 8 segundos.

Te recomendamos que pruebes con distintas cantidades de clientes y observes el comportamiento del servidor. Llévalo al límite, prueba con cantidades de clientes cada vez mayores y trata de entender lo que ocurre en cada caso. Y por supuesto, no olvides probarlo también con los servidores que veremos en las siguientes secciones.

15.6. Servidor TCP multiproceso

El servidor multiproceso *(forking server)* es la alternativa más frecuente al multihilo. El código es más complejo por la necesaria gestión de procesos. El programa en sí también es más costoso que el multihilo. Los procesos son mucho más pesados que los hilos. Implican una copia completa de la memoria del proceso padre. Por otro lado, los procesos son independientes unos de otros, de modo que los errores de uno no afectan a los demás y no hay ningún GIL que afecte a las tareas intensivas en CPU.

Respecto al código, la diferencia clave es que la función `handle()` se ejecuta ahora dentro de un nuevo proceso hijo. Aparte de eso, las funciones `main()`, `handle()` y `upper()` son prácticamente las mismas.

```
import sys
import os
import time
import socket

class ProcessThrottler(object):
    def __init__(self, max_procs=40):
        self.max_procs = max_procs
        self.procs = []

    def collect_children(self):
        # from socketserver module
        while self.procs:
            opts = os.WNOHANG if len(self.procs) < self.max_procs else 0
            pid, status = os.waitpid(0, opts)
            if not pid:
                break

            self.procs.remove(pid)

    def start_new_process(self, func, args):
        self.collect_children()
        pid = os.fork()
        if pid:
```

```
            self.procs.append(pid)
        else:
            func(*args)
            sys.exit()

def upper(msg):
    time.sleep(1)  # simulates a complex job
    return msg.upper()

def handle(sock, client, n):
    print(f"Client {n:>3} connected: {client}")
    while 1:
        data = sock.recv(32)
        if not data:
            break
        sock.sendall(upper(data))

    sock.close()
    print(f"Client {n:>3} disconnected: {client}")

def main(port):
    sock = socket.socket()
    sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    sock.bind(('', port))
    sock.listen(5)

    throttler = ProcessThrottler()
    n = 0

    while 1:
        conn, client = sock.accept()
        throttler.start_new_process(
            handle, (conn, client, n := n+1))

if len(sys.argv) != 2:
    print("Usage: {0} <port>".format(sys.argv[0]))
    sys.exit(1)

try:
    main(int(sys.argv[1]))
except KeyboardInterrupt:
    print("shut down")
```

LISTADO 15.4: Servidor TCP multiproceso
/upper/tcp_fork.py

La creación de los procesos recae exclusivamente en la clase `ProcessThrottler`. La clase guarda una lista de los PID de los procesos que va creando (`procs`). El método `start_new_process()` es el que ejecuta la llamada `os.fork()` e invoca la función indicada como argumento. El método `collect_children()`[1] se encarga de limitar la cantidad de procesos que se crearán, invocando `os.waitpid()` de forma bloqueante si se alcanza el máximo, evitando así que se cree un nuevo proceso hasta que no haya terminado

[1] Este código pertenece al módulo `socketserver` de la librería estándar.

uno de los existentes. Ese máximo se indica en el constructor con el parámetro `max_procs` (por defecto: 40).

Python dispone de algunas abstracciones de más alto nivel para la creación y gestión de procesos. Por ejemplo, la clase `multiprocessing.Process` ofrece un API similar a la de `threading.Thread` para manejar un único proceso, mientras que `multiprocessing.Pool` y `ProcessPoolExecutor` del módulo `concurrent.futures` permiten crear un *pool*[2] de procesos en los que se puede ejecutar la función proporcionada.

El Listado 15.5 muestra una versión del servidor multiproceso que utiliza la clase `multiprocessing.Pool`. Sin embargo, este programa funciona de un modo diferente. A diferencia del servidor anterior (Listado 15.4) que crea y destruye un proceso por cada conexión, en este los procesos existen desde que se instancia la clase `Pool` y se reutilizan una y otra vez para las nuevas conexiones. Por eso a esta técnica se le llama *preforking* y la ventaja es evidente: la aplicación se ahorra la sobrecarga de crear y destruir procesos continuamente, y puede ser una diferencia significativa si el servidor recibe muchas peticiones que requieren poco procesamiento. También tiene una desventaja: el consumo de memoria es el mismo aunque el servidor esté completamente ocioso.

```
import sys
import time
import socket
from multiprocessing import Pool

MAX_PROCS = 10

def upper(msg):
    time.sleep(1)  # simulates a complex job
    return msg.upper()

def handle(sock, client):
    print(f"Client connected: {client}")
    while 1:
        data = sock.recv(32)
        if not data:
            break
        sock.sendall(upper(data))

    sock.close()
    print(f"Client disconnected: {client}")

def main(port):
    sock = socket.socket()
    sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    sock.bind(('', port))
```

[2] No conozco ninguna traducción razonable para esto y me niego a llamarlo *piscina de procesos*, lo siento.

```
    sock.listen(5)

    with Pool(MAX_PROCS) as pool:
        while 1:
            conn, client = sock.accept()
            pool.apply_async(handle, (conn, client))

if len(sys.argv) != 2:
    print("Usage: {0} <port>".format(sys.argv[0]))
    sys.exit(1)

try:
    main(int(sys.argv[1]))
except KeyboardInterrupt:
    print("shut down")
```

LISTADO 15.5: Servidor TCP con *preforking*
/upper/tcp_prefork_pool.py

Hay otro modo más agresivo de implementar *preforking* con TCP que consiste en compartir el *master socket* entre los procesos hijos y todos ellos invocan `accept()`. Eso es posible cuando el SO ofrece soporte específico, en el caso de Linux (Listado 15.6).

```
import sys
import os
import time
import socket
from multiprocessing import Pool

MAX_PROCS = 10

def upper(msg):
    time.sleep(1)  # simulates a complex job
    return msg.upper()

def handle(sock, client):
    print(f"Client connected: {client}, PID: {os.getpid()}")
    while 1:
        data = sock.recv(32)
        if not data:
            break
        sock.sendall(upper(data))

    sock.close()
    print(f"Client disconnected: {client}")

def worker(sock):
    try:
        while 1:
            conn, client = sock.accept()
            handle(conn, client)
    except KeyboardInterrupt:
        sys.exit(0)

def main(port):
```

```
    sock = socket.socket()
    sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    sock.bind(('', port))
    sock.listen(5)

    with Pool(MAX_PROCS) as pool:
        pool.map(worker, [sock] * MAX_PROCS)

if len(sys.argv) != 2:
    print("Usage: {0} <port>".format(sys.argv[0]))
    sys.exit(1)

try:
    main(int(sys.argv[1]))
except KeyboardInterrupt:
    print("shut down")
```

LISTADO 15.6: Servidor TCP con *preforking* para el método `accept()`
/upper/tcp_prefork_pool_accept.py

15.7. Servidor UDP

Es momento de ver las variantes UDP del servicio *upper*. El Listado 15.7 es una implementación funcional y correcta del servidor salvo porque, como de costumbre en el libro, obviamos el tratamiento de errores en favor de la legibilidad[3]. Hay unas cuantas diferencias evidentes y alguna no tan evidente que vale la pena destacar respecto al servidor TCP.

En la función `main()` hay un bucle que recibe mensajes, e imitando la estructura del servidor TCP, invoca la función `handle()` con cada mensaje[4] y el endpoint del cliente que lo ha enviado. La función `handle()` hace el trabajo y envía la respuesta. No hay un bucle en esa función, lo que deja patente que este es un servidor orientado a mensajes, no a conexiones.

Recuerda que con UDP la E/S parcial no es un problema. Si no hay pérdida de datos[5], cada llamada a `recvfrom()` devuelve un mensaje completo y cada llamada a `sendto()` envía el mensaje completo.

```
import sys
import time
import socket

def upper(msg):
    time.sleep(1)  # simulates a complex job
    return msg.upper()

def handle(sock, msg, client, n):
```

[3]No olvides tenerlo en cuenta si escribes código para producción.
[4]No con cada conexión, que no hay.
[5]Y recuerda que eso es factible con UDP.

```
    print(f"New request: {n} {client}")
    sock.sendto(upper(msg), client)

def main(port):
    sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    sock.bind(('', port))

    n = 0
    while 1:
        msg, client = sock.recvfrom(1024)
        handle(sock, msg, client, n := n+1)

if len(sys.argv) != 2:
    print("Usage: {0} <port>".format(sys.argv[0]))
    sys.exit(1)

try:
    main(int(sys.argv[1]))
except KeyboardInterrupt:
    print("shut down")
```

LISTADO 15.7: Servidor UDP
/upper/udp_server.py

El Listado 15.8 muestra el cliente UDP. Como en TCP, puedes sustituirlo por `ncat` si quieres.

```
import sys
from socket import socket, SOCK_DGRAM

def main(host, port):
    with socket(type=SOCK_DGRAM) as sock:
        server_endpoint = (host, port)

        while 1:
            data = sys.stdin.readline().strip().encode()
            if not data:
                break

            sock.sendto(data, server_endpoint)
            msg, _ = sock.recvfrom(1024)
            print("Reply is '{}'".format(msg.decode()))

if len(sys.argv) != 3:
    print("Usage: {0} <host> <port>".format(sys.argv[0]))
    sys.exit(1)

try:
    main(sys.argv[1], int(sys.argv[2]))
except KeyboardInterrupt:
    print("shut down")
```

LISTADO 15.8: Cliente UDP
/upper/udp_client.py

También tenemos /upper/udp_stress_client.py, el cliente de *stress* para UDP. Pruébalo con este servidor y con las variantes de las siguientes secciones para ver las diferencias. Así se comporta con el servidor UDP básico:

Cliente

```
$ ./udp_stress_client.py 127.0.0.1 2000 20
- [  0] Reply: TWENTY
- [  1] Reply: TWENTY
- [  2] Reply: TWENTY
- [  3] Reply: TWENTY
- [  4] Reply: TWENTY
- [  5] Reply: TWENTY
- [  6] Reply: TWENTY
```

Servidor

```
$ ./udp_server.py 2000
New request: 1 ('127.0.0.1', 40923)
New request: 2 ('127.0.0.1', 48631)
New request: 3 ('127.0.0.1', 40484)
New request: 4 ('127.0.0.1', 55612)
New request: 5 ('127.0.0.1', 51594)
New request: 6 ('127.0.0.1', 55917)
New request: 7 ('127.0.0.1', 55545)
```

Figura 15.4: Servidor y cliente upper TCP

A la vista de esta salida se puede apreciar esa diferencia no tan evidente. Resulta que este no es un servidor iterativo, al menos no en el sentido en el que lo es el TCP. Cada llamada a `recvfrom()` puede recoger un mensaje de un cliente diferente, lo que eventualmente permite progresar a todos los clientes que estén enviando solicitudes. Por eso, este es un servidor **concurrente** (ver Figura 15.5). Es importante recordar aquí la diferencia entre *concurrente* y *paralelo*. Este servidor no es paralelo, no puede ejecutar la función `handle()` exactamente al mismo tiempo para varios clientes, pero eso no es necesario para considerarlo concurrente. La tabla 15.1 resume este aspecto para los servidores anteriores. Es una consecuencia directa del diferente modo de trabajar de TCP y UDP.

Servidor	**Concurrente**	**Paralelo**
TCP iterativo	No	No
TCP multihilo	Sí	Sí
TCP multiproceso	Sí	Sí
UDP básico	Sí	No

Cuadro 15.1: Concurrencia y paralelismo de los servidores TCP y UDP

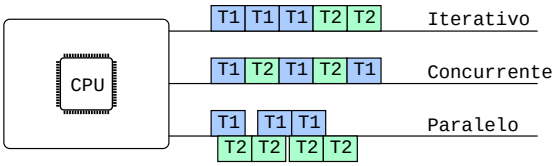


FIGURA 15.5: Comparación entre procesamiento iterativo, concurrente y paralelo

15.8. Servidor UDP multiproceso

Por supuesto es posible crear un servidor UDP que cree un proceso por cada mensaje recibido consiguiendo así un procesamiento paralelo. El Listado 15.9 muestra una posible implementación, en este caso utilizando la clase `multiprocessing.Process` en lugar de `fork()` directamente.

```
import sys
import time
import socket
import multiprocessing as mp

MAX_PROCS = 10

def start_new_process(func, args):
    for p in mp.active_children()[::-1][MAX_PROCS:]:
        p.join()

    ps = mp.Process(target=func, args=args)
    ps.start()

def upper(msg):
    time.sleep(1)  # simulates a complex job
    return msg.upper()

def handle(sock, msg, client, n):
    print(f"New request: {n} {client}")
    sock.sendto(upper(msg), client)
    sys.exit(0)

def main(port):
    sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    sock.bind(('', port))

    n = 0
    while 1:
        msg, client = sock.recvfrom(1024)
        start_new_process(handle, (sock, msg, client, n := n + 1))

if len(sys.argv) != 2:
    print("Usage: {0} <port>".format(sys.argv[0]))
    sys.exit(1)

try:
```

```
    main(int(sys.argv[1]))
except KeyboardInterrupt:
    print("shut down")
```

LISTADO 15.9: Servidor UDP multiproceso
/upper/udp_process.py

Sin embargo, si la tarea que realiza en cada petición es simple, la sobrecarga relativa por la creación y destrucción de procesos es mucho más grave que en el caso de TCP. Aquí se está creando un proceso por ¡cada mensaje!, no por cada conexión. Lo que sí puede ser razonable es utilizar la técnica de *preforking*, que evita completamente esa sobrecarga (Listado 15.10).

```
import sys
import time
import socket
from multiprocessing import Pool

MAX_PROCS = 20

def upper(msg):
    time.sleep(1)  # simulates a complex job
    return msg.upper()

def handle(sock, msg, client, n):
    print(f"New request: {n} {client}")
    sock.sendto(upper(msg), client)

def main(port):
    sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    sock.bind(('', port))

    with Pool(MAX_PROCS) as pool:
        n = 0
        while 1:
            msg, client = sock.recvfrom(1024)
            pool.apply_async(handle, (sock, msg, client, n := n+1))

if len(sys.argv) != 2:
    print("Usage: {0} <port>".format(sys.argv[0]))
    sys.exit(1)

try:
    main(int(sys.argv[1]))
except KeyboardInterrupt:
    print("shut down")
```

LISTADO 15.10: Servidor UDP con *preforking*
/upper/udp_prefork.py

15.9. Servidor UDP multihilo

Implementar un servidor UDP creando un hilo cada vez que llega un mensaje para ejecutar en él la función `handle()` no es una buena idea. La sobrecarga

de crear hilos no es despreciable, aunque no tan grave ni de lejos como con procesos. Pero el mayor problema no es ese. El problema es que en UDP todos los hilos tienen acceso al mismo socket, a diferencia del servidor TCP multihilo, en el que cada hilo opera su propio socket. Compartir el socket provoca condiciones de carrera, bloqueos, etc. Por eso, un servidor UDP multihilo de este tipo no es nada recomendable.

Sin embargo, es posible aplicar un enfoque distinto. Utilizar el paralelismo que proporcionan los hilos solo a la tarea del servidor, sin involucrar a los sockets. La idea es tener un hilo para recibir los peticiones (el principal), otro para enviar las respuestas *(responder)* y un pool para el procesamiento de los mensajes. Los mensajes recibidos se pasan como argumento a la función `handle()`, que se ejecuta en el pool. Las respuestas se envían al hilo *responder* por medio de una cola (`queue.Queue`). De ese modo, no hay ningún socket compartido, tenemos las ventajas, pero no los inconvenientes (Listado 15.11).

```
import sys
import time
import queue
from socket import socket, SOCK_DGRAM
from threading import Thread, current_thread
from concurrent.futures import ThreadPoolExecutor

MAX_THREADS = 20
output_queue = queue.Queue()

def upper(msg):
    time.sleep(1)  # Simulate heavy processing
    return msg.upper()

def handle(msg, client, n):
    print(f"Processing request {n} from {client}, thread {current_thread().name}")
    response = upper(msg)
    output_queue.put((response, client))

def responder(sock):
    while 1:
        response, client = output_queue.get()
        sock.sendto(response, client)

def main(port):
    sock = socket(type=SOCK_DGRAM)
    sock.bind(('', port))

    Thread(target=responder, args=(sock,), daemon=True).start()

    with ThreadPoolExecutor(max_workers=MAX_THREADS) as executor:
        n = 0
        while 1:
            msg, client = sock.recvfrom(1024)
            executor.submit(handle, msg, client, n := n+1)
```

```
if __name__ == '__main__':
    if len(sys.argv) != 2:
        print("Usage: {0} <port>".format(sys.argv[0]))
        sys.exit(1)

    try:
        main(int(sys.argv[1]))
    except KeyboardInterrupt:
        print("shut down")
```

LISTADO 15.11: Servidor UDP con pool de hilos
/upper/udp_threadpool.py

15.10. socketserver

El módulo `socketserver` de la librería estándar de Python ofrece clases y *mixins* para implementar servidores TCP y UDP de un modo muy sencillo. El Listado 15.12 es la versión equivalente al servidor TCP iterativo.

```
import sys
import os
import time
from socketserver import StreamRequestHandler, TCPServer

def upper(msg):
    time.sleep(1)  # simulates a complex job
    return msg.upper()

class Handler(StreamRequestHandler):
    def handle(self):
        print(f"Client connected: {self.client_address}")
        while 1:
            data = os.read(self.rfile.fileno(), 32)
            if not data:
                break

            self.wfile.write(upper(data))
        print(f"Client disconnected: {self.client_address}")

class CustomTCPServer(TCPServer):
    allow_reuse_address = True

if len(sys.argv) != 2:
    print("Usage: {0} <port>".format(sys.argv[0]))
    sys.exit(1)

server = CustomTCPServer(('', int(sys.argv[1])), Handler)
server.serve_forever()
```

LISTADO 15.12: Servidor TCP iterativo con `socketserver`
/upper/tcp_ss.py

La clase `TCPServer` permite crear un servidor TCP iterativo. Solo hay que pasarle como argumento el endpoint donde debe escuchar y una clase que herede de `StreamHandlerRequest` que proporcione a su vez un método `handle()` que es esencialmente equivalente a la función del mismo nombre de los ejemplos anteriores. Una diferencia muy evidente es que esa función no invoca `send()` y `recv()` sobre el socket conectado. En su lugar, invoca `read()` y `write()` sobre los *file objects* llamados `rfile` y `wfile`, pero que obviamente se refieren al socket.

Otra peculiaridad es que no se usa directamente la clase `TCPServer`. El programa define una clase `CustomTCPServer` que hereda de ella para redefinir el atributo `allow_reuse_address` como `True`. Eso es equivalente a la activación de la opción `SO_REUSEADDR`.

El módulo `socketserver` proporciona las clases `ForkingTCPServer` y `ThreadingTCPServer` alternativas a `TCPServer` para implementar directamente servidores multiproceso y multihilo, respectivamente. Y efectivamente, basta con cambiar la clase como puedes comprobar en /`upper/tcp_ss_thread.py`, que sería equivalente al Listado 15.3 y /`upper/tcp_ss_fork.py` que sería equivalente al Listado 15.4.

También proporciona la clase `UDPServer` (en uso en el Listado 15.13), equivalente al servidor del Listado 15.7. Aquí tiene como manejador una clase que hereda de `DatagramRequestHandler` y especializa el método `handle()` que recibe una petición y envía la respuesta.

```
import sys
import time
from socketserver import DatagramRequestHandler, UDPServer

def upper(msg):
    time.sleep(1)  # simulates a complex job
    return msg.upper()

class Handler(DatagramRequestHandler):
    def handle(self):
        print(f"New request: {self.client_address}")
        msg = self.rfile.read()
        self.wfile.write(upper(msg))

if len(sys.argv) != 2:
    print("Usage: {0} <port>".format(sys.argv[0]))
    sys.exit(1)

server = UDPServer(('', int(sys.argv[1])), Handler)
server.serve_forever()
```

LISTADO 15.13: Servidor UDP con `socketserver`
/`upper/udp_ss.py`

También existen las clases `ForkingUDPServer` y `ThreadingUDPServer` para implementar servidores multiproceso y multihilo, respectivamente, pero que tienen los inconvenientes ya expuestos.

El módulo no tiene soporte para *preforking* en ninguna de las dos modalidades y tampoco son sencillas de implementar porque pasar las instancias de las clases involucradas a los subprocesos es complejo.

15.11. Operaciones bloqueantes

En § 6.8 vimos algunas de las consecuencias de que los sockets al igual que los archivos requieran de operaciones de E/S. Pero hay otra consecuencia muy importante que no habíamos abordado aún: Estas operaciones son **bloqueantes**. Igual que leer de la consola esperando que el usuario pulse una tecla puede conllevar que el proceso quede bloqueado indefinidamente, lo mismo ocurre con el método `recv()`. En estos casos, el SO bloquea el proceso que invoca estas funciones hasta que lleguen los datos esperados. Pero no es algo que afecte solo a `recv()`; también afecta a `accept()` y por supuesto a `recvfrom()`, pues también en esos casos el proceso espera a la llegada de mensajes procedentes de la red para continuar su ejecución.

Es bastante intuitivo pensar que estas operaciones de «lectura» sean bloqueantes, pero resulta que también lo son las operaciones de «escritura» como `send()`, `sendto()` o `connect()`. Aunque es menos habitual que el envío de un mensaje conlleve el bloqueo de forma indefinida[6], también son operaciones de E/S y por eso el SO puede bloquear el proceso a la espera de que la operación se complete; en este caso, que los datos sean enviados, o almacenados en el buffer correspondiente, antes de continuar la ejecución del programa.

Que estas operaciones sean bloqueantes puede sonar como un problema que hay que resolver, pero es lógico que sea así e incluso resulta conveniente. Por norma general es **más sencillo** escribir —y más eficiente ejecutar— un programa de comunicaciones, o en general intensivo en E/S, si se comporta de este modo.

15.12. Alternativas a la E/S bloqueante

Pese a lo dicho, hay algunas situaciones en las que el programa podría necesitar aprovechar los tiempos «muertos» mientras el SO espera a que estas operaciones terminen, o quizá quiera ejecutar otras operaciones de E/S. Recuerda que cuando un proceso está bloqueado, la CPU no está ociosa; el

[6] Volveremos sobre esto cuando hablemos de control de flujo en el capítulo 12

planificador del SO pone otro proceso en ejecución, pero ciertamente, tu aplicación está parada.

Normalmente hay tres opciones para aprovechar los bloqueos: operaciones no bloqueantes, timeouts y programación asíncrona. Aunque esto es potencialmente aplicable a cualquier operación de E/S, lógicamente aquí vamos a hablar de sockets. Veámoslas brevemente:

Operaciones no bloqueantes

Los sockets están por defecto en modo bloqueante. Se puede conseguir que un socket concreto tenga un comportamiento no bloqueante invocando su método `setblocking(False)`. A partir de ese momento, si por ejemplo se invoca `recv()` y no hay datos disponibles, eleva inmediatamente una excepción `BlockingIOError` y es responsabilidad del programador capturar la excepción e intentar la recepción de nuevo más tarde si así lo estima oportuno. No es difícil imaginar cómo esto nos puede complicar rápidamente las cosas... Además es necesario señalar que el comportamiento de los sockets no bloqueantes puede diferir entre plataformas, algo que dificulta la escritura de código portable, otra buena razón para evitarlos si no está realmente justificado.

Timeouts

Hay otras situaciones en las que las operaciones bloqueantes son adecuadas, pero no es aceptable que el proceso quede bloqueado indefinidamente. En esta situación se puede usar un timeout, es decir, definir un tiempo máximo que de alcanzarse provoca la interrupción de la operación y eleva una excepción `TimeoutError` que avisa de la situación devolviendo así el control al programa. El método `socket.settimeout()` permite fijar ese límite (expresado en segundos).

Este método acepta dos valores especiales. Si se llama con `None`, elimina el timeout y el socket queda en modo bloqueante, pero si se llama con `0`, el socket cambia a modo no bloqueante, es decir, es como invocar `setblocking(False)`.

Entrada/salida asíncrona

El SO ofrece algunas llamadas al sistema para gestionar operaciones de E/S de forma asíncrona. La más básica y rudimentaria, que veremos a continuación, es `select()`, que permite esperar a que uno o más descriptores de archivo—en el sentido amplio del concepto— estén listos para leer o escribir.

Aparte de eso, Python proporciona un modelo de programación asíncrona basado en corutinas que funcionan en un único hilo de ejecución. La programación asíncrona es potente y compleja, y requiere del programador un enfoque muy diferente a la programación secuencial habitual.

Veremos una pequeña introducción a estas dos soluciones en las siguientes secciones.

15.13. Servidor TCP asíncrono con `select()`

Con `select()` se puede resolver de otro modo el problema del servidor iterativo: los dos puntos de bloqueo en `accept()` y `recv()`.

La función `select()`, que en Python se encuentra en el módulo del mismo nombre, permite manejar una serie de descriptores de archivo cuyo estado puede cambiar en cualquier momento. El prototipo de la función es este:

```
read_ready, write_ready, error_ready = select.select(rlist, wlist, xlist, timeout=None)
```

Los parámetros `rlist`, `wlist` y `xlist` son listas de descriptores de archivo que quieres que `select()` monitorice. En la lista `rlist` pones descriptores de los que quieres leer, en `wlist` de los que quieres escribir y en `xlist` de los que quieres comprobar si han tenido algún error. En los sistemas POSIX los descriptores de archivo incluyen muchas otras cosas como tuberías, dispositivos, terminales, y por supuesto, también sockets.

El valor de retorno es una tupla con tres listas, que contienen los descriptores que efectivamente están listos para ser leídos, escritos o han tenido un error, es decir, subconjuntos de los argumentos de la función.

La función `select()` bloquea el proceso hasta que ocurre algo en algún descriptor de cualquiera de las listas. Eso proporciona un único punto de bloqueo en el programa y por tanto, permite manejar múltiples entradas asíncronas simultáneas sin necesidad de hilos u otros mecanismos para conseguir paralelismo.

En este tipo de programas se suele asociar un bloque de código (un *manejador* o *handler*) a cada evento asíncrono (no predecible). Este paradigma se denomina *programación dirigida por eventos*[7] y es típico también de las GUI, los videojuegos u otro software interactivo.

[7] `https://es.wikipedia.org/wiki/Programación_dirigida_por_eventos`

El código del servidor completo aparece en el Listado 15.14. Como hay dos tipos de sockets: el máster y los conectados a los clientes, el programa debe hacer un tratamiento diferente para cada uno, que corresponde con los métodos `master_handler()` y `child_handler()` respectivamente. La lista `socks` contiene inicialmente el socket maestro, y conforme el servidor acepta conexiones, los nuevos sockets se añaden a esa misma lista. La lista `socks` se pasa como primer argumento a `select()` porque en este programa tan sencillo solo nos interesa evitar el bloqueo relacionado con las lecturas.

```
import sys
import socket
import time
import select
from utils import show_select_status

def upper(msg):
    time.sleep(1)  # simulates a complex job
    return msg.upper()

class Server:
    def __init__(self, port):
        self.master = socket.socket()
        self.master.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        self.master.bind(('', port))
        self.master.listen(5)
        self.socks = [self.master]

    def master_handler(self):
        conn, client = self.master.accept()
        self.socks.append(conn)
        print(f"- Client connected: {client}, Total {len(self.socks)} sockets")

    def child_handler(self, conn):
        data = conn.recv(32)
        if not data:
            self.socks.remove(conn)
            conn.close()
            return

        conn.sendall(upper(data))

    def run(self):
        while 1:
            read_ready = select.select(self.socks, [], [])[0]
            show_select_status(self.socks, read_ready)
            for s in read_ready:
                if s == self.master:
                    self.master_handler()
                else:
                    self.child_handler(s)

if __name__ == "__main__":
    if len(sys.argv) != 2:
        print("Usage: {0} <port>".format(sys.argv[0]))
        exit(1)
```

```
    try:
        server = Server(int(sys.argv[1]))
        server.run()
    except KeyboardInterrupt:
        print("\nServer shutting down.")
        exit(0)
```

LISTADO 15.14: Servidor TCP con `select()`
/upper/tcp_select.py

Si pruebas este servidor con el cliente de stress podrás ver que su comportamiento es, salvando las diferencias, similar al del servidor UDP básico. Ofrece concurrencia: todos los clientes progresan, pero no paralelismo: la función `upper()` no se ejecuta a la vez para varios clientes.

```
$ ./tcp_stress_client.py 127.0.0.1 2000 4
- [  0] Reply: TWENTY
- [  1] Reply: TWENTY
- [  2] Reply: TWENTY
- [  3] Reply: TWENTY
- [  0] Reply: TINY
```

La librería estándar de Python incluye un módulo llamado `selectors` con un nivel de abstracción algo más alto para conseguir lo mismo. Puedes ver y probar un servidor para el servicio upper con `selectors` en el archivo /upper/tcp_selectors.py. Se basa en la creación de un objeto de clase `DefaultSelector` en el que se registran manejadores para un descriptor de archivo y un motivo específico.

```
selector = selectors.DefaultSelector()
selector.register(f, selectors.EVENT_READ, handler)
```

Esto le indica al `selector` que si hay datos disponibles para leer en el descriptor de fichero `f`, debe invocar la función `handler`.

15.14. Servidor TCP asíncrono con asyncio

Python, como muchos otros lenguajes modernos, ofrece una solución más general: concurrencia basada en *corutinas* o tareas cooperativas. Con este modelo, el código se organiza de un modo similar a un programa secuencial convencional, pero cuando se invoca una primitiva de E/S bloqueante, el proceso no se bloquea, sino que suspende la ejecución de esa función y cede el control a otra. De ese modo, el proceso puede aprovechar los tiempos «muertos» sin bloqueos y sin necesidad de crear procesos u hilos adicionales.

El programador debe indicar explícitamente qué funciones son corutinas con la palabra clave `async`, y debe llamarlas con `await`. Las corutinas pueden a su vez invocar otras corutinas, mientras que la función más externa la debe gestionar un bucle de eventos. Este bucle es el que se encarga de ejecutar las corutinas, suspenderlas y reanudarlas cuando corresponda. El bucle de eventos y otras muchas utilerías las ofrece el módulo `asyncio`, que literalmente es la abreviatura de *asynchronous I/O*.

Existen dos modalidades para implementar un servidor TCP con `asyncio`:

- *Streams*
- *Transports and Protocols*

Streams es la de más alto nivel y cómo su nombre indica se basa en el manejo de dos flujos: uno de lectura y otro de escritura. El Listado 15.15 aplica ese enfoque. Se proporciona una función —que aquí hemos llamado `handle()`— que recibe los flujos (`reader` y `writer`). Para crear el servidor se utiliza `asyncio.start_server()`. Fíjate que todas las funciones bloqueantes (o las que las llaman) son asíncronas (declaradas `async`) y son invocadas con `await`. Aunque se parece bastante al servidor TCP iterativo, este puede intercalar la recepción de mensajes, que se hace en `reader.read`, con la aceptación de nuevas conexiones.

Con `async` la función que simula un `upper()` costoso no puede ser una llamada a `sleep()` porque el bucle de eventos utilizaría esa pausa para ejecutar otras corutinas, de modo que no conseguiría su propósito de simular una tarea costosa. En su lugar se ha añadido un bucle vacío que dura 1 segundo —función `fake_heavy_upper()`.

```
import sys
import asyncio
import time

async def upper(msg):
    def fake_heavy_upper(msg):
        start = time.time()
        while time.time() - start < 1:
            pass
        return msg.upper()

    return await asyncio.to_thread(fake_heavy_upper, msg)

async def handle(reader, writer):
    peername = writer.get_extra_info('peername')
    print(f"Client connected: {peername}")

    try:
        while 1:
```

```
            data = await reader.read(32)
            if not data:
                break
            writer.write(await upper(data))
            await writer.drain()

    except asyncio.CancelledError:
        pass
    finally:
        print(f"Client disconnected: {peername}")
        writer.close()
        await writer.wait_closed()

async def main(port):
    server = await asyncio.start_server(handle, '', port)

    async with server:
        await server.serve_forever()

if len(sys.argv) != 2:
    print("Usage: {0} <port>".format(sys.argv[0]))
    sys.exit(1)

try:
    asyncio.run(main(sys.argv[1]))
except KeyboardInterrupt:
    print("shut down")
```

LISTADO 15.15: Servidor TCP con asyncio Streams
/upper/tcp_async_streams.py

La segunda modalidad para implementar un servidor es *Transports and Protocols*, que es de más bajo nivel que *Streams*. El *transporte* representa el medio para transmitir los mensajes (el socket), mientras que el *protocolo* dice qué información hay que transmitir y cómo se debe interpretar.

En el caso más sencillo consiste en implementar una clase que hereda de `asyncio.Protocol` y encapsula el comportamiento del protocolo de aplicación. Se consigue especializando los métodos de la clase `Protocol`: `connection_made()`, `data_received()`, etc., que son invocados cuando ocurren esos eventos. A esto se le llama programación basada en *hooks*. El Listado 15.16 muestra el servidor TCP con esta modalidad.

```
import sys
import asyncio
import time

async def upper(msg):
    def fake_heavy_upper(msg):
        start = time.time()
        while time.time() - start < 1:
            pass
        return msg.upper()
```

```
    return await asyncio.to_thread(fake_heavy_upper, msg)

class UpperProtocol(asyncio.Protocol):
    def connection_made(self, transport):
        self.peername = transport.get_extra_info("peername")
        print(f"Client connected: {self.peername}")
        self.transport = transport

    def data_received(self, data):
        message = data.decode()
        loop = asyncio.get_running_loop()
        loop.create_task(self.handle_message(message))

    async def handle_message(self, message):
        reply = await upper(message)
        self.transport.write(reply.encode())

    def connection_lost(self, exc):
        print(f"Client disconnected: {self.peername}")
        self.transport.close()

async def main(port):
    loop = asyncio.get_running_loop()
    server = await loop.create_server(UpperProtocol, '', port)

    async with server:
        await server.serve_forever()

if len(sys.argv) != 2:
    print("Usage: {0} <port>".format(sys.argv[0]))
    sys.exit(1)

try:
    asyncio.run(main(int(sys.argv[1])))
except KeyboardInterrupt:
    print('shut down')
```

LISTADO 15.16: Servidor TCP con `asyncio` Transport and Protocols
`/upper/tcp_async_protocol.py`

15.15. Cliente `TCP` asíncrono con `asyncio`

Una posible versión asíncrona del cliente TCP (§ 15.3) podría ser el del Listado 15.17. Es destacable el uso del método `run_in_executor()` que permite ejecutar la función `stdin.readline()`, que es bloqueante, en un hilo independiente para evitar que bloquee el bucle de eventos. En realidad aunque el envío y la recepción son concurrentes, no aporta ninguna ventaja en este caso y no tiene demasiado interés práctico. Evitar el bloqueo de la lectura desde la consola y desde el socket no va a mejorar el rendimiento del cliente, seguirá pudiendo enviar un mensaje cada segundo como máximo.

```
import sys
import asyncio

def readline():
    return sys.stdin.readline().strip().encode()

async def main(host, port):
    reader, writer = await asyncio.open_connection(host, port)

    try:
        while True:
            data = await asyncio.get_event_loop().run_in_executor(None, readline)
            if not data:
                break

            writer.write(data)
            await writer.drain()

            msg = b''
            while len(msg) < len(data):
                chunk = await reader.read(32)
                if not chunk:
                    break
                msg += chunk

            print(f"Reply is '{msg.decode()}'")

    except asyncio.CancelledError:
        pass
    finally:
        writer.close()
        await writer.wait_closed()

if len(sys.argv) != 3:
    print("Usage: {0} <host> <port>".format(sys.argv[0]))
    sys.exit(1)

try:
    asyncio.run(main(sys.argv[1], int(sys.argv[2])))
except KeyboardInterrupt:
    print("shut down")
```

Listado 15.17: Cliente TCP con asyncio
/upper/tcp_client_async.py

Eso no significa que no sea interesante en otros casos. Si el cliente tiene otras cosas que hacer, puede suponer una importante ventaja. Un ejemplo práctico de esto es el cliente de *stress* que hemos estado usando a lo largo del capítulo: /upper/tcp_stress_client.py. Este cliente envía mensajes a varios servidores, de modo que mientras espera la respuesta de uno, puede estar enviando mensajes a otros. Te recomendamos estudiar ese programa y compararlo con el cliente 15.17 anterior.

15.16. Servidor UDP asíncrono con `asyncio`

También puedes implementar un servidor UDP con la modalidad *Transports and Protocols* (ver Listado 15.18), heredando de `asyncio.DatagramProtocol`. Sin embargo, no es posible implementar un servidor UDP con *Streams* porque está diseñado solo para transferencia de datos orientada a flujo.

```
import sys
import asyncio
import time

async def upper(msg):
    def fake_heavy_upper(msg):
        start = time.time()
        while time.time() - start < 1:
            pass
        return msg.upper()

    return await asyncio.to_thread(fake_heavy_upper, msg)

class UpperUDPProtocol(asyncio.DatagramProtocol):
    def connection_made(self, transport):
        self.transport = transport

    def datagram_received(self, data, addr):
        print(f"New request: {addr}")
        asyncio.create_task(self.handle_request(data, addr))

    async def handle_request(self, data, addr):
        response = await upper(data.decode())
        self.transport.sendto(response.encode(), addr)

    def error_received(self, exc):
        print(f"Comm error: {exc}")

async def main(port):
    loop = asyncio.get_running_loop()

    transport, _ = await loop.create_datagram_endpoint(
        lambda: UpperUDPProtocol(),
        local_addr=('', port)
    )

    try:
        await asyncio.sleep(3600)
    except asyncio.CancelledError:
        pass
    finally:
        transport.close()

if len(sys.argv) != 2:
    print("Usage: {0} <port>".format(sys.argv[0]))
    sys.exit(1)

try:
    asyncio.run(main(int(sys.argv[1])))
```

```
except KeyboardInterrupt:
    print("shut down")
```

LISTADO 15.18: Servidor UDP con asyncio *Transport and Protocols*
/upper/udp_async_protocol.py

Y ¿qué más?

Este capítulo ha mostrado las posibilidades *nativas* de concurrencia que ofrece un servidor UDP debido a su naturaleza orientada a mensajes individuales. Al mismo tiempo eso implica limitaciones debido a que es necesario realizar todas sus operaciones sobre un único socket, por la contención que supone proteger el socket adecuadamente. A pesar de ello, existen técnicas como el *pool* de hilos o el *preforking* que permiten un alto nivel de paralelismo evitando el acceso compartido. Por contra, TCP ofrece más opciones para hacer servidores de alto rendimiento al estar basado en un modelo de conexiones que permite que cada una de esas conexiones disponga de su propio socket, con el aislamiento que ello conlleva. Sea como sea, no se deben despreciar las posibilidades de concurrencia que ofrece la gestión de E/S asíncrona, ya sea con `select` y sus derivados, o el soporte integrado en el lenguaje gracias a `asyncio`.

Aunque esto es solo una pincelada de cada una de estas técnicas, es sencillo imaginar las diversas opciones disponibles, tanto con Python como con otros lenguajes. Te invitamos a experimentar por ti mismo y aplicar estas y otras técnicas más avanzadas a tus propios proyectos.

Capítulo 16

Publicador-Suscriptor

Al terminar este capítulo, entenderás:

- Qué es el modelo de interacción publicador-suscriptor.
- Cómo implementar un chat con este modelo de interacción, tanto con UDP como con TCP.
- Qué es y qué ofrece el protocolo MQTT para implementar este tipo de comunicación.

El segundo modelo de interacción más habitual, por detrás del cliente-servidor, es publicador-suscriptor[1]. En lugar de aplicar una interacción basada en petición-respuesta, este modelo asume que existen dos tipos de participantes: los *publicadores* envían mensajes a un espacio común (el *canal*) y los *suscriptores* reciben los mensajes que les interesan en función de a qué estén suscritos. El matiz clave es que esos mensajes se envían sin esperar una respuesta. Si los publicadores esperan algún tipo de información de vuelta, lo harán también con un rol de suscriptores, pero no como parte de la misma interacción, por lo que no podemos considerar que se trate de respuestas.

El modelo se caracteriza por el uso de un componente intermedio llamado **broker** que es quien se encarga de recibir los mensajes que proceden de los publicadores y hacerlos llegar a los suscriptores. Lógicamente, no tiene sentido llamar *peticiones* a estos mensajes porque no buscan un resultado. En su lugar se les llama *publicaciones* o más comúnmente *eventos*.

El hecho de que haya mensajes solo en un sentido y que todos ellos pasen por el *broker* desacopla completamente a publicadores y suscriptores. Lo único que deben conocer todos los participantes es el endpoint del broker.

[1] Abreviado a menudo como «pubsub».

La otra peculiaridad de este modelo es que puede haber múltiples suscriptores para un mismo canal. El broker es el encargado de realizar y distribuir las copias necesarias para entregar a cada suscriptor.

16.1. Chat UDP para dos

Para entender cómo funciona este modelo, vamos a desarrollar paso a paso un ejemplo práctico típico: un *chat*. Es un programa sencillo que permite a varias personas enviar mensajes a una *sala* de modo que todos los usuarios presentes en la sala pueden verlos.

Empezaremos desde una versión extremadamente simple con solo dos participantes. Después iremos añadiendo funcionalidad y resolviendo los problemas que vayan apareciendo. Para facilitar el desarrollo inicialmente le vamos a dar la estructura cliente-servidor, pero conforme evolucione, incorporaremos un broker y se convertirá en publicación-suscripción. También por simplicidad, empezaremos con una versión UDP.

16.1.1. Paso 1: Mensaje unidireccional

En este primer paso los objetivos son crear:

- Un servidor que recibe un único mensaje, lo imprime en consola y termina.
- Un cliente que envía la cadena `hello` al servidor y termina.

Servidor

Las tareas que realiza el servidor son muy simples:

1. Crea un socket UDP.
2. Vincula el socket a un puerto libre.
3. Espera un datagrama que contiene un mensaje de texto.
4. Imprime el mensaje en consola.

Estas tareas corresponden línea a línea con el Listado 16.1.

```
import socket

sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
sock.bind(('', 12345))
message, client = sock.recvfrom(1024)
print(message.decode(), client)
sock.close()
```

LISTADO 16.1: Servidor de chat UDP básico
/chat-udp/server1.py

No hay mucho que comentar. Es un programa aún más simple que los anteriores. Recuerda que al ser UDP, el argumento de `recvfrom()` limita el tamaño de los mensajes que podrán intercambiar los usuarios, pero para un chat sencillo como este, 1024 bytes parece más que suficiente.

Para ejecutarlo simplemente escribe el siguiente comando en la consola:

```
$ python3 chat-udp/server1.py
```

El programa (y la consola) queda inmediatamente bloqueado, en concreto en la invocación del método `recvfrom()`, a la espera de recibir el mensaje de un cliente.

Cliente

Antes de escribir el programa cliente en Python es buena idea probar que el servidor funciona como debe. Y para eso puedes utilizar `ncat` (ver 6.12).

Con el siguiente comando puedes ejecutar un cliente UDP que envía la cadena `hola` al servidor que está escuchando en el puerto 12345 (el de acabas de poner en marcha). En este instante el servidor debería imprimir el saludo y terminar.

```
$ echo hola | ncat --udp --send-only 127.0.0.1 12345
```

Ahora que has comprobado que el servidor funciona, puedes pasar a escribir un cliente que imite esta misma funcionalidad. También es muy simple: crea el socket UDP, envía un mensaje y termina (Listado 16.2).

```
import socket

sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
sock.sendto("hola".encode(), ('127.0.0.1', 12345))
sock.close()
```

Listado 16.2: Cliente de chat UDP básico
/chat-udp/client1.py

El segundo parámetro de la función `sendto()` es el endpoint destino del mensaje (`127.0.0.1:12345`), es decir, el servidor y el cliente se ejecutan en la misma máquina, algo recomendable al menos mientras escribes y pruebas el programa.

Lo puedes ejecutar como sigue, y el efecto debería ser el mismo que con `ncat`.

```
$ python3 chat-udp/client.py
```

16.1.2. Paso 2: Devuelve el saludo

El servidor del paso anterior solo imprime el mensaje recibido. Con un pequeño cambio podrá devolver el saludo al cliente.

Utilizando la dirección del cliente, que se obtiene como valor de retorno del método `recvfrom()`, la aplicación puede a su vez utilizar el método `sendto()` y enviar un mensaje de vuelta (Listado 16.3).

```
import socket

sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
sock.bind(('', 12345))
message, peer = sock.recvfrom(1024)
print(message.decode(), peer)
sock.sendto("qué tal?".encode(), peer)
sock.close()
```

LISTADO 16.3: Servidor de chat UDP con respuesta
/chat-udp/server2.py

El cliente del paso anterior terminaba inmediatamente después de enviar su mensaje. Ahora debe esperar para recibir el mensaje del servidor. Como es lógico, basta con imitar lo que hace el servidor: usar el método `recvfrom()` (Listado 16.4).

```
import socket

sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
sock.sendto("hola".encode(), ('127.0.0.1', 12345))
message, peer = sock.recvfrom(1024)
print("{} from {}".format(message.decode(), peer))
sock.close()
```

LISTADO 16.4: Cliente de chat UDP con respuesta
/chat-udp/client2.py

16.1.3. Paso 3: Libertad de expresión

Con este paso los usuarios que ejecutan cliente y servidor tendrán realmente la posibilidad de conversar. En lugar de enviar una cadena literal, ambos programas van a leer de consola lo que el usuario teclee y lo van a enviar hacia el interlocutor. La conversación se mantendrá hasta que cualquiera de ellos envíe la cadena `bye`. El Listado 16.5 muestra el código completo del servidor después de este cambio:

```
import socket
QUIT = b"bye"

sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
```

```
sock.bind(('', 12345))

while 1:
    message_in, peer = sock.recvfrom(1024)
    print(message_in.decode())

    if message_in == QUIT:
        break

    message_out = input().encode()
    sock.sendto(message_out, peer)

    if message_out == QUIT:
        break

sock.close()
```

Listado 16.5: Servidor de chat UDP por turnos
/chat-udp/server3.py

La diferencia principal respecto a las versiones anteriores es el bucle `while`. Este bucle termina tanto si el usuario local introduce la cadena `bye` como si es recibida a través del socket. Para leer una cadena de texto de la consola se utiliza la función `input()`. El código del cliente (Listado 16.6) es muy similar.

```
import socket
QUIT = b"bye"

sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
server = ('', 12345)

while 1:
    message_out = input().encode()
    sock.sendto(message_out, server)

    if message_out == QUIT:
        break

    message_in, peer = sock.recvfrom(1024)
    print(message_in.decode())

    if message_in == QUIT:
        break
```

Listado 16.6: Cliente de chat UDP por turnos
/chat-udp/client3.py

Únicamente hay dos diferencias entre cliente y servidor: solo el servidor ejecuta `bind()` y es el que empieza recibiendo un mensaje en el socket, mientras el cliente empieza leyendo de teclado. En UDP, el cliente tiene que ser el primero en enviar un mensaje. Como no hay conexión previa, el

servidor no puede saber quién es el cliente (su endpoint) hasta que reciba algo.

Esta versión tiene un problema grave en cuanto a la interacción entre los usuarios. Tanto el cliente como el servidor tienen dos puntos diferentes en los que el programa queda bloqueado: la función `input()` para leer de consola y el método `recvfrom()` para leer del socket. Es el mismo problema que los servidores del capítulo 15: la imposibilidad de atender múltiples entradas asíncronas simultáneamente[2]. Pero en este caso concreto no tiene que ver con concurrencia. Aquí implica que los usuarios han de esperar a que su interlocutor envíe un mensaje antes de poder escribir de nuevo. O dicho de otro modo, los usuarios no pueden enviar 2 o más mensajes consecutivos, que es lo que se espera de un chat.

Y con esto, la situación tampoco encaja con un protocolo petición-respuesta. No queremos que haya una correspondencia uno a uno entre los mensajes que envía el cliente y los que envía el servidor. Los mensajes que envía el servidor no son respuestas a solicitudes del cliente, cada usuario debe poder enviar tantos mensajes como quiera en cualquier momento.

16.1.4. Paso 4: Habla cuando quieras

Para resolver el problema de la E/S asíncrona vamos a utilizar hilos, que es la más sencilla al menos a nivel de código. Atenderemos la entrada procedente de la consola en un hilo adicional (el envío), mientras que el hilo principal se encargará de atender el socket (la recepción). El código del servidor aparece en el Listado 16.7.

```
4  import socket
5  import _thread
6  server = ('', 12345)
7  QUIT = b"bye"
8
9  class Chat:
10     def __init__(self, sock, peer):
11         self.sock = sock
12         self.peer = peer
13
14     def run(self):
15         _thread.start_new_thread(self.sending, ())
16         self.receiving()
17         self.sock.close()
18
19     def sending(self):
20         while 1:
21             message = input().encode()
22             self.sock.sendto(message, self.peer)
```

[2] a menos que se utilicen hilos, procesos o E/S asíncrona.

```
23
24              if message == QUIT:
25                  break
26
27      def receiving(self):
28          while 1:
29              message, peer = self.sock.recvfrom(1024)
30              print(message.decode())
31
32              if message == QUIT:
33                  self.sock.sendto(QUIT, self.peer)
34                  break
35
36  if __name__ == '__main__':
37      sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
38      sock.bind(server)
39      message, client = sock.recvfrom(0, socket.MSG_PEEK)
40      Chat(sock, client).run()
```

Listado 16.7: Servidor de chat UDP simultaneo
/chat-udp/server4.py

El programa está compuesto por una clase `Chat` con tres métodos, aparte del constructor. El método `sending()` se ocupa de leer líneas de texto de la consola y enviarlas a través del socket. El método `receiving()` lee líneas de texto del socket y las imprime en la consola. En ambos casos, si el mensaje leído o recibido es `bye`, la función termina. En el caso de la función `receiving()` además devuelve el mensaje para que el hilo de recepción del otro extremo también termine. El método `run()` crea y arranca el hilo para la tarea de envío y ejecuta la tarea de recepción.

En cuanto a la función principal, la llamada a `recvfrom()` (**línea 39**) se utiliza únicamente para obtener el endpoint del cliente, pero sin consumir nada del buffer del socket gracias al flag MSG_PEEK. La **línea 40** crea la instancia de la clase `Chat` pasando el socket y el endpoint del cliente como parámetros.

El cliente solo difiere en la creación del socket de modo que se puede reutilizar la clase `Chat` del servidor tal cual. Simplemente crea el socket y la instancia de `Chat`, a la que le pasa el socket y el endpoint del servidor (Listado 16.8).

```
import socket
from server4 import Chat, server

sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
Chat(sock, server).run()
```

Listado 16.8: Cliente de chat UDP simultáneo
/chat-udp/client4.py

Sigue habiendo un pequeño problema estético. Como el punto de entrada del usuario y el lugar donde se escriben los mensajes que se reciben es el mismo (la salida estándar) es fácil que se mezclen, dificultando la lectura de la salida. Para solucionarlo habría que hacer que las tareas de recepción y envío escribieran en partes diferentes de la pantalla, o quizá construir un pequeño GUI. Sin embargo, ésta es una cuestión al margen de la finalidad de este ejemplo.

16.1.5. Paso 5: Todo en uno

El servidor y el cliente del paso anterior utilizan la misma clase `Chat` para resolver la mayor parte del problema. Lo único diferente entre servidor y cliente es el código de la función principal (lo que está fuera de clase `Chat`). Eso significa que es posible crear un único programa que se comporte como servidor o cliente en función de un parámetro de línea de comandos (Listado 16.9).

```
import sys
import socket
from threading import Thread

SERVER = ('', 12345)
QUIT = b'bye'

class Chat:
    def __init__(self, sock, peer):
        self.sock = sock
        self.peer = peer

    def run(self):
        sender_thread = Thread(target=self.sending, daemon=True)
        sender_thread.start()
        self.receiving()
        sender_thread.join()
        self.sock.close()

    def sending(self):
        while True:
            message = input().encode()
            self.sock.sendto(message, self.peer)
            if message == QUIT:
                break

    def receiving(self):
        while 1:
            message, _ = self.sock.recvfrom(1024)
            print("other> {}".format(message.decode()))
            if message == QUIT:
                self.sock.sendto(QUIT, self.peer)
                break

if __name__ == '__main__':
```

```
    if len(sys.argv) != 2:
        print("usage: %s [--server|--client]".format(sys.argv[0]))
        sys.exit()

    mode = sys.argv[1]
    sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

    if mode == '--server':
        sock.bind(SERVER)
        message, client = sock.recvfrom(0, socket.MSG_PEEK)
        Chat(sock, client).run()
    else:
        Chat(sock, SERVER).run()
```

LISTADO 16.9: Chat UDP multihilo (servidor y cliente)
/chat-udp/chat-thread.py

16.2. Chat asíncrono con `select()`

Como alternativa a los hilos, el chat anterior también se puede implementar con `select()`. Las dos entradas asíncronas a vigilar son la consola y el socket, así que se le pueden pasar esos descriptores a `select()`. Es un uso más simple que el del servidor TCP de § 15.13, pero que sigue la misma idea: proporcionar un único punto de bloqueo que permite atender cualquiera de las dos situaciones. La llamada sería algo como:

```
while 1:
    ready = select([sys.stdin, sock], [], [])[0]
    for fd in ready:
        if fd == sock:
            # leer del socket
        else:
            # leer de consola
```

El bucle `for` es necesario porque ambos descriptores podrían estar listos para leer al mismo tiempo, o muy próximos en el tiempo. Para leer se va a seguir usando las funciones `recvfrom()` e `input()`. Recuerda que, después de que `select()` haya confirmado que hay datos disponibles, tienes garantía de que esas funciones no van a bloquear el programa. El programa completo (que se sigue pudiendo usar como cliente o servidor) aparece en el Listado 16.10.

```
import sys
import socket
from select import select
SERVER = ('', 12345)
QUIT = b'bye'

class Chat:
    def __init__(self, sock, peer):
        self.sock = sock
```

```
        self.peer = peer

    def run(self):
        fds = [sys.stdin, self.sock]
        while 1:
            ready = select(fds, [], [])[0]
            for fd in ready:
                if self.sock in ready:
                    msg = self.receiving()
                else:
                    msg = self.sending()

                if msg == QUIT:
                    return

    def sending(self):
        message = input().encode()
        self.sock.sendto(message, self.peer)
        return message

    def receiving(self):
        message, peer = self.sock.recvfrom(1024)
        print("other> {}".format(message.decode()))
        return message

if __name__ == '__main__':
    if len(sys.argv) != 2:
        print("usage: %s [--server|--client]".format(sys.argv[0]))
        sys.exit()

    mode = sys.argv[1]
    sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

    if mode == '--server':
        sock.bind(SERVER)
        message, client = sock.recvfrom(0, socket.MSG_PEEK)
        Chat(sock, client).run()

    else:
        Chat(sock, SERVER).run()
```

LISTADO 16.10: Chat UDP (servidor y cliente) con `select()`
`/chat-udp/chat-select.py`

Aquí los métodos `sending()` y `receiving()` actúan como *manejadores* de los eventos de lectura de consola y del socket respectivamente. A diferencia de la versión con hilos, no hay bucles. Son funciones que se ejecutan cuando se detecta cada una de las condiciones, hacen una única lectura, procesan el mensaje y terminan.

Es sencillo adaptarlo para usar el módulo `selectors`, que indudablemente gana en limpieza, cambiando el método `run()` a algo como:

```
def run(self):
    selector = selectors.DefaultSelector()
```

```
    selector.register(sys.stdin, selectors.EVENT_READ, self.sending)
    selector.register(self.sock, selectors.EVENT_READ, self.receiving)

    while 1:
        for key, mask in selector.select():
            if key.data() == QUIT:
                return
```

El código completo del servidor está en el archivo /chat-udp/chat-selectors.py.

16.3. Chatroom UDP

Ahora que están claros los mecanismos para el envío y recepción de mensajes, podemos plantear una solución general para un chat con múltiples usuarios. Llamaremos a esta modalidad 'chatroom' para distinguirlo de la versión para dos usuarios. Esta «sala de chat» es el *canal* al que los publicadores envían sus mensajes.

El nuevo servidor va a tomar el papel de *broker* que recibirá los mensajes de una cantidad arbitraria de participantes. Estos participantes serán a la vez publicadores, por su tarea de envío; y suscriptores, por su tarea de recepción.

La Figura 16.1 representa esta idea, pero ten en cuenta que los óvalos que representan a los participantes aparecen por duplicado, en sus dos roles de publicadores y suscriptores, para enfatizar el flujo de los mensajes.

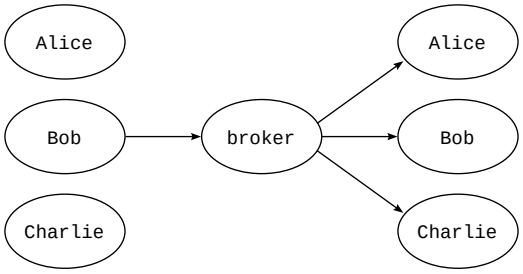


FIGURA 16.1: Arquitectura del chatroom

El código del broker aparece en el Listado 16.11. Mantiene un diccionario llamado `members` que asocia el endpoint de cada participante con su *nick*. Al recibir el primer mensaje de un participante, lo añade al diccionario. Los siguientes mensajes ya se envían a todos los demás participantes precediendo el nombre del emisor. Por último, si el mensaje que envía el participante es `bye`, el broker lo elimina del diccionario y de la sala de chat.

Este programa es un buen ejemplo de cómo un socket UDP puede operar múltiples flujos (es concurrente) a pesar de trabajar en un único hilo de ejecución.

Es interesante que el broker no necesita ningún mecanismo de concurrencia o gestión asíncrona porque solo usa un socket, mientras que el participante sí lo necesita (`selectors` en este caso) porque tiene dos entradas que atender: consola y socket.

```
import socket
QUIT = 'bye'

def main():
    sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    sock.bind(('', 12345))
    members = {}

    while 1:
        data, endpoint = sock.recvfrom(1024)
        message = data.decode()
        if endpoint not in members.keys():
            members[endpoint] = message
            print("User '{}' has joined the chat.".format(message))
            continue

        sender = members[endpoint]
        for member_endpoint, nick in members.items():
            if member_endpoint == endpoint:
                continue

            print(nick, '<-', message)
            encoded = "{}: {}\n".format(sender, message).encode()
            sock.sendto(encoded, member_endpoint)

        if message == QUIT:
            del members[endpoint]
            print("User '{}' has left the chat.".format(sender))

try:
    main()
except KeyboardInterrupt:
    print("shut down.")
```

LISTADO 16.11: Broker de chatroom UDP
/chat-udp/chatroom-broker.py

El cliente (en el rol de participante) es en realidad muy parecido al del paso 4 del chat de dos usuarios (lo tienes en el archivo /chat-udp/chatroom-member.py). Te animamos a probar el broker y unos cuantos participantes en terminales diferentes para que puedas comprobar cómo funciona.

Broker

```
$ ./chatroom-broker.py
New connection from ('127.0.0.1', 41656)
User 'alice' has joined the chat.
New connection from ('127.0.0.1', 53202)
User 'bob' has joined the chat.
New connection from ('127.0.0.1', 47864)
User 'charlie' has joined the chat.
alice <- hi everyone
bob <- hi everyone
```

Alice

```
$ ./chatroom-member.py 127.0.0.1 2000
alice
charlie: hi everyone
```

Bob

```
$ ./chatroom-member.py 127.0.0.1 2000
bob
charlie: hi everyone
```

Charlie

```
$ ./chatroom-member.py 127.0.0.1 2000
charlie
hi everyone
```

FIGURA 16.2: Chatroom con tres participantes

16.4. Chatroom TCP

Por supuesto también tiene mucho sentido hacer una versión TCP, no ya por rendimiento, sino por fiabilidad. Recuerda que UDP puede perder cualquier mensaje en cualquier momento, lo que no es lo que más conviene para una aplicación de mensajería.

El Listado 16.12 muestra las diferencias más importantes respecto a la versión UDP.

```
import socket
import selectors

QUIT = 'bye'
PORT = 12345

class ChatroomBroker:
    def main(self):
        with socket.socket() as self.master:
            self.master.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
            self.master.bind(('', PORT))
            self.master.listen(5)

            self.members = {}
            self.selector = selectors.DefaultSelector()
            self.selector.register(self.master, selectors.EVENT_READ, self.acceptor)

            while True:
                for key, mask in self.selector.select():
                    key.data(key.fileobj)

    def acceptor(self, sock):
        conn, addr = self.master.accept()
        print("New connection from", addr)
        self.members[conn] = None
        self.selector.register(conn, selectors.EVENT_READ, self.receiver)
```

```
    def receiver(self, conn):
        message = conn.recv(1024).decode().strip()
        if message == QUIT or not message:
            print("User '{}' has left the chat.".format(self.members[conn]))
            self.selector.unregister(conn)
            conn.close()
            del self.members[conn]
            return

        if not self.members[conn]:
            self.members[conn] = message
            print("User '{}' has joined the chat.".format(message))
            return

        sender = self.members[conn]
        for member_conn, nick in self.members.items():
            if member_conn == conn:
                continue

            print(nick, '<-', message)
            encoded = "{}: {}\n".format(sender, message).encode()
            member_conn.sendall(encoded)

try:
    ChatroomBroker().main()
except KeyboardInterrupt:
    print("shut down.")
```

LISTADO 16.12: Broker de chatroom TCP
/chat-tcp/chatroom-broker.py

El broker mantiene el diccionario `members` cuya clave es el socket conectado y cuyo valor es el *nick* del participante. Lo hemos implementado con dos manejadores para un `selector`: `receiver()` y `reader()`. El primero, como su nombre indica, acepta cada nueva conexión, y la registra en el selector para recibir los mensajes de ese participante. El segundo es el que se encarga de esto. Tiene que manejar varias situaciones:

- Eliminación del participante cuando envía el mensaje `bye` o este cierra la conexión.
- Fijar el *nick* del participante cuando se recibe el primer mensaje.
- Reenviar a todos los demás participantes cuando se recibe un mensaje. En este caso, concatena el nick del emisor al comienzo del mensaje.

El participante también lo hemos implementado con un `selector` y dos manejadores: `sending()` y `receiving()`. El primero lee de la consola y envía el mensaje al broker, mientras que el segundo lee del socket y lo imprime en la consola. El código del participante se muestra en el Listado 16.13.

```
import sys
import socket
import selectors

QUIT = b'bye'

class ChatroomMember:
    def __init__(self, broker, nick):
        self.broker = broker
        self.nick = nick

    def sending(self):
        message = input().encode()
        self.sock.sendall(message)
        return message

    def receiving(self):
        message = self.sock.makefile().readline().strip()
        print(message)
        return message

    def run(self):
        self.sock = socket.socket()
        self.sock.connect(self.broker)
        self.sock.sendall(self.nick.encode())

        selector = selectors.DefaultSelector()
        selector.register(sys.stdin, selectors.EVENT_READ, self.sending)
        selector.register(self.sock, selectors.EVENT_READ, self.receiving)

        while 1:
            for key, mask in selector.select():
                if key.data() in [QUIT, '']:
                    return+33

if len(sys.argv) != 3:
    exit("Usage: ./chatroom-member.py <broker_address> <nick>")

try:
    broker = (sys.argv[1], 12345)
    ChatroomMember(broker, nick=sys.argv[2]).run()
except (KeyboardInterrupt, EOFError):
    print("shut down.")
```

LISTADO 16.13: Participante de chatroom
/chat-tcp/chatroom-member.py

Recuerda que con `selectors` o `select()` conseguimos un comportamiento asíncrono y concurrente (no paralelo) equivalente en ese sentido al broker UDP.

16.5. Chatroom con MQTT

La sección anterior demuestra cómo programar un broker TCP sencillo que se encarga de la distribución de mensajes desde los productores a los consu-

midores. MQTT es precisamente un protocolo diseñado para crear este tipo de aplicaciones. Las implementaciones de este protocolo suelen proporcionar un broker genérico. El más utilizado por mucho (y que también vamos a usar aquí) se llama `mosquitto`.

Para una sala de chat tan simple como la de la sección anterior, solo necesitamos escribir el código del participante (Listado 16.14) que hace uso de una librería cliente de MQTT llamada `paho-mqtt`, también con mucho la más usada en Python.

```
import json
import sys
import paho.mqtt.client as mqtt

TOPIC = 'chatroom'
QUIT  = 'bye'

class ChatroomMember:
    def __init__(self, broker, nick):
        self.broker = broker
        self.nick = nick

    def on_connect(self, client, userdata, flags, reason_code, properties):
        self.client.subscribe(TOPIC)

    def on_message(self, client, userdata, msg):
        data = json.loads(msg.payload.decode())
        if data['nick'] == self.nick:
            return

        print("{}: {}".format(data['nick'], data['message']))

    def run(self):
        self.client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)
        self.client.on_connect = self.on_connect
        self.client.on_message = self.on_message
        self.client.connect(*self.broker)
        self.client.loop_start()

        while 1:
            line = input()
            if line == QUIT:
                break

            payload = json.dumps({'nick': self.nick, 'message': line})
            self.client.publish(TOPIC, payload)

        self.client.loop_stop()
        self.client.disconnect()

if len(sys.argv) != 3:
    exit("Usage: ./chatroom-member.py <broker_address> <nick>")

try:
    broker = (sys.argv[1], 1883)
```

```
    ChatroomMember(broker, nick=sys.argv[2]).run()
except (KeyboardInterrupt, EOFError):
    print("shut down.")
```

LISTADO 16.14: Participante de chatroom con MQTT
/chat-mqtt/chatroom-member.py

La estructura de este programa se parece mucho a la del Listado 16.13. Hay un método `run()` en el que se configura un *callback* `on_connect()` que realiza la suscripción al canal `chatroom` y otro `on_message()` que se encarga de procesar e imprimir los mensajes recibidos.

Con MQTT tanto los productores como los suscriptores utilizan la clase `Client`. En este caso, al igual que la versión TCP básica, el participante es a la vez publicador —invocando el método `publish()`— y suscriptor —con el callback de `on_message()`.

Este *callback* para `on_message()` funciona de un modo muy similar a los manejadores de `selectors`. Cuando hay datos, el *runtime* invoca el método y los pasa como argumento (`userdata`). Este mensaje puede ser de cualquier tipo que se pueda serializar como cadena o secuencia de bytes, pero lo más habitual con MQTT es usar JSON. A pesar de lo simple que es la carga útil de estos mensajes (nick, mensaje de usuario) hemos preferido usarlo también aquí.

El bucle de eventos de MQTT arranca con `loop_start()` y se ejecuta en un hilo independiente, por eso el método `run()` queda libre para la lectura de mensajes desde la entrada estándar.

El broker `mosquitto` se puede utilizar como un servicio del sistema, sin más que instalar el paquete oficial del mismo nombre. También es posible arrancarlo como un contenedor docker si lo prefieres con algo como:

```
$ docker run --rm -p 1883:1883 eclipse-mosquitto mosquitto -c /mosquitto-no-auth.conf
```

Para esta sencilla prueba, vamos a suponer que el broker está disponible como servicio del sistema en tu máquina. Puedes ejecutar varios participantes en varios terminales indicando la IP del broker y el nick de cada participante.

```
$ ./chatroom-member.py 127.0.0.1 alice
$ ./chatroom-member.py 127.0.0.1 bob
```

También hay un pequeño script `tmux` /chat-mqtt/run.sh que lanza 3 participantes e ilustra el funcionamiento del chatroom.

Y ¿qué más?

El modelo publicador-suscriptor es muy común en aplicaciones distribuidas por su naturaleza asíncrona y desacoplada. Es especialmente útil para aplicaciones de monitorización, notificación o actualización de estado como por ejemplo IoT. También es relevante su buen nivel de escalabilidad y adaptabilidad ya que permite que publicadores y suscriptores se añadan y eliminen sin afectar a los demás participantes. Por otro lado, la crítica más frecuente es que su broker representa un cuello de botella y un punto único de fallo. Solucionar esa limitación lleva a arquitecturas más complejas basadas en brokers federados o replicados, y en última instancia a arquitecturas sin broker, lo que da lugar a otro importante modelo de interacción: *peer-to-peer* (P2P), llamadas comúnmente en español como *redes de pares*.

Capítulo 17

Calidad de servicio

Al terminar este capítulo, entenderás:

- Los parámetros básicos de Calidad de Servicio: tasa de transferencia, latencia, *jitter*, pérdida de paquetes.
- Cómo medir tasa de transferencia, latencia y *jitter*.
- Cómo el control de flujo TCP actúa como limitador de tasa.
- Qué es el marcado y clasificación de paquetes.
- Cómo operan policing y shaping.
- Cómo funciona la Calidad de Servicio en Linux.

Se dice de IP que es un protocolo de «mejor esfuerzo» *(best effort)*, que no es más que una forma bonita de decir que hace lo que buenamente puede y eso, en función de las condiciones de la red, podría ser absolutamente nada. La integridad, el orden o incluso la propia entrega son prestaciones que IP consigue eventualmente, pero no están en absoluto aseguradas. Aunque TCP sí ofrece estas garantías, no todas las aplicaciones funcionan bien con TCP, e incluso cuando es así, hay otras prestaciones deseables que no ofrece. Por ejemplo, la tecnología TCP/IP básica no puede garantizar (ni siquiera lo intenta) una tasa de transferencia mínima, o una latencia o *jitter* por debajo de ciertos umbrales.

Una de las limitaciones clave de la tecnología de conmutación de paquetes, en la que se basa IP e Internet, es que no dispone de *control de admisión*, es decir, nunca niega una nueva solicitud de envío o conexión por mucho que la interred ya esté sufriendo una carga excesiva o incluso congestión.

El control de admisión es la característica típica de las redes de conmutación de circuitos, como la de telefonía. La red telefónica puede cuantificar la cantidad de recursos disponibles y rechazar el servicio solicitado por un cliente si estima que no lo va a poder satisfacer adecuadamente: el clásico aviso de «todas las líneas están ocupadas». La conveniencia del control de admisión es fácil de entender, pero existen otras formas, no tan obvias, de garantizar el servicio.

La QoS (Quality of Service) engloba los mecanismos que tratan de garantizar determinado nivel en los parámetros de tráfico para los servicios que ofrece una interred de conmutación de paquetes. La idea que subyace en la QoS es que la red debería asignar sus recursos en función de las necesidades de cada usuario, tipo de flujo o servicio, bajo la premisa de que no todos necesitan las mismas prestaciones. Por ejemplo, una llamada de voz requiere baja latencia, pero puede aceptar cierta pérdida de mensajes, mientras que la descarga de un archivo puede tolerar alta latencia, pero ninguna pérdida de datos. Fíjate que QoS no implica cotas muy exigentes de rendimiento, solo garantizar ciertos mínimos, que no tienen por qué ser altos, pero que deben cumplirse.

Un flujo de red (en los términos que se definió en § 6.3) es la unidad de tráfico que se trata como un todo a la hora de clasificar, priorizar o asignar recursos, y es el elemento básico sobre el que trabja el pipeline de QoS.

Los parámetros QoS básicos de un flujo se han comentado ya: tasa de transferencia, latencia, *jitter* y tasa de pérdida de paquetes —o confiabilidad en general. Es importante entender que QoS es un juego de suma cero: los recursos son limitados (y normalmente conocidos) y los que consume un flujo no están disponibles para los demás.

Un ejemplo con la tasa de transferencia, que es un parámetro fácil de entender: imagina un enlace que, por construcción, está limitado a 100 Mbps: QoS debe garantizar que la suma de las tasas utilizadas por todos los flujos no exceda ese límite. Su papel, por tanto, es doble: asignar a cada flujo los recursos que necesita —aquí, ancho de banda— y evitar que, en conjunto, excedan la capacidad disponible. Si ese tipo de control no existe, cuando el enlace se satura, se descartan paquetes arbitrarios con consecuencias impredecibles; con él, es posible elegir qué paquetes conviene sacrificar para minimizar el daño.

A veces, el límite no es la capacidad física del canal, sino un valor acordado, parte de un contrato comercial entre un proveedor y un cliente, en concreto el ancho de banda contratado se denomina CIR (Committed Information Rate). Imagina un ISP que ofrece a sus clientes conexión de fibra con 1 Gbps de descarga (ese es el CIR). Por encima de eso, el proveedor puede ofrecer un extra si tiene capacidad libre, pero sin garantías. Esa capacidad extra es el EIR (Excess Information Rate)[1].

El contrato no es solo retórica. Literalmente existe un contrato que se conoce como SLA (Service Level Agreement) o en español Acuerdo de Nivel de Servicio, y en los casos más exigentes, aparte de la tasa, incluye objeti-

[1] También se suele hablar de PIR (Peak Information Rate), que es CIR + EIR

vos cuantificables (SLO) y métricas para validar su cumplimiento (SLI). Un ejemplo del tipo de información que incluye un SLA:

- Tasa de transferencia garantizada: 100 Mbps.
- Tasa de transferencia máxima: 200 Mbps.
- Latencia máxima: 20 ms.
- *jitter* máximo: 5 ms.
- Tasa máxima de pérdida de paquetes: 0,1 %.
- Disponibilidad: 99,95 %.

Este capítulo estudia estos parámetros, su importancia, causas y consecuencias, y la forma de medirlos; qué es y cómo funciona un pipeline QoS, qué componentes lo forman y qué puede ofrecer; y cómo Linux implementa QoS y cómo se configura.

17.1. Latencia y *jitter*

La latencia se ha mencionado ya de forma informal en capítulos anteriores; conviene ahora dar una definición concreta y, de paso, la de algunos conceptos relacionados.

El **retardo** *(delay)*, erróneamente utilizado como sinónimo de latencia, se refiere al tiempo que le lleva al mensaje atravesar un elemento específico. Se consideran varios tipos de retardo: de procesamiento, de transmisión, de cola, de propagación, etc.

La **latencia** es el tiempo que le lleva a un mensaje viajar desde el emisor hasta el destino, es decir, es la suma de los retardos de todos los elementos que componen el camino.

El **tiempo de respuesta** *(response time)* es el tiempo total que transcurre desde que la solicitud sale del emisor hasta que la respuesta le llega de vuelta. Incluye el retardo de la red en el camino de ida y vuelta, el tiempo de procesamiento del servidor y el que requiere el SO de cada nodo para procesar ambos mensajes.

El **tiempo de ida y vuelta** (RTT) es el tiempo que tarda un mensaje en ir del emisor al destino y regresar al emisor. A diferencia del tiempo de respuesta, en este solo se contabiliza el tiempo de transmisión.

El ***jitter*** es la variación del retardo o la latencia, específicamente en relación a los otros mensajes del mismo flujo.

17.1.1. Causas del retardo y del *jitter*

De los tipos de retardo, el más relacionado con la red es el de propagación. Es el tiempo que le lleva a un mensaje viajar a través del medio físico

en el enlace. Se debe a dos factores principales: la distancia física entre los extremos y la velocidad de propagación de la señal en ese medio de transmisión. Se calcula como d/v siendo d la longitud del enlace y v la velocidad de propagación.

El retardo de transmisión es el tiempo que lleva colocar los bits del mensaje en el canal por medio de la interfaz de red. Se calcula como L/R siendo L el tamaño del mensaje (en bits) y R la velocidad de transmisión.

Por ejemplo, la latencia para enviar un mensaje por un canal de fibra óptica de 50 km, con velocidad de propagación de $2 \cdot 10^8$ m/s y un mensaje de 1 kb a través de un enlace de 1 Gbps, sería:

$$\text{latencia} = \text{retardo de transmisión} + \text{retardo de propagación}$$

$$\text{latencia} = \frac{1 \cdot 10^3}{1 \cdot 10^9} + \frac{50 \cdot 10^3}{2 \cdot 10^8} = 1\ \mu\text{s} + 0{,}25\ \text{ms} \approx 251\ \mu\text{s}$$

Este cálculo desprecia el retardo de procesamiento y de cola que ocurre en el emisor para preparar el mensaje y pedirle al SO que lo envíe a través de un socket, así como la gestión del SO con la NIC. También asume que no hay dispositivos intermedios como routers o switches. El cálculo de la latencia real (extremo a extremo) depende de otros elementos y factores en una situación real.

Ambos retardos de transmisión y propagación son bastante deterministas y dependen únicamente de valores conocidos y fijos. El retardo de procesamiento por otra parte puede variar dependiendo de la carga del nodo e incluso de los propios datos si requiere por ejemplo tareas de codificación o cifrado.

Por último, el retardo de cola *(queuing delay)* es el tiempo que el mensaje tiene que esperar en las colas de salida y entrada de los nodos origen y destino, y de dispositivos de interconexión. Es el más variable porque depende de decisiones de los algoritmos involucrados aparte de por la propia carga de la red. La variación del retardo de cola es la causa principal del *jitter*. Si en un flujo, uno de los paquetes se ve obligado a esperar en la cola de un router durante 10ms y el siguiente solo tiene que esperar 2ms, está introduciendo una variación (*jitter*) de 8ms.

El *jitter* es especialmente perjudicial para las aplicaciones que reproducen un flujo en tiempo real, como la telefonía IP, el streaming multimedia o los videojuegos en línea, porque necesitan entregar los datos a un ritmo constante. Para absorber esa variación, estas aplicaciones intercalan un *jitter buffer* entre la recepción y la reproducción: retienen brevemente los

mensajes recibidos y los entregan a la aplicación al ritmo de consumo del reproductor. Cuanto mayor es el *jitter*, mayor debe ser el buffer para así evitar cortes en la reproducción, aunque eso implica retardo adicional. Por esta razón, el *jitter* resulta may problemático para el tráfico en tiempo real mientras que es prácticamente inocuo para una transferencia masiva de datos *(bulk data)*, donde los mensajes se pueden almacenar y reordenar sin ninguna urgencia.

17.1.2. Medición del retardo

Los distintos tipos de retardo se pueden calcular en términos teóricos, pero es complicado medirlos en la práctica porque implicaría tener puntos de control sincronizados en todos los componentes involucrados. También la latencia es difícil de calcular ya que es necesario que emisor y receptor cuenten con relojes locales sincronizados con suficiente precisión. Los protocolos de sincronización como NTP tienen una precisión limitada precisamente por esa misma latencia. Para lograr ese tipo de sincronización se requiere una fuente externa de sincronización como GPS.

Por todo esto, es mucho más sencillo y práctico medir tiempo de ida y vuelta, como hace `ping`, y estimar la latencia como la mitad de ese valor, aunque teniendo muy claro que no es exacto: la latencia de uno de los sentidos podría ser mayor que la del otro, y no hay forma de saberlo.

```
$ ping example.net
PING example.net (172.66.175.59) 56(84) bytes of data.
64 bytes from 172.66.175.59: icmp_seq=1 ttl=57 time=8.24 ms
64 bytes from 172.66.175.59: icmp_seq=2 ttl=57 time=8.90 ms
64 bytes from 172.66.175.59: icmp_seq=3 ttl=57 time=10.1 ms
64 bytes from 172.66.175.59: icmp_seq=4 ttl=57 time=7.21 ms
^C
--- example.net ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3004ms
rtt min/avg/max/mdev = 7.206/8.606/10.083/1.044 ms
```

LISTADO 17.1: Medición de RTT con `ping`.

17.1.3. Cálculo del *jitter*

Si estimamos la latencia como RTT/2, deberemos estimar el *jitter* como la mitad de la variación del RTT. La forma más simple para obtener el *jitter* es calcular las diferencias entre medidas consecutivas de RTT y calcular después una media.

$$J = \frac{1}{2}\frac{1}{N-1}\sum_{i=2}^{N}|RTT_i - RTT_{i-1}| \tag{17.1}$$

donde:

J es el *jitter*, RTT_i es la medida de RTT del paquete i, y N es el número total de paquetes.

Aplicada a los datos de la captura 17.1, el *jitter* que obtenemos es:

$$\begin{aligned} J &= \frac{1}{2}\frac{1}{4-1}\left(|8{,}90 - 8{,}24| + |10{,}1 - 8{,}90| + |7{,}21 - 10{,}1|\right) \\ &= \frac{1}{6}\left(0{,}66 + 1{,}20 + 2{,}89\right) = \frac{1}{6}\cdot 4{,}75 \\ &\approx 0{,}792 \text{ ms} \end{aligned}$$

Sin embargo, el *jitter* se suele calcular a menudo como una EMA denominada *interarrival jitter*. Es la fórmula que aplica RTP tal como está definida en la RFC 1889 [53]. De ese modo, el cálculo es más estable y resiste mejor el ruido (valores atípicos puntuales). La fórmula es:

$$J_i = \frac{15}{16}J_{i-1} + \frac{1}{16}|L_i - L_{i-1}| \tag{17.2}$$

donde:

L es la latencia, J_{i-1} es el *jitter* previamente calculado y 1/16 es el factor de suavizado.

ECUACIÓN 17.1: Cálculo del *jitter* como media móvil exponencial.

Sin embargo, no es eso lo que hace `ping`. El valor `mdev` que muestra en sus estadísticas (Listado 17.1) no es el *jitter* sino la desviación estándar de los RTT, que también da una buena idea de la dispersión de la latencia.

$$\text{mdev} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}\left(RTT_i - \overline{RTT}\right)^2} \tag{17.3}$$

donde:

$\overline{RTT}$ es el RTT promedio y N es el número de muestras.

Aplicando esta fórmula a los datos de la captura 17.1 se obtiene:

$$\begin{aligned}\text{mdev} &= \sqrt{\frac{(8{,}24-8{,}6125)^2+(8{,}90-8{,}6125)^2+(10{,}1-8{,}6125)^2+(7{,}21-8{,}6125)^2}{4}}\\ &= \sqrt{\frac{0{,}1388+0{,}0827+2{,}2127+1{,}9670}{4}}\\ &\approx 1{,}049 \text{ ms}\end{aligned}$$

La pequeña diferencia con el valor que aparece en la salida de `ping` (1,044) se debe al redondeo que realiza el programa al mostrar los datos.

Con `mtr` también puedes medir RTT y desviación típica, con la diferencia de que este realiza la medida para todos los routers intermedios.

```
~$ mtr --report-wide --report-cycles 4 sidney.example.net
Start: 2026-07-15T16:31:23+0200
HOST: amy                Loss%  Snt  Last   Avg  Best  Wrst StDev
  1.|-- _gateway          0.0%    4   2.5   6.0   2.5  14.5   5.7
  2.|-- 192.168.0.1       0.0%    4  10.5   5.6   3.5  10.5   3.3
  3.|-- ???              100.0    4   0.0   0.0   0.0   0.0   0.0
  4.|-- 188.114.108.49   75.0%    4   8.2   8.2   8.2   8.2   0.0
  5.|-- 188.114.108.48    0.0%    4  16.5  10.4   8.2  16.5   4.0
  6.|-- 188.114.108.1     0.0%    4  20.1  12.2   7.9  20.1   5.5
  7.|-- 188.114.108.7     0.0%    4  10.0  31.9  10.0  58.4  25.3
  8.|-- 172.66.175.59     0.0%    4   8.5   9.1   7.4  12.9   2.6
```

El programa `iperf3` sí ofrece una medición de *jitter* canónica aplicando la Fórmula 17.1. Aunque este es un programa cliente-servidor puede medir el *jitter* sin necesidad de sincronización precisa de los relojes. Como el cálculo depende de diferencias entre medidas, la discrepancia entre relojes local y remoto *(drift)* no afecta al resultado.

```
~$ iperf3 --time 5 --bitrate 500M --client sidney.example.net --udp
Connecting to host sidney.example.net, port 5201
[  5] local 192.168.0.37 port 54954 connected to sidney.example.net port 5201
[ ID] Interval      Transfer     Bitrate    otal Datagrams
[  5]   0.00-1.00   7.30 MBytes  61.2 Mbps  5314
[  5]   1.00-2.00   6.90 MBytes  57.9 Mbps  5028
[  5]   2.00-3.00   7.15 MBytes  59.9 Mbps  5203
[  5]   3.00-4.00   7.07 MBytes  59.3 Mbps  5148
[  5]   4.00-5.00   6.29 MBytes  52.7 Mbps  4578
- - - - - - - - - - - - - - - - - - - - - - - - -
[ ID] Interval      Transfer     Bitrate    Jitter    Lost/Total Datagrams
[  5]   0.00-5.00   34.7 MBytes  58.2 Mbps  0.000 ms  0/25271 (0%)  sender
[  5]   0.00-5.09   34.7 MBytes  57.2 Mbps  0.141 ms  0/25269 (0%)  receiver
```

17.2. Cálculo de la tasa de transferencia

La tasa de transferencia mide la velocidad con la que se mueven los datos, ya sea a través de una red, un enlace, un flujo UDP, una conexión TCP, etc.

La tasa se mide en bits por segundo (b/s o bps), bytes por segundo (B/s) y sus múltiplos (ver § D.3).

Se distingue entre dos tipos de tasa:

Throughput
: o tasa bruta, es la cantidad total de datos transmitidos en un período de tiempo. Además de la carga útil, incluye cabeceras, reconocimientos, retransmisiones o cualquier otro tipo de información de control; todo eso que llamamos sobrecarga de protocolos.

Goodput
: o tasa neta, considera solo los datos efectivos, la carga útil procedente del usuario o la aplicación.

En realidad, ambas tasas se pueden medir en distintas capas, considerando diferentes cabeceras y sus mecanismos de control. La tasa bruta de más bajo nivel debería contabilizar hasta las cabeceras de enlace, por lo que tiene que medirse directamente desde la interfaz de red.

La tasa neta se suele medir sobre el nivel de transporte, es decir, con los datos que se envían o reciben con las primitivas del socket (como veremos en los ejemplos de código) o sobre el nivel de aplicación, que sería el caso en el que únicamente se cuentan los datos relevantes para el usuario, como por ejemplo, un fichero descargado desde un servidor web con HTTP.

También es útil distinguir la tasa en los extremos y en la red:

Tasa de envío (T_s)
: es la velocidad con la que el proceso emisor entrega datos al SO a través de un socket, lo que, en el caso de TCP, no implica necesariamente que esos datos estén saliendo a la red y puede que simplemente se estén almacenando en el buffer de envío.

Tasa de recepción[2] **(T_r)**
: es la velocidad con la que el proceso receptor recoge datos a través de un socket, que con TCP implica leer datos del buffer de recepción.

Tasa de red[3] **(T_n)**
: es la velocidad a la que los datos se mueven a través de la red. Esta tasa no se puede medir directamente en los extremos, pero se puede estimar a partir de las tasas de envío y recepción.

[2] en inglés: *drain rate*
[3] en inglés: *network rate*

Lógicamente para cualquiera de las tres, pueden ser útiles tanto la tasa bruta como la neta. Los datos se envían en bloques (segmentos o datagramas) que pueden tener tamaños distintos y que son enviados/recibidos a intervalos posiblemente irregulares. Eso condiciona el cálculo y da lugar a los métodos más habituales, descritos a continuación.

17.2.1. Tasa instantánea

Es el cálculo más sencillo, y probablemente también el menos útil. Consiste en contabilizar únicamente los datos transmitidos desde la medición anterior considerando el tiempo transcurrido desde entonces. Formalmente:

$$R_{\text{inst}} = \frac{\Delta B}{\Delta t} \tag{17.4}$$

donde:

> R_{inst} es la tasa *(rate)* instantánea (bytes por segundo), ΔB es la cantidad de datos transmitida en el último bloque (bytes), y Δt es el tiempo transcurrido desde el envío del bloque anterior (segundos).

La tasa instantánea puede variar drásticamente de un mensaje al siguiente. Se dice que es muy «sensible al ruido», y por eso raramente resulta útil.

En Python la tasa de envío neta instantánea podría calcularse como en el siguiente fragmento[4]. Fíjate que usamos `time.monotonic()` en lugar de `time.time()` para que el cálculo de los delta no se vea afectado por ajustes en el reloj del sistema.

```
while 1:
    last_time = time.monotonic()
    data = get_data()
    sock.sendall(data)
    byte_rate = len(data) / (time.monotonic() - last_time)
    print(byte_rate)
```

17.2.2. Promedio acumulado (CA)

En el otro extremo de la tasa instantánea está el promedio total acumulado o CA (Cumulative Average). Consiste simplemente en calcular la media de toda la transmisión hasta el momento, es decir, el total de datos transmitidos partido por el tiempo transcurrido desde el inicio.

$$R_{\text{avg}}(t) = \frac{B(t) - B(t_0)}{t - t_0} \tag{17.5}$$

[4] Adaptarlo a la tasa de recepción es directo

donde:

> $R_{\text{avg}}(t)$ es la tasa media acumulada (bytes por segundo), $B(t)$ es la cantidad total de datos transmitida hasta ahora (bytes), $B(t_0)$ es la cantidad total de datos transmitida en el instante t_0, y $t - t_0$ es el tiempo transcurrido desde el comienzo (segundos).

Lógicamente sufre del efecto contrario a la tasa instantánea. No se adapta en absoluto a las variaciones (nada reactivo), por lo que una reducción o aumento momentáneo de la tasa queda completamente enmascarado.

En Python:

```
start_time = time.monotonic()
sent = 0
while 1:
    data = get_data()
    sock.sendall(data)
    sent += len(data)
    elapsed = time.monotonic() - start_time
    byte_rate = sent / elapsed
    print(byte_rate)
```

LISTADO 17.2: Promedio acumulado

17.2.3. Media móvil simple (SMA)

Una técnica intermedia que ofrece información actualizada, pero más estable que la tasa instantánea, es la 'media móvil simple' o SMA (Simple Moving Average). Se basa en definir una ventana de tiempo que abarca solo los w segundos previos considerando únicamente los datos transmitidos en ese lapso.

$$R_{\text{SMA}}(t) = \frac{B(t) - B(t-w)}{w} \tag{17.6}$$

donde:

> $R_{\text{SMA}}(t)$ es la tasa media móvil simple (bytes por segundo), $B(t)$ es la cantidad total de datos transmitida hasta ahora (bytes), $B(t-w)$ es la cantidad total de datos transmitida hace w segundos, y w es el tamaño de la ventana (segundos).

Aunque es una medida mucho más reactiva que el promedio acumulado, los nuevos datos tardan algún tiempo ($w/2$ segundos) en reflejarse en la tasa.

En Python:

```
class SMA_RateMeter:
    def __init__(self, window_size=5):
        self.window_size = window_size
        self.window = collections.deque()
```

```
    def update(self, sent_bytes):
        now = time.monotonic()
        self.window.append((now, sent_bytes))

        # Eliminar datos fuera de la ventana
        while self.window and (now - self.window[0][0]) > self.window_size:
            self.window.popleft()

    @property
    def rate(self):
        if not self.window:
            return 0

        window_bytes = sum(b for t, b in self.window)
        elapsed = self.window[-1][0] - self.window[0][0]
        return window_bytes / elapsed if elapsed > 0 else 0

sma = SMA_RateMeter(window_size=5)
while 1:
    data = get_data()
    sock.sendall(data)
    sma.update(len(data))
    print(sma.rate)
```

17.2.4. Media móvil exponencial (EMA)

SMA también tiene un problema: los datos nuevos de la ventana tienen el mismo peso que los antiguos, por lo que la tasa no refleja bien las variaciones en el tráfico (sigue siendo poco reactivo). Para paliar ese efecto se puede usar una 'media móvil exponencial', o EMA (Exponential Moving Average), que aplica un decaimiento exponencial, asignando menos peso a los datos más antiguos.

Se calcula utilizando la siguiente fórmula:

$$R_{\text{EMA}}(t) = \alpha \cdot R_{\text{inst}}(t) + (1 - \alpha) \cdot R_{\text{EMA}}(t-1) \tag{17.7}$$

donde:

> $R_{\text{EMA}}(t)$ es la tasa media móvil exponencial, $R_{\text{inst}}(t)$ es la tasa instantánea en el instante t, y α es el factor de suavizado ($0 < \alpha < 1$).

El factor de suavizado α permite ajustar la reactividad del cálculo. Así un valor de 1 la hace equivalente a la tasa instantánea, y valores menores aumentan el peso de los cálculos anteriores. Un valor típico es $\alpha = 0{,}1$.

En Python, una implementación básica del EMA podría ser:

```
class EMA_RateMeter:
    def __init__(self, alpha=0.1):
        self.alpha = alpha
```

```
        self.rate = 0
        self.last = time.monotonic()

    def update(self, sent_bytes):
        elapsed = time.monotonic() - self.last
        self.rate = self.alpha * sent_bytes / elapsed + (1 - self.alpha) * self.rate
        self.last = time.monotonic()

ema = EMA_RateMeter()
while 1:
    data = get_data()
    sock.sendall(data)
    ema.update(len(data))
    print(ema.rate)
```

17.2.5. Consideraciones sobre el cálculo de tasa

Tampoco EMA es perfecto. Al comienzo de una conexión TCP, el emisor puede enviar —invocando `send()`— una gran cantidad de datos en muy poco tiempo porque en realidad esos datos se almacenan en el buffer de envío y recepción, pero no están siendo aún consumidos realmente por el proceso receptor.

Esto genera cálculos de tasa instantánea (en los que se basa EMA) enormes e irreales, porque a fin de cuentas lo que se está midiendo en esos primeros instantes es la tasa entre el proceso y la memoria local y remota, pero no entre los procesos emisor y receptor, que es lo que proporciona una medida representativa. Estos valores tan altos falsean el resultado, e incluso con el decaimiento exponencial, puede llevar mucho tiempo compensar ese efecto. En esta situación no mejora la reactividad, que acaba siendo tan pobre como la del promedio acumulado.

Hay algunas opciones que pueden ayudar a paliar este problema:

- Ignorar la tasa instantánea los primeros segundos de la conexión (período de calentamiento o *warmup*).
- Descartar lapsos (Δt) demasiado pequeños.
- Utilizar un α bajo para Δt pequeños. Se puede calcular como $\alpha = 1 - e^{-dt/\tau}$ siendo τ una constante que determina la reactividad.
- Reducir el tamaño de los buffers TCP de envío y recepción.
- No usar EMA en el emisor.
- Usar SMA en lugar de EMA durante el calentamiento o si no se necesita alta reactividad.

Aplicaremos algunas de estas técnicas a los ejemplos de este capítulo.

17.3. Control de flujo TCP como limitador de tasa

Sabemos que el control de flujo TCP permite al receptor controlar la cantidad de datos que el emisor le envía de acuerdo al espacio del que dispone (la ventana que anuncia). Aunque no es su objetivo principal, este mecanismo se puede utilizar para conseguir un control de la tasa de transferencia rudimentario.

Considerando solo uno de los sentidos de la comunicación, asumamos por un momento que el emisor siempre tiene datos disponibles y los puede enviar siempre que sea posible. Con esta premisa se pueden dar tres situaciones en función de la tasa de recepción (T_r) respecto a la velocidad de la red (T_n). Lógicamente la tasa de emisión (T_s) estará limitada por la menor de las otras dos.

- Si $T_r < T_n$, el buffer de recepción se llena, la ventana se cierra, el buffer de emisión también se llena y el emisor se bloquea *(stall)* en espera de que se libere espacio cuando la ventana vuelva a abrir. En esta situación, la tasa de emisión está limitada por el proceso receptor.
- Si $T_r > T_n$, el buffer de recepción queda vacío y el receptor quedará bloqueado en espera de nuevos datos. En esta situación, la tasa de emisión está limitada por la red.
- Si $T_r \approx T_n$, el buffer de recepción nunca se llena ni queda vacío. Ni el emisor ni el receptor se bloquean.

Lógicamente, los buffers de emisión y recepción existen precisamente porque el tercer caso, aunque deseable, es poco frecuente. Estas tres velocidades: emisión, red y recepción suelen ser dispares y cambiantes. Los buffers amortiguan esas diferencias y permiten un flujo más estable. Sin embargo, obviando otros controles —como el de congestión— cuando la ventana está abierta el emisor TCP envía datos a la máxima tasa posible en ese momento, y eso frecuentemente provoca bloqueos en ambos extremos, lo que influye en parte en que T_r y T_n varíen constantemente. Además el algoritmo de Nagle trata de evitar segmentos pequeños, lo que alarga esos bloqueos. La conclusión es que cuando existe disparidad en el desempeño de emisor y receptor, el mecanismo de control de flujo (y en menor medida el de congestión) provoca un flujo a ráfagas, es decir, momentos en los que se envían datos a la máxima velocidad posible y momentos en los que no se envía nada (porque la ventana está cerrada).

La consecuencia directa de todo esto es que si el receptor limita intencionadamente la tasa de recepción, la tasa media de emisión se verá también limitada, aunque su dispersión (*p. ej.* la desviación típica) será muy alta debido al flujo en ráfagas.

En principio una limitación de tasa de este tipo no es incorrecta, pero cuando ocurre de forma generalizada puede provocar efectos perjudiciales como el aumento de la latencia o el *jitter*. Lo veremos más adelante.

17.3.1. Limitación de tasa en el receptor

Se puede conseguir bloqueando intencionadamente el proceso con `time.sleep()` el tiempo necesario para aproximar la tasa medida a la tasa deseada. El siguiente código calcula la tasa con el método de promedio acumulado (`ca_rate`), aunque puede sustituirse por cualquier otro método de cálculo de tasa.

```
1  target_rate_kBps = 200 * 1000  # 200 kB/s
2  start_time = time.monotonic()
3  received = 0
4  while 1:
5      data = sock.recv(4096)
6      if not data:
7          break
8
9      received += len(data)
10     elapsed = time.monotonic() - start_time
11     ca_rate = received / elapsed
12
13     if ca_rate <= target_rate_kBps:
14         continue
15
16     adjust_time = (received / target_rate_kBps) - elapsed
17     if adjust_time > 0:
18         time.sleep(adjust_time)
```

LISTADO 17.3: Limitación de la tasa de recepción (medida con promedio acumulado)

Hasta la **línea 14** es equivalente al Listado 17.2 salvo que aquí estamos calculando la tasa de recepción. Si este cálculo está por debajo de la tasa objetivo (`target_rate_kBps`) no hace nada, pero si está por encima, realiza un ajuste aumentando el tiempo (el denominador de la Fórmula 17.5). El ajuste se obtiene como la diferencia entre el tiempo realmente transcurrido y el tiempo que corresponde a la tasa objetivo (**línea 16**).

El ejemplo del directorio /`flow-control`, ya utilizado en el capítulo 12, ilustra cómo lograrlo. El emisor (cliente) envía datos al receptor (servidor) tan rápido como puede, pero el receptor limita la tasa aplicando la técnica anterior. Ambos programas calculan la tasa de envío y recepción respectivamente mediante promedio acumulado (§ 17.2.2).

Cliente

```
~$ ./client.py 127.0.0.1 2000
SNDBUF: 2,626,560 bytes
 (-) sent:14,550.0 kB, CA:245.2 kB/s
```

Servidor

```
~$ ./server.py --limit 200 2000
Client connected: ('127.0.0.1', 42244)
RCVBUF: 131,072 bytes
 received:11,869.8 kB, CA:200.1 kB/s
```

FIGURA 17.1: Limitación de la tasa en el receptor
/flow-control

Puedes probar el ejemplo con los siguientes comandos. El segundo parámetro del servidor es la tasa máxima deseada: 200 kB/s.

Al ejecutarlo, la transferencia en el lado del cliente se detiene durante unos segundos de vez en cuando. El receptor no para en ningún momento porque sigue consumiendo los datos que hay en el buffer de recepción. Cuando se libera suficiente espacio, la ventana se abre y el emisor envía nuevos datos durante un tiempo, hasta que el buffer se llena y la ventana se cierra de nuevo.

La Figura 17.2 es un esquema de su funcionamiento y la Figura 17.3 muestra la cantidad de datos enviados por el emisor a partir de la información obtenida de su ejecución (almacenada en el archivo `client-stats.csv`). El diagrama en escalera permite apreciar claramente los períodos de bloqueo *(stall)* debidos al cierre de la ventana.

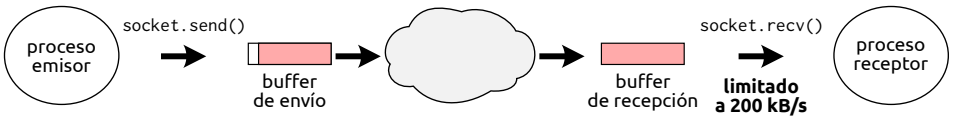


FIGURA 17.2: Limitación de tasa en el receptor

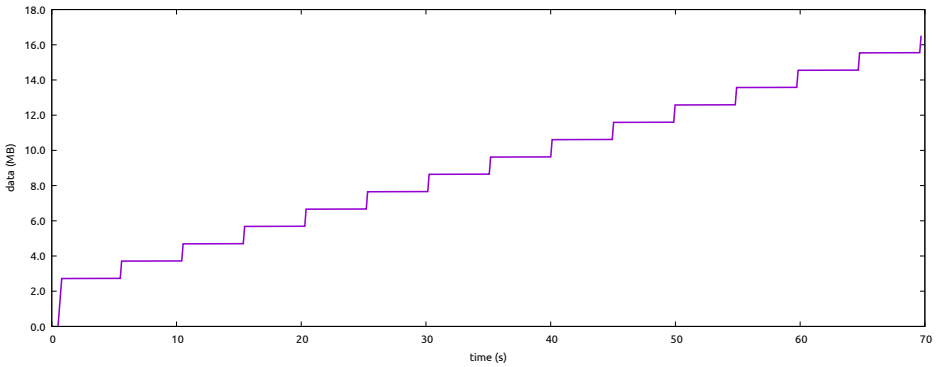


FIGURA 17.3: Datos enviados por el emisor

En la evolución de la tasa se aprecia claramente el efecto del calentamiento o *warmup* (§ 17.2.5). Debido a que los buffers de envío y recepción son grandes: 2627 kB y 131 kB respectivamente[5], el emisor al comienzo puede enviar muchos datos ($\sim$ 2,7 MB) muy rápido. Una vez llenos los buffers, bloquea por el cierre de la ventana. Con el tiempo la tasa media se irá aproximando a los 200 kB/s impuestos por el receptor. Puedes ver esa evolución en la gráfica 17.4 obtenida también a partir de los datos generados por el emisor.

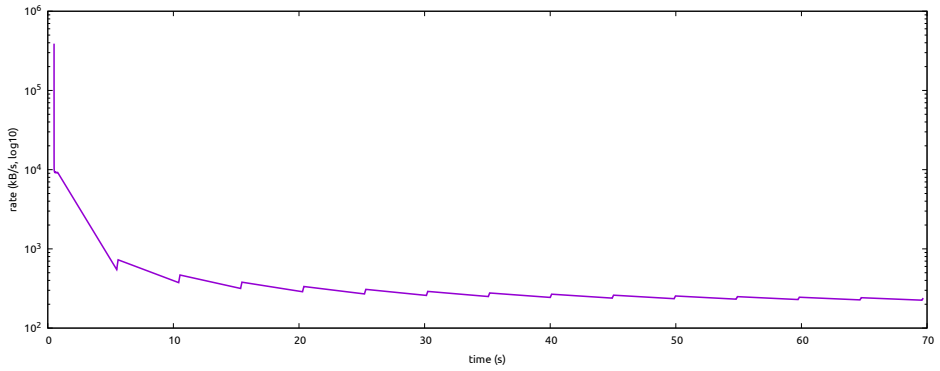


FIGURA 17.4: Tasa de envío medida en el emisor (promedio acumulado)

Los dientes de sierra se deben a que el emisor envía muchos datos en poco tiempo y bloquea (rampa ascendente); cuando reanuda, han pasado unos segundos por lo que la tasa media baja un poco (rampa descendente). Usando otro método para el cálculo de tasa, lógicamente obtendríamos gráficas sin este efecto.

Este mismo servidor proporciona alternativamente un modo diferente de trabajo (que se activa con `--step`). Este argumento permite especificar una cantidad de bytes, que una vez recibidos hacen que el servidor se detenga en espera de que el usuario pulse `ENTER` para continuar. De ese modo puedes comprobar fácilmente cómo el emisor bloquea al llenarse los buffers de envío y recepción y no continúa hasta que quede espacio libre en el receptor. Esta lógica está implementada en el método `step_receiving()` (Listado 17.4).

```
159     def step_receiving(self):
160         input('Press ENTER to receive > ')
161         received = 0
162         while 1:
163             count = 0
164             pending = self.step_size * 1000
```

[5]Esos son los valores por defecto en GNU/Linux para `localhost`.

```
165             while count < pending:
166                 data = self.conn.recv(min(4096, pending - count))
167                 if not data:
168                     return
169                 count += len(data)
170             received += count
171
172             input(f'received: {received//1000:,} kB > ')
```

Listado 17.4: Servidor con control manual de la recepción
/flow-control/server.py

En los ejemplos previos el cliente simplemente envía datos sin sentido, y el servidor los descarta. Sin embargo, tienen otro modo que permite indicar al cliente que consuma datos desde su entrada estándar (con `--stdin`) y al servidor que envíe los datos recibidos a su salida estándar (con `--stdout`). De este modo podemos crear un sistema sencillo para transmitir un fichero `mp3` a través de la red, que será consumido en el servidor directamente por un reproductor de audio, vía redirección. Obviamente para que funcione correctamente, el cliente debe enviar un archivo adecuado (un `mp3`). Este escenario es interesante porque es el reproductor el que determina la tasa de recepción en función de la calidad y compresión del archivo (su *bitrate*). Puedes probar esa posibilidad con estos comandos:

```
~$ ./server.py --stdout --rcvbuf 4000 2000 | mpg123 -q -
~$ ./client.py --stdin --sndbuf 4000 < audio.mp3
```

Figura 17.5: Tasa limitada por el reproductor de `mp3`
/flow-control-player

Fíjate en las opciones `--sndbuf` y `--rcvbuf` que establecen el tamaño de los respectivos buffers de envío y recepción. Estos valores favorecen períodos de bloqueo más cortos que ayudan a entender cómo funciona la transferencia.

Tanto el cliente como el servidor muestran la tasa con promedio acumulado (CA). El servidor además puede mostrar la tasa SMA si se activa el argumento `--sma` y la tasa EMA con compensación de calentamiento basada en el ajuste de α respecto Δt si se activa el argumento `--ema`. La Figura 17.7 representa ambas tasas para comparación.

En la ejecución de la Figura 17.5 la tasa de recepción ronda los 41 kB/s (328 kbps), muy próxima al bitrate con el que está codificado el `mp3` con el que hemos realizado la prueba: 320 kbps.

```
flow-control$$ ./server.py --ema --sma --stdout --rcvbuf 4000 2000 | mpg123 -q -
Client connected: ('127.0.0.1', 52406)
RCVBUF: 8,000 bytes
 received:968.7 kB, CA:43.0 kB/s, EMA:40.8 kB/s, SMA:41.0 kB/s
```

FIGURA 17.6: Opciones para medición de tasa en el servidor: EMA y SMA /flow-control-player/server.py

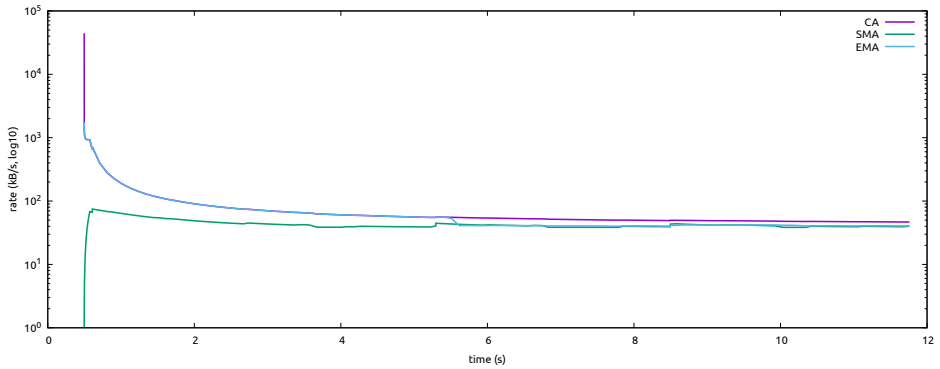


FIGURA 17.7: Comparación de CA, SMA y EMA en el servidor

La consecuencia de tener el reproductor en el receptor es la misma que en los otros ejemplos. Como la tasa de recepción es menor que la tasa de emisión, ambos buffers se llenan y el emisor queda eventualmente bloqueado de manera intermitente. En este caso, también el receptor puede quedar bloqueado porque la entrada estándar del reproductor dispone de un buffer adicional, con lo que la llamada `stdout.flush()` puede bloquear el proceso receptor hasta que haya espacio libre en ese buffer. Ten en cuenta que esto ocurre porque cliente y servidor se están ejecutando en el mismo nodo y se comunican a través de `localhost`. En una conexión remota y en función del ancho de banda disponible, este comportamiento puede variar mucho, hasta el punto de no ocurrir bloqueos si ese ancho de banda es similar al bitrate al que el reproductor consume el flujo.

```
~$ file audio.mp3
audio.mp3: Audio file with ID3 version 2.4.0, contains: MPEG ADTS, layer III, v1,
320 kbps, 44.1 kHz, JntStereo
```

FIGURA 17.8: Información del `mp3` utilizado en la prueba

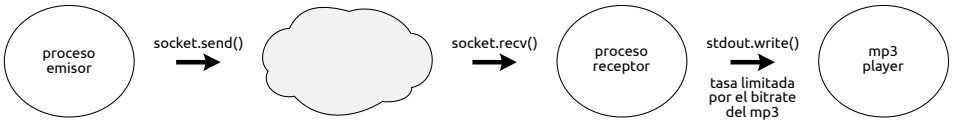


FIGURA 17.9: Datos enviados por el emisor

17.3.2. Limitación de tasa en el emisor

Por supuesto, el emisor puede aplicar exactamente la misma técnica intercalando pausas para conseguir una tasa de emisión concreta. Sin embargo, en este caso la tasa será mucho más estable. El emisor puede enviar bloques de datos más pequeños más a menudo, lo que reduce mucho la dispersión.

El problema es que el emisor no sabe a qué tasa consume el receptor. Si se excede, en algún momento se llenarán los buffers y se bloqueará el emisor. Si es demasiado baja, será el receptor el que se bloquee en espera de datos nuevos y la reproducción del audio se interrumpirá.

17.4. Medición de tasa de transferencia y de pérdida

El programa `iperf3` es una excelente herramienta para medir la tasa de transferencia entre dos nodos y también la tasa de pérdida de paquetes. Se puede utilizar con UDP y TCP con múltiples flujos de datos concurrentes, en uno o en ambos sentidos. Es posible establecer la duración de la prueba tanto en tiempo como en cantidad de datos y fijar la tasa de transferencia deseada, incluso saturando la ruta. La salida del programa muestra tasa de transferencia, latencia y *jitter*.

El programa calcula y muestra la tasa de transferencia instantánea en intervalos de 1s (por defecto). Al acabar muestra el promedio acumulado de toda la prueba.

Este ejemplo muestra las medidas que realizan tanto el servidor como el cliente para un flujo UDP con una tasa solicitada de 50 Mbps, que claramente la ruta puede satisfacer. Los datos van únicamente del cliente al servidor.

```
alice@sidney:~$ iperf3 --server
Accepted connection from 203.0.113.2, port 57850
[  5] local sidney.example.net port 5201 connected to 203.0.113.2 port 47103
[ ID] Interval      Transfer     Bitrate         Jitter    Lost/Total Datagrams
[  5]   0.00-1.00   5.76 MBytes  48.3 Mbps  0.113 ms  0/4196 (0%)
[  5]   1.00-2.00   5.96 MBytes  50.0 Mbps  0.114 ms  1/4339 (0.023%)
[  5]   2.00-3.00   5.96 MBytes  50.0 Mbps  0.152 ms  0/4340 (0%)
[  5]   3.00-4.00   5.96 MBytes  50.0 Mbps  0.078 ms  0/4339 (0%)
[  5]   4.00-5.00   5.96 MBytes  50.0 Mbps  0.093 ms  0/4341 (0%)
[  5]   5.00-5.03    207 KBytes  50.1 Mbps  0.175 ms  0/147 (0%)
```

```
- - - - - - - - - - - - - - - - - - - - - - - - -
[ ID] Interval      Transfer     Bitrate    Jitter    Lost/Total Datagrams
[  5]   0.00-5.03   29.8 MBytes  49.7 Mbps  0.175 ms  1/21702 (0.0046%) receiver
```

```
$ iperf3 --time 5 --bitrate 50M --client sidney.example.net --udp
Connecting to host sidney.example.net, port 5201
[  5] local 192.168.0.37 port 47103 connected to sidney.example.net port 5201
[ ID] Interval      Transfer     Bitrate         Total Datagrams
[  5]   0.00-1.00   5.97 MBytes  50.0 Mbps  4345
[  5]   1.00-2.00   5.96 MBytes  50.0 Mbps  4340
[  5]   2.00-3.00   5.96 MBytes  50.0 Mbps  4341
[  5]   3.00-4.00   5.96 MBytes  50.0 Mbps  4340
[  5]   4.00-5.00   5.96 MBytes  50.0 Mbps  4337
- - - - - - - - - - - - - - - - - - - - - - - - -
[ ID] Interval      Transfer     Bitrate    Jitter    Lost/Total Datagrams
[  5]   0.00-5.00   29.8 MBytes  50.0 Mbps  0.000 ms  0/21703 (0%)  sender
[  5]   0.00-5.03   29.8 MBytes  49.7 Mbps  0.175 ms  1/21702 (0.0046%) receiver
```

Para cada intervalo, aparece la cantidad de datos enviados o recibidos y la tasa media. Al terminar, muestra la cantidad total y la media de toda la prueba. Al terminar, el servidor envía al cliente los datos de resumen, por eso el cliente muestra una línea `sender` y otra `receiver`. Únicamente se perdió un datagrama UDP (última línea).

En esta segunda prueba, el bitrate solicitado es mucho mayor: 1 Gbps. La ruta no puede satisfacer esa demanda, se queda en 90,6 Mbps, y por eso se pierden muchos datagramas, concretamente el 90 %.

```
$ iperf3 --time 5 --bitrate 1G --client sidney.example.net --udp
Connecting to host sidney.example.net, port 5201
[  5] local 192.168.0.37 port 55549 connected to sidney.example.net port 5201
[ ID] Interval      Transfer     Bitrate         Total Datagrams
[  5]   0.00-1.00   114 MBytes   957 Mbps  83134
[  5]   1.00-2.00   114 MBytes   956 Mbps  82973
[  5]   2.00-3.00   114 MBytes   956 Mbps  82968
[  5]   3.00-4.00   114 MBytes   956 Mbps  82976
[  5]   4.00-5.00   114 MBytes   956 Mbps  83013
- - - - - - - - - - - - - - - - - - - - - - - - -
[ ID] Interval      Transfer     Bitrate    Jitter    Lost/Total Datagrams
[  5]   0.00-5.00    570 MBytes   956 Mbps  0.000 ms  0/415064 (0%)  sender
[  5]   0.00-5.28   57.1 MBytes  90.6 Mbps  0.190 ms  373469/415040 (90%)  receiver
```

Y finalmente la misma prueba con TCP. En este caso también se pierden paquetes, pero TCP se encarga de retransmitirlos (columna `Retr`), en total 86. Le hemos establecido un intervalo de 0,1 s para poder apreciar mejor el crecimiento de la ventana de congestión (`Cwnd`) aunque claramente los RTO la han dejado cerca de 350 ms. La tasa media de transferencia es de 80,5 Mbps, muy inferior a la solicitada de 1 Gbps, y menor también que la obtenida con el cliente UDP (80,2 vs. 90,6 Mbps) aunque el nodo remoto y probablemente la ruta son los mismos. Esta diferencia se debe sin duda a la contención que provoca la ventana de congestión.

```
$ iperf3 --time 5 --bitrate 1G --client sidney.example.net --interval 0.1
Connecting to host sidney.example.net, port 5201
[ ID] Interval      Transfer     Bitrate    Retr  Cwnd
[  5]   0.00-0.10    128 KBytes  10.5 Mbps    0   56.2 KBytes
[  5]   0.10-0.20   2.50 MBytes   210 Mbps   13    356 KBytes
[  5]   0.20-0.30   0.00 Bytes      0 bps    73    277 KBytes
[  5]   0.30-0.40    768 KBytes  62.9 Mbps    0    283 KBytes
```

```
[  5]   0.40-0.50    768 KBytes  62.9 Mbps    0    288 KBytes
[...]
[  5]   4.80-4.90   1.50 MBytes   126 Mbps    0    394 KBytes
[  5]   4.90-5.00    896 KBytes  73.2 Mbps    0    394 KBytes
- - - - - - - - - - - - - - - - - - - - - - - - -
[ ID] Interval      Transfer     Bitrate    Retr
[  5]   0.00-5.00   49.8 MBytes  83.5 Mbps   86            sender
[  5]   0.00-5.03   48.1 MBytes  80.2 Mbps                 receiver
```

17.5. Garantizar prestaciones

Lo primero que debes saber es que eso de «garantizar» es bastante relativo. Lo que puede hacer QoS en una interred IP es priorizar un flujo sobre otro, pero la garantía solo la puede dar si el enlace es capaz de ofrecer las prestaciones totales solicitadas o bien se dispone de algún mecanismo de control de admisión.

Hay dos formas básicas de aplicar prioridades al tráfico:

- IntServ (Integrated Services) identifica flujos y aplica prioridades diferentes a cada flujo.
- DiffServ (Differentiated Services) utiliza un conjunto de clases de tráfico predefinidas, marca el tráfico con esas clases y después prioriza los recursos en función de ellas.

IntServ realiza reserva de recursos en cada router antes de comenzar la transmisión. Debido a ello, su escalabilidad es limitada. Los routers que deban manejar miles de flujos deberán mantener información sobre la reserva de recursos de cada uno de ellos. DiffServ es un enfoque más general y escala mucho mejor porque no requiere configuración previa, el router toma sus decisiones únicamente con la información del paquete.

17.6. Servicios diferenciados

Como cabría esperar, la operación de DiffServ ocurre principalmente en los routers, aunque los conmutadores y los equipos terminales también pueden participar en determinadas situaciones. Un esquema simplificado del procesamiento *(pipeline)* QoS podria ser el de la Figura 17.10.

La finalidad de cada una de esas tareas es la siguiente:

- Clasificación: determina a qué clase de tráfico corresponde el paquete.
- Marcado: escribe el identificador de clase en el paquete.
- Policing: comprueba si el paquete cumple con los requisitos de QoS.
- Encolado: coloca el paquete en la cola que corresponde a su clase.

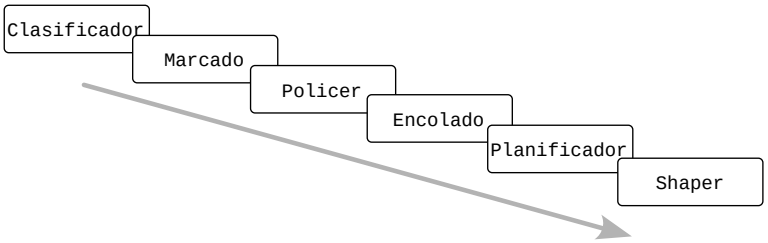


FIGURA 17.10: Pipeline QoS

- Planificación: decide de qué cola se extrae el siguiente paquete a reenviar.
- Shaping: ajusta la tasa de salida conforme a la contratada.
- Reenvío: transmite el paquete hacia su siguiente salto como en un router convencional.

Antes de entrar en el detalle de cada una de estas tareas, conviene introducir dos conceptos que aparecen de forma recurrente en redes, y especialmente en QoS:

ingress es el tráfico que entra.
egress es el tráfico que sale.

La interpretación de ingress y egress dependen completamente del componente que se tome como referencia. El mismo tráfico puede ser una cosa u otra en función de cada caso. Para el pipeline QoS, ingress es el tráfico que procede de la red (si es un router), o de un proceso emisor si es un nodo. El tráfico egress es el que envía a la interfaz de red.

Dentro del propio pipeline, las colas donde se almacenan temporalmente los paquetes son el punto de referencia más intuitivo. Asi, las tareas de clasificación, marcado y policing procesas el tráfico ingress, mientras que planificación, shaping y reenvío se encargan del tráfico egress.

17.6.1. Cubos de tokens

Para saber si un flujo cumple una determinada tasa (o forzar que así sea) se suelen utilizar variantes del algoritmo de medición *token bucket*, que podemos traducir como *cubo de fichas* o simplemente *cubo de tokens*.

El cubo[6] se llena de tokens a una tasa constante de CIR, que representa la tasa media garantizada. El cubo tiene una capacidad de CBS, que representa el tamaño de ráfaga máxima permitida. Cada paquete que llega se

[6] ficticio, obviamente

canjea por tokens del cubo (un token por byte). Si el cubo contiene tokens suficientes, el paquete es conforme y sigue su camino. En caso contrario, se dice que el paquete «excede» y se aplica la acción programada que decida el servicio que está usando el algoritmo. Si eso ocurre, no se consumen tokens.

El algoritmo del cubo de tokens deja pasar los paquetes si su tasa media fluctúa en torno a CIR, pero además tolera ráfagas ocasionales superiores siempre que compensen los tokens no consumidos (ahorrados) durante la emisión por debajo de CIR. Esta es la versión básica y se conoce como *single token bucket* o *single rate two color marker*. Esos dos colores representan el resultado: conforme (verde) o excedente (rojo). La Figura 17.11 representa el algoritmo.

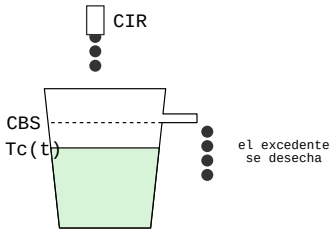


FIGURA 17.11: Cubo de tokens básico: dos colores

La Tabla 17.1 es un ejemplo del funcionamiento de este cubo de tokens. El cubo se llena a una tasa CIR=1000 kBps y tiene una capacidad CBS=3000 bytes. Inicialmente el cubo está lleno ($T_c(t) = 3000$ tokens). El paso (Δt) es 1 ms, es decir, cada fila muestra la llegada de un paquete con esa diferencia de tiempo. Las columnas «$T_c(t)$ prev.» y «$T_c(t)$ post.» son los tokens acumulados en el cubo antes y después de evaluar cada paquete; B es el tamaño del paquete entrante (bytes) y «Resultado» es el color con el que se marca el paquete. Por último, la columna «media» muestra la tasa media acumulada (bytes recibidos / tiempo transcurrido). Se mantiene en torno a los 1000 kBps aunque la tasa de entrada suba hasta los 1800 kBps; esto es una ráfaga y se puede ver que si dura demasiado, el cubo la corta.

La variante SRTCM (Single Rate Three Color Marker) [54][7] utiliza dos cubos `C` *(Commited)* y `E` *(Exceed)*. El cubo `C` tiene una capacidad de CBS tokens que juega el mismo papel que en el algoritmo básico. El cubo `E` tiene una capacidad de EBS, se llena con el excedente de `C` y representa el tamaño de ráfaga excedente (son tokens ahorrados). El paquete se marca como

[7] Estamos aplicando el modo *color blind*, es decir, los paquetes llegan sin colorear.

$t(ms)$	$T_c(t)$ prev.	B	$T_c(t)$ post.	Resultado	media
0	3000	1000	2000	verde	1000,0
1	3000	1200	1800	verde	1100,0
2	2800	700	2100	verde	966,7
3	3000	1800	1200	verde	1175,0
4	2200	1800	400	verde	1300,0
5	1400	1800	1400	**rojo**	1083,3
6	2400	1800	600	verde	1185,7
7	1600	1800	1600	**rojo**	1037,5
8	2600	1000	1600	verde	1033,3

CUADRO 17.1: Ejemplo de cubo de tokens

«conforme» (verde) si los tokens acumulados en `C` ($T_c(t)$) son suficientes. Si no lo son, pero los tokens acumulados en `E` ($T_e(t)$) son suficientes, se marca como «excedente» (amarillo). En otro caso se marca como «fuera de perfil» (rojo). El paquete marcado en verde consume tokens de `C`, el marcado en amarillo consume tokens de `E` y el marcado en rojo no consume tokens. La Figura 17.12 representa el algoritmo y el Listado 17.5 muestra su pseudocódigo.

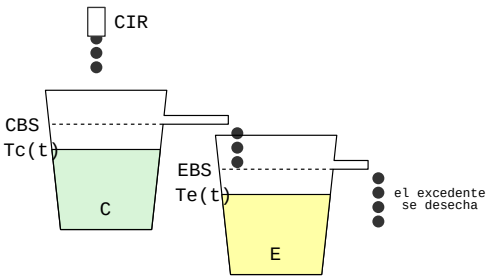


FIGURA 17.12: Algoritmo de cubo de tokens de tipo srTCM

```
B = paquete.longitud
si Tc >= B:
    color = VERDE
    Tc = Tc - B
sino si Te >= B:
    color = AMARILLO
    Te = Te - B
sino:
    color = ROJO
```

LISTADO 17.5: Algoritmo de srTCM en pseudocódigo

Una segunda variante, llamada TRTCM (Two Rate Three Color Marker) [55], utiliza dos tasas de llenado: CIR y PIR para los cubos `C` *(Commited)* y `P` *(Peak)*, que tienen capacidades CBS y PBS respectivamente. Para un paquete de B bytes, marca en rojo si `P` tiene menos de B tokens, amarillo si `P` los tiene, pero `C` no, y verde si ambos los tienen. Cuando se marca verde, se descuentan B tokens de ambos cubos. El amarillo indica que el paquete excede la ráfaga máxima garantizada (CBS), pero no la máxima permitida (PBS). Se representa en la Figura 17.13.

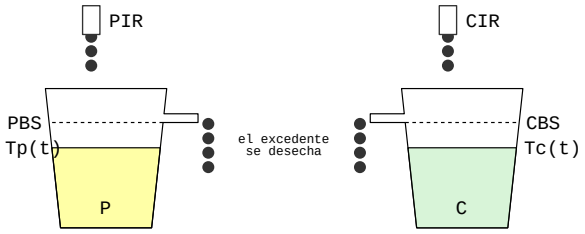


FIGURA 17.13: Algoritmo de cubo de tokens de tipo TRTCM

Estos algoritmos solo miden. El uso que se le dé depende de la acción que se aplique para cada color resultante: aceptar, descartar, remarcar, esperar, enviar a otra cola, etc.

17.6.2. Clasificación y marcado

El marcado debería realizarse lo más cerca posible del origen del tráfico. Sin embargo, normalmente no se hace en el propio origen, a menos que se trate de nodos bajo el control de la organización o terminales cerrados como teléfonos VOIP. Piensa que si cualquiera pudiera marcar su tráfico en su propio computador, todos los usuarios elegirían la máxima prioridad y todo el sistema sería inútil. Los dispositivos en los que sí se confía —routers, switches u otros nodos bajo control de la organización— forman la «frontera de confianza» *(trust boundary)*.

Cuando el marcado lo hace un router se realiza considerando los datos de la cabecera del paquete: dirección origen y destino, puerto, protocolo, etc., pero también información de enlace como la etiqueta VLAN, por lo que todos los paquetes de un mismo flujo se marcan probablemente con la misma clase. Si el paquete llega marcado, el router podría decidir respetar la marca o sobreescribirla si no se considera fiable.

El marcado del tráfico consiste en colocar un identificador de clase —un valor de 6 bits llamado PHB (Per-Hop Behavior)— en el campo DSCP de la cabecera IP. En IPV6, el campo `Traffic Class` cumple la misma función.

El campo `DSCP` ocupa los 6 bits MSB del campo TOS (Type of Service) de la cabecera; los 2 bits restantes se utilizan para ECN, la notificación de congestión (§ 14.6).

El significado de ese octeto ha cambiado con cada especificación que lo ha revisado. La Figura 17.14 compara esas interpretaciones, una por fila:

- La RFC 791 [9] (1981), la especificación original de IP, lo definió como campo `TOS`: tres bits de precedencia y tres indicadores independientes para solicitar menor retardo (D:*delay*), mayor tasa (T:*throughput*) o mayor fiabilidad (R:*reliability*), con los dos últimos bits reservados a cero.
- La RFC 1349 [56] (1992) añadió un cuarto indicador, el coste económico (C:*cost*), que ocupó uno de esos bits reservados.
- La RFC 2474 [57] (1998) abandonó ese esquema y reinterpretó los seis bits más significativos como un único valor, el DSCP, dejando los dos bits restantes sin asignar; la RFC 3168 [51] (2001) los destinó a ECN.
- La RFC 2597 [58] (1999) da estructura interna al propio DSCP: para las clases AFxy reparte sus seis bits en tres de clase, dos de tolerancia al descarte y uno fijo a cero. Aunque es anterior a la RFC 3168, ocupa el último lugar porque no reinterpreta el octeto completo, sino que subdivide el DSCP que definió la RFC 2474.

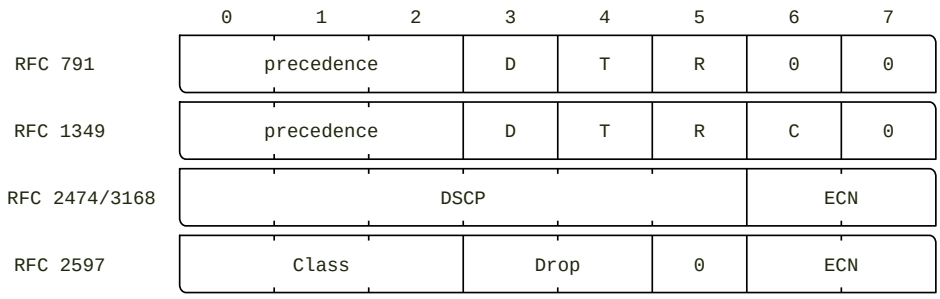


FIGURA 17.14: Interpretaciones del campo `TOS` de IP

Los PHB estandarizados son:

AC (Class Selector)
: (bits `xxx000`) indica una clase de tráfico codificada en los tres bits MSB (CS1-CS7). DF es en realidad la clase 0 (CS0). Estas clases están definidas en la RFC 2474 [57] y son compatibles con el campo `Precedence` de la especificación original de IP RFC 791 [9].

Las clases, de menor a mayor prioridad, son:

CS1 Tráfico desechable. De hecho, tiene menor prioridad que CS0, *p. ej.* P2P, actualizaciones en segundo plano.

CS0 Tráfico sin garantías *best effort*, la clase por defecto.

CS2 OAM (Operations, Administration and Maintenance), *p. ej.* SSH, SNMP, NTP.

CS3 Video broadcast, *p. ej.* IPTV.

CS4 Vídeo interactivo en tiempo real, *p. ej.* videojuegos, telemetría, telepresencia.

CS5 Señalización de voz y vídeo, *p. ej.* SIP, H.323, H.245.

CS6 Control de la red y encaminamiento, *p. ej.* OSPF, BGP, STP.

CS7 Control crítico / reservado.

AFxy (Assured Forwarding)
(bits `xxx100`) indica una de cuatro clases con tres niveles de descarte. Se representan como AFxy, siendo x la prioridad de reenvío (1-4) e y la tolerancia que ese tráfico tiene al descarte (1-3). Por ejemplo, AF41 indica alta prioridad (4) y baja tolerancia al descarte (1).

EF (Expedited Forwarding)
(bits `101110`) es el tráfico de máxima prioridad, RTP/VoIP.

Estos modos diferentes de codificar las clases en realidad se intercalan. La RFC 4594 [59] define un conjunto de 12 perfiles de tráfico, que puedes ver resumidos en el Cuadro 17.2.

Clase de tráfico	PHB / DSCP	Uso típico
Control de red	CS6	Protocolos de encaminamiento
Telefonía	EF	Voz sobre IP
Señalización	CS5	Señalización de llamadas
Videoconferencia	AF41/42/43	Videoconferencia interactiva
Interactivo en tiempo real	CS4	Vídeo interactivo, juegos en red
Streaming multimedia	AF31/32/33	Streaming de vídeo y audio
Vídeo broadcast	CS3	Vídeo broadcast en tiempo real
Datos de baja latencia	AF21/22/23	Transacciones interactivas (ERP, BD)
OAM	CS2	Gestión de red
Datos de alto rendimiento	AF11/12/13	Transferencias masivas, correo, copias de seguridad
Estándar	DF / CS0	Tráfico genérico
Datos de baja prioridad	CS1	Tráfico desechable

CUADRO 17.2: Clases de tráfico recomendadas

17.6.3. Policing

El policing (en español «vigilancia» o «regulación») comprueba que el tráfico no excede los valores de tasa (CIR/PIR) establecidos en el SLA. Aunque evalúa paquetes individuales, tiene en cuenta el comportamiento acumulado de cada flujo. Si el paquete analizado hace que el flujo exceda la tasa prevista, puede descartarlo o remarcarlo con una prioridad inferior o preferencia de descarte mayor, por ejemplo cambiando un marcado AF41 a AF43.

El policer a menudo utiliza un medidor de cubo de tokens de tres colores (srTCM o trTCM) para determinar si el paquete es conforme (verde), excedente (amarillo) o fuera de perfil (rojo). Las acciones que aplica el policer suelen ser las siguientes:

- Si el paquete es conforme, lo deja pasar sin cambios.
- Si el paquete excede, lo deja pasar, pero lo marca con una prioridad inferior o una preferencia de descarte mayor.
- Si el paquete está fuera de perfil, lo descarta.

Con estas acciones puedes deducir que el policer no añade latencia ni *jitter* al flujo; nunca encola ni retrasa, pero como descarta, puede provocar retransmisiones. Por eso, el policer se aplica sobre todo con tráfico prioritario transportado sobre UDP, como VoIP, en el que es preferible perder algún pequeño fragmento que retrasar todo el flujo.

17.6.4. Encolado

A cada interfaz de salida del router se asocia un conjunto de colas (una por clase) en lugar de una sola (§ 14.1). Es una tarea sencilla, simplemente se coloca cada paquete en la cola que le corresponde según la clase con la que está etiquetado.

Los paquetes quedan almacenados en estas colas en espera de que el canal quede libre y el router pueda reenviar el siguiente paquete. Al igual que en un router sin QoS, la ocupación de estas colas crece durante episodios de alta carga. En esa situación (ver § 14.6) se utiliza un algoritmo de descarte aleatorio (RED) en lugar de esperar a que la cola se llene.

Sin embargo, con un router QoS, la situación es distinta porque disponemos de información adicional sobre el tráfico. Se aplican algoritmos como WRED (Weighted RED), que considera tanto la clase del paquete como su tolerancia al descarte (la y en las clases AFxy) y que elige paquetes cuyo descarte resulta menos dañino.

Como norma, no se aplica WRED a la cola de máxima prioridad (EF) porque ese tipo de tráfico (*p. ej.* VoIP) tolera muy mal el descarte. Como suele trasportarse sobre UDP no se retransmite, no soporta ECN ni su descarte ayuda a reducir la congestión.

17.6.5. Planificación

El planificador *(scheduler)* es el algoritmo utilizado para determinar de qué cola se toma el siguiente paquete. Estos son los tipos de planificador más comunes, en orden de complejidad:

FIFO
: (First In First Out) representa la ausencia de prioridad. Todos los paquetes son tratados del mismo modo, y se reenvían en el orden de llegada. Se podría decir que es el planificador de los routers sin QoS.

SP
: (Strict Priority) como su nombre indica, aplica una prioridad estricta, es decir, siempre atiende primero a la cola no vacía de mayor prioridad. Muy simple, pero produce inanición para las colas de menor prioridad en episodios de alta carga.

  Al atender siempre primero la cola de mayor prioridad, un paquete de esa clase nunca espera detrás de tráfico de otra. Así acota el retardo máximo de la clase y, al reducir su competencia por la cola, también el *jitter* (§ 17.1.1).

WRR
: (Weighted Round Robin) reparte el ancho de banda disponible en el enlace entre las colas proporcionalmente a su prioridad.

WFQ
: (Weighted Fair Queuing) es similar a WRR pero considera el tamaño de los paquetes para determinar la proporción de ancho de banda consumida por cada cola. De ese modo, una cola de baja prioridad con paquetes grandes no distorsiona la proporción asignada.

CBWFQ
: (Class Based WFQ) es un WFQ configurable. Permite al administrador definir las clases y el reparto del ancho de banda, garantizando a cada una una tasa mínima incluso cuando el resto de clases satura el enlace.

LLQ
: (Low Latency Queuing) es el planificador más usado. Es una combinación de SP para tráfico de alta prioridad y baja latencia, y CBWFQ para el resto del tráfico. SP se aplica hasta una tasa máxima configurable, para evitar inanición.

La cota que SP impone al retardo es fácil de calcular. Como es un planificador no apropiativo *(non-preemptive)*, no puede interrumpir el envío un paquete en curso aunque llegue otro de mayor prioridad, pero cuando acaba atiende de inmediato la cola prioritaria. Ese es el peor caso y corresponde con el retardo de envío de § 17.1.1 aplicado al mayor paquete posible:

$$\text{retardo}_{\max} \approx \frac{L_{\max}}{R} \tag{17.8}$$

donde:

> $L_{\max}$ es el tamaño del mayor paquete que puede estar enviándose en el enlace —seguramente MTU—, y R la velocidad de transmisión del enlace.

Por ejemplo, en un enlace Gigabit Ethernet ($R = 1$ Gbps) con tramas de hasta 1500 bytes, el retardo adicional es como mucho de $1500{\cdot}8/10^9 = 12\ \mu s$, un valor insignificante frente al orden de milisegundos en el que se suele definir un SLA de voz o vídeo. Lógicamente la cota crece en enlaces más lentos porque el mismo paquete tarda más en enviarse.

17.6.6. Shaping

El *shaping* (en español «modelado» o «conformado») es la tarea que extrae paquetes de la cola elegida por el planificador. Decide el momento adecuado para obtener una tasa de salida estable en torno al CIR configurado, y para eso utiliza también el algoritmo de cubo de tokens, aunque no exactamente del mismo modo que el policer. Mientras el cubo contenga tokens suficientes, el shaper los canjea por paquetes de la cola; si no hay, espera, bloquea la cola.

Este cubo de tokens se configura con tasa de llenado CIR y capacidad CBS. Con estos valores se calcula $\Delta t = CBS/CIR$, que es el período de extracción. Un CBS alto produce Δt grandes, lo que conlleva ráfagas grandes y períodos largos; un CBS bajo conlleva ráfagas pequeñas y períodos cortos. Por ejemplo, si la tasa garantizada es $CIR = 6Mbps$ y $CBS = 8Kb$ obtenemos un $\Delta t = 1{,}33$ ms. Eso significa que el shaper extrae 8 Kb cada 1,33 ms. Si para el mismo caso se fija $CBS = 800Kb$ se extraerán esos 800 Kb cada 133 ms. Ese mismo Δt es también el límite superior del *jitter* que el propio shaping puede introducir en la clase: un CBS pequeño lo mantiene bajo, a costa de admitir ráfagas menores.

El valor Δt es el período del reloj que controla la acción del shaper, pero también del planificador. También aquí se puede utilizar un algoritmo de dos cubos con capacidades CBS y EBS del tipo SRTCM.

17.7. QoS en Linux

La mayoría de la «maquinaria» de QoS de Internet se encuentra en equipos de gama alta, como switches y routers en grandes empresas, pero sobre todo en ISP y en los grandes operadores de telecomunicaciones en las redes WAN. El fabricante CISCO destaca por su sofisticado soporte de QoS en este tipo de equipamiento con su sistema operativo IOS.

Linux también dispone de un buen soporte de QoS, que puede ser suficiente para despliegues de mediano y pequeño tamaño, y para el propósito de este libro resulta muy útil para bajar al detalle, experimentar y comprobar en la práctica lo explicado hasta ahora.

El soporte de QoS de Linux se llama *Traffic Control*, o simplemente `tc` (`iproute2`), que es también el nombre del programa con el que se administran la mayoría de sus opciones. Aunque la nomenclatura de Linux difiere un poco de la teoría de las secciones anteriores (extraída principalmente de las RFC), los conceptos y la forma en que funciona son muy similares. *Traffic Control* se apoya en tres conceptos:

qdisc
: de *queuing discipline*, que podemos traducir como «disciplina de encolado», es equivalente al planificador.

class
: son las clases de tráfico. Cada clase implica una cola, pero en función del tipo de qdisc, pueden ser varias.

filter
: son los predicados que indican a qué clase debe ir cada paquete. El conjunto de filtros forma el clasificador.

Los siguientes apartados explican cómo se configuran estos tres elementos y cómo se relacionan entre sí.

17.7.1. qdisc

Aunque es probable que lo hayas pasado por alto hasta ahora (y nosotros lo hemos omitido en muchas capturas del libro) cada interfaz de red puede tener asociada una qdisc. El comando `ip link` lo muestra de nuevo:

```
$ ip link
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP qlen 1000
    link/ether f8:5f:2a:c0:ff:ee brd ff:ff:ff:ff:ff:ff
```

```
3: wlan0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP qlen 1000
    link/ether 08:00:46:de:ca:f0 brd ff:ff:ff:ff:ff:ff
```

Los valores de qdisc para cada interfaz:

```
lo: qdisc noqueue
eth0: qdisc fq_codel
wlan0: qdisc noqueue
```

La interfaz Ethernet está configurada con el qdisc `fq_codel` —la opción por defecto hoy día. Sin embargo, las interfaces *loopback* y WiFi tienen configurado `noqueue`, que es como decir 'sin qdisc', pero es así por motivos diferentes.

Loopback no es una interfaz real, no está asociada a un medio físico, o sea que no puede sufrir congestión y los flujos que la utilizan no compiten por su ancho de banda. La interfaz WiFi por otra parte, sí que utiliza QoS, pero lo hace en una capa inferior. WiFi utiliza un medio compartido (el aire) por el que los dispositivos tienen que competir, y lo hacen con un sistema llamado CSMA/CA. Es la capa MAC, que está implementada en la propia NIC, la que se encarga en gran medida de afrontar el problema. Ya tiene sus propias colas y su lógica de prioridad como parte de la implementación del protocolo de enlace. A pesar de todo eso, puede haber situaciones en las que interese completar las capacidades QoS desde el SO, pero no es lo habitual.

El Cuadro 17.3 muestra algunos de los qdisc disponibles, y su relación con los tipos de QoS:

`pfifo` y `bfifo`	FIFO con conteo de paquetes y bytes
`prio`	SP con varias bandas
`sfq`	WRR
`htb`	WFQ y CBWFQ

CUADRO 17.3: Equivalencia entre qdisc y los tipos de QoS

Esta es la configuración por defecto en un Linux actual de una interfaz Ethernet, con el comando `tc`:

```
~# tc qdisc show dev eth0
qdisc fq_codel 0: root refcnt 2 limit 10240p flows 1024 quantum 1514 target 5ms
    interval 100ms memory_limit 32Mb ecn drop_batch 64
```

Esta interfaz utiliza `fq_codel`, que combina dos mecanismos: FQ *(fair queuing)*, un clasificador que utiliza un hash para direcciones/puertos/protocolo/etc. y con ello organiza los flujos de tráfico en un número especificado de colas

internas. Sirve esas colas con un planificador de tipo *round robin* similar a WRR/DRR para hacer un reparto equitativo (de ahí su nombre) de la tasa excedente entre esas colas. CoDel *(controlled delay)* es la técnica de AQM que aplica en caso de congestión (capítulo 14, § 14.6.1). A diferencia de SP/LLQ (§ 17.6), que acotan el retardo dando prioridad completa a una clase, CoDel trata igual a todas las colas internas de `fq_codel`: es el control de retardo que aplica Linux por defecto actualmente.

Este comando ofrece mucha información útil que requiere explicación:

`0: root`
: Dice que es el planificador raíz o principal. Más adelante en este mismo capítulo se muestra cómo definir jerarquías de planificadores y colas.

`limit 10240p`
: Es el tamaño de las colas internas: 10240 paquetes cada una.

`flows 1024`
: Es el número de colas internas. Fíjate que puede haber más flujos que colas. La cola se comparte entre varios flujos si tienen el mismo valor de hash.

`quantum 1514`
: Es la cantidad de bytes que cada cola puede transmitir en cada ronda antes de ceder el turno a la siguiente cola (DRR). El valor de 1514 bytes no es casual; es justo el tamaño de una trama Ethernet con su cabecera. La idea es que se pueda extraer al menos un mensaje por ronda y no haya que esperar al siguiente turno para acumular déficit.

`target 5ms interval 100ms`
: Es configuración para CoDel: `target` es el retardo aceptable de los paquetes. Por encima de ese valor, se considera su descarte; `interval` es el plazo máximo que el flujo tiene permitido superar `target`. En este caso, si el retardo del flujo se mantiene por encima de 5 ms durante 100 ms, se empezarán a descartar paquetes o se aplicará ECN si está disponible.

`memory_limit 32Mb`
: Es el espacio total (en megabytes) que ocupan los paquetes encolados (los 10240 paquetes de `limit` en este ejemplo).

`ecn` ECN está habilitado.

`drop_batch 64`
: La cantidad de paquetes que se pueden descartar de una vez si se supera `limit` o `memory_limit`. Solo se hace en caso extremo como una forma de tail-drop eficiente.

El programa `tc` ofrece estadísticas detalladas sobre el comportamiento de la política QoS establecida en la interfaz[8]. Para la interfaz anterior, nos dice lo siguiente:

```
$ tc -s qdisc show dev eth0
 Sent 1879915814 bytes 2378552 pkt (dropped 0, overlimits 0 requeues 8993)
 backlog 130204b 2p requeues 8993
  maxpacket 65102 drop_overlimit 0 new_flow_count 3065 ecn_mark 0 memory_used 126448
  new_flows_len 0 old_flows_len 1
```

[8]Las cifras mostradas están contabilizadas desde que se definió el planificador.

donde:

`Sent 1879915814 bytes 2378552 pkt`
: Es la cantidad de bytes y paquetes que han sido transmitidos a través de este pipeline.

`dropped 0`
: Paquetes descartados debidos a la congestión por el algoritmo AQM.

`overlimits 0`
: Se superó la tasa garantizada. Con `fq_codel`, este valor siempre es 0 porque no tiene tasa máxima.

`requeues 8993`
: Paquetes reencolados, normalmente por problemas esporádicos, no por congestión o exceso de tasa.

`backlog 130204b 2p`
: Cantidad de bytes y paquetes que hay en la cola en el momento de ejecutar el comando.

`maxpacket 13554`
: El tamaño del paquete más grande que se ha enviado.

`drop_overlimit 0`
: Cantidad de paquetes descartados por superar los límites de la cola (`limit` o `memory_limit`). Es decir, son descartes *tail drop* por congestión.

`new_flow_count 2680`
: Cantidad de flujos que han sido encolados.

`ecn_mark 0`
: Cantidad de paquetes marcados con ECN por congestión.

`memory_used 126448`
: Tamaño total (en bytes) que ocupan los paquetes encolados.

`new_flows_len 0`
: Cantidad de flujos nuevos, que no han sido atendidos aún.

`old_flows_len 1`
: Cantidad de flujos antiguos, de los cuales ya se han atendido algunos paquetes.

Para obtener una salida como la del comando anterior, con datos algo más interesantes, hemos forzado tráfico sintético con el programa `iperf3` (Figura 17.15). Consiste en un servidor (por defecto escucha en el puerto 5201) y un cliente que le envía tráfico. Si no se indica otra cosa, creará conexiones TCP y transmitirá datos a la tasa máxima posible.

Cliente

```
$ iperf3 --time 20 --client
↪  sidney.example.net
    --parallel 4
```

Servidor

```
$ iperf3 --server
```

FIGURA 17.15: Generando tráfico con `iperf3`:
4 flujos TCP en paralelo durante 20 segundos

Aquí en concreto le hemos pedido que envíe tráfico con 4 conexiones en paralelo durante 20 segundos. Cuando acaba, tanto el servidor como el cliente muestran unas estadísticas sobre la transferencia (Listado 17.6).

```
~$ iperf3 --time 3 --client sidney.example.net --port 5201
Connecting to host example.net, port 5201
[  5] local 192.168.0.37 port 53204 connected to example.net port 5201
[ ID] Interval      Transfer     Bitrate    Retr  Cwnd
[  5]   0.00-1.00   6.88 MBytes  57.6 Mbps  103    208 KBytes
[  5]   1.00-2.00   6.38 MBytes  53.4 Mbps    0    224 KBytes
[  5]   2.00-3.00   6.25 MBytes  52.4 Mbps    0    246 KBytes
- - - - - - - - - - - - - - - - - - - - - - - - -
[ ID] Interval      Transfer     Bitrate    Retr
[  5]   0.00-3.00   19.5 MBytes  54.5 Mbps  103        sender
[  5]   0.00-3.04   17.9 MBytes  49.4 Mbps             receiver
```

Listado 17.6: Estadísticas de transferencia de `iperf3`

17.7.2. Configurando un sistema de colas

El planificador `fq_codel` resulta muy adecuado para las necesidades cotidianas y por eso es la configuración por defecto en muchas distribuciones GNU/Linux. Pero hay situaciones con necesidades específicas. En esos casos, puede ser más adecuado un planificador que permita crear clases y jerarquía de colas. Además nos viene bien para experimentar y aprender de QoS.

Empezamos por configurar un planificador `htb` como raíz, sustituyendo así el `fq_codel` por defecto. Este planificador es de tipo CBWFQ, dos cubos de tokens con tasa garantizada y máxima, similar a srTCM.

```
~# tc qdisc add dev eth0 root handle 1: htb default 99
```

donde:

`root handle 1:`
: Es el identificador del planificador raíz.

`htb` Es el tipo de planificador.

`default 99`
: Es la clase por defecto, que se utilizará para marcar los paquetes que no cumplan ningún filtro.

Definimos una jerarquía de clases sencilla: una clase padre y dos hijas. Eso permite una gestión flexible del ancho de banda disponible entre las clases hermanas. En primer lugar, la clase padre:

```
~# tc class add dev eth0 parent 1: classid 1:1 htb rate 2mbit ceil 2mbit
```

donde:

`parent 1:`
: Es una clase hija del planificador `1:`.

classid 1:1
: Es el identificador de esta clase. El *classid* es un entero de 32 bits con el formato *major:minor*. El *major* es el identificador de la qdisc y el `minor` es el identificador de la clase dentro de esa qdisc. Por eso todo lo que definamos aquí comparte el `major 1:` del qdisc raíz.

rate 2mbit
: Es la tasa garantizada (2 Mbps).

ceil 2mbit
: Es la tasa máxima permitida (2 Mbps).



Es conveniente aclarar que la nomenclatura de `tc` para las unidades es bastante «pintoresca», lo que puede dar lugar a confusiones. Las cantidades en bytes se denotan con una '`b`' minúscula (*p. ej.* `1000b`) mientras que para bits se escribe la palabra completa (`1000 bits`). También se utiliza arbitrariamente «mbits» o «Mbits» para megabits. Y en muchas ocasiones las tasas se expresan como `mbit` para indicar `Mbps` (megabits por segundo). Revisa el anexo D para ver la nomenclatura del SI que se usa en este libro.

Dos clases hijas idénticas (`1:10` y `1:20`) completan la jerarquía y van a compartir la tasa de la clase padre. Para que el tráfico convencional no afecte a los resultados del experimento, se define la clase por defecto (`1:99`).

```
~# tc class add dev eth0 parent 1:1 classid 1:10 htb rate 1mbit ceil 2mbit
~# tc class add dev eth0 parent 1:1 classid 1:20 htb rate 1mbit ceil 2mbit
```

donde:

parent 1:1
: Son clases hijas de la clase `1:1`.

classid 1:*
: Son identificadores de clase.

rate 1mbit ceil 2mbit
: Son las tasas garantizada (1 Mbps) y tasa máxima permitida (2 Mbps).

Configuración resultante con el comando `tc class show`:

```
~# tc class show dev eth0
class htb 1:1 root rate 2Mbit ceil 2Mbit burst 1600b cburst 1600b
class htb 1:10 parent 1:1 prio 0 rate 1Mbit ceil 2Mbit burst 1600b cburst 1600b
class htb 1:20 parent 1:1 prio 0 rate 1Mbit ceil 2Mbit burst 1600b cburst 1600b
class htb 1:99 root prio 0 rate 100Mbit ceil 100Mbit burst 1600b cburst 1600b
```

Además de la información del comando anterior, aparecen algunos datos más que son importantes y han sido fijados a valores por defecto:

prio 0
: Es la prioridad de la clase. Por defecto es 0 (máxima prioridad)[9].

[9]Como ocurre a menudo, valores menores indican prioridades mayores.

burst 1600b cburst 1600b
: Son las ráfagas máximas (las capacidades de los cubos de tokens), expresadas aquí en bytes.

Que ambas clases tengan la misma prioridad hará que el planificador reparta la tasa excedente de forma proporcional a su `rate` a través de la clase padre. Si fueran prioridades diferentes, la clase con mayor prioridad (menor valor) utilizaría toda la tasa excedente que necesite, y solo una vez satisfecha, el sobrante estaría disponible para las otras clases.

Cada clase hija tiene garantizado 1 Mbps, pero su `ceil` le permite llegar a 2 Mbps. Cuando una de ellas no consume su tasa garantizada, esa capacidad queda disponible en la clase padre y la otra puede tomarla prestada *(borrow)* hasta alcanzar su `ceil`. Esta es la razón de ser de la jerarquía de clases.

También es muy buena idea que la clase padre tenga una tasa garantizada que sea al menos la suma de las tasas de las hijas. De otro modo se dice que hay sobresuscripción *(oversubscription)*, con lo que el planificador no puede garantizar todas las tasas garantizadas —algo similar al *overbooking* de las líneas aéreas.

Cada clase `htb` usa un mecanismo de dos cubos de tokens muy similar a trTCM con la siguiente configuración: un cubo `C` con tasa de llenado CIR=`rate` y capacidad CBS=`burst`, y un cubo `P` con tasa de llenado PIR=`ceil` y capacidad PBS=`cburst`. El resultado (los colores) se interpreta como:

- verde: puede enviar a la tasa garantizada.
- amarillo: puede pedir prestado a la clase padre (indirectamente a las clases hermanas).
- rojo: no puede enviar, tiene que esperar.

Se pueden obtener estadísticas para cada clase que nos dan una idea muy concreta del comportamiento de la política QoS. Puedes verlas con `tc -s class show`.

```
~# tc -s class show dev eth0 classid 1:10
class htb 1:10 parent 1:1 prio 0 rate 1Mbit ceil 2Mbit burst 1600b cburst 1600b
 Sent 0 bytes 0 pkt (dropped 0, overlimits 0 requeues 0)
 backlog 0b 0p requeues 0
 lended: 0 borrowed: 0 giants: 0
 tokens: 200000 ctokens: 100000
```

Las líneas que empiezan por «Sent» y «backlog» tienen el mismo significado que en el planificador, salvo que aplican solo a esta cola. Pero tenemos algunos campos nuevos que son interesantes:

borrowed: 0
: Se incrementa cada vez que, para enviar un paquete, la cola tuvo que pedir prestada tasa a otras clases.

lended: 0
: Cantidad de veces que el envío se hizo a cargo de la tasa de esta clase.

tokens: 200000
: Cantidad de tokens que quedan en el cubo `C` *(commited)*.

ctokens: 100000
: Cantidad de tokens que quedan en el cubo `P` *(peak)*.

Estas cantidades de tokens requieren una explicación. En la implementación de Linux, estos tokens son canjeables por tiempo de transmisión, en lugar de bytes, es decir, cada token representa una cantidad de tiempo que la cola puede utilizar para transmitir.

Por ejemplo, si según `burst` la capacidad del cubo `C` es 1600 bytes y su tasa garantizada es 1 Mbps (125 kBps), con el cubo lleno sería posible transmitir a esa tasa durante:

$$t = burst/rate = 1600\ bytes/125000\ bytes/s = 0{,}0128\ s = 12{,}8\ ms$$

Que con la equivalencia de 1 token=64 ns que utiliza Linux actualmente, nos da la cantidad de tokens que caben en el cubo:

$$12{,}8\ ms/64\ ns = 0{,}0128/0{,}000000064 = 200000\ tokens$$

Como el cubo `P` *(peak)* tiene una tasa de 2 Mbps, para transmitir la misma cantidad de bytes (`cburst`) solo necesita la mitad de tiempo, y por tanto la mitad de tokens.

Filtros

Los filtros conforman la lógica que decide qué clase se asocia a cada flujo, lo que a fin de cuentas constituye el clasificador del pipeline. En este pequeño ejemplo, vamos a clasificar el tráfico IP dirigido al puerto 5201 como clase `1:10` y el tráfico dirigido al puerto 5202 como clase `1:20`. El resto de tráfico se clasificará en la clase por defecto, `1:99`.

```
~# tc filter add dev eth0 protocol ip parent 1: prio 1 flower ip_proto tcp \
    dst_port 5201 flowid 1:10
~# tc filter add dev eth0 protocol ip parent 1: prio 2 flower ip_proto tcp \
    dst_port 5202 flowid 1:20
```

donde:

protocol ip
: Selecciona tráfico IPv4.

parent 1:
: Es un filtro asociado al planificador raíz.

prio 1
: Indica la prioridad (orden) en que se evalúa el filtro.

flower
: El clasificador seleccionado (`flower`) es potente y de alto nivel, permitiendo referir campos de las cabeceras de muchos protocolos. Una de sus ventajas más destacables respecto a alternativas como `u32` es que permite *hardware offload*.

ip_proto tcp
: Selecciona tráfico TCP.

dst_port 5201
: Selecciona tráfico dirigido al puerto 5201.

flowid 1:10
: Si el paquete cumple el predicado, se le clasifica como `1:10`.



Hardware offload permite instalar el clasificador en la NIC, con la consiguiente mejora de velocidad y reducción de carga de la CPU. Sin embargo, solo las NIC más modernas y de gama alta permiten esta opción.

Esquemáticamente, la configuración de las clases queda así:

```
1: (root, htb, default 99)
 +-- 1:1 (rate 2Mbps ceil 2Mbps)
 |   +-- 1:10 (rate 1Mbps ceil 2Mbps) puerto 5201
 |   +-- 1:20 (rate 1Mbps ceil 2Mbps) puerto 5202
 +-- 1:99 (rate 100Mbps) resto del tráfico
```

El comando `tc filter show` confirma esa configuración:

```
~# tc filter show dev eth0
filter parent 1: protocol ip pref 1 flower chain 0
filter parent 1: protocol ip pref 1 flower chain 0 handle 0x1 classid 1:10
  eth_type ipv4
  ip_proto tcp
  dst_port 5201
  not_in_hw
filter parent 1: protocol ip pref 2 flower chain 0
filter parent 1: protocol ip pref 2 flower chain 0 handle 0x1 classid 1:20
  eth_type ipv4
  ip_proto tcp
  dst_port 5202
  not_in_hw
```

donde:

pref *
: Es la prioridad del filtro, que determina el orden en el que se evaluarán por cada paquete. Esto es importante: los filtros más específicos deben tener mayor prioridad (menor valor) que los más generales, o serán ignorados.

chain 0
: Indica la cadena a la que pertenece el filtro. Las cadenas permiten encadenar clasificadores para construir pipelines complejos, en los que una acción puede pasar de una cadena a otra. Si no se indica otra cosa, los filtros se crean en la cadena 0.

handle 0x1
: Es el identificador del filtro, que permite referenciarlo después para modificarlo o eliminarlo. Si no se especifica, `tc` le asigna uno automáticamente.

not_in_hw
: Indica que el filtro no se ha instalado en la NIC (no hay *hardware offload*), y se ejecutará en la CPU.

Para ver los efectos de esta configuración, ejecutamos un cliente `iperf3` durante 16s contra el puerto 5201 de un servidor remoto. A los 8s y sin parar los clientes, tomamos las estadísticas de las tres clases. El script /qos/2-class-1-idle.sh implementa esta prueba; esta es la salida que genera:

```
~# tc -s class show dev eth0
class htb 1:1 root rate 2Mbit ceil 2Mbit burst 1600b cburst 1600b
 Sent 1964834 bytes 1316 pkt (dropped 0, overlimits 327 requeues 0)
 backlog 0b 0p requeues 0
 lended: 326 borrowed: 0 giants: 0
 tokens: -188223 ctokens: -188223

class htb 1:10 parent 1:1 prio 0 rate 1Mbit ceil 2Mbit burst 1600b cburst 1600b
 Sent 1964834 bytes 1316 pkt (dropped 0, overlimits 652 requeues 0)
 backlog 6024b 2p requeues 0
 lended: 338 borrowed: 326 giants: 0
 tokens: -96473 ctokens: -188223

class htb 1:20 parent 1:1 prio 0 rate 1Mbit ceil 2Mbit burst 1600b cburst 1600b
 Sent 0 bytes 0 pkt (dropped 0, overlimits 0 requeues 0)
 backlog 0b 0p requeues 0
 lended: 0 borrowed: 0 giants: 0
 tokens: 200000 ctokens: 100000

class htb 1:99 root prio 0 rate 100Mbit ceil 100Mbit burst 1600b cburst 1600b
 Sent 68794 bytes 222 pkt (dropped 0, overlimits 15 requeues 0)
 backlog 0b 0p requeues 0
 lended: 209 borrowed: 0 giants: 0
 tokens: 1846 ctokens: 1846
```

En una segunda prueba (script /qos/2-class.sh) ejecutamos la misma configuración, pero con 2 clientes `iperf3` con flujos hacia los puertos 5201 y 5202 respectivamente:

```
~# tc -s class show dev eth0
class htb 1:1 root rate 2Mbit ceil 2Mbit burst 1600b cburst 1600b
 Sent 1971868 bytes 1332 pkt (dropped 0, overlimits 1 requeues 0)
 backlog 0b 0p requeues 0
 lended: 0 borrowed: 0 giants: 0
 tokens: -471048 ctokens: -471048

class htb 1:10 parent 1:1 prio 0 rate 1Mbit ceil 2Mbit burst 1600b cburst 1600b
 Sent 985934 bytes 666 pkt (dropped 0, overlimits 327 requeues 0)
 backlog 6024b 2p requeues 0
 lended: 339 borrowed: 0 giants: 0
 tokens: -376481 ctokens: -88250
```

```
class htb 1:20 parent 1:1 prio 0 rate 1Mbit ceil 2Mbit burst 1600b cburst 1600b
 Sent 985934 bytes 666 pkt (dropped 0, overlimits 327 requeues 0)
 backlog 6024b 2p requeues 0
 lended: 339 borrowed: 0 giants: 0
 tokens: -376443 ctokens: -88250

class htb 1:99 root prio 0 rate 100Mbit ceil 100Mbit burst 1600b cburst 1600b
 Sent 350881 bytes 572 pkt (dropped 0, overlimits 108 requeues 0)
 backlog 0b 0p requeues 0
 lended: 467 borrowed: 0 giants: 0
 tokens: 1873 ctokens: 1873
```

Y con el comando `tc qdisc show` vemos las estadísticas del planificador, con los clientes aún en ejecución:

```
~# tc -s qdisc show dev eth0
qdisc htb 1: root refcnt 2 r2q 10 default 0x99 direct_packets_stat 4 direct_qlen 1000
 Sent 2323244 bytes 1908 pkt (dropped 0, overlimits 763 requeues 0)
 backlog 12048b 4p requeues 0
```

Resultados de las clases antes y después de ejecutar el segundo cliente `iperf3`:

variable	un cliente			dos clientes		
classid	1:1	1:10	1:20	1:1	1:10	1:20
sent (pkt)	1316	1316	0	1332	666	666
overlimits	327	652	0	1	327	327
backlog (pkt)	0	2	0	0	2	2
lended	326	338	0	0	339	339
borrowed	0	326	0	0	0	0
tokens	−188 223	−96 473	200 000	−471 048	−376 481	−376 443
ctokens	−188 223	−188 223	100 000	−471 048	−88 250	−88 250

Cuadro 17.4: Estadísticas de las clases con uno o dos clientes `iperf3` contra 2 puertos distintos.

La clase `1:1` es instrumental: ningún filtro clasifica tráfico hacia ella. Su función es puramente estructural, sirve de contenedor de la tasa que sus clases hijas comparten y de la que pueden tomar prestado. Por eso sus estadísticas de envío no reflejan tráfico propio, sino la suma del de sus clases hijas.

Puedes comprobar que cuando únicamente hay tráfico en la cola `1:10`, la clase padre contabiliza su misma cantidad de paquetes enviados (1316). La cola `1:10` consumió toda la tasa disponible en el padre (2 Mbps) porque no compite con nadie. La cifra 326 indica la cantidad de paquetes que el planificador extrajo de la cola `1:10` con tokens prestados.

Cada vez que la cola `1:10` consume todos sus tokens al llegar a 1 Mbps, le pide prestado a la clase padre el permiso para enviar el paquete (326 veces `lended` en la 1:1 y `borrowed` en la `1:10`). Esto consume tokens en la clase hija y en la padre, y la cantidad consumida puede ser distinta si las tasas de esas clases son distintas (recuerda que es tiempo). Todo esto es posible solo porque la clase `1:20` no está utilizando su tasa garantizada (`rate`). Con esto, la clase `1:10` puede llegar a su tasa máxima (`ceil`) de 2 Mbps.

Resulta llamativo ver cantidades negativas de tokens. El valor negativo es el tiempo que debe transcurrir hasta que el cubo recupere tokens y la cola pueda volver a transmitir. Por tanto, encontrar estos valores negativos implica que la clase se comporta como un shaper, retrasando y no descartando, algo que queda confirmado al observar todos los `dropped` a 0.

Los signos de las variables `tokens` y `ctokens` indican además en qué modo está trabajando la clase: Si ambos son positivos, la clase está transmitiendo a su tasa garantizada o `rate` (verde). Si `tokens` es negativo y `ctokens` positivo, la clase está transmitiendo por encima de `rate`, pero por debajo de `ceil` (amarillo). Si ambos son negativos, la clase está probablemente difiriendo el tráfico (rojo), porque ha alcanzado la tasa máxima (`ceil`).

La variable `overlimits` indica la cantidad de paquetes no-verdes: es decir los que han requerido préstamos (`borrowed`) o los que tuvieron que esperar (dado que `dropped`=0). Por ejemplo, para la clase `1:10` podemos deducir:

- 1316 paquetes enviados en total.
- 326 paquetes prestados (amarillos o `borrowed`).
- 990 (1316 − 326) paquetes verdes.
- 652 por encima de la tasa garantizada (no-verdes u `overlimits`).
- 326 *overlimits* − *borrowed* = 652 − 326 **intentos** diferidos o rojos.

Lo de *intentos* es importante porque los paquetes nunca se marcan en rojo. Rojo significa que el envío se retrasa, y el mismo paquete podría ser retrasado varias veces antes de ser finalmente transmitido. Por eso, podríamos hablar de paquetes rojos si se descartaran, pero no cuando se retrasan.

Al arrancar el segundo cliente —mira las 3 últimas columnas de la Tabla 17.4— la clase `1:20` empieza a transmitir y a competir por el ancho de banda disponible. Como ahora ya no hay excedente, las dos clases tenderán a transmitir a su tasa garantizada (1 Mbps).

Policer

Para ver el efecto del policer añadimos al pipeline una tercera clase (`1:30`). También es necesario modificar la clase padre para que pueda ofrecer una tasa agregada de 3 Mbps.

```
~# tc class add dev eth0 parent 1: classid 1:1 htb rate 3mbit ceil 3mbit
~# tc class add dev eth0 parent 1:1 classid 1:30 htb rate 1mbit ceil 2mbit
```

Definimos el policer, que es un filtro con `action police` que clasifica el tráfico dirigido al puerto 5203 hacia la nueva cola 1:30.

```
~# tc filter add dev eth0 protocol ip parent 1: prio 3 flower \
  ip_proto tcp dst_port 5203 \
  action police rate 1.5mbit burst 32k conform-exceed drop \
  flowid 1:30
```

donde:

`rate 1.5mbit`
: La tasa límite (CIR): 1,5 Mbps.

`burst 32k`
: La ráfaga máxima permitida (CBS): 32 kB.

`conform-exceed drop`
: Cuando el cubo se quede sin tokens, los paquetes deben descartarse.

Este policer limita la tasa a 1,5 Mbps con ráfaga de 32 kB con un cubo simple de 2 colores. También es posible configurar un cubo trTCM con 3 colores, pero quedémonos con el de 2 colores para no complicar el ejemplo.

Hemos fijado intencionadamente la tasa objetivo del policer a 1,5 Mbps entre la tasa `rate` de la clase 1:30 (1 Mbps) y la `ceil` (2 Mbps). De este modo la nueva clase puede pedir prestado para acercarse a su `ceil` antes de llegar al techo del policer. Así podemos ver actuar a los dos mecanismos —shaper y policer, retraso y descarte— de forma complementaria.

Como en los casos anteriores, la configuración establecida incluye otros parámetros que no se han indicado al definir el filtro:

```
~# tc -s filter show dev eth0
^^Iaction order 1:  police 0x1 rate 1500Kbit burst 32Kb mtu 2Kb action drop overhead 0b
^^Iref 1 bind 1  installed 0 sec used 0 sec
^^IAction statistics:
^^ISent 0 bytes 0 pkt (dropped 0, overlimits 0 requeues 0)
^^Ibacklog 0b 0p requeues 0
```

Otros campos con valores por defecto

`action order 1:`
: El orden de ejecución de la acción, dado que puede definir varias.

`police 0x1`
: Este filtro es un policer con identificador 0x1.

`mtu 2Kb`
: Tamaño máximo de paquete que puede pasar: 2 kB.

`action drop`
: Los paquetes que excedan (rojos) serán descartados.

overhead 0b
: Tamaño de la cabecera que se resta del tamaño del paquete para calcular el tamaño del paquete a contar.

installed 0
: El filtro se creó hace 0 segundos.

used 0
: El filtro se usó hace 0 segundos, o no se ha usado.

requeues 0
: Paquetes reencolados.

Esquemáticamente, la configuración de clases queda así:

```
1: (root, htb, default 99)
 +-- 1:1 (rate 3Mbps ceil 3Mbps)
 |   +-- 1:10 (rate 1Mbps ceil 2Mbps) puerto 5201, shaper
 |   +-- 1:20 (rate 1Mbps ceil 2Mbps) puerto 5202, shaper
 |   +-- 1:30 (rate 1Mbps ceil 2Mbps) puerto 5203, policer (1.5Mbps) + shaper
 +-- 1:99 (rate 100Mbps) resto del tráfico
```

Para probar este pipeline usamos el script /qos/policer-3-class.sh que genera flujos TCP sobre cada uno de los puertos 5201, 5202 y 5203, si bien en el caso del puerto 5203 genera 16 flujos en paralelo. Esto es necesario porque con uno o pocos flujos, el control de congestión haría su trabajo y ajustaría automáticamente la ventana en cuanto empezasen a ocurrir los RTO por los primeros descartes. Con los flujos en paralelo, el policer se ve obligado a descartar paquetes y podemos ver su efecto.

Capturamos estadísticas después de 8 segundos de ejecución, y lo primero que llama la atención es la ocupación de la cola `1:30`:

```
~# tc -s class show dev eth0 classid 1:30
class htb 1:30 parent 1:1 prio 0 rate 1Mbit ceil 2Mbit burst 1600b cburst 1600b
 Sent 924822 bytes 668 pkt (dropped 0, overlimits 256 requeues 0)
 backlog 96384b 32p requeues 0
 lended: 412 borrowed: 0 giants: 0
 tokens: -376475 ctokens: -88250
```

El backlog de 32 paquetes encaja con los 16 flujos TCP: los descartes del policer provocan continuas situaciones de *3dupack* y expiraciones de RTO en cada flujo, lo que mantiene su cwnd cerca del mínimo (§ 14.5). Con 16 flujos compitiendo por la misma clase, la suma de los pocos segmentos que cada uno mantiene en vuelo da como resultado ese backlog de, en promedio, 2 paquetes por flujo. La cola se ocupa porque el policer permite hasta 1,5 Mbps, pero la clase está limitada a 1 Mbps y no hay excedente que permita superarla, porque las otras clases también están emitiendo a su tope. Igual que antes, la clase se comporta como shaper, retrasando el tráfico en espera de tokens. La cola ha procesado (transmitido + retenido) un total de:

$$924\,822 + 96\,384 = 1\,021\,206 \text{ bytes}$$

Pasando a bits/s y calculando para los 8 segundos, ha aceptado datos a una tasa media de:

$$1\,021\,206 \times 8/8 \text{ s} \approx 1{,}02 \text{ Mbps}$$

Pero la tasa de transmisión efectiva fue:

$$924\,822 \times 8/8 \text{ s} \approx 0{,}92 \text{ Mbps}$$

Lo más interesante en este caso está en las estadísticas del filtro.

```
~# tc -s filter show dev eth0
^^Iaction order 1:  police 0x1 rate 1500Kbit burst 32Kb mtu 2Kb action drop overhead 0b
^^Iref 1 bind 1  installed 11 sec used 0 sec firstused 8 sec
^^IAction statistics:
^^ISent 1224516 bytes 867 pkt (dropped 71, overlimits 71 requeues 0)
^^Ibacklog 0b 0p requeues 0
```

El policer se ha encontrado 71 veces con paquetes que excedieron la ráfaga máxima (tokens agotados) y los ha descartado todos. El contador `Sent` del policer contabiliza todo lo que evalúa, aunque lo descarte. El policer ha descartado:

$$procesado - transmitido = descartado$$
$$1\,224\,516 - 1\,021\,206 = 203\,310 \text{ bytes}$$

La tasa de entrada en el filtro y la de descartes son:

$$entrada = 1\,224\,516 \times 8/8 \text{ s} \approx 1{,}22 \text{ Mbps}$$
$$descarte = 203\,310 \times 8/8 \text{ s} \approx 0{,}20 \text{ Mbps}$$

De lo que podemos concluir es que una vez se «llena» la cola, el policer descarta el tráfico excedente (25 kB/s). La tasa efectiva de la cola `1:30` no ha llegado al `rate` de 1 Mbps, pero probablemente el motivo es el arranque lento de los clientes TCP que desaprovechan tasa disponible al comienzo.

```
~# tc -s qdisc show dev eth0
qdisc htb 1: root refcnt 2 r2q 10 default 0x99 direct_packets_stat 0 direct_qlen 1000
 Sent 3171692 bytes 2484 pkt (dropped 71, overlimits 1039 requeues 0)
 backlog 108432b 36p requeues 0
```

Con este último experimento puedes ver claramente la diferencia entre la función del policer y el shaper actuando sobre el mismo tráfico. La clase `1:30` (el shaper) indica `backlog 96384b 32p`: nunca descarta, retiene para lograr una tasa media que cumpla los parámetros de la cola (y de su padre). En cambio el filtro (el policer) indica `dropped 71`, que son paquetes que superaron sus parámetros. Como descarta y no encola, el policer no añade retardo, lo que puede ser más adecuado para tráfico interactivo como voz o vídeo, que suele usar transporte UDP. Con TCP, los descartes provocarán retransmisiones y el típico perfil de tráfico en diente de sierra, algo que no es deseable para lograr flujos estables y predecibles.

17.7.3. Marcado de paquetes con netfilter

En esta sección se explica cómo usar *netfilter* (§ 11.7) para hacer marcado DSCP de los paquetes salientes de un nodo terminal, pero del mismo modo se puede usar en un router de la organización.

Por simplicidad, en este ejemplo vamos a definir solo 3 de los 12 perfiles DIFFSERV (§ 17.6).

Lo primero es crear una tabla y una cadena propias para no interferir con las que seguramente ya hay definidas en tu sistema.

```
~# nft add table ip qos_lab
~# nft add chain ip qos_lab output_chain '{ type filter hook output priority mangle;
↪  policy accept; }'
```

La primera línea crea la tabla `qos_lab` para protocolos de la familia IPV4. La segunda línea es una cadena asociada a la tabla `qos_lab` con los siguientes parámetros:

`type filter`
: Es una cadena de tipo *filter*. Permite aceptar, descartar o modificar los paquetes que la atraviesan.

`hook output`
: Está asociada al *hook output*, que significa que se aplicará a todos los paquetes generados por procesos del propio nodo. Para hacer marcado en un nodo que hace de router (reenvía) tendría que elegir el hook *forward*.

`priority mangle`
: Indica el orden en el que se evalúa la cadena. La prioridad *mangle* implica que se ejecutará antes del planificador.

`policy accept`
: Si no se indica lo contrario, la cadena aceptará los paquetes, es decir, seguirán su camino.

Por el momento hemos creado una cadena vacía que no hace nada:

```
~# nft list table ip qos_lab
table ip qos_lab {
^^Ichain output_chain {
^^I^^Itype filter hook output priority mangle; policy accept;
^^I}
}
```

Es momento de añadir la primera regla. Esto marca con el perfil EF todo el tráfico de voz/control (protocolo SIP), que usa el puerto UDP 5060, y el tráfico RTP (audio de las llamadas), que suele usar el rango de puertos UDP 10000-20000.

```
~# nft add rule ip qos_lab output_chain udp dport 5060 ip dscp set ef
~# nft add rule ip qos_lab output_chain udp dport 10000-20000 ip dscp set ef
```

La segunda clase será para datos interactivos del protocolo RDP, que utiliza el puerto TCP 3389. Este tráfico se marcará con el perfil AF31.

```
~# nft add rule ip qos_lab output_chain tcp dport 3389 ip dscp set af31
```

Y la tercera clase será para tráfico genérico *best effort*, como podría ser el generado por el programa `rsync`[10] que usa el puerto TCP 873. Lo marcaremos con el perfil CS1.

```
~# nft add rule ip qos_lab output_chain tcp dport 873 ip dscp set cs1
```

Aspecto de la configuración completa de la tabla:

```
~# nft list table ip qos_lab
table ip qos_lab {
    chain output_chain {
        type filter hook output priority mangle; policy accept;
        udp dport 5060 ip dscp set ef
        udp dport 10000-20000 ip dscp set ef
        tcp dport 3389 ip dscp set af31
        tcp dport 873 ip dscp set cs1
    }
}
```

Para clasificar este tráfico configura como raíz un planificador de tipo `prio` que es adecuado para el tráfico prioritario.

```
~# tc qdisc add dev eth0 root handle 1: prio bands 3
```

Las tres bandas especificadas se utilizan del siguiente modo:

- Banda 0 (1:1): máxima prioridad, tráfico EF (voz/control).
- Banda 1 (1:2): prioridad media, clasificado con un `htb` para los otros dos perfiles: AF31 y CS1.

[10] Un programa de copia remota masiva de ficheros

- Banda 2 (1:3): prioridad baja, resto del tráfico.

Configuración:

```
~# tc qdisc show dev eth0
qdisc prio 1: root refcnt 2 bands 3 priomap 1 2 2 2 1 2 0 0 1 1 1 1 1 1 1 1
```

El mapa de prioridades `priomap` asigna a cada banda (valores 0-2) el tráfico correspondiente a cada valor de `SO_PRIORITY` (0-15), pero está pensado para los valores TOS obsoletos. Para solventarlo, vamos a crear filtros que clasifiquen según los valores DSCP que acabamos de fijar en las reglas `netfilter`.

```
# Filtro banda 0 (EF/voz) con policer
~# tc filter add dev eth0 parent 1: protocol ip prio 1 flower \
  ip_tos 0xb8/0xfc \
  action police rate 1mbit burst 10k drop \
  classid 1:1

# Filtro banda 1 (AF31/datos interactivos)
~# tc filter add dev eth0 parent 1: protocol ip prio 2 flower \
  ip_tos 0x68/0xfc \
  classid 1:2

# Filtro banda 2 (CS1/bulk)
~# tc filter add dev eth0 parent 1: protocol ip prio 3 flower \
  ip_tos 0x20/0xfc \
  classid 1:3
```

Configuración de filtros:

```
~# tc filter show dev eth0
filter parent 1: protocol ip pref 1 flower chain 0
filter parent 1: protocol ip pref 1 flower chain 0 handle 0x1 classid 1:1
  eth_type ipv4
  ip_tos 184/0xfc
  not_in_hw
filter parent 1: protocol ip pref 2 flower chain 0
filter parent 1: protocol ip pref 2 flower chain 0 handle 0x1 classid 1:2
  eth_type ipv4
  ip_tos 104/0xfc
  not_in_hw
filter parent 1: protocol ip pref 3 flower chain 0
filter parent 1: protocol ip pref 3 flower chain 0 handle 0x1 classid 1:3
  eth_type ipv4
  ip_tos 32/0xfc
  not_in_hw
```

Con lo que tenemos ahora, todo el tráfico marcado con EF se encola en la banda 1:1 y el planificador no atenderá nada más mientras haya tráfico en esa cola. Para evitar inanición en las otras clases, vamos a sustituir el primer filtro por un policer con limitación de tasa, que corresponde con un

planificador de tipo LLQ (Low Latency Queuing) y lo sustituimos por el filtro con policer:

```
~# tc filter del dev eth0 parent 1: prio 1
~# tc filter add dev eth0 parent 1: protocol ip prio 1 flower \
  ip_tos 0xb8/0xfc \
  action police rate 1mbit burst 10k drop \
  classid 1:1
```

Esto encaja directamente con un planificador LLQ de tres bandas, con la banda 0 limitada a 1 Mbps y las otras dos bandas sin limitación.

Podemos conseguir algo más flexible sustituyendo las colas FIFO de las bandas 1:2 y 1:3 por planificadores estilo CBWFQ usando `htb`.

Para ello creamos un qdisc hijo `htb` para la banda 1:2:

```
~# tc qdisc add dev eth0 parent 1:2 handle 20: htb default 1
```

Dentro de él, creamos una clase con el rate/ceil para el tráfico AF31 (datos interactivos):

```
~# tc class add dev eth0 parent 20: classid 20:1 htb rate 2mbit ceil 4mbit
```

Y para la banda 1:3 otro `htb` para el tráfico CS1 (datos genéricos):

```
~# tc qdisc add dev eth0 parent 1:3 handle 30: htb default 1
```

Ahora la clase correspondiente, con un rate/ceil menor, porque *best effort* no necesita garantías altas, solo aprovechar lo que sobre:

```
~# tc class add dev eth0 parent 30: classid 30:1 htb rate 500kbit ceil 4mbit
```

Resultado:

```
~# tc qdisc show dev eth0;
qdisc prio 1: root refcnt 2 bands 3 priomap 1 2 2 2 1 2 0 0 1 1 1 1 1 1 1 1
qdisc htb 20: parent 1:2 r2q 10 default 0x1 direct_packets_stat 0 direct_qlen 1000
qdisc htb 30: parent 1:3 r2q 10 default 0x1 direct_packets_stat 0 direct_qlen 1000

~# tc class show dev eth0
class prio 1:1 parent 1:
class prio 1:2 parent 1: leaf 20:
class prio 1:3 parent 1: leaf 30:
class htb 20:1 root prio 0 rate 2Mbit ceil 4Mbit burst 1600b cburst 1600b
class htb 30:1 root prio 0 rate 500Kbit ceil 4Mbit burst 1600b cburst 1600b
```

17.8. QoS en origen

El mecanismo de encaminamiento en el propio nodo origen también puede priorizar tráfico, aunque no utilizando el marcado DSCP. Linux tiene un sistema automático llamado `ip_tos2prio`, que como su nombre indica, convierte el valor del campo `TOS` (concretamente el campo `TOS` de la RFC 1349 [56]) en una prioridad local. Desgraciadamente ese sistema utiliza la especificación obsoleta «IP precedence» de la RFC 791 [9] que no es compatible con DSCP (Figura 17.14).

En un programa actual, para tener prioridad local tendrás que hacer tanto el marcado DSCP como una selección de prioridad explícita, que tiene que ejecutarse necesariamente después para anular el efecto de la conversión automática errónea que dijimos antes. Ambas cosas se pueden hacer en la propia aplicación directamente sobre los sockets implicados. El Listado 17.7 muestra un fragmento de un servidor TCP que marca con la clase EF (la más prioritaria) usando la opción `IP_TOS` del socket y la prioridad local 6 usando la opción `SO_PRIORITY`. Estos son los niveles de prioridad local:

- `TC_PRIO_BEST_EFFORT = 0`
- `TC_PRIO_FILLER = 1`
- `TC_PRIO_BULK = 2`
- `TC_PRIO_INTERACTIVE_BULK = 4`
- `TC_PRIO_INTERACTIVE = 6`
- `TC_PRIO_CONTROL = 7`

```
DSCP_EF = 46 << 2
TC_PRIO_INTERACTIVE = 6

sock = socket.socket()
sock.bind(('', 1234))

sock.setsockopt(socket.IPPROTO_IP, socket.IP_TOS, DSCP_EF)
sock.setsockopt(socket.SOL_SOCKET, socket.SO_PRIORITY, TC_PRIO_INTERACTIVE)

sock.listen(5)

conn, addr = sock.accept()
```

LISTADO 17.7: Marcado de un flujo TCP con máxima prioridad (EF)

El servidor anterior 17.7 marca todos los segmentos de todas las conexiones que acepte ya desde su paquete SYN. Es posible marcar cada conexión independiente simplemente haciendo las mismas llamadas `setsockopt()`, pero sobre el socket conectado (`conn` en el listado). Fíjate en el desplazamiento de 2 bits (`<<2`) que se hace al valor de DSCP como corresponde a su posición

dentro del campo `TOS`. Esto deja los bits ECN a 0, pero es correcto porque el valor de esos bits es responsabilidad del núcleo.

Con UDP se puede hacer algo equivalente, es decir, marcar todos los datagramas enviados por un socket, aunque también es posible marcar cada datagrama por separado incluso con clases. Lo puedes ver en el Listado 17.8.

```
sock.sendmsg(
    [payload], [(socket.IPPROTO_IP, socket.IP_TOS, struct.pack('i', DSCP_EF))],
    0, ('example.net', 1234))
```

Listado 17.8: Marcado de un datagrama UDP individual

En cualquier caso, ten en cuenta que el efecto de la prioridad local depende del planificador configurado en la interfaz de red. El planificador `pfifo_fast` tiene tres bandas (colas de prioridad estricta) y coloca cada paquete en una u otra según el valor de `skb->priority`, traducido a número de banda mediante una tabla de 16 entradas llamada *priomap*. Otros planificadores, como `fq_codel`, ignoran totalmente la prioridad local.

Hay una vuelta de tuerca más. El valor de `SO_PRIORITY` es un entero de 32 bits, de los que la codificación de prioridad local solo emplea los 4 menos significativos (valores 0–15). El planificador `htb` interpreta ese entero como un *classid* con formato *major*:*minor*. Si el *major* es cero, algo que ocurre con todos los valores de prioridad local, es ignorado. Pero si el *major* no es cero, `htb` comprueba si hay una clase definida con ese *classid* y, si existe, encola el paquete en ella directamente, sin necesidad de definir un filtro.

Sabiendo esto, puedes escribir un programa que directamente clasifique tráfico en la cola elegida. Por ejemplo, el Listado 17.9 es un cliente UDP que coloca el flujo directamente en la cola de la clase `1:10` del ejemplo de la sección anterior:

```
CLASSID_1_10 = 0x00010010  # clase 1:10

sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
sock.setsockopt(socket.SOL_SOCKET, socket.SO_PRIORITY, CLASSID_1_10)
sock.sendto(b"hola", ('example.net', 1234))
```

Listado 17.9: Marcado de un flujo UDP para la clase 1:10

Debes tener en cuenta que escribir valores de `SO_PRIORITY` mayores que 6 requiere privilegios especiales, concretamente la capacidad `CAP_NET_ADMIN`, dado que afecta de forma directa al tráfico en la red.

Y ¿qué más?

La gestión del tráfico y la QoS ofrecen formas de optimizar el uso y aprovechamiento de los recursos de la red; este capítulo ha repasado los mecanismos más importantes y cómo llevarlos a la práctica en sistemas GNU/Linux.

El tipo de QoS descrito aquí lo consigue principalmente el SO del router o nodo origen, pero estas mismas ideas se pueden aplicar en la capa de aplicación: un proxy web puede priorizar ciertas peticiones, o puede limitar la tasa de descarga o la cantidad de conexiones simultáneas que pueden realizar ciertos clientes en ciertos momentos. Estas técnicas de limitación se suelen denominar *throttling* y, a diferencia de la priorización, no buscan optimizar el uso de la red, sino salvaguardar la disponibilidad del servicio.

Capítulo 18

Serialización

Al terminar este capítulo, entenderás:

- Qué es la serialización y la des-serialización.
- Por qué es importante definir un formato de datos común.
- Cómo es la serialización binaria de datos multibyte y texto.
- Cómo utilizar `struct` para manejar datos binarios simples de tipos diferentes, como por ejemplo una cabecera de mensaje binario.

Empezamos este capítulo con una afirmación que podría aparecer en un curso de informática para niños: «La única forma en la que un computador digital procesa y almacena datos es mediante código binario». Ya, lo tenemos claro, pero eso también tiene consecuencias cuando esos datos tienen que viajar por la red.

El problema es que, salvo en unas pocas excepciones, el binario rara vez resulta suficientemente expresivo o amigable para representar información útil para las personas. Por ese motivo se utilizan distintas formas de interpretar el código binario en función del tipo de dato que se necesita: enteros, decimales, cadenas de caracteres, fechas, etc. Lo importante es recordar que, sea cual sea su representación en un lenguaje de programación de alto nivel, en la memoria o registros del computador, todo dato es a fin de cuentas una secuencia de bits.

La **serialización** es el proceso que transforma los datos que manejan los programas (enteros, cadenas, imágenes, etc.) en secuencias de bytes susceptibles de ser almacenadas en un archivo o enviadas a través de la red. La des-serialización es el proceso inverso. Existen dos tipos de serialización: binaria y textual. La binaria codifica los datos como secuencias de bytes, mientras que la textual utiliza marcado legible por humanos[1], con identificadores, espacios y comillas.

[1] El significado del adjetivo «legible» es bastante debatible según qué protocolo.

El siguiente listado es un ejemplo de serialización binaria (con pickle) y textual (con JSON) de los mismos datos:

Binaria

```
>>> import pickle
>>> data = {'width': 10, 'height': 20}
>>> pickle.dumps(data)
b'\x80\x04\x95\x1a\x00\x00\x00\x00...'
```

Textual

```
>>> import json
>>> data = {'width': 10, 'height': 20}
>>> json.dumps(data)
'{"width": 10, "height": 20}'
```

FIGURA 18.1: Serialización binaria y textual

Por supuesto, existen muchos formatos o sistemas de codificación, tanto binaria como textual. Este capítulo trata algunos de los más sencillos para ilustrar el concepto con mínima complejidad.

Es buen momento para aclarar una confusión habitual con la terminología. No debes confundir «codificación» con «cifrado» (o «encriptación»). «Codificar» es simplemente aplicar una transformación que modifica el modo en que se representan los datos, pero no implica ninguna clave, ocultación u ofuscación del mensaje para conseguir confidencialidad. Por ejemplo, un mensaje codificado en *código* Morse es perfectamente entendible por cualquier persona o sistema que conozca el código, que por supuesto, es público. El cifrado pretende ocultar el contenido del mensaje, para que únicamente aquellos que conozcan la clave secreta puedan verlo. Un mensaje cifrado con Blowfish solo podrá ser descifrado por alguien que conozca la clave.

18.1. Representación, solo eso

Una de las excepciones en las que el binario es útil directamente es la *programación de sistemas*, es decir, aquellos programas que consumen directamente servicios del SO. El binario resulta útil para manejar campos de bits, flags (banderas binarias) o máscaras, muy comunes cuando se manipulan registros de control, operaciones de E/S, etc. Por eso, cualquier programador también debería manejar con soltura la representación binaria.

La codificación más básica y común es la que aplica una base numérica. Los lenguajes de programación, como las personas, utilizan la base decimal para expresar cantidades. Python permite expresar literales numéricos en varias bases. Todos los casos del siguiente ejemplo representan el número 42; y en todos los casos, si se asigna a una variable, se está creando un entero (tipo `int`) con el mismo valor. Puedes comprobarlo fácilmente en el Listado 18.1, que se está ejecutando en el modo interactivo del intérprete.

```
>>> 0b101010  # binario
42
>>> 0o52      # octal
42
>>> 0x2A      # hexadecimal
42
```

Listado 18.1: Literales numéricos en Python

Asimismo ofrece funciones para convertir entre bases: `bin()`, `oct()` y `hex()`; pero una consideración importante a tener en cuenta es que estas funciones devuelven cadenas (`str`) porque su objetivo es precisamente ofrecer diferentes *representaciones textuales* del mismo dato. Observa las comillas simples en los valores de retorno en el Listado 18.2 que indican claramente que se trata de cadenas.

```
>>> bin(42)
'0b101010'
>>> oct(42)
'0o52'
>>> hex(42)
'0x2A'
```

Listado 18.2: Conversión a representación binaria, octal y hexadecimal

Opcionalmente, el constructor de la clase `int` acepta un número expresado como cadena de caracteres pudiendo además indicar la base, incluso con bases tan exóticas como 23 (Listado 18.3).

```
>>> int('42')
42
>>> int('52', 8)
42
>>> int('1J', 23)
42
```

Listado 18.3: Especificando la base en el constructor de `int`

Insistimos: debes tener claro que 42, 052, 0x2A o 101010 no son más que diferentes representaciones del mismo dato, y que el computador lo manejará **siempre** en su forma binaria.

18.2. Los enteros de Python

El tipo `byte` es el más simple de cualquier lenguaje de programación y corresponde con una secuencia de 8 bits. Suele ser un entero sin signo, es decir, puede representar números enteros en el rango [0, 255]. Sin embargo, Python no dispone de ese tipo[2]. Python, por su naturaleza dinámica, solo

[2]No lo debes confundir con `bytes`, que es un tipo de secuencia.

tiene un tipo de datos para enteros: `int`. Estos enteros pueden ser arbitrariamente grandes puesto que el *runtime* se encarga de gestionar la memoria necesaria:

```
>>> googol = 10 ** 100
>>> type(googol)
<class 'int'>
```

Pero cuando se serializan datos en un archivo o una conexión de red, necesitamos precisar explícitamente el tamaño de los datos. Aunque Python no disponga de los tipos `byte`, `short`, `long`, etc., usaremos estos conceptos para entender los tamaños de los datos. Las siguientes secciones explican cómo se logra todo esto.

18.3. Caracteres

La codificación de caracteres más simple consiste en asignar un número a cada carácter del alfabeto. El código más común y veterano es ASCII, que fue creado por ANSI en 1963 como una evolución de la codificación utilizada anteriormente en telegrafía. Es un código de 7 bits (128 símbolos) que incluye los caracteres alfa-numéricos de la lengua inglesa (mayúsculas y minúsculas) y la mayoría de los signos de puntuación y tipográficos habituales. Además incluye caracteres de control para indicar salto de línea, de página, tabulador, etc. Más tarde se crearon extensiones, como por ejemplo ISO 8859-1, popularmente conocida como *latin-1* que dispone de 256 símbolos, entre los que se encuentran caracteres acentuados y otros símbolos de uso común en idiomas europeos (ß, ç, ñ, ¿, etc.), monedas (£, ¥) o signos matemáticos (±, ÷, ×, ¼, etc).



El carácter «retorno de carro» (CR, código 13 o 0x0D), pide mover el cursor a la primera columna (en el borde izquierdo), mientras que el carácter de «avance de línea» (LF, código 10 o 0x0A) pide mover el cursor en la siguiente línea. Claramente alude a las máquinas de escribir automáticas, teletipos e impresoras en los que la máquina debe situar su cabezal físico para comenzar a escribir en la siguiente línea del papel. Las computadoras, por analogía, utilizaban la secuencia CR-LF para indicar la misma operación en un terminal (una pantalla). Sigue siendo de este modo en los sistemas operativos de Microsoft a día de hoy. Por contra, los creadores de los sistemas UNIX entendieron que el salto de línea sin retorno de carro no tenía sentido en un terminal y, por tanto, se utiliza únicamente el carácter LF para conseguir el mismo efecto. En muchos lenguajes de programación se representa con el carácter de control \n y se denomina simplemente «nueva línea» o EOL

Los lenguajes de programación incluyen funciones elementales para manejar la conversión entre bytes (números de 8 bits) y sus caracteres equivalentes. El siguiente fragmento de código Python lo demuestra mediante las funciones `ord()` y `chr()`.

```
>>> ord('a')
97
>>> chr(97)
'a'
>>> ord('0')
48
>>> ord('\0')
0
>>> chr(0)
'\x00'
>>> ord('\n')
10
>>> ord(' ')
32
```

Listado 18.4: Conversión entre caracteres y enteros en Python

Fíjate en la **línea 6** que el código del **carácter** `0` es 48, mientras que (**línea 10**) el carácter equivalente al código 0 es `\x00`. Es especialmente importante tener claro que los caracteres numéricos **no corresponden** con los valores que representan. También resulta digno de mención que la secuencia `\n` es *un solo carácter*, ya que la barra es lo que se conoce como un «carácter de escape», es decir, cambia el significado del siguiente carácter. En este caso significa «nueva línea».

De modo similar, la secuencia `\x` indica que los siguientes dígitos deben entenderse como un código hexadecimal. La función `chr()` devuelve una secuencia de este tipo cuando no existe un carácter *imprimible* asociado al código indicado.

El siguiente listado muestra un ejemplo de la equivalencia entre una cadena y su representación numérica.

```
>>> for i in '\xd2a3\n':
...     print(ord(i))
210
97
51
10
```

La cadena `\xd2a3\n` está compuesta por los caracteres `\xd2`, `a`, `3` y `\n`.

No puedes manipular el contenido de una cadena; recuerda que el tipo `str` es inmutable. Sin embargo, tenemos el tipo `bytearray`, que puede almacenar una secuencia de bytes, modificar su contenido —acepta tanto caracteres

como enteros— y permite obtener fácilmente la lista de caracteres o secuencia de enteros equivalente:

```
>>> buf = bytearray('abcd', 'ascii')
>>> buf[0] = 20
>>> buf
bytearray(b'\x14bcd')
>>> buf.decode()
'\x14bcd'
>>> bytes(buf)
b'\x14bcd'
>>> list(buf)
[20, 98, 99, 100]
```

18.4. Tipos multibyte y ordenamiento

Como sabes, un único byte solo puede representar 256 valores; obviamente casi cualquier programa o algoritmo, por simple que sea, necesita manejar enteros mayores, reales en coma flotante y otros tipos de datos que no «caben» en un byte. El más sencillo de estos tipos es el `short` o entero de 16 bits[3]. Aquí aparece una cuestión interesante: el ordenamiento de bytes (*endianness* o *byte order*) o, lo que es lo mismo, ¿en qué orden se deberían colocar en memoria los dos bytes que forman un `short`?

Dependiendo de la respuesta se distingue entre *little endian* y *big endian*. *Little endian* significa que el byte de *menor* peso[4] se coloca en la dirección más baja de memoria, mientras que en *big endian* es el byte de *mayor* peso[5]. La Figura 18.2 lo muestra para un dato de 4 bytes.

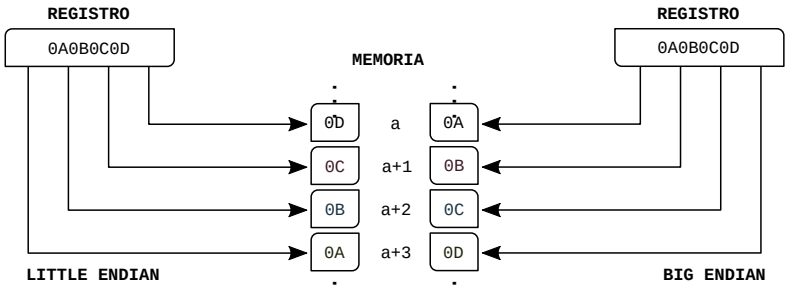


FIGURA 18.2: Ordenación de bytes

[3] `short` tampoco existe como tipo nativo en Python.
[4] el que tiene los bits menos significativos (LSB)
[5] el MSB

Para que el computador realice las operaciones (aritméticas, lógicas, etc.) que correspondan sobre el dato, es esencial que los programas manipulen la memoria de acuerdo al ordenamiento de la arquitectura.

Puedes comprobar de una manera muy sencilla qué tipo de ordenamiento tiene tu computadora, utilizando el módulo `struct` (más adelante lo veremos con detalle).

```
if struct.pack('H', 1) == b'\x00\x01':
    print("big endian")
else:
    print("little endian")
```

LISTADO 18.5: Averiguar el ordenamiento de bytes con Python.

Aunque Python ofrece una forma directa:

```
>>> sys.byteorder
'little'
```

Algo parecido ocurre también con la red. Cuando se coloca un dato multi-byte *en el cable*[6] debe elegirse un ordenamiento. Pues bien, los protocolos de la pila TCP/IP utilizan siempre ordenamiento *big-endian* [60], es decir, se envía primero el byte más significativo. Y de hecho, se le conoce como *ordenamiento de la red (network byte order)*.

Eso significa que las arquitecturas *little-endian* (típicamente Intel y AMD) deben convertir sus datos multi-byte antes de enviarlos a la red sobre un `socket`. Para evitar que el programa tenga que comprobar por sí mismo qué ordenamiento utiliza la arquitectura en la que se está ejecutando, las librerías de sockets proporcionan funciones que realizan la conversión. Nótese que en un nodo *big-endian* estas funciones no harán nada[7], pero deben usarse para conseguir que el programa sea 'portable', es decir, que se pueda ejecutar en nodos de diferente arquitectura sin ninguna modificación.

Las funciones que ofrece Python, tomadas de las llamadas al sistema POSIX homónimas, son:

- `socket.ntohs()`. Convierte un entero de 16 bits *(short)* del ordenamiento de la red al del host: ***n**etwork **to** **h**ost **s**hort*.
- `socket.ntohl()`. Convierte un entero de 32 bits *(long)* del ordenamiento de la red al del host: ***n**etwork **to** **h**ost **l**ong*.
- `socket.htons()`. Convierte un entero de 16 bits *(short)* del ordenamiento del host al de la red: ***h**ost **to** **n**etwork **s**hort*.

[6] por analogía con la expresión en inglés *on the wire*

[7] retornan el mismo valor que se les pasa como parámetro

- `socket.htonl()`. Convierte un entero de 32 bits *(long)* del ordenamiento de host al de la red: ***h**ost **to** **n**etwork **l**ong*.

El siguiente listado muestra unos ejemplos de uso de estas funciones en un computador *little-endian*.

```
>>> socket.htons(32)
8192
>>> socket.htonl(32)
536870912
>>> socket.htons(0)
0
```

LISTADO 18.6: Funciones de conversión de ordenamiento del módulo `socket`.

Esta es, de nuevo, la primera transformación en hexadecimal, para observar mejor los 2 bytes que la forman:

```
>>> hex(32)
'0x20'
>>> hex(socket.htons(32))
'0x2000'
```

Se puede ver claramente que al convertir el valor `0x20` a *big-endian* se coloca en el primer byte del entero de 16 bits. Si el computador receptor fuese *little-endian* no habría cambios.

18.5. Cadenas de caracteres y secuencias de bytes

En Python las cadenas de caracteres (de tipo `str`) utilizan Unicode. Sin embargo, este tipo de datos no se puede leer o escribir en un archivo (salvo que sea de texto), ni enviar o recibir de un socket. Todas esas operaciones requieren secuencias de bytes (de tipo `bytes`). Convertir una cadena a una secuencia de bytes requiere aplicar una codificación (un *encoding*) que establece la transformación. El siguiente listado ilustra la diferencia entre ambos tipos de datos:

```
>>> string = "hello world"
>>> type(string)
<class 'str'>
>>> sequence = bytes(string, 'ascii')
>>> type(sequence)
<class 'bytes'>
>>> string
'hello world'
>>> sequence
b'hello world'
```

LISTADO 18.7: Cadena codificada en ASCII

Python puede utilizar muchos sistemas de codificación *(encodings)* diferentes. El más habitual en los sistemas POSIX es UTF8 y también lo es para Python. UTF8 puede representar cualquier carácter Unicode. Es compatible con ASCII para su rango (127 caracteres), es decir, requiere un byte, mientras que para el resto de caracteres necesita 2 o más bytes. Por ejemplo el carácter ñ se codifica con 2 bytes: '0xc3b1' y € se codifica con 3 bytes: '0xe282ac'.

En la cadena de caracteres, cada símbolo representa un carácter, mientras que en la secuencia de bytes cada símbolo representa un byte. Y, ¿cuál es la diferencia? Parece que las líneas 8 y 10 son iguales salvo por la `b` que precede a la secuencia de bytes. Esto se debe a que este ejemplo usa codificación «ascii». ASCII codifica cada carácter con un único byte, por eso coinciden. Otro ejemplo más ilustrativo:

```
1  >>> string = "ñandú"
2  >>> sequence = bytes(string, 'ascii')
3  UnicodeEncodeError: 'ascii' codec can't encode character [...]
4  >>> sequence = bytes(string, 'utf-8')
5  >>> string
6  'ñandú'
7  >>> sequence
8  b'\xc3\xb1and\xc3\xba'
9  >>> len(sequence)
10 7
```

Listado 18.8: Cadena codificada en UTF8

Esta vez, al intentar codificar la cadena «`ñandú`» con el *encoding* ASCII se produce un error (**línea 2**), porque ASCII no puede representar las letras `ñ` y `ú`. Pero UTF8 sí que puede, aunque requiere 2 bytes para esos caracteres 'no ASCII'. Por eso, la secuencia equivalente requiere 7 bytes (**línea 10**) a pesar de que la cadena solo tiene 5 caracteres. Nótese que la conversión puede lograrse utilizando los constructores de ambos tipos (`bytes` y `str`) o bien los métodos `encode()` y `decode()` respectivamente:

```
>>> bytes('ñandú', 'utf-8')
b'\xc3\xb1and\xc3\xba'
>>> 'ñandú'.encode('utf-8')
b'\xc3\xb1and\xc3\xba'
>>> str(b'\xc3\xb1and\xc3\xba', 'utf-8')
'ñandú'
>>> b'\xc3\xb1and\xc3\xba'.decode('utf-8')
'ñandú'
```

Listado 18.9: Constructores y métodos `str.encode()` y `bytes.decode()`

18.6. Empaquetado

El módulo `struct` de la librería estándar de Python puede hacer transformaciones hacia y desde binario de un modo mucho más flexible y, sobre todo, cómodo. La función `struct.pack()`[8] *serializa* datos nativos de Python generando una secuencia de bytes (tipo `bytes`) según la especificación de tamaño y ordenamiento que se le indique. Por ejemplo, el siguiente listado serializa el número 5 como un entero de 32 bits con ordenamiento *big-endian* y después como *little-endian*:

```
>>> struct.pack('>i', 5)
b'\x00\x00\x00\x05'
>>> struct.pack('<i', 5)
b'\x05\x00\x00\x00'
```

LISTADO 18.10: `struct`: alternativas de ordenamiento de un entero de 32 bits

El primer parámetro de `pack()` es la especificación de la conversión. Hay dos conjuntos de símbolos: uno para especificar ordenamiento (ver tabla La tabla 18.1) y otro para especificar formato (ver tabla 18.2).

El siguiente listado muestra el resultado de aplicar ambos ordenamientos al mismo dato en un computador *little-endian*, pero con un entero de 16 bits.

```
>>> struct.pack('>h', 5)
b'\x00\x05'
>>> struct.pack('<h', 5)
b'\x05\x00'
```

LISTADO 18.11: `struct`: alternativas de ordenamiento de un entero de 16 bits

`@`	ordenamiento nativo del computador (realiza alineamiento)
`=`	ordenamiento nativo
`<`	*little endian*
`>`	*big endian*
`!`	ordenamiento de la red *(big-endian)*

CUADRO 18.1: `struct`: especificación de ordenamiento

El Listado 18.12 muestra el resultado de aplicar los diferentes formatos al mismo dato en un computador *little-endian*.

```
>>> struct.pack('b', 5)
b'\x05'
>>> struct.pack('?', 5)
```

[8] *empaquetar*

`x`	relleno (alineado al siguiente dato)
`c`	carácter (`char`)
`b`	byte con signo
`B`	byte sin signo
`?`	booleano/char
`h`	entero de 16 bits con signo
`H`	entero de 16 bits sin signo
`i`	entero de 32 bits con signo
`I`	entero de 32 bits sin signo
`q`	entero de 64 bits con signo (nativo)
`Q`	entero de 64 bits sin signo (nativo)
`f`	`float`
`d`	`double`
`s`	cadena de caracteres (un número previo indica tamaño)
`P`	entero que puede almacenar una dirección de memoria
`q` ó `Q`	entero equivalente al `long long` de C en la misma arquitectura.

CUADRO 18.2: `struct`: especificación de formato

```
b'\x01'
>>> struct.pack('h', 5)
b'\x05\x00'
>>> struct.pack('i', 5)
b'\x05\x00\x00\x00'
>>> struct.pack('l', 5)
b'\x05\x00\x00\x00\x00\x00\x00\x00'
>>> struct.pack('f', 5)
b'\x00\x00\xa0@'
>>> struct.pack('d', 5)
b'\x00\x00\x00\x00\x00\x00\x14@'
```

LISTADO 18.12: `struct`: empaquetado en diferentes tamaños

Pero lo verdaderamente interesante de `struct` es que la cadena de formato puede especificar un número arbitrario de campos, que corresponden a parámetros sucesivos de la función `pack()`. El siguiente ejemplo empaqueta la cabecera de un mensaje ARP sobre una trama Ethernet (§ 5.3).

El valor para los campos de la trama será:

MAC destino
: `FF:FF:FF:FF:FF:FF` (es una trama broadcast).

MAC origen
: `C4:85:08:ED:D3:07`.

tipo `0x0806`, que corresponde con el protocolo ARP.

En el Listado 18.13 se puede ver cómo construir dicha cabecera, la secuencia de bytes que se obtiene y su equivalente numérico. La cadena de formato (`!6s6sh`) indica que debe codificarse con ordenamiento de red (`!`) y que está compuesto de dos cadenas de 6 bytes (`6s`) y un entero de 16 bits sin signo (`h`).

Lo interesante es que la secuencia resultante siempre tendrá una longitud de 14 bytes independientemente del valor de sus tres argumentos.

```
>>> header = struct.pack('!6s6sh', b'\xFF' * 6, b'\xC4\x85\x08\xED\xD3\x07', 0x0806)
>>> header
b'\xff\xff\xff\xff\xff\xff\xc4\x85\x08\xed\xd3\x07\x08\x06'
>>> list(header)
[255, 255, 255, 255, 255, 255, 196, 133, 8, 237, 211, 7, 8, 6]
```

LISTADO 18.13: `struct`: empaquetando una cabecera Ethernet

18.7. Desempaquetado

La contrapartida de `pack()` es `unpack()`. Esta función toma una cadena de formato con las misma reglas que `pack()` y una secuencia de bytes, que puede haber sido obtenida con `file.read()`, `socket.recv()` o cualquier otra función orientada a lectura de flujos *(streams)*. La función `unpack()` retorna una tupla con los valores que corresponden a cada uno de los campos especificados en la cadena de formato. La función `unpack()` realiza por tanto la *des-serialización*.

El Listado 18.14 realiza la función inversa al anterior. Es decir, a partir de la secuencia de bytes devuelve una tupla con las dos direcciones MAC y el tipo de la trama. La **línea 5** simplemente corrobora que el tercer valor de la tupla (`2054`) coincide efectivamente con el valor hexadecimal `0x0806`.

```
1 >>> header
2 b'\xff\xff\xff\xff\xff\xff\xc4\x85\x08\xed\xd3\x07\x08\x06'
3 >>> struct.unpack('!6s6sh', header)
4 (b'\xff\xff\xff\xff\xff\xff', b'\xc4\x85\x08\xed\xd3\x07', 2054)
5 >>> hex(2054)
6 '0x806'
```

LISTADO 18.14: `struct`: desempaquetando una cabecera Ethernet

18.8. Formatos de serialización binaria

El módulo `struct` permite trabajar con datos binarios de un modo muy sencillo, pero es bastante limitado. Si los datos a serializar son de tamaño variable, como cadenas o secuencias de objetos, la cosa se complica bastante. Para usos más avanzados, sobre todo cuando queremos definir nuestras propias estructuras, existen librerías mucho más potentes como `construct`, Google Protocol Buffers o Apache Avro.

18.9. Serialización textual

En las capas inferiores —en realidad desde transporte hacia abajo— la gran mayoría de los protocolos utilizan serialización binaria porque normalmente eso genera mensajes más compactos y su procesamiento es más rápido y requiere menos recursos. En la capa de aplicación sin embargo, es común encontrar muchos protocolos que utilizan serialización textual.

El formato textual para codificación de datos más usado en la actualidad con mucha diferencia es JSON. Como su nombre indica, JSON genera/reconoce cadenas de texto que replican el formato de la definición de objetos con la sintaxis de JavaScript[9]. El gran éxito de JSON se debe a que es muy legible por humanos y su procesamiento automatizado es muy sencillo, aunque costoso en recursos comparado con los formatos binarios. Hoy en día es una norma estandarizada y muchos lenguajes ofrecen soporte incluso en su librería estándar.

Desde Python, también resulta muy sencillo de manipular. El Listado 18.15 muestra como serializar la hipotética lectura de un sensor de temperatura y humedad, que incluye además un identificar y el estado de carga de su batería. El resultado de la serialización (`payload`) se puede enviar perfectamente como carga útil de un datagrama UDP.

```
>>> import json
>>> import socket

>>> data = {'sensor-id': 'temp-hum-01', 'temperature': 22.5, 'humidity': 45.2,
...         'battery': 0.85}
>>> payload = json.dumps(data)
>>> print(payload)
{"sensor-id": "temp-hum-01", "temperature": 22.5, "humidity": 45.2, "battery": 0.85}
>>> print(type(payload))
<class 'str'>

>>> with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as sock:
...     sock.sendto(payload.encode(), ('203.0.113.2', 55000))
```

LISTADO 18.15: Serialización JSON y envío de la lectura de un sensor

El código con la deserialización (Listado 18.16) muestra la recepción del mensaje y su des-serialización.

```
>>> import json
>>> import socket

>>> with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as sock:
...     sock.bind(('', 55000))
...     data, addr = sock.recvfrom(1024)
```

[9]Hay mucho que puntualizar en esta frase, pero sirve como aproximación.

```
>>> data = json.loads(data.decode())
>>> print(data['temperature'])
22.5
```

LISTADO 18.16: Deserialización JSON de la lectura de un sensor

También, para serialización textual, hay innumerables formatos entre los que podemos destacar XML, YAML, CSV o TOML entre otros.

Y ¿qué más?

La serialización es una parte esencial de las comunicaciones en Internet, pero también una fuente inagotable de errores y confusión, sobre todo entre programadores noveles. El objetivo de este capítulo ha sido aclarar los conceptos más básicos, mostrando ejemplos prácticos que ilustren su funcionamiento. Quedan fuera muchos detalles técnicos, pero a partir de este punto, el lector tendrá la base para profundizar por su cuenta en librerías y formatos con los que se encuentre en su día a día, que a buen seguro serán muchos y variados.

Capítulo 19

Captura y análisis

Al terminar este capítulo, entenderás:

- Algunas de las opciones que `tshark` y `wireshark` ofrecen para captura y análisis de tráfico de red.
- Qué son los filtros de captura y cómo se usan.
- Qué son los filtros de visualización y qué posibilidades ofrecen.
- Cómo generar estadísticas y resultados a medida.
- Cómo recuperar la carga útil de un flujo UDP o conexión TCP.

Ya has realizado muchas capturas de tráfico con `tshark` desde los primeros capítulos. Pero todas esas veces ha sido con un uso muy básico del programa, `tshark` es capaz de mucho más: puede diseccionar mensajes, interpretar el significado de cada campo, mostrar la información de diferentes formas, generar estadísticas... entre otras cosas. En este capítulo verás de qué más es capaz `tshark`, y su hermano mayor `wireshark`, una versión con interfaz gráfica más potente y versátil.

Este tipo de programas se llaman «analizadores de protocolos» o *sniffers*, y resultan extremadamente útiles para detectar problemas en la red, optimizar protocolos y depurar aplicaciones. Permiten capturar tráfico de una interfaz de red específica o de todas, visualizarlo en tiempo real, o almacenarlo en un archivo para poder analizarlo después, y estudiar capturas que han hecho otros.

Además, permiten aplicar filtros de captura para procesar únicamente los mensajes que coinciden con el criterio que te interese, y también filtros de visualización, para elegir qué mensajes de la captura se muestran en cada momento. Es importante entender esta diferencia. El filtro de captura es permanente, los mensajes descartados por el filtro pierden y no formarán parte de la captura. El filtro de visualización, en cambio, se puede aplicar, modificar o eliminar en cualquier momento y no afecta a la captura, solo a lo que se muestra.

19.1. tshark

Después de las múltiples capturas realizadas con `tshark` en capítulos anteriores, seguro que ya tienes bastante clara su utilidad. `tshark` es un analizador similar al veterano `tcpdump`, aunque con mucha más funcionalidad y posibilidades. Esta es una captura sencilla de tráfico ICMP, similar a las anteriores:

```
$ ping ietf.org &
```

```
~# tshark -i eth0 -f icmp
Capturing on eth0
192.168.0.37 -> 64.170.98.30 ICMP 98 Echo (ping) request  id=0x405d, seq=10/2560, ttl=64
64.170.98.30 -> 192.168.0.37 ICMP 98 Echo (ping) reply    id=0x405d, seq=10/2560, ttl=60
192.168.0.37 -> 64.170.98.30 ICMP 98 Echo (ping) request  id=0x405d, seq=11/2816, ttl=64
64.170.98.30 -> 192.168.0.37 ICMP 98 Echo (ping) reply    id=0x405d, seq=11/2816, ttl=60
192.168.0.37 -> 64.170.98.30 ICMP 98 Echo (ping) request  id=0x405d, seq=12/3072, ttl=64
64.170.98.30 -> 192.168.0.37 ICMP 98 Echo (ping) reply    id=0x405d, seq=12/3072, ttl=60
6 packets captured
```

Listado 19.1: `tshark` capturando tráfico ICMP

Como muchos otros programas de consola habituales en los sistemas POSIX, `tshark` acepta una gran variedad de opciones y argumentos que permiten modificar su comportamiento. En el ejemplo anterior, el significado de sus argumentos es el siguiente:

`-i eth0` Capturar tráfico de la interfaz `eth0`.
`-f icmp` Capturar solo mensajes ICMP y descartar el resto.
`-c 6` Capturar únicamente 6 paquetes y terminar.

La información de salida está organizada en columnas:

1. Instante de captura, relativo al primer mensaje[1].
2. Dirección IP origen.
3. Dirección IP destino.
4. Protocolo de la carga útil.
5. Tamaño total del mensaje.
6. Tipo de mensaje.
7. Identificador ICMP.
8. Número de secuencia ICMP.
9. TTL.

La información que se muestra depende del tipo de mensaje capturado. Por ejemplo, si capturas tráfico ARP verás algo muy distinto después de la columna de tamaño:

[1] el instante de captura se ha omitido en este ejemplo.

```
~# tshark -i eth0 -f arp
Capturing on eth0
f8:99:47:ad:f0:0d → f8:5f:2a:c0:ff:ee ARP 60 Who has 192.168.0.37? Tell 192.168.0.1
f8:5f:2a:c0:ff:ee → f8:99:47:ad:f0:0d ARP 42 192.168.0.37 is at f8:5f:2a:c0:ff:ee
f8:e4:45:95:b3:7c → f8:5f:2a:c0:ff:ee ARP 60 Who has 192.168.0.37? Tell 192.168.0.42
f8:5f:2a:c0:ff:ee → f8:e4:45:95:b3:7c ARP 42 192.168.0.37 is at f8:5f:2a:c0:ff:ee
```

Mediante los argumentos es posible afinar la captura con un alto grado de detalle. Además de capturar tráfico de una interfaz de red específica e imprimir un resumen como el anterior, `tshark` también puede cargar una captura desde un archivo (opción `-r`) y, por supuesto, también crear este tipo de archivos para su análisis posterior (opción `-w`).

19.1.1. Acceso privilegiado

Quizá hayas advertido por el '#' que los comandos anteriores se ejecutan con privilegios de `root`. Esto se debe a que para capturar tráfico «crudo» de la interfaz de red se requieren privilegios específicos. Sin embargo, ejecutar cualquier programa con privilegios de administrador es siempre un riesgo de seguridad, de hecho, así lo advierte el propio `tshark` en su salida.

```
~# tshark
Running as user "root" and group "root". This could be dangerous.
```

Existe una alternativa más conveniente para ejecutar `tshark` o programas similares. El núcleo Linux permite asignar «capacidades» *(capabilities)* específicas a un programa (un archivo binario). Las capacidades necesarias para capturar tráfico son `CAP_NET_ADMIN` y `CAP_NET_RAW`. Para aplicarlas al programa que realiza la captura de tráfico para `tshark` (`dumpcap` (`wireshark-common`)) basta ejecutar:

```
~# setcap cap_net_raw,cap_net_admin=eip /usr/bin/dumpcap
```

Esta solución otorga esas capacidades a cualquier usuario que ejecute el programa `dumpcap`. Una alternativa más conservadora es otorgarlas solo a los miembros de un grupo de usuarios específico (*p. ej.* grupo `wireshark`)[2].

19.1.2. Selección de la interfaz de captura

Como en el ejemplo anterior, se puede elegir de qué interfaz de red capturar (con la opción `-i`). Si no se especifica esta opción, `tshark` elegirá la interfaz que se esté utilizando para conectar a Internet. Recuerda que para obtener una lista de interfaces puedes utilizar el comando `ip link` o el veterano `ifconfig` (`net-tools`). También el propio `tshark` puede mostrar un lista

[2]Consulta `http://wiki.wireshark.org/CaptureSetup/CapturePrivileges` para más detalles

de interfaces disponibles, que además incluye algunas pseudo-interfaces que él mismo define y que en realidad no existen en el sistema.

```
$ tshark -D
1. eth0
2. any
3. lo (Loopback)
4. bluetooth-monitor
5. nflog
6. nfqueue
7. dbus-system
8. dbus-session
9. veth2728659
10. veth1362a83
11. br-6018fd38a646
12. br-7e03d0bfa152
13. ciscodump (Cisco remote capture)
14. randpkt (Random packet generator)
15. sdjournal (systemd Journal Export)
16. sshdump (SSH remote capture)
17. udpdump (UDP Listener remote capture)
```

Conviene repasar rápidamente las interfaces que ha encontrado en este caso. Es probable que muchas de ellas también aparezcan cuando ejecutes este comando en tu propio sistema.

eth0 La primera NIC Ethernet, y única en este caso.

any Interfaz virtual que permite capturar tráfico de todas las interfaces.

lo La interfaz *loopback*.

bluetooth-monitor
: Captura de tráfico Bluetooth.

nflog y nfqueue
: Captura de paquetes de Netfilter, el sistema de cortafuegos de Linux.

dbus-system y dbus-session
: Interfaz de captura de mensajes del bus de mensajes local (D-BUS) que utilizan muchos sistemas GNU/Linux.

veth-<id> y br-<id>
: Interfaces Ethernet virtuales y *bridge* respectivamente, comúnmente utilizadas por aplicaciones de virtualización como como Virtualbox, VMWare, KVM, docker, etc. para conexión a red de máquinas virtuales.

ciscodump
: Captura remota de Cisco.

sdjournal
: Captura de mensajes del sistema `systemd`.

randpkt
: Generación de paquetes aleatorios.

sshdump
: Captura remota basado en un túnel SSH.

udpdump
Captura remota de paquetes basado en un flujo UDP.

19.1.3. Limitando la captura

Si no se le indica lo contrario, `tshark` continuará capturando tráfico indefinidamente, ya sea mostrando la salida en consola o almacenándolo en un archivo.

Es posible limitar la captura de varias formas:

-c N
un número de paquetes (que cumplen los filtros).

-a duration:N
durante un número de segundos.

-a filesize:N
un tamaño de archivo de la captura resultante, indicando en KiB.

-a files:N
un número de archivos.

El siguiente comando captura todo el tráfico de red de la interfaz `wlan0` y lo almacena en un archivo llamado `wlan0.pcap` hasta un máximo de 1 MiB.

```
$ tshark -i wlan0 -w wlan0.pcap -a filesize:1024
Capturing on 'wlan0'
529
```

Durante el proceso, `tshark` mostrará en consola la cantidad actual de paquetes capturados.

19.1.4. Filtros de captura

El argumento `-f` sirve para especificar un filtro de captura. El formato de estos filtros constituye todo un lenguaje en sí mismo. La tabla 19.1 contiene unos cuantos ejemplos extraídos de `http://wiki.wireshark.org/CaptureFilters`. Puedes encontrar información detallada en la página de manual de `pcap-filter`.

19.1.5. Formato de salida

Por defecto el programa muestra una línea que resume cada mensaje, como en los ejemplos anteriores. Pero existen otras muchas posibilidades para obtener información detallada de cada campo. La opción `-V` muestra los valores de los campos de cada mensaje (por capa) en un formato de texto legible (Listado 19.2). Una parte de esto ya apareció en la captura 5.1, que

Filtro	Significado
ip	tráfico IP
ip or arp	IP o ARP
tcp and not dst port 80	cualquier protocolo sobre TCP excepto el dirigido al puerto 80 (HTTP)
host 192.168.0.1	hacia o desde la IP indicada
net 192.168.0.0/24 net 192.168	hacia o desde la red indicada
src host 192.168.0.1 src net 192.168	procedente del host o red indicado
port 22	TCP o UDP hacia o desde el puerto 22
tcp dst port 22	hacia el puerto 22 TCP (SSH)
dst host 10.0.0.1 and port 443	tráfico HTTP con destino al puerto 443 del host 10.0.0.1
not ether dst FF:FF:FF:FF:FF:FF	tramas Ethernet no broadcast

CUADRO 19.1: `tshark`: ejemplos de filtros de captura

introduce brevemente la información que ofrece. Aparte de los valores directos que se obtienen al diseccionar las cabeceras, `tshark` también muestra información derivada o calculada que no aparece en los mensajes, como por ejemplo, los intervalos de tiempo entre mensajes o el resultado de la comprobación de los checksums. Esa información aparece entre corchetes.

```
$ tshark -f icmp -c 1 -V
Frame 1: 98 bytes on wire (784 bits), 98 bytes captured (784 bits) on interface 0
    Interface id: 0
    WTAP_ENCAP: 1
    Arrival Time: Feb 12, 2013 16:59:13.856520000 CET
    [Time shift for this packet: 0.000000000 seconds]
    Epoch Time: 1360684753.856520000 seconds
    [Time delta from previous captured frame: 0.000000000 seconds]
    [Time delta from previous displayed frame: 0.000000000 seconds]
    [Time since reference or first frame: 0.000000000 seconds]
    Frame Number: 1
    Frame Length: 98 bytes (784 bits)
    Capture Length: 98 bytes (784 bits)
    [Frame is marked: False]
    [Frame is ignored: False]
    [Protocols in frame: eth:ip:icmp:data]
Ethernet II, Src: Intel_ef:d4:96, Dst: Cisco_3a:c9:40
    Destination: Cisco_3a:c9:40 (00:64:40:3a:c9:40)
        Address: Cisco_3a:c9:40 (00:64:40:3a:c9:40)
        .... ..0. .... .... .... .... = LG bit: Globally unique address (factory default)
        .... ...0 .... .... .... .... = IG bit: Individual address (unicast)
    Source: Intel_ef:d4:96 (c4:85:08:ef:d4:96)
        Address: Intel_ef:d4:96 (c4:85:08:ef:d4:96)
        .... ..0. .... .... .... .... = LG bit: Globally unique address (factory default)
        .... ...0 .... .... .... .... = IG bit: Individual address (unicast)
    Type: IP (0x0800)
Internet Protocol Version 4, Src: 120.12.17.214, Dst: 12.22.58.30
    Version: 4
```

```
    Header length: 20 bytes
    Differentiated Services Field: 0x00 (DSCP 0x00: Default; ECN: 0x00: Not-ECT (Not
    ↪  ECN-Capable Transport))
    Total Length: 84
    Identification: 0x0000 (0)
    Flags: 0x02 (Don't Fragment)
        0... .... = Reserved bit: Not set
        .1.. .... = Don't fragment: Set
        ..0. .... = More fragments: Not set
    Fragment offset: 0
    Time to live: 64
    Protocol: ICMP (1)
    Header checksum: 0x415c [correct]
    Source: 120.12.17.214 (120.12.17.214)
    Destination: 12.22.58.30 (12.22.58.30)
    [Source GeoIP: Unknown]
    [Destination GeoIP: Unknown]
Internet Control Message Protocol
    Type: 8 (Echo (ping) request)
    Code: 0
    Checksum: 0x1fb8 [correct]
    Identifier (BE): 27502 (0x6b6e)
    Identifier (LE): 28267 (0x6e6b)
    Sequence number (BE): 10557 (0x293d)
    Sequence number (LE): 15657 (0x3d29)
    Timestamp from icmp data: Feb 12, 2013 16:59:13.000000000 CET
    [Timestamp from icmp data (relative): 0.856520000 seconds]
    Data (48 bytes)

0000  8c 11 0d 00 00 00 00 00 10 11 12 13 14 15 16 17   ................
0010  18 19 1a 1b 1c 1d 1e 1f 20 21 22 23 24 25 26 27   ........ !"#$%&'
0020  28 29 2a 2b 2c 2d 2e 2f 30 31 32 33 34 35 36 37   ()*+,-./01234567
       Data: 8c110d000000000010111213141516171819 1a1b1c1d1e1f...
       [Length: 48]
```

Listado 19.2: Modo «verboso» de `tshark`

Normalmente esto es demasiada información y lo interesante es aislar específicamente los campos relacionados con la cuestión que te interese en cada caso. Para lograrlo se puede indicar el formato de salida por campos (`-T fields`) y la lista con los campos concretos, que pueden depender de las condiciones de filtrado. El ejemplo del Listado 19.3 filtra los mensajes TCP dirigidos al puerto 80 (presuntamente HTTP) y muestra únicamente las direcciones MAC e IP origen y destino.

```
$ tshark -f "tcp and dst port 80" -T fields -e eth.src -e ip.src -e eth.dst -e ip.dst
c4:85:08:ef:d4:96    120.12.17.214    00:64:40:3a:c9:40    198.252.206.25
c4:85:08:ef:d4:96    120.12.17.214    00:64:40:3a:c9:40    65.121.208.106
c4:85:08:ef:d4:96    120.12.17.214    08:20:12:3d:f4:32     120.12.27.170
c4:85:08:ef:d4:96    120.12.17.214    00:64:40:3a:c9:40    198.252.206.25
c4:85:08:ef:d4:96    120.12.17.214    00:64:40:3a:c9:40    173.194.34.196
```

```
c4:85:08:ef:d4:96    120.12.17.214    00:64:40:3a:c9:40    173.194.34.196
c4:85:08:ef:d4:96    120.12.17.214    00:64:40:3a:c9:40    173.194.34.196
c4:85:08:ef:d4:96    120.12.17.214    00:64:40:3a:c9:40    108.160.160.160
```

Listado 19.3: `tshark` permite elegir los campos a mostrar

Los nombres de los campos posibles son los mismos que para los «filtros de visualización» (que se tratan más adelante).

Además, es posible generar una representación de la captura en otros formatos, como PostScript XML, por si necesitaras hacer un tratamiento automatizado de los valores capturados.

19.1.6. Filtros de visualización

A partir de la captura obtenida (o cargada desde archivo) puedes aplicar un filtro que determina qué mensajes se muestran, es decir, un filtro de visualización (*display filter*) altera únicamente lo que el usuario puede ver en cada momento, pero a diferencia de los filtros de captura, estos sí se pueden editar obteniendo un subconjunto diferente de entre los mensajes capturados. El formato de estos filtros es distinto al de los filtros de captura, y mucho más potente.

Como ejemplo, la siguiente consola muestra las peticiones HTTP que soliciten una URI que contenga la cadena `'favicon.ico'`, es decir, el icono que los navegadores asocian al guardar una página como favorito *(bookmark)*.

```
$ tshark -R 'http.request.uri contains "favicon.ico"'
Capturing on wlan0
192.168.0.37 -> 93.184.220.111 HTTP 380 GET /i/favicon.ico?m=1317424629g HTTP/1.1
192.168.0.37 -> 217.148.71.165 HTTP 365 GET /favicon.ico HTTP/1.1
192.168.0.37 -> 23.37.161.157 HTTP 633 GET /favicon.ico HTTP/1.1
11 packets dropped
3 packets captured
```

El programa proporciona literalmente miles de filtros. Puedes consultarlos en la referencia en línea[3], en la página de manual para `wireshark-filter`[4] o con el comando `tshark -G fields`. Estos filtros están organizados jerárquicamente siendo el primer componente el nombre del protocolo al que aplican. En el ejemplo anterior, `http.request.uri` se refiere a la URI que aparece en la petición (GET) de los mensajes HTTP[5]. La tabla 19.2 muestra algunos ejemplos representativos similares a `http://wiki.wireshark.org/DisplayFilters`.

[3] `http://www.wireshark.org/docs/dfref/`

[4] `man wireshark-filter`

[5] `http://www.wireshark.org/docs/dfref/t/tcp.html`

Filtro	Significado
icmp.type == 8	Peticiones ICMP Echo
tcp.port == 25	Segmentos TCP hacia o desde el puerto 25
tcp.dstport == 25	Únicamente los dirigidos al puerto 25
arp or icmp	ICMP or ARP de cualquier tipo
ip.addr == 192.168.2.0/24	Generado/dirigido por/hacia un computador de la red indicada
!(ip.dst == 127.0.0.1)	No dirigido a la interfaz *loopback*

CUADRO 19.2: `tshark`: ejemplos de filtros de visualización

Al igual que los filtros de captura, también es posible utilizar operadores lógicos y relacionales que permiten escribir expresiones muy elaboradas[6].

19.1.7. Estadísticas

`tshark` genera una gran variedad de estadísticas útiles[7]. En esta sección se incluyen solo algunas a modo de ejemplo.

proto,colinfo,*filter*,*field*

Añade columnas a la salida habitual. Por ejemplo:

```
$ tshark -z proto,colinfo,tcp.srcport,tcp.srcport
  5.508997 108.160.160.160 -> 192.168.0.37 HTTP 245 HTTP/1.1 200 OK
          (text/plain)  tcp.srcport == 80
  8.928316 173.194.78.125 -> 192.168.0.37 TCP 471
          [TCP segment of a reassembled PDU]  tcp.srcport == 5222
 17.660298 173.194.78.125 -> 192.168.0.37 TCP 199
          [TCP segment of a reassembled PDU]  tcp.srcport == 5222
```

icmp,srt[,*filter*]

Calcula las estadísticas SRT (Service Response Time) del tráfico ICMP Echo de forma similar al programa `ping`. Un ejemplo:

```
$ tshark -z icmp,srt
[...]
10024 packets captured
ICMP Service Response Time (SRT) Statistics (all times in ms):
Filter: <none>
Requests  Replies   Lost      % Loss
1784      1783      1           0,1%
Minimum   Maximum   Mean      Median    SDeviation     Min Frame Max Frame
55,228    818,109   62,742    56,894    37,105         7831      5373
```

[6] `http://www.wireshark.org/docs/wsug_html_chunked/ChWorkBuildDisplayFilterSection.html`

[7] Puedes ver todas las estadísticas disponibles con `tshark-z help`

io,phs[,*filter*]

Contabiliza mensajes y bytes de todos los mensajes (conforme al filtro) desglosando los resultados según su encapsulación.

```
$ tshark -z io,phs -q -c 100
Capturing on wlan0
100 packets captured
Protocol Hierarchy Statistics
Filter:
eth                                      frames:100 bytes:18503
  ip                                     frames:98 bytes:18289
    icmp                                 frames:32 bytes:3136
    udp                                  frames:34 bytes:3760
      dns                                frames:32 bytes:3408
      db-lsp-disc                        frames:2 bytes:352
    tcp                                  frames:32 bytes:11393
      ssl                                frames:14 bytes:10189
        tcp.segments                     frames:2 bytes:1157
  llc                                    frames:2 bytes:214
    data                                 frames:2 bytes:214
```

io,stat,*interval*,*filter*,*filter*

Contabiliza mensajes y bytes en intervalos del tiempo especificado. Permite indicar una cantidad arbitraria de filtros que serán representados como columnas.

```
$ LANG=C tshark -nzio,stat,0.5,udp,"tcp.dstport==80",
[...]
794 packets captured
================================================================
| IO Statistics                                                |
|                                                              |
| Interval size: 0.5 secs                                      |
| Col 1: udp                                                   |
|     2: tcp.dstport==80                                       |
|     3: Frames and bytes                                      |
|--------------------------------------------------------------|
|            |1              |2              |3               |
| Interval   | Frames | Bytes | Frames | Bytes | Frames | Bytes |
|--------------------------------------------------------------|
| 0.0 <> 0.5 |      2 |   211 |      2 |   132 |     30 |  9012 |
| 0.5 <> 1.0 |      0 |     0 |      9 |   498 |     18 |  1044 |
| 1.0 <> 1.5 |      2 |   211 |      0 |     0 |      4 |   407 |
| 1.5 <> 2.0 |      0 |     0 |      7 |   378 |     14 |   798 |
| 2.0 <> 2.5 |      6 |   806 |     11 |  1597 |     24 |  3376 |
| 2.5 <> 3.0 |     17 |  1780 |     61 | 11138 |    133 | 42969 |
================================================================
```

io,stat,*interval*,*func*

Aplica funciones de agregación (`COUNT`, `SUM`, `MIN`, `MAX`, `AVG` y `LOAD`) que se pueden aplicar a campos cualesquiera para generar columnas en la tabla de

resultados. El siguiente ejemplo cuenta el número de paquetes IP, y calcula la media y la suma de los tamaños de esos paquetes en intervalos de 2 segundos.

```
$ LANG=C tshark -nzio,stat,2,"COUNT(ip)ip","AVG(ip.len)ip.len","SUM(ip.len)ip.len"
[...]
803 packets captured
==================================
| IO Statistics                  |
|                                |
| Interval size: 2 secs          |
| Col 1: COUNT(ip)ip             |
|     2: AVG(ip.len)ip.len       |
|     3: SUM(ip.len)ip.len       |
|--------------------------------|
|          |1      |2    |3      |
| Interval | COUNT | AVG |   SUM  |
|--------------------------------|
|  0 <>  2 |     7 |  61 |    432 |
|  2 <>  4 |     0 |   0 |      0 |
|  4 <>  6 |   107 | 115 |  12354 |
|  6 <>  8 |   486 | 267 | 130039 |
|  8 <> 10 |   126 | 497 |  62628 |
| 10 <> 12 |    20 |  45 |    904 |
| 12 <> 14 |    38 |  40 |   1544 |
==================================
```

follow,*proto*,*mode*,*filter*[,*range*]

Muestra el contenido de un flujo de mensajes entre dos nodos, es decir, concatena la carga útil de los segmentos sucesivos que corresponden a la misma sesión o conexión. De este modo, si por ejemplo se está transmitiendo un archivo podría recuperarse a partir de la captura. Los protocolos soportados son TCP y UDP. Y puede hacer el volcado en tres modos diferentes: ascii, hex y raw.

```
$ tshark -z follow,tcp,hex,192.168.2.12:47147,129.42.58.216:80
```

conv,*type*[,*filter*]

Muestra las «conversaciones» o flujos, es decir, nodos que intercambian mensajes entre sí de forma recurrente. El parámetro *type* puede ser uno de: eth, fc, fddi, ip, ipv6, ipx, tcp, tr, udp.

```
$ tshark -z conv,tcp
TCP Conversations
Filter:<No Filter>                       |  <-  |  ->   | Total  | Start   | Durat |
                                         | F  B | F  B  | F   B  |         |       |
192.168.0.37:38216 <-> 73.233.104.123:80   1  74  2  140  3   214  0,002912  0,2126
192.168.0.37:58949 <-> 92.184.220.111:80   1  66  2  128  3   194  0,002801  0,1134
192.168.0.37:58948 <-> 92.184.220.111:80   1  66  2  128  3   194  0,002754  0,1118
192.168.0.37:58947 <-> 92.184.220.111:80   1  66  2  128  3   194  0,002704  0,1027
```

```
192.168.0.37:58946 <-> 92.184.220.111:80   1  66  2  128  3   194  0,002660  0,1015
192.168.0.37:56569 <-> 92.100.127.144:80   1  74  2  140  3   214  0,002398  0,0727
192.168.0.37:38838 <-> 95.172.94.11:80     1  62  2  128  3   190  0,002319  0,0952
192.168.0.37:38837 <-> 95.172.94.11:80     1  62  2  128  3   190  0,002270  0,0935
```

19.2. wireshark

`wireshark` es una herramienta gráfica que amplía las posibilidades de `tshark`. Permite visualizar el tráfico capturado de un modo más flexible, potente e intuitivo, y además ofrece herramientas de análisis más sofisticadas.

19.2.1. Interfaz gráfica

Al arrancar `wireshark` aparece una ventana (Figura 19.1) que muestra la lista de interfaces disponibles para captura (las mismas de `tshark`, § 19.1.2). Desde aquí puedes especificar un filtro de captura (opcional), seleccionar una interfaz y pulsar el botón de captura: .

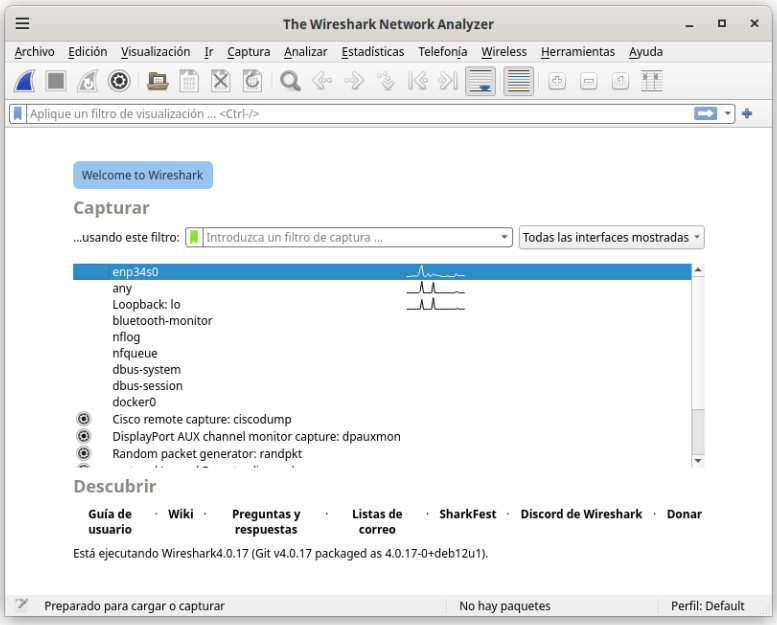


FIGURA 19.1: `wireshark`: interfaz inicial

Inmediatamente la ventana muestra 3 paneles:

1. El panel superior muestra la lista de mensajes capturados, con una línea por mensaje, que se va actualizando en tiempo real conforme

van llegando nuevos mensajes. Se muestran por defectos las columnas No., Time, Source, Destination, Protocol, Length e Info.

2. Abajo a la izquierda, aparece un árbol expandible que incluye todos los detalles de los campos de cada una de las cabeceras de los distintos protocolos encapsulados en cada mensaje.

3. Abajo a la derecha, el contenido completo del mensaje en formato hexadecimal en un formato prácticamente idéntico al de `tshark` con la opción `-V` (§ 5.1).

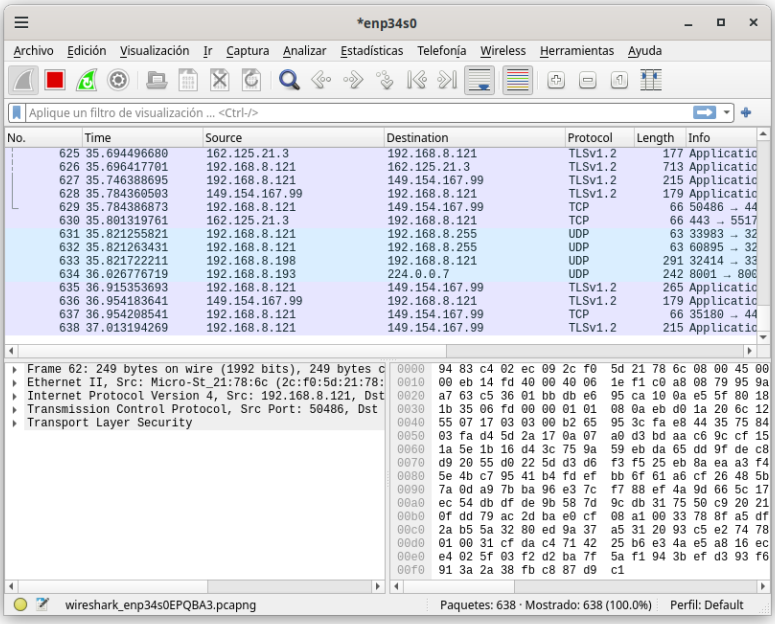


FIGURA 19.2: `wireshark`: interfaz de captura por defecto

Esta vista por defecto no parece la más útil. Puedes cambiar entre distintas disposiciones en el menú Edición→Preferencias→Apariencia→Diseño y elegir el diseño de 3 filas (el primero) que corresponde con lista de mensajes, detalles y contenido (bytes) tal como muestra la Figura 19.3. De ese modo la interfaz se verá como en la Figura 19.4.

Habrás comprobado que el programa sigue capturando tráfico indefinidamente, hasta que pulses el botón de parada: ■.

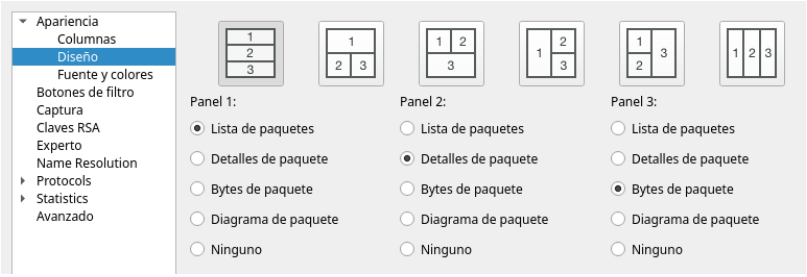


FIGURA 19.3: `wireshark`: configuración de paneles

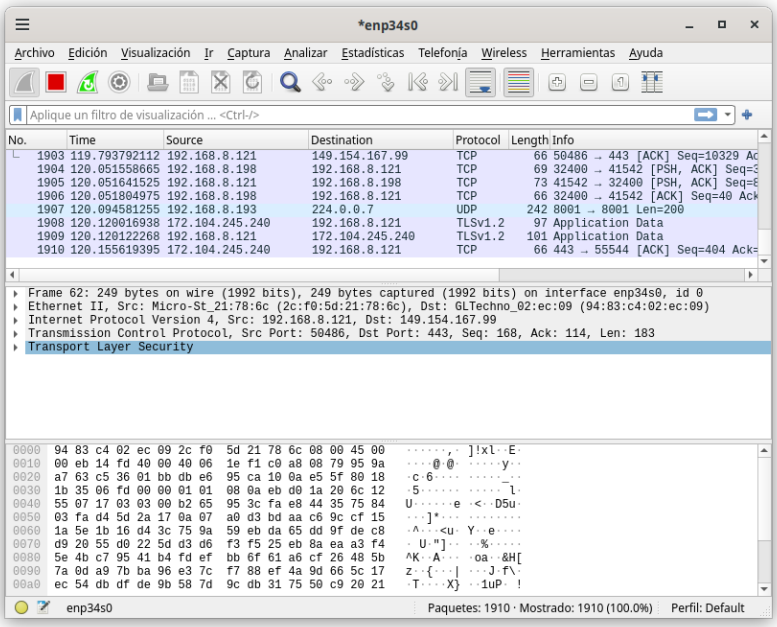


FIGURA 19.4: `wireshark`: interfaz con las 3 filas: lista, detalle y bytes

19.2.2. Captura y filtrado

Como ejemplo, repetamos con `wireshark` la captura TCP de §5.9.2. En la ventana inicial escribe `tcp port 2000` como filtro de captura, selecciona la interfaz *loopback* y lanza la captura (Figura 19.5).

Comprobarás que no aparece nada... es normal, por el momento no hay tráfico que coincida con el filtro de captura. Vamos a generarlo recreando la misma sesión TCP con `ncat`, es decir, en una consola ejecuta el servidor con `ncat -l -p 2000` y en otra el cliente con `echo hola | ncat 127.0.0.1`

`2000`. Inmediatamente verás que aparecen 8 mensajes en el panel superior, como en la Figura 19.6.

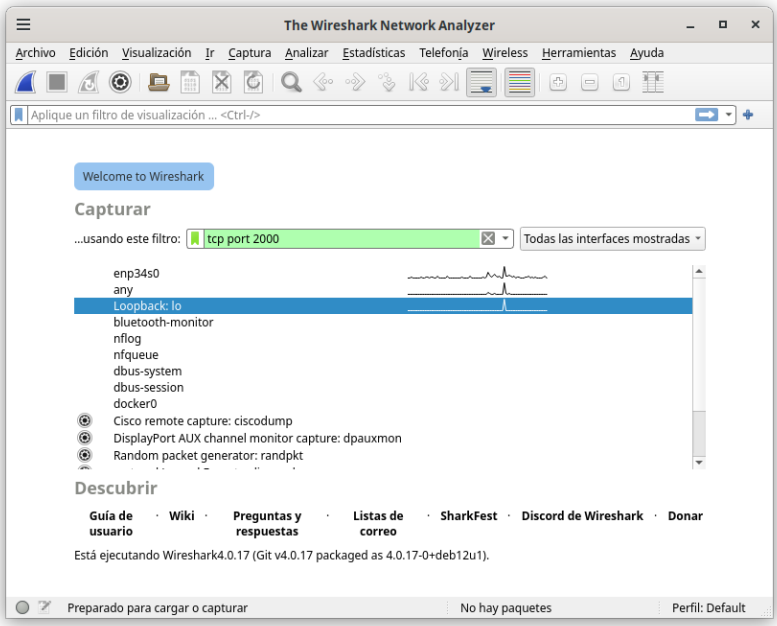


FIGURA 19.5: `wireshark`: estableciendo una captura en la interfaz *loopback* para el puerto TCP 2000

El primero de los mensajes está seleccionado y eso permite explorar todos sus campos en el panel central. En la figura, la parte correspondiente a la cabecera TCP se muestra expandida y aparece el valor de cada campo. En esta segunda sección, el campo seleccionado es `flags` con el valor `0x002`, que corresponde a la activación del flag SYN. Efectivamente se trata del segmento de establecimiento de conexión enviado por el cliente. A su vez, esto provoca que los bytes del mensaje que corresponden a ese campo de la cabecera aparezcan resaltados en el panel inferior.

Aquí puedes establecer un filtro de visualización para, por ejemplo, quedarte únicamente con los segmentos TCP que tienen carga útil. Ese filtro (`tcp.len > 0`) se escribe en la barra, que queda resaltada con fondo verde cuando es correcto (Figura 19.7). El único mensaje que cumple el filtro es el que lleva la cadena «hola\n», y lo puedes ver resaltado en los paneles central e inferior.

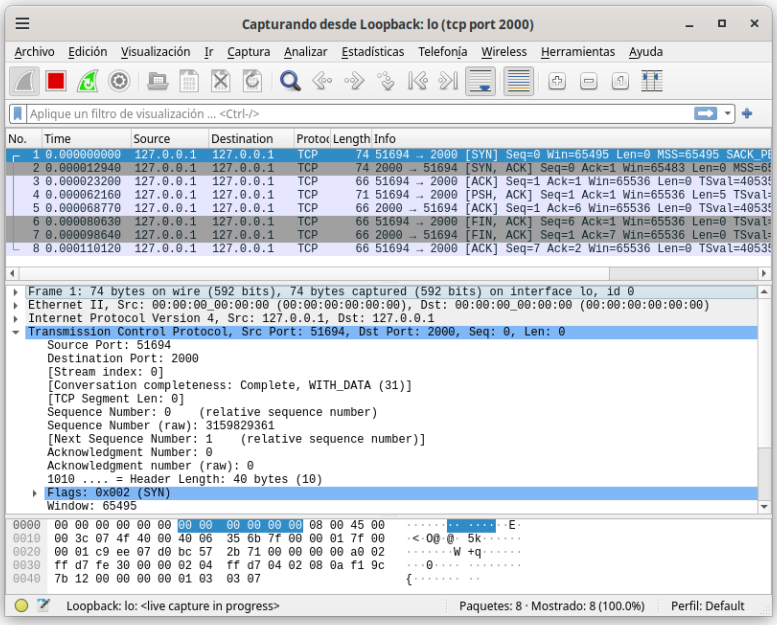


FIGURA 19.6: `wireshark`: captura de una conexión TCP simple

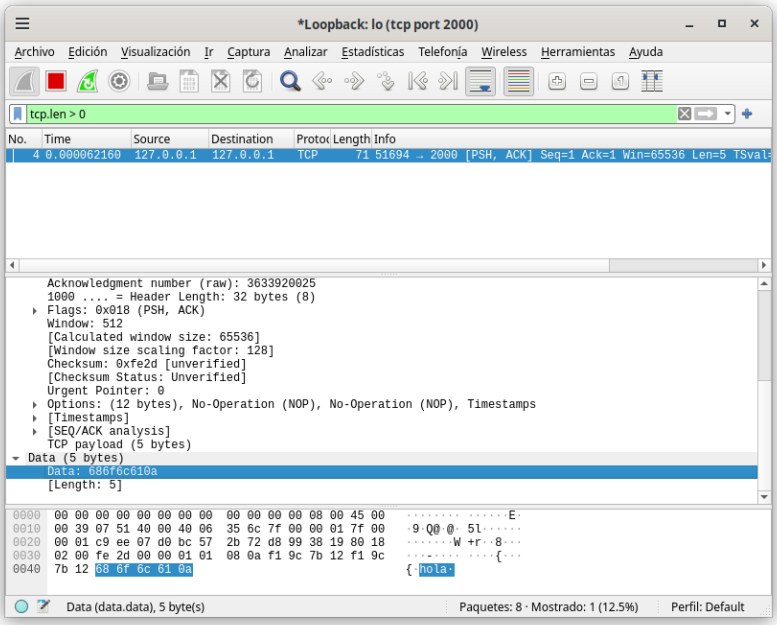


FIGURA 19.7: `wireshark`: filtro de visualización (mensajes con carga útil)

19.3. Captura de flujos

`wireshark` ofrece la posibilidad de capturar la carga útil de una conexión TCP o de un flujo UDP en uno o en ambos sentidos. Esto significa que, por ejemplo, es posible recuperar y reconstruir un archivo que se está transmitiendo a través de la red, asumiendo, por supuesto, que has sido capaz de capturar todos los mensajes.

Para empezar debes utilizar `tshark` para capturar el tráfico dirigido al puerto TCP 2000 y lo puedes guardar en un archivo. Después analizaremos ese archivo para recuperar la carga útil de la conexión completa tanto con `wireshark` como con herramientas de línea de comandos. Por supuesto, con `tshark` por ejemplo, aunque por supuesto lo puedes hacer igualmente con `wireshark`.

```
$ tshark -i lo -f "tcp port 2000" -w captura.pcap
```

Ahora envía un archivo (un `.mp3` concretamente, aunque sirve cualquier cosa) a través de una conexión TCP que puedes conseguir con un servidor `ncat` a la escucha en el puerto 2000 y un cliente (también `ncat`) con el que enviar el archivo:

Cliente

```
$ cat original.mp3 | ncat localhost 2000
```

Servidor

```
$ ncat -l -p 2000 > recibido.mp3
```

Figura 19.8: Transfiriendo un `.mp3` con `ncat`

Una vez terminado el envío, puedes parar la captura con `Ctrl+C`. Y la puedes cargar en `wireshark` con la opción Archivo→Abrir o desde consola simplemente con:

```
$ wireshark captura.pcap
```

Ahora verás la lista de todos los mensajes que forman la captura. Puedes abrir el menú contextual pulsando con el botón derecho del ratón en cualquier de los mensajes. Elige Seguir→TCP Stream. Aparecerá una nueva ventana como la de la Figura 19.9.

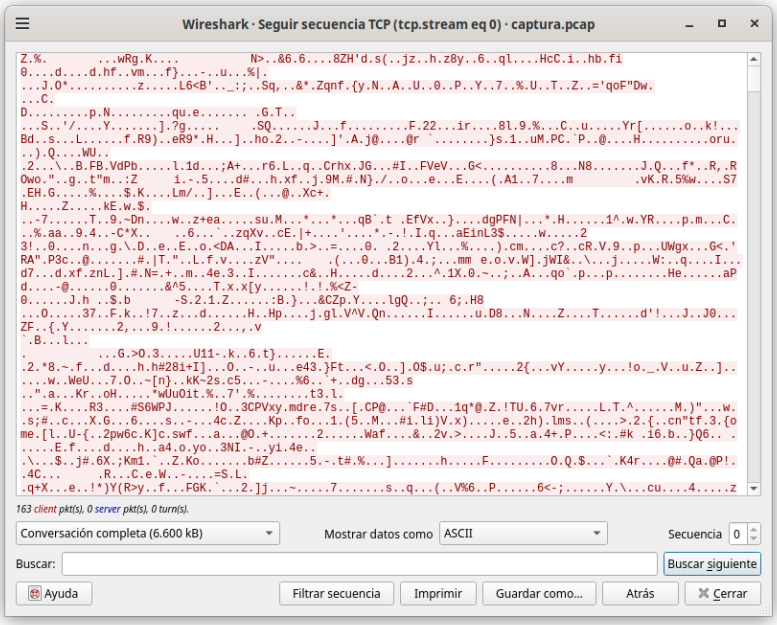


Figura 19.9: `wireshark`: reconstrucción de un flujo TCP

En los controles que aparecen en la parte inferior elige «Mostrar como: raw» y pulsa el botón «Guardar como...». Guarda el archivo con el nombre captura.mp3».

Por supuesto, puedes abrirlo con cualquier reproductor multimedia, pero también puedes comprobar que el archivo recuperado es idéntico al original utilizando el comando `diff` o calculando el MD5 de ambos archivos.

```
$ md5sum *
c2b91faddfec1d01c4f81a8a6d34d537  original.mp3
c2b91faddfec1d01c4f81a8a6d34d537  captura.mp3
```

Puedes conseguir lo mismo desde línea de comandos con:

```
$ tcpflow -r captura.pcap
```

El programa genera un archivo con la carga útil para cada uno de los flujos y sentidos contenidos en la captura. En este caso solo crea un archivo, ya que el sentido servidor a cliente de la conexión TCP no ha transportado ningún dato. El archivo se llama en este caso `127.000.000.001.58008-127.000.000.001.02000` que está formado por las direcciones IP y puertos de origen y destino de la conexión. Igual que antes, puedes comprobar que el archivo es idéntico al original.

```
$ md5sum *
c2b91faddfec1d01c4f81a8a6d34d537  original.mp3
c2b91faddfec1d01c4f81a8a6d34d537  captura.mp3
c2b91faddfec1d01c4f81a8a6d34d537  127.000.000.001.58008-127.000.000.001.02000
```

Resulta sorprendentemente sencillo recuperar un archivo completo que se transfiere por la red si se dan las condiciones y se dispone de las herramientas adecuadas. Es fácil entender las consecuencias que esto tiene para la privacidad. Por supuesto, si el archivo se transmite sobre un canal cifrado (como TLS) esto no será posible, o al menos, no tan sencillo.

Y ¿qué más?

Este capítulo ha sido un vistazo a algunas de las posibilidades de captura, visualización, filtrado, análisis y generación de estadísticas que ofrecen `tshark` y `wireshark`. Echando un vistazo a su documentación es fácil percatarse de que se trata de herramientas extraordinariamente potentes, que se utilizan habitualmente en ámbitos profesionales de la seguridad informática, la administración de redes, el desarrollo de software de comunicaciones, etc. A partir de aquí, invitamos al lector a explorar su gran potencial. Hay mucho que descubrir y aprender.

Capítulo 20

Sockets raw

Los sockets más usados con diferencia son los TCP seguidos de los UDP. Pero hay muchos otros tipos de socket. Este capítulo es una introducción muy práctica a los «sockets raw»[1].

El término *raw* en informática se suele utilizar para indicar acceso directo a los datos que proporciona un dispositivo, o al menos con menor intervención del SO o librerías involucradas[2]. Este acceso directo tiene tres implicaciones principales:

- Mayor flexibilidad, al no estar limitado por las reglas o normas que impongan las capas de alto nivel que ofrece el sistema operativo.
- Acceso privilegiado, debido precisamente a que dichas posibilidades tienen un impacto directo sobre la seguridad del sistema y la privacidad de sus usuarios.
- Menos soporte, ya que son precisamente las capas del sistema operativo que se dejan a un lado las que simplifican el manejo del recurso. El «modo raw» conlleva un nivel de abstracción mucho menor y por tanto, más complejidad técnica.

Estas tres cuestiones se pueden aplicar casi a cualquier dispositivo que permita un acceso «raw», sea un periférico USB, una consola o, como en este caso, un socket.

Con los sockets `AF_INET:SOCK_STREAM` o `AF_INET:SOCK_DGRAM` no es posible acceder (para leer o escribir) a las cabeceras de ninguno de los protocolos de TCP/IP de la capa de transporte o inferior, ya sea IP, ICMP, ARP, TCP, etc. Esos sockets únicamente permiten indicar cuál será la carga útil de los segmentos TCP o UDP y solo indirectamente se puede influir en algunos de los campos de sus cabeceras: puerto origen y destino y poco más[3].

[1] A veces (dolorosamente) traducido como «conector directo».
[2] El adjetivo *raw* (crudo) se utiliza como contraposición a *cooked* (cocinado).
[3] a menos que acudamos a llamadas al sistema como `setsockopt()`[sc].

En raras situaciones se necesita ofrecer servicios que implican a protocolos de capas 2 y 3, o a las cabeceras de tramas, paquetes y segmentos, que normalmente quedan fuera de la vista del programador. Algunos programas de este tipo pueden ser `ping`, `traceroute`, `arping` o un *sniffer* cualquiera. Entonces ¿cómo se hacen estos programas? La respuesta, como habrás imaginado, pasa por los sockets raw.

20.1. Acceso privilegiado

En § 19.1.1 hablamos de la necesidad de `sudo` o *capabilities* para ejecutar un sniffer y eso se debe precisamente a que utilizan sockets raw, de modo que todo eso es aplicable a cualquier programa que escribamos y también los utilice.

Lamentablemente, las *capabilities* solo se pueden aplicar a programas binarios, no a scripts de Python. Por eso, para ejecutar los programas que veremos en este capítulo, tendrás que utilizar `sudo`.

Aparte de la ejecución del programa, también necesitas configurar la interfaz de red en «modo promiscuo». Si tu interfaz de red es una tarjeta Ethernet o WiFi, únicamente las tramas broadcast, multicast o que vayan dirigidas específicamente a su dirección MAC serán capturadas y entregadas al subsistema de red. Sin embargo, si pretendes utilizar un socket raw, es muy probable que te interese recibir todo el tráfico que llegue a la interfaz de red de tu computador, y no solo el mencionado.

Para lograrlo debes activar el «modo promiscuo» de la NIC, algo que puede hacer con el comando `ip`:

```
~# ip link set eth0 promisc on
```

O con `ifconfig`:

```
~# ifconfig eth0 promisc
```

De forma análoga puedes saber si la interfaz está en modo promiscuo con el siguiente comando (fíjate en el flag PROMISC).

```
$ ip link show eth0
2: eth0: <BROADCAST,MULTICAST,PROMISC,UP,LOWER_UP> mtu 1500 qdisc \
         pfifo_fast state UP qlen 1000
    link/ether f8:5f:2a:c0:ff:ee brd ff:ff:ff:ff:ff:ff
```

Y el equivalente con `ifconfig`:

```
$ /sbin/ifconfig eth0
eth0      Link encap:Ethernet  HWaddr f8:5f:2a:c0:ff:ee
          inet addr:192.168.2.4  Bcast:192.168.2.255  Mask:255.255.255.0
```

```
          inet6 addr: fe80::21e:c9ff:fe34:7e92/64 Scope:Link
          UP BROADCAST RUNNING PROMISC MULTICAST  MTU:1500  Metric:1
          RX packets:160791 errors:0 dropped:0 overruns:0 frame:0
          TX packets:121923 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:177101459 (168.8 MiB)  TX bytes:18361567 (17.5 MiB)
          Memory:fdfe0000-fe000000
```

20.2. Tipos de sockets raw

Lo primero a tener en cuenta es que hay dos tipos básicos de socket raw, y que la decisión de cuál utilizar depende totalmente del objetivo y requisitos de la aplicación que se desea:

Familia `AF_PACKET`
: Los sockets raw de la familia `AF_PACKET` son los de más bajo nivel y permiten leer y escribir cabeceras de protocolos de cualquier capa.

Familia `AF_INET`
: Los sockets raw `AF_INET` delegan al sistema operativo la construcción de las cabeceras de enlace y permiten una manipulación «compartida» de las cabeceras de red.

En las próximas secciones veremos en detalle la utilidad y funcionamiento de ambas familias.

20.3. Sockets `AF_PACKET:SOCK_RAW`

Son los sockets raw más flexibles y de más bajo nivel, y representan la elección obligada si el objetivo es crear un *sniffer* o algo parecido. Precisamente el siguiente listado es un *sniffer* extremadamente básico que imprime por consola las tramas Ethernet/WiFi completas recibidas por cualquier interfaz y portando cualquier protocolo.

```
import socket

ETH_P_ALL = 3

sock = socket.socket(socket.AF_PACKET, socket.SOCK_RAW,
                     socket.htons(ETH_P_ALL))

while 1:
    print("--\n{!r}".format(sock.recvfrom(1600)))
```

LISTADO 20.1: Sniffer básico con `AF_PACKET:SOCK_RAW`
/raw/sniff-all.py

Y a continuación el programa en funcionamiento:

```
~# ./sniff-all.py
--
(b'\xff\xff\xff\xff\xff\xff\xa8\x92,
\xce\xcd\xb3\x08\x06\x00\x01\x08\x00\x06\x04\x00\x01\xa8\x92,
\xce\xcd\xb3\xac\x13\xb0O\x00\x00\x00\x00\x00\x00\xac\x13\xb0\x01',
('eth0', 2054, 1, 1, b'\xa8\x9d,\xde\xce\xb5'))
```

Haciendo modificaciones mínimas a este programa es posible filtrar el tráfico en dos aspectos:

Tipo de trama
: Es decir, el código que identifica el protocolo encapsulado como carga útil.[4] Para ello se utiliza el tercer campo del constructor de `socket`.

La interfaz de red
: Se logra vinculando el socket a una interfaz de red concreta por medio del método `bind()`.

El uso de ambos «filtros» queda demostrado en el siguiente programa, llamado `sniff-arp.py`. Solo muestra mensajes ARP recibidos por la interfaz que se indique como argumento:

```
import sys
import socket

if len(sys.argv) != 2:
    print("usage: {} <iface>".format(sys.argv[0]))
    exit(1)

ETH_P_ARP = 0x0806

sock = socket.socket(socket.AF_PACKET, socket.SOCK_RAW,
                     socket.htons(ETH_P_ARP))
sock.bind((sys.argv[1], ETH_P_ARP))

while 1:
    print("--\n{!r}".format(sock.recv(1600)))
```

LISTADO 20.2: Sniffer `AF_PACKET:SOCK_RAW` filtrando ARP
`/raw/sniff-arp.py`

Y el programa en acción:

```
~# ./sniff-arp.py wlan0
--
b'\xff\xff\xff\xff\xff\xffl>m\x84y\x1d\x08\x06\x00\x01\x08\x00\x06\x04\x00\x01l>m\x84y
\x1d\xa1C\x11<\x00\x00\x00\x00\x00\x00\xa1C\x11\x01\x00\x00\x00\x00\x00\x00\x00\x00\x00
\x00\x00\x00\x00\x00\x00\x00\x00\x00'
```

[4] `http://www.iana.org/assignments/ethernet-numbers`

Si te fijas, es fácil identificar la cabecera Ethernet en esa secuencia de bytes. Aparecen 6 bytes `0xff`, que corresponden a la dirección broadcast de Ethernet; y más adelante `0x0806`, que es el tipo para payload ARP.

De este modo tan sencillo es posible realizar un *sniffer* completamente a medida de las necesidades concretas. Pero todo esto solo sirve para leer tramas. El siguiente apartado explica cómo enviar, lo que abre un interesante mundo de posibilidades (y riesgos de seguridad).

Si quieres identificar el origen del paquete puedes utilizar el método `recvfrom()` en lugar de `recv()`. En ese caso el valor de retorno es una tupla que incluye, entre otras cosas, el nombre de la interfaz (*p. ej.* «eth0») y la dirección MAC origen como una secuencia de bytes.

20.3.1. Construir y enviar tramas

El mismo socket creado en los ejemplos anteriores se puede utilizar para enviar datos. Para sintetizar un paquete, es decir, construir cabeceras de acuerdo a las especificaciones, se utiliza normalmente el módulo `struct`[5].

El siguiente listado envía una cabecera Ethernet cuyos campos son:

`Destino`: `FF:FF:FF:FF:FF:FF`
`Origen`: `00:01:02:03:04:05`
`Protocolo`: 0x0806 (ARP)

```
import sys
import socket
import struct

if len(sys.argv) != 2:
    print("usage: {} <iface>".format(sys.argv[0]))
    exit(1)

ETH_P_ARP = 0x0806

sock = socket.socket(socket.AF_PACKET, socket.SOCK_RAW,
                     socket.htons(ETH_P_ARP))
sock.bind((sys.argv[1], ETH_P_ARP))

sock.send(struct.pack('!6s6sh', 6 * b'\xFF',
                      b'\x00\x01\x02\x03\x04\x05', ETH_P_ARP))
```

LISTADO 20.3: Enviando una trama Ethernet con `AF_PACKET:SOCK_RAW`
/raw/send-wrong-eth.py

[5]Consulte el capítulo 18 para más información sobre dicho módulo.

Si capturas esa trama con `wireshark` o `tshark` verás que aparece con un «malformed packet», y con razón: es solo una cabecera ¡no tiene carga útil!

```
$ tshark -a duration:7 -n -f arp
Capturing on 'wlan0'
    1 0.000000000 00:01:02:03:04:05 → ff:ff:ff:ff:ff:ff ARP 14 [Malformed Packet]
```

Y eso lógicamente contradice todas las normas del protocolo Ethernet. Resumiendo, este programa no sirve para nada, solo para que veas que se puede construir y enviar lo que quieras a la red, incluso aunque sea un completo sinsentido.

20.3.2. Implementando un `arping`

Aunque hay muchas variantes, el programa `arping` envía una petición ARP Request y espera la respuesta correspondiente. Esta sección presenta una implementación que sirve para ilustrar el uso de los sockets raw de la familia `AF_PACKET`.

20.3.2.1. Generando mensajes

El programa necesita enviar mensajes ARP Request, que irán encapsulados en tramas Ethernet. Una forma de implementar esta tarea (llamada a veces «sintetizar paquetes») y aprovechar la POO es escribir una clase por cada tipo de mensaje. Por tanto, la clase para generar el mensaje ARP Request es algo tan sencillo como esto:

```
class Ether:
    def __init__(self, hwsrc, hwdst):
        self.hwsrc = hwsrc
        self.hwdst = hwdst
        self.payload = None

    def set_payload(self, payload):
        self.payload = payload
        payload.frame = self

    def serialize(self):
        retval = struct.pack("!6s6sh", self.hwdst, self.hwsrc,
                             self.payload.proto) + self.payload.serialize()

        return retval + (60-len(retval)) * b"\x00"
```

Lo único a destacar de la clase `Ether` es el método `serialize()` que se encarga de generar la representación binaria de los datos que corresponden a la cabecera, concretamente dirección MAC destino, MAC origen, protocolo (el que indique el payload) y a continuación el payload propiamente dicho.

Esos datos se empaquetan en binario gracias a `struct.pack()`[6] indicando que se trata de 2 secuencias de 6 bytes (`6s6s`) y un entero de 16 bits (`h`). La última línea de ese método calcula y concatena el relleno (*padding*) necesario para que la trama alcance el tamaño mínimo necesario de 60 bytes.

La clase para generar mensajes ARP Request es incluso más sencilla:

```
class ArpRequest:
    proto = ETH_P_ARP

    def __init__(self, psrc, pdst):
        self.psrc = socket.inet_aton(psrc)
        self.pdst = socket.inet_aton(pdst)
        self.frame = None

    def serialize(self):
        return struct.pack("!HHbbH6s4s6s4s", 0x1, 0x0800, 6, 4, 1,
                           self.frame.hwsrc, self.psrc, b"\x00",  self.pdst)
```

20.3.2.2. Leyendo mensajes

La otra funcionalidad importante del programa es reconocer los mensajes que se obtendrán como respuesta si todo va bien. Se trata de discretizar el valor de cada campo representándolo en un formato adecuado. Esa tarea se suele llamar «disección de paquetes». Como en el caso anterior, una buena forma de hacer esto es delegar el reconocimiento (*parsing*) de cada tipo de mensaje en una clase específica. Hace falta una clase para reconocer tramas Ethernet y otra para reconocer mensajes ARP Reply.

La clase para reconocer tramas Ethernet puede ser algo tan sencillo como esto:

```
class EtherDissector:
    def __init__(self, frame):
        try:
            (self.hwdst,
            self.hwsrc,
            self.proto) = struct.unpack("!6s6sh", frame[:14])
        except struct.error:
            raise DissectionError

        self.payload = frame[14:]
```

El constructor acepta por parámetro una secuencia de bytes, es decir, la trama tal como se lee del socket. Los valores que «desempaqueta» con `struct` y que estarán accesibles como atributos públicos son: dirección MAC destino, MAC origen, protocolo y payload.

[6] Ver `http://docs.python.org/library/struct.html#format-characters`

El disector del mensaje ARP Reply, llamado `ArpReplyDissector`, es también muy sencillo:

```
class ArpReplyDissector:
    def __init__(self, msg):
        self.msg = msg

        if struct.unpack("!H", self.msg[6:8])[0] != ARP_REPLY:
            raise DissectionError

        try:
            (self.hwsrc, self.psrc,
            self.hwdst, self.pdst) = struct.unpack("!6s4s6s4s", msg[8:28])
        except struct.error:
            raise DissectionError
```

El constructor de la clase acepta por parámetro una secuencia de bytes, que corresponden con la carga útil de una trama. Como antes, los valores de todos los campos quedan disponibles como atributos de la instancia. Si algún campo o formato no corresponde, el constructor lanza la excepción `DissectionError`.

20.3.2.3. Programa principal

Solo queda escribir la función principal, la que realmente crea, lee y escribe en el socket. Aparece en el siguiente listado:

```
def main(ipsrc, ipdst, iface):
    print("Request: Who has {0}? Tell {1}".format(ipdst, ipsrc))

    sock = socket.socket(socket.AF_PACKET, socket.SOCK_RAW,
                         socket.htons(ETH_P_ARP))
    sock.bind((iface, ETH_P_ARP))

    frame = Ether(sock.getsockname()[-1], BROADCAST)
    frame.set_payload(ArpRequest(ipsrc, ipdst))

    sock.send(frame.serialize())

    while 1:
        eth = EtherDissector(sock.recv(2048))

        try:
            arp_reply = ArpReplyDissector(eth.payload)
            if arp_reply.hwdst == frame.hwsrc:
                print("Reply:   {0} is at {1}".format(
                      ipdst, display_mac(arp_reply.hwsrc)))
                break
        except DissectionError:
            print(".")
```

La función `main()` acepta las direcciones IP del host origen y destino, y la interfaz de red (línea 1). Primero crea y vincula el socket a la interfaz solicitada (lineas 4-6). A continuación crea una trama Ethernet con destino

broadcast y origen la MAC de la interfaz (línea 8), y le fija como payload una instancia de `ArpRequest`. El método `send()` envía la trama en su formato binario (línea 11).

El bucle `while` espera la respuesta. En cada iteración se lee y disecciona una trama (línea 14). Si esa trama contiene un mensaje ARP Reply, es decir, si `ArpReplyDissector` no lanza la excepción `DissectionError`, se comprueba además que esa sea la respuesta ARP que se espera y no otra (línea 18). Si es así se imprime la dirección IP del destino y la dirección MAC asociada a esa IP, que es el objetivo final del programa (líneas 19-20). Puedes encontrar una versión ampliada en el archivo `raw/arping.py`.

20.4. Sockets `AF_INET:SOCK_RAW`

A pesar de la flexibilidad y potencia de los sockets `AF_PACKET`, no siempre son la mejor elección ya que el programador debe parsear y generar el contenido de todas las cabeceras. Eso puede ser bastante engorroso cuando entra en juego el cálculo de checksums u otros datos no tan directos.

Los sockets `AF_INET:SOCK_RAW` pueden ser una buena alternativa si solo te interesa «tocar» las cabeceras de transporte, dejando al sistema operativo todo el trabajo relacionado con las de enlace, y opcionalmente las de red.

20.4.1. Capturando mensajes

El siguiente programa imprime por consola todos los paquetes IP que contengan un segmento UDP:

```
import socket

sock = socket.socket(socket.AF_INET, socket.SOCK_RAW,
                     socket.getprotobyname('udp'))
while 1:
    print("--\n{!r}".format(sock.recv(1600)))
```

LISTADO 20.4: Sniffer de mensajes UDP con `AF_INET:SOCK_RAW`
`/raw/sniff-udp.py`

La función `getprotobyname()` devuelve el número de protocolo[7] a partir de su nombre (linea 4). Es interesante destacar que el resultado del método `recv()` es el paquete IP completo, incluyendo cabecera (línea 6).

[7] `http://www.iana.org/assignments/protocol-numbers/protocol-numbers.xml`

Como en el caso de los socket `AF_PACKET` puedes identificar el origen del paquete (su dirección IP) sin tener que parsear la cabecera IP. Para lograrlo utiliza el método `recvfrom()` en lugar de `recv()`. En ese caso el valor de retorno es una tupla con la forma (datos, dirección), teniendo en cuenta que la dirección es su vez una tupla (IP, 0).

20.4.2. Enviando

Para enviar datos sobre este tipo de socket debes utilizar el método `sendto()` indicando la dirección destino. El Listado 20.5 envía envía un segmento UDP «sintético», pero válido, que contiene el texto «hello Inet».

```
import socket
import struct

sock = socket.socket(socket.AF_INET, socket.SOCK_RAW,
                     socket.getprotobyname('udp'))

payload = b'hello Inet'
udp_pkt = struct.pack('!4h', 0, 2000, 8+len(payload), 0) + payload
sock.sendto(udp_pkt, ('127.0.0.1', 0))
```

LISTADO 20.5: Sintetizando un mensaje UDP con `AF_INET:SOCK_RAW`
`/raw/send-udp.py`

Puedes comprobar su funcionamiento ejecutando un servidor UDP en el puerto 2000 gracias a `ncat`. En un terminal ejecuta:

```
$ ncat -l -p 2000
```

Y en otro terminal, pero en la misma máquina, ejecuta:

```
$ ./send-udp.py
```

Si todo ha ido bien, en el primer terminal debería aparecer el texto «Hello Inet».

20.4.2.1. `IP_HDRINCL`

Como has podido comprobar en el ejemplo anterior, es posible enviar un segmento sin tener que construir la cabecera IP, únicamente la UDP. Sin embargo, puede haber ocasiones en las que el programador necesite «tocar» también la cabecera IP. Eso se consigue con la opción `IP_HDRINCL`.

La ventaja respecto al socket `AF_PACKET` es doble: no hay que molestarse con la cabecera de enlace, y además el SO puede rellenar por nosotros algunos de los campos más latosos si así queremos (poniendo ceros en ellos). Esos campos son:

- El `checksum`.
- La dirección IP origen.
- El `identificador` del mensaje.
- El campo `longitud total`.

Esta opción, como la gran mayoría, debe fijarse explícitamente después de crear el socket por medio del método `setsockopt()`, tal como se indica:

```
sock.setsockopt(socket.SOL_IP, socket.IP_HDRINCL, 1)
```

Esto resulta muy útil cuando quieres utilizar el socket para enviar distintos protocolos, y por tanto necesitas tener acceso al campo `proto`. Para poder hacer eso ha de crearse un socket de un *protocolo* especial identificado como `IPPROTO_RAW`:

```
sock = socket.socket(socket.AF_INET, socket.SOCK_RAW, socket.IPPROTO_RAW)
```

Aunque tiene un pequeño inconveniente: no se puede leer de este tipo de socket, tendrás que crear un socket adicional para poder leer los mensajes entrantes.

Capítulo 21

DNS

Al terminar este capítulo, entenderás:

- Qué es DNS y por qué son necesarios los nombres de dominio.
- Cómo se organiza el espacio de nombres: dominios, zonas y servidores autoritativos.
- Cómo se resuelven los nombres: el resolvedor y la resolución recursiva e iterativa.
- Qué papel juegan las cachés y el TTL.
- Cómo se define una zona y cuáles son los registros esenciales.
- Cómo se logra baja latencia con réplicas *anycast* y redes CDN.
- Qué son mDNS, DDNS y los sumideros DNS.

DNS es el servicio de nombres para redes TCP/IP, es decir, el sistema que asocia nombres únicos a direcciones IP, de forma similar a como una agenda asocia nombres a números de teléfono. Y esto es necesario —o muy conveniente— por varios motivos:

- Las direcciones IP son difíciles de recordar para las personas, especialmente las IPv6.
- Es sencillo asociar significados a los nombres, mientras que las direcciones no lo tienen. Eso facilita la identificación de los servicios que proporcionan determinados nodos (*p. ej.* `mail.example.com` probablemente aloja un servidor de correo).
- Tener nombres permite mantener referencias fijas para los nodos, permitiendo que las direcciones cambien. Eso permite al proveedor migrar el servidor (moverlo a otro nodo) sin que los usuarios, u otros servicios, lo noten[1].

[1] Esto se denomina *transparencia de localización*.

- Un mismo nombre puede asociarse a varias direcciones, lo que proporciona redundancia, y con ello mejora de la disponibilidad, tolerancia a fallos y balanceo de carga.

Formalmente se denomina Sistema de Nombres de Dominio (Domain Name System). Pero ¿qué es un 'dominio'? La definición oficial resulta un tanto... recursiva [61] y nada intuitiva. Por eso aquí lo vamos a enfocar de otro modo.

DNS define un espacio de nombres jerárquico organizado como un árbol invertido, es decir, con la raíz arriba. Cada nodo de este árbol tiene un nombre asociado. El dominio es el subárbol que se obtiene a partir de cualquiera de esos nodos. El *nombre de dominio* para un nodo se obtiene concatenando los nombres de los nodos que van desde ese hacia arriba hasta la raíz, y por convenio esos nombres se separan con puntos. Por ejemplo, en la Figura 21.1, el dominio `ietf.org` está formado, aparte del propio nodo `ietf`, por los nodos `www`, `mail` y `dev`.

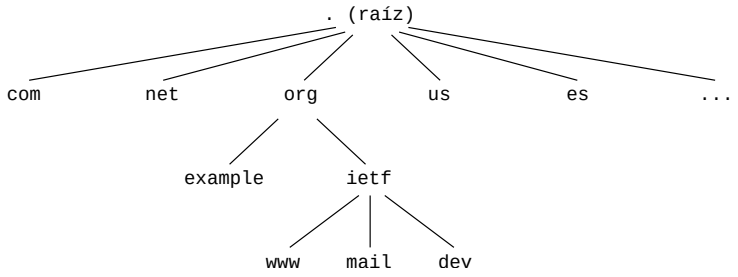


FIGURA 21.1: Estructura jerárquica de los nombres de dominio

Cualquier dominio puede tener subdominios si se toman subárboles a partir del nodo base que lo define. Es fácil entender porqué todo lo relacionado con DNS suena recursivo: el árbol es una estructura recursiva por definición.

Como puede ser confuso, aclaremos que DNS da nombre a varias cosas. Es el nombre del propio servicio distribuido que traduce nombres a direcciones, el del protocolo que define los mensajes que se intercambian los servidores y clientes, y también se utiliza para referirse a los servidores que implementan este servicio: Cuando se habla de «un DNS» se entiende que hablamos de un servidor concreto, que puede ser un proceso o un nodo, en función del contexto.

Conceptualmente, el servicio DNS funciona como una base de datos distribuida y redundante que almacena la información en forma de **registros** nombre–tipo–valor. Esto convierte el protocolo asociado en un mecanismo

de consulta. A estas consultas las llamamos en español «resolución de nombres».

La resolución de nombres es una tarea muy frecuente, las aplicaciones lo hacen constantemente como parte de su funcionamiento. Lo hacen cada vez que demandan cualquier servicio de red. Pero también disponemos de herramientas para hacer esas consultas manualmente, y resultan muy útiles para diagnosticar problemas, probar entender la configuración o estudiar su funcionamiento. Una de estas herramientas es `dig` (`bind9-dnsutils`). Un ejemplo básico de uso:

```
$ dig www.ietf.org

; <<>> DiG 9.20.15-1~deb13u1-Debian <<>> www.ietf.org
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 12494
;; flags: qr rd ra ad; QUERY: 1, ANSWER: 2, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
;; QUESTION SECTION:
;www.ietf.org.            IN    A

;; ANSWER SECTION:
www.ietf.org.        113    IN    A    104.16.45.99
www.ietf.org.        113    IN    A    104.16.44.99

;; Query time: 7 msec
;; SERVER: 1.1.1.1#53(1.1.1.1) (UDP)
;; WHEN: Tue Jan 06 14:33:58 UTC 2026
;; MSG SIZE  rcvd: 69
```

Sin embargo, `dig` produce mucha información que puede no ser relevante. Por suerte, acepta varias opciones para limitar su salida. Ademas puedes escribir algunas de esas opciones en el archivo `~/.digrc` para que el programa las aplique siempre y así no tener que escribirlas cada vez. Por ejemplo, para este capítulo usamos un archivo `~/.digrc` que contiene lo siguiente:

```
+nocmd
+noedns
+noquestion
+nostats
```

Con eso, si repites el comando anterior, la salida será algo así:

```
$ dig www.ietf.org
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 6684
;; flags: qr rd ra ad; QUERY: 1, ANSWER: 2, AUTHORITY: 0, ADDITIONAL: 0

;; ANSWER SECTION:
```

```
www.ietf.org.       113    IN    A    104.16.45.99
www.ietf.org.       113    IN    A    104.16.44.99
```

Ahora que la salida es manejable, se puede analizar más cómodamente su contenido. Este comando en concreto pregunta al servidor DNS configurado en tu sistema acerca del dominio `www.ietf.org`. La respuesta (`ANSWER SECTION`) informa de dos registros tipo A. Estos indican las direcciones IPv4 asociadas: `104.16.44.99` y `104.16.45.99`. Quizá te sorprenda que sean dos y no una, pero eso es perfectamente posible y, de hecho, bastante común. Eso mejora la disponibilidad del servicio, porque el cliente normalmente usará la primera, pero si falla, empleará la segunda.

Además, el servidor puede reordenar esos registros cada vez que se le piden, mejorando el balanceo de la carga entre ambos nodos. Repite el comando varias veces y fíjate en el orden en que aparecen las direcciones cambia en cada ocasión. Más adelante veremos cómo se saca provecho de esta característica.

Ya que estamos, esto es lo que significa la información que aparece en la cabecera:

- `opcode: QUERY`: Se trata de una consulta estándar.
- `status: NOERROR`: La consulta se ha ejecutado sin errores.
- `flags: qr rd ra ad`:
  - `qr`: El mensaje es una respuesta *(query response)*.
  - `rd`: El cliente ha solicitado resolución recursiva *(recursion desired)*.
  - `ra`: El servidor soporta resolución recursiva *(recursion available)*.
  - `ad`: Los datos están autenticados *(authenticated data)*.

El «servidor DNS configurado en tu sistema» es la dirección de un servidor —en realidad suelen ser dos— que el SO utiliza para realizar las consultas DNS. Normalmente se obtienen automáticamente con DHCP, así que no es algo que nos suela preocupar. En § 21.3 veremos cómo se pueden fijar de forma manual, y por qué puede ser interesante.

21.1. Dominios y zonas

El nombre `www.ietf.org` es efectivamente un dominio, pero también es un subdominio de `ietf.org`, que a su vez es un subdominio de `net`. El nombre `net` es lo que se llama un dominio de nivel superior o TLD (Top Level Domain). Los TLD cuelgan directamente de la raíz del árbol, que se denota con un punto (`.`) al final del nombre, como en `www.ietf.org.` —aunque ese punto final normalmente se omite en los usos menos formales. Un nombre de dominio de este tipo (que termina en el raíz) se denomina FQDN (Fully

Qualified Domain Name), que se suele traducir como *nombre de dominio completamente calificado*.

Esta nomenclatura es meramente conceptual y no tiene relación con el modo en que se almacena la información en los servidores. Para cada dominio existe al menos un servidor específico que define la correspondencia oficial entre nombres y direcciones IP para ese domino —si bien suelen ser dos por redundancia. De estos servidores se dice que son los **autoritativos** del dominio. Como ejemplo, estos son los servidores autoritativos para el dominio `ietf.org`, obtenidos pidiendo los registros NS:

```
$ dig ietf.org NS
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 45975
;; flags: qr rd ra ad; QUERY: 1, ANSWER: 2, AUTHORITY: 0, ADDITIONAL: 0

;; ANSWER SECTION:
ietf.org.        86400    IN    NS    ken.ns.cloudflare.com.
ietf.org.        86400    IN    NS    jill.ns.cloudflare.com.
```

Miles de servidores (no autoritativos) repartidos por todo el mundo pueden resolver también el nombre `ietf.org`, pero eso es simplemente porque tienen una copia de la información que proporcionan estos servidores autoritativos.

El servidor autoritativo define la llamada *zona* del dominio, que incluye los registros asociados, y responde a consultas específicas sobre esta información. La zona puede incluir la definición de los subdominios o bien, puede delegarlos a servidores autoritativos específicos. Sin embargo, los servidores autoritativos no proporcionan información sobre dominios ajenos, únicamente sobre el que administran.

Estos son los tipos de registro esenciales que se pueden definir en una zona DNS.

Tipo	**Significado**	**Descripción**
SOA	Start of Authority	Indica el servidor autoritativo primario *(master)*, correo del administrador, número de serie, etc.
NS	Name Server	Indica los servidores autoritativos de la zona o delega sus subdominios.
A	Address	Asocia un nombre con una dirección IPv4.
CNAME	Canonical Name	Define un alias para otro nombre.

CUADRO 21.1: Registros básicos para definir zonas DNS

Aparte de los NS, ya presentados, el registro más importante es el SOA, que define parámetros esenciales de la zona. Este es su aspecto real. La Tabla 21.2 detalla el significado de los campos incluidos en el registro.

```
$ dig +multiline ietf.org SOA
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 49304
;; flags: qr rd ra ad; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 0

;; ANSWER SECTION:
ietf.org.       1800 IN   SOA jill.ns.cloudflare.com. dns.cloudflare.com. (
                2392675761 ; serial
                10000      ; refresh (2 hours 46 minutes 40 seconds)
                2400       ; retry (40 minutes)
                604800     ; expire (1 week)
                1800       ; negative TTL (30 minutes)
                )
```

Campo	Significado
`Primary NS`	Servidor autoritativo primario declarado para la zona: `jill.ns.cloudflare.com`
`Responsible Mail`	Correo del administrador, aunque no siempre tiene formato de correo electrónico.
`Serial`	Número de serie de la zona.
`Refresh (s)`	Cada cuanto un secundario comprueba si hay cambios.
`Retry (s)`	Tiempo de reintento si falla el refresco.
`Expire (s)`	Tras este tiempo sin contacto, el secundario deja de servir la zona.
`Negative TTL (s)`	Tiempo de caché para respuestas negativas.

CUADRO 21.2: Valores reales e interpretación del registro SOA de `ietf.org`

21.2. La zona raíz

Como para cualquier otro dominio, el dominio raíz también tiene su propia zona. La zona raíz gestiona los registros NS para todos los TLD. Está coordinada por la ICANN y la IANA, y consta de 13 servidores replicados a centenares por todo el mundo. Están nombrados desde `a.root-servers.net` a `m.root-servers.net`.

Puedes consultar la zona raíz y ver los nombres de esos servidores, y también su SOA:

```
$ dig . NS
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 23022
;; flags: qr rd ra ad; QUERY: 1, ANSWER: 13, AUTHORITY: 0, ADDITIONAL: 0
```

```
;; ANSWER SECTION:
.            510508    IN    NS    a.root-servers.net.
.            510508    IN    NS    b.root-servers.net.
.            510508    IN    NS    c.root-servers.net.
.            510508    IN    NS    d.root-servers.net.
.            510508    IN    NS    e.root-servers.net.
.            510508    IN    NS    f.root-servers.net.
.            510508    IN    NS    g.root-servers.net.
.            510508    IN    NS    h.root-servers.net.
.            510508    IN    NS    i.root-servers.net.
.            510508    IN    NS    j.root-servers.net.
.            510508    IN    NS    k.root-servers.net.
.            510508    IN    NS    l.root-servers.net.
.            510508    IN    NS    m.root-servers.net.

$ dig . SOA
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 45090
;; flags: qr rd ra ad; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 0

;; ANSWER SECTION:
.            86351    IN    SOA    a.root-servers.net. nstld.verisign-grs.com.
2026010600 1800 900 604800 86400
```

21.3. Resolución de nombres

En los nodos de los usuarios, al componente que realiza las resoluciones de nombres lo llamaremos *resolvedor* como calco léxico del original en inglés *local resolver* o *stub resolver*. Suele ser de una librería o servicio del SO que recibe y responde peticiones de parte de las aplicaciones a través de la llamada al sistema `getaddrinfo()` o de las librerías que cada lenguaje de programación ofrezca por encima. Por ejemplo, en Python:

```
import socket
for addr in socket.getaddrinfo("www.example.com", None):
    print(addr[4][0])
104.18.5.106
104.18.4.106
```

El módulo `socket` también utiliza automáticamente el resolvedor del SO cuando en lugar de una IP, utilizas un nombre al establecer una conexión TCP o flujo UDP, como en:

```
import socket
s = socket.socket()
s.connect(("www.example.com", 80))
```

El resolvedor determina las fuentes de información que debe usar consultando la línea `hosts` en el archivo `/etc/nsswitch.conf`, que suele tener este aspecto:

```
hosts: files dns
```

Ese «files» indica que primero debe consultar el archivo `/etc/hosts`, que contendrá algo similar a esto:

```
127.0.0.1    localhost
127.0.0.1    maine
::1     ip6-localhost ip6-loopback
ff02::1 ip6-allnodes
ff02::2 ip6-allrouters
```

Si no encuentra el nombre aquí, la palabra «dns» de `nsswitch.conf` indica que debe consultar los servidores DNS configurados en el archivo `resolv.conf`, que suele tener este aspecto:

```
nameserver 1.1.1.1
nameserver 8.8.8.8
```

Estas direcciones IP corresponden a los servidores DNS llamados *recursive resolvers*, y que aquí llamaremos simplemente «servidores DNS» para distinguirlos de los autoritativos. Estos servidores son los encargados de realizar las consultas necesarias para resolver el nombre solicitado por el cliente y suelen proporcionarlos los ISP. Por eso, el contenido de este fichero normalmente se obtiene por DHCP u otros servicios locales como `systemd-resolved` o `resolvconf`.

También existe la posibilidad de editarlo manualmente si ninguno de esos servicios está en uso, o si se quiere forzar un servidor específico ajeno al ISP, como es el caso del ejemplo. La Figura 21.2 muestra cómo realizar esa configuración en el entorno de escritorio GNOME.

La razón por la que se establecen dos de estos servidores es la habitual: redundancia. Si el primero no responde u ocasionalmente lo hace muy lento, el SO puede puede utilizar el segundo.

Normalmente el cliente pide a estos servidores una *resolución recursiva*, que implica los siguientes pasos del servidor DNS:

1. Si el nombre solicitado está en su caché, responde inmediatamente al cliente.
2. Si no está en la caché, realiza una consulta a un servidor raíz, que le indica el servidor autoritativo para el TLD solicitado.
3. Pregunta al servidor autoritativo del TLD, que a su vez delega la consulta al servidor autoritativo del dominio solicitado.
4. Realiza una consulta al servidor autoritativo específico, que puede disponer de la dirección o bien responder con otra delegación: el registro NS de un subdominio.

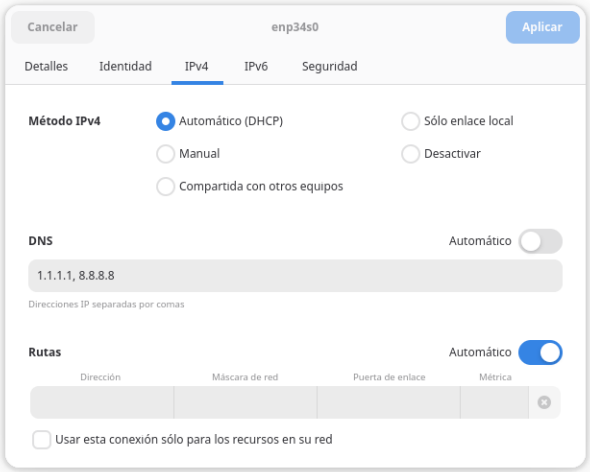


FIGURA 21.2: Configuración DNS en GNOME

5. Guarda la respuesta en caché y la devuelve al cliente.

Es *recursivo* porque el servidor (y no el cliente) se encarga de seguir las delegaciones, es decir, realiza las consultas necesarias hasta llegar hasta la respuesta final. Por defecto, el cliente solicita resoluciones recursivas al servidor (RD=1 en la cabecera del mensaje DNS), pero opcionalmente es posible hacer solicitudes iterativas (RD=0). En ese caso, el servidor responde con la información de que dispone, pero no hace peticiones adicionales. Lo importante es entender que el funcionamiento típico del servidor DNS es ofrecer resolución recursiva realizando peticiones iterativas.

Para entender mejor el trabajo que realiza el servidor, podemos ver un proceso similar forzando una resolución iterativa desde el cliente:

```
$ dig +trace www.ietf.org
.            513823    IN    NS    a.root-servers.net.
.            513823    IN    NS    b.root-servers.net.
.            513823    IN    NS    c.root-servers.net.
.            513823    IN    NS    d.root-servers.net.
.            513823    IN    NS    e.root-servers.net.
.            513823    IN    NS    f.root-servers.net.
.            513823    IN    NS    g.root-servers.net.
.            513823    IN    NS    h.root-servers.net.
.            513823    IN    NS    i.root-servers.net.
.            513823    IN    NS    j.root-servers.net.
.            513823    IN    NS    k.root-servers.net.
.            513823    IN    NS    l.root-servers.net.
.            513823    IN    NS    m.root-servers.net.
;; Received 525 bytes from 1.1.1.1#53(1.1.1.1) in 7 ms
```

```
16
17  org.            172800    IN    NS    a0.org.afilias-nst.info.
18  org.            172800    IN    NS    a2.org.afilias-nst.info.
19  org.            172800    IN    NS    b0.org.afilias-nst.org.
20  org.            172800    IN    NS    b2.org.afilias-nst.org.
21  org.            172800    IN    NS    c0.org.afilias-nst.info.
22  org.            172800    IN    NS    d0.org.afilias-nst.org.
23  ;; Received 812 bytes from 2001:7fe::53#53(i.root-servers.net) in 191 ms
24
25  ietf.org.        3600    IN    NS    ken.ns.cloudflare.com.
26  ietf.org.        3600    IN    NS    jill.ns.cloudflare.com.
27  ;; Received 306 bytes from 199.19.54.1#53(b0.org.afilias-nst.org) in 19 ms
28
29  www.ietf.org.        300    IN    A    104.16.45.99
30  www.ietf.org.        300    IN    A    104.16.44.99
31  ;; Received 177 bytes from 173.245.58.122#53(jill.ns.cloudflare.com) in 3 ms
```

Este comando se ha ejecutado teniendo configurado `1.1.1.1` en el archivo `/etc/resolv.conf`. Estas son las diferentes partes que la componen, las respuestas a las consultas sucesivas que hace el cliente a partir de la información que va recibiendo de cada servidor:

- Consulta al servidor raíz (**línea 2**). Responde con los 13 servidores autoritativos para el dominio raíz.
- Consulta a `i.root-servers.net` acerca del TLD `org`, que responde con 6 servidores autoritativos (**línea 17**).
- Consulta a `b0.org.afilias-nst.org` sobre `ietf.org` (**línea 25**). Responde con dos servidores autoritativos.
- Consulta a `jill.ns.cloudflare.com` para resolver `www.ietf.org`, que finalmente responde con las direcciones IP solicitadas (**línea 29**).

Después de cada sección aparece el servidor concreto al que se ha consultado y cuanto tiempo tardó en responder.

21.4. Cachés e información obsoleta

Las cachés de los servidores DNS juegan un papel clave en el funcionamiento del sistema, proporcionando un equilibrio entre disponibilidad de información actualizada y rendimiento. Cuando un servidor recibe una petición desde un cliente, primero comprueba su caché. Si tiene la respuesta y aún es válida (no ha expirado), devuelve inmediatamente ese valor y no realiza consultas adicionales. Sin la caché, los servidores continuamente tendrían que hacer consultas a los mismos servidores autoritativos, lo que conllevaría una sobrecarga inadminible para todas las partes y degradaría de forma muy notable la operación del sistema en su conjunto.

La invalidación de la caché la determina el campo TTL (expresado en segundos) asociado a cada registro. El servidor autoritativo define el valor

del TTL por lo que no cambia, a menos que el administrador de la zona lo modifique manualmente. Puedes averiguar el valor del TTL tal como está definido dirigiendo la consulta directamente al servidor autoritativo (usando la `@`) obviando el servidor configurado en tu sistema.

```
$ dig @jill.ns.cloudflare.com ietf.org
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 11996
;; flags: qr aa rd; QUERY: 1, ANSWER: 2, AUTHORITY: 0, ADDITIONAL: 0
;; WARNING: recursion requested but not available

;; ANSWER SECTION:
ietf.org.        300    IN    A    104.16.45.99
ietf.org.        300    IN    A    104.16.44.99
```

Hay dos pistas en esta respuesta que confirman que hemos preguntado al servidor autoritativo:

- En la cabecera aparece el flag `aa` *(authoritative answer)*.
- La advertencia indicando *recursion requested but not available* es lo esperable para servidores autoritativos.

El TTL es el valor que aparece en la segunda columna junto a cada registro (300 segundos en este caso). Si repites esta misma consulta verás que siempre aparece el mismo valor, mientras que si envías la consulta al servidor proporcionado por el ISP, el valor irá disminuyendo hasta llegar a cero, momento en el que hará una nueva consulta al autoritativo y volverá a empezar desde 300. Este sistema tiene una consecuencia importante: la información que ofrecen los servidores siempre está potencialmente obsoleta[2]. ¿Cuánto? Depende completamente del valor del TTL definido en cada zona, si bien es cierto que muchos ISP imponen un valor mínimo (5 minutos) y un valor máximo (1 día).

Y así es cómo se distribuye la información a través de la red DNS. No hay ningún mecanismo que detecte cambios y sea proactivo en la distribución de actualizaciones. Cuando por ejemplo el administrador de una zona modifica la dirección IP asociada a un nombre, los servidores seguirán ofreciendo información incorrecta a sus clientes hasta que sus cachés expiren.

Es común escuchar que las asociaciones DNS «tardan en propagarse», pero en realidad solo depende de la configuración del TTL del servidor autoritativo y de los servidores de terceros que estén involucrados. Por ese motivo, cuando se realiza una migración que implica un cambio de dirección IP es buena práctica reducir el TTL horas o días antes, dando tiempo para que

[2]Se dice que ofrece *consistencia eventual*

expiren las cachés en toda la red. Una vez terminada la migración, puede volver a aumentarse el TTL para evitar consultas demasiado frecuentes.

La información negativa, es decir, la situación en la que un servidor autoritativo dice que no conoce un nombre, también se almacena en caché. En ese caso se utiliza el valor llamado *Negative TTL* que aparece en el registro SOA (ver Cuadro 21.2). Así se evitan consultas reiteradas para nombres desconocidos a los mismos servidores.

21.5. Configuración de zona

La configuración de una zona en un servidor autoritativo se especifica en un archivo llamado *fichero de zona*[3] que contiene los registros que la definen.

El siguiente listado muestra una definición de zona hipotética para el dominio `example.net`[4]. Este fichero de zona se aplicaría en los servidores autoritativos `ns1.example.net` y `ns2.example.net` para responder a las consultas sobre ese dominio.

```
$TTL 3600
@ IN SOA ns1.example.net. hostmaster.example.net. (
    2026012901 ; serial
    3600       ; refresh
    900        ; retry
    604800     ; expire
    300        ; negative TTL
)

@   IN NS    ns1.example.net.
@   IN NS    ns2.example.net.

@   IN A     192.0.2.10
@   IN AAAA  2001:db8::10

www IN CNAME @
@   IN MX 10 mail.example.net.
_sip._tcp IN SRV 10 60 5060 sip.example.net.
```

Fíjate que ambos servidores autoritativos sirven la misma información, ambos definen el registro SOA que apunta al servidor primario (`ns1`). Ambos incluyen los dos registros NS.

Esta zona define también:

- El valor por defecto de TTL para los registros que no lo especifiquen explícitamente (3600 segundos).

[3] O alguna estructura de datos equivalente.

[4] El dominio `example.net` existe realmente con otra configuración

- Registro A para el dominio base[5] `example.net` (representado por '`@`').
- Registro AAAA para el dominio base, que proporciona una dirección IPv6 asociada.
- Registro CNAME que define un alias `www.example.net` para el dominio base, es decir, devuelve la misma dirección IP.
- El registro MX que define el servidor de correo para el dominio.
- El registro SRV que define el servicio SIP sobre TCP para el dominio.

Para que esta zona sea legítima y se pueda utilizar, la zona del `net.` debe definir obligatoriamente los mismos valores para los registros NS. Los registros correspondientes en el fichero de zona de `net.` serían algo así:

```
example.net.      3600  IN  NS    ns1.example.net.
example.net.      3600  IN  NS    ns2.example.net.
ns1.example.net.        IN  A     192.0.2.53
ns2.example.net.        IN  A     192.0.2.54
```

Los registro NS representan aquí la delegación del dominio `example.net` a los servidores autoritativos indicados (`ns1` y `ns2`). Los registros A adicionales se denominan *registros glue* y son necesarios para evitar un posible bucle de resolución cuando el nombre del servidor autoritativo (como en este caso) está dentro del dominio delegado.

21.6. Resolución inversa

ToDo!

21.7. Replicación

Algunas compañías ofrecen servidores DNS públicos que proporcionan algunas ventajas sobre los que facilitan los ISP. Esta tabla recoge los más conocidos, sus direcciones IP, compañía y propósito principal:

[5] También se denomina en inglés *apex domain* o *root domain*, pero evitaremos esta segunda opción porque se puede confundir con el dominio raíz: '`.`'

IP	Compañía	Propósito
8.8.8.8 / 8.8.4.4	Google	Rapidez y estabilidad.
1.1.1.1 / 1.0.0.1	Cloudflare	Privacidad y baja latencia.
9.9.9.9	Quad9 Foundation	Bloqueo contra dominios ma phishing.
208.67.222.123 / 208.67.220.123	Cisco (OpenDNS FamilyShield)	Control parental.
208.67.222.222 / 208.67.220.220	Cisco (OpenDNS Home)	Filtrado configurable y estad
185.228.168.9 / 185.228.169.9	CleanBrowsing	Bloqueo de malware.
185.228.168.168 / 185.228.169.168	CleanBrowsing	Control parental.
94.140.14.14 / 94.140.15.15	AdGuard	Bloqueo de publicidad.
84.200.69.80 / 84.200.70.40	DNS.Watch	Neutralidad y privacidad.
156.154.70.1 / 156.154.71.1	Neustar	Protección contra dominios n
64.6.64.6 / 64.6.65.6	Verisign	Estabilidad y privacidad.

Cosas como la protección contra malware, phishing o control parental la consiguen simplemente manteniendo una lista negra de dominios y respondiendo con un registro A falso o un dominio que lleva a una página que informa del bloqueo. Pero ¿cómo puede un servidor DNS ofrecer baja latencia a usuarios de todo el planeta? Por lógica, el servidor ofrecerá menor latencia a los usuarios que estén próximos geográficamente que a aquellos que estén lejos. La respuesta es que en realidad el servidor DNS no está en un nodo, sino que hay decenas o centenares de servidores (réplicas) con la misma dirección IP repartidos por el mundo en instalaciones PoP (Point of Presence) bajo el control administrativo de la compañía. De este modo, la latencia es baja para cualquier usuario, en cualquier lugar.

Pero eso nos lleva a otra cuestión aún más básica: ¿cómo puede haber centenares de nodos con la misma dirección IP? Esto es posible con *anycast*, una técnica de encaminamiento que puede llevar paquetes IP al nodo más próximo (mejor métrica) con la dirección destino solicitada. Realmente IPv4 no soporta *anycast* como sí lo hace IPv6, pero en la práctica el encaminamiento BGP puede lograr el mismo efecto.

21.7.1. CDN

Una CDN es un servicio de caché distribuida para recursos web estáticos: archivos JavaScript, CSS, imágenes, etc. El objetivo, como de costumbre, es aumentar la disponibilidad y reducir la latencia para el acceso desde los usuarios finales a estos recursos.

Para lograr esto, cuando un usuario pide la resolución del dominio de una CDN (*p. ej.* ajax.googleapis.com), el servidor autoritativo (gestionado por la CDN) responde con una dirección *anycast* o bien con la dirección IP de

un PoP próximo al usuario, en cuyo caso además, puede tener en cuenta la carga a la hora de elegirlo, que es algo que no puede hacer *anycast* únicamente mediante encaminamiento BGP.

En este ejemplo se puede apreciar cómo, al caducar la caché, el servidor DNS configurado en el sistema empieza a devolver una dirección diferente.

```
~$ dig ajax.googleapis.com
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 17254
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; ANSWER SECTION:
ajax.googleapis.com.    1    IN    A    142.250.200.106

~$ dig ajax.googleapis.com
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 13269
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; ANSWER SECTION:
ajax.googleapis.com.    290    IN    A    216.58.209.74
```

El sistema CDN también utiliza un mecanismo de caché con expiración, aunque en este caso se aprovechan los campos específicos de las cabeceras HTTP para determinar la validez de los recursos en caché. Cuando el objeto solicitado no se encuentra o ha caducado, el nodo CDN *edge* lo obtiene del servidor origen o de un nodo intermedio de la red CDN. Los nodos intermedios se utilizan en redes CDN jerárquicas, algo habitual cuando esta alcanza un tamaño importante.

21.8. Transporte

El protocolo DNS original se encapsula sobre UDP y el servidor escucha en el puerto 53. Este sigue siendo el transporte más usado también hoy día, especialmente para consultas desde el cliente. El motivo es que UDP es ligero y de baja latencia, algo muy importante para un servicio como este que se utiliza a menudo en sesiones muy cortas.

Sin embargo, existen otros transportes estandarizados:

- TCP en el puerto 53, utilizado para transferencia de zona entre servidores autoritativos y para consultas cuyas respuestas que no caben en un único datagrama UDP.
- DNS sobre TLS (DoT) en el puerto 853, que proporciona privacidad y cifrando en consultas y respuestas.
- DNS sobre HTTPS (DoH) en el puerto 443, que también proporciona privacidad y cifrado, pero además permite *disimular* las consultas DNS dentro de tráfico HTTPS incluso sobre HTTP/2 o HTTP/3.

- DNS sobre QUIC (DoQ) en el puerto 853. Utiliza QUIC que es fiable, cifrado y con control de congestión sobre UDP.

21.9. Multicast DNS

Multicast DNS, abreviado como mDNS, permite la resolución de nombres en redes LAN sin necesidad de servidores. Utiliza el mismo formato de mensajes DNS estándar, pero los clientes envían las solicitudes a la dirección multicast `224.0.0.251`, puerto 5353.

Los nodos se autoasignan nombres basados en su hostname en el dominio especial `.local`. Este mecanismo es la base de *zero-configuration networking (zeroconf)* que se utiliza en redes LAN para la configuración y, sobre todo, descubrimiento automático de servicios y dispositivos gracias a DNS-SD.

Con DNS-SD y mDNS es posible anunciar y descubrir impresoras, servidores de ficheros, televisores, altavoces, dispositivos IoT, etc. Se utilizan nombres de servicio con el formato `_servicio._protocolo.local`, por ejemplo `_ssh._tcp.local` para un servidor SSH o `_ipp._tcp.local` para una impresora. La información se proporciona con los registros estándar PTR, SRV y TXT.

En GNU/Linux ambos protocolos se implementan con el sistema `avahi` (`avahi-daemon`). Por ejemplo, el siguiente comando lista los servicios que se están anunciando en la LAN:

```
$ avahi-browse -a
+ wlp1s0 IPv4 shellyplus1-c83d56432                Web Site              local
+ wlp1s0 IPv4 TV                                   AirPlay Remote Video  local
+ wlp1s0 IPv4 Google-Cast-Group-c2623452345234523   _googlecast._tcp      local
```

Y se puede obtener información detallada:

```
= wlp1s0 IPv4 TV                                   AirPlay Remote Video  local
   hostname = [tv.local]
   address = [192.168.0.173]
   port = [51620]
   txt = ["serialNumber=0CAK3SDMA87463F" "manufacturer=Samsung" "model=QRQ60"
   "flags=0x244" "fv=p20.0.1" "rsf=0x3" "features=0x7F8AD0,0x38BCB46"
   "deviceid=D5:7D:D0:DF:D5:46" "acl=0"]
```

21.10. Dynamic DNS

Un servicio de DDNS lo proporciona un servidor DNS autoritativo que ofrece al administrador la posibilidad de modificar dinámicamente los registros de la zona (específicamente los A) normalmente a través de un API. Un servicio que se ejecuta en el nodo detecta automáticamente cuando cambia su

dirección IP (con los mecanismos de § 11.6.1) y lo notifica inmediatamente al servidor DDNS mediante ese API para que actualice la resolución.

Esta funcionalidad obviamente es útil para nodos con direcciones IP dinámicas, como típicamente proporcionan los ISP a sus clientes domésticos o pequeñas oficinas. Esto permite disponer de un nombre de dominio público y fijo que apunta a un nodo que no dispone de una dirección IP fija.

Existen varios servicios comerciales que ofrecen este servicio, tales como `no-ip.com`, `dyn.com`, `duckdns.org` o `freedns.afraid.org`. Aunque existe un protocolo estándar para este propósito [62], la mayoría usan API propietarias basadas en REST HTTP.

21.11. Sumideros DNS *(sinkhole)*

Un sumidero DNS *(sinkhole)* es un servidor muy sencillo que se suele instalar en algún nodo de la LAN. Al igual que algunos de los servidores de la sección anterior, su objetivo es bloquear el acceso a dominios maliciosos o conocidos por participar en campañas de *phishing*, *spam*, publicidad, etc. La diferencia principal es que al ejecutarse en nuestra LAN tenemos plena capacidad para administrar con precisión el tipo de bloqueo que realizan. Algunos de los más conocidos son `Pi-hole` y `AdGuard Home`, que además están disponibles como contenedores `docker` con lo que su instalación resulta muy sencilla.

Y ¿qué más?

Este capítulo ha presentado DNS como lo que es: una base de datos global distribuida y jerárquica que asocia registros a nombres de dominio. Has visto cómo se organiza en zonas administradas por servidores autoritativos, cómo los servidores recursivos resuelven los nombres por delegación desde la raíz, y cómo las cachés y el TTL controlan la propagación —y la potencial obsolescencia— de la información. También se han repasado sus transportes y algunas variantes de interés práctico como MDNS, DDNS o los sumideros DNS.

Sin embargo, han quedado fuera aspectos más avanzados relacionados con la seguridad. La especificación original no ofrece ninguna garantía de autenticidad, lo que lo hace vulnerable a la suplantación de respuestas y al envenenamiento de cachés *(cache poisoning)*. Para remediarlo existe DNSSEC, una extensión que firma criptográficamente los registros y permite asegurar que la respuesta procede realmente del administrador de la zona.

Además, DNS se ha convertido en una pieza clave de otros servicios: los registros TXT se utilizan para verificar la propiedad de dominios y para

combatir la suplantación en el correo electrónico mediante mecanismos como SPF, DKIM y DMARC.

Capítulo 22

SSH

Al terminar este capítulo, entenderás:

- Qué es y qué ofrece SSH.
- Cómo establecer una shell remota.
- Cómo utilizar el fichero `~/.ssh/config`.
- Qué ventajas ofrece la autenticación con clave pública/privada.
- Cómo funciona la autenticación con certificados.
- Cómo copiar archivos de forma segura entre nodos con SCP.

SSH (Secure SHell) es un protocolo de aplicación que permite establecer conexiones con máquinas remotas. Podríamos decir que es una versión mejorada del veterano y muy inseguro `telnet`. Sin embargo, es mucho más que eso: SSH proporciona un canal seguro (cifrado) para ejecutar comandos, transferir archivos[1], redireccionar conexiones, crear túneles que pueden transportar datos de conexiones arbitrarias o simplemente como sistema de autenticación.

22.1. Shell segura

Como ya indica su nombre, el uso más común de la aplicación SSH es el de «shell remota», es decir, un programa que permite abrir una sesión en un computador remoto y ejecutar programas. En este documento vamos a usar la aplicación `OpenSSH`, que en el caso de sistemas GNU/Linux suele estar disponible como dos paquetes: `openssh-server` y `openssh-client`. Por supuesto, existen otras muchas implementaciones para casi cualquier SO.

Se trata, por supuesto, de una aplicación cliente-servidor: El servidor SSH, que debe estar instalado en cada computador que queramos que sea accesible, siempre está a la espera de conexiones, mientras que el cliente SSH es el que inicia la conexión.

[1] con el programa `scp`

Para probar SSH puedes utilizar un contenedor docker cuya configuracióne está en el directorio /ssh-docker. El siguiente comando, que se debe ejecutar dentro de ese directorio, crea la imagen y arranca el contenedor que ejecuta el servidor SSH:

```
alice@maine:~/code/ssh-docker$ docker compose up -d
[...]
```

La imagen contiene un sistema GNU/Linux con un servidor SSH listo para conectar. Abre un terminal en tu sistema y ejecuta el programa ssh, que es el cliente del protocolo.

```
alice@maine:~$ ssh user@172.20.0.2
user@172.20.0.2's password:
user@sidney:~$
```

El argumento del programa incluye el usuario (user) y la dirección IP o el nombre del servidor. Si el puerto es distinto del 22 (valor por defecto) también se puede indicar aquí con otro parámetro.

El programa ssh trata de establecer una conexión segura con el servidor SSH que hay en 172.20.0.2 (el contenedor docker). Este proceso pide la contraseña de sistema asociada a ese usuario, que no se muestra al introducirla. Para este ejemplo la contraseña es **secret**.

22.2. Configuración

Para simplificar el comando de conexión se puede escribir un archivo de configuración llamado ~/.ssh/config en el **computador cliente**. Para el caso anterior, el archivo podría ser algo como:

```
Host sidney
    hostname 172.20.0.2
    User user
```

Es importante señalar que este archivo debe tener permisos de lectura/escritura solo para el usuario (-rw-------) o será ignorado. Una vez tengas esta configuración puedes conectar simplemente con:

```
alice@maine:~$ ssh sidney
user@172.20.0.2's password:
```

Este modo de autenticación con usuario/clave se llama de «clave privada» porque se asume que solamente el usuario conoce dicha clave.

22.3. Acceso con clave pública

Para evitar tener que escribir constantemente la clave cada vez que conectas a una misma máquina, SSH ofrece un sistema de autenticación de «clave pública». Este sistema utiliza un par de claves privada/pública. La clave pública se coloca en el sistema al que quieres acceder y, de hecho, no hay ningún problema en que cualquiera la pueda leer o copiar (por eso es pública). Sin embargo, la clave privada debe quedar custodiada por el usuario y nunca debe ser accesible para nadie más.

Para generar el par de claves ejecuta el siguiente comando aceptando todos los valores por defecto (pulsa `ENTER`):

```
alice@maine:~$ ssh-keygen
Generating public/private rsa key pair.
Enter file in which to save the key (/home/alice/.ssh/id_rsa):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/alice/.ssh/id_rsa
Your public key has been saved in /home/alice/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:KXUrUqPxk/EPiXEBfgqK3HlSqvhphSeh6O+qm9tz1DM alice@maine
```

Esto genera dos archivos llamados `id_rsa` (clave privada) y `id_rsa.pub` (clave pública) en el directorio `/home/alice/.ssh/`.

Ahora se debe enviar la clave pública al servidor. Esto se puede hacer fácilmente con:

```
alice@maine:~$ ssh-copy-id sidney
user@172.20.0.2's password:

Number of key(s) added: 1

Now try logging into the machine, with:   "ssh 'sidney'"
and check to make sure that only the key(s) you wanted were added.
```

Si no se le indica otra cosa, este programa almacena la clave pública del usuario en `~/.ssh/id_rsa.pub`— en el archivo `~/.ssh/authorized_keys` del servidor (`sidney` en este ejemplo).

Y tal como indica, ahora deberíamos poder entrar en la máquina `sidney` simplemente con:

```
alice@maine:~$ ssh sidney
Last login: Mon Mar 14 21:07:15 2022 from 172.20.0.1
user@sidney:~$
```

Esto no solo es interesante porque ahorra al usuario tener que escribir su contraseña constantemente. Además permite que las aplicaciones puedan automatizar tareas sin la intervención de un usuario. Los sistemas de control de versiones (como `git`) o sistemas de Integración o Despliegue Continuo (CI/CD) utilizan habitualmente autenticación SSH con clave pública.

También es posible ejecutar un único comando en el servidor y ver la salida directamente en el computador cliente. Simplemente hay que escribir el comando a continuación:

```
alice@maine:~$ ssh sidney ls /home/
user
```

22.3.1. Agente SSH

Cuando se generan las claves con `ssh-keygen`, existe la opción de proteger la clave privada con una contraseña *(passphrase)*, que en el ejemplo anterior quedó vacía. Esta contraseña evita que otra persona pueda utilizar la clave privada sin permiso, incluso disponiendo del archivo. El inconveniente es que `ssh` la pedirá en cada conexión.

El programa `ssh-agent` resuelve ese problema. Es un proceso que permanece en ejecución durante la sesión del usuario y que mantiene en memoria las claves privadas ya descifradas. La *passphrase* solo se introduce una vez por sesión. La clave se añade al agente con `ssh-add`:

```
alice@maine:~$ ssh-add
Enter passphrase for /home/alice/.ssh/id_rsa:
Identity added: /home/alice/.ssh/id_rsa (alice@maine)
```

La opción `-l` lista las claves cargadas en el agente:

```
alice@maine:~$ ssh-add -l
3072 SHA256:KXUrUqPxk/EPiXEBfgqK3HlSqvhphSeh6O+qm9tz1DM alice@maine (RSA)
```

Los entornos de escritorio habituales arrancan el agente automáticamente al iniciar la sesión. Además, el agente puede «reenviarse» al nodo remoto (opción `ForwardAgent`), lo que permite saltar desde ella a otros servidores utilizando las claves custodiadas en el nodo origen, pero sin copiarlas fuera de él.

22.4. Autenticación con certificado

Otra forma de autenticar un usuario es mediante certificados, una opción algo más avanzada que la autenticación con clave pública de la sección anterior. Estos certificados son emitidos por una autoridad de certificación

o CA (Certificate Authority), que firma (con su clave privada) el par de claves (privada/pública) de los usuarios. El resultado son certificados que permiten autenticar usuarios en hosts y viceversa.

La ventaja de este sistema es que el administrador de un servidor puede expedir certificados para un número arbitrario de usuarios. Un mismo par de claves firmado por un administrador puede otorgar derecho de acceso a múltiples servidores sin tener que modificar su configuración.

Las claves de la CA se crean también con `ssh-keygen`:

```
$ ssh-keygen -t rsa -f ca_key -P "" -C ca_key
Generating public/private rsa key pair.
Your identification has been saved in ca_key
Your public key has been saved in ca_key.pub
The key fingerprint is:
SHA256:nbN+pdyB3paUUnoPe60KJbkC3flKiXCVbSIH51wQXbo ca_key
```

Aunque `ssh-keygen` proporciona una gran cantidad de opciones, en este caso solamente utilizamos algunas de ellas:

- **`-t`** permite indicar el algoritmo (en este caso cifrado RSA).
- **`-f`** especifica el nombre del archivo destino.
- **`-P`** define la contraseña (vacía en este caso).
- **`-C`** añade un comentario que aparece al final de la clave pública.

La clave privada (`ca_key`), por supuesto, debe almacenarse de forma segura, ya que es con la que se firma y emite los certificados. La clave pública (`ca_key.pub`) es la que se utiliza el servidor SSH en el proceso de autenticación de los clientes.

22.4.1. Certificado de usuario

Los certificados SSH de usuario hacen posible autenticar usuarios en un servidor determinado. Para ilustrar cómo funcionan vamos a generar un certificado para el usuario `user` de la máquina `sidney` como lo haría su administrador. Primero creamos un par de claves pública/privada para el usuario, con el nombre `user_key`.

```
$ ssh-keygen -f user_key
```

Después, se firma la clave pública del usuario (`user_key.pub`) con la clave privada de la CA (`ca_key`):

```
$ ssh-keygen -s ca_key -I sidney -n user user_key.pub
```

Con este comando se ha generado también el certificado `user_key-cert.pub`.

Las opciones que se han usado en la firma son:

-s para indicar la clave con la que se quiere firmar *(sign)*.

-I para determinar la identidad del certificado, normalmente el nombre del *host* (puede utilizarse en el futuro para revocar un certificado, por ejemplo).

-n que permite definir una lista de usuarios y/o *hosts* donde el certificado es válido (en este caso únicamente el usuario `user`).

En el servidor, es necesario almacenar la clave pública de la CA con los permisos adecuados (644). En la máquina `sidney`:

```
root@sidney:/etc/ssh# chmod 644 ca_key.pub
root@sidney:/etc/ssh# ls -la ca_key.pub
-rw-r--r-- 1 user user    562 Mar  3 09:13 ca_key.pub
```

Para indicar que todas las claves de usuario firmadas por la CA se pueden autenticar en el servidor, se debe añadir lo siguiente en el archivo de configuración `/etc/ssh/sshd_config`:

```
TrustedUserCAKeys /etc/ssh/ca_key.pub
```

Para que los cambios tengan efecto reiniciamos el servidor SSH (programa `sshd`) con:

```
root@sidney:/home/user# systemctl restart sshd
```

Con esto termina la configuración el servidor (`sidney` en este ejemplo). Este proceso lo realizar automáticamente la construcción de la imagen docker y se puede ver en el archivo `Dockerfile`.

Por otro lado, el cliente tiene que copiar su clave privada (`user_key`) y su certificado (`user_key-cert.pub`) en su directorio `~/.ssh/`:

```
alice@maine:~$ ls -l ~/.ssh
-rw------- 1 alice    alice    2590 mar  3 09:35 user_key
-rw-rw-r-- 1 alice    alice    2020 mar  3 09:40 user_key-cert.pub
-rw-rw-r-- 1 alice    alice     137 mar  3 10:04 config
```

Además, habrá de indicar que quiere usar esa clave a la hora de acceder a la máquina `sidney`:

```
alice@maine:~$ ssh -i ~/.ssh/user_key sidney
```

Para mayor comodidad, puede añadirse la clave a la configuración de `sidney` en `~/.ssh/config`:

```
Host sidney
    hostname 172.20.0.2
    User user
    IdentityFile ~/.ssh/user_key
```

Y con esto se podrá acceder simplemente con:

```
alice@maine:~$ ssh sidney
```

La diferencia respecto a lo explicado en la sección 22.3 es que el usuario nunca necesitó una clave textual para acceder al servidor. De hecho, el servidor se puede configurar para que la única forma de autenticación permitida sea mediante clave pública [2], de modo que no hay posibilidad de usar `ssh-copy-id`. Esto se considera más seguro.

22.5. Copia de archivos con SCP

SCP (Secure Copy Protocol) es un protocolo de copia segura de archivos entre computadores cualesquiera conectados a Internet. Resulta especialmente cómodo ya que el propio servidor de `OpenSSH` lo incorpora y está activado por defecto.

Con la configuración previa, es tan sencillo como:

```
$ scp un-\file sidney:
un-\file                                      100%    5    16.6KB/s   00:00
```

También es posible copiar directorios recursivamente con la opción `-r`.

Y ¿qué más?

Este capítulo ha mostrado SSH como protocolo de shell remota, así como sus esquemas de autenticación —usuario/contraseña, clave pública y certificados firmados por una CA— y también como mecanismo de transferencia segura de archivos.

Sin embargo, el canal cifrado y autenticado que ofrece SSH tiene muchas más aplicaciones: redirección remota de puertos, túneles cifrados o su uso como proxy SOCKS para encaminar tráfico arbitrario a través de una única conexión. Estas capacidades resultan especialmente útiles para acceder a redes privadas o atravesar cortafuegos.

[2] `PasswordAuthentication no`

Capítulo 23

Invocación remota

Al terminar este capítulo, entenderás:

- Qué es un protocolo de petición-respuesta.
- Cuáles son las distintas semánticas de invocación.
- Qué es y cómo funciona la invocación remota.
- Qué son RPC y RMI.
- Qué es la transparencia de localización.
- Qué alternativas plantean Cap'n Proto y REST.
- Cuánto cuesta realmente la invocación remota.

El capítulo 15 presentó el patrón petición-respuesta; el cliente envía una petición —normalmente una operación sobre un recurso— y el servidor responde con un resultado. Ese patrón lo define, en cada caso, un protocolo de aplicación propio: HTTP para la web, SMTP para el correo, DNS para la resolución de nombres, etc. Cada uno especifica su propio formato de mensajes, sus propias operaciones y su propia forma de indicar errores, adaptados al problema concreto que resuelven.

Diseñar un protocolo de aplicación desde cero no es trivial; hay que definir la sintaxis de los mensajes, decidir cómo se delimitan sobre el flujo de bytes subyacente, considerar la evolución del protocolo (versiones, compatibilidad) y, finalmente, implementar esa especificación en cada lenguaje o plataforma que vaya a usarla. Para muchas aplicaciones, ese coste es desproporcionado en relación con lo que realmente necesitan: simplemente ejecutar código en un nodo remoto y obtener el resultado, igual que se invoca una función o un método local.

La **invocación remota** es una solución general para esa necesidad. En lugar de definir un protocolo específico por aplicación, ofrece al programador una infraestructura genérica —el **middleware** de comunicaciones—, común a cualquier aplicación, que se encarga de transportar la invocación hasta el nodo remoto, ejecutarla allí y devolver el resultado. RPC (Remote Procedure Call) sigue este enfoque para procedimientos (funciones indepen-

dientes); RMI (Remote Method Invocation) hace lo mismo para métodos de objetos. En ambos casos, el middleware genera el código encargado de establecer la comunicación, serializar los argumentos, enviarlos, ejecutar el código, devolver la respuesta y entregar el resultado de vuelta al cliente, de modo que invocar una operación remota se parece, desde el punto de vista del programador, a invocar una operación local.

Lo que realmente diferencia una invocación remota de un mero intercambio de mensajes es que la invocación implica la ejecución de un fragmento de código en el receptor a expensas del emisor. La petición indica qué pieza de código se ha de ejecutar y con qué argumentos, mientras que la respuesta transporta el resultado.

En la invocación remota, el que envía el mensaje de petición (la invocación) es el cliente, y el que recibe y ejecuta el código asociado es el servidor. Sin embargo, aunque es lo habitual, esos roles no tienen por qué coincidir con los de los procesos involucrados, es decir, un servidor TCP podría ser el que envía la invocación hacia el cliente TCP.

23.1. Protocolos de petición-respuesta

Dos procesos que se ejecutan en nodos distintos no comparten memoria, así que la única forma de interactuar es enviándose mensajes a través de la red: eso es el *paso de mensajes*. Se reduce, en su forma más básica, a dos primitivas sobre un socket, `send(destino, mensaje)` y `receive(origen, mensaje)`, sin que deba existir ninguna relación entre mensajes sucesivos.

El patrón *petición-respuesta* es la forma más simple de paso de mensajes en una comunicación extremo a extremo: asocia un mensaje de respuesta al de petición que la originó. Este patrón lo emplean protocolos de varias capas, pero es especialmente frecuente en la de aplicación.

Diseñar un protocolo de aplicación obliga al diseñador a tomar esencialmente las mismas decisiones, independientemente del problema que resuelva el protocolo, aunque el resultado de esas decisiones varía según las necesidades de la aplicación.

La invocación remota no es diferente, aunque no resuelve todas esas decisiones del mismo modo: el middleware fija de antemano algunas de ellas al generar automáticamente el protocolo subyacente, mientras que deja otras en manos del programador. Estas son algunas:

- **Síncrono vs. asíncrono**: el cliente de la petición puede bloquear en espera de la respuesta, o continuar inmediatamente con otra tarea. La primera es una invocación síncrona y es más fácil de entender,

porque el cliente sabe en todo momento si la petición sigue pendiente; además es más similar a una llamada local. Una invocación asíncrona aprovecha el tiempo de espera, pero debe encargarse de correlacionar más adelante cada respuesta con la petición que le corresponda.

- **Con o sin estado**: el servidor puede mantener el estado entre peticiones sucesivas del mismo cliente, o resolver cada una de forma independiente, sin memoria de las anteriores. Un protocolo sin estado es más sencillo de escalar y de recuperar tras un fallo, porque una réplica del servicio puede atender la siguiente petición sin compartir contexto previo con la que atendió la anterior; a cambio, cualquier información que dependa de peticiones anteriores tiene que incluirse explícitamente en cada una.
- **Transporte y fiabilidad**: sobre TCP la conexión ya garantiza entrega y orden, pero establecerla tiene un coste que no siempre compensa para un intercambio breve. Sobre UDP no hay tal garantía: si se pierde el mensaje con la petición o el que lleva la respuesta, nada lo detecta ni corrige.

Entre la fiabilidad completa que ofrece TCP y la ausencia total de UDP hay un término medio (o varios): el protocolo de petición-respuesta puede construir sobre UDP solo las medidas de fiabilidad que necesite, sin la sobrecarga que implican las que no necesita. El identificador de mensaje, como el que llevan las consultas DNS, permite cotejar cada respuesta con la petición que la originó, aunque haya varias en curso o el transporte las reordene; ese mismo identificador le permite al servidor detectar y descartar duplicados en lugar de procesarlos de nuevo. Un temporizador permite detectar peticiones o respuestas perdidas. Con ese mismo temporizador se puede añadir la retransmisión, reintentando la petición un número limitado de veces antes de darla por fallida definitivamente. Identificador, temporizador y retransmisión son las piezas básicas de cualquier protocolo ARQ; cada protocolo de petición-respuesta elige cuáles de ellas necesita, y con qué grado de exigencia.

Estos elementos definen tres estilos de intercambio, según qué mensajes intervienen y con qué fin:

- R (Request): el cliente envía la petición y no espera ninguna respuesta (no bloqueante). Sirve para peticiones *one-way* habitualmente sobre transporte no confiable. No hay un valor de retorno, confirmación, temporizadores ni retransmisiones. También podría ocurrir que hubiera un resultado o confirmación, pero al cliente de la petición no le interese, por lo que no espera ni recoge el mensaje.

- RR (Request-Reply): El cliente espera un mensaje de respuesta (bloqueante). La respuesta, además de posiblemente contener un resultado, puede llevar un código de confirmación o de error. Puede utilizar TCP como transporte o bien implementar confiabilidad básica con un ARQ. RR es el estilo más común en protocolos de aplicación.
- RRA (Request-Reply-Acknowledge): cuando el servidor detecta una petición duplicada, no basta con descartarla: si ya ejecutó la operación, tiene que reenviar el mismo resultado en lugar de ejecutarla de nuevo, así que necesita conservarlo en una caché llamada *historia*, indexada por el identificador de mensaje, en lugar de descartarlo justo después de enviarlo. La duda es cuánto tiempo mantener cada entrada: sin ninguna señal del cliente, el servidor solo puede adivinarlo con un temporizador propio y un margen de seguridad. Este estilo añade un tercer mensaje con el que el cliente confirma la recepción de la respuesta, y con el que el servidor sabe explícitamente cuándo puede liberar la entrada de la historia. Sin este mensaje, la historia crecería indefinidamente.

23.2. Medidas de tolerancia a fallos

Frente a la pérdida de un mensaje, la invocación remota puede aplicar tres medidas, que ofrecen fiabilidad creciente:

- **Reintentar la petición**: si el temporizador expira sin recibir respuesta, el cliente retransmite el mismo mensaje, con el mismo identificador. Se aplica un número limitado de repeticiones, que de alcanzarse, será considerado un error. Obviamente implica obligatoriamente un mensaje de respuesta/confirmación. Sin esta medida, el cliente no puede saber si la petición llegó al servidor.
- **Filtrar duplicados**[1]: el servidor conserva los identificadores de las peticiones ya atendidas, para reconocer una retransmisión y no repetir la ejecución de la operación que le corresponde. Sin este filtro, cada retransmisión se procesa de nuevo.
- **Retransmisión del resultado**: al detectar una petición duplicada, el servidor no se limita a descartarla, sino que reenvía el resultado que ya calculó y tiene guardado, en lugar de dejar al cliente sin respuesta pese a que la operación sí llegó a ejecutarse.

[1] denominado a menudo «deduplicación»

23.3. Operaciones idempotentes

Una operación es **idempotente** cuando ejecutarla varias veces (con los mismos argumentos) produce sobre el estado del servidor el mismo efecto que ejecutarla una sola vez. La idempotencia es una propiedad de la semántica propia de la operación. Es interesante porque estas operaciones no requieren detección de duplicados, lo que simplifica su implementación.

Un estilo RRA necesita cachear la respuesta de cada petición precisamente en el caso de las operaciones no idempotentes, que normalmente se asume que es caso más común.

Algunos ejemplos de operaciones idempotentes podrían ser:

```
set_size(25)
stop()
car.get_color()
get_time()
```

Es interesante destacar que `get_time()()` sea idempotente, porque aunque el valor que retorne sea diferente en cada llamada, cumple la condición de no modificar el estado del servidor.

Mientras que los siguientes son no idempotentes:

```
push("item")
counter.inc()
send_mail("user@example.com", "Hello")
```

Este concepto de idempotencia no coincide con el que se utiliza en matemáticas. Aquel está relacionado con el resultado de la aplicación de una función al resultado previo de esa misma función, que de hecho ni siquiera aplica a varios de los ejemplos anteriores porque requiere que la función tenga al menos un argumento.

23.4. Semántica de invocación

Combinar las tres medidas de tolerancia a fallos —reintentar petición, filtrar duplicados y retransmitir resultado— da lugar a distintas garantías sobre potencialmente cuántas veces se llega a ejecutar la operación remota. Estas garantías se conocen como semánticas de invocación:

- *maybe*: no aplica ninguna medida. Si se pierde la petición o la respuesta, el cliente no tiene forma de saberlo ni de corregirlo. La operación se ejecuta cero o una vez, sin que el cliente pueda distinguir cuál de los dos casos ha ocurrido. Es la semántica que resulta, por ejemplo, de un estilo R (Request) sobre un transporte no confiable. Por ejemplo

es útil cuando el cliente envía mensajes actualizando periódicamente el valor de una medida (*p. ej.* la temperatura de una habitación), y no es un problema que se pierda alguno de esos mensajes.

- *at-least-once*: reintenta la petición hasta tener una confirmación, pero no filtra duplicados. Cada retransmisión que llega al servidor se ejecuta de nuevo, así que la operación puede llegar a ejecutarse una o más veces. Garantiza que se ejecuta al menos una vez, a costa de no poder garantizar que no se repita. Se puede implementar con el estilo RR o R con transporte confiable si no requiere valor de retorno.

- *at-most-once*: además de reintentar, el servidor filtra los duplicados. Conserva el resultado ya calculado de la primera ejecución para volver a enviarlo si la petición se repite. La operación se ejecuta como mucho una vez (0 o 1 veces). El cliente siempre recibe una respuesta, ya sea para informar del resultado o de un error, es decir, a diferencia de *maybe* el cliente sabe si la operación se ejecutó o no. Es la semántica que ofrece el estilo RRA (Request-Reply-Acknowledge).

- *exactly-once*: ejecuta la operación exactamente una vez. Esto es algo que las tres medidas de tolerancia a fallos no pueden garantizar por sí solas, porque únicamente actúan sobre la comunicación. La garantía *exactly-once* requiere asegurar la continuidad del estado del servidor, particularmente entre la ejecución de la operación y su registro en la historia, por lo que ambas se ejecutan como una operación atómica. Además requiere de un sistema adicional para recuperar al proceso servidor tras un *crash* con recuperación transparente del estado previo. Eso lo convierte en un problema realmente complejo y difícilmente justificable. En la práctica, lo que se suele implementar es una variante más simple llamada *effectively-once*, que es una combinación de *at-least-once* sobre operaciones idempotentes.

El fabricante de cada middleware decide qué semánticas de invocación ofrece, en función de los estilos de intercambio y de las medidas de tolerancia a fallos que implementa. Al programador le corresponde elegir, entre las disponibles, la más adecuada en cada caso, y tomar otras decisiones que el middleware no impone, como invocar de forma síncrona o asíncrona. Esto es una ventaja, le otorga mayor control para adaptar cada invocación a las necesidades y requisitos de la aplicación.

23.5. Mecanismo de invocación remota

Con independencia de la tecnología concreta —RPC, RMI u otra—, todo mecanismo de invocación remota comparte una arquitectura similar. Comienza con una *interfaz remota* conocida tanto por el cliente como por el servidor. La interfaz es una lista de las operaciones que ofrecerá (implementará) el servidor, con sus nombres, parámetros y tipos de retorno, mientras que el cliente será quien invoque esas operaciones.

El middleware proporciona habitualmente un compilador que genera el código necesario para transportar las invocaciones (identificadores y argumentos) hasta el servidor y llevar los resultados de vuelta al cliente.

Esos fragmentos de código se denominan *stubs* (cabos) y realizan tareas diferentes para cliente y el servidor. Los *stubs* del cliente proporcionan funciones que implementan la interfaz remota, pero no tienen funcionalidad de la aplicación. Lo que hacen es serializar y transportar las invocaciones hasta el servidor. Los *stubs* del servidor incluyen la lógica de multiplexación que localiza y ejecuta funciones con las mismas firmas. Es habitual que ese compilador ofrezca la posibilidad de generar implementaciones huecas para las operaciones, que suelen denominarse *esqueletos (skeletons)*.

La Figura 23.1 resume el proceso completo. El compilador de interfaces genera los *stubs* de cliente y de servidor a partir de la interfaz, y el compilador de cada lenguaje los combina con el código que escribe el programador para obtener los ejecutables. En la figura, el cliente está escrito en Java y el servidor en C++, de modo que el compilador de interfaces genera el *stub* de cliente en Java y el de servidor en C++.

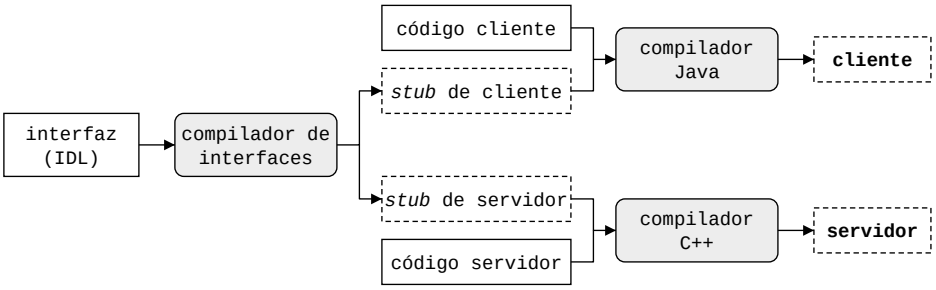


FIGURA 23.1: Generación de *stubs* y ejecutables a partir de la interfaz (en discontinua los ficheros generados)

El recorrido completo de una invocación remota, desde que el cliente invoca su *stub* hasta que recibe el resultado, sigue siempre los pasos del Cuadro 23.1.

Cliente	**Servidor**
Invoca la operación remota pasando los argumentos correspondientes, de forma similar a como invocaría una función o un método local. En realidad está invocando al *stub*.	
El *stub* serializa el identificador de la operación junto con los argumentos , y con eso construye el mensaje de petición.	
El *stub* del cliente envía la petición al nodo servidor, con el estilo de intercambio y las medidas de fiabilidad aplicados.	
(bloquea en espera de la respuesta)	El *dispatcher* del servidor recibe la petición. El identificador de operación incluido en el mensaje indica qué procedimiento o método se ha de ejecutar.
	El *stub* del servidor deserializa los argumentos contenidos en el mensaje, reconstruyendo a partir de ellos los valores nativos que necesita la operación.
	El *stub* del servidor invoca localmente la implementación real de la operación.
	Al termianr la ejecución, el *stub* del servidor serializa el resultado —o el error, si la operación falló— en un mensaje de respuesta.
El *stub* recibe la respuesta, deserializa el resultado y lo devuelve como valor de retorno de la llamada original.	

CUADRO 23.1: Pasos de una invocación remota

El identificador de la operación (paso 4) le indica al servidor qué operación quiere ejecutar. No se debe confundir con el identificador de mensaje (§ 23.1), que se utiliza para distinguir ocurrencias distintas, incluso aunque correspondan a la misma operación.

El mecanismo de invocación remota esencialmente ofrece transparencia: la invocación al método remoto es muy similar sintácticamente a una llamada local equivalente. Sin embargo, hay varias diferencias que no se pueden (y no se deben) ocultar: latencia, errores en el servidor, fallo parcial o las garantías de entrega aplicadas (§ 23.4).

23.5.1. pico-rpc

Pico-rpc es un middleware RPC mínimo escrito en Python y con fines exclusivamente docentes. Implementa las piezas elementales del mecanismo de invocación remota: lenguaje de descripción de interfaces, compilador, generación de los *stubs* de cliente y servidor, y *dispatcher*; pero no implementa la mayoría del control de errores o mecanismos de seguridad. Con pico-rpc resulta sencillo hacer el recorrido completo de la descripción de RPC de Coulouris *et al.* [63]. Consta de los siguientes ficheros:

- `picorpc.py` es el *runtime* del middleware. Contiene las primitivas para describir interfaces y el *dispatcher* del servidor.
- `rpcgen.py` es el compilador de interfaces. Genera los *stubs* a partir de la definición de la interfaz.
- `math.idl.py` es la definición de la interfaz del ejemplo.
- `server.py` corresponde con el servidor de la aplicación, que incluye implementación de los procedimientos remotos que aporta el programador de la aplicación. Por último `client.py` es el cliente que realiza la invocación.

Interfaz

Normalmente, la definición de la interfaz se escribe con un lenguaje específico y puramente declarativo, llamado IDL, que es independiente del lenguaje de programación en el que se implementa cliente y servidor. El compilador genera los *stubs* en el lenguaje de destino a partir de esa definición. Por simplicidad, en pico-rpc, la definición de la interfaz se escribe también en Python mediante una clase que hereda de `Interface`, y un método sin cuerpo por cada operación. Las anotaciones de tipo de los parámetros y del valor de retorno determinan cómo se serializa cada valor (Listado 23.1).

```python
from picorpc import Interface, uint8, uint64

class Math(Interface):
    def factorial(n: uint8) -> uint64: ...
    def power(base: uint8, exp: uint8) -> uint64: ...
```

Listado 23.1: Interfaz del servicio `Math`
/pico-rpc/math.idl.py

Los tipos `uint8` y `uint64` corresponden directamente con los caracteres de formato de `struct` para enteros sin signo de 1 y 8 bytes (`'B'` y `'Q'`). El identificador de cada operación es su posición en la interfaz: 0 para `factorial()` y 1 para `power()`.

El «compilador» lee esta definición y genera los *stubs* de cliente y servidor:

```
$ ./rpcgen.py math.idl.py
generated math_client_stub.py and math_server_stub.py from math.idl.py
```

Cliente

El *stub* de cliente es una clase (`MathStub`) con un método por cada operación de la interfaz (Listado 23.2). Cada método abre una nueva conexión TCP, construye el mensaje de petición con el identificador de la operación (1 byte) seguido de los argumentos serializados, y espera la respuesta, que contiene el resultado serializado como un entero de 8 bytes. Esto encaja con el estilo RR en el que cada conexión transporta una única petición y su respuesta. No hace falta un identificador de mensaje para asociarlas, porque la conexión no se reutiliza y TCP resuelve las retransmisiones de segmentos sin que la aplicación reciba duplicados.

```
class MathStub:
    FACTORIAL = 0
    POWER = 1

    def __init__(self, host, port):
        self.server_endpoint = host, port

    def factorial(self, n):
        sock = socket.socket()
        sock.connect(self.server_endpoint)

        request = struct.pack('BB', self.FACTORIAL, n)
        sock.sendall(request)

        reply = sock.recv(MAX_MESSAGE_SIZE)
        result = struct.unpack('Q', reply)[0]

        sock.close()
        return result
```

LISTADO 23.2: *Stub* de cliente generado (fragmento)
`math_client_stub.py`

El programa cliente solo necesita instanciar el *stub* indicando el *endpoint* del servidor e invocar sus métodos del mismo mode que lo haría con un objeto convencional (Listado 23.3).

```
import sys

from math_client_stub import MathStub

if len(sys.argv) != 3:
    print("usage: ./client <server> <arg>")
```

```
    exit(1)

server = sys.argv[1]
arg = int(sys.argv[2])

stub = MathStub(server, 2000)
print("factorial({}) = '{}'".format(arg, stub.factorial(arg)))
print("power({}, {}) = '{}'".format(arg, arg-1, stub.power(arg, arg-1)))
```

LISTADO 23.3: Cliente del servicio `Math`
`pico-rpc/client.py`

Servidor

En el lado del servidor, el compilador genera una función *stub* por cada operación y una tabla (`STUBS`) que asocia cada identificador de operación con su función *stub* (Listado 23.4). Cada una de esas funciones deserializa los parámetros, invoca la implementación real de la operación y serializa el resultado.

```
def factorial_stub(args, implementation):
    args = struct.unpack('B', args)
    result = implementation.factorial(*args)
    return struct.pack('Q', result)

def power_stub(args, implementation):
    args = struct.unpack('BB', args)
    result = implementation.power(*args)
    return struct.pack('Q', result)

FACTORIAL = 0
POWER = 1

STUBS = {
    FACTORIAL: factorial_stub,
    POWER: power_stub,
}
```

LISTADO 23.4: *Stub* de servidor generado
`math_server_stub.py`

La multiplexación la realiza el *dispatcher* (clase `Dispatcher` de `picorpc.py`). Acepta conexiones de forma iterativa (como en § 15.2), lee el mensaje de petición, utiliza su primer byte para localizar la función *stub* correspondiente en la tabla `STUBS` y le entrega el resto del mensaje, es decir, los argumentos serializados. Por último, envía al cliente el resultado que devuelve el *stub*.

En el servidor el programador define una clase que implementa las operaciones de la interfaz (`MathI`), y registra una instancia en el *dispatcher* junto con el módulo del *stub* de servidor (Listado 23.5).

```python
from picorpc import Dispatcher
import math_server_stub

class MathI:
    def factorial(self, n):
        if n == 0:
            return 1

        return n * self.factorial(n - 1)

    def power(self, base, exp):
        return base ** exp

dispatcher = Dispatcher(port=2000)
dispatcher.register(math_server_stub, MathI())
dispatcher.run()
```

Listado 23.5: Servidor del servicio `Math`
`/pico-rpc/server.py`

Es importante aclarar que a pesar de que pico-rpc utiliza clases tanto para la definición de la interfaz y para la implementación de las operaciones, lo hace solo por conveniencia de implementación. Sus métodos no tienen relación entre sí y funcionan como operaciones independientes.

Ejecución del servidor y del cliente en el mismo computador:

Cliente

```
$ ./client.py 127.0.0.1 5
factorial(5) = '120'
power(5, 4) = '625'
```

Servidor

```
$ ./server.py
Server ready: ('0.0.0.0', 2000)
```

Figura 23.2: Servidor y cliente pico-rpc

Para mantener el código al mínimo, pico-rpc prescinde deliberadamente de aspectos que ningún middleware real puede ignorar. Solo admite enteros sin signo, supone que cada mensaje llega completo en una única llamada a `recv()` —ignora la E/S parcial— el servidor es iterativo y no aplica ninguna medida de tolerancia a fallos más allá de las que proporciona TCP. Tampoco transporta errores de vuelta al cliente si la ejecución de la operación o la serialización falla, *p. ej.* el resultado de `factorial(21)` no cabe en un `uint64`.

23.5.2. ONC RPC

ONC RPC—también llamado Sun RPC— fue uno de los middlewares pioneros de la invocación remota. Lo desarrolló Sun Microsystems en los años

80 y sobre él está construido el popular sistema de ficheros en red NFS que aún hoy día se sigue utilizando.

Serializa los datos con un formato llamado XDR [64], y las interfaces de los procedimientos remotos se describen en ficheros `.x`, un lenguaje que extiende XDR para declarar programas y procedimientos. La implementación original de Sun, de la que derivan las actuales, solo permite escribir clientes y servidores en C, aunque existen implementaciones independientes para otros lenguajes.

El ejemplo /`onc-rpc-factorial` es un servicio equivalente al de pico-rpc, aunque únicamente con la operación `factorial()`. La interfaz (Listado 23.6) declara un «programa», identificado por un número (`600000002`), con una versión (`1`) que contiene el procedimiento `FACTORIAL`, también numerado (`1`). El número de procedimiento cumple la misma función que el identificador de operación de pico-rpc, pero también en otros middlewares reales.

```
program FACTORIAL_PROG {
   version FACTORIAL_VERS {
    unsigned long FACTORIAL(int n) = 1;
   } = 1;
} = 600000002;
```

Listado 23.6: Interfaz del servicio `FACTORIAL_PROG`
/`onc-rpc-factorial/factorial.x`

A partir de la interfaz, `rpcgen` (`rpcsvc-proto`) genera tres ficheros: `factorial.h`, con las constantes y los prototipos; `factorial_clnt.c`, el *stub* de cliente, y `factorial_svc.c`, el *stub* de servidor, que incluye el *dispatcher* y la función `main()` del servidor. El programador aporta lo demás:

- `factorial_server.c` es la implementación del procedimiento en la función `factorial_1_svc()`, cuyo nombre se forma con el del procedimiento y el número de versión. Devuelve un puntero a una variable estática porque el *stub* serializa el resultado después de que la función haya retornado (Listado 23.7).
- `factorial_client.c` obtiene un manejador de cliente con `clnt_create()`, indicando el nodo, el programa, la versión y el transporte (UDP), e invoca la operación con `factorial_1()`, que es la función del *stub* de cliente (Listado 23.8).

```
u_long *factorial_1_svc(int *n, struct svc_req *rqstp) {
    static u_long result;
    result = 1;

    printf("Received: n=%d \n", *n);
```

```
    for (int i=*n; i>0; i--)
        result *= i;

    return &result;
}
```

LISTADO 23.7: Implementación del procedimiento `FACTORIAL`
/onc-rpc-factorial/factorial_server.c

```
void factorial_prog_1(char *host, int n) {
    CLIENT *clnt;
    u_long *result_1;

#ifndef DEBUG
    clnt = clnt_create (host, FACTORIAL_PROG, FACTORIAL_VERS, "udp");
    if (clnt == NULL) {
        clnt_pcreateerror (host);
        exit (1);
    }
#endif

    result_1 = factorial_1(&n, clnt);
    if (result_1 == NULL)
        clnt_perror(clnt, "call failed");
    else
        printf("factorial(%d): %lu", n, *result_1);

#ifndef DEBUG
    clnt_destroy (clnt);
#endif
}
```

LISTADO 23.8: Cliente del servicio `FACTORIAL_PROG`
/onc-rpc-factorial/factorial_client.c

Al arrancar, el servidor crea un socket UDP y otro TCP, sin puerto fijo, y los registra en `rpcbind` (versión actual del antiguo «portmapper»), un servicio que asocia cada combinación de programa, versión y transporte con el puerto en el que está disponible.

El servidor ofrece los dos transportes para que el cliente pueda elegir cuál usar. Para eso se utiliza el último argumento de `clnt_create()` (Listado 23.8). Lógicamente cada transporte implica una semántica de invocación distinta (§ 23.4). La función `clnt_create()` consulta a `rpcbind` en el nodo remoto para averiguar el puerto que corresponde al transporte elegido.

- Con UDP el *runtime* del cliente retransmite la petición si la respuesta no llega a tiempo, y cada mensaje lleva un identificador de transacción (`xid`) para asociar la respuesta con su petición. Sin embargo, el servidor no filtra duplicados salvo que se active expresamente su caché de peticiones, así que la semántica es *at-least-once*. Con la caché

activa, el servidor reenvía la respuesta guardada en lugar de ejecutar de nuevo el procedimiento, lo que se aproxima a *at-most-once*, aunque la caché tiene un tamaño limitado y se pierde si el servidor se reinicia.

- Con TCP el *runtime* del cliente no retransmite. La petición se envía una sola vez y es TCP quien garantiza que llega sin pérdidas ni duplicados. Si la conexión se interrumpe o expira la espera, la llamada devuelve un error sin reintentarlo y el cliente no puede saber si el procedimiento llegó a ejecutarse. El procedimiento se ejecuta, por tanto, como mucho una vez: la semántica es *at-most-once*.

El cliente del ejemplo utiliza UDP. Para este servicio es suficiente, porque calcular el factorial es una operación idempotente.

El ejemplo /`onc-rpc-hello` es aún más básico. Solo define un procedimiento que recibe una cadena y la imprime en el servidor. Está tomado, con algunas simplificaciones, de la documentación oficial de ONC RPC [65]. Incluye el mismo programa en dos versiones, una local con una llamada a función convencional (directorio `local`) y otra en la que esa función pasa a ser un procedimiento remoto (directorio `remote`), lo que permite comparar qué cambia al convertir una llamada local en remota.

23.5.3. gRPC

gRPC es un middleware RPC de código abierto que Google publicó en 2015 a partir de su experiencia con su sistema interno de invocación remota. Actualmente es uno de los más utilizados para comunicar servicios dentro de la infraestructura interna o un centro de datos, sobre todo para arquitecturas de microservicios.

Las interfaces se describen en ficheros con extensión `.proto` en el lenguaje Protocol Buffers, que es a la vez el IDL y el formato de serialización binario. A diferencia de ONC RPC, el compilador genera *stubs* para muchos lenguajes (C++, Java, Go, Python, etc.), de modo que cliente y servidor se pueden escribir en lenguajes distintos. Como transporte utiliza HTTP/2 sobre TCP, y opcionalmente acepta cifrado con TLS.

El ejemplo /`grpc-factorial` es equivalente al ejemplo de ONC RPC, salvo que en este caso se puede escribir en Python. La interfaz (Listado 23.9) declara un «servicio» `Math` con la operación `factorial()`. Una operación unaria de gRPC recibe exactamente un mensaje y devuelve exactamente otro, por lo que el parámetro y el resultado se definen como mensajes (`FactorialRequest` y `FactorialReply`), aunque cada uno contenga un único campo. El número que acompaña a cada campo (`= 1`) es el que lo identifica en el mensaje serializado. Eso permite añadir campos nuevos a un mensaje

sin romper la compatibilidad con clientes o servidores que usan la versión anterior de la interfaz, algo que Google consideró una característica esencial en el diseño de Protocol Buffers.

```
syntax = "proto3";

package math;

service Math {
  rpc factorial (FactorialRequest) returns (FactorialReply) {}
}

message FactorialRequest {
  int32 value = 1;
}

message FactorialReply {
  int64 result = 1;
}
```

LISTADO 23.9: Interfaz del servicio `Math` en gRPC
/`grpc-factorial/math.proto`

El compilador `protoc`, que para Python se ejecuta como el módulo `grpc\_tools` (`python3-grpc-tools`), genera dos ficheros a partir de la interfaz: `math_pb2.py`, con las clases de los mensajes y su serialización, y `math_pb2_grpc.py`, con los *stubs* de cliente (`MathStub`) y servidor (`MathServicer`). El programador aporta lo demás:

- `server.py` implementa la operación en una clase que hereda de `MathServicer` (Listado 23.10). El método `factorial()` recibe el mensaje de petición ya deserializado y un contexto, que da acceso a los metadatos de la llamada y permite, entre otras cosas, devolver un código de error. La función `add_MathServicer_to_server()` registra esa implementación en el servidor, que la asocia con las operaciones de la interfaz. Es el equivalente a la tabla `STUBS` de pico-rpc.
- `client.py` crea un canal hacia el *endpoint* del servidor, instancia el *stub* sobre ese canal e invoca la operación con un objeto `FactorialRequest` como argumento (Listado 23.11).

```
from concurrent import futures

import grpc
import math_pb2
import math_pb2_grpc

class Math(math_pb2_grpc.MathServicer):
    def factorial(self, request, context):
        result = 1
```

```
        n = request.value

        while n:
            result *= n
            n -= 1

        return math_pb2.FactorialReply(result=result)

server = grpc.server(futures.ThreadPoolExecutor(max_workers=10))
math_pb2_grpc.add_MathServicer_to_server(Math(), server)
server.add_insecure_port('[::]:2000')
server.start()

try:
    server.wait_for_termination()

except KeyboardInterrupt:
    server.stop(0)
```

LISTADO 23.10: Servidor del servicio `Math` en gRPC
/grpc-factorial/server.py

```
import sys

import grpc
import math_pb2
import math_pb2_grpc

if len(sys.argv) != 3:
    print("usage: ./client <server> <value>")
    sys.exit(1)

server = sys.argv[1]
value = int(sys.argv[2])

channel = grpc.insecure_channel('{}:2000'.format(server))
stub = math_pb2_grpc.MathStub(channel)
response = stub.factorial(math_pb2.FactorialRequest(value=value))
print("result = '{}'".format(response.result))
```

LISTADO 23.11: Cliente del servicio `Math` en gRPC
/grpc-factorial/client.py

El servidor escucha en un puerto TCP indicado (2000). No existe ningún equivalente a `rpcbind`, el cliente debe conocer de antemano el `d:el` servidor. Tanto `insecure_channel()` en el cliente como `add_insecure_port()` en el servidor indican que la comunicación no está cifrada. En un despliegue real se aconseja usar sus equivalentes con TLS. Además, a diferencia del de pico-rpc, el servidor es concurrente; atiende las peticiones con un *pool* de hasta 10 hilos (`ThreadPoolExecutor`).

Cada invocación se envía como una petición `POST` de HTTP/2 dirigida a una ruta formada por el nombre del servicio y el de la operación (`/math.Math/factorial`). El cuerpo contiene el mensaje serializado precedido de 5 bytes:

uno que indica si va comprimido y cuatro con su longitud. El resultado de la ejecución —éxito o un código de error— se devuelve en las cabeceras finales de la respuesta. HTTP/2 permite multiplexar varias invocaciones simultáneas sobre la misma conexión TCP, así que el cliente no necesita abrir una conexión por cada llamada, como hace pico-rpc.

Respecto a la semántica de invocación, el *runtime* del cliente no reintenta por defecto una petición que haya podido llegar al servidor, y TCP evita pérdidas y duplicados. Si la llamada falla o expira su plazo, el cliente recibe un error y no puede saber si la operación llegó a ejecutarse, de modo que, igual que ONC RPC sobre TCP, la semántica es *at-most-once*. Los reintentos automáticos se pueden activar, pero en ese caso es responsabilidad del programador determinar si la operación es idempotente.

Por supuesto, gRPC transporta los errores hasta el cliente. Si la implementación de `factorial()` lanza una excepción —*p. ej.* porque el resultado de `factorial(21)` no cabe en el campo `int64` de `FactorialReply`— el cliente recibe un error con el código de estado `UNKNOWN` y el texto de la excepción, en lugar de quedarse esperando o recibir un mensaje corrupto.

Por último, además de las invocaciones «unarias» como la del ejemplo (una petición y una respuesta), gRPC admite otras tres modalidades en las que uno o ambos extremos envían un flujo de mensajes:

- *server streaming*, en el que el servidor devuelve una secuencia de respuestas a una única petición;
- *client streaming*, en el que el cliente envía una secuencia de mensajes y recibe una única respuesta; y
- *bidireccional*, en la que ambos envían secuencias de forma independiente.

Estos modos se declaran en la interfaz anteponiendo la palabra clave `stream` al tipo del parámetro, al del resultado o a ambos.

El ejemplo /`grpc-streaming` amplía el servicio `Math` con una operación para cada una de esas modalidades (Listado 23.12): `factorials()` obtiene un valor límite y el servidor devuelve la secuencia de factoriales hasta ese límite; `sum()` recibe una secuencia de enteros y devuelve su suma; y `factorial_each()` devuelve el factorial de cada valor que recibe.

```
syntax = "proto3";

package math;

service Math {
```

```
  // the server returns 1!, 2!, ..., n!
  rpc factorials (Limit) returns (stream Number) {}

  // the client sends numbers and the server returns their sum
  rpc sum (stream Number) returns (Number) {}

  // the server returns the factorial of each number as it arrives
  rpc factorial_each (stream Number) returns (stream Number) {}
}

message Limit {
  int32 value = 1;
}

message Number {
  int64 value = 1;
}
```

Listado 23.12: Interfaz del servicio `Math` con flujos
`/grpc-streaming/math.proto`

En Python esos flujos se implementan con iteradores, de modo que el *stub* traduce cada modalidad al mecanismo de generadores (Listado 23.13). Cuando el resultado es un flujo, la implementación es un generador que entrega un mensaje cada vez con `yield`, y el *runtime* lo envía en cuanto se produce. Cuando el parámetro es un flujo, la implementación recibe iterador (`request_iterator`) del que extrae los mensajes conforme llegan; ese iterador se agota cuando el cliente cierra su mitad del flujo.

```
import math
import time
from concurrent import futures

import grpc
import math_pb2
import math_pb2_grpc

class Math(math_pb2_grpc.MathServicer):
    def factorials(self, request, context):
        for n in range(1, request.value + 1):
            time.sleep(0.5)
            yield math_pb2.Number(value=math.factorial(n))

    def sum(self, request_iterator, context):
        total = 0
        for number in request_iterator:
            total += number.value

        return math_pb2.Number(value=total)

    def factorial_each(self, request_iterator, context):
        for number in request_iterator:
            yield math_pb2.Number(value=math.factorial(number.value))

server = grpc.server(futures.ThreadPoolExecutor(max_workers=10))
```

```
math_pb2_grpc.add_MathServicer_to_server(Math(), server)
server.add_insecure_port('[::]:2000')
server.start()

try:
    server.wait_for_termination()

except KeyboardInterrupt:
    server.stop(0)
```

Listado 23.13: Servidor con las tres modalidades de flujo
/grpc-streaming/server.py

En el cliente, la correspondencia es la opuesta (Listado 23.14): la invocación de una operación que devuelve un flujo devuelve en realidad un iterador que el cliente recorre, y una operación cuyo parámetro es un flujo admite un iterable como argumento, en este caso el generador `numbers()`.

```
def numbers(values):
    for value in values:
        yield math_pb2.Number(value=value)

print("server streaming:")
for reply in stub.factorials(math_pb2.Limit(value=limit)):
    print("  ", reply.value)

print("client streaming:")
reply = stub.sum(numbers(range(1, limit + 1)))
print("  ", reply.value)

print("bidirectional streaming:")
for reply in stub.factorial_each(numbers(range(1, limit + 1))):
    print("  ", reply.value)
```

Listado 23.14: Cliente con las tres modalidades de flujo
/grpc-streaming/client.py

La diferencia respecto a encadenar varias invocaciones unarias no es solo sintáctica. Todos los mensajes de un mismo flujo se envían sobre el mismo *stream* HTTP/2, que es la forma en la que gRPC delimita una invocación. Hay una sola petición, un solo plazo *(deadline)* para toda la operación y un único código de estado final, que llega en las cabeceras al cerrarse el flujo. Pero cada mensaje se entrega al receptor en cuanto el emisor lo produce, sin esperar a los demás. El retardo artificial de medio segundo de `factorials()` evidencia ese comportamiento. El cliente imprime cada resultado delfactorial conforme lo recibe, no todos de golpe al terminar.

En la modalidad bidireccional los dos flujos son independientes. El servidor no está obligado a consumir todos los mensajes antes de empezar a responder, ni a emitir uno por cada uno que recibe. La función `factorial_each()` implementa la alternancia más simple —un resultado por cada argumento—

, pero igualmente podría agrupar varios, descartar algunos o emitir mensajes sin haber recibido nada.

23.5.4. JSON-RPC

JSON-RPC es un protocolo RPC cuyos mensajes son objetos JSON. A pesar de su sencillez, o quizá precisamente por eso, tiene bastante aceptación. Sobre él se construyen, por ejemplo, MCP (Model Context Protocol), con el que las aplicaciones basadas en LLM descubren e interaccionan con otros programas, y LSP, con el que los editores de código solicitan a un servidor especializado en un lenguaje de programación cosas como completar identificadores o marcar errores del código. También lo emplean las API de los nodos de la red Ethereum. En MCP y LSP ambos extremos emiten peticiones.

La especificación de su versión 2.0 [66] es un documento breve, que se limita a definir el formato de los mensajes y la correspondencia entre cada respuesta y su petición. No existe un IDL, un compilador ni *stubs*, ni tampoco un *runtime* de referencia. La especificación tampoco establece el transporte ni el modo de delimitar los mensajes sobre él, de manera que un mensaje puede transportarse en el cuerpo de una petición HTTP, en una línea de texto sobre TCP o precedido de una cabecera que indique su longitud, como hace LSP. Adopta el mismo planteamiento que XML-RPC, un protocolo anterior, con JSON en lugar de XML.

La invocación `subtract(42, 23)` y una invocación a un método inexistente dan lugar al siguiente intercambio, expresado en la notación de la propia especificación, en la que `-->` señala el mensaje que envía el cliente y `<--` el que devuelve el servidor:

```
--> {"jsonrpc": "2.0", "method": "subtract", "params": [42, 23], "id": 1}
<-- {"jsonrpc": "2.0", "result": 19, "id": 1}
--> {"jsonrpc": "2.0", "method": "foobar", "params": [42], "id": 2}
<-- {"jsonrpc": "2.0", "error": {"code": -32601, "message": "Method not found"}, "id": 2}
```

Una petición es un objeto con cuatro miembros. `jsonrpc` indica la versión (siempre la cadena `2.0`), `method` es el nombre de la operación, `params` contiene los argumentos, ya sea por posición (un *array*) o por nombre (un objeto), e `id` identifica la petición. La respuesta incluye el mismo `id` y, además, un miembro `result` con el resultado o un miembro `error` con un código numérico y un mensaje. Ese `id` corresponde al identificador de mensaje de § 23.1, y permite al cliente mantener varias peticiones en curso sobre la misma conexión y asociar cada respuesta a la petición correspondiente. El identificador de operación, en cambio, es el nombre del método, es decir, una cadena, y no un número como en pico-rpc o en ONC RPC.

Una petición que carece de `id` es una *notificación*. El servidor la ejecuta sin enviar respuesta alguna, ni siquiera en caso de error, por lo que corresponde al estilo R y el cliente no puede saber si la operación llegó a ejecutarse. Asimismo, es posible agrupar varias peticiones en un *array*, que constituye un *lote (batch)*. El servidor responde con otro *array* que contiene las respuestas a todas las peticiones salvo a las notificaciones, en cualquier orden, puesto que cada una incluye su `id`. Un lote reduce el número de tiempos de ida y vuelta cuando las peticiones son independientes entre sí.

El ejemplo /`jsonrpc-factorial` utiliza la biblioteca `jsonrpclib` (`python3-jsonrpclib-pelix`). Ofrece la operación `factorial()` de los ejemplos anteriores y otra, `log()`, que imprime un texto en la consola del servidor y sirve para ilustrar las notificaciones. Cada mensaje se envía en el cuerpo de una petición `POST` de HTTP, siempre a la misma URI, de modo que, igual que en gRPC, HTTP actúa únicamente como transporte.

El servidor registra cada función con `register_function()`, que la asocia a su nombre, y para despachar una petición busca ese nombre en un diccionario, del mismo modo que la tabla `STUBS` de pico-rpc. No se requiere un *stub* de servidor, ya que el módulo `json` entrega los argumentos como valores nativos de Python. El servidor, en cambio, debe validar todo lo que recibe, porque nada garantiza que la petición tenga la forma esperada. La especificación reserva un código de error para cada caso, *p. ej.* `-32601` si el método no existe o `-32602` si los argumentos no son válidos.

Tampoco el cliente dispone de *stubs*. La clase `ServerProxy` construye cada petición a partir del nombre del método invocado, de modo que `server.factorial(5)` se convierte en una invocación remota sin que la operación se haya declarado previamente en ninguna interfaz. La contrapartida de esa flexibilidad es que los errores que se detectarían al generar los *stubs* a partir de una interfaz solo se manifiestan en tiempo de ejecución, y es el servidor quien los detecta.

Puesto que los mensajes son texto, cualquier cliente HTTP, como `curl`, puede efectuar invocaciones:

```
$ curl localhost:2000 \
  -d '{"jsonrpc": "2.0", "method": "factorial", "params": [5], "id": 1}'
{"result": 120, "id": 1, "jsonrpc": "2.0"}
$ curl localhost:2000 \
  -d '{"jsonrpc": "2.0", "method": "factorial", "params": {"n": 20}, "id": 2}'
{"result": 2432902008176640000, "id": 2, "jsonrpc": "2.0"}
$ curl localhost:2000 \
  -d '{"jsonrpc": "2.0", "method": "log", "params": ["hello"]}'
$ curl localhost:2000 -d '[
  {"jsonrpc": "2.0", "method": "factorial", "params": [3], "id": 3},
  {"jsonrpc": "2.0", "method": "power", "params": [2, 8], "id": 4}]'
```

```
[{"result": 6, "id": 3, "jsonrpc": "2.0"},
 {"id": 4, "jsonrpc": "2.0",
  "error": {"code": -32601, "message": "Method power not supported."}}]
```

La segunda invocación proporciona el argumento por nombre. La tercera es una notificación, de modo que el servidor contesta a la petición HTTP, como exige ese protocolo, pero sin ningún mensaje JSON-RPC en el cuerpo. La cuarta es un lote de dos peticiones, la segunda de ellas dirigida a un método inexistente.

JSON no distingue entre enteros y reales, y muchas implementaciones representan los números como reales de doble precisión, así que un entero mayor que 2^{53} puede llegar redondeado [67]. En gRPC, en cambio, el tipo `int64` de la interfaz produce un error si el valor no cabe, en lugar de entregar uno incorrecto.

23.6. RMI

RMI traslada el mecanismo de invocación remota al modelo de objetos. La arquitectura es la misma que en RPC—interfaz, compilador, *stubs* y *runtime*—, pero el destino de una invocación no es un procedimiento del servidor, sino un método de un objeto concreto. Esa diferencia, aparentemente pequeña, tiene varias consecuencias.

En RPC, el servidor ofrece un conjunto plano de procedimientos, y toda invocación de una operación llega a la misma implementación. Si el servicio gestiona varias entidades independientes —varios ficheros, varias cuentas o varios contadores— el procedimiento no tiene forma de distinguirlas por sí mismo; el cliente tiene que incluir en cada invocación un argumento adicional que identifique la entidad, y el servidor tiene que mantener una tabla que asocie ese identificador con su estado. Eso es lo que hace NFS, por ejemplo, con los manejadores de fichero.

En RMI, en cambio, cada objeto remoto encapsula su propio estado, y el cliente invoca los métodos directamente sobre el objeto que le interesa. Un mismo servidor puede alojar muchos objetos que implementan la misma interfaz: dos contadores remotos llevan cuentas independientes aunque compartan servidor, tipo y código, igual que dos instancias de una misma clase en un programa convencional.

Para lograrlo, cada objeto remoto tiene una **referencia a objeto remoto**, que lo identifica de forma única en todo el sistema. De un modo u otro, esta referencia remota incluye el *endpoint* del servidor que lo aloja y una *identidad* que identifica el objeto. En el cliente, la referencia se materializa en un *proxy*, que es parte del *stub* de cliente y está ligado a un objeto concreto de

ese servidor. La invocación, por tanto, requiere dos niveles de despachado[2] en el servidor. Primero un componente del *runtime*, habitualmente llamado *adaptador de objetos*, localiza el objeto a partir de su identidad. Después, el *stub* de servidor de ese objeto localiza el método a partir del identificador de operación. El objeto que implementa la interfaz en el servidor se suele denominar *sirviente (servant)*.

La ventaja más importante de este modelo es que las referencias a objetos remotos son «ciudadanos de primera clase» *(first-class citizens)*, es decir, valores que se pueden pasar como argumento o devolver como resultado de una invocación, igual que una referencia a un objeto convencional. Eso tiene varias consecuencias:

- El conjunto de objetos accesibles no está fijado de antemano. Un servidor puede crear objetos nuevos durante la ejecución y devolver sus referencias al cliente, *p. ej.* un objeto «factoría» cuyo método crea una sesión y devuelve la referencia al nuevo objeto.
- Los parámetros admiten dos modos de paso. Los objetos remotos se pasan por referencia: el receptor obtiene un proxy y sus invocaciones llegan al objeto original. El resto de valores (números, cadenas, estructuras) se pasan por valor: se serializan y el receptor obtiene una copia. En RPC solo existe el paso por valor.
- El cliente puede pasar al servidor la referencia de un objeto propio para que el servidor lo invoque más tarde, *p. ej.* para notificarle un evento. Esto es lo que llamamos *callback*, y es una de las situaciones en las que los roles de cliente y servidor de la invocación se invierten para una interacción concreta.

Por lo demás, las interfaces de RMI heredan las características del modelo de objetos del lenguaje: pueden extender otras interfaces (herencia de interfaz), y un proxy de una interfaz derivada se puede usar allí donde se espera una base *(upcasting)*. Los errores del servidor se propagan como excepciones, que el *stub* de cliente vuelve a lanzar en el cliente.

Todo esto por supuesto tiene un coste. La vida de un objeto remoto debería terminar cuando ningún cliente lo necesita, pero el servidor no puede saber cuándo han dejado de existir los *proxies* que lo referencian en otros nodos. Algunos middlewares incorporan para eso un recolector de basura distribuido; otros dejan esa responsabilidad al programador, que tiene que destruir explícitamente los objetos que ya no necesita. Además, la facilidad para crear y repartir referencias invita a diseños con muchas invocaciones

[2] también llamado «indirección»

pequeñas, en las que la latencia de la red —que el modelo de objetos no oculta— acaba suponiendo una sobrecarga no trivial.

23.6.1. pico-rmi

Pico-rmi es la versión orientada a objetos de pico-rpc (§ 23.5.1). Mantiene la misma estructura —*runtime* (`picormi.py`), compilador (`rmigen.py`), definición de la interfaz en Python y serialización con `struct`— y solo cambia lo necesario para que un servidor pueda alojar varios objetos independientes.

La interfaz `Counter`, del ejemplo /`pico-rmi`, declara dos operaciones sin parámetros: `increment()` y `get()`. La mayor diferencia respecto a pico-rpc no está en la interfaz, sino en que ahora puede haber varios objetos que la implementan.

Cliente

El *stub* del cliente para un objeto remoto concreto se llama *proxy*, y para instanciarlo se requiere, además del *endpoint* del servidor, la identidad del objeto. Esta combinación es la «referencia de objeto remoto» o simplemente «referencia remota».

```
from counter_client_stub import CounterProxy

if len(sys.argv) != 3:
    print("usage: ./client <server> <identity>")
    exit(1)

server = sys.argv[1]
identity = sys.argv[2]

counter = CounterProxy(identity, server, 2001)
print("get() = '{}'".format(counter.get()))
print("increment() = '{}'".format(counter.increment()))
print("increment() = '{}'".format(counter.increment()))
print("get() = '{}'".format(counter.get()))
```

Listado 23.15: Cliente del ejemplo `Counter` de pico-rmi
/`pico-rmi/client.py`

El mensaje de petición incluye en este caso la identidad del objeto (16 bytes) seguida del identificador de operación (1 byte), y los argumentos si los hubiera.

```
request = struct.pack('16sB', self.identity, self.INCREMENT)
```

Listado 23.16: Mensaje de petición pico-rmi generado en el *stub* del cliente

Servidor

En pico-rpc el *stub* de servidor es un módulo con funciones sueltas que reciben la implementación como argumento. En pico-rmi el compilador genera una clase (`Counter`) que hereda de `Servant`, con un método *stub* por cada operación y la misma tabla `STUBS`. El sirviente (`CounterI`) hereda de esa clase, así que cada *stub* invoca la operación sobre la propia instancia (`self`) y, por tanto, sobre su estado.

El *dispatcher* de pico-rpc se sustituye por el «adaptador de objetos» según la terminología habitual de RMI, que mantiene un diccionario de sirvientes indexado por la identidad del objeto (Listado 23.17). Al recibir una petición, obtiene la identidad y localiza el sirviente, que a su vez localiza el *stub* a partir del identificador de operación. Estos son los dos niveles de despachado descritos al principio de esta sección.

```
class Servant:
    STUBS: dict[int, Callable] = {}

    def dispatch(self, function_id, args):
        return self.STUBS[function_id](self, args)

class ObjectAdapter:
    def __init__(self, port):
        self.port = port
        self.servants = {}

    def add(self, identity, servant):
        self.servants[identity] = servant

    def dispatch(self, sock):
        request = sock.recv(MAX_MESSAGE_SIZE)
        identity = decode_identity(request[:IDENTITY_SIZE])
        function_id, args = request[IDENTITY_SIZE], request[IDENTITY_SIZE + 1:]

        servant = self.servants[identity]
        result = servant.dispatch(function_id, args)
        sock.sendall(result)
```

LISTADO 23.17: Sirviente base y adaptador de objetos de *runtime* de pico-rmi
/pico-rmi/picormi.py

El servidor crea dos sirvientes y los registra en el adaptador con las identidades `counter1` y `counter2` (Listado 23.18).

```
from picormi import ObjectAdapter
from counter_server_stub import Counter

class CounterI(Counter):
    def __init__(self):
        self.value = 0
```

```
    def increment(self):
        self.value += 1
        return self.value

    def get(self):
        return self.value

adapter = ObjectAdapter(port=2001)
adapter.add('counter1', CounterI())
adapter.add('counter2', CounterI())
adapter.run()
```

LISTADO 23.18: Servidor del servicio `Counter`
`pico-rmi/server.py`

Cada ejecución del cliente invoca `get()`, dos veces `increment()` y de nuevo `get()` sobre el objeto cuya identidad recibe como argumento. Los dos contadores llevan cuentas independientes, aunque estén en el mismo servidor y compartan la misma interfaz. En pico-rpc eso habría requerido añadir un parámetro que identificara sobre qué contador debe aplicar la operación.

Cliente

```
$ ./client.py 127.0.0.1 counter1
get() = '0'
increment() = '1'
increment() = '2'
get() = '2'
$ ./client.py 127.0.0.1 counter1
get() = '2'
increment() = '3'
increment() = '4'
get() = '4'
$ ./client.py 127.0.0.1 counter2
get() = '0'
increment() = '1'
increment() = '2'
get() = '2'
```

Servidor

```
$ ./server.py
Object adapter ready: ('0.0.0.0', 2001)
```

FIGURA 23.3: Cliente y servidor pico-rmi

Pico-rmi aplica todas las simplificaciones de pico-rpc, y añade otras propias del modelo de objetos. Los sirvientes se registran al arrancar el servidor y no se pueden crear ni destruir después. Tampoco es posible pasar una referencia de objeto remoto como argumento o como valor de retorno, porque solo admite enteros sin signo. Por último, si la identidad de la petición no corresponde a ningún sirviente, el adaptador no informa al cliente del error: la excepción termina el servidor.

23.6.2. Java RMI

Java RMI es el middleware de invocación remota que Sun incorporó a la plataforma Java en 1997. No dispone de un IDL ni de compilador de interfaces. La interfaz remota es una interfaz Java convencional, y el *stub* es un *proxy* dinámico que el *runtime* construye mediante reflexión. Eso simplifica mucho su uso, pero cliente y servidor han de estar escritos necesariamente en Java (o, al menos, deben ejecutarse en una JVM). Utiliza su propio protocolo llamado JRMP encapsulado sobre TCP y serializa los datos con la serialización nativa de Java. La semántica de invocación es *at-most-once*.

El ejemplo /`java-rmi` es el «hola mundo» de la documentación oficial de Java. Una interfaz remota (Listado 23.19) debe cumplir dos requisitos : extender *java.rmi.Remote*, una interfaz vacía que solo sirve para marcar el tipo como remoto, y declarar que todos sus métodos pueden lanzar `RemoteException`. Esta es una decisión de diseño deliberada; los fallos de comunicación no se deben ocultar al programador, por eso el compilador obliga al programador a tratarlo en cada invocación.

```
public interface Hello extends Remote {
    String sayHello() throws RemoteException;
}
```

LISTADO 23.19: Interfaz remota *Hello* en Java RMI
/`java-rmi/Hello.java`

El sirviente es una clase que implementa esa interfaz remota declarada (Listado 23.20). El método `exportObject()` de `UnicastRemoteObject` lo registra en el *runtime*, que queda escuchando en el puerto TCP indicado (5000), y devuelve el *stub* de cliente para ese objeto. El servidor Java RMI y el sirviente quedan definidos en este caso en una única clase `Server`.

El *runtime* mantiene activos sus hilos mientras haya objetos exportados, así que el proceso servidor no termina al salir de `main()`.

```
public class Server implements Hello {
    public String sayHello() {
        return "Hello, world!";
    }

    public static void main(String args[]) {
        try {
            Server obj = new Server();
            Hello stub = (Hello) UnicastRemoteObject.exportObject(obj, 5000);

            // Bind the remote object's stub in the registry
            Registry registry = LocateRegistry.getRegistry();
            registry.rebind("Hello", stub);
```

```
            System.err.println("Server ready");

        } catch (Exception e) {
            System.err.println("Server exception: " + e.toString());
            e.printStackTrace();
        }
    }
}
```

LISTADO 23.20: Servidor del ejemplo `Hello` en Java RMI
/java-rmi/Server.java

23.6.2.1. Registry

Para que el cliente pueda obtener ese *stub*, el servidor lo publica en el Registry con el nombre `Hello`. El Registry es un servicio de nombres muy sencillo que se ejecuta con el programa `rmiregistry`[3] y escucha por defecto en el puerto 1099. El propio Registry es un objeto remoto, con identidad fija y conocida. Por eso, `LocateRegistry.getRegistry()` no necesita comunicarse con nada; construye localmente un *stub* a partir del nombre del nodo, el puerto 1099 y esa identidad fija. Así se resuelve el problema de obtener la primera referencia remota. Sin ella no se puede invocar nada.

La invocación `registry.rebind("Hello", stub)` es a su vez una invocación remota sobre el Registry, en la que el *stub* se pasa como argumento. Ese *stub* contiene la dirección IP y el puerto del servidor y la identidad del objeto. La dirección se obtiene resolviendo el nombre del nodo del servidor; si resuelve a una dirección de *loopback*, la referencia no sirve a los clientes de otros nodos. La propiedad `java.rmi.server.hostname` permite indicar la dirección correcta.

El cliente (Listado 23.21) obtiene una copia de ese *stub* con `lookup()`, y a partir de ese momento invoca `sayHello()` directamente sobre el servidor, en el puerto 5000; el Registry no interviene en las invocaciones.

```
public class Client {
    private Client() {}

    public static void main(String[] args) {
        String host = (args.length < 1) ? null : args[0];
        try {
            Registry registry = LocateRegistry.getRegistry(host);
            Hello stub = (Hello) registry.lookup("Hello");
            String response = stub.sayHello();
            System.out.println("response: " + response);
```

[3]El Registry necesita acceder a la clase *Hello* para deserializar el *stub* que recibe, así que `rmiregistry` se debe lanzar desde el directorio que contiene las clases compiladas. Otra opción es que el propio servidor cree el Registry en su proceso con `LocateRegistry.createRegistry()`.

```
        } catch (Exception e) {
            System.err.println("Client exception: " + e.toString());
            e.printStackTrace();
        }
    }
}
```

LISTADO 23.21: Cliente del ejemplo `Hello` en Java RMI
/java-rmi/Client.java

Por seguridad, el Registry solo acepta las operaciones de escritura —`bind()`, `rebind()` y `unbind()`— cuando proceden del mismo nodo. La consulta (`lookup`), en cambio, se puede invocar desde cualquier nodo.

23.6.2.2. Paso por valor y por referencia

Los objetos que implementan *Remote* y han sido exportados se pasan **por referencia**: al serializar un argumento o un resultado, el *runtime* los sustituye por su *stub*, como ocurre con el que el servidor entrega al Registry. El resto se pasan **por valor**, para lo que su clase debe implementar *java.io.Serializable*. Java RMI utiliza la serialización nativa de Java que, a diferencia de XDR o Protocol Buffers, no se limita a tipos declarados en una interfaz: serializa el grafo completo de objetos alcanzables desde el argumento, conservando su estructura.

23.6.2.3. Recolección de basura distribuida

Java RMI incorpora un recolector de basura distribuido que se basa en el conteo de referencias remotas. Cuando un proceso deserializa un *stub*, su *runtime* informa al servidor que aloja el objeto mediante una invocación `dirty()` sobre el DGC de ese servidor (otro objeto remoto con identidad fija), y cuando el recolector local elimina el último *proxy*, lo notifica con `clean()`.

Como el cliente puede terminar sin avisar, cada referencia se concede como un «préstamo» *(lease)* con una duración limitada, 10 minutos por defecto (propiedad `java.rmi.dgc.leaseValue`), que el cliente debe renovar periódicamente. Si el *lease* expira, el servidor decrementa el conteo. Cuando llega a cero, el *runtime* deja de mantener el objeto y el recolector local de la JVM del servidor lo elimina. Si el sirviente implementa la interfaz *Unreferenced*, se le notifica. Como cualquier esquema de conteo de referencias, no es capaz de recolectar ciclos de objetos que se referencian entre sí desde distintos nodos.

En el ejemplo, la única referencia local al sirviente es la variable `obj` de `main()`, que deja de existir cuando esta termina. El objeto sigue vivo porque el Registry mantiene una referencia remota —el *stub*— y renueva su *lease*.

23.6.2.4. Seguridad

JRMP no cifra ni autentica las comunicaciones. Para hacerlo, `exportObject()` admite una variante que recibe las factorías de *sockets* del cliente y del servidor; las clases `SslRMIClientSocketFactory` y `SslRMIServerSocketFactory` permiten usar TLS.

Las primeras versiones de Java RMI permitían que el receptor descargara desde una URL indicada por el emisor *(codebase)* las clases que le faltaban. Las versiones actuales ya no lo permiten, así que ambos extremos necesitan tenerlas en su *classpath*.

Hay otro problema de seguridad más grave. La deserialización de objetos arbitrarios recibidos de la red es un vector de ataque para introducir código malicioso. Un atacante puede enviar un grafo de objetos, construido con clases presentes en el *classpath* del servidor, cuya deserialización acaba ejecutando código arbitrario. Desde Java 9, la plataforma incluye filtros de deserialización que limitan qué clases se aceptan. Aun así, exponer un servicio Java RMI en una red no confiable se considera una mala práctica.

23.6.3. ZeroC Ice

Ice es un middleware RMI desarrollado por la empresa ZeroC, cuya primera versión apareció en 2003. Sus autores procedían del mundo de CORBA y lo concibieron como un sustituto que conservara el modelo de objetos distribuidos, pero con una especificación mas sencilla y un *runtime* más ligero. Genera *stubs* para muchos lenguajes de modo que, igual que en gRPC, cliente y servidor pueden estar escritos en lenguajes distintos. Utiliza un protocolo de aplicación propio llamado IceP, independiente del transporte. Sus mensajes se encapsulan sobre el que indique el *endpoint*.

Las piezas son las habituales de cualquier middleware RMI (§ 23.6): un IDL, un compilador que genera los *stubs* de cliente y servidor, un *runtime* con su adaptador de objetos y una referencia de objeto remoto que el cliente materializa en un proxy.

El ejemplo /`py/counter-by-id` es el servicio `Counter` de pico-rmi (§ 23.6.1) reescrito para Ice: un servidor aloja dos contadores independientes y un cliente invoca sus operaciones.

La interfaz se describe en el lenguaje Slice —el IDL de Ice— (Listado 23.22). Permite declarar módulos, interfaces, estructuras, secuencias, diccionarios, enumerados y excepciones; las interfaces admiten herencia, incluso múltiple.

```
module Example {
    interface Counter {
        long increment();
        idempotent long get();
    };
};
```

Listado 23.22: Interfaz *Counter* en Slice
/py/counter-by-id/counter.ice

Una peculiaridad de Slice es que es posible declarar qué operaciones son idempotentes (`idempotent`). Sabiendo esto, el *runtime* del cliente puede decidir cómo gestionar un fallo. Una operación marcada así se puede reintentar sin romper la semántica *at-most-once* (§ 23.3), algo que no se puede aplicar a un método no idempotente. En `Counter`, `get()` es idempotente e `increment()` no lo es.

Cliente y servidor conocen esa marca porque sus *stubs* se han generado desde la misma interfaz, así que ninguno de los dos necesita que se la indiquen. Aun así, el «modo» de la operación se incluye en cada petición. Está ahí para componentes que no tienen la interfaz, *p. ej.* un intermediario que reenvía invocaciones desconocidas, como el servicio de propagación de eventos IceStorm.

El compilador `slice2py` (`zeroc-ice-compilers`) genera el módulo `counter_ice.py` y un paquete `Example` con el proxy de cliente (`CounterPrx`) y el *stub* de servidor (`Counter`). Pero como Python es un lenguaje dinámico, ese paso se puede evitar. La función `Ice.loadSlice()` compila el fichero `.ice` al vuelo durante la ejecución y construye los tipos en memoria.

El sirviente (`CounterI`) hereda del *stub* de servidor e implementa las dos operaciones (Listado 23.23). Cada método recibe un parámetro que no aparece en la interfaz, `current`, que permite al sirviente acceder a los datos de la propia invocación: la identidad del objeto invocado, el nombre de la operación, el adaptador, o la conexión entre otros. Con ese parámetro, un único sirviente puede encargarse de muchos objetos distintos y distinguirlos por su identidad. Eso es lo que se llama «sirviente por defecto» *(default servant)*, y evita crear un sirviente por cada entidad cuando son miles o millones.

```
class CounterI(Example.Counter):
    def __init__(self):
        self.value = 0

    def increment(self, current):
        self.value += 1
        return self.value

    def get(self, current):
        return self.value

def main(ic):
    adapter = ic.createObjectAdapter('CounterAdapter')

    proxy1 = adapter.add(CounterI(), Ice.stringToIdentity('counter1'))
    proxy2 = adapter.add(CounterI(), Ice.stringToIdentity('counter2'))

    print(proxy1)
    print(proxy2)

    adapter.activate()
    ic.waitForShutdown()
    return 0
```

LISTADO 23.23: Servidor del servicio `Counter` en ICE
/py/counter-by-id/server.py

La función `main()` recibe en `ic` el *comunicador (communicator)*, el objeto principal del *runtime* de ICE, a partir del cual se crean los adaptadores de objetos y los proxies. El programa lo obtiene al arrancar con `Ice.initialize(sys.argv)`, que además extrae de los argumentos de la línea de comandos las propiedades de configuración del *runtime*.

El servidor registra cada sirviente en el adaptador de objetos con `adapter.add()` bajo una identidad única, que se puede construir a partir d euna cadena con `Ice.stringToIdentity()`. Ahí están los dos niveles de despachado: el adaptador localiza el sirviente a partir de la identidad que viene en la petición, y el *stub* del servidor localiza el método a partir del identificador de la operación. Es exactamente el mismo papel del `ObjectAdapter` de pico-rmi, pero el *endpoint* en el que escucha no se le pasa como argumento, sino que lo toma de una propiedad de configuración:

```
CounterAdapter.Endpoints=tcp -h 127.0.0.1 -p 2002
```

Prácticamente todo el comportamiento del *runtime* se ajusta con propiedades como esta, que se leen del fichero indicado en `--Ice.Config` o bien de la línea de comandos. El protocolo de transporte por defecto, el tamaño máximo de mensaje, los plazos de espera, la cantidad de reintentos, el tamaño del *pool* de hilos, las trazas de red y muchas otras características son configurables con propiedades (hay cientos de ellas). El código no cambia

al pasar de TCP a TLS, ni al mover el servidor a otro puerto, solo hay que ajustar las propiedades.

23.6.3.1. Referencias remotas textuales

El método `adapter.add()` devuelve el proxy del objeto recién registrado, e ICE le da una representación textual completa, lo que su documentación llama *stringified proxy*. Eso es precisamente lo que imprime el servidor del ejemplo:

```
counter1 -t -e 1.1:tcp -h 127.0.0.1 -p 2002 -t 60000
```

Esa cadena es una referencia de objeto remoto (§ 23.6) representada como texto, y contiene los dos elementos que la definen: la identidad del objeto (`counter1`), y el *endpoint* del servidor que lo aloja (`tcp -h 127.0.0.1 -p 2002`), después. Entre ambos hay varias opciones de la referencia: `-t` es el modo de invocación (*twoway*), `-e 1.1` la versión del formato de serialización y `-t 60000` el *timeout* de invocación en milisegundos.

En realidad cuando se utiliza `print()` con un proxy, se está invocando `proxyToString()`. Y también existe la función inversa `stringToProxy()`, que construye un proxy que el cliente puede invocar.

Que la referencia remota se pueda transportar como texto resuelve el problema de «la primera referencia», el mismo que Java RMI resuelve con el Registry (§ 23.6.2): el cliente la obtiene por cualquier medio —un fichero de configuración, un argumento en la línea de comandos, un mensaje de correo— sin necesidad de un servicio de nombres. También hace evidente algo que otros middlewares ocultan: una referencia remota puede llevar varios *endpoints*, incluso con protocolos distintos (`tcp`, `ssl`, `udp`), y el *runtime* prueba cada uno de ellos hasta conseguir establecer la comunicación. Si el adaptador del ejemplo no fijara la dirección con `-h`, escucharía en todas las del nodo y la referencia incluiría un *endpoint* por cada una.

23.6.3.2. Reflexión remota

El método `stringToProxy()` solo considera la cadena que se le pasa (el proxy textual); no comprueba que el objeto exista ni que sea del tipo esperado. Devuelve un proxy de tipo `Ice::ObjectPrx`, la clase base de todos los proxies. Para invocar un método de la interfaz declarada —`increment()` en este ejemplo— es necesario que el tipo de este proxy sea del tipo específico (`CounterPrx`). De eso se encarga `checkedCast` (Listado 23.24).

```
def main(ic):
    proxy = ic.stringToProxy(sys.argv[1])
    counter = Example.CounterPrx.checkedCast(proxy)

    if counter is None:
        raise RuntimeError('invalid proxy')

    print("get() = '{}'".format(counter.get()))
    print("increment() = '{}'".format(counter.increment()))
    print("increment() = '{}'".format(counter.increment()))
    print("get() = '{}'".format(counter.get()))
```

LISTADO 23.24: Cliente del servicio `Counter` en ICE
/py/counter-by-id/client.py

Convertir una referencia de un tipo base a uno derivado es un **molde hacia abajo** *(downcasting)*, una operación habitual en cualquier lenguaje orientado a objetos y que no es nada particular de ICE o RMI. Lo que sí cambia es quién puede comprobarlo. En local, el objeto está en memoria y el lenguaje puede hacerlo, pero con RMI el único que sabe de qué tipo es el objeto es el servidor. Por eso, `checkedCast()` en realidad implica una invocación remota que interroga al objeto para verificar su tipo. Hay una variante `uncheckedCast()` que construye el proxy, pero no comprueba, algo que lógicamente implica un riesgo de fallo, pero hay situaciones en las que simplemente no es posible hacer la comprobación.

Preguntarle a un objeto por su propio tipo es un mecanismo de **reflexión**, «remota» en este caso. Toda interfaz SLICE hereda implícitamente de *Ice::Object*, que aporta a cualquier objeto remoto las operaciones necesarias para preguntarle sobre sí mismo. Su alcance, eso sí, es corto: el objeto revela qué interfaces implementa (método `ice_ids`), pero no qué operaciones tiene cada una ni cómo se serializan sus argumentos. ICE no ofrece un repositorio de interfaces que se pueda consultar en ejecución, como sí tenía CORBA.

En el ejemplo `Counter`, cada ejecución del cliente invoca `get()`, dos veces `increment()` y de nuevo `get()` sobre el objeto de la referencia recibida como argumento. Igual que en pico-rmi, los dos contadores llevan cuentas independientes aunque compartan servidor, adaptador e interfaz.

23.6.3.3. Paso de referencias y ciclo de vida

En SLICE, el tipo de un proxy se escribe con un asterisco tras el nombre de la interfaz (`Counter*`), y se puede usar como parámetro o como resultado de una operación. Las referencias de objeto remoto son, por tanto, «ciudadanos de primera clase» (§ 23.6), con los dos modos de paso que eso implica. Los objetos remotos se envían por referencia, y el receptor construye con

Servidor

```
$ ./server.py --Ice.Config=server.config
counter1 -t -e 1.1:tcp -h 127.0.0.1 -p 2002 -t 60000
counter2 -t -e 1.1:tcp -h 127.0.0.1 -p 2002 -t 60000
```

Cliente

```
$ ./client.py 'counter1 -t:tcp -h 127.0.0.1 -p 2002'
get() = '0'
increment() = '1'
increment() = '2'
get() = '2'
$ ./client.py 'counter1 -t:tcp -h 127.0.0.1 -p 2002'
get() = '2'
increment() = '3'
increment() = '4'
get() = '4'
$ ./client.py 'counter2 -t:tcp -h 127.0.0.1 -p 2002'
get() = '0'
increment() = '1'
increment() = '2'
get() = '2'
```

FIGURA 23.4: Servidor y cliente ICE
/py/counter-by-id

ella un proxy cuyas invocaciones llegan al original; el resto de los tipos —números, cadenas, estructuras, secuencias, diccionarios— se envían por valor. La diferencia con Java RMI, que serializa el grafo completo de objetos alcanzables (§ 23.6.2), es que aquí solo se envía lo que declare la interfaz.

Donde ICE se aparta de Java RMI es en el ciclo de vida. ICE no incorpora recolector de basura distribuido. Es responsabilidad del programador. A cambio, el servidor no mantiene conteos de referencias ni *(leases)* que renovar, y un cliente que desaparece sin avisar no deja nada pendiente.

23.6.3.4. Modos de invocación

El modo de invocación no forma parte de la interfaz; es un atributo de la referencia remota, que el cliente puede cambiar en su proxy cuando necesite. Corresponde con el `-t` que aparece en el proxy textual, aunque el objeto `Ice::ObjectPrx` también tiene métodos para lograr lo mismo en ejecución. El modo determina el estilo de intercambio y, por tanto, la semántica de invocación que obtiene el cliente (Cuadro 23.2).

El modo *twoway* (por defecto) es el único que permite obtener respuestas. El cliente recibe el resultado o una excepción, y como TCP evita pérdidas y duplicados, y el *runtime* no reintenta por su cuenta una operación que no sea idempotente, la semántica es *at-most-once* (§ 23.4).

Opción	Modo	Intercambio	Semántica	Método `ObjectPrx`
`-t`	*twoway*	RR sobre TCP	*at-most-once*	`ice_twoway()`
`-o`	*oneway*	R sobre TCP	*maybe*	`ice_oneway()`
`-d`	*datagram*	R sobre UDP	*maybe*	`ice_datagram()`
`-O`	*batch oneway*	R sobre TCP	*maybe*	`ice_batchOneway()`
`-D`	*batch datagram*	R sobre UDP	*maybe*	`ice_batchDatagram()`

CUADRO 23.2: Modos de invocación en una referencia remota ICE

Los otros cuatro cubren solo el mensaje de petición. El *stub* de cliente lo entrega al transporte y devuelve el control inmediatamente, sin esperar. El cliente no puede saber si la operación llegó a ejecutarse. Incluso con TCP, que garantiza que el mensaje llega al extremo remoto, no es posible saber si el sirviente ejecutó correctamente la operación (fallo parcial). Por eso la semántica es *maybe* en los cuatro casos. Lo que los distingue es el coste y el riesgo: con `-d` y `-D` el mensaje puede perderse por el camino, mientras que las variantes «por lotes» *(batch)* no envían nada hasta que se vacía la cola, lo que amortiza la sobrecarga cuando las invocaciones son muchas y pequeñas, a cambio de perderlas todas juntas si hay un problema en la conexión.

Por coherencia, un proxy de solo petición únicamente admite operaciones sin resultado ni excepciones declaradas; invocar `get()` en esos casos produce un error local (`TwowayOnlyException`) antes de enviar nada. La misma interfaz sirve para los dos estilos de intercambio (§ 23.1) sin cambiar la definición, algo que por ejemplo en gRPC exigiría declararlo en el fichero `.proto`.

Existe una opción más en los proxies textuales: `-s`, que no es un modo sino un requisito: obliga al *runtime* a usar únicamente *endpoints* seguros, y por eso aparece combinada con la del modo (`-t -s`).

23.7. Localización de servicios

De uno u otro modo, cualquier referencia de objeto remoto incluye el *endpoint* del servidor que aloja al objeto (§ 23.6). Eso resuelve el problema de encontrarlo, pero crea otro: acopla el cliente a un `c`:oncreto. Así lo hacen todos los ejemplos de este capítulo: pico-rmi recibe el host como argumento, gRPC construye su canal con un *endpoint* fijo, el proxy de ICE lo lleva incluido, etc. Esto funciona mientras nada cambie, pero en un despliegue las cosas cambian. Los servidores cambian de nodo (se «migran»), se replican, o se reinician en otro puerto. También es útil poder arrancar los servidores solo cuando hacen falta («bajo demanda»).

La **transparencia de localización** ofrece a los clientes nombres alternativos para los servidores que no incluyen endpoints. Para lograr la comunicación obviamente los endpoints son necesarios, pero se puede aplazar el momento de averiguarlos hasta que se produzca la invocación. Para lograr esto, el middleware debe incluir un servicio que proporcione los endpoints —potencialmente efímeros— para los identicadores estables de los servicios. El componente que hace esto es el *binder*.

Todo *binder* tiene el problema del *bootstrapping*, es decir, ¿cómo se obtienen sus endpoints, si para eso hace falta precisamente el servicio que ofrece el *binder*? Es el mismo problema de los servidores DNS, y la solución también suele ser la misma. El *binder* (al igual que el servidor DNS) tiene un *endpoint* fijo, conocido y estable, ya sea por convención o por configuración. El *binder* de ONC-RPC es `rpcbind` (§ 23.5.2), que escucha siempre en el puerto 111, y el de Java RMI es el Registry (§ 23.6.2), en el 1099 y con una identidad fija. En ICE ese papel lo realiza un servicio llamado `Locator`, cuyo proxy se fija con la propiedad `Ice.Default.Locator`, que tiene este aspecto:

```
Ice.Default.Locator=IceGrid/Locator:tcp -h maine -p 4061
```

La alternativa al *binder* es el «descubrimiento de servicios». En lugar de preguntarle a un servicio conocido, se envía una consulta a la red con algún mecanismo de difusión —`rpcbind` e ICE lo tienen. El problema es que la difusión normalmente está localizada únicamente a la LAN o la subred.

23.7.1. Registro de servidores y objetos

Para que el *binder* pueda hacer su trabajo, los servidores tienen registrarse en él. En ONC-RPC el servidor registra en `rpcbind` cada combinación de programa, versión y protocolo junto al puerto que el sistema le ha asignado. La tabla resultante se consulta con `rpcinfo -p`. Con el servidor del ejemplo /`onc-rpc-hello` en marcha, sus dos últimas entradas corresponden al programa `600000000` que declara su interfaz, registrado por partida doble porque acepta los dos transportes:

```
$ rpcinfo -p
   program vers proto   port  service
    100000    4   tcp    111  portmapper
    100000    3   tcp    111  portmapper
    100000    2   tcp    111  portmapper
    100000    4   udp    111  portmapper
    100000    3   udp    111  portmapper
    100000    2   udp    111  portmapper
 600000000    1   udp  57269
 600000000    1   tcp  62357
```

Los puertos son aleatorios y cambian en cada arranque; eso es justamente por lo que hace falta un *binder*, y lo que permite por ejemplo que un servidor NFS no necesite un puerto fijo. Obsérva que `rpcbind` figura en su propia tabla (`100000 «portmapper`). El binder es un programa RPC más, igual que el Registry de Java RMI es un objeto remoto Java y el localizador de Ice un objeto Ice.

En Java RMI el registro es explícito y lo hace el programador con `rebind()`, que publica el *stub* bajo un nombre. El Registry *stub* completo, no solo el endpoint.

Ice lo automatiza por completo. El adaptador de objetos, al activarse, registra por su cuenta en el *binder* el «proxy directo» *(direct proxy)* —el que lleva *endpoints*, como los de § 23.6.3— en el que va a escuchar; el programador no tiene que escribir nada de código, solo le da al adaptador un identificador y configura un `Locator`, ambos por configuración (Listado 23.25). Obsérva que el adaptador ni siquiera fija el puerto (`tcp`), porque no hace falta: el *runtime* elige uno libre y es el *binder* quien lo almacena.

```
PrinterAdapter.Endpoints=tcp
PrinterAdapter.AdapterId=PrinterAdapter1
PrinterAdapter.Locator=IceGrid/Locator -t:tcp -h 127.0.0.1 -p 4061
```

Listado 23.25: Configuración del servidor con localizador
/locator/server.config

El servidor es exactamente el mismo del «hola mundo» (/py/hello/server.py). Lo único que cambia es lo que imprime: el proxy ya no lleva *endpoints*, sino el nombre del adaptador tras una `@`. Es un «proxy indirecto» *(indirect proxy)*.

```
$ ./server.py --Ice.Config=../locator/server.config
printer1 -t -e 1.1 @ PrinterAdapter1
```

23.7.2. Resolución de nombres

La resolución es transparente para el programador. En Java RMI la consulta al Registry es una línea de código que alguien tiene que escribir (`lookup()`), y ocurre una vez, antes de tener el proxy. En ONC-RPC, `clnt_create()` hace lo propio contra `rpcbind`. En Ice no hay ninguna llamada equivalente; el cliente construye el proxy y lo invoca, y es el *runtime* el que, al intentar establecer la comunicación, observa que la referencia no tiene *endpoints* y consulta al localizador antes de conectar. El cliente solo necesita saber dónde está el localizador (Listado 23.26) y usar como referencia la que imprimió el servidor. Los *endpoints* que el *runtime* obtiene quedan en una caché para evitar resoluciones adicionales innecesarias.

```
Ice.Default.Locator=IceGrid/Locator -t:tcp -h 127.0.0.1 -p 4061
```

Listado 23.26: Configuración del cliente con localizador
/locator/locator.config

```
$ ./client.py --Ice.Config=locator.config "printer1 -t -e 1.1 @ PrinterAdapter1"
```

Esa caché caduca por fallo, no por tiempo. Cuando el *runtime* no consigue conectar con los *endpoints* cacheados, descarta la entrada, vuelve a preguntarle al localizador y reintenta la invocación. Esa *re-resolución* es lo que da transparencia de verdad. Permite mover un servidor de puerto o de nodo mientras un cliente lo está usando, sin que reciba ningún error. Aunque solo oculta los fallos de conexión; si la petición ya salió y la operación no es `idempotent`, reintentar rompería la semántica *at-most-once* y el error sí llega al cliente.

La diferencia con los otros dos binders es evidente. El *stub* que devuelve el Registry de Java RMI lleva el *endpoint* y no se renueva nunca. Si el servidor cambia de puerto, ese *stub* queda inservible y el programador es quien tiene que repetir la llamada a `lookup()`. Lo mismo ocurre con `clnt_create()` en ONC-RPC, que consulta a `rpcbind` al crear el manejador y no vuelve a hacerlo. En ambos casos el *binder* resuelve el arranque; en Ice, además, vigila la validez de la referencia durante toda su vida. Cap'n Proto adopta un enfoque distinto, que renuncia al binder por diseño hasta el punto de considerarlo una importante debilidad (§ 23.9).

23.8. Invocación y despachado asíncrono

Las invocaciones que se han tratado hasta ahora son «síncronas», es decir, el hilo del cliente queda bloqueado desde que envía la petición hasta que llega la respuesta. Se hace así como analogía a la forma en que funcionan las llamadas locales convencionales. Sin embargo, en una invocación remota ese tiempo de espera —RTT más lo que tarde el servidor en ejecutar la operación— se pierde. Hay dos mecanismos para aprovecharlo, uno en cada extremo, y son independientes porque resuelven esperas distintas. La *invocación asíncrona*, de AMI (Asynchronous Method Invocation), permite al cliente seguir ejecutando código en lugar de esperar, y el *despachado asíncrono*, AMD (Asynchronous Method Dispatching), evita que se bloquee el servidor. Ninguno de los extremos puede saber cómo lo está haciendo el otro, porque el intercambio de mensajes es el mismo RR de siempre. Una invocación asíncrona sigue esperando una respuesta, la diferencia es que

el programa no se detiene a esperarla. No se debe confundir la invocación asíncrona con la invocación de solo petición, que no tiene respuesta.

Los ejemplos de esta sección utilizan Ice, que es el middleware más completo de los tratados en este aspecto.

23.8.1. Invocación asíncrona

En Ice la decisión es del cliente y no afecta a la interfaz. Para la operación `factorial()` de la interfaz *Math*, declarada como `long factorial(int value)`, el compilador genera siempre dos formas de invocarla en el proxy: `factorial()`, que bloquea, y `factorialAsync()`, que devuelve inmediatamente un *future*. Un *future* representa un resultado que todavía no está disponible.

El programa puede recoger el resultado de varias formas:

- Esperando más tarde: el cliente sigue con su trabajo y bloquea solo cuando necesita el valor. La llamada `result()` espera si la respuesta aún no ha llegado (Listado 23.27).
- Con un *callback*: el cliente registra una función que el *runtime* invoca cuando el *future* se completa, y no espera en ningún momento (Listado 23.28).
- En un bucle de eventos: `Ice.wrap_future()` convierte el *future* en uno de `asyncio`, de modo que se puede usar `await` (Listado 23.29).
- Consultando periódicamente si ya ha llegado el resultado *(polling)*, con `done()` (Listado 23.30).

```
future = math.factorialAsync(value)
print("That was an async call")
print(f"Async result is: {future.result()}")
```

Listado 23.27: Cliente que espera el resultado más tarde
/py/ami/client-block.py

```
def callback(future):
    try:
        print(f"Callback: value is: {future.result()}")
    except Exception as ex:
        print(f"Exception is: {ex}")

def main(ic):
    if len(sys.argv) != 3:
        print(f"usage: {sys.argv[0]} <proxy> <value>")
        return 1
```

```
    proxy = ic.stringToProxy(sys.argv[1])
    math = Example.MathPrx.checkedCast(proxy)
    value = int(sys.argv[2])

    if not math:
        raise RuntimeError('Invalid proxy')

    future = math.factorialAsync(value)
    future.add_done_callback(callback)

    print("That was an async call")

    return 0
```

Listado 23.28: Cliente con *callback*
/py/ami/client-callback.py

```
result = await Ice.wrap_future(math.factorialAsync(value))
print(f"Async result is: {result}")
```

Listado 23.29: Cliente integrado en un bucle de eventos
/py/ami/client-await.py

```
future = math.factorialAsync(value)
while not future.done():
    print("Still waiting...")
    time.sleep(0.1)
print(f"Async result is: {future.result()}")
```

Listado 23.30: Cliente que consulta periódicamente
/py/ami/client-poll.py

Que la invocación no bloquee tiene consecuencias. La excepción no se puede lanzar en el punto de la llamada, que ya ha terminado, sino que incluye en el *future* y se eleva al pedir el resultado; por eso el *callback* del ejemplo captura la excepción al leerlo. Además, el *callback* normalmente se ejecuta en un hilo del *runtime*, no en el que hizo la invocación, así que si comparte datos con el resto del programa hay que sincronizar el acceso. Y la semántica de invocación no cambia; una invocación asíncrona sigue siendo *at-most-once*, igual que la síncrona. Cambia cuándo la recibe el cliente, no qué garantías tiene.

23.8.2. Despachado asíncrono

En el servidor el problema es otro. El *runtime* atiende las peticiones con un *pool* de hilos limitado —por omisión, de un solo hilo, ajustable con la propiedad `Ice.ThreadPool.Server.Size`—, y el hilo que recibe una petición ejecuta el sirviente y bloquea hasta que retorna. Si la operación es lenta, los hilos del pool quedan ocupados. Unas pocas peticiones simultaneas pueden

agotar el *pool* e impedir que se atiendan las demás. El despachado asíncrono desacopla la ejecución de la operación del envío de la respuesta. Con AMD el sirviente devuelve el control al *runtime* sin tener el resultado, y este envía la respuesta más tarde, cuando el resultado esté disponible.

A diferencia de la invocación asíncrona, esta decisión sí aparece en la interfaz, porque cambia la firma que tiene el sirviente. En el Slice se declara con una anotación en la operación, `[.ªmd"] long factorial(int value)`. El sirviente del ejemplo (Listado 23.31) no calcula nada; crea un *future*, lo entrega con el argumento a una cola de trabajo que atiende otro hilo, y devuelve un *future*. La respuesta no sale al retornar el método, sino cuando se complete ese *future*. En el ejemplo lo hace el *job* al ejecutarse (Listado 23.32), con `set_result()`, o con `set_exception()` si tiene que cancelarse, para que el cliente reciba un error en lugar de esperar indefinidamente.

```
class MathI(Example.Math):
    def __init__(self, work_queue):
        self.work_queue = work_queue

    def factorial(self, value, current=None):
        future = Ice.Future()
        self.work_queue.add(future, value)
        return future
```

Listado 23.31: Sirviente con despachado asíncrono
/py/amd/server.py

```
class Job:
    def __init__(self, future, value):
        self.future = future
        self.value = value

    def execute(self):
        self.future.set_result(factorial(self.value))

    def cancel(self):
        self.future.set_exception(Example.RequestCanceledException())
```

Listado 23.32: Tarea *(job)* que completa el *future*
/py/amd/work_queue.py

El cliente que se utiliza con este servidor AMD es uno síncrono corriente, que invoca `factorial()` y espera el resultado. Este mecanismo no hace que el cálculo sea más rápido. El factorial es cálculo puro y la cola de trabajo no le aporta nada. La ventaja aparece cuando la operación espera, *p. ej.* una consulta a una base de datos o a otro servicio remoto, porque mientras espera no bloquea ningún hilo del *runtime*.

23.9. Cap'n Proto

La arquitectura de Cap'n Proto es la de cualquier RMI (§ 23.6): un IDL, un compilador, *stubs* y un *runtime*, con un modelo de objetos distribuidos. Sin embargo, su diseño cuestiona y, de hecho va contra, algunos de los pilares que rigen la mayoría de los otros middlewares: la necesidad de serializar los datos, el modelo de referencias remotas y la forma de obtenerlas.

Las siguientes secciones examinan cada una de ellas sobre el mismo ejemplo (/`capn-upper`), un servicio *TextProcessor* con una única operación, `upper()`, que devuelve una versión en mayúsculas del texto recibido. Existe en C++ y en Python, con el mismo fichero de interfaz (Listado 23.33). El fichero comienza con un identificador de 64 bits que lo identifica de forma inequívoca, y cada método lleva un número de orden (`@0`) que cumple el mismo papel que el número de campo de Protocol Buffers, es decir, permite ampliar la interfaz con métodos nuevos sin romper la compatibilidad.

```
@0x8b96e1f756a41043;

interface TextProcessor {
  upper @0 (message :Text) -> (result :Text);
}
```

Listado 23.33: Interfaz *TextProcessor* en Cap'n Proto
/`capn-upper/py/upper.capnp`

El compilador `capnp` (`capnproto`) genera el código de los *stubs* para C++. En Python no hace falta ese paso: el módulo `capnp` carga el fichero de interfaz directamente al importar `upper_capnp`, de un modo similar a `Ice.loadSlice()`.

23.9.1. ¿Sin serialización?

Enviar un argumento con XDR, Protocol Buffers o la serialización de Java implica convertirlo desde su representación en memoria a una secuencia de bytes, y el receptor debe hacer la conversión inversa. Cap'n Proto elimina ese paso: el mensaje se construye directamente en un *buffer* con la misma disposición que tendrá en el cable, y el receptor lo lee sin transformarlo. Esto es lo que se llama serialización *zero-copy*.

Para que esto sea viable, los tipos que genera el compilador no son objetos cuyos campos se rellenan al deserializar, sino accesores que operan sobre el *buffer*. En el cliente del ejemplo (Listado 23.35), el argumento `message` de `upper()` escribe el texto directamente en el mensaje que se va a enviar, y el resultado (`reply.result`) se lee del *buffer* tal como llegó.

Un mensaje que no necesita interpretarse para leerlo se puede leer en parte sin recorrer el resto, procesar antes de haberlo recibido completo, o guardar en un fichero y proyectar en memoria sin releerlo. Como contrapartida, ocupa más que uno con codificación compacta.

23.9.2. Referencias como *capabilities*

En Cap'n Proto las referencias a objetos remotos se llaman *capacidades (capabilities)*. Son la referencia de objeto remoto de § 23.6, pero con una propiedad añadida: designan un objeto y, además, le dan el permiso para invocarlo. En Java RMI y en Ice una referencia es un dato, una identidad más un *endpoint*, y con eso basta para poder invocar el objeto. En Cap'n Proto una capacidad no se puede fabricar ni se puede delegar a un tercero. Cuando un nodo envía una referencia a otro, el identificador únicamente es válido para esa conexión. Nadie que no lo haya recibido directamente puede invocar.

Colocar la referencia y el permiso juntos no es un mero detalle de implementación; es la respuesta a un problema de seguridad. Cuando una referencia se puede conseguir sin que nadie la haya concedido, el permiso hay que comprobarlo en otro lugar. El servidor debe averiguar quién es el cliente e implementar un mecanismos de autorización.

Ese enfoque conlleva dos inconvenientes:

- Cada cliente tiene permiso para acceder a todo aquello que pueda nombrar, y no solo de lo que necesita para su tarea. Viola el el principio de mínimo privilegio.
- El problema del «diputado confundido» *(confused deputy)*: un servicio con autoridad que actúa en nombre de un cliente, sobre un recurso cuyo nombre le indica el cliente, puede acabar usando esa autoridad sobre un recurso distinto para el que el cliente no tiene permiso.

Con capacidades, el cliente no le indica al servicio un nombre, sino que le entrega la referencia, que designa el recurso y prueba a la vez que tiene permiso para usarlo. La política de acceso deja de ser una tabla que mantener y pasa a ser una decisión de diseño de las interfaces. Es la misma idea que se aplica en un sistema de ficheros en un SO; para conceder a un usuario acceso de solo lectura a un fichero, se le entrega un objeto que no permite escribir —puede que ni exista la operación de escritura— y no hay nada que el cliente pueda hacer.

Esta idea también cambia la forma de obtener la primera referencia. No hay Registry, servicio de nombres ni referencia con identidad conocida. Al

establecer la conexión, cada extremo puede ofrecer un único objeto inicial, el *bootstrap*, que el otro obtiene como primera capacidad. En el servidor del ejemplo (Listado 23.34), es el argumento `bootstrap` de `TwoPartyServer`, y el cliente lo recoge con el método `bootstrap()` (Listado 23.35). La conexión es a la vez el canal y el nombre.

```
class TextProcessorI(upper_capnp.TextProcessor.Server):
    async def upper(self, message, _context):
        print("Client sent:", message)
        return message.upper()

async def new_connection(stream):
    await capnp.TwoPartyServer(stream, bootstrap=TextProcessorI()).on_disconnect()
```

LISTADO 23.34: Servidor de *TextProcessor* en Cap'n Proto
/capn-upper/py/server.py

```
async def main(host, port):
    connection = await capnp.AsyncIoStream.create_connection(
        host=host, port=port)

    client = capnp.TwoPartyClient(connection)
    text_processor = client.bootstrap().cast_as(upper_capnp.TextProcessor)

    reply = await text_processor.upper(message="hello, world!")
    print("Server reply:", reply.result)
```

LISTADO 23.35: Cliente de *TextProcessor* en Cap'n Proto
/capn-upper/py/client.py

La ausencia de un servicio de nombres es deliberada, no una carencia. Un servicio de nombres es justamente una fuente de referencias que los clientes pueden conseguir sin que nadie les haya concedido permiso expreso para usarlas, como el *Registry* de Java RMI o el *Locator* de ICE. Con Cap'n proto, los objetos se alcanzan navegando desde el objeto inicial, que entrega a cada cliente solo lo que debe ver; si una aplicación necesita algo parecido a un servicio de nombres, lo implementa como un objeto más. Esto no resuelve el problema de localización de § 23.7; el cliente sigue necesitando el *endpoint* del servidor al que se conecta.

Las capacidades están ligadas a la conexión, lo que simplifica el ciclo de vida sin necesidad de *leases*. Un objeto se destruye cuando se pierden todas sus referencias o la conexión por la que se recibieron. En el ejemplo, el servidor crea un sirviente por conexión (`new_connection()` en el listado), que desaparece con ella y obliga al cliente a reconectar. El protocolo se transporta sobre un flujo de bytes, como TCP en el ejemplo.

Este modelo tiene inconvenientes. Revocar una capacidad ya entregada es difícil, y responder a quién tiene acceso a un objeto exige reconstruir el grafo de referencias repartidas, sin una tabla central que consultar. Una caída de conexión invalida todas sus capacidades, y el cliente debe reconectar y reconstruir los objetos que usaba. Para guardar una capacidad y restaurarla más tarde en otra conexión hay que convertirla en un *token* persistente, una parte opcional del protocolo, y no todas las capacidades se pueden guardar. Por último, sin servicio de nombres, el descubrimiento queda fuera del middleware.

23.9.3. Promesas y *pipelining*

Cap'n Proto no intenta que una invocación remota se parezca a una local. Toda invocación devuelve inmediatamente una promesa *(promise)*[4], un resultado que todavía no está disponible. El cliente decide con `await` cuándo esperarlo. El diseño de Cap'n Proto pone mucho incapié es no ocultar la red y sus limitaciones, y para lograrlo, la promesa es esencial para aplicar optimizaciones.

Si las referencias son ciudadanos de primera clase, un patrón habitual es invocar una operación que devuelve un objeto e invocar de inmediato otra sobre ese resultado. Con un servicio de ficheros, cuyo método `open()` devuelve un objeto *Directory* o *File*, la lectura de un fichero requiere tres invocaciones encadenadas, aquí en pseudocódigo:

```
directory = root.open("bar")
file = directory.open("foo")
data = file.read()
```

En cualquier otro middleware, cada una tiene que esperar a que llegue la respuesta de la anterior para poder usar su resultado, lo que cuesta tres viajes de ida y vuelta. En Cap'n Proto, la promesa que devuelve `open()` ya ofrece los métodos del objeto que devolverá, y las invocaciones sobre ella se envían al servidor sin esperar: el cliente cursa las tres peticiones juntas, cada una con una referencia al resultado de la anterior, y el servidor las va ejecutando en cadena. Es lo que se llama *promise pipelining*, y reduce la secuencia a un único viaje de ida y vuelta. Su documentación lo presenta como un «viaje en el tiempo»: el cliente recibe la promesa, y puede usarla como destino de nuevas invocaciones, antes de que el servidor haya recibido siquiera la primera petición. Solo funciona cuando todos los objetos de la cadena los aloja el mismo servidor.

[4] El mismo concepto que el *future* de § 23.8

Esto ataca directamente el coste que § 23.6 atribuía al modelo de objetos: la facilidad para repartir referencias invita a interfaces con muchas invocaciones pequeñas, y la latencia acumulada las penaliza. Es la pieza que hace viable la ausencia de nombres: si todo se alcanza navegando desde el objeto inicial, cada paso de la navegación costaría un viaje de ida y vuelta, y sin *pipelining* la tentación sería sustituirlos por rutas y nombres globales. Con *pipelining* no hace falta sacrificar la interfaz, fusionando operaciones para ahorrar viajes —*p. ej.* un método que reciba la ruta completa—, ni la ventaja de poder entregar a cada cliente solo el objeto al que debe llegar. El ejemplo, con una única invocación, no lo necesita.

23.10. REST

REST no es un middleware como los anteriores, es un *estilo arquitectural*, es decir, un conjunto de normas y restricciones que debe cumplir un sistema distribuido [68].

La abstracción central de REST es el *recurso*. Cualquier información que se pueda nombrar puede ser un recurso, ya sea un documento, una imagen, una colección de otros recursos o incluso un servicio cuyo valor cambia con el tiempo, como «el tiempo que hace hoy en Brazatortas».

Estas son las restricciones:

1. Cliente-servidor.
2. El servidor no tiene estado.
3. Las respuestas se pueden cachear.
4. La interfaz es uniforme, es decir, la misma para cualquier recurso.
5. Permite intermediarios entre cliente y servidor.

Hay una sexta restricción (aunque opcional); la de *código bajo demanda*, que permite al servidor enviar código ejecutable al cliente.

Sin embargo, REST no impone ningún protocolo ni formato de serialización específico. Aunque HTTP es, con mucha diferencia, el protocolo más utilizado para implementar REST, en realidad se podría utilizar cualquiera que encaje con esas restricciones. Por ejemplo, CoAP [69] sigue el mismo modelo sobre UDP para dispositivos con recursos muy limitados. Con la serialización pasa lo mismo. JSON es lo habitual, pero un recurso se puede representar en cualquier formato que entiendan ambos extremos, como podría ser Protocol Buffers, XML o incluso texto plano, y no dejaría de ser REST.

Al igual que en JSON-RPC (§ 23.5.4), no hay IDL, compilador, *stubs* ni *runtime* específico. Si el protocolo elegido es HTTP, se puede utilizar cual-

quier cliente desde `curl` hasta un navegador, y lo mismo se puede decir del servidor.

Aunque gRPC también se encapsula en HTTP (§ 23.5.3), lo usa de un modo muy distinto. Para gRPC, HTTP es solo el transporte. Todas sus invocaciones son `POST` y la operación va en la ruta (`/math.Math/factorial`). En REST, en cambio, la ruta identifica un recurso y el método HTTP es la operación. Para REST, HTTP es el protocolo de aplicación, no de transporte.

23.10.1. Quizzy

El resto de la sección se apoya en /`quizzy`, una pequeñísima aplicación de cuestionarios de elección múltiple escrita específicamente para ilustrar los conceptos de REST. Permite crear preguntas, agruparlas en cuestionarios y responder un cuestionario para obtener una calificación. Por simplicidad, no hay usuarios y las calificaciones no se almacenan.

Quizzy maneja tres tipos de datos:

- Una pregunta tiene un enunciado, varias opciones y el índice de la opción correcta.
- Un cuestionario tiene un título y una lista de preguntas. Una misma pregunta puede formar parte de varios cuestionarios.
- Una calificación indica los puntos obtenidos sobre el total de preguntas del cuestionario. Cada respuesta correcta vale un punto.

Las preguntas y los cuestionarios son los recursos del servicio, y cada uno tiene su propia URI. Lo habitual es nombrar cada colección en plural (`/questions`, `/quizzes`) e identificar cada elemento añadiendo su identificador a la ruta de la colección (`/questions/1`). Para crear una pregunta, el cliente envía un `POST` a la colección. El servidor le asigna un identificador y responde con el código 201 (*Created*) y una cabecera `Location` con la ruta del nuevo recurso.

La calificación es distinta, porque no se almacena. La ruta `/quizzes/1/grade` no identifica nada que se pueda consultar después, sino un recurso cuya única función es procesar las respuestas que recibe.

Todo el servidor está en un único fichero, `server.py`, que tiene tres partes: el modelo de datos del ORM, la comprobación de esquema y los manejadores de cada operación.

Se apoya en varios módulos:

FastAPI (`python3-fastapi`)
: Es un *framework* web. Asocia cada combinación de método y ruta con una función Python, el *manejador*, mediante un decorador. Con ayuda de Pydan-

tic, convierte el JSON de cada petición en objetos Python, y lo que devuelve el manejador en una respuesta. También genera la descripción del API.

uvicorn (`python3-uvicorn`)
: Es el servidor HTTP. Acepta las conexiones TCP, interpreta las peticiones y se las entrega a FastAPI.

Pydantic (`python3-pydantic`)
: Permite definir los *esquemas*, clases que describen el formato de los datos recibidos y devueltos, y comprueba que se cumple.

SQLAlchemy (`python3-sqlalchemy`)
: Es el ORM. Asocia las clases `Question` y `Quiz` con tablas de la base de datos, y traduce las operaciones sobre esos objetos a consultas SQL.

SQLite
: Es la base de datos. Se guarda en un único fichero (`quizzy.db`). El módulo que la maneja (`sqlite3`) forma parte de la librería estándar de Python.

En el manejador que crea una pregunta (Listado 23.36), el decorador lo asocia con `POST` sobre `/questions` y fija el código de estado 201. FastAPI le entrega los datos ya validados y convierte en JSON el objeto que devuelve, así que lo único que hace el manejador con HTTP es añadir la cabecera `Location`.

```
@app.post('/questions', status_code=201)
def create_question(data: QuestionData, response: Response,
                    session: SessionDep) -> QuestionWithSolution:
    question = Question(**data.model_dump())
    session.add(question)
    session.commit()
    response.headers['Location'] = f'/questions/{question.id}'
    return question
```

LISTADO 23.36: Manejador que crea una pregunta
/quizzy/server.py

El programa `client.py` no depende de ninguna de ellas. Utiliza `requests` (`python3-requests`), una biblioteca HTTP de propósito general, para crear dos preguntas y un cuestionario, consultarlo y calificarlo. Las mismas peticiones se pueden hacer con `curl`, como se muestra en /quizzy/README.md. Esto demuestra que el servidor ofrece una interfaz que se puede invocar desde clientes distintos, escritos en lenguajes distintos, sin necesidad de *stubs* ni de un *runtime*.

23.10.2. Recursos y representaciones

En Quizzy, cada pregunta y cada cuestionario es un recurso. Su nombre se mapea a una URI, que tiene una función similar a la referencia textual de

ICE (§ 23.6.3). Contiene el *endpoint* (`http://host:puerto`) y la identidad del recurso (la ruta).

> Es muy habitual ver el término «endpoint» para designar una ruta, o una combinación de método y ruta (*p. ej.* `PUT /quizzes/1`). Es un uso de la jerga que no coincide con la terminología de la tesis de Fielding [68], el documento fundacional de REST, que habla de recursos y de sus identificadores (URI). En nuestro caso, *endpoint* designa siempre la tupla dirección-puerto-protocolo, como en el resto del libro.

El cliente REST no obtiene el recurso, ni un proxy para invocar. El cliente REST solo maneja representaciones del estado de los recursos. Por eso, su nombre lo define perfectamente: transferencia de representación de estado. Sin embargo, un mismo recurso puede tener varias representaciones. En Quizzy, una pregunta incluye la respuesta correcta cuando se pide directamente, pero no cuando forma parte de un cuestionario.

En el servidor, la misma pregunta existe de tres formas: una fila en una tabla de la base de datos, un objeto Python y su representación JSON. El ORM convierte entre las dos primeras, y Pydantic entre las dos últimas.

El modelo del ORM (Listado 23.37) define una clase por cada tabla, cuyos atributos corresponden a las columnas. Por simplicidad, las opciones de una pregunta se guardan en una única columna de tipo JSON, en lugar de en una tabla de opciones con clave ajena a la de preguntas. Como una misma pregunta puede formar parte de varios cuestionarios, la relación entre ambos requiere una tabla intermedia sin clase propia (`quiz_questions`). El atributo `questions` de `Quiz` no corresponde a ninguna columna sino a esa relación, y el ORM lo resuelve devolviendo la lista de objetos `Question`.

```
quiz_questions = Table(
    'quiz_questions', Base.metadata,
    Column('quiz_id', ForeignKey('quizzes.id'), primary_key=True),
    Column('question_id', ForeignKey('questions.id'), primary_key=True),
)

class Question(Base):
    __tablename__ = 'questions'
    id: Mapped[int] = mapped_column(primary_key=True)
    text: Mapped[str]
    options: Mapped[list[str]] = mapped_column(JSON)
    correct_option: Mapped[int]

class Quiz(Base):
    __tablename__ = 'quizzes'
    id: Mapped[int] = mapped_column(primary_key=True)
    title: Mapped[str]
```

```
    questions: Mapped[list[Question]] = relationship(
        secondary=quiz_questions, order_by=Question.id)
```

LISTADO 23.37: Modelo de datos del ORM
/quizzy/server.py

Cada representación es un esquema de Pydantic (Listado 23.38). La de un cuestionario incluye sus preguntas como `QuestionWithoutSolution`, que no tiene el campo `correct_option`.

```
class QuestionWithSolution(QuestionData):
    id: int

class QuestionWithoutSolution(BaseModel):
    id: int
    text: str
    options: list[str]
```

LISTADO 23.38: Representaciones de una pregunta
/quizzy/server.py

23.10.3. Interfaz uniforme

En gRPC o en ICE, cada interfaz define sus propias operaciones. En REST, todos los recursos comparten el mismo conjunto (reducido) de operaciones, de modo que el vocabulario del dominio de aplicación se limita a los nombres de los recursos. Si se usa HTTP, esas operaciones son sus mensajes (o «métodos») [70] (Cuadro 23.3).

Método	Seguro	Idempotente	Uso habitual
`GET`	sí	sí	obtener una representación
`PUT`	no	sí	crear o reemplazar un recurso
`DELETE`	no	sí	eliminar un recurso
`POST`	no	no	crear un recurso o procesar datos
`PATCH`	no	no	modificar parcialmente un recurso
`QUERY`	sí	sí	consultar con datos en el cuerpo

CUADRO 23.3: Propiedades REST de los métodos HTTP más habituales

Un método *seguro* es uno que no modifica el estado del servidor. La semántica de cada método forma parte del protocolo, así que cualquier intermediario (un proxy, una caché, el propio navegador) sabe si puede repetir una petición o guardar su respuesta, sin conocer la aplicación. Es la misma información que lleva cada petición ICE sobre las operaciones idempotentes, pero aquí no depende de cada interfaz.

El Cuadro 23.4 muestra la interfaz completa de Quizzy, donde `{id}` representa el identificador de una pregunta o de un cuestionario. Con solo tres métodos se manejan todos sus recursos.

Método	Ruta	Acción
`POST`	`/questions`	crea una pregunta
`GET`	`/questions`	devuelve la lista de preguntas
`GET`	`/questions/{id}`	devuelve una pregunta y su solución
`PUT`	`/questions/{id}`	modifica una pregunta
`POST`	`/quizzes`	crea un cuestionario
`GET`	`/quizzes`	devuelve la lista de cuestionarios
`GET`	`/quizzes/{id}`	devuelve un cuestionario, sin soluciones
`PUT`	`/quizzes/{id}`	modifica un cuestionario
`POST`	`/quizzes/{id}/grade`	califica las respuestas enviadas

CUADRO 23.4: Recursos y operaciones de Quizzy

Quizzy modifica las preguntas y los cuestionarios con `PUT`. El cliente envía la pregunta o el cuestionario completos, y el servidor sustituye el estado anterior por el que recibe. Por eso la operación es idempotente, porque repetirla deja el recurso en el mismo estado. Es posible hacer una actualización parcial de un recurso con `PATCH`, pero no se utiliza en Quizzy.

Calificar un cuestionario no encaja bien en este esquema. No modifica nada en el servidor, así que sería una operación segura e idempotente, pero necesita enviar las respuestas en el cuerpo, y en una petición `GET` el cuerpo no tiene una semántica definida. Quizzy, como muchas aplicaciones, utiliza `POST`, que no garantiza ninguna de esas dos propiedades. El método `QUERY` [71], que acaba de entrar en el estándar de HTTP en 2026, resuelve este problema: es seguro e idempotente, y permite enviar datos en el cuerpo de la petición, pero es demasiado reciente y la mayoría de clientes, servidores, librerías y frameworks no lo soportan aún.

La interfaz uniforme incluye otra restricción que casi ninguna API cumple. Las representaciones deberían incluir enlaces a los recursos relacionados, de modo que el cliente pueda navegar por ellos sin conocer de antemano las URI, igual que una persona navega por la web. Esto se conoce como Hypermedia As The Engine Of Application State (HATEOAS).

23.10.4. Sin estado

Cada petición debe incluir todo lo que el servidor necesita para atenderla. El servidor no guarda nada sobre su conversación con un cliente concreto, aunque sí guarda el estado de los recursos (en Quizzy, las preguntas y los cuestionarios).

La calificación del cuestionario también es un buen ejemplo de esto. El cliente envía todas las respuestas en una sola petición y recibe la nota. Si las enviara una a una, el servidor tendría que recordar el progreso de cada cliente, es decir, mantener una sesión con estado. Para hacer algo así con REST, ese progreso tendría que modelarse como un recurso más (*p. ej.* `/attempts/{id}`).

Al no tener estado, el servidor es mucho más fácil de replicar y por tanto, el servicio escala mejor.

23.10.5. Validación y documentación

Sin un IDL compartido ni *stubs* generados, nada garantiza que una petición tenga la forma esperada. Cualquiera puede enviar cualquier cosa con `curl` o con una librería de HTTP como `requests`. Por eso el servidor tiene que validar todo lo que recibe.

Quizzy aplica tres niveles:

- Tipos y restricciones simples, *p. ej.* una pregunta debe tener entre 2 y 6 opciones. Pydantic las comprueba antes de ejecutar el manejador y, si fallan, FastAPI responde con un error 422 (*Unprocessable Content*).
- Restricciones entre campos, *p. ej.* el índice de la respuesta correcta debe ser menor que el número de opciones. Ninguno de los dos campos es incorrecto por separado. Pydantic también las comprueba, pero con un método escrito a mano, no con una declaración.
- Restricciones que dependen del estado, *p. ej.* que existan las preguntas que forman un cuestionario. Solo se pueden comprobar consultando la base de datos.

Los dos primeros niveles se declaran en el esquema `QuestionData` (Listado 23.39), con los argumentos de `Field()` y con el método `check_correct_option()`, respectivamente.

```
class QuestionData(BaseModel):
    text: str = Field(min_length=1)
    options: list[str] = Field(min_length=2, max_length=6)
    correct_option: int = Field(ge=0)

    @model_validator(mode='after')
    def check_correct_option(self) -> Self:
        if self.correct_option >= len(self.options):
```

```
            raise ValueError('correct_option must be the index of one of the options')
        return self
```

LISTADO 23.39: Esquema de los datos de una pregunta
/quizzy/server.py

Para describir una API REST se suele utilizar OpenAPI [72][5]. Un documento OpenAPI describe los recursos, sus operaciones y los esquemas de los datos, estos últimos con JSON Schema [73]. A partir de él, distintas herramientas generan documentación interactiva (como `Swagger UI`), *stubs* de cliente o de servidor y validadores.

Permite dos formas de trabajar:

- *design-first*: primero se escribe el documento OpenAPI y después el código, igual que con un IDL.
- *code-first*: el documento se genera a partir del código. Es lo que hace FastAPI, que lo publica en `/openapi.json`.

Un documento generado a partir del código solo recoge lo que está declarado. En Quizzy, el esquema de una pregunta limita el número de opciones, pero no dice nada de la relación entre la respuesta correcta y las opciones, porque esa regla solo está expresada en la implementación Python.

La gran diferencia con un IDL es que el documento OpenAPI es opcional. Ni el cliente ni el servidor lo necesitan para funcionar.

Sin embargo, describir así una API entra en conflicto con HATEOAS. El documento OpenAPI establece de antemano las rutas de todos los recursos (*p. ej.* `/quizzes/{id}`), y un cliente construido a partir de él depende de ellas, cuando en principio solo debería conocer una URI inicial y obtener las demás de los enlaces que incluye cada representación. Quizzy tampoco cumple esa restricción. Sus representaciones no incluyen enlaces y, aunque el servidor indica la ruta de cada recurso nuevo en la cabecera `Location`, `client.py` no la utiliza y compone las rutas a partir del identificador.

23.10.6. Errores y semántica de invocación

Con HTTP, los errores se notifican con el código de estado de la respuesta. Quizzy responde 404 (*Not Found*) si el recurso no existe y 422 si los datos no son válidos. El cuerpo puede llevar más detalles, y existe un formato estándar para eso en la RFC 9457 [74].

[5] Existen otros lenguajes con el mismo propósito, aunque su uso es mucho menos común

Con REST sobre HTTP (y por tanto sobre TCP), la semántica es *at-most-once*, igual que en gRPC. Un cliente solo debería reintentar automáticamente una petición si es idempotente [70].

23.11. Versionado

La interfaz de un servicio cambia con el tiempo. Aparecen operaciones nuevas, y otras necesitan un parámetro adicional o un tipo de mayor tamaño. En un programa local basta con recompilarlo todo, pero en un sistema distribuido los clientes y los servidores se compilan y despliegan por separado, a menudo por organizaciones distintas, así que durante un tiempo deben convivir versiones distintas de la misma interfaz. Un cambio es *compatible hacia atrás (backward compatible)* si los clientes antiguos pueden invocar servidores nuevos, y es *compatible hacia delante (forward compatible)* si los servidores antiguos admiten peticiones de clientes nuevos. Un cambio incompatible obliga al servidor a proporcionar ambas versiones a la vez y al cilente a indicar en las peticiones a qué versión se dirige.

Qué cambios son compatibles depende de dos decisiones de diseño del middleware: cómo identifica la petición la operación invocada y al modo en que serialización empaqueta los datos; si los etiqueta o simplemente los coloca uno detrás de otro. El caso más simple (como pico-rpc) es el caso más desfavorable en ambos aspectos. El identificador de cada operación es su posición en la interfa, así que si se inserta una operación delante de `factorial()`, cambian los identificadores arruinando la compatibilidad hacia atrás:

```
class Math(Interface):
    def square(n: uint8) -> uint64: ...               # 0 (nueva)
    def factorial(n: uint8) -> uint64: ...            # 1 (antes 0)
    def power(base: uint8, exp: uint8) -> uint64: ...  # 2 (antes 1)
```

Un cliente antiguo que invoca `factorial()` ejecuta en realidad `square()`, sin que ninguno de los dos extremos lo advierta.

ONC RPC incorpora la versión de la interfaz del propio protocolo. Cada petición incluye la versión como campo, junto a los números de programa y de procedimiento [75], y un mismo servidor puede ofrecer varias versiones de un programa. La contrapartida es que XDR no etiqueta los datos, así que cualquier cambio de tipos requiere una versión nueva. Es lo que necesitaría el ejemplo /`onc-rpc-factorial`, cuyo resultado (`unsigned long`) se codifica con 32 bits y llega truncado, sin ningún aviso, a partir de 13!. Para devolverlo como `unsigned hyper` (64 bits) basta con añadir un segundo bloque `version` al fichero `.x`:

```
program FACTORIAL_PROG {
   version FACTORIAL_VERS {
      unsigned long FACTORIAL(int n) = 1;
   } = 1;
   version FACTORIAL_VERS2 {
      unsigned hyper FACTORIAL(unsigned int n) = 1;
   } = 2;
} = 600000002;
```

`rpcgen` genera un *dispatcher* por versión, que el servidor registra por separado en `rpcbind`, y el número de versión que forma parte del nombre de las funciones permite que las implementaciones de ambas (`factorial_1_svc` y `factorial_2_svc`) coexistan en el mismo programa. Si el cliente solicita una versión que el servidor no ofrece, la respuesta de error (`PROG_MISMATCH`) indica la más baja y la más alta disponibles, y `clnt_create_vers()` utiliza esa información para conectar con la más alta del rango que admite el cliente:

```
rpcvers_t vers;   /* versión acordada: 1 o 2 */
clnt = clnt_create_vers(host, FACTORIAL_PROG, &vers, 1, 2, "udp");
```

Así ha evolucionado NFS (programa `100003`), y un mismo servidor puede atender a la vez a clientes de varias versiones.

gRPC no gestiona la compatibilidad con versiones de la interfaz, sino campo a campo. Cada campo de un mensaje se identifica por su número (§ 23.5.3), y el receptor ignora los números que no conoce, de modo que añadir un campo es compatible en ambos sentidos. Por eso tiene sentido que gRPC exija que cada operación reciba un mensaje, aunque contenga un único campo (`FactorialRequest`); añadir un parámetro equivale a añadir un campo a ese mensaje. Un campo se puede renombrar, porque el formato binario no incluye nombres, pero su número no se debe reutilizar nunca para otro campo, y la palabra clave `reserved` lo impide para los campos eliminados. Declarar el campo nuevo como `optional` permite al servidor saber si el cliente lo envió, ya que sin esa palabra un campo ausente se lee como 0:

```
message FactorialRequest {
  int32 value = 1;
  reserved 2;                    // campo eliminado
  optional uint64 modulus = 3;   // parámetro añadido
}
```

Las operaciones, en cambio, se identifican por su nombre completo, que forma la ruta de cada invocación (`/math.Math/factorial`), así que se pueden añadir, pero no renombrar. Para los cambios incompatibles, la convención es incluir la versión en el nombre del paquete (`package math.v2`). Cada versión se convierte así en un servicio distinto, con rutas distintas, y un mismo servidor puede ofrecer ambas.

JSON-RPC carece de interfaz y, por tanto, de cualquier mecanismo para versionarla. El campo `jsonrpc` indica la versión del protocolo, no la de la aplicación, y lo único que posibilita la evolución de la «interfaz» es el paso de argumentos por nombre, que permite añadir parámetros opcionales sin depender de su posición. Son los protocolos construidos sobre JSON-RPC los que resuelven el problema, cada uno a su manera. En LSP, cliente y servidor intercambian en la primera petición (`initialize`) la lista de funcionalidades opcionales que admite cada uno, sin ningún número de versión, y esa negociación vale para toda la sesión.

En el siguiente ejemplo, el cliente declara que puede mostrar en Markdown la información emergente sobre un símbolo *(hover)*, y el servidor, que es capaz de proporcionarla:

```
--> {"jsonrpc": "2.0", "id": 1, "method": "initialize", "params": {..., "capabilities":
      {"textDocument": {"hover": {"contentFormat": ["markdown", "plaintext"]}}}}}
<-- {"jsonrpc": "2.0", "id": 1, "result": {"capabilities": {"hoverProvider": true}}}
```

MCP identifica cada versión del protocolo con la fecha de su último cambio incompatible y, desde la revisión `2026-07-28`, la incluye en cada petición, junto con las funcionalidades del cliente, en lugar de negociarla al iniciar la sesión, para que el servidor no tenga que mantener estado entre peticiones:

```
--> {"jsonrpc": "2.0", "id": 7, "method": "tools/call", "params": {
      "name": "factorial", "arguments": {"n": 5},
      "_meta": {"io.modelcontextprotocol/protocolVersion": "2026-07-28",
                "io.modelcontextprotocol/clientCapabilities": {}}}}
```

Si el servidor no admite esa versión, responde con un error que enumera las que sí admite.

Java RMI identifica cada método por un *hash* de 64 bits calculado a partir de su nombre y de los tipos de sus parámetros y de su resultado. Añadir métodos a una interfaz remota es compatible, pero cualquier cambio en la firma de un método produce un *hash* distinto, y el servidor rechaza la invocación con una excepción (*unrecognized method hash*); la incompatibilidad se detecta. La peculiaridad es que la sobrecarga de métodos de Java permite conservar la versión antigua de una operación junto a la nueva. La interfaz *Hello* (Listado 23.19) puede incorporar una variante de `sayHello()` con un parámetro, y los clientes antiguos siguen invocando la original, porque para el middleware son métodos distintos; es extensible por sobrecarga:

```
public interface Hello extends Remote {
    String sayHello() throws RemoteException;
    String sayHello(String name) throws RemoteException;  // versión 2
}
```

Los objetos que se pasan por valor tienen su propia versión, el atributo `serialVersionUID` de su clase. Si el programador no lo declara, se calcula a partir de la estructura de la clase, y casi cualquier cambio impide deserializar los objetos de la versión anterior (`InvalidClassException`). Si lo declara y lo mantiene, la especificación de la serialización de Java establece qué cambios son compatibles, como añadir atributos, que toman su valor por omisión al leer un objeto antiguo:

```
public class Question implements Serializable {
    private static final long serialVersionUID = 1L;
    private String text;
    private String hint;   // añadido en la versión 2
}
```

En ICE cada petición indica la operación por su nombre y siempre lo hará de ese modo precisamente para que añadir operaciones nunca obligue a recompilar los clientes. Invocar una operación que el servidor desconoce produce un error explícito (`OperationNotExistException`). Los argumentos, en cambio, se serializan uno tras otro sin etiquetas, y nada en la petición identifica la firma. Si el servidor espera un parámetro que un cliente antiguo no envía, la invocación falla, pero un servidor antiguo que recibe un argumento de más lo ignora sin avisar. Para añadir parámetros de forma compatible, SLICE ofrece los *opcionales*, que llevan una etiqueta numérica con el mismo objetivo que el número de campo de Protocol Buffers y que también admiten los atributos de clases y excepciones:

```
interface Counter {
    long increment(optional(1) int step);
    idempotent long get();
};
```

Si el cliente no envía el argumento, el sirviente recibe el valor especial `Ice.Unset`:

```
def increment(self, step, current):
    self.value += 1 if step is Ice.Unset else step
    return self.value
```

Lo más peculiar de ICE son las *facetas (facets)*. Un objeto puede tener varias bajo la misma identidad, cada una con su propio sirviente y su propia interfaz, y cada petición indica a cuál de ellas se dirige (por omisión, a la faceta sin nombre). El servidor puede añadir una versión nueva de la interfaz como otra faceta de los objetos existentes sin alterar la que usan los clientes antiguos. El cliente nuevo la selecciona en el proxy, con la opción `-f` del proxy textual o con `ice_facet()`, y `checkedCast()` le permite comprobar si el objeto la tiene; si no la tiene, devuelve `None`:

```
# servidor
adapter.add(CounterI(), identity)
adapter.addFacet(CounterV2I(), identity, "v2")

# cliente
counter = Example.CounterV2Prx.checkedCast(base, "v2")
```

ICE advierte de que, a cambio, la elección de la interfaz se aplaza hasta la ejecución, con menos garantías de tipos *(type safety)* que si se hiciera en la compilación.

Cap'n Proto aplica el mismo enfoque Protocol Buffers, con una restricción que se deriva de su formato sin serialización. Los ordinales de los campos y de los métodos indican el orden en que se añadieron y deben ser consecutivos a partir de `@0`. El compilador rechaza cualquier hueco *(Ordinals must be sequential with no holes)* y asigna las posiciones en memoria siguiendo ese orden, de modo que un campo nuevo nunca desplaza a los existentes. Como no se admiten huecos, un campo no se puede eliminar, solo renombrar; el propio protocolo de Cap'n Proto conserva así algunos en desuso, como `obsoleteSave`.

Los parámetros de un método forman una estructura implícita, así que se pueden añadir al final con un valor por omisión, sin declarar un mensaje explícito como en GRPC:

```
interface TextProcessor {
  upper @0 (message :Text, prefix :Text = "") -> (result :Text);
}
```

Ccontempla incluso el caso de descubrir tarde que los elementos de una lista necesitan datos adicionales. Una lista de enteros (`List(UInt32)`) se puede convertir en una lista de estructuras cuyo primer campo (`@0`) es de ese mismo tipo, y cada versión lee correctamente los mensajes de la otra:

```
struct Stats {
  obsoleteName @0 :Text;      # en desuso, no se puede eliminar
  samples @1 :List(Sample);   # en la versión 1, List(UInt32)
  label @2 :Text;             # campo añadido
}

struct Sample {
  value @0 :UInt32;
  time @1 :UInt64;
}
```

En REST las operaciones no pueden cambiar, porque son las del protocolo (interfaz uniforme), y lo que evoluciona son los recursos y sus representaciones. Añadir un miembro a una representación JSON es compatible siempre que el receptor ignore los que no conoce, como hace Pydantic por omisión,

de modo que el servidor de Quizzy acepta sin más una pregunta con campos adicionales.

Para los cambios incompatibles, la versión se puede indicar en la URI (`/v2/quizzes`), en una cabecera propia o en el tipo de medio de la representación, que el cliente solicita mediante la negociación de contenido de HTTP. La primera opción es, con mucha diferencia, la más habitual, pero choca con el propio estilo, porque la URI identifica el recurso, y una petición es la misma independientemente de la versión de su representación. Fielding llegó a afirmar que un «v1» en la URI delata una API RPC sobre HTTP y no REST[6].

HTTP dispone de cabeceras específicas para anunciar en cada respuesta que un recurso está obsoleto (`Deprecation` [76]) y la fecha a partir de la cual dejará de estar disponible (`Sunset` [77]). Un cliente que solicitara la versión 1 de la representación de un cuestionario podría recibir una respuesta como esta, que la declara obsoleta desde el 1 de enero de 2026 (`Deprecation` expresa la fecha en segundos desde 1970) y anuncia su retirada el 30 de junio de 2027:

```
GET /quizzes/1 HTTP/1.1
Accept: application/vnd.quizzy.v1+json

HTTP/1.1 200 OK
Content-Type: application/vnd.quizzy.v1+json
Deprecation: @1767225600
Sunset: Wed, 30 Jun 2027 23:59:59 GMT
```

En cualquier caso, ningún middleware evita mantener dos versiones durante un tiempo cuando un cambio es incompatible. Lo que distingue a cada uno es dónde se indica qué versión invoca el cliente: un campo del mensaje en ONC RPC, el nombre del paquete en gRPC, la firma del método en Java RMI, la faceta en Ice, y la URI o el tipo de medio en REST.

23.12. Rendimiento

La invocación remota no es gratis. Esta sección cuantifica su coste comparando el tiempo de respuesta para una misma operación —`factorial(20)`— implementada con cada uno de los middlewares del capítulo. El tiempo se mide en el cliente, desde que invoca la operación hasta que recibe el resultado, así que incluye la serialización, la transmisión de los mensajes y el despachado en el servidor. El cálculo del factorial es despreciable frente a todo lo demás.

[6] `https://x.com/fielding/status/376835835670167552`

Las invocaciones son síncronas y secuenciales, sobre una conexión ya establecida. Cada dato es la media de 1000 invocaciones, descartando el 3 % más rápido y más lento para que unas pocas invocaciones anómalas, como las que sufren una retransmisión TCP, no distorsionen el resultado[7]. Se consideran tres destinos: el mismo proceso, otro proceso del mismo nodo y un proceso en otro nodo. En el mismo proceso se mide una llamada a función local convencional, es decir, la operación sin ninguna comunicación.

El lenguaje por sí solo ya influye en el resultado. Por eso comparamos implementaciones escritas en el mismo lenguaje (C++). En la comparación se incluye también cliente-servidor con sockets TCP sin middleware, que sirven de referencia; la columna «factor» indica cuántas veces más tarda cada middleware que los sockets. El Cuadro 23.6 compara cada mecanismo en C++, Java y Python, siempre en el mismo nodo. Java RMI aparece únicamente en este último, porque solo existe para Java.

	mismo nodo		**nodo remoto**	
Middleware	media	factor	media	factor
local	0,0075	-	-	-
sockets	15,4	1,00	35 406	1,0
ONC RPC	17,9	1,16	35 158	1,0
gRPC	79,0	5,13	35 819	1,0
Ice	27,7	1,80	35 220	1,0
Cap'n Proto	30,9	2,01	35 800	1,0
REST	37,3	2,42	35 455	1,0

CUADRO 23.5: Tiempo de respuesta de una invocación medido en el cliente (μs)

En el nodo de referencia, una llamada local en C++ tarda 0,0075 μs, mientras que la misma operación con sockets TCP hacia otro proceso del mismo nodo tarda 15,4 μs. Cruzar la frontera entre procesos multiplica el coste por 2052 antes incluso de elegir un middleware, debido a los cambios de contexto, las copias entre el espacio de usuario y el núcleo, y el paso por la pila de protocolos. Lo que añade cada middleware se suma a esa cifra.

ONC RPC es el que menos sobrecarga añade, porque su protocolo apenas envía algo más que el identificador de la operación y los argumentos. ICE añade el despachado en dos niveles (objeto y método) y una cabecera al-

[7]Antes de cada medida se descartan 5000 invocaciones. Esto se hace por el calentamiento *(warm-up)*. En el mismo nodo, cada medida se repite 10 veces, intercalada con las de los demás mecanismos, y se toma la mediana, porque el reparto de cliente y servidor entre los núcleos de la CPU varía de una ejecución a otra. La frecuencia de la CPU se mantiene fija en su máximo.

go mayor. Cap'n Proto queda ligeramente por encima de ICE. No serializa (§ 23.9), pero con un único entero como argumento esa ventaja no se aprecia. gRPC es, con diferencia, el más caro, porque cada invocación es una petición HTTP/2 completa, con su marco de cabeceras, su marco de datos y su control de flujo. A cambio, ofrece multiplexación sobre una misma conexión, *streaming* en ambos sentidos y la posibilidad de atravesar cualquier intermediario HTTP. REST, sobre HTTP/1.1 y con el resultado en JSON, queda entre Cap'n Proto y gRPC. Cada mensaje lleva cabeceras de texto que hay que componer e interpretar, pero no los marcos ni el control de flujo de HTTP/2.

Middleware	C++	Java	Python	cociente Python/C++
local	0,0075	0,0090	0,794	106
sockets	15,4	21,4	18,6	1,21
ONC RPC	17,9	-	-	-
gRPC	79,0	-	255,3	3,23
Java RMI	-	34,1	-	-
Ice	27,7	-	45,1	1,63
Cap'n Proto	30,9	-	165,5	5,35
REST	37,3	-	722,3	19,4

CUADRO 23.6: Tiempo de respuesta en el mismo nodo según el lenguaje (μs) y cociente entre el de Python y el de C++

El Cuadro 23.6 permite separar el coste del middleware del coste del lenguaje. En ICE, gRPC y Cap'n Proto, la versión en Python y la versión en C++ comparten middleware, protocolo y formato de serialización, y aun así el coste por invocación se multiplica por 1,63 en ICE, por 3,23 en gRPC y por 5,35 en Cap'n Proto. Buena parte de lo que se suele atribuir al middleware se debe en realidad al intérprete. En Cap'n Proto influye además que su soporte para Python no ofrece invocaciones síncronas. En REST la diferencia es aún mayor (19,4 veces), porque las dos versiones comparten el protocolo y el formato, pero no la implementación. Las bibliotecas de Python que utiliza Quizzy (§ 23.10.1) interpretan HTTP en el propio intérprete, mientras que las de ICE, gRPC y Cap'n Proto delegan el protocolo al *runtime* de C++.

Contra otro nodo, la escena cambia por completo. Todos los middlewares convergen alrededor de 35 406 μs, que es prácticamente el RTT al nodo remoto, y las diferencias entre ellos quedan por debajo del ruido. En cuanto hay una red de por medio, lo que determina el rendimiento es *cuántas*

invocaciones se hacen, no cuánto cuesta cada una. Un diseño que encadena muchas llamadas pequeñas (§ 23.6) cuesta RTT por cada una, así que la estrategia adecuada es evitar la espera —con invocaciones de solo petición, por lotes o asíncronas— más que elegir un middleware más ligero o un lenguaje más eficiente.

La media tampoco refleja el coste de la *primera* invocación, que incluye establecer la conexión y lo que cada middleware necesita antes de invocar, como la consulta al registro o `checkedCast()` en ICE. En Java incluye además la carga de las clases. En Java RMI, la primera invocación tarda 103 048 μs, frente a los 34,1 μs de media. Para un servicio que se ejecuta durante mucho tiempo es irrelevante, pero para un proceso que arranca, invoca una vez y termina, esa primera invocación es todo el coste.

A la vista de todas estas medidas, podemos concluir que:

- Una invocación remota es varios órdenes de magnitud más costosa que una local.
- La mayor parte de ese coste se debe a la comunicación entre procesos y sobre todo a la red, no al middleware.
- Cuanto mayor sea el coste computacional de la operación remota, más rentable resulta.
- El rendimiento depende principalmente del número de invocaciones. Cuantos menos mensajes y más grandes, mejor. La latencia de la red es el factor dominante, algo que ocurre a todos los niveles de la pila de protocolos.
- El lenguaje de implementación puede influir tanto como el middleware, que sigue siendo muy poco en relación al total.

Y ¿qué más?

La invocación remota sustituye el diseño de un protocolo de aplicación para cada servicio por una infraestructura genérica: el middleware. Genera los *stubs* a partir de una interfaz y se encarga de serializar, transportar y despachar cada invocación. Por debajo, sigue habiendo un protocolo de petición-respuesta, y las decisiones sobre el estilo de intercambio y las medidas de tolerancia a fallos determinan la semántica de invocación, es decir, qué garantías tiene el cliente cuando se pierde un mensaje.

Los middlewares docentes pico-rpc y pico-rmi muestran esas piezas en su forma más simple. ONC RPC, GRPC, Java RMI y ICE muestran cómo las resuelven los middlewares reales, que además se ocupan de localizar los servidores y de aprovechar el tiempo de espera con la invocación y el despachado asíncronos.

Cap'n Proto y REST cuestionan parte de ese planteamiento, cada uno por un camino distinto. Cap'n Proto mantiene el modelo de objetos, pero no intenta que una invocación remota se parezca a una local. Cada invocación devuelve una promesa, y cada referencia designa un objeto y concede a la vez el permiso para invocarlo. REST prescinde de IDL, compilador y *stubs*, y en lugar de que cada interfaz defina sus propias operaciones, aplica el mismo conjunto reducido a todos los recursos. JSON-RPC prescinde igualmente de ellos, aunque sin abandonar el modelo RPC; se limita a definir el formato de los mensajes, cada uno de los cuales indica el nombre de la operación invocada.

La petición-respuesta tampoco es la única base sobre la que se pueden construir abstracciones de comunicación de alto nivel. Las colas de mensajes y el modelo de actores, por ejemplo, resuelven necesidades similares con mensajes asíncronos en un solo sentido, sin intentar parecerse a una llamada a función, en la línea del modelo publicador-suscriptor (capítulo 16).

Capítulo 24

Epílogo

Este libro es el resultado de nuestra experiencia como docentes en las asignaturas de redes de computadores y sistemas distribuidos en la Escuela Superior de Informática de la UCLM. Como tal, esta experiencia se amplía y cambia cada curso, probando nuevas ideas y enfoques, por lo que el libro a buen seguro también lo hará, corrigiendo erratas y errores, actualizando y ampliando contenido, ejemplos y referencias.

ANEXOS

Anexo A

Comandos habituales

Este capítulo presenta brevemente algunos de los programas más comunes y útiles cuando se trabaja con la shell. Salvo que se indique otro paquete, todos ellos pertenecen a GNU `coreutils`.

A.1. Ficheros y directorios

`cp` (*copy*)
: dados un nombre de archivo existente y un directorio o nombre de archivo, copia el primero en el segundo. Si el archivo destino existe, lo sobrescribe.

`cut` escribe en su salida partes de las líneas de entrada según sus opciones:

```
alice@maine:~$ date
  Sun Aug 26 12:47:35 CEST 2012
  alice@maine:~$ date | cut --delimiter=" " --fields=2
  Aug
```

`diff` muestra las diferencias entre dos archivos.

`find` encuentra archivos a partir de su nombre.

`head` escribe a su salida los primeros bytes o líneas de su entrada o del archivo indicado como argumento.

```
alice@maine:~$ head --lines=3 /etc/passwd
  root:x:0:0:root:/root:/bin/bash
  daemon:x:1:1:daemon:/usr/sbin:/bin/sh
  bin:x:2:2:bin:/bin:/bin/sh
```

`ln` (*link*)
: crea enlaces a otros archivos o directorios.

`md5sum`
: calcula (o comprueba) la suma MD5 del archivo indicado (o de su entrada estándar).

`mkdir` (*make dir*)
: crea un directorio.

mkfifo
: crea una tubería con nombre ligado al sistema de archivos.

mv (*move*)
: es equivalente a `cp` pero borra el archivo original.

nl (*number lines*)
: lee líneas de un archivo y las escribe a su salida precedidas del número de línea.

pwd (*print working directory*)
: indica el nombre del directorio actual.

rm (*remove*)
: elimina el archivo indicado.

rmdir (*remove dir*)
: borra un directorio vacío.

seq (*sequence*)
: escribe un rango de enteros. Puede usarse como variable de control de un bucle `for`.

split
: divide un archivo en trozos del tamaño indicado.

stat produce información detallada de archivos.

tac (*reverse cat*)
: escribe a su salida líneas de su entrada en orden inverso.

tail escribe a su salida los últimos bytes o líneas de su entrada o del archivo indicado como argumento. Con la opción `-f/--follow` monitoriza el archivo mostrando el nuevo contenido tan pronto como se añade. Muy útil para hacer el seguimiento de un archivo de log.

tee crea una «te». Lee de su entrada estándar y lo escribe al mismo tiempo a su salida estándar y al archivo indicado.

test, [[1]
: realiza comprobaciones lógicas sobre archivos, cadenas de texto y datos numéricos. Se utiliza habitualmente como condición en estructuras de control `if`, `while`, etc.

touch
: cambia la fecha de un archivo (por defecto ahora). Si el archivo indicado no existe, crea uno vacío.

[1] No es una errata: `[`, el corchete de apertura

tr (*translate*)
lee líneas de su entrada y las escribe a su salida reemplazando *tokens*[2] tal como lo indiquen sus opciones.

uniq lee líneas de su entrada y las escribe a su salida, pero omitiendo líneas consecutivas idénticas.

wc (*word count*)
cuenta letras, palabras y líneas del archivo indicado (o de su entrada estándar) y escribe los totales en su salida.

yes escribe «y» y un salto de línea continuamente a su salida. Se utiliza para contestar afirmativamente a cualquier pregunta que haga un programa por línea de comando:

```
alice@maine:~$ touch kk
alice@maine:~$ yes | rm --interactive --verbose kk
rm: remove regular empty file `kk'? removed `kk'
```

A.2. Sistema

dd copia bloques de bytes en dispositivos (archivos o discos).

df (*disk free*) muestra información sobre el uso de los sistemas de archivos montados en el computador.

du (*disk usage*) calcula el espacio de disco utilizado por archivos y directorios.

fdisk
muestra y manipula tablas de particiones.

mount
monta dispositivos de almacenamiento de cualquier tipo sobre el sistema de archivos.

uname
(*unix name*) muestra información del sistema: núcleo, hostname, versión, arquitectura, etc.

```
alice@maine:~$ uname -a
Linux maine 6.12.38+deb13-amd64 #1 SMP Debian 6.12.38-1 (2025-07-16) x86_64
↪  GNU/Linux
```

sync escribe a disco inmediatamente las operaciones pendientes sobre archivos.

[2] Un *token* es cualquier combinación de caracteres que cumpla una expresión regular concreta.

A.3. Procesos

nice ejecuta un programa fijando un nivel de prioridad.

nohup
ejecuta un comando ignorando las señales para su finalización (SIGHUP). Permite que un proceso creado por una shell sobreviva a la propia shell.

sleep
realiza una pausa del número de segundos indicados.

true, false
simplemente retornan 0 y 1, los valores que corresponden a una ejecución correcta e incorrecta respectivamente.

top (procps)
, muestra una lista actualizada de los procesos ordenada por el consumo de CPU.

A.4. Usuarios y permisos

chmod (*change mode*)
cambia los permisos de lectura, escritura y ejecución de un archivo o directorio.

chown (*change owner*)
cambia el propietario (usuario y grupo) de un archivo.

chgrp (*change group*)
cambia el grupo propietario de un archivo.

groups
muestra los nombres de los grupos a los que pertenece el usuario.

id muestra los identificadores del usuario y los grupos a los que pertenece.

sudo (*superuser do*) permite ejecutar un comando con los privilegios de otro usuario, por defecto el administrador.

who muestra los usuarios conectados junto con los terminales que utilizan y la hora de la conexión (*login*).

whoami
muestra el nombre del usuario conectado.

Anexo B

Otros comandos de red

B.1. Otros *ping*

Algunas variantes del programa `ping` disponibles en sistemas GNU:

fping
: Permite especificar múltiples destinos que serán comprobados secuencialmente.

oping
: También permite especificar múltiples destinos, pero las peticiones son enviadas en paralelo.

B.2. mtr

`mtr` es muy similar a `traceroute`, pero puede utilizarse con interfaz gráfica, tal como se muestra en la Figura B.1.

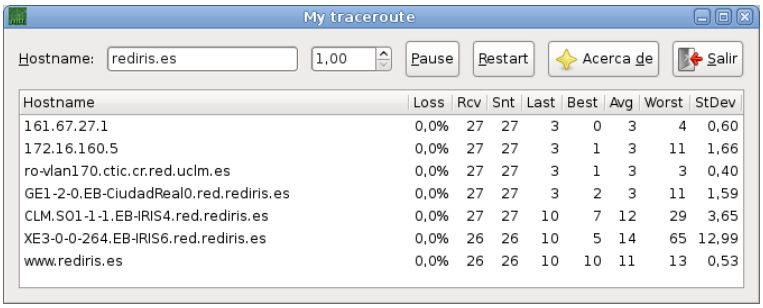


FIGURA B.1: Aspecto del programa `mtr`

B.3. ss

El comando `ss` (🎁 `iproute2`) proporciona mucha información sobre el estado de las conexiones TCP del propio nodo. Puedes utilizar el ejemplo de la §13.6 para analizar la información que ofrece. El Listado B.1 muestra un fragmento a modo de ejemplo.

```
~$ ss -timon
State   Recv-Q  Send-Q   Local Address:Port  Peer Address:Port
ESTAB   0       2150144  127.0.0.1:33188     127.0.0.1:2000    timer:(persist,348ms,0)
^^I skmem:(r0,rb131072,t0,tb2626560,f2176,w2185088,o0,bl0,d0) ts sack cubic wscale:7,7
↪  rto:220 backoff:1 rtt:17.255/0.045 mss:54976 pmtu:65535 rcvmss:536 advmss:65483
↪  cwnd:10 bytes_sent:1894649664 bytes_retrans:164928 bytes_acked:1894484737
↪  segs_out:51699 segs_in:51693 data_segs_out:34467 send 254887279bps lastsnd:332
↪  lastrcv:9472192 lastack:92 pacing_rate 509767168bps delivery_rate 50712936bps
↪  delivered:34468 busy:9472188ms rwnd_limited:9472180ms(100.0%) retrans:0/3
↪  dsack_dups:3 rcv_space:65495 rcv_ssthresh:65495 notsent:2150144 minrtt:12.982
↪  rcv_wnd:65536
```

LISTADO B.1: TCP: Análisis de una conexión con el programa `ss`

El significado de cada campo es el siguiente:

- `State`: Estado de la conexión.
- `Recv-Q` y `Send-Q`: Datos pendientes en los buffers de recepción y envío respectivamente.
- `Local Address:Port` y `Peer Address:Port`: Direcciones y puertos de los extremos de la conexión.
- `skmem`: Información sobre la memoria del socket (socket memory):
  - `r`: Memoria ocupada en el buffer de recepción (bytes).
  - `rb`: Memoria total en el buffer de recepción (bytes).
  - `t`: Memoria ocupada en el buffer de envío (bytes).
  - `tb`: Memoria total en el buffer de envío (bytes).
  - `f`: Memoria utilizada como caché (bytes).
  - `w`: Memoria ocupada por paquetes encolados (bytes).
  - `o`: Memoria ocupada por opciones del socket (bytes).
  - `bl`: Memoria ocupada por el backlog (bytes).
  - `d`: Número de paquetes descartados.
- `ts`: La conexión está usando timestamps para medir el RTT.
- `sack`: La confirmación selectiva está habilitada.
- `cubic`: Algoritmo de control de congestión.
- `wscale:<envío>,<recepción>`: Factor de escala de las ventanas.
- `rto`: RTO actual (ms).
- `rtt:<medio>/<dev>`: RTT medio y desviación estándar (ms).
- `ato`: Auto TimeOut (ms) antes de enviar un ACK.
- `mss`: MSS (bytes).
- `pmtu`: Path MTU (bytes).
- `rcvmss`: MSS de recepción (bytes).
- `advmss`: MSS anunciado por el receptor (bytes).

- `cwnd`: Ventana de congestión (en segmentos).
- `bytes_sent`: Bytes enviados.
- `bytes_retrans`: Bytes retransmitidos.
- `bytes_acked`: Bytes confirmados.
- `bytes_received`: Bytes recibidos.
- `segs_out`: Segmentos enviados.
- `segs_in`: Segmentos recibidos.
- `data_segs_out`: Segmentos enviados que contienen datos.
- `data_segs_in`: Segmentos recibidos que contienen datos.
- `send`: Velocidad de envío.
- `lastsnd`: Último segmento enviado (ms desde el inicio).
- `lastrcv`: Última vez que se recibió un segmento (ms desde el inicio).
- `lastack`: Última vez que se recibió un ACK.
- `pacing_rate`: Velocidad de envío para evitar congestión.
- `delivery_rate`: Tasa de entrega efectiva.
- `delivered`: Número de segmentos entregados con éxito.
- `app_limited`: La tasa está limitada por la aplicación, no por la red.
- `busy`: Tiempo total de la conexión.
- `retrans`: Número de retransmisiones.
- `dsack_dups`: Número de segmentos duplicados.
- `rcv_rtt`: RTT de recepción estimado (ms).
- `rcv_space`: Tamaño del buffer de recepción.
- `rcv_ssthresh`: ssthresh (bytes).
- `minrtt`: RTT mínimo observado (ms).
- `snd_wnd`: Ventana de envío (bytes).
- `rcv_wnd`: Ventana de recepción (bytes).

Anexo C

Docker

A lo largo del libro se han incluido varios ejemplos que requieren cierta infraestructura y que hemos llamado 'laboratorios'. Para implementarlos hemos utilizado `docker`, una herramienta que permite ejecutar aplicaciones dentro de contenedores. Un contenedor es un entorno de ejecución aislado, separado del SO anfitrión que tiene su propio sistema de archivos, configuración de red y procesos. Esto nos permite simular varios nodos que se comunican entre sí, obviamente muy útil cuando hablamos de redes, aunque no sea ese su propósito principal.

Al empezar a utilizar `docker` puede recordar bastante a sistemas de VM (Virtual Machine) como `VirtualBox` o `VMware`, sin embargo, hay diferencias importantes:

- El contenedor `docker` no emula el hardware de un computador, ni ejecuta un SO completo, sino que utiliza los programas sobre el SO del anfitrión. No permite ejecutar un SO Windows sobre un SO GNU/Linux o viceversa, ni siquiera de una versión diferente del núcleo.
- A diferencia de las VM, un contenedor `docker` está pensado para ejecutar un único proceso. Cuando el proceso termina, el contenedor también. Esto hace que la imagen del contenedor sea normalmente mucho más pequeña que la de una VM.
- Los contenedores son efímeros, es decir, no tienen estado. Cualquier cambio (escritura) que se aplique al sistema de archivos del contenedor se pierde al pararlo, aunque es posible montar volúmenes externos persistentes.

C.1. Imágenes

Para ejecutar un contenedor `docker` necesitas una imagen. La imagen la puedes obtener de un repositorio de imágenes o *hub* (como `https://hub.docker.com`) o bien la puedes crearla tu mismo a partir de otra.

La prueba habitual para comprobar que tienes una instalación correcta y funcional es ejecutar un contenedor con la imagen `hello-world`, que siendo la primera vez, conllevará la descarga automática de dicha imagen.

```
1  ~$ docker run hello-world
2  Unable to find image 'hello-world:latest' locally
3  latest: Pulling from library/hello-world
4  17eec7bbc9d7: Pull complete
5  Digest: sha256:1d983076c7facf19a9a53a7a4855dbd736bf5206e3bfc5142a4c1da4ed44dedc
6  Status: Downloaded newer image for hello-world:latest
7
8  Hello from Docker!
9  This message shows that your installation appears to be working correctly.
10
11 To generate this message, Docker took the following steps:
12  1. The Docker client contacted the Docker daemon.
13  2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
14     (amd64)
15  3. The Docker daemon created a new container from that image which runs the
16     executable that produces the output you are currently reading.
17  4. The Docker daemon streamed that output to the Docker client, which sent it
18     to your terminal.
19
20 To try something more ambitious, you can run an Ubuntu container with:
21  $ docker run -it ubuntu bash
22
23 Share images, automate workflows, and more with a free Docker ID:
24  https://hub.docker.com/
25
26 For more examples and ideas, visit:
27  https://docs.docker.com/get-started/
```

La **línea 2** dice que la imagen solicitada no está en tu computador, por lo que que procede a descargarla. Las **líneas 3-6** muestran información sobre la descarga. Finalmente, a partir de la **línea 7** aparece la salida del programa `hello` que es un programa incluido en la imagen y que se ejecuta dentro del contenedor.

Cualquier imagen descargada se almacena es tu computador y se puede reutilizar. Puedes comprobarlo con el siguiente comando:

```
~$ docker images
REPOSITORY      TAG           IMAGE ID       CREATED         SIZE
hello-world     latest        1b44b5a3e06a   6 days ago      10.1kB
```

C.2. Contenedores

El contenedor representa la ejecución del programa definido en una imagen, y de hecho puedes crear varios contenedores sobre la misma imagen. Puedes probar un contenedor que ejecuta un servidor web (`nginx`). De hecho eso, es lo habitual: `docker` se suele usar para ejecutar un servidor o un servicio que queda en ejecución indefinidamente.

```
~$ docker run -d -p 8000:80 -v .:/data nginx
fc085a4e035a6d07d0ab22ed6d03887d99a0ad3c6ff84cc221ee6b4eb09dbac5
```

Los argumentos de ese comando son los más frecuentes:

- `-d`: ejecuta el contenedor en segundo plano. La secuencia alfanumérica que ves en este caso es el identificador del contenedor.
- `-p 8000:80`: consigue que el puerto TCP 80 del contenedor esté accesible a través del 8000 de tu computador y con ello dar acceso al servicio desde el exterior. Puedes probar a acceder a ese servidor web cargando `http://localhost:8000` en tu navegador y verás la página de bienvenida de `nginx`.
- `-v .:/data`: monta el directorio actual (filename.) de tu computador en el directorio `/data` del contenedor. Eso da al contenedor acceso a los archivos de ese directorio y además le otorga persistencia.
- `nginx`: es el nombre de la imagen, de forma análoga al `hello-world` del ejemplo anterior.

Puedes comprobar el estado de los contenedores y sus procesos asociados con `docker ps` o `docker container ls`, aunque este segundo comando muestra también los contenedores parados.

```
~$ docker container ls
CONTAINER ID   IMAGE   COMMAND                  PORTS                  NAMES
fc085a4e035a   nginx   "/docker-entrypoint.…"   0.0.0.0:8000->80/tcp   hardcore_cannon
```

En esta sencilla información de estado del contenedor aparece el comando que se ejecuta en el interior del contenedor (llamado menudo *entrypoint*), el puerto mapeado 8000->80 y el nombre del contenedor (`hardcore_cannon`) que se genera aleatoriamente y no debes confundir con el nombre de la imagen.

El contenedor seguirá en ejecución indefinidamente hasta que lo detengas u ocurra un error grave. En este situación `docker` te permite ejecutar otros comandos del contenedor con la orden `docker exec`. Por ejemplo, esto ejecuta una shell interactiva (gracias a la opción `-ti`) dentro del contenedor:

```
~$ docker exec -ti hardcore_cannon bash
root@1ad721a249fe:/#
```

Por último puedes parar un contenedor activo con:

```
~$ docker stop hardcore_cannon
hardcore_cannon
```

C.3. `Dockerfile`

Si quieres crear una imagen propia, o mejor dicho, personalizar una imagen existente puedes crear un archivo `Dockerfile`. Por ejemplo, supongamos que necesitas tener Python disponible en el contenedor anterior. Puedes crear un `Dockerfile` con lo siguiente:

```
FROM nginx
RUN apt-get update && apt-get install -y python3
EXPOSE 80
```

La primera línea es la imagen base, la segunda es la ejecución de dos comandos para instalar el paquete (`python3`) y la tercera informa de que el contenedor expondrá el puerto 80. Para poder usar esta imagen, primero debes construirla:

```
~$ docker build -t nginx-python .
[+] Building 11.3s (6/6)FINISHED                                    docker:default
 => [internal] load build definition from Dockerfile                          0.0s
 => => transferring dockerfile: 107B                                          0.0s
 => [internal] load metadata for docker.io/library/nginx:latest               0.0s
 => [internal] load .dockerignore                                             0.0s
 => => transferring context: 2B                                               0.0s
 => [1/2] FROM docker.io/library/nginx:latest                                 0.1s
 => [2/2] RUN apt-get update && apt-get install -y python3                   10.2s
 => exporting to image                                                        0.9s
 => => exporting layers                                                       0.8s
 => => writing image sha256:b1141380ef52995902f0c568a82feb4fcfe122d8a...      0.0s
 => => naming to docker.io/library/nginx-python                               0.0s
```

El parámetro `-t` asigna un nombre a la imagen y el punto final se refiere al directorio actual, que es donde debe buscar el archivo `Dockerfile`. Ahora la nueva imagen aparecerá en la lista:

```
~$ docker images
nginx-python              latest         b1141380ef52   2 minutes ago    248MB
nginx                     latest         ad5708199ec7   44 hours ago     192MB
hello-world               latest         1b44b5a3e06a   6 days ago       10.1kB
```

Y con esto puede arrancar un contenedor con la imagen recién creada:

```
~$ docker run -d -p 8000:80 nginx-python
```

C.4. Docker Compose

Cuando se necesitan varios contenedores que deben cooperar o comunicarse entre sí, es mucho más cómodo utilizar `docker-compose`. Este programa leer un archivo llamado `docker-compose.yml` que indica la configuración de varios contenedores y sus relaciones.

```yaml
services:
  nginx:
    image: nginx
    container_name: web-server
    ports:
      - "8080:80"
    volumes:
      - ./html:/usr/share/nginx/html:ro
    depends_on:
      - db

  db:
    image: postgres
    container_name: db
    environment:
      POSTGRES_USER: usuario
      POSTGRES_PASSWORD: password
      POSTGRES_DB: storage
    ports:
      - "5432:5432"
    volumes:
      - db_data:/var/lib/postgresql/data

volumes:
  db_data:
```

Este archivo define dos servicios (contenedores):

- `nginx` es el servidor web, equivalente al del ejemplo de la sección anterior.
  - `ports` define redirecciones de puertos como el parámetro `-p` en el comando `run`.
  - `volumes` define un punto de montaje, como la opción `-p` aunque en este caso se indica que es solo lectura (`ro`), de modo que el contenedor no podrá modificar el contenido de este directorio.
  - `depends_on` dice que el contenedor `db` debería arrancar antes que este.
- `db` ejecuta la base de datos PostgreSQL (versión 15).
  - `environment` define variables de entorno. En este caso se define el nombre de la base de datos y el nombre de usuario y clave con el que se accede. Las variables de entorno son la forma más habitual de configurar servicios `docker`.
  - `volumes` en este caso especifica un volumen gestionado por `docker`. Fíjate que la parte antes del `:` no es una ruta. Es un identificador que aparece al final en la sección `volumes`.

Para arrancar los contenedores basta con ejecutar el siguiente comando en el directorio dónde está `docker-compose.yml`:

```
~$ docker-compose up -d
[+] Running 4/4
 - Network compose-example_default   Created                                0.0s
 - Volume "compose-example_db_data"  Created                                0.0s
 - Container db                      Started                                0.3s
 - Container web-server              Started                                0.4s
```

Además, `docker` crea automáticamente una red interna con direccionamiento privado a la que conecta todos los contenedores. El nombre de la red se basa en el directorio que contiene el archivo `docker-compose.yml`. Puedes obtener información de las redes disponibles con:

```
~$ docker network ls
NETWORK ID     NAME                      DRIVER    SCOPE
32ef38990d8e   bridge                    bridge    local
dc3b83ef6011   compose-example_default   bridge    local
059b37dd8ae6   host                      host      local
030f578f0ff0   none                      null      local
```

Y puedes ver detalles sobre esa red y las direcciones asignadas a los contenedores:

```
~$ docker network inspect compose-example_default
[...]
        "IPAM": {"Config": [{
            "Subnet": "172.19.0.0/16",
            "Gateway": "172.19.0.1"
         }]},
[...]
        "Containers": {
            "3748abed03d58138129ee0f17d6f1a5a9a4df22584fef184fd1d7968f8c7aeed": {
                "Name": "db",
                "MacAddress": "c2:d7:a3:24:c0:b5",
                "IPv4Address": "172.19.0.2/16",
            },
            "5e6cb60767f0b5e00ba8e60eec9692dd073da56826af27ea3cc8db659d3151e6": {
                "Name": "web-server",
                "MacAddress": "86:47:2d:6b:5a:20",
                "IPv4Address": "172.19.0.3/16",
            }
        },
[...]
```

Anexo D

Unidades

Una pequeña referencia de las unidades de medida más comunes en redes y comunicaciones, que por supuesto, se utilizan en este libro.

D.1. Memoria

La unidad mínima de memoria es el byte (que se abrevia como B mayúscula) y para sus múltiplos se aplican los prefijos del SI (Sistema Internacional) para potencias de 2, es decir:

múltiplo	prefijo	relación	en bytes
kibibyte	KiB	1024 B	2^{10} B
mebibyte	MiB	1024 KiB	2^{20} B
gibibyte	GiB	1024 MiB	2^{30} B
tebibyte	TiB	1024 GiB	2^{40} B
pebibyte	PiB	1024 TiB	2^{50} B

CUADRO D.1: Múltiplos (binarios) de byte como medida de memoria.

D.2. Almacenamiento

El almacenamiento también se mide en bytes, pero se utilizan los prefijos del SI para potencias de 10, es decir:

múltiplo	prefijo	relación	en bytes
kilobyte	kB	1000 B	10^3 B
megabyte	MB	1000 kB	10^6 B
gigabyte	GB	1000 MB	10^9 B
terabyte	TB	1000 GB	10^{12} B
petabyte	PB	1000 TB	10^{15} B

CUADRO D.2: Múltiplos (decimales) de byte como medida de almacenamiento

Fíjate que el prefijo de 'kilo' se escribe con `k` minúscula mientras que para 'kibi' se escribe con `K` mayúscula. También es importante tener claro que en muchos textos y especificaciones técnicas se asumen potencias de 2 aunque se utilicen los prefijos decimales si se refieren a cantidades de memoria.

D.3. Tasa o velocidad de transmisión

Lo más habitual es medir la velocidad de transmisión *(data rate)* en múltiplos decimales del 'bit por segundo', denotado como bit/s[1], b/s o más comúnmente bps.

prefijo	unidad SI	relación	en bps
kbps	kbit/s	1000 bps	10^3 bps
Mbps	Mbit/s	1000 kbps	10^6 bps
Gbps	Gbit/s	1000 Mbps	10^9 bps
Tbps	Tbit/s	1000 Gbps	10^{12} bps
Pbps	Pbit/s	1000 Tbps	10^{15} bps

CUADRO D.3: Múltiplos del bps.

Aunque menos común, también se emplean múltiplos decimales del 'byte por segundo', denotado como B/s.

prefijo	relación	en B/s
kB/s	1000 B/s	10^3 B/s
MB/s	1000 kB/s	10^6 B/s
GB/s	1000 MB/s	10^9 B/s
TB/s	1000 GB/s	10^{12} B/s
PB/s	1000 TB/s	10^{15} B/s

CUADRO D.4: Múltiplos del B/s.

Es común asimilar el concepto de velocidad (o tasa) de transmisión con el *ancho de banda*. Aunque da una idea de la capacidad de un canal, no son lo mismo. El ancho de banda, como su nombre indica, define la anchura de la banda de frecuencias (medidas en herzios) que utiliza un canal para transportar señales que codifican datos —digitales para el caso que nos ocupa. Por ejemplo, un canal que transmita entre los 2 kHz y los 5 kHz tiene un ancho de banda de 3 kHz. Pero la velocidad de transmisión depende del sistema de codificación. Si el sistema solo permite codificar 1 bit por ciclo, en ese canal solo se puede transmitir datos a 3 kbps; en cambio un

[1] bit/s es la opción correcta según el SI.

sistema como 16-QAM codifica 4 bits por ciclo, de modo que permitiría una velocidad de 12 kbps para el mismo ancho de banda de 3 kHz.

Índice alfabético

3dupack, 285
8P8C, 65

A, 447
AAAA, 454
ABR, 180
`accept()`, 95, 96
Acceptor, 95
ACK, 232
ADF, 223
ADM, 223
AFxy, 370
ALG, 217
almacenamiento y reenvío, 132
AMD, 510
AMI, 509
APDF, 223
APDM, 223
aplicación distribuida, 295
AQM, 290
ARP, 71
ARQ, 232
AS, 169
ASBR, 180
ASCII, 400
ASN, 169
ASPATH, 181
`asyncio`, 319

BDP, 240, 277
BGP, 181
binder, 505
broadcast, 70
broker, 327
BSD, 2

CA, 464
cabecera, 56
Cap'n Proto, 512
  bootstrap, 514
  capacidad, 513
  promise pipelining, 515
  serialización, 512
payload, 56
CBS, 366
CBWFQ, 373
CDN, 456
CE, 291
CGN, 220
CGNAT, 220
CIDR, 123
CIR, 346
CLI, 9
cliente, 295
CMA, 353
CNAME, 447
CoDel, 291
compatibilidad hacia atrás, 524
compatibilidad hacia delante, 524
conectividad, 56
confiabilidad, 231
confirmación acumulativa, 239

confirmación duplicada, 239
congestión, 276
conmutación de paquetes, 46, 132
conmutador, 65
`connect()`, 96
control de congestión, 278
control de flujo, 231
CR, 400
CRC, 232
CWR, 292

datagrama, 78, 132
DDNS, 458
DF, 156
DGC, 498
DHCP, 197
DiffServ, 365
dig, 445
Dijkstra (algoritmo), 173
DNAT, 215
DNS, 443
DNS autoritativo, 446
DNS-SD, 458
DNSSEC, 459
DoH, 457
Don't fragment, 162
DoQ, 457
DoT, 457
DRR, 377
DSCP, 369
dupack, 285

E/S parcial, 98
eBGP, 181
EBS, 367
ECE, 292
ECMP, 140
ECN, 290
ECT, 290
EGP, 181
EIF, 223
EIM, 223
EIR, 346
EMA, 355
encaminamiento
  algoritmo, 168
  dinámico, 167
  estático, 167
  protocolo, 168
encapsulación, 56
endianness, *véase* ordenamiento de bytes
endpoint, 89
entrega directa, 135
entrega indirecta, 135
EOL, 400
esqueleto, 475
estilos de intercambio, 471
Ethernet, 61
extremo a extremo, 49

FCS, 62
FIFO, 373
FIN, *véase* TCP, FIN
FLSM, 120
flujo, 89
FQDN, 446
fragmentación, 154
FRR, 188
FSF, 4
FTP, 217

GID, 21
GIL, 302
glue record, 455
GNU, 4
go back N, *véase* repetición continua
goodput, 352
GPL, 4
gRPC, 483
  streaming, 486

HATEOAS, 521

hole punching, 228
htb, 379
Hurd, 4

iBGP, 181
ICANN, 118
ICE, 229
Ice, 499
    faceta, 527
IceP, 499
ICMP, 74, 145
    Address mask, 154
    Destination unreachable, 147
    Echo, 151
    Information, 154
    Parameter problem, 150
    Redirect, 150
    Router advertisement, 154
    Router solicitation, 154
    Time exceeded, 148
    Timestamp, 152
Identificador de proceso, 13
IDL, 477
IGP, 169
IHL, 68
interfaz de red, 61
internet, 43
interred, 43
IntServ, 365
invocación asíncrona, 508
invocation semantics, 473
IP, 67
iperf3, 363
IPv4, 67
IPv4LL, 209
IPX, 207
IRDP, 154
ISN, 246

Jacobson (algoritmo), 261
Java RMI, 496
JRMP, 496
JSON, 409
JSON-RPC, 489

Karn (algoritmo), 261
keep-alive (temporizador), 272
KiB, 553
kill (comando), 13

LCP, 66
LF, 400
`listen()`, 95
listener socket, 95
LLQ, 373
loopback, 68
LPM, 136
LSA, 180
LSDB, 180

MAC, 63
MCP, 489
mDNS, 458
MF, 156
MiB, 553
modelo híbrido, 51
MQTT, 341
MSL, 252
MSS, 248
mtr,
appmtr543
MTU, 154
MTU path discovery, 164
MTU path negotiation, 164
multicast, 70
multiplexación, 47
MX, 454
máscara de red, 118

NACK, 243
Nagle (algoritmo), 264
NAPT, 213

NAT, 211
NAT traversal, 224
`ncat`, 111
NCP, 66
netcat, 111
netfilter, 218
netmask, *véase* máscara de red
NIC, 61
NS, 447

objeto distribuido, 491
ONC, 480
ONC RPC, 480
ONT, 210
ordenamiento de bytes, 402
ORM, 517
OSI, 49
OSI (modelo), 49
OSPF, 180
OUI, 64

P2P, 216
packet switching, 46
paquete, 46
parada y espera, 233
partial I/O, *véase* E/S parcial
paso de mensajes, 470
PBS, 368
persistencia (temporizador), 265
petición-respuesta, 470
PHB, 369
PHB-CS, 370
PHB-DF, 370
PHB-EF, 370
pico-rmi, 493
pico-rpc, 477
PID, 13
pila de protocolos, 48
`ping`, 77
PIR, 346
PLPMTUD, 164
PMTU, 155
PMTUD, 164
PoP, 456
PPP, 66
preforking, 305
programación dirigida por eventos, 317
protocolo, 47
puerta de enlace, 139
puerto, 78, 89, 91, 439
puerto 0, 92
puerto abierto, 91

QoS, 345
quiet time, *véase* tempo de silencio252
Quizzy, 517

R, 471
RD, 451
recolección de basura distribuida, 498
`recv()`, 95
`recvfrom()`, 93
RED, 290
red, 43
red de computadores, 43
red de datagramas, 132
registro DNS, 444
rendimiento, 529
repetición continua, 235
repetición selectiva, 237
REST, 516
retransmisión (temporizador), 260
RIP, 179
RIR, 118
RJ45, 65
RMI, 491
  referencia remota, 502
  reflexión remota, 502

rmiregistry, 497
router, 132
RPC, 469
RR, 471
RRA, 471
RST, *véase* TCP, RST
RTO, 260
RTT, 77, 260
RTTVAR, 261

SACK, 266
SCP, 467
SDP, 217
segmento, 82
`select()`, 317, 335
selective repeat, *véase* repetición selectiva
`selectors`, 319, 336
semántica de invocación, 473
`send()`, 95
`sendall()`, 99
sending buffer (TCP), 98
`send()`, 98
`sendto()`, 92
servidor, 295
silly window, *véase* ventana tonta
SIP, 217
sistema autónomo, 169
skeleton, 475
SLA, 346
SLI, 346
Slice, 499
SLO, 346
SMA, 354
SNAT, 211
SND.UNA, 262
sniffer, 411
`SO_REUSEADDR`, 252
SOA, 447
sobrecarga de cabeceras, 248
`socket`, 88
`socket.fileno()`, 104
`socket.makefile()`, 104
`socket.recv()`, 99
`socketserver`, 313
SP, 373
SPT, 173
srTCM, 367
SRTT, 260
SRV, 454
SSH, 461
SSL, 48
stall, 357
stop and wait, *véase* parada y espera
store and forward, *véase* almacenamiento y reenvío
stub, 475
STUN, 224
style exchange protocols, 471
switch, *véase* conmutador
SWS, 264
SYN, *véase* TCP, SYN

tabla de encaminamiento, 135
tasa de transferencia, 352
TCB, 245
TCP, 81
  ACK, 245
  cliente, 299
  confiabilidad, 245
  confirmación duplicada, 271
  FIN, 250
  flags, 82
  mensaje, 82
  RST, 252
  SACK, 266
  servidor, 297
  servidor multihilo, 300
  servidor multiproceso, 303

SYN, 247
TCP/IP, 50
TCP/IP (modelo), 50
throughput, 352
tiempo de silencio, 252
TLD, 446
TLS, 48
TOS, 370
traceroute, `traceroute`149
trama, 62
transparencia de localización, 505
transporte, 89
trTCM, 368
`tshark`, 57, 412
TTL, 134
TTY, 13
TURN, 229
TXT, 459

UDP, 78
mensaje, 79
servidor, 307
UDP, 92
UID, 21
UPnP, 216
URI, 518
UTF8, 404
UUID, 64

vector-ruta, 181
ventana tonta, 264
versionado, 524
VLSM, 123

WFQ, 373
WiFi, 61
`wireshark`, 422
WRED, 372
WRR, 373
WWW, 52

XDR, 481
XML-RPC, 489

ZeroC Ice, 499
ZWP, 265

Referencias

[1] "Debian GNU/Linux." http://www.debian.org/.

[2] D. M. Ritchie and B. W. Kernighan, *The C Programming Language*. Englewood Cliffs, NJ: Prentice Hall, 1978.

[3] B. Gates, "An open letter to hobbyists." Carta abierta distribuida en el entorno del Homebrew Computer Club, Feb. 1976. `https://www.digibarn.com/collections/newsletters/homebrew/V2_01/gatesletter.html`.

[4] R. Stallman, "The gnu manifesto," 1985. `https://www.gnu.org/gnu/manifesto.en.html`.

[5] L. Torvalds, "What would you like to see most in minix?." Mensaje en el grupo de noticias comp.os.minix, Aug. 1991. `https://web.archive.org/web/20130509134305/http://groups.google.com/group/comp.os.minix/msg/b813d52cbc5a044b`.

[6] I. Murdock, "Debian announcement." Mensaje en el grupo de noticias comp.os.linux, Aug. 1993. `https://groups.google.com/g/comp.os.linux.development/c/Md3Modzg5TU/m/xty88y5OLaMJ`.

[7] L. Delgrossi and L. Berger, "Internet Stream Protocol Version 2 (ST2) Protocol Specification - Version ST2+." RFC 1819 (Experimental), Aug. 1995.

[8] W. Simpson, "The Point-to-Point Protocol (PPP)." RFC 1661 (Standard), July 1994. Updated by RFC 2153.

[9] J. Postel, "Internet Protocol." RFC 791 (Standard), Sept. 1981. Updated by RFC 1349.

[10] W. Fenner, "Internet Group Management Protocol, Version 2." RFC 2236 (Proposed Standard), Nov. 1997. Obsoleted by RFC 3376.

[11] J. Postel, "Internet Control Message Protocol." RFC 792 (Standard), Sept. 1981. Updated by RFCs 950, 4884.

[12] T. Pummill and B. Manning, "Variable Length Subnet Table For IPv4." RFC 1878 (Historic), Dec. 1995.

[13] V. Fuller, T. Li, J. Yu, and K. Varadhan, "Classless Inter-Domain Routing (CIDR): an Address Assignment and Aggregation Strategy." RFC 1519 (Proposed Standard), Sept. 1993. Obsoleted by RFC 4632.

[14] A. Retana, R. White, V. Fuller, and D. McPherson, "Using 31-Bit Prefixes on IPv4 Point-to-Point Links." RFC 3021 (Proposed Standard), Dec. 2000.

[15] F. Gont, "Deprecation of ICMP Source Quench Messages." RFC 6633, May 2012.

[16] F. Gont and C. Pignataro, "Formally Deprecating Some ICMPv4 Message Types." RFC 6918, Apr. 2013.

[17] S. Deering, "ICMP Router Discovery Messages." RFC 1256 (Proposed Standard), Sept. 1991.

[18] E. W. Dijkstra, "A note on two problems in connection with graphs," *Numerische Mathematik*, vol. 1, no. 1, pp. 269–271, 1959.

[19] C. L. Hedrick, "Routing Information Protocol." RFC 1058 (Historic), June 1988. Updated by RFCs 1388, 1723.

[20] G. Malkin, "RIP Version 2 Carrying Additional Information." RFC 1388 (Proposed Standard), Jan. 1993. Obsoleted by RFC 1723.

[21] J. Moy, "OSPF Version 2." RFC 1247 (Draft Standard), July 1991. Obsoleted by RFC 1583, updated by RFC 1349.

[22] R. Droms, "Dynamic Host Configuration Protocol." RFC 2131 (Draft Standard), Mar. 1997. Updated by RFCs 3396, 4361.

[23] Y. Rekhter, B. Moskowitz, D. Karrenberg, G. J. de Groot, and E. Lear, "Address Allocation for Private Internets." RFC 1918 (Best Current Practice), Feb. 1996.

[24] S. Cheshire, B. Aboba, and E. Guttman, "Dynamic Configuration of IPv4 Link-Local Addresses." RFC 3927 (Proposed Standard), May 2005.

[25] P. Srisuresh and K. Egevang, "Traditional IP Network Address Translator (Traditional NAT)." RFC 3022 (Informational), Jan. 2001.

[26] P. Srisuresh and M. Holdrege, "IP Network Address Translator (NAT) Terminology and Considerations." RFC 2663 (Informational), Aug. 1999.

[27] S. Guha, K. Biswas, B. Ford, S. Sivakumar, and P. Srisuresh, "NAT Behavioral Requirements for TCP." RFC 5382 (Best Current Practice), Oct. 2008.

[28] F. Audet and C. Jennings, "Network Address Translation (NAT) Behavioral Requirements for Unicast UDP." RFC 4787 (Best Current Practice), Jan. 2007.

[29] J. Postel and J. Reynolds, "File Transfer Protocol." RFC 959 (Standard), Oct. 1985. Updated by RFCs 2228, 2640, 2773, 3659.

[30] J. Rosenberg, H. Schulzrinne, G. Camarillo, A. Johnston, J. Peterson, R. Sparks, M. Handley, and E. Schooler, "SIP: Session Initiation Protocol." RFC 3261 (Proposed Standard), June 2002. Updated by RFCs 3265, 3853, 4320, 4916, 5393.

[31] M. Handley, V. Jacobson, and C. Perkins, "SDP: Session Description Protocol." RFC 4566 (Proposed Standard), July 2006.

[32] D. Senie, "Network Address Translator (NAT)-Friendly Application Design Guidelines." RFC 3235 (Informational), Jan. 2002.

[33] S. Perreault, I. Yamagata, S. Miyakawa, A. Nakagawa, and H. Ashida, "Common Requirements for Carrier-Grade NATs (CGNs)." RFC 6888, Apr. 2013.

[34] J. Weil, V. Kuarsingh, C. Donley, C. Liljenstolpe, and M. Azinger, "IANA-Reserved IPv4 Prefix for Shared Address Space." RFC 6598, Apr. 2012.

[35] J. Rosenberg, J. Weinberger, C. Huitema, and R. Mahy, "STUN - Simple Traversal of User Datagram Protocol (UDP) Through Network Address Translators (NATs)." RFC 3489 (Proposed Standard), Mar. 2003. Obsoleted by RFC 5389.

[36] P. Srisuresh, B. Ford, and D. Kegel, "State of Peer-to-Peer (P2P) Communication across Network Address Translators (NATs)." RFC 5128 (Informational), Mar. 2008.

[37] J. Rosenberg, R. Mahy, P. Matthews, and D. Wing, "Session Traversal Utilities for NAT (STUN)." RFC 5389 (Proposed Standard), Oct. 2008.

[38] D. MacDonald and B. Lowekamp, "NAT Behavior Discovery Using Session Traversal Utilities for NAT (STUN)." RFC 5780, May 2010.

[39] T. Reddy.K, A. Johnston, P. Matthews, and J. Rosenberg, "Traversal Using Relays around NAT (TURN): Relay Extensions to Session Traversal Utilities for NAT (STUN)." RFC 8656, Feb. 2020.

[40] A. Keränen, C. Holmberg, and J. Rosenberg, "Interactive Connectivity Establishment (ICE): A Protocol for Network Address Translator (NAT) Traversal." RFC 8445, July 2018.

[41] W. Eddy, "Transmission Control Protocol (TCP)." RFC 9293, Aug. 2022.

[42] J. Postel, "Transmission Control Protocol." RFC 793 (Standard), Sept. 1981. Updated by RFCs 1122, 3168.

[43] D. Borman, B. Braden, V. Jacobson, and R. Scheffenegger, "TCP Extensions for High Performance." RFC 7323, Sept. 2014.

[44] M. Sargent, J. Chu, D. V. Paxson, and M. Allman, "Computing TCP's Retransmission Timer." RFC 6298, June 2011.

[45] V. Jacobson, "Congestion avoidance and control," in *Proceedings of the ACM SIGCOMM '88 Conference on Communications Architectures and Protocols*, p. 314–329, ACM, 1988.

[46] P. Karn and C. Partridge, "Improving round-trip time estimates in reliable transport protocols," *ACM SIGCOMM Computer Communication Review*, vol. 17, no. 5, p. 2–7, 1987.

[47] J. Nagle, "Congestion control in IP/TCP internetworks." RFC 896, Jan. 1984.

[48] R. Braden, "Requirements for Internet Hosts - Communication Layers." RFC 1122 (Standard), Oct. 1989. Updated by RFCs 1349, 4379.

[49] F. Baker, "Requirements for IP Version 4 Routers." RFC 1812 (Proposed Standard), June 1995. Updated by RFC 2644.

[50] M. Allman, V. Paxson, and E. Blanton, "TCP Congestion Control." RFC 5681, Sept. 2009.

[51] K. Ramakrishnan, S. Floyd, and D. Black, "The Addition of Explicit Congestion Notification (ECN) to IP." RFC 3168 (Proposed Standard), Sept. 2001.

[52] J. Iyengar and M. Thomson, "QUIC: A UDP-Based Multiplexed and Secure Transport." RFC 9000, May 2021.

[53] A.-V. T. W. Group, H. Schulzrinne, S. Casner, R. Frederick, and V. Jacobson, "RTP: A Transport Protocol for Real-Time Applications." RFC 1889 (Proposed Standard), Jan. 1996. Obsoleted by RFC 3550.

[54] J. Heinanen and R. Guerin, "A Single Rate Three Color Marker." RFC 2697 (Informational), Sept. 1999.

[55] J. Heinanen and R. Guerin, "A Two Rate Three Color Marker." RFC 2698 (Informational), Sept. 1999.

[56] P. Almquist, "Type of Service in the Internet Protocol Suite." RFC 1349 (Proposed Standard), July 1992. Obsoleted by RFC 2474.

[57] K. Nichols, S. Blake, F. Baker, and D. Black, "Definition of the Differentiated Services Field (DS Field) in the IPv4 and IPv6 Headers." RFC 2474 (Proposed Standard), Dec. 1998. Updated by RFCs 3168, 3260.

[58] J. Heinanen, F. Baker, W. Weiss, and J. Wroclawski, "Assured Forwarding PHB Group." RFC 2597 (Proposed Standard), June 1999. Updated by RFC 3260.

[59] J. Babiarz, K. Chan, and F. Baker, "Configuration Guidelines for DiffServ Service Classes." RFC 4594 (Informational), Aug. 2006.

[60] J. Reynolds and J. Postel, "Assigned Numbers." RFC 1700 (Historic), Oct. 1994. Obsoleted by RFC 3232.

[61] P. V. Mockapetris, "Domain names - concepts and facilities." RFC 1034 (Standard), Nov. 1987. Updated by RFCs 1101, 1183, 1348, 1876, 1982, 2065, 2181, 2308, 2535, 4033, 4034, 4035, 4343, 4035, 4592.

[62] P. Vixie, S. Thomson, Y. Rekhter, and J. Bound, "Dynamic Updates in the Domain Name System (DNS UPDATE)." RFC 2136 (Proposed Standard), Apr. 1997. Updated by RFCs 3007, 4035, 4033, 4034.

[63] G. Coulouris, J. Dollimore, T. Kindberg, and G. Blair, *Distributed Systems: Concepts and Design*. Boston, MA: Addison-Wesley, 5th ed., 2012.

[64] M. Eisler, "XDR: External Data Representation Standard." RFC 4506 (Standard), May 2006.

[65] Sun Microsystems, *ONC+ Developer's Guide*. Santa Clara, CA, 2002. Part No. 816-1435-10. https://docs.oracle.com/cd/E19683-01/816-1435/index.html.

[66] JSON-RPC Working Group, *JSON-RPC 2.0 Specification*, Jan. 2013. https://www.jsonrpc.org/specification.

[67] T. Bray, "The JavaScript Object Notation (JSON) Data Interchange Format." RFC 8259, Dec. 2017.

[68] R. T. Fielding, *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis, University of California, Irvine, 2000. https://ics.uci.edu/~fielding/pubs/dissertation/top.htm.

[69] Z. Shelby, K. Hartke, and C. Bormann, "The Constrained Application Protocol (CoAP)." RFC 7252, June 2014.

[70] R. Fielding, M. Nottingham, and J. Reschke, "HTTP Semantics." RFC 9110, June 2022.

[71] J. Reschke, J. Snell, and M. Bishop, "The HTTP QUERY Method." RFC 10008, June 2026.

[72] OpenAPI Initiative, *OpenAPI Specification v3.2.1*, Sept. 2026. https://spec.openapis.org/oas/v3.2.1.html.

[73] A. Wright, H. Andrews, B. Hutton, and G. Dennis, "JSON Schema: A media type for describing JSON documents." Internet-Draft draft-bhutton-json-schema-01 (versión 2020-12), June 2022. https://json-schema.org/draft/2020-12/json-schema-core.

[74] M. Nottingham, E. Wilde, and S. Dalal, "Problem Details for HTTP APIs." RFC 9457, July 2023.

[75] R. Thurlow, "RPC: Remote Procedure Call Protocol Specification Version 2." RFC 5531, May 2009.

[76] S. Dalal and E. Wilde, "The Deprecation HTTP Response Header Field." RFC 9745, Mar. 2025.

[77] E. Wilde, "The Sunset HTTP Header Field." RFC 8594, May 2019.